Stefan Hoffmann

The Set of Synchronizing Words and the Constrained Synchronization Problem

November 6, 2026

Springer Nature

Abstract

Have you ever wanted to start all over again? Has your smartphone or computer ever frozen and you had to restart it? If so, then you are familiar with the idea of resetting things, the central concept of the present work. More specifically, it is about resetting (also called synchronizing) finite state transition systems, also called finite automata. Finite automata process input sequences (also called words) and can be used to model a wide variety of systems, for example, switching circuits, networks, or simple behavior patterns in artificial intelligence. In all these situations, knowing whether we can reset the system is of interest. A circuit component might be out of sync with its environment, can we reset it? A robot might be stuck, or we are simply unsure about its current state, so can we reset its state of operation? However, while this problem is tractable for completely specified and deterministic automata, only a small deviation from this "optimal" situation, i.e., having only partial information, allowing nondeterminism, or constraining the admissible input sequences, results in intractable algorithmic problems.

Here, we are concerned with one particular problem variant: constraining the admissible input sequences (the *constrained synchronization problem*). In this setting, what computational complexity can we deduce from the kind of constraint put on the input sequences and/or types of input automata? We state various results on how to manipulate sets of admissible synchronizing input sequences so as to retain a certain complexity of the problem. We also identify various conditions that either yield tractable or intractable problems. More specifically, the following complexity classes will arise: AC^0, L, NL, P, NP, PSPACE. Except for AC^0 and P, we give sets of admissible synchronizing sequences yielding constrained problems that are complete for these classes. Moreover, we investigate the following classes of admissible sets of synchronizing sequences: finite, infinite, sparse, letter-bounded, commutative, or constraints specified by certain small-state automata; where we give a complete complexity classifications for the latter three classes. Lastly, we consider the following types of input automata to the problem: monotonic, partially ordered, commutative, solvable, SI- and ρ–automata. We obtain, for example, that for commutative input automata, the problem is always in P.

Now, the set of synchronizing sequences, i.e., our search space, of a given input automaton is acceptable by a finite automaton as well. This leads naturally to the investigation of accepting automata for sets of synchronizing sequences, specifically for

the question of when they become largest possible. Here, we identify and investigate two conditions that control the size of a minimal automaton for the set of synchronizing sequences: reachability and distinguishability. For both notions, we establish various structural results and connections with concepts from the literature such as, for example, completely reachable automata. Decision problems and algorithmic questions that naturally arise from these notions are also investigated. We state conditions yielding extremal minimal automata accepting sets of synchronizing sequences. Moreover, we identify a connection to permutation group theory, which is linked to automata theory by the group of units of transformation monoids of automata. Specifically, via graph-theoretical constructions, we obtain new characterizations of primitive permutation groups based on associated synchronizing automata. Our results link permutation groups and vertex-transitive graphs to the theory of synchronizing automata. Furthermore, we explore relations of the size of automata accepting sets of synchronizing sequences to the length of shortest synchronizing sequences.

Zusammenfassung

Wollten Sie schon einmal wieder von vorne anfangen? Hat sich Ihr Smartphone oder Computer schon einmal aufgehängt und nur ein Neustart schuf Abhilfe? Falls ja, so wissen Sie, was es heißt etwas zurückzusetzen, ein zentrales Konzept der vorliegenden Arbeit. Im Speziellen handelt sie von der Zurücksetzbarkeit (oder synonym Synchronisierbarkeit) endlicher Zustandsübergangssysteme, auch endliche Automaten genannt. Automaten interagieren über Eingabefolgen (auch Wörter genannt), welche Zustandsübergänge bewirken, mit ihrer Umwelt und dienen der Modellierung einer Vielzahl von Systemen wie Schaltkreisen, Betriebszuständen von Maschinen oder einfachen Agenten in der künstlichen Intelligenz. Ein Automat heißt synchronisierbar, sofern es eine Eingabefolge gibt, welche ihn in einen wohldefinierten Zielzustand versetzt, unabhängig von seinem aktuellen Zustand. Die Schwierigkeit auf Synchronisierbarkeit zu testen hängt von verschiedenen Parametern ab, zum Beispiel ob wir das System vollständig kennen, welches Verhalten beobachtbar ist, ob Nichtdeterminismus erlaubt ist oder ob bestimmte synchronisierende Folgen ausgeschlossen sind.

In dieser Arbeit beschäftigen wir uns mit der Einschränkung möglicher synchronisierender Eingabefolgen, das sog. *constrained synchronization problem* (dt. *Problem der eingeschränkten Synchronisierbarkeit*). Es stellt sich nun in natürlicher Weise die Frage, wie die Art der Einschränkung die algorithmische Komplexität beeinflusst. Kann man anhand einer gegebenen Einschränkung die resultierende Problemstellung in schwierig oder praktikabel lösbar einteilen? Wir untersuchen komplexitätserhaltende Operationen auf Mengen von erlaubten synchronisierenden Eingabefolgen. Weiterhin arbeiten wir verschiedene hinreichende Bedingungen für praktikabel lösbare oder schwierige Probleme heraus. Im Rahmen unserer Untersuchungen tauchen folgende Komplexitätsklassen auf: AC^0, L, NL, P, NP, PSPACE, wobei wir Bedingungen für Mitgliedschaft in einer dieser Klassen oder Schwierigkeit bezüglich NP oder PSPACE angeben sowie, außer für AC^0 und P, eine Vielzahl von vollständigen Problemen für die jeweiligen Komplexitätsklassen. Die folgenden Arten von Einschränkungen für Mengen synchronisierender Eingabefolgen untersuchen wir: endlich, unendlich, spärlich, buchstaben-beschränkt, kommutativ oder durch kleine Automaten beschrieben. Für kommutative, buchstabenbeschränkte und durch bestimmte kleine Automaton beschriebene Einschränkungen geben wir eine komplette Klassifikation der algorithmischen Komplexität des zugehö-

rigen eingeschränkten Synchronisierbarkeitsproblems. Darüber hinaus untersuchen wir die folgenden Klassen von Eingabeautomaten: monotone, partiell geordnete, kommutative, auflösbare SI- und ρ-Automaten. Unsere Ergebnisse zeigen, dass die resultierenden algorithmischen Komplexitäten stark vom betrachteten Eingabetyp abhängen. Beispielsweise ist das Problem für kommutative Eingabeautomaten stets in Polynomialzeit lösbar.

Die Menge an Eingabefolgen, die einen Automaten synchronisieren, das heißt unser Suchraum, ist durch einen weiteren Automaton akzeptierbar, was die Untersuchung von akzeptierenden Automaton für jene Mengen nahelegt. Von besonderem Interesse in Zusammenhang mit unseren Fragestellungen ist der Fall, wenn ein akzeptierender Automat zur Beschreibung synchronisierender Eingabefolgen maximal groß ist. Wie wir hier herausarbeiten, wird die Größe solcher Automaten durch zwei Faktoren beeinflusst: Erreichbarkeit und Unterscheidbarkeit. Wir untersuchen beide Begriffe, stellen Zusammenhänge her und setzen diese in Beziehung zu etablierten Konzepten aus der Literatur, wie zum Beispiel zu vollständig erreichbare Automaten (engl. completely reachable automata). Verschiedene Bedingungen für die Maximalität von Automaten für synchronisierende Eingabefolden werden herausgearbeitet. Weiterhin formulieren und untersuchen wir Entscheidungsprobleme und algorithmische Fragestellungen, welche sich aus diesen Begriffen ergeben. Über ihren Transformationsmonoid haben Automaten einen natürlichen Bezug zur Algebra. Auf dieser Grundlage werden wir Verbindungen zur Theorie der Permutationsgruppen herstellen. Im Speziellen arbeiten wir, unter Zuhilfenahme graphentheoretischer Konstruktionen, verschiedene neuartige Charakterisierungen für primitive Permutationsgruppen heraus. Unsere Ergebnisse verbinden Permutationsgruppen und ecken-transitive Graphen (engl. vertex-transitive graphs) mit der Theorie synchronisierbarer Automaten. Abschließend untersuchen wir, inwiefern die Größe eines akzeptierenden Automaten für die synchronisierenden Eingabefolgen eines gegebenen Automaten mit der Länge einer kürzesten synchronisierenden Eingabefolge in Beziehung steht.

Acknowledgement

First, I thank my supervisor Prof. Dr. Henning Fernau for giving me the freedom and opportunity to pursue my research and the topics that interest me and also for encouragement to develop (and write down) my ideas! His open-mindedness, broad knowledge, diplomatic attitude, and willingness to engage into different topics created just the right (research) environment. I learned a lot from him, both technically and humanly. Furthermore, I express my gratitude to him for having infinite patience with me; in particular with respect to this thesis which took way more time than intended. In this regard, I also have to thank Prof. Dr. Müller-Hannemann from the Martin-Luther University Halle-Wittenberg for his patience and understanding with my delays; even if things did not worked out as planned, for which I am deeply sorry.

I thank Prof. Dr. Mikhail Volkov (Ural Federal University) for introducing our working group to synchronizing automata at a common workshop in 2019 and suggesting the topics that form the basis of the present thesis. I also thank him for agreeing to be my co-supervisor, despite this was not possible in the end due to political reasons beyond our control.

I thank Prof. Igor Rystsov for agreeing to be my co-supervisor on short notice. Though I never met him in person, his research and ideas, as is certainly noticeable, influenced the content of the present work.

Furthermore, I am grateful to all the friends that made my time in Trier so enjoyable, in particular Richard Spang, Anna Andres, and Ray Kandar for the parties and moments we shared! Thanks go to the Gladiators Trier Soccer Team of the Institute of Archeology for letting me join their training despite not being an archaeologist (special thanks to Lukas Märker here!). Without the regular physical activity, my intellectual activity would certainly not be possible.

I also thank the university choir for letting me participate, and the concerts and rehearsals I enjoyed. In particular, I thank our bass leader Steffen Kramer, for many fun moments and also, together with Malte Busch, many nice movie nights and subsequent discussions. I am also thankful for all the other moments I shared with Malte traveling or spending time at a bar, or the wonderful people I meet due to him, as, for instance, Jana Thielen.

And I thank my flatmates: Andre Janipour, Florian Höller, Chemseddine Amrani, and Asek Ako-Agbor Ashu for getting along so well and a relaxed and open atmosphere, despite me not being very social for the last years due to my graduation efforts. In particular, I thank Florian Höller for sharing his ”good mood 90’s” music playlist with me that played (and put me in a good mood) during the last weeks of writing up the present thesis.

Furthermore, I thank Henning Fernau, Richard Spang, Malte Busch, Anna Andres, and (for a short glance before falling asleep on a plane) Giuseppe Romana for reading through a preliminary version of the present work and giving valuable feedback.

Lastly, I thank my parents for raising and supporting me, and making it possible for me to study and graduate.

Stefan Hoffmann, 2025

Notation Index

Standard logical and set-theoretical notation that is briefly mentioned (and fixed) at the beginnings of Sections 1.1 and 3.1 and common expressions, as, for example, $\binom{n}{2} = \frac{n(n-1)}{2}$, u^n or t^n for the n-fold concatenation product (resp. composition) of a word u or transformation t, etc., do not appear in the notation index. We assume a sufficient level of familiarity with such notions on the part of the reader to not feel it necessary to include them in this index. Furthermore, notations that are only mentioned in a single paragraph (for example, in a discussion section) and are not used beyond that are also not included.

$f\colon X \to Y$	(Total) function f from X to Y	3
Σ	A finite alphabet, i.e., set of symbols	3
Σ^*	Set of words over Σ with concatenation	3
$u \cdot v$ or uv	Concatenation of two words u, v	3
$\lvert u \rvert$	Length of word u	3
ε	The empty word	3
$\mathbb{N}_0$	$\{0, 1, 2, 3, \ldots\}$	4
$\mathbb{Z}$	Set of integers	4
$\mathbb{R}$	Set of real numbers	4
$[n]$	The set $\{0, 1, \ldots, n-1\}$	4
$\max(M)/\min(M)$	Maximum/minimum of finite $\emptyset \neq M \subseteq \mathbb{N}_0$	4
$\max(a_1, \ldots, a_n)$	Maximum of $a_1, \ldots, a_n \in N_0$	4
$\min(a_1, \ldots, a_n)$	Minimum of $a_1, \ldots, a_n \in N_0$	4
$\mathcal{A} = (Q, \Sigma, \delta)$	Automaton specification of $\mathcal{A}$	4
$\delta(S, u)$	Image of $S \subseteq Q$ in $\mathcal{A} = (Q, \Sigma, \delta)$	4
$\mathrm{Syn}(\mathcal{A})$	Set of synchronizing words of $\mathcal{A}$	4
$\mathrm{rt}(\mathcal{A})$	Reset length of $\mathcal{A}$	10
$C_n = ([n], \{a, b\}, \delta_n)$	Automata from the Černý family	10
$[p, q]$	Interval between $p, q \in Q$ in C_n	11
$\mathcal{P}(X)$	Power set of set X	27
$(f(x))_{x \in A}$	Notation to specify function $f\colon X \to Y$	28
$\pi_i\colon X_1 \times \cdots \times X_n \to X_i$	Component projection mapping	28
$a \bmod b$	Unique $0 \leq r < b$ with $a = bn + r$ for $r, n \in \mathbb{N}_0$	28

$a \equiv a' \pmod b$ or $a \equiv_b a'$	a and a' are congruent modulo b	28
$\gcd(M), \operatorname{lcm}(M)$	Greatest com. div. / least common multiple	28
$\lfloor x \rfloor$	$\lfloor x \rfloor = \min\{k \in \mathbb{Z} \mid k \leq x\}$	28
Y^X	Set of functions $f\colon X \to Y$	30
$f(S)$	Extension of $f\colon X \to Y$ to subsets $S \subseteq X$	30
$f^{-1}(S)$, $f^{-1}(x)$	Inverse image of S (resp. x) for $f\colon X \to Y$	30
$\operatorname{dupl}(f)$, $\operatorname{dupl}(u)$	Duplicate states of $f\colon X \to Y$ / word $u \in \Sigma^*$	30, 132
$\operatorname{excl}(f)$, $\operatorname{excl}(u)$	Excluded states of $f\colon X \to Y$ / word $u \in \Sigma^*$	30, 132
$g \circ f$ or gf	Function composition	31
$\operatorname{rank}(f)$, $\operatorname{rank}(u)$	Rank of $f\colon X \to Y$ / word $u \in \Sigma^*$	36, 132
$\operatorname{def}(f)$, $\operatorname{def}(u)$	Defect of $f\colon X \to Y$ / word $u \in \Sigma^*$	36, 132
$\operatorname{exc}_f(A)$	Excess of A under f	36
$(k_1, \ldots, k_l)$	Kernel-type notation	36
$\operatorname{fix}(t), \operatorname{move}(t)$	Set of fixpoints / moved points	38
$\mathrm{id}_X\colon X \to X$	Identity transformation on X	38
$(a_0\ a_1\ \cdots\ a_{m-1})_X, ()$	Cycle notation	93
$[a_1 \mapsto \cdots \mapsto a_m]_X$	Tail transformation notation	94
$[x]_\sim$	Equivalence class of x w.r.t. $\sim$	39
$X/\sim$	Set of equivalence classes w.r.t. $\sim$	39
$\pi_\sim\colon X \to X/\sim$	Canonical projection map	39
$\sim_f$	Kernel rel. of f (equiv. rel. induced by f)	39
$\uparrow T$	Smallest upper set containing T	46
$+_n, \cdot_n$	Cropped operations on $[n]$	41
AB	Product on subsets $A, B \subseteq S$, S semigroup	41
A^n, $A^{\leq n}$, $A^{<n}$, aA, Aa	Operations derived from products on subsets	42
$\leq_S, \leq_M, \leq_G$	Subsemigroup, submonoid, subgroup	41
$Z(G)$	Center of a group G	43
$\langle T \rangle_S$, $\langle T \rangle_M$, $\langle T \rangle_G$	Semigroup, monoid, group generated by T	43
$A = (a_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,m\}}$	$n \times m$ matrix	43
I_n, O_n	$n \times n$ identity and zero matrices, respectively.	44
$\mathcal{T}_X$	Full transformation semigroup on X	45
$\mathcal{T}_n$	Full transformation semigroup on $[n]$	45
$\mathcal{S}_X$	Full symmetric group on X	111
$\operatorname{Orb}_G(a)$	Orbit of a in permutation group G	112
G_a	Point stabilizer of a in permutation G	112
t^g	Conjugation of $t\colon X \to X$ by $g\colon X \to X$	112
$\mathcal{S}_n$	Full symmetric group on $[n]$	111
$\mathcal{A}_n$	Alternating group on $[n]$	112
L^*, L^+	Kleene star and plus	56
$\operatorname{Pref}(u), \operatorname{Pref}(L)$	Set of prefixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\operatorname{Suff}(u), \operatorname{Suff}(L)$	Set of suffixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\operatorname{Fact}(u), \operatorname{Fact}(L)$	Set of factors/infixes of word u (resp. $L \subseteq \Sigma^*$)	57
$\lvert u \rvert_a$	Number of occurrences of $a \in \Sigma$ in $u \in \Sigma^*$	57
$u^{-1}L$	Language quotient of L by u	57
$\pi_\Gamma\colon \Sigma^* \to \Gamma^*$	Projection morphism	57

$\mathcal{RE}(\Sigma)$	Set of regular expressions over Σ	58
$L(R)$	Language described by $R \in \mathcal{RE}(\Sigma)$	58
$\psi\colon \Sigma^* \to \mathbb{N}_0^\Sigma$	Parikh map/Parikh morphism	57
$\mathrm{Aut}(G)$	Automorphism group of a graph $G = (V, E)$	59
G_t / G_R	Functional graph / graph of a relation	62
$N^+(U)$	Vertices v with edge (u, v) s.t. $u \in U$, $v \notin U$	127
$O(g)$, $o(g)$	Big O and small o notation for $g\colon \mathbb{N}_0 \to \mathbb{N}_0$	64
(L-uniform) NC^k, AC^k	Circuit families or languages decidable by circuit families in (L-uniform) NC^k/AC^k	66, 71
L	Languages decidable in det. log. space	71
NL	Languages decidable in nondet. log. space	71
NC	Language class $\bigcup_{k\geq 0}$ L-uniform NC^k	71
P	Languages decidable in det. poly. time	71
NP	Languages decidable in nondet. poly. time	71
PSPACE	Languages decidable in det. poly. space	71
NPSPACE	Languages decidable in nondet. poly. space	71
$\leq_m^{\log}$ / $\equiv_m^{\log}$	log-space many-one reduction / equivalent	72
$\leq_m^{\mathsf{P}}$ / $\equiv_m^{\mathsf{P}}$	polynomial time many-one reduction / equivalent	72
$\leq_m^{\mathsf{AC}^0}$ / $\leq_m^{\mathsf{NC}^1}$	L-uniform AC^0 / NC^1 many-one reduction	72
$\equiv_m^{\mathsf{AC}^0}$ / $\equiv_m^{\mathsf{NC}^1}$	L-uniform AC^0 / NC^1 many-one equivalent	72
$\leq_T^{\mathsf{NC}^1}$ / $\equiv_T^{\mathsf{NC}^1}$	L-uniform NC^1 oracle reduction / equivalent	72
G^D	Diagonal action on G	123
$\delta^{-1}(q, u)$	Inverse transition of δ	131
$G(\mathcal{A})$	Automaton graph of automaton $\mathcal{A}$	131
$\preceq_R$, $\preceq_R^{\mathcal{A}}$	Reachability quasiorder w.r.t. $\mathcal{A}$	134
$\simeq_R$	Equivalence relation induced by $\preceq_R$	134
δ_u	$\delta_u\colon Q \to Q$ induced by $u \in \Sigma^*$ in $\mathcal{A} = (Q, \Sigma, \delta)$	132
$\mathcal{T}_S(\mathcal{A})$, $\mathcal{T}_M(\mathcal{A})$	Transformation semigroup/monoid of $\mathcal{A}$	132
$[a]_{\mathcal{A}}$, $[u]_{\mathcal{A}}$, $[a]$, $[u]$	Transition matrix for letter/word a/u w.r.t. $\mathcal{A}$	132
$\mathcal{A} \cong \mathcal{B}$	$\mathcal{A}$ and $\mathcal{B}$ are isomorphic	137
$\mathcal{A}_{\mid S}$, $\mathcal{A}_{\mid \Gamma}$, $\mathcal{A}_{\mid \Gamma, S}$	Restriction of $\mathcal{A}$ to upper set S, Γ or both.	148
$\mathcal{A}/{\sim} = (Q/{\sim}, \Sigma, \delta_\sim)$	Quotient automaton w.r.t. $\mathcal{T}_M(\mathcal{A})$-congruence $\sim$	149
$\sim_S$	Rees congruence for upper set S	151
$\mathcal{A}_1 \times \cdots \times \mathcal{A}_m$, $\mathcal{A}^n$	Product aut. of $\mathcal{A}_1, \ldots, \mathcal{A}_m$ / n-fold product	154
$\mathcal{P}(\mathcal{A})$	Power automaton of $\mathcal{A}$	154
$\mathcal{A}^{\leq 2}$	Auxiliary construction for pair automaton	157
$\mathcal{A}^{\leq 2}/{\sim_F}$	Pair automaton of $\mathcal{A}$	157
$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$	Automaton as language acceptor	159
$L(\mathcal{A})$	Language accepted by automaton $\mathcal{A}$	159
$\mathcal{A}_{q,S}$	Changing/adding start s & final set F to $\mathcal{A}$	159
$\equiv_L$	Myhill-Nerode relation	161
$\mathrm{MinAut}(L)$	Minimal automaton of $L \subseteq \Sigma^*$	161
$\pi_{\mathcal{A}}$	Morphism onto $\mathcal{A}_L$ from accessible $\mathcal{A}$	161
$\leq_{\mathcal{R}}$	$\mathcal{R}$-quasiorder on words w.r.t. automaton	172
$\sim_s$	Stability relation	183

Σ_I, Σ_G	Subalphabets w.r.t. OI-automaton	186
$\mathcal{B}_{n,m,r}$	Strongly conn. comm. automata	203
$\Gamma_k(\mathcal{A}) = (Q_k, E_k)$	Bondar-Casas-Volkov graph construction	??
$I(\mathcal{A}) = (Q, E)$	The Rystsov graph	??
$I_i(\mathcal{A}) = (Q, E_i)$	Auxiliary graph introduced by Rystsov	??
$I(S) = ([n], E)$	Rystsov graph extended to $S \leq_S \mathcal{T}_n$	??
$\Gamma_1(S) = ([n], E)$	$\Gamma_1(\mathcal{A})$-graph extended to $S \leq_S \mathcal{T}_n$	??
$\Gamma_{1,i}(\mathcal{A})$	Variant of $\Gamma_1(\mathcal{A})$-graph	??
$\mathcal{A}_{f,a}$	Adding letter a to $\mathcal{A}$ defined via $f\colon Q \to Q$	??
$a \rightsquigarrow_T b$ / $a \rightsquigarrow b$	Quasiorder used in ??.	??
$\mathbf{D}, \mathbf{E}, \mathbf{F}, \mathbf{G}, \mathbf{H}, \mathbf{I}, \mathbf{J}, \mathbf{K}$	Exceptional automata for Černý bound	??
$\mathcal{V}_n = \mathcal{W}_n$	Class of slowly synchronizing automata	??
$\mathcal{L}_n = \mathcal{D}'_n$	Class of slowly synchronizing automata	??
$\mathcal{F}_n$, $n > 3$ odd	Class of slowly synchronizing automata	??
$\mathcal{K}_n = \mathcal{G}_n$, $n > 5$ odd	Class of slowly synchronizing automata	??
$\overline{\mathcal{A}_n}$, $n \geq 8$ even	Class of slowly synchronizing automata	??
$\mathcal{M}_n$, $n \geq 3$	Class of slowly synchronizing automata	??
$\mathcal{B}_n$, $n \geq 5$ odd	Class of slowly synchronizing automata	??
$f\colon X \nrightarrow Y$	Partial function f from X to Y	??
$\sigma_L\colon \Sigma^* \to \Sigma^*$	Levenshtein-Romanov mapping	??
$\rho\colon \Sigma^* \to \mathbb{N}_0$	Equal-letter runs	??
$\equiv_L^{\sim,U}$	Relation derived from $\equiv_L$	??
$\equiv_L^{>n}$	Relation defined with $\equiv_L^{\sim,U}$	??
$\sim_{C,V}$	Relation used to define $\equiv_{C,V}$	??
$\equiv_{C,V}$	Relation defined with $\equiv_L^{\sim,U}$	??
$\Sigma_{i,j}$	Subalphabet w.r.t. small constraint automaton	??
$\psi_n\colon \Sigma^* \to \{0, 1, \ldots, n\}^\Sigma$	n-cropped Parikh map	??
L_i^F	Shorthand $L_i^F = L(\mathcal{B}_{i,F})$ for Table ??	??
$\mathcal{A}_t, \mathcal{A}_c$	Sync. binary SI-automata, see Prop. 7.31	??

List of Figures

1.1 Example automaton 5
1.2 Encoding of five symbols e, h, l, o, and r by binary sequences. 6
1.3 Introductory decoder automaton 6
1.4 Ashby's ghost taming automaton 8
1.5 A polygonal component and its possible orientations 8
1.6 Part orienter problem modeled as a finite automaton 9
1.7 Černý family C_n 11

3.1 Illustration for the identities from Theorems 3.4 and 3.6 33
3.2 Cases so that $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ 34
3.3 Scheme of an $m \times n$ matrix 44
3.4 Relations among connectivity notions 61
3.5 Examples for connectedness notions 62
3.6 Example functional graph 63

4.1 Composition of cycle and simple idempotent 96
4.2 Functional graph of a one-point singular 97

5.1 Directed orbital graphs of $G = \langle\{(0\ 1\ 2\ 3)\}\rangle_G$ 128
5.2 Directed orbital graphs of $G = \langle\{(0\ 1)(2\ 3), (0\ 2)(1\ 3)\}\rangle_G$ 128

6.1 The transition matrices for the Černý automata C_n 133
6.2 Example morphisms and non-morphisms on automata 138
6.3 Automaton not admitting any non-identity endomorphism. 138
6.4 Some strongly connected component fixed and others swapped 145
6.5 Morphism between automata 148
6.6 Example power automaton 155
6.7 Automaton constructed from transformation semigroup. 166

7.1 Relations among classes of automata 176
7.2 Automata with a trap state attaining the maximal reset length 179
7.3 Unary automaton for lower bound of rank r words 186

7.4 Synchronizing binary SI-automata 188
7.5 Two synchronizing monotonic automata 192
7.6 Partially ordered automaton that is not monotonic 196
7.7 Example ρ-automata 198
7.8 Example quotient automaton of ρ-automaton 200
7.9 Example automata $\mathcal{B}_{n,m,r}$ 203
7.10 Extremal synchronizing commutative automaton 209
7.11 Non-solvable automaton with solvable pair automaton 212
7.12 Solvable automaton with sequence of upper sets 217
7.13 Inclusion relations for solvability notions 221

8.1 Haase diagram for reductions 227
8.2 Reduction of OrderBetweenVertices to SyncWords. 234
8.3 Example for reduction used in the proof of Prop. 8.10 236
8.4 Example for reduction of 3SAT to Intersection on unary automata . . . 241
8.5 Non-locally confluent synchronizing partially ordered automaton 257

List of Tables

1.1 Published upper bounds on the reset length 13

4.1 Cases for composition being a one-point singular 100
4.2 Conditions covering cases for composition being one-point singular . . . 101

8.1 Overview complexities on classes of automata 233

Contents

Part I Introductory Part

1 Short Introduction to Synchronizing Automata 3
1.1 Notions Used in the Introduction 3
1.2 Formal Definition of Automata Synchronization 4
1.3 Introductory Examples 5
1.3.1 Error-Prone Communication Channels 5
1.3.2 The Haunted Mansion 7
1.3.3 Part Orienters 8
1.3.4 Lost in a Maze 10
1.4 Reset Length and the Černý Conjecture 10

2 Outline and Contribution 15
2.1 The Problems 15
2.1.1 The Constrained Synchronization Problem 15
2.1.2 Automata for Sets of Synchronizing Words 16
2.2 Organization of the Thesis 17
2.2.1 Structure of Part II: Preliminaries 17
2.2.2 Structure of Part III: Automata for Sets of Synchronizing Words 18
2.2.3 Structure of Part IV: The Constrained Synchronization Problem 19
2.3 Work by the Author Relevant to the Thesis 20
2.3.1 Published Work 20
2.3.2 Unpublished Work 22

Part II Preliminaries

3 General Notions 27
3.1 More on Logic, Set Theory, and Number Theory 27
3.2 More on Functions 28
3.2.1 Basic Notions 30
3.2.2 The Duplicate Set and Function Composition 32
3.2.3 Functions on a Finite Set 36

3.3 Transformations 38
3.4 Relations 39
3.5 Algebraic Notions 40
3.5.1 Semigroups, Monoids, Groups, and Semirings 40
3.5.2 Matrices 43
3.5.3 Transformation Semigroups 45
3.6 Order Theory 45
3.6.1 Definitions and Basic Facts 46
3.6.2 Equivalence Classes and Upper and Lower Sets 49
3.6.3 Maximal and Minimal Elements 50
3.6.4 The Set of Maximal Elements 53
3.6.5 Sequences of Upper and Lower Sets 54
3.7 Formal Language Theory 55
3.7.1 More Basic Notions 55
3.7.2 Regular Expressions 58
3.8 Graph Theory 59
3.8.1 Basic Notions 59
3.8.2 Connectedness Notions 60
3.8.3 Graphs for Transformations and Homogeneous Relations 62
3.9 Computational Complexity Theory 64
3.9.1 The Random Access Machine Model 64
3.9.2 Turing Machines and Transducers 65
3.9.3 Circuits 66
3.9.4 Decision Problems 69
3.9.5 Complexity Classes and Reductions 71
3.9.6 Circuits and Encodings 76
3.10 Remarks on Different Notions of Reduction 80
3.11 Remarks on the Order of Function Composition 81
3.12 Discussion 84

4 Classes of Transformations 91
4.1 Permutations and Quasipermutations 91
4.2 Cycles and Cycles Induced by a Transformation 93
4.3 Tail Transformations, Simple Idempotents, and Cycles 94
4.4 One-Point Singular Transformations 97
4.5 Idempotent Transformations 102
4.6 Idempotent One-Point Singulars 106
4.7 Discussion 109

5 Permutation Groups 111
5.1 Basic Notions 111
5.2 Transitive Permutation Groups 115
5.2.1 Separating Sets and Idempotents 115
5.2.2 Blocks and Point Stabilizers 116
5.2.3 Mapping 2-Sets, Orbit Sizes, and Transitivity 118
5.3 Permutation Groups and Abstract Groups 122

5.4 Permutation Groups and Non-Permutations 124
5.5 Graphs and Permutation Groups 125
5.5.1 Generating Sets for the Full Symmetric Group 125
5.5.2 Vertex-Transitive Graphs 126
5.5.3 Orbital Graphs 127
5.6 Discussion 129

6 Notions From Automata Theory 131
6.1 More Basic Notions 131
6.2 The Transformation Semigroup of an Automaton 132
6.3 The Reachability Order 134
6.3.1 Definition and a Characterization 134
6.3.2 Connectivity and Upper Sets 134
6.4 Morphisms 137
6.4.1 Definitions and Basic Facts 137
6.4.2 Morphisms and the Reachability Order 140
6.4.3 Endomorphisms 143
6.4.4 Cyclic Unary Automata 146
6.5 Operations on Automata 148
6.5.1 Subautomata and Restricted Automata 148
6.5.2 The Quotient Automaton 149
6.5.3 The Rees Quotient Automaton 150
6.5.4 The Product Automaton 154
6.5.5 The Power Automaton 154
6.5.6 The Pair Automaton 157
6.6 Sequences of Upper Sets and Maximal States 157
6.7 Automata as Language Acceptors 159
6.7.1 Regular Languages 159
6.7.2 The Minimal Automaton 160
6.8 Remarks on Transformation Semigroups and Automata 166
6.9 Discussion 167

7 Classes of Automata 175
7.1 Synchronizing Automata 176
7.1.1 Sets of Synchronizing Words and Ideal Languages 176
7.1.2 Subautomata, Morphic Images, and Quotient Automata 177
7.1.3 Synchronizing Automata with a Trap State 178
7.1.4 Synchronizing Automata and the Reachability Order 180
7.1.5 The Pair Criterion 182
7.1.6 Algorithmic Questions 183
7.2 Unary Automata 185
7.3 OI- and SI-Automata 186
7.4 Permutation-Reset Automata 190
7.5 Monotonic Automata 191
7.6 Automata Whose Strongly Connected Components are Blocks 192
7.7 Partially Ordered Automata 196

7.8 ρ-Automata 198
7.9 Commutative Automata 200
7.9.1 Commutativity of Rees Quotients 201
7.9.2 Strongly Connected Automata 202
7.9.3 Subset Mapping Properties 205
7.9.4 Extremal Synchronizing Automata 209
7.10 Solvable Automata 210
7.10.1 Operations 210
7.10.2 Characterizations 213
7.10.3 Synchronization and Subset Mapping Properties 218
7.10.4 Comparison with Original Definition 220
7.11 Discussion 222

8 Subset Mapping Problems 225
8.1 The Problems 225
8.2 Reductions 226
8.3 Classes of Automata 230
8.3.1 Known Results and Overview 231
8.3.2 General Automata 233
8.3.3 Solvable Automata 240
8.3.4 Commutative Automata 244
8.3.5 Partially Ordered Automata 248
8.4 Discussion 249

Part III Automata for Sets of Synchronizing Words

Part IV The Constrained Synchronization Problem

References 263

Part I
Introductory Part

"That fondness for science has encouraged me to compose a short work on calculating by al-jabr (completion) and al-muqabala (balancing), confining it to what is easiest and most useful in arithmetic."

—Muhammad ibn Musa Al-Khwarizmi (780–850)

Chapter 1
Short Introduction to Synchronizing Automata

This chapter gives a short introduction to synchronizing automata for readers unfamiliar with the topic. Section 1.1 fixes the notation needed here. The notation for the remainder of the thesis is given in Chapter 3. In Section 1.2 we formally introduce synchronizing automata. Then, in Section 1.3, we present several introductory examples and problems and discuss their relation to synchronizing automata. Finally, Section 1.4 defines the reset length, states Černý's conjecture, and presents the Černý automata, which serve as a running example throughout the thesis.

For readers who want to learn more about synchronizing automata, the most recent survey articles are by Kari and Volkov [360] (see also its precursor [613]) and by Volkov [617], the latter containing a good coverage of applications from different fields and types of automata considered in the literature. The surveys by Jürgensen [348], Kari and Volkov [360], and Béal and Perrin [71] discuss connections to codes (see also Chapters 3 and 10 of [87]). Sandberg [548] and Kohavi [375, Chap. 13] include relations to automata with outputs as well as applications in circuit theory and testing. The survey by Bogdanović et al. [97] presents structural results not covered in the other surveys. Another survey is by Mateescu and Salomaa [444]. Finally, we recommend [36] on synchronizing permutation groups, motivated by synchronizing automata, as results presented in that survey were inspirational for our results in **??** from Part III.

1.1 Notions Used in the Introduction

If X is a set, by $|X|$ we denote its *cardinality* and $x \in X$ denotes *set membership*, i.e., the fact that x is an element in X. We use standard *set builder notation* $\{x \mid P(x)\}$ or $\{x : P(x)\}$ for the set of all elements x satisfying a predicate $P(x)$. The *empty set* is denoted by $\emptyset$. For sets X, Y, we write $f\colon X \to Y$ for a *(total) function* (also called a *map* or *mapping*) with *domain* X and *codomain* Y, assigning to each $x \in X$ an element $f(x) \in Y$.

Let Σ denote a *finite alphabet*, i.e., a set of *symbols* or *letters* (we always use Σ to denote an alphabet in the present work). A *word (over* Σ*)*, also called a *string*, is a finite

sequence of elements from Σ. By Σ^* we denote the *set of all words* over Σ. We identify each symbol in Σ with the corresponding word of length one. In this way, we have $\Sigma \subseteq \Sigma^*$. For $u, v \in \Sigma^*$ we write $u \cdot v$, or uv for short, for the *concatenation (product)* of both words.[1] We denote the *length* of $u \in \Sigma^*$ by $|u|$. The *empty word*, i.e., the word of length zero, is denoted by ε. For $u \in \Sigma^*$, we set $u^0 = \varepsilon$ and $u^n = u \cdots u$ (n times) for $n > 0$. Subsets of Σ^* are called *(formal) languages (over Σ)*.

By $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ we denote the *set of natural numbers*, including zero, with the usual orders $\leq$ and $<$ on $\mathbb{N}_0$. For $n \in \mathbb{N}_0$ we set $[n] = \{m \in \mathbb{N}_0 \mid m < n\}$. For a non-empty subset M of $\mathbb{N}_0$, by $\min(M)$, or $\min M$ for short, (resp. $\max(M)$, or $\max M$, for finite M) we denote the *smallest* (resp. *largest*) *number* in M w.r.t. the usual order. We also set $\min(a_1, \ldots, a_n) = \min\{a_1, \ldots, a_n\}$ (resp. $\max(a_1, \ldots, a_n) = \max\{a_1, \ldots, a_n\}$) for $a_1, \ldots, a_n \in \mathbb{N}_0$ with $n \geq 1$. We also denote the integers and the real numbers by $\mathbb{Z}$ and $\mathbb{R}$, respectively.

A *(deterministic and complete) finite semiautomaton* over Σ (or just *automaton* for short) is a tuple $\mathcal{A} = (Q, \Sigma, \delta)$ where Q is the *finite state set*, Σ the *finite input alphabet*, and $\delta \colon Q \times \Sigma \to Q$ the *(totally defined) transition function*.[2] We extend δ to a function $\hat{\delta} \colon Q \times \Sigma^* \to Q$ by setting $\hat{\delta}(q, \varepsilon) = q$ and $\hat{\delta}(q, ux) = \delta(\hat{\delta}(q, u), x)$ for $q \in Q$, $u \in \Sigma^*$, and $x \in \Sigma$. For simplicity, we denote the extension $\hat{\delta}$ by δ as well. Given $S \subseteq Q$ and $u \in \Sigma^*$ the *image (of S under u)* is $\delta(S, u) = \{\delta(q, u) \mid q \in S\}$.

Automata are often specified graphically as *state diagrams*, where states are represented as points or circles, and a transition $\delta(p, a) = q$ for $p, q \in Q$ and $a \in \Sigma$ is depicted by an arrow from p to q labeled by a. Multiple transitions between the same pair of states are typically combined into a single arrow labeled with the corresponding set of input symbols. See Theorem 1.1 for an example of an automaton together with its state diagram.

Example 1.1 Consider Figure 1.1 for the state diagram of the automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with state set $Q = \{0, 1, 2, 3\}$, two letters $\Sigma = \{a, b\}$ and where $\delta \colon Q \times \Sigma \to Q$ is given by $\delta(q, b) = q + 1$ for $q \in \{0, 1, 2\}$, $\delta(3, b) = 0$, $\delta(0, a) = 1$, and $\delta(q, a) = q$ for $q \in \{1, 2, 3\}$.

1.2 Formal Definition of Automata Synchronization

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *synchronizing* (or *resettable*) if there exists a word, called a *reset* or *synchronizing word*, driving it into the same state regardless of its current state. Formally, a word $u \in \Sigma^*$ is said to be a *synchronizing* (or *reset*) *word* for $\mathcal{A}$ if $\delta(Q, u) = \{q\}$ for some $q \in Q$. In this case, we say that *u synchronizes $\mathcal{A}$*, and that q is a *synchronizing state* of $\mathcal{A}$. The *set of synchronizing words* of $\mathcal{A}$ is denoted by

[1] For a more formal construction of Σ^* and concatenation, see, for example, the textbook [194, p. 5].

[2] *Deterministic* means that δ is a function rather than a relation, and *complete* means that it is total. Both modifiers are redundant here, as we consider no other kind of automaton, but we state them as a reminder for readers used to the nondeterministic setting. In Part IV, we also introduce partial automata, where the transition function may be partially defined.

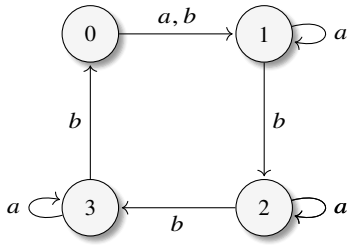


Fig. 1.1: The state diagram of the automaton from Theorem 1.1.

$\mathrm{Syn}(\mathcal{A}) = \{u \in \Sigma^* \mid \exists q \in Q\colon \delta(Q, u) = \{q\}\}$. Then $\mathcal{A}$ is synchronizing if and only if $\mathrm{Syn}(\mathcal{A})$ is non-empty.

Synchronizing words correspond to a sequence of actions (or inputs) that allow an operator (or observer) to regain control by resetting the automaton into a well-defined state. This notion arises in different contexts and applications in coding theory [87, 348, 360], robotics [198, 252, 457, 613], symbolic dynamics [605], model checking and verification [548], and group theory [36].

The introduction of synchronizing automata is often attributed to Černý and his seminal paper [140]. However, the actual history is more complicated and the concept can be traced back to earlier work of Laemmel [396] and Liu [416]. For a detailed account on the genesis of this concept, see the survey [617, Sect. 1.3].

1.3 Introductory Examples

Here, we introduce several situations and problems which can be modeled using synchronizing automata: a problem from coding theory in Section 1.3.1, a puzzle from Ashby in Section 1.3.2, a conveyor belt and part orienter design problem in Section 1.3.3, and a maze orientation problem in Section 1.3.4.

1.3.1 Error-Prone Communication Channels

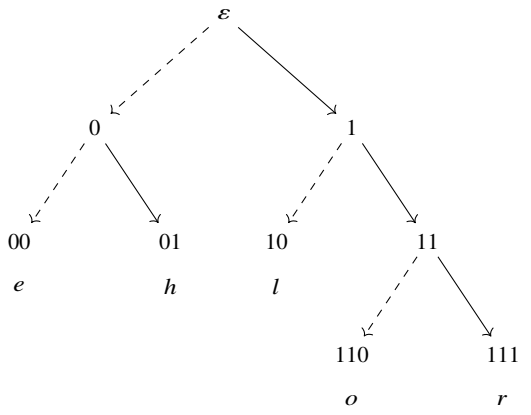
Suppose five symbols e, h, l, o, r are encoded by five binary sequences 00, 01, 10, 110, and 111, respectively, see Figure 1.2. This coding scheme is taken from Neumann [462].

When transmitting a message using this code, is it possible to recover from a loss of synchronization between the sender (encoder) and receiver (decoder) caused by channel errors? For example, suppose the sender wants to transmit "hello" to the receiver, which corresponds to the bit sequence "01 00 10 10 110". However, after sending the first three bits, the sender is uncertain whether they were received correctly.

On the receiving end, a stream of code words can be decoded with the help of the automaton shown in Figure 1.3a, which starts in the state q_ε and reflects different states

Symbol	Binary Encoding
e	00
h	01
l	10
o	110
r	111

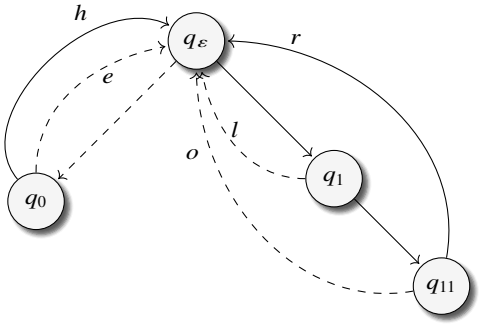
(a) Encoding of source symbols.



(b) Decision tree to decode symbols for binary inputs where dashed and solid branches correspond to 0 and 1, respectively.

Fig. 1.2: Encoding of five symbols e, h, l, o, and r by binary sequences.

of the receiver. A path starting in q_ε and going back to q_ε corresponds to a received code word, where the transition leading into q_ε indicates which code word was received. For example, when reading "01 00 10 10 110", starting from q_ε, the transitions returning to q_ε indicate the sequence "hello" in that order.



(a) Starting from q_ε, each return to q_ε corresponds to the decoding of one code word; the label on the return transition indicates the decoded source symbol.

	0	1	1	0
q_ε	q_0	q_ε	q_1	q_ε
q_0	q_ε	q_1	q_{11}	q_ε
q_1	q_ε	q_1	q_{11}	q_ε
q_{11}	q_ε	q_1	q_{11}	q_ε

(b) The state transitions for the input sequence 0110 starting from each state.

Fig. 1.3: Decoder automaton (left) for the code from Figure 1.2 where dashed (resp. solid) transitions represent input symbol 0 (resp. 1) with the synchronizing word 0110 (right).

Now suppose that the current state of the decoder automaton is unknown. This raises the following question.

Can the sender transmit a sequence of bits that guarantees the receiver is brought to the well-defined state q_ε in Figure 1.3a, from which it can correctly decode the next code word?

The answer is positive: as shown in Figure 1.3b, reading the sequence 0110 from any state of the automaton leads to q_ε. Thus, by transmitting 0110, the sender can reset the receiver in case it has lost synchronization with the stream of code words due to errors, ensuring correct decoding of subsequent bits.

Applied to the situation above, where the sender wants to transmit "hello" but is unsure whether the first three bits were transmitted correctly, the sender can send the message

$$\underbrace{010}_{\text{uncertain bits}}\ 0110\,01\,00\,10\,10\,110.$$

After reading "010 0110", the automaton is in the state q_ε, from which it can correctly decode the next code word.

1.3.2 The Haunted Mansion

Ashby [47] presented the following puzzle. A mansion is haunted by two ghostly noises, Singing and Laughter. Each of the noises can independently be either on or off, and they can be influenced by two possible actions, organ playing or burning incense, according to the following scheme described by Ashby [47, pp. 60–61].

> The Singing, in the succeeding minute, will go on as it was during the preceding minute (sounding or silent) unless there was organ-playing with no Laughter, in which case it will change to the opposite (sounding to silent, or vice versa).
>
> As for the Laughter, if there was incense burning, then it will sound or not according as the Singing was sounding or not (so that the Laughter copies the Singing a minute later). If however there was no incense burning, the Laughter will do the opposite of what the Singing did.

The puzzle asks for a sequence of actions that silences both noises when they are both active. However, more is possible: as we shall see, there exists a sequence of actions that leads to both noises becoming silent in every possible situation.

We can model the situation by an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with state set

$$Q = \{(S, L), (\overline{S}, L), (\overline{S}, \overline{L}), (S, \overline{L})\},$$

where S and $\overline{S}$ represent the presence and absence of singing, respectively, and similarly for L and $\overline{L}$ with laughter. Furthermore, the input alphabet is $\Sigma = \{a_{OB}, a_{\overline{O}B}, a_{\overline{OB}}, a_{O\overline{B}}\}$ where the symbols represent the four possible combinations of playing the organ (O), not playing organ ($\overline{O}$), burning incense (B), or not burning incense ($\overline{B}$). The transitions are given as in Figure 1.4.

The word $a_{\overline{O}\overline{B}}a_{O\overline{B}}a_{\overline{O}B}$ is a synchronizing word ending in the synchronizing state $(\overline{S}, \overline{L})$. Hence, by executing the corresponding actions, the ghosts end up silent regardless of their current behavior. As pointed out by Volkov [613], this seems to be the first

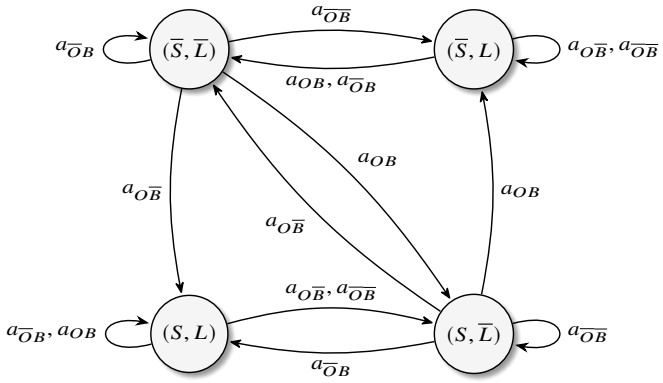


Fig. 1.4: Ashby's ghost taming automaton. The word $a_{\overline{OB}}a_{O\overline{B}}a_{\overline{OB}}$ is synchronizing.

publication in which a synchronizing automaton occurs. However, Ashby did not comment on the existence of a sequence that silences both ghosts under all circumstances.

1.3.3 Part Orienters

Suppose that, at the end of an assembly line, square-shaped components with a small square inset on one side (see Figure 1.5a) must arrive with the inset on the left side (Ananichev and Volkov [22]; see also [496, 613, 617]). For simplicity, we assume that there are only four possible orientations, shown in Figure 1.5b. As the components move along the assembly line from left to right, they encounter two kinds of obstacles: high and low. A high obstacle always rotates a component by 90-degree counterclockwise. A low obstacle rotates the component in the same way only when the inset is on the top side, since only in this orientation does the obstacle touch the component. Can one place a sequence of obstacles along the assembly line such that, regardless of the initial orientation, the final orientation always has the inset on the left side?



(a) A polygonal component.

(b) The four possible orientations.

Fig. 1.5: A polygonal component and its possible orientations on an assembly line.

Taking the four possible orientations as the states of an automaton, the effects of the two types of obstacles can be modeled by two input letters, as illustrated on the left-hand

side of Figure 1.6. As the reader might notice, this construction yields, in essence, the same automaton as the one given earlier in Figure 1.1. The original problem reduces to determining whether this automaton admits a synchronizing word. One possible synchronizing sequence is low, high, high, high, low, high, high, high, low. In the case where a component arrives with its insert oriented to the left, the effect of this sequence is shown on the right-hand side of Figure 1.6.

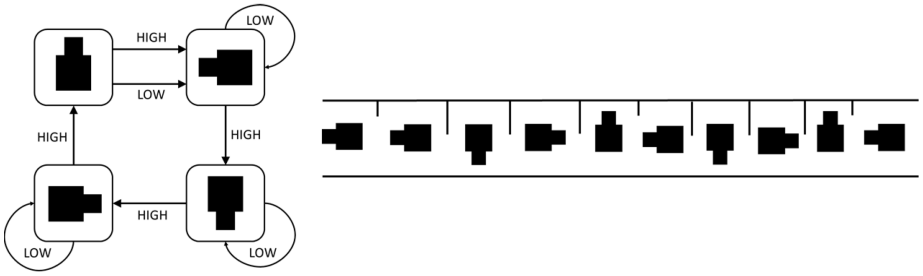


Fig. 1.6: Left: The part orienter problem modeled as a finite automaton. Right: Sequence of effects on one component moving from left to right. Image taken from Podolak et al. [496].

The above scheme works without the use of sensors and does not require any observer or external mechanism to orient the parts. Employing sensorless methods for part orientation is often preferable to using more complicated machinery because of their robustness, simplicity, and ease of maintenance. In another sensorless approach to part feeding, parts are placed on a vibrating surface and must pass certain obstacles. Such orientation schemes have apparently been employed since the mid-20th century; see [541] for a survey on vibratory part feeders. Sensorless methods for orienting parts without vibrating surfaces were first described by Erdmann and Mason [200], who simplified and mathematically analyzed a mechanical device originally introduced by Grossman and Blasgen [264].

The first author to investigate the computational complexity of designing sensorless part orienters was Natarajan [457, 458]. He studied the engineering problem of orienting polygonal objects on conveyor belts using simple, robust, and sensorless methods based on mechanical contact points installed along the belt. Mathematically, he modeled this problem both as a pebbling game and as the problem of finding a sequence of functions from a given set that realizes a constant function. He noted (without proof) that a generalization of the latter formulation is PSPACE-complete and gave polynomial-time algorithms for several variants of these problems.

Natarajan's algorithmic approach to part orienting stimulated substantial further research in this area. For example, Goldberg [252] introduced a scheme that orients parts by squeezing them with parallel-jaw grippers and gave a polynomial-time algorithm for designing a sequence of squeezes that orients any part, conjecturing that a linear number of squeezes always suffices. Chen and Ierardi [146] later confirmed this conjecture.

Eppstein [198] observed the connection to synchronizing automata previously studied in automata theory, characterized the class of automata investigated by Natarajan [457], and improved Natarajan's algorithms. In this sense, synchronizing automata emerged independently in the mechanical engineering literature. Part orienting as a motivating example for synchronizing automata was later popularized by Ananichev and Volkov [22, 23]; see also the widely cited survey [613].

1.3.4 Lost in a Maze

Imagine a maze in which every junction has exactly k outgoing corridors and all junctions look alike, so that a visitor can never tell where they currently are. The corridors leaving a junction are to be marked with k distinct colors, one color per corridor, and a visitor who is told a sequence of colors follows the corresponding corridors one after the other. The task of whoever installs the signs is the following.

> Can the corridors be colored so that some sequence of colors leads every visitor to one and the same junction, no matter where in the maze they started?

Such a sequence does not point at the exit directly; it brings all visitors to a common junction, and from there a single fixed route takes them out. Coloring the corridors turns the maze into a deterministic finite automaton: the junctions are the states, the colors form an alphabet of size k, and a corridor colored a from junction p to junction q gives the transition $\delta(p, a) = q$. The question above then asks whether the corridors admit a coloring for which the resulting automaton is synchronizing.

Not every maze does. If, for instance, the junctions can be split into two groups such that every corridor leads from one group to the other, then two visitors starting in different groups stay in different groups forever, whatever the coloring and whatever sequence they are given. Ruling out obstructions of this kind is in fact enough: the *road coloring theorem*, conjectured by Adler, Goodwyn, and Weiss [1] and proved by Trahtman [605] some thirty years later, states that a synchronizing coloring exists if and only if the maze is strongly connected and the lengths of its closed walks have greatest common divisor one.

1.4 Reset Length and the Černý Conjecture

The *reset length* of a synchronizing automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is

$$\mathrm{rt}(\mathcal{A}) = \min\{|u| : u \in \mathrm{Syn}(\mathcal{A})\}.$$

Černý [140] proved that, for $n \geq 1$, we have

$$(n-1)^2 \leq \max\{\mathrm{rt}(\mathcal{A}) \mid \mathcal{A} \text{ has } n \text{ states and } \mathrm{Syn}(\mathcal{A}) \neq \emptyset\} \leq 2^n - n - 1.$$

For the lower bound, he gave the *Černý family of automata* $C_n = ([n], \{a, b\}, \delta_n)$ where $n \geq 2$ and

$$\delta_n(i, b) = \begin{cases} i+1 & \text{if } i < n-1, \\ 0 & \text{if } i = n-1, \end{cases} \qquad \delta_n(i, a) = \begin{cases} 1 & \text{if } i = 0, \\ i & \text{if } i \neq 0, \end{cases}$$

see Figure 1.7. The automaton from the part orientation problem in Figure 1.6 equals C_4. Below, we give a proof that $\mathrm{rt}(C_n) = (n-1)^2$, which also shows that $(ab^{n-1})^{n-2}a$ is the only shortest synchronizing word for C_n.

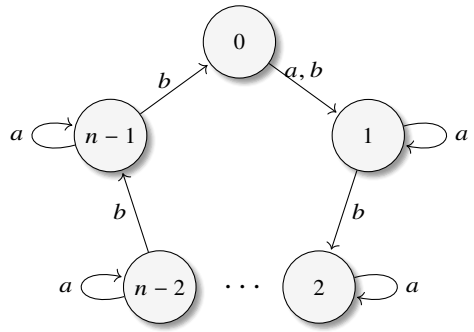


Fig. 1.7: Automata from the Černý family C_n. See Section 1.4.

Theorem 1.2 (Černý [140]) *For $n \geq 2$, we have $\mathrm{rt}(C_n) = (n-1)^2$. Moreover, $(ab^{n-1})^{n-2}a$ is the unique shortest synchronizing word of C_n.*

Proof. Identify $Q = [n]$ with $\mathbb{Z}_n$, so that $\delta_n(p, b) = p + 1$ (modulo n) for $p \in Q$, while a maps 0 to 1 and fixes all other states. An *interval* is a non-empty set $[p, q] = \{p, p+1, \ldots, q\}$ with $p, q \in Q$. Note that $Q = [p, p-1]$ for every $p \in Q$, and that for $I \neq Q$ the states p and q with $I = [p, q]$ are determined by I, as p is the only state in I with $p - 1 \notin I$ and q the only one with $q + 1 \notin I$. In particular, an interval containing 1 but not 0 begins at 1, and an interval containing 0 but not 1 ends at 0.

One verifies inductively that $\delta_n(Q, (ab^{n-1})^j a) = [1, n-1-j]$ for $0 \leq j \leq n-2$, so $(ab^{n-1})^{n-2}a$ synchronizes C_n and a shortest synchronizing word exists. Let $w = x_1 \cdots x_m$ with $x_1, \ldots, x_m \in \Sigma$ be one with $\delta_n(Q, w) = \{q\}$ and let

$$I_j = \{\, p \in Q \mid \delta_n(p, x_{j+1} \cdots x_m) = q \,\} \qquad (0 \leq j \leq m).$$

Then $I_m = \{q\}$, $I_0 = Q$ and $I_{j-1} = \{\, p \in Q \mid \delta_n(p, x_j) \in I_j \,\}$ for $j \geq 1$. Every state p satisfies $\delta_n(p, x_1 \cdots x_j) \in I_j$; in particular, no I_j is empty. Moreover $I_j \neq Q$ for $j \geq 1$, as $I_j = Q$ means that $x_{j+1} \cdots x_m$ maps every state to q and is thus a shorter synchronizing word.

Let $1 \leq j \leq m$ and assume that I_j is an interval, say $I_j = [p_j, q_j]$, which is possible in a unique way as $I_j \neq Q$. Since $\delta_n(p, a) \in I_j$ holds for $p \neq 0$ exactly when $p \in I_j$, and for $p = 0$ exactly when $1 \in I_j$, exactly one of the following cases occurs.

1. $x_j = b$; then $I_{j-1} = [p_j - 1, q_j - 1]$, so $|I_{j-1}| = |I_j|$ and $p_{j-1} = p_j - 1$.
2. $x_j = a$ and $\{0, 1\} \subseteq I_j$ or $\{0, 1\} \cap I_j = \emptyset$; then $I_{j-1} = I_j$.
3. $x_j = a$ and $0 \in I_j$ but $1 \notin I_j$; then $I_j = [p_j, 0]$ with $p_j \neq 0$ as $I_{j-1} \neq \emptyset$, and $I_{j-1} = I_j \setminus \{0\} = [p_j, n - 1]$.
4. $x_j = a$ and $1 \in I_j$ but $0 \notin I_j$; then $p_j = 1$, and $I_{j-1} = I_j \cup \{0\} = [0, q_j]$, so $|I_{j-1}| = |I_j| + 1$ and $p_{j-1} = 0 = p_j - 1$.

In each case I_{j-1} is again an interval. So, starting from $I_m = [q, q]$ and descending, every I_j is an interval.

The cases (2) and (3) never occur: in both we have $I_{j-1} \subseteq I_j$, and, since every state p satisfies $\delta_n(p, x_1 \cdots x_{j-1}) \in I_{j-1} \subseteq I_j$ and hence $\delta_n(p, x_1 \cdots x_{j-1} x_{j+1} \cdots x_m) = q$, deleting the letter x_j from w leaves a shorter synchronizing word in these cases.

Since $|I_m| = 1$ and $|I_0| = n$, the word w contains exactly $n-1$ letters a. Let $j_1 < \cdots < j_{n-1}$ be their positions, so that j_1 is the position of the first letter a in w and j_{n-1} that of the last. Here $j_1 = 1$, as otherwise $x_1 = b$ and $Q = I_0 = [p_1 - 1, q_1 - 1]$, giving $I_1 = Q$, which we excluded; and $j_{n-1} = m$, as otherwise $x_m = b$ and $I_{m-1} = \{q - 1\}$ by case (1), so that $x_1 \cdots x_{m-1}$ maps every state to $q - 1$ and is shorter. Now let $2 \leq k \leq n - 1$. The positions j with $j_{k-1} < j < j_k$ all carry the letter b, and by case (4) we have $p_{j_k - 1} = 0$ and $p_{j_{k-1}} = 1$. Since each b decreases the first state by one and these run from $p_{j_k - 1} = 0$ down to $p_{j_{k-1}} = 1$, their number $j_k - j_{k-1} - 1$ is congruent to -1 modulo n. It equals $n - 1$, since otherwise it is at least $2n - 1$ and deleting n consecutive letters b, which changes no transition as $\delta_n(p, b^n) = p$, would again shorten w. Hence $j_k - j_{k-1} = n$ for all k and therefore

$$m = \sum_{k=2}^{n-1} (j_k - j_{k-1}) + 1 = (n - 2)n + 1 = (n - 1)^2.$$

So the letters a occupy the positions $1, n + 1, \ldots, (n - 2)n + 1 = m$, while all remaining positions carry the letter b. Thus $\mathrm{rt}(C_n) = (n - 1)^2$ and $w = (ab^{n-1})^{n-2}a$. □

Eppstein [198] introduced the notion of an interval for a more general class of automata. For a shortest synchronizing word $w = uv$ of C_n, he proved $\mathrm{rt}(C_n) = (n - 1)^2$ by bounding the length of a shortest interval containing $\delta_n([n], u)$ [198, Thm. 3], whereas the original proof by Černý [140, Lem. 1] considered the length of a largest interval in $[n] \setminus \delta_n([n], u)$. The proof above instead uses the sets $\{p \in [n] \mid \delta_n(p, v) = q\}$, which are intervals, whenever non-empty, for every word v, and this also yields the uniqueness asserted in Theorem 1.2. Salomaa [547, Sec. 7] later gave a proof related to Černý's in the form of a solitaire game in which states are marked by stones (see also [546, Thm. 8], where the argument is adapted to the case in which all states are mapped to a fixed state). A graph-theoretical proof relating the reset length to the exponent of certain digraphs was given by Ananichev et al. [21, Thm. 3] (see also [360, Prop. 3.1]).

In [141, 580] the authors conjectured the following bound (the so-called *Černý bound*).

Černý conjecture: We have $\mathrm{rt}(\mathcal{A}) \leq (n-1)^2$ for every synchronizing n-state $\mathcal{A}$.

This conjecture, which has remained open for more than five decades, has stimulated extensive research since its formulation. The reader might wonder why the conjecture is not named after Starke, even though his paper [580] predates [141]. One possible reason is that older papers often referred only to the seminal work [140] when discussing the conjecture, despite the fact that, although it introduced synchronizing automata and the Černý automata, it did not explicitly state the conjecture itself. Another possible reason is that Černý discussed the conjecture at two conferences shortly after the publication of [140]. In any case, even the term "Černý–Starke conjecture" was never adopted in the literature; see [615] for further discussion on the origin of this conjecture.

Table 1.1: Published upper bounds for the reset length. Kfoury's bound was communicated to him by Paterson, see [366].

Upper Bound	Remark	Reference
$2^n - n - 1$		(1964, Černý [140])
$\frac{1}{2}n^3 - \frac{3}{2}n^2 + n + 1$		(1966, Starke [580])
$\frac{1}{2}n^3 - n^2 + \frac{1}{2}n$		(1970, Kohavi [375])
$\frac{1}{3}n^3 - n^2 - \frac{1}{3}n + 6$		(1970, Kfoury [366])
$\frac{1}{3}n^3 - \frac{3}{2}n^2 + \frac{25}{6}n - 4$		(1971, Černý et al. [141])
$\frac{7}{27}n^3 - \frac{17}{18}n^2 + \frac{17}{6}n - 3$	$n = 3k$	(1977, Pin [485])
$\left(\frac{1}{2} + \frac{\pi}{36}\right)n^3 + o(n^3)$		(1978, Pin [487, 484, 488])
$\frac{1}{6}n^3 - \frac{1}{6}n$		(1983, Pin/Frankl [227, 488], 1987, Klyachko et al. [372])
$\alpha n^3 + o(n^3)$	$\alpha \approx 0.1664$	(2018, Szykuła [593])
$\alpha n^3 + o(n^3)$	$\alpha \leq 0.1654$	(2019, Shitov [568])

Table 1.1 lists the published upper bounds in chronological order. Klyachko et al. [372] independently proved the Pin–Frankl bound from Table 1.1 behind the Iron Curtain. Pin [488] actually concluded the bound $(n^3 - n)/6 - 1$ for $n \geq 4$ with Frankl's result [227]. The bound $(n^3 - n)/6$ had previously been stated by Kohavi and Winograd [376, 377], yet, somewhat mysteriously, their work was never cited in the subsequent literature. Their argument broadly follows the same approach as that of Pin [488], analyzing traces of 2-sets in shortest synchronizing words. However, their treatment of certain maximal traces appears to lack sufficient justification, which may explain why their work was largely ignored (for this reason we also left it out in Table 1.1). Additionally, Salomaa [546, Thm. 4] proved the bound $\frac{1}{2}n^3 - \frac{3}{2}n^2 + 2n$ for the length of a shortest word mapping all states to a fixed state, rather than merely to some state as required for synchronization.

The Černý conjecture [140] has been confirmed for a variety of classes of automata, to name a few of them (without further explanation and with no claim for completeness):

1. circular automata [186, 187, 486],
2. oriented automata [198],
3. (weakly generalized) monotonic automata [22, 23, 24] and 0-monotonic automata [18],
4. automata with a trap state [52, 534],

5. commutative [54, 212, 533], partially ordered [537], solvable [534], and automata whose transition monoid belongs to the varieties **DS** [16] or **EDS** [15],
6. strongly reset automata [534],
7. nilpotent and (reverse and generalized) definite automata [54, 534],
8. automata whose set of synchronizing words is a shuffle ideal [481],
9. Eulerian automata [359],
10. automata preserving a chain of partial orders [614],
11. automata whose transition monoid contains a $\mathbb{Q}$I-group [36, 40],
12. aperiodic automata [604],
13. automata with a blocking state [52],
14. automata whose transformation semigroup contains a group fulfilling certain representation-theoretical conditions [582],
15. one-cluster automata with a cycle of prime length [583],
16. automata $\mathcal{A}$ with restrictions on generating words for $\mathrm{Syn}(\mathcal{A})$ [228, 501],
17. automata that cannot recognize $\{a,b\}^*ab\{a,b\}^*$ [15],
18. certain aperiodically 1-contracting automata [182],
19. decoder automata for finite maximal prefix codes [85],
20. automata having letters of a certain rank [85],
21. automata respecting intervals of a digraph [259],
22. strongly semisimple automata [14] (related to Item 16, see also [514]),
23. automata with two idempotent letters [616, Prop. 5],
24. 2-junction automata [260],
25. automata with simple idempotents having no permutational letter or containing a regular permutation group in the transition monoid [536],
26. automata for which their sets of synchronizing words have minimal generating words not containing a word of a certain length [228],
27. binary automata with a simple idempotent letter [618],
28. automata with strongly connected restricted Rystsov graph [135, 618],
29. media automata [199],
30. automata without involutions [601].

Furthermore, quadratic upper bounds on the reset length have been obtained for automata whose transition monoid contains every transformation [256], for one-cluster automata [70], for quasi-Eulerian and quasi-one-cluster automata [81], for regular automata [532, 531], for automata with coinciding cycles [522], for automata with simple idempotents [535], for automata having a one-point singular and satisfying additional conditions [524],[3] for automata whose transformation monoid contains a transitive group [647], and for completely reachable automata [205], the latter improving and extending the results of [256]. For several subclasses of automata, bounds tighter than $(n-1)^2$ are known. For instance, for solvable automata [534], the bound is $n-1$. A comprehensive overview of such bounds and subclasses can be found in the living list [618] of results related to the Černý conjecture maintained by Mikhail Volkov.

[3] Both [22] and [524] also contain more general formulations of the Černý conjecture for transformation monoids.

Chapter 2
Outline and Contribution

First, in Section 2.1 we introduce the two central problems with which the thesis is concerned: the constrained synchronization problem (Section 2.1.1) and the description of automata for sets of synchronizing words (Section 2.1.2). Section 2.2 gives an overview on the organization of the present work. Finally, Section 2.3 lists the papers on which this work is based, as well as the unpublished results that we have included.

2.1 The Problems

2.1.1 The Constrained Synchronization Problem

The *constrained synchronization problem* asks whether a given deterministic and complete semiautomaton has a synchronizing word contained in a given regular *constraint language*. Formally, let Σ be an alphabet and $L \subseteq \Sigma^*$ be a regular language (see Sections 3.9.4 and 6.7.1 for formal definitions of decision problems and regular languages, respectively). The problem Sync$[L]$ of *constrained synchronization* with respect to the *constraint language* L is the following decision problem.

Sync$[L]$
Input: A deterministic and complete semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$.
Question: Does there exist a word $w \in L$ synchronizing $\mathcal{A}$?

This problem, systematically studied in [215], firstly appeared in connection with a reduction procedure introduced in [271], as well as implicitly in reductions used in [438]. **??** provides further motivation and details.

2.1.2 Automata for Sets of Synchronizing Words

The set of synchronizing words of a given automaton can be accepted by another automaton of size exponential in the size of the original automaton (see Chapter 3 for the definition of automata as language acceptors). In this thesis, we also study automata for sets of synchronizing words. Our motivation comes from the constrained synchronization problem, whose computational complexity is governed by two parameters: the *structure of the constraint language* and the *structure of the set of synchronizing words* of the given input automata. If an automaton recognizing the set of synchronizing words can be constructed in polynomial time for a class of input automata, then the constrained synchronization problem (with any constraint language) is solvable in polynomial time for this class. An example is the class of commutative automata over a fixed alphabet (see **??**). Consequently, NP- or PSPACE-hardness results (see Section 3.9.5 for definitions of complexity classes) imply that automata for the corresponding sets of synchronizing words cannot, in general, be constructed in polynomial time. This motivates our interest in automata needing exponentially larger accepting automata for their sets of synchronizing words.

Beyond its connection to the constrained synchronization problem, the study of automata for sets of synchronizing words is also motivated by the following observations.

1. For known families of *slowly synchronizing automata*, that is, automata whose reset length is close to the Černý bound, the minimal automata recognizing their sets of synchronizing words are exponentially larger [440, 441]. This raises the question of whether there is a deeper connection between the reset length of an automaton and the size of a minimal automaton recognizing its set of synchronizing words.
2. The operation that maps a regular language to the set of synchronizing words of its minimal automaton is regularity-preserving and can therefore be studied from the perspective of the state complexity of operations [237, 643].
3. The class of sets of synchronizing words of automata coincides with the class of regular ideal languages (Theorem 7.1). Thus, every regular ideal language can be represented by an automaton having it as its set of synchronizing words. This representation can be exponentially more succinct than a representation by an accepting automaton, as witnessed by known families of slowly synchronizing automata [440, 441]. From the perspective of the descriptional complexity of regular languages, it is therefore natural to investigate those regular ideal languages for which the gap between these two representations is maximal. In automata-theoretic terms, this leads to the study of automata for which minimal automata accepting their sets of synchronizing words are as large as possible.
   Furthermore, these observations motivate the definition of the *reset complexity* of a regular ideal language as the size of a smallest automaton having this language as its set of synchronizing words. The Černý conjecture is equivalent to the statement that the reset complexity of a regular ideal language is at least the square root of the length of a shortest word in the language, plus one [440, 441]. This yields a language-theoretic approach to the Černý conjecture through the investigation of shortest words in regular ideal languages [197, 228, 433, 442, 506, 513].

4. As we will see, our results reveal connections between subset reachability properties, the size of automata for sets of synchronizing words, and the well-known class of primitive permutation groups.

2.2 Organization of the Thesis

The thesis consists of four parts. In Part I we give a short introduction on synchronizing automata and an overview of the thesis. Part II introduces further notions, automata classes, and gives preliminary computational complexity results. A detailed overview is given in Section 2.2.1. Part III is concerned with automata accepting the set of synchronizing words of a given automaton and is outlined in Section 2.2.2 below. Part IV deals with the constrained synchronization problem and we give an overview in Section 2.2.3.

Parts III and IV are the main parts. Unless stated otherwise, all the results stated in these parts are due to the author of this thesis.

Historically, and as outlined in Section 2.1, our research on the constrained synchronization problem led us to investigate automata for sets of synchronizing words. However, in the present work, we first discuss automata for sets of synchronizing words in Part III before turning to the constrained synchronization problem in Part IV. We chose this order of presentation because Part IV relies on results from Part III, whereas Part III is independent of Part IV. Moreover, the study of automata accepting sets of synchronizing words is a natural topic in its own right, and we expand it beyond its application to the constrained synchronization problem.

2.2.1 Structure of Part II: Preliminaries

Here, we introduce further notions in Chapter 3. In Chapter 4 we investigate classes of transformations. Permutation groups are investigated in Chapter 5. More automata-theoretical notions are given in Chapter 6. Classes of automata and their properties are investigated in Chapter 7. Computational complexity results on subset mapping problems in automata with respect to certain subclasses are investigated in Chapter 8.

Most results in this part are taken from the literature. Some results on idempotent and one-point singular transformations from Chapter 4 are new. The notion of strong $\frac{3}{2}$-connectivity for digraphs introduced here is new. Further, some results in Chapter 7 are new, see also Section 2.3.2 for more details. Some structural results on solvable automata in Section 7.10 appear in [313]. Chapter 6 on automata theory contains some results on morphism and quotient automata that we have not seen elsewhere. The results in Chapter 8 were first presented in [313].

2.2.2 Structure of Part III: Automata for Sets of Synchronizing Words

Part III studies automata that accept the set of synchronizing words of a given automaton. We derive new notions and conditions related to the size of accepting automata for sets of synchronizing words, also giving conditions under which minimal automata for such sets have maximal size. We further relate our findings to classical automata-theoretic results from the field of descriptional complexity. This part is based on the papers [297, 299, 303, 304, 306, 307, 309, 315], but the presentation has been extended and unified, with additional new results included.

Except in **????????** and the discussion sections, which review notions and results from the literature, all results presented are novel and due to the author. Next, we describe the organization of Part III and give a broad overview of each chapter and its relation to the work on which this thesis is based. A more detailed overview of the chapters, including descriptions of their individual sections, is given at the beginning of each chapter.

In **??**, we first note that the set of synchronizing words is a regular language acceptable by the power automaton. To obtain lower bounds on the size of automata accepting the set of synchronizing words, we apply minimality conditions to the power automaton. This leads us to the notions of reachability of subsets and distinguishability of pairs of subsets. The rest of the chapter investigates these two notions, giving sufficient conditions for reachability and distinguishability. We also introduce completely reachable automata, a class previously studied in the literature, and the new class of completely distinguishable automata, for which we prove several equivalent characterizations. The results in this chapter are based on [308, 315], where completely distinguishable automata were introduced, but it also contains new unpublished material, see Section 2.3.2.

?? introduces sync-maximal automata as synchronizing automata for which an automaton accepting the set of synchronizing words is as large as possible, i.e., for which the gap between the state complexity and the reset complexity of the set of synchronizing words is maximal. We investigate sufficient conditions and derive structural properties, showing that sync-maximal automata are closely related to completely reachable automata. We also give various examples. The notion of sync-maximal automata, as well as the structural results, were first introduced in [308, 315].

?? discusses graph-theoretical constructions that yield sufficient conditions for complete reachability and complete distinguishability. First, we review constructions and results from the literature. We then extend some of these constructions to arbitrary transformation semigroups. Finally, we give graph-theoretical conditions ensuring complete reachability or distinguishability of certain pairs of subsets. The results from this chapter are new and have not been published before.

?? investigates the following subclasses of automata: commutative automata, automata in which every letter has defect at most one, SI-automata, and circular automata. For commutative automata, we show that an automaton recognizing the set of synchronizing words can be computed in polynomial time when the alphabet is fixed. This result was published in [299] in the context of the constrained synchronization problem for commutative constraint languages, and will be used later in Part IV. We then consider automata in which every letter has defect at most one. In this setting, we show that if

the group of units of the transformation monoid is transitive, then synchronizability implies complete distinguishability. As a consequence, a binary automaton with at least three states is completely reachable if and only if it is sync-maximal. These results are taken from [315]. Then we investigate SI-automata. For example, we show that in synchronizing SI-automata distinct pairs 2-sets are distinguishable. We derive this from more general and unpublished results, but it also follows from [306, Lem. 11]. However, the paper [306] contains a mistake stemming from a false statement [306, Prop. 2]. We discuss this in detail at the end of **??**. Lastly, we investigate circular automata and state a sufficient condition for complete reachability and complete distinguishability. A special case of this condition was published in [307], with a preliminary version presented in [303].

?? studies the decision problems for complete reachability, complete distinguishability, and sync-maximality, as well as related questions. For example, we prove that these problems are NL-hard for SI-automata, with complete reachability even being NL-complete for SI-automata, and in PSPACE for general automata. The results on SI-automata were published in [308, 315].

?? discusses relations to permutation groups. We introduce primitive groups and characterize them in terms of complete reachability, complete distinguishability, and in terms of sync-maximality of associated automata. The relation of primitive groups to complete reachability was first published in [297], and the relation to sync-maximality in [304]. The extended journal publication [309] of [297, 304] contains both of these characterizations. We then introduce A-primitivity and study its connection to congruences, orbitals, generating sets, synchronization, reachability, and homogeneity properties. Our results generalize existing characterizations of primitivity from the literature, and a characterization by a homogeneity property also gives a new characterization. The notion of A-primitivity is introduced here for the first time, and the results of this chapter have not been published previously.

?? discusses families of slowly synchronizing automata. We observe that, for most of these families of automata, the minimal automata accepting their sets of synchronizing words have exponential size. In particular, we use results from Chapter **??** on circular automata to deduce sync-maximality for several families of slowly synchronizing circular automata from the literature, including one family for which sync-maximality has previously only been conjectured. This confirmation of the conjecture was presented in [303, 307]. We then show, for three families of slowly synchronizing but not sync-maximal automata, that minimal automata for their sets of synchronizing words are exponentially larger. Based on our observations, we formulate various conjectures. Apart from the confirmation of the aforementioned conjecture, the results from this chapter have not been published.

2.2.3 Structure of Part IV: The Constrained Synchronization Problem

Part IV deals with the constrained synchronization problem and is based on the works [215, 294, 296, 298, 299, 301, 305, 316]. The results from [294, 298, 299, 301, 316] were revisited with an emphasis on a more streamlined, coherent, and sim-

plified presentation. This was achieved primarily through the use of general methods introduced in [301, 316].

In **??**, we give motivational examples for the constrained synchronization problem.

?? introduces additional notions that we will need in Part IV. We also present a new characterization of solid codes that, to the best of our knowledge, has not been published before.

?? presents general results on manipulating constraint languages while preserving the complexity or to deduce complexity lower bounds. Furthermore, we give sufficient conditions for NP- and PSPACE-hardness. Some of these results were presented in [301, 316], which generalized results from [215], and some results are new.

The central question investigated in **??** is the computational complexity of Sync[L] for different classes of regular constraint languages $L \subseteq \Sigma^*$. While a complete classification remains an *open problem*, we present several partial results by giving complete classifications for the following classes of constraint languages: letter-bounded languages, languages accepted by partial automata (PDFAs, introduced in **??**) with at most two states over an arbitrary alphabet and at most three states over a binary alphabet, and commutative languages. We also obtain results for infinite, sparse, and monoid languages, as well as for languages accepted by strongly connected PDFAs. The classification for commutative constraint languages appeared in [294, 299]. Here, we present a simplified formulation and proof. The results for sparse and letter-bounded languages appeared in [296, 298].

Lastly, motivated by the question of what features are needed in input automata to realize certain computational complexities, in **??** we investigate specific types of input automata. Some of the results from this chapter appeared in [305], and some mistakes from this paper have been corrected.

2.3 Work by the Author Relevant to the Thesis

2.3.1 Published Work

The present thesis contains and extends material from the following papers and conference contributions published since 2019, in which the author was involved.

[215] Henning Fernau et al. “Computational Complexity of Synchronization under Regular Constraints”. In: *44th International Symposium on Mathematical Foundations of Computer Science, MFCS 2019, August 26-30, 2019, Aachen, Germany*. Ed. by Peter Rossmanith, Pinar Heggernes, and Joost-Pieter Katoen. Vol. 138. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, 63:1–63:14. ISBN: 978-3-95977-117-7. DOI: 10.4230/LIPIcs.MFCS.2019.63

[296] Stefan Hoffmann. “On A Class of Constrained Synchronization Problems in NP”. In: *Proceedings of the 21st Italian Conference on Theoretical Computer Science, Ischia, Italy, September 14-16, 2020*. Ed. by Gennaro Cordasco and Luisa Gargano and Adele A. Rescigno. Vol. 2756. CEUR Workshop Proceedings. available online:

http://ceur-ws.org/Vol-2756/paper_19.pdf. CEUR-WS.org, 2020, pp. 145–157

[294] Stefan Hoffmann. "Computational Complexity of Synchronization Under Regular Commutative Constraints". In: *Computing and Combinatorics - 26th International Conference, COCOON 2020, Atlanta, GA, USA, August 29-31, 2020, Proceedings*. Ed. by Donghyun Kim and R. N. Uma and Zhipeng Cai and Dong Hoon Lee. Vol. 12273. Lecture Notes in Computer Science. Springer, 2020, pp. 460–471. DOI: 10.1007/978-3-030-58150-3_37

[303] Stefan Hoffmann. "State Complexity of the Set of Synchronizing Words for Circular Automata and Automata over Binary Alphabets". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 318–330. DOI: 10.1007/978-3-030-68195-1_25

[297] Stefan Hoffmann. "Completely Reachable Automata, Primitive Groups and the State Complexity of the Set of Synchronizing Words". In: *Language and Automata Theory and Applications - 15th International Conference, LATA 2021, Milan, Italy, March 1-5, 2021, Proceedings*. Ed. by Alberto Leporati et al. Vol. 12638. Lecture Notes in Computer Science. Springer, 2021, pp. 305–317. DOI: 10.1007/978-3-030-68195-1_24

[301] Stefan Hoffmann. "Ideal Separation and General Theorems for Constrained Synchronization and Their Application to Small Constraint Automata". In: *Computing and Combinatorics - 27th International Conference, COCOON 2021, Tainan, Taiwan, October 24-26, 2021, Proceedings*. Ed. by Chi-Yeh Chen et al. Vol. 13025. Lecture Notes in Computer Science. Springer, 2021, pp. 176–188. DOI: 10.1007/978-3-030-89543-3_15

[300] Stefan Hoffmann. "Constrained Synchronization and Subset Synchronization Problems for Weakly Acyclic Automata". In: *Developments in Language Theory - 25th International Conference, DLT 2021, Porto, Portugal, August 16-20, 2021, Proceedings*. Ed. by Nelma Moreira and Rogério Reis. Vol. 12811. Lecture Notes in Computer Science. Springer, 2021, pp. 204–216. DOI: 10.1007/978-3-030-81508-0_17

[299] Stefan Hoffmann. "Constrained Synchronization and Commutativity". In: *Theoretical Computer Science* 890 (2021), pp. 147–170. DOI: 10.1016/j.tcs.2021.08.030 (extended version of [294])

[298] Stefan Hoffmann. "Computational Complexity of Synchronization Under Sparse Regular Constraints". In: *Fundamentals of Computation Theory - 23rd International Symposium, FCT 2021, Athens, Greece, September 12-15, 2021, Proceedings*. Ed. by Evripidis Bampis and Aris Pagourtzis. Vol. 12867. Lecture Notes in Computer Science. Springer, 2021, pp. 272–286. DOI: 10.1007/978-3-030-86593-1_19. arXiv: 2108.00081

[304] Stefan Hoffmann. "Sync-Maximal Permutation Groups Equal Primitive Permutation Groups". In: *Descriptional Complexity of Formal Systems - 23rd IFIP WG 1.02 International Conference, DCFS 2021, Virtual Event, September 5, 2021, Proceedings*. Ed. by Yo-Sub Han and Sang-Ki Ko. Vol. 13037. Lecture Notes in Computer Science. Springer, 2021, pp. 38–50. DOI: 10.1007/978-3-030-93489-7_4

[306] Stefan Hoffmann. “Reset Complexity and Completely Reachable Automata with Simple Idempotents”. In: *Descriptional Complexity of Formal Systems - 24rd IFIP WG 1.02 International Conference, DCFS 2022, August 29 - 31, 2022, Debrecen, Hungary, Proceedings*. Ed. by Yo-Sub Han and György Vaszil. Vol. 13439. Lecture Notes in Computer Science. Springer, 2022, pp. 85–99. DOI: 10.1007/978-3-031-13257-5_7

[305] Stefan Hoffmann. “Constrained Synchronization for Monotonic and Solvable Automata and Automata with Simple Idempotents”. In: *Implementation and Application of Automata - 26th International Conference, CIAA 2022, Rouen, France, June 28 - July 1, 2022, Proceedings*. Ed. by Pascal Caron and Ludovic Mignot. Vol. 13266. Lecture Notes in Computer Science. Springer, 2022, pp. 225–237. DOI: 10.1007/978-3-031-07469-1_18

[316] Stefan Hoffmann. “General Theorems for Constrained Synchronization and their Application to Small Constraint Automata”. In: *Journal of Combinatorial Optimization* (to appear) (extended version of [301])

[309] Stefan Hoffmann. “New Characterizations of Primitive Permutation Groups with Applications to Synchronizing Automata”. In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021, p. 105086. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105086 (extended version of [297])

[307] Stefan Hoffmann. “Binary and Circular Automata Having Maximal State Complexity for the Set of Synchronizing Words”. In: *Information and Computation* 295 (2023). Special Issue: Selected papers of the 15th International Conference on Language and Automata Theory and Applications, LATA 2021, p. 105076. ISSN: 0890-5401. DOI: 10.1016/j.ic.2023.105076 (extended version of [303])

[308] Stefan Hoffmann. “Completely Distinguishable Automata and the Set of Synchronizing Words”. In: *Developments in Language Theory - 27th International Conference, DLT 2023, Umeå, Sweden, June 12-16, 2023, Proceedings*. Ed. by Frank Drewes and Mikhail Volkov. Vol. 13911. Lecture Notes in Computer Science. Springer, 2023, pp. 128–142. DOI: 10.1007/978-3-031-33264-7_11

[315] Stefan Hoffmann. “Completely Distinguishable Automata and the Set of Synchronizing Words”. In: *Theory of Computing Systems* 70.20 (2 2026). DOI: 10.1007/s00224-024-10204-9 (extended version of [308])

[313] Stefan Hoffmann. “Subset Mapping Problems in Solvable Automata”. In: *Journal of Automata, Languages and Combinatorics* 29.2-4 (2024), pp. 199–236. DOI: 10.25596/JALC-2024-199

2.3.2 Unpublished Work

Here, we discuss results contained in the present thesis that were not published before.

In Part II, the results on one-point singulars from Sections 4.4 and 4.6 are new. We were also unable to find the statements from Section 4.5 on compositions of idempotent transformations in the literature (but most of them are rather technical or perhaps folklore). We also note that our treatment of automata theory in Chapter 6, tailored to

our needs, differs from those found in the literature of which we are aware, in particular in its usage of the reachability order and morphisms. Furthermore, many results related to morphisms and isomorphisms between automata that we give are new or scattered throughout the literature under different terminology and notation.

In Part III, the results on reachable subsets from **??**, on pairs of distinguishable subsets from **????**, the graph-theoretical results from **??**, on automata where every letter has defect at most one from **??**, and on subset mapping properties for pairs of equal-sized sets in SI-automata from **??** are new (but one preliminary result appeared in [306, Lem. 11]). Furthermore, the notion of A-primitivity is novel and all results from **??** appear in this thesis for the first time.

In Part IV, some characterizations of solid codes from **??** are (to the best of our knowledge) new. The complexity lower bounds involving concatenations from **??** have not been published previously. In **??** some published results have been corrected or extended. For example, the classification for partially ordered automata in **??** refines the results of [300]; the bound for monotonic automata claimed in **??** corrects that of [305]; and the claims on solvable automata in **??** have likewise been corrected. See **??** for details. Also, some material has been presented differently. Further details on new, extended, or revised material are given in the discussion section at the end of each chapter.

Part II
Preliminaries

"Es steht alles schon bei Dedekind."
—Emmy Noether (1882–1935)

Chapter 3
General Notions

Recall the definitions from Section 1.1 regarding set theory, functions, automata, and formal languages. Here, we introduce additional notions, facts, and results not covered previously. Section 3.1 gives more basic notions from logic, set and number theory. Further notions related to functions, in particular for functions on finite sets, are introduced in Section 3.2. Section 3.3 introduces transformations and gives some results for transformations on finite sets. In Section 3.4 we introduce relations. Semigroups, monoids, groups, semirings, matrices, and transformation semigroups are introduced in Section 3.5. Order-theoretical notions are introduced in Section 3.6. Section 3.7 gives more background on formal language theory and introduces regular expressions. Graphs and connectedness notions are introduced in Section 3.8. Section 3.9 introduces computational complexity theory and our theoretical framework to measure the resource requirements of algorithms. Sections 3.10 and 3.11 discusses conventions and notational choices. Lastly, Section 3.12 gives an overview of the literature related to the introduced notions.

The present chapter serves to fix notation. We assume some familiarity and prior exposure with general concepts from logic, set theory, number theory, basic algebra, formal language theory, and complexity theory.

3.1 More on Logic, Set Theory, and Number Theory

Some notions were already introduced in Section 1.1, for example $[n] = \{0, 1, \ldots, n-1\}$; see also the notation index (page xi) for a list of all notations. We use standard logical notation: $\wedge$ for *conjunction* (and), $\vee$ for *disjunction* (or), $\neg$ for *negation*, $\Rightarrow$ (*implication*), $\Leftrightarrow$ (*equivalence*), $\forall$ and $\exists$ for the *universal quantifier* and the *existential quantifier*, respectively, etc. We also assume familiarity with standard arguments on finite sets, like the *pigeonhole principle*, *proofs by induction*, and the *well-ordering principle*. Furthermore, we use standard (naive) set-theoretical notation: *union* ($\bigcup$), *disjoint union* ($\uplus$),[1] *intersection* ($\cap$), *set difference* ($\setminus$), *subset* ($\subseteq$), *proper subset* ($\subsetneq$) etc. For a set X, we

[1] Meant to be pairwise disjoint when more than two sets are involved.

denote by $\mathcal{P}(X) = \{Y \mid Y \subseteq X\}$ the *power set (of X)*. A finite set containing k elements is called a *k-set*. A 1-set is also called a *singleton set*. If $X_1, \ldots, X_n$ are sets, their *Cartesian product* is $X_1 \times \cdots \times X_n = \{(x_1, \ldots, x_n) \mid \forall i \in \{1, \ldots, n\} : x_i \in X_i\}$, i.e., the set of n-tuples where the i-th component is an element from X_i for $i \in \{1, \ldots, n\}$. For $X_1 = \cdots = X_n$, we set $X_1^n = X_1 \times \cdots \times X_n$ where X_1^0 equals the singleton set containing the empty tuple. Elements from X_1^n for $n \geq 0$ are also identified with sequences of length n and, for finite X_1, we also call them words over X_1 (see Section 1.1) where the empty tuple corresponds to the empty word. The *(component) projection mappings* are the mappings $\pi_i \colon X_1 \times \cdots \times X_n \to X_i$ given by $\pi_i((x_1, \ldots, x_i, \ldots, x_n)) = x_i$ for $i \in \{1, \ldots, n\}$. Two sets X, Y are called *disjoint* if $X \cap Y = \emptyset$. A *partition of X* is a family of non-empty sets $\{X_i \mid i \in I\}$ for some index set I such that $X = \biguplus_{i \in I} X_i$. We call $\{X\}$ and $\{\{x\} \mid x \in X\}$ the *trivial partitions*. A *section* (or *transversal*) of a partition $\{X_i \mid i \in I\}$ of X is a subset $S \subseteq X$ such that $S \cap X_i$ is a singleton for each $i \in I$.

Every finite set X, by ordering its elements, can be identified with $[n]$ for $n = |X|$. So, statements involving $[n]$ for some $n \geq 0$ hold for all finite sets X of size n by choosing some identification for X. Thus, in statements involving finite sets we can assume, without loss of generality, the finite sets to be of the form $[n]$ for some $n \geq 0$, which will be done in the present work.

For $a, a', b \in \mathbb{N}_0$ with $b > 0$, we denote by $a \bmod b$ the unique number $0 \leq r < b$ such that $a = bn + r$ for some $n \geq 0$. If $r = 0$, we say that b *divides* a (or a is a *multiple* of b). We write $a \equiv_b a'$ or $a \equiv a' \pmod{b}$ if $a \bmod b = a' \bmod b$ (equivalently b divides $(a - a')$). Suppose c divides b and $a \equiv a' \pmod{b}$. Then $a \equiv a' \pmod{c}$ and we get

$$a \equiv 0 \pmod{c} \Leftrightarrow a' \equiv 0 \pmod{c}. \tag{3.1}$$

For a finite set $M \subseteq \mathbb{N}_0$ we denote by $\gcd(M)$ (resp. $\operatorname{lcm}(M)$ if $0 \notin M$), also for short $\gcd M$ (resp. $\operatorname{lcm} M$), the *greatest common divisor* (resp. *least common multiple*) of the numbers in M where we set $\gcd(\emptyset) = 0$ (resp. $\operatorname{lcm}(\emptyset) = 1$). Also we set $\gcd(a_1, \ldots, a_n) = \gcd\{a_1, \ldots, a_n\}$ (resp. $\operatorname{lcm}(a_1, \ldots, a_n) = \operatorname{lcm}\{a_1, \ldots, a_n\}$) for $a_1, \ldots, a_n \geq 0$ (resp. $a_1, \ldots, a_n \geq 1$). Two number a, b are called *coprime* if $\gcd(a, b) = 1$. A number $p > 1$ is *prime* if it is only divisible by 1 or p. For a real number x we set $\lfloor x \rfloor = \max\{k \in \mathbb{Z} \mid k \leq x\}$.

Theorem 3.1 (Prime Number Theorem (see, for example, [404])) *Suppose we have a numbering $p_1 < p_2 < p_3 < \ldots$ of the prime numbers. Then for every $\varepsilon > 0$ there exists $N > 0$ such that $(1 - \varepsilon) \cdot n \log(n) \leq p_n \leq (1 + \varepsilon) \cdot n \log(n)$ for all $n \geq N$.*

Corollary 3.2 *There exists a number $N \geq 1$ such that $p_n < n^2$ for all $n \geq N$.*

3.2 More on Functions

We already introduced (total) functions in Section 1.1. Here, Section 3.2.1 introduces more basic notions and states some equations. We investigate the duplicate set of the composition of two functions in Section 3.2.2. We need these results when we later

investigate the composition of certain classes of functions. Section 3.2.3 investigates functions on finite sets.

3.2.1 Basic Notions

Given sets X, Y we set $Y^X = \{f \mid f\colon X \to Y \text{ is a function}\}$. For $n \geq 0$, $X^{\{1,\dots,n\}}$ can be identified with the Cartesian product X^n by identifying $f\colon \{1,\dots,n\} \to X$ for $n \geq 1$ with the tuple $(f(1),\dots,f(n))$ and the unique *empty function* $\emptyset \to X$ with the empty tuple over X. Occasionally, we use the notation $(f(x))_{x\in X}$ to denote or specify a function $f\colon X \to Y$.

Let $f\colon X \to Y$. The *extension of f to subsets* is the function $\hat{f}\colon \mathcal{P}(X) \to \mathcal{P}(Y)$ given by $\hat{f}(S) = \{f(x) : x \in S\}$ for $S \subseteq X$. In the following, we denote $\hat{f}$ also by f. We call $f(S)$ the *image of S under f* (for $S = X$ just the *image of f*). The function f also induces a function $f^{-1}\colon \mathcal{P}(Y) \to \mathcal{P}(X)$ given by $f^{-1}(T) = \{x \in X \mid f(x) \in T\}$ for $T \subseteq Y$. We call $f^{-1}(T)$ the *inverse image* (or *preimage*) *of $T \subseteq Y$ under f*. We have $f^{-1}(T) = f^{-1}(T \cap f(X))$ for $T \subseteq Y$. For $y \in Y$, we set $f^{-1}(y) = f^{-1}(\{y\})$. The *restriction of f* to a subset $S \subseteq X$ is $f_{|S}\colon S \to Y$ given by $f_{|S}(x) = f(x)$ for $x \in S$. We have $f_{|S}^{-1}(T) = f^{-1}(T) \cap S$ for $T \subseteq Y$.

The *excluded set of f* (the set of excluded elements) is

$$\operatorname{excl}(f) = Y \setminus f(X).$$

The *duplicate set of f* (the set of duplicate elements) is the subset of $f(X)$ given by

$$\operatorname{dupl}(f) = \{f(x) \mid x \in X \wedge f^{-1}(f(x)) \neq \{x\}\}.$$

For $S \subseteq X$ and $y \in Y$ we have

$$y \in \operatorname{dupl}(f_{|S}) \Leftrightarrow f^{-1}(y) \cap S \text{ contains at least two distinct elements,} \tag{3.2}$$

i.e., $y \in \operatorname{dupl}(f_{|S})$ if and only if $y = f(x_1) = f(x_2)$ for distinct $x_1, x_2 \in S$. Thus, for $|S| = 1$ we get $\operatorname{dupl}(f_{|S}) = \emptyset$. We also have

$$\operatorname{dupl}(f_{|S}) \subseteq f(S). \tag{3.3}$$

Next, for $S \subseteq T \subseteq X$ we get

$$\operatorname{dupl}(f_{|T}) = \operatorname{dupl}(f_{|S}) \cup \operatorname{dupl}(f_{|T\setminus S}) \cup (f(S) \cap f(T \setminus S)). \tag{3.4}$$

We call f *surjective* if $\operatorname{excl}(f) = \emptyset$, *injective* if $\operatorname{dupl}(f) = \emptyset$, i.e., if $f(x) = f(y)$ for $x, y \in X$ implies $x = y$, and *bijective* if it is surjective and injective. The function f is said to be *constant* if $f^{-1}(f(x)) = X$ for some $x \in X$.

The function f is bijective if and only if the preimage $f^{-1}(y)$ is a singleton for every $y \in Y$. For bijective f, we identify the map that sends each $y \in Y$ to the unique element in $f^{-1}(y)$ with f^{-1}, that is, we regard the set-valued map $f^{-1}\colon Y \to \mathcal{P}(X)$ as a function $f^{-1}\colon Y \to X$, called the *inverse function* of f. We note that, for every $S \subseteq Y$, there is no ambiguity in the notation $f^{-1}(S)$: whether interpreted as the preimage of S under f or as the image of S under the inverse function f^{-1}, i.e., the set $\{f^{-1}(x) \mid x \in S\}$, we obtain the same set $\{x \in X \mid f(x) \in S\}$.

The *identity transformation (on X)* is $\mathrm{id}_X \colon X \to X$ (or just $\mathrm{id} \colon X \to X$) given by $\mathrm{id}_X(x) = x$ for all $x \in X$.

Given $f \colon X \to Y$ and $g \colon Y \to Z$, the *(function) composition* of g and f is $g \circ f \colon X \to Z$ (or $gf \colon X \to Z$ for short) given by $(gf)(x) = g(f(x))$ for each $x \in X$. Obviously, we have $gf = g_{|f(X)}f$. For $T \subseteq Z$ we have $(gf)^{-1}(T) = f^{-1}(g^{-1}(T))$. This gives

$$(gf)^{-1} = f^{-1}g^{-1}. \tag{3.5}$$

Let I and J be index sets. For sets $T_i \subseteq Y$ and $S_j \subseteq X$ where $i \in I$, $j \in J$ we have

$$f\left(\bigcup_{j\in J} S_j\right) = \bigcup_{j\in J} f(S_j), \qquad f^{-1}\left(\bigcup_{i\in I} T_i\right) = \bigcup_{i\in I} f^{-1}(T_i),$$
$$f^{-1}\left(\bigcap_{i\in I} T_i\right) = \bigcap_{i\in I} f^{-1}(T_i), \qquad f\left(\bigcap_{j\in J} S_j\right) \subseteq \bigcap_{j\in J} f(S_j).$$

This also yields that for $S_1, S_2 \subseteq X$ with $S_1 \subseteq S_2$ we have $f(S_1) \subseteq f(S_2)$ and $f^{-1}(S_1) \subseteq f^{-1}(S_2)$.

Furthermore, for $T \subseteq Y$ and $S \subseteq X$ we have

$$f(f^{-1}(T)) = T \cap f(X) \tag{3.6}$$
$$T \setminus f(f^{-1}(T)) = T \cap \mathrm{excl}(f) \tag{3.7}$$
$$f^{-1}(f(S)) = S \cup f^{-1}(f(S) \cap \mathrm{dupl}(f)) \tag{3.8}$$
$$f^{-1}(f(S)) = (S \setminus f^{-1}(\mathrm{dupl}(f))) \uplus f^{-1}(f(S) \cap \mathrm{dupl}(f)) \tag{3.9}$$

Equations (3.6) to (3.8) are easy to verify. For Equation (3.9), split $f(S)$ into $f(S) \cap \mathrm{dupl}(f)$ and $f(S) \cap (Y \setminus \mathrm{dupl}(f))$ whose inverse images are disjoint. So it remains to note that $f^{-1}(f(S) \cap (Y \setminus \mathrm{dupl}(f))) = S \setminus f^{-1}(\mathrm{dupl}(f))$, where $\supseteq$ is clear and, conversely, $f(x) \in f(S)$ gives $f(x) = f(x')$ for some $x' \in S$, while $f(x) \notin \mathrm{dupl}(f)$ gives $f^{-1}(f(x)) = \{x\}$, so that $x = x' \in S \setminus f^{-1}(\mathrm{dupl}(f))$.

By Equation (3.6), we have $f(f^{-1}(T)) = T$ if and only if $T \subseteq f(X)$. The function f is surjective (resp. injective) if and only if $f(f^{-1}(T)) = T$ for each $T \subseteq Y$ (resp. $f^{-1}(f(S)) = S$ for each $S \subseteq X$) by Equation (3.6) (resp. Equation (3.9)). Note that, by removing S on both sides in Equation (3.9), we find $f^{-1}(f(S)) \setminus S = f^{-1}(f(S) \cap \mathrm{dupl}(f)) \setminus S$.

In the following, we will often use these relations and the above equations and inclusions without special mention.

Proposition 3.3 *A function $f \colon X \to Y$ is surjective (resp. injective for $X \neq \emptyset$) if and only if $fg = \mathrm{id}_Y$ (resp. $gf = \mathrm{id}_X$) for some $g \colon Y \to X$.*

Proof. Suppose f is surjective. Since $f^{-1}(y)$ is non-empty for $y \in Y$, we can find a function $g \colon Y \to X$ such that $fg = \mathrm{id}_Y$ by choosing $g(y) \in f^{-1}(y)$ for each $y \in Y$.[2] If

[2] Here, we need the axiom of choice.

f is not surjective, equivalently $f(X)$ is a proper subset of Y, then $f(g(Y))$ is a proper subset of Y for each $g\colon Y \to X$. This yields that $fg \neq \mathrm{id}_Y$ since $\mathrm{id}_Y(Y) = Y$.

Now, suppose f is injective. Let $g\colon Y \to X$ where $g(y)$ for $y \in Y$ equals the unique element in $f^{-1}(y)$ if $f^{-1}(y) \neq \emptyset$, and $g(y) \in X$ is arbitrary otherwise, which is possible as $X \neq \emptyset$. Then $gf = \mathrm{id}_X$. Next, suppose f is not injective. Then we have $f(x) = f(y)$ for distinct $x, y \in X$. Let $g\colon Y \to X$. We have $g(f(x)) = g(f(y))$. Since $\mathrm{id}_X(x) \neq \mathrm{id}_X(y)$ we find $gf \neq \mathrm{id}_X$. □

For bijective f, the functions from Theorem 3.3 are uniquely determined and equal to f^{-1}. In fact, the inverse function for bijective f is characterized as the unique function f^{-1} such that $f^{-1}f = \mathrm{id}_X$ and $ff^{-1} = \mathrm{id}_Y$.

3.2.2 The Duplicate Set and Function Composition

In Theorem 3.4 we present identities for the duplicate set of a composition of two functions. Theorem 3.6 gives an identity for the inverse image of the duplicate set of a composition. Figure 3.1 given an illustration for these identities. We also note some corollaries and give examples.

Proposition 3.4 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then the following equations hold:*

$$\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f)) \tag{3.10}$$

$$\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)\setminus \mathrm{dupl}(f)}) \cup g(\mathrm{dupl}(f)) \tag{3.11}$$

$$\mathrm{dupl}(gf) = \{z \in Z \mid g^{-1}(z) \cap \mathrm{dupl}(f) \neq \emptyset \text{ or } |g^{-1}(z) \setminus \mathrm{excl}(f)| \geq 2\} \tag{3.12}$$

Proof. 3.10: Clearly, we have $\mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(gf)$. Next, let $y \in \mathrm{dupl}(gf)$. Then $y = g(f(x_1)) = g(f(x_2))$ for distinct $x_1, x_2 \in X$. If $f(x_1) \neq f(x_2)$ then $y \in \mathrm{dupl}(g_{|f(X)})$, and $f(x_1) = f(x_2)$ implies $y \in g(\mathrm{dupl}(f))$. So, $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|f(X)}) \cup g(\mathrm{dupl}(f))$.

3.11: Set $S = f(X) \setminus \mathrm{dupl}(f)$. By Equation (3.4) we have

$$\mathrm{dupl}(g_{|f(X)}) = \mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \cup \mathrm{dupl}(g_{|S}) \cup (g(\mathrm{dupl}(f)) \cap g(S)).$$

Since $g(\mathrm{dupl}(f)) \cap g(S) \subseteq g(\mathrm{dupl}(f))$ and, by Equation (3.3), $\mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \subseteq g(\mathrm{dupl}(f))$, we obtain the claim from Equation (3.10).

3.12: We have $g^{-1}(z) \cap \mathrm{dupl}(f) \neq \emptyset$ if and only if $z \in g(\mathrm{dupl}(f))$, and $|g^{-1}(z) \setminus \mathrm{excl}(f)| \geq 2$ if and only if $z \in \mathrm{dupl}(g_{|f(X)})$. Hence, the formula follows from Equation (3.10). □

Volkov and Casas [134] gave Equation (3.12) formulated for words and automata.

For finite sets, Equation (3.11) gives

$$\max\{|\,\mathrm{dupl}(g_{|f(X)})|, |g(\mathrm{dupl}(f))|\} \leq |\,\mathrm{dupl}(gf)|. \tag{3.13}$$

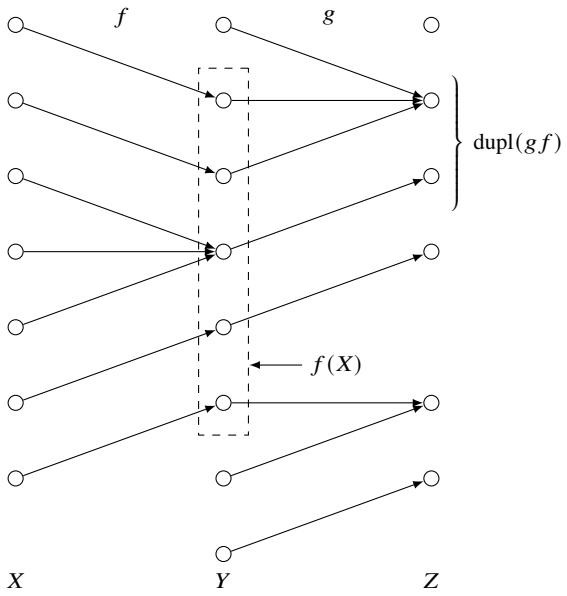


Fig. 3.1: An illustration for the identities from Theorems 3.4 and 3.6.

Corollary 3.5 *Let $f\colon X \to Y$, $g\colon Y \to Z$, and $S \subseteq X$ with $f(X)\backslash \mathrm{dupl}(f) \subseteq S \subseteq f(X)$. Then the following are equivalent.*

1. $\mathrm{dupl}(gf) \subseteq g(\mathrm{dupl}(f))$
2. $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$
3. $\mathrm{dupl}(g_{|S}) \subseteq g(\mathrm{dupl}(f))$
4. $g^{-1}(\mathrm{dupl}(g_{|S})) \subseteq g^{-1}(g(\mathrm{dupl}(f)))$
5. $S \cap g^{-1}(\mathrm{dupl}(g_{|S})) \subseteq g^{-1}(g(\mathrm{dupl}(f)))$
6. *For all $z \in \mathrm{dupl}(g)$, if $S \cap g^{-1}(z)$ contains two distinct elements, then $z \in g(\mathrm{dupl}(f))$.*
7. *For all $z \in \mathrm{dupl}(g)$, if $f(X) \cap g^{-1}(z)$ contains two distinct elements, then it contains an element from $\mathrm{dupl}(f)$, i.e., $y, y' \in f(X) \cap g^{-1}(z)$ for $y \neq y'$ implies $z \in g(\mathrm{dupl}(f))$.*

Proof. $1 \Rightarrow 2$: We have $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(gf)$ by Equation (3.11), giving the claim.

$2 \Rightarrow 3$: With Equation (3.10) we have $\mathrm{dupl}(g_{|S}) \subseteq \mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$.

$3 \Rightarrow 4$: This is clear.

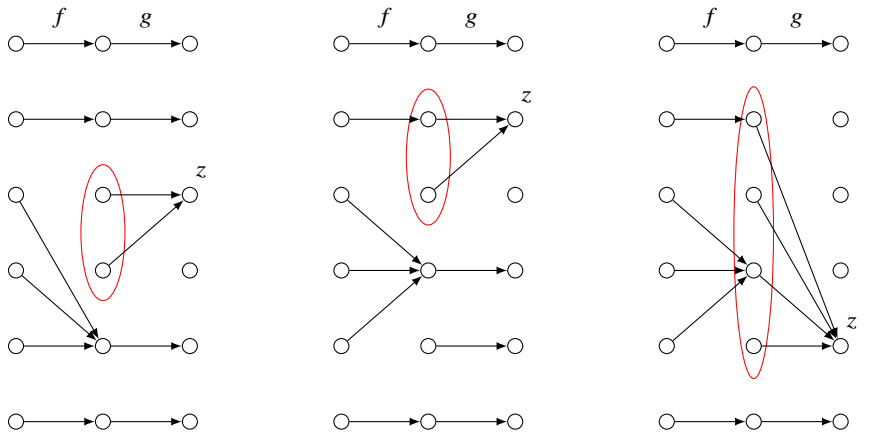
$4 \Rightarrow 5$: This is clear.

$5 \Rightarrow 6$: Let $z \in \mathrm{dupl}(g)$ such that $x, y \in S \cap g^{-1}(z)$ for distinct $x, y \in Y$. Then we have $z \in \mathrm{dupl}(g_{|S})$ and so $x, y \in S \cap g^{-1}(\mathrm{dupl}(g_{|S}))$. By assumption, we get $x, y \in g^{-1}(g(\mathrm{dupl}(f)))$. Thus we find $z = g(x) \in g(\mathrm{dupl}(f))$, which gives the claim.

$6 \Rightarrow 7$: Let $f(x), f(x') \in g^{-1}(z)$ with $f(x) \neq f(x')$ for $x, x' \in X$. If $f(x), f(x') \in S$, then, by assumption, $z = g(y)$ for some $y \in \mathrm{dupl}(f)$. We have $y \in f(X) \cap g^{-1}(z)$.

Otherwise, without loss of generality, let $f(x) \notin S$. Then, since $f(X) \setminus \mathrm{dupl}(f) \subseteq S$ by choice of S, we have $f(x) \in \mathrm{dupl}(f)$.

$7 \Rightarrow 1$: Let $z \in \mathrm{dupl}(g_{|f(X)})$. Then $f(X) \cap g^{-1}(z)$ contains two elements. Thus, by assumption, we have $z \in g(\mathrm{dupl}(f))$. This shows $\mathrm{dupl}(g_{|f(X)}) \subseteq g(\mathrm{dupl}(f))$. Then, with Equation (3.10), we find $\mathrm{dupl}(gf) \subseteq g(\mathrm{dupl}(f))$. □



(a) $f(X) \cap g^{-1}(z) = \emptyset$ (b) $f(X) \cap g^{-1}(z)$ singleton (c) $\mathrm{dupl}(f) \cap g^{-1}(z) \neq \emptyset$

Fig. 3.2: Cases that, when fulfilled for all $z \in \mathrm{dupl}(g)$, yield $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ by Item 7 of Theorem 3.5. The set $g^{-1}(z)$ is indicated by a red-colored elliptical shape.

Figure 3.2 depicts the following three cases: $f(X) \cap g^{-1}(z)$ is empty, $f(X) \cap g^{-1}(z)$ is a singleton, or $f(X) \cap g^{-1}(z)$ contains two distinct elements and an element from $\mathrm{dupl}(f)$. If for all $z \in \mathrm{dupl}(g)$ one of these cases holds, then $\mathrm{dupl}(gf) = g(\mathrm{dupl}(f))$ by Item 7 from Theorem 3.5.

Proposition 3.6 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then*

$$(gf)^{-1}(\mathrm{dupl}(gf)) = f^{-1}(\mathrm{dupl}(f)) \cup (gf)^{-1}(\mathrm{dupl}(g_{|f(X)})).$$

Proof. By Equation (3.8) and as $g(\mathrm{dupl}(f)) = g_{|f(X)}(\mathrm{dupl}(f))$ we have

$$g^{-1}_{|f(X)}(g(\mathrm{dupl}(f))) = \mathrm{dupl}(f) \cup g^{-1}_{|f(X)}(g(\mathrm{dupl}(f)) \cap \mathrm{dupl}(g_{|f(X)})). \tag{3.14}$$

With $g^{-1}_{|f(X)}(g(\mathrm{dupl}(f))) = g^{-1}(g(\mathrm{dupl}(f))) \cap f(X)$ this then gives

$$\begin{aligned}
&f^{-1}(g^{-1}(\mathrm{dupl}(gf))) \\
&= f^{-1}(g^{-1}(g(\mathrm{dupl}(f))) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)}))) && \text{(Eqn. (3.10))} \\
&= f^{-1}(g^{-1}(g(\mathrm{dupl}(f)) \cap f(X)) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)}))) \\
&= f^{-1}(\mathrm{dupl}(f)) \cup f^{-1}(g^{-1}(\mathrm{dupl}(g_{|f(X)}))) && \text{(Eqn. (3.14))}
\end{aligned}$$

This shows the claim. □

The next example shows that we cannot replace $g_{|f(X)}$ by g in Theorem 3.6.

Example 3.7 Let $X = \{1,2,3,4\}$ and $f, g\colon X \to X$ with $f(1) = 1, f(2) = f(3) = 2, f(4) = 4$ and $g(1) = g(2) = 1$, $g(3) = g(4) = 3$. Then

1. $\mathrm{dupl}(gf) = \{1\}$ and $\mathrm{dupl}(g) = \{1,3\}$,
2. $f^{-1}(\mathrm{dupl}(f)) = \{2,3\}$ and $g^{-1}(\mathrm{dupl}(g)) = X$,
3. $\mathrm{dupl}(g_{|\{1,2,4\}}) = \{1\}$,
4. $(gf)^{-1}(\mathrm{dupl}(gf)) = \{1,2,3\}$.

Here $f^{-1}(g^{-1}(\mathrm{dupl}(g))) = X$ properly contains $(gf)^{-1}(\mathrm{dupl}(gf))$.

Proposition 3.8 *Let $f\colon X \to Y$, $g\colon Y \to Z$, and $S, T, R \subseteq Y$ with $f(X) \setminus \mathrm{dupl}(f) \subseteq S$, $\mathrm{dupl}(f) \subseteq T$, and $R \subseteq f(X)$. Then we have the following:*

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|S})$ *if and only if* $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|S})$.
2. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|R})$ *implies* $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|f(X)})$.
3. $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$ *implies* $\mathrm{dupl}(g_{|R}) = \mathrm{dupl}(g_{|f(X)})$.
4. $\mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$ *if and only if* $\mathrm{dupl}(gf) \subseteq \mathrm{dupl}(g_{|R})$.
5. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|T})$ *if and only if for all $y \in \mathrm{dupl}(f)$ there exists $y' \in T$ with $y \neq y'$ and $g(y) = g(y')$.*

Proof. 1: Since $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)}) \subseteq \mathrm{dupl}(g_{|S})$, this follows from Equation (3.11).

2: This follows from Equation (3.10) with $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)})$.

3: Since $R \subseteq f(X)$ we have $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)})$. By assumption and with Equation (3.10), we find $\mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf) = \mathrm{dupl}(g_{|R})$.

4: This follows since $\mathrm{dupl}(g_{|R}) \subseteq \mathrm{dupl}(g_{|f(X)}) \subseteq \mathrm{dupl}(gf)$ with Equation (3.10).

5: First, suppose $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|T})$. Consider $y \in \mathrm{dupl}(f)$. Then the assumption gives $g(y) \in \mathrm{dupl}(g_{|T})$. Thus, there exist distinct $y', y'' \in T$ with $g(y) = g(y') = g(y'')$ and we have $y \neq y'$ or $y \neq y''$, which gives the claim.

Conversely, suppose the other condition holds. Let $y \in \mathrm{dupl}(f)$. Then $g(y) = g(y')$ for some $y' \in T \setminus \{y\}$. Since $\mathrm{dupl}(t) \subseteq T$ we have $y, y' \in T$. So $g(y) \in \mathrm{dupl}(g_{|T})$. □

The condition $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$ implies $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$ by Equation (3.4), but the converse does not hold, as shown by the following example. By Equation (3.4) the difference between $\mathrm{dupl}(g_{|f(X)})$ and $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$ is the set $\mathrm{dupl}(g_{|\,\mathrm{dupl}(f)}) \cup S$ with $S = g(\mathrm{dupl}(f)) \cap g(f(X) \setminus \mathrm{dupl}(f))$.

Example 3.9 Let $X = \{1,2,3\}$ and $Y = \{2,3\}$. Consider $f\colon X \to Y$ with $f(1) = f(2) = 2$ and $f(3) = 3$, and $g\colon Y \to \{3\}$ with $g(2) = g(3) = 3$. Then $g(\mathrm{dupl}(f)) = \{3\}$, $\mathrm{dupl}(g_{|f(X)}) = \{3\}$, and $\mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)}) = \emptyset$. So

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$, but
2. $g(\mathrm{dupl}(f)) \not\subseteq \mathrm{dupl}(g_{|f(X)\setminus\mathrm{dupl}(f)})$.

Corollary 3.10 *Let $f\colon X \to Y$ and $g\colon Y \to Z$. Then the following are equivalent:*

1. $g(\mathrm{dupl}(f)) \subseteq \mathrm{dupl}(g_{|f(X)})$.

2. $\operatorname{dupl}(gf) \subseteq \operatorname{dupl}(g_{|f(X)})$.
3. $\operatorname{dupl}(gf) = \operatorname{dupl}(g_{|f(X)})$.
4. *For all* $y \in \operatorname{dupl}(f)$ *exists* $y' \in f(X)$ *with* $y \neq y'$ *such that* $g(y) = g(y')$.
5. $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g_{|\operatorname{dupl}(f)}) \cup (g(\operatorname{dupl}(f)) \cap g(f(X) \setminus \operatorname{dupl}(f)))$.

Proof. The first four equivalences follow from Theorem 3.8 by considering $S = T = R = f(X)$.

$1 \Leftrightarrow 5$: First, let $g(f(x))$ for some $x \in X$ such that $f(x) \in \operatorname{dupl}(f)$. By Item 1 there exists $x' \in X$ such that $f(x) \neq f(x')$ and $g(f(x)) = g(f(x'))$. If $f(x') \in \operatorname{dupl}(f)$, then $g(f(x)) \in \operatorname{dupl}(g_{|\operatorname{dupl}(f)})$. Otherwise, we have $g(f(x)) \in g(f(X) \setminus \operatorname{dupl}(f))$. This gives Item 5.

Conversely, first not that by Equation (3.4) we have

$$\operatorname{dupl}(g_{|\operatorname{dupl}(f)}) \cup (g(\operatorname{dupl}(f)) \cap g(f(X) \setminus \operatorname{dupl}(f))) \subseteq \operatorname{dupl}(g_{|f(X)}).$$

Thus Item 5 implies Item 1. □

Note that $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g)$ is not equivalent to $g(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(g_{|f(X)})$ as demonstrated by the next example.

Example 3.11 Let $X = \{1, 2\}$ with $f\colon X \to X$ with $f(1) = f(2)$. Then $\operatorname{dupl}(f) = \{f(1)\}$, $f(\operatorname{dupl}(f)) = \{f(1)\}$, and $\operatorname{dupl}(f_{|f(X)}) = \emptyset$. So

1. $f(\operatorname{dupl}(f)) \subseteq \operatorname{dupl}(f)$, but
2. $f(\operatorname{dupl}(f)) \not\subseteq \operatorname{dupl}(f_{|f(X)})$.

3.2.3 Functions on a Finite Set

Let X, Y be finite sets and $f\colon X \to Y$ be a function. Then the *defect* (resp. *rank*) of f is $\operatorname{def}(f) = |\operatorname{excl}(f)|$ (resp. $\operatorname{rank}(f) = |f(X)|$). As $Y = f(X) \uplus \operatorname{excl}(f)$, we have

$$|Y| = \operatorname{rank}(f) + \operatorname{def}(f). \tag{3.15}$$

For $A \subseteq Y$ we call $\operatorname{exc}_f(A) = |f^{-1}(A)| - |A|$ the *excess of* A *under* f. As $|f^{-1}(y)| = 0$ if and only if $y \in \operatorname{excl}(f)$, and $|f^{-1}(y)| \geq 2$ if and only if $y \in \operatorname{dupl}(f)$, we have, for $y \in Y$,

$$\operatorname{exc}_f(\{y\}) \begin{cases} = -1 & \text{if } y \in \operatorname{excl}(f), \\ = 0 & \text{if } y \in f(X) \setminus \operatorname{dupl}(f), \\ \geq 1 & \text{if } y \in \operatorname{dupl}(f). \end{cases} \tag{3.16}$$

The *kernel type* [34] of f is the ordered r-tuple with $r = \operatorname{rank}(f)$ given by the sizes of the non-empty inverse images $f^{-1}(y)$ for $y \in Y$, i.e., the partition of $|X|$ into these sizes. For example, f has kernel type $(k, 1, \ldots, 1)$ for $k \geq 1$ if and only if

1. $|f^{-1}(y)| = k$ for some $y \in Y$ and
2. $|f^{-1}(z)| \leq 1$ for all $z \in Y \setminus \{y\}$.

Clearly, for $S \subseteq X$ we have

$$|f(S)| \leq |S|, \tag{3.17}$$

and $|f(S)| = |S|$ if and only if $f_{|S}$ is injective.

Proposition 3.12 *Let $f\colon X \to Y$ and $g\colon Y \to Z$ for finite sets X, Y, and Z. Then*

$$\operatorname{rank}(gf) \leq \min\{\operatorname{rank}(f), \operatorname{rank}(g)\}, \tag{3.18}$$

where

1. $\operatorname{rank}(gf) = \operatorname{rank}(f)$ *if and only if $g_{|f(X)}$ is injective, and*
2. $\operatorname{rank}(gf) = \operatorname{rank}(g)$ *if and only if $g(Y) = g(f(X))$.*

Proof. We have $\operatorname{rank}(gf) = |g(f(X))|$. By Equation (3.17), $|g(f(X))| \leq |f(X)| = \operatorname{rank}(f)$ with equality if and only if $g_{|f(X)}$ is injective. Furthermore, $g(f(X)) \subseteq g(Y)$ gives $|g(f(X))| \leq \operatorname{rank}(g)$, and, as these sets are finite, equality of the cardinalities holds if and only if $g(f(X)) = g(Y)$. □

Proposition 3.13 *Let $f\colon X \to Y$ with X, Y finite. Then the following holds:*[3]

1. *For $A \subseteq Y$ we have*

$$\operatorname{exc}_f(A) = \sum_{y \in A} (|f^{-1}(y)| - 1), \tag{3.19}$$

$$\operatorname{exc}_f(A) = \operatorname{exc}_f(A \cap \operatorname{dupl}(f)) - |A \cap \operatorname{excl}(f)|. \tag{3.20}$$

2. *For disjoint $A, B \subseteq Y$ we have* $\operatorname{exc}_f(A \uplus B) = \operatorname{exc}_f(A) + \operatorname{exc}_f(B)$.
3. $\operatorname{exc}_f(A) \geq |A|$ *for $A \subseteq \operatorname{dupl}(f)$.*
4. $\operatorname{exc}_f(A) \leq \operatorname{exc}_f(B)$ *for $A \subseteq B \subseteq f(X)$. Moreover, we have*

$$\operatorname{exc}_f(A) < \operatorname{exc}_f(B) \Leftrightarrow (B \setminus A) \cap \operatorname{dupl}(f) \neq \emptyset.$$

5. $\operatorname{exc}_f(f(X)) = \operatorname{exc}_f(\operatorname{dupl}(f)) = |X| - \operatorname{rank}(f) = \sum_{i=1}^{r}(k_i - 1)$ *where f has kernel type $(k_1, \ldots, k_r)$.*
6. *For $A \subseteq Y$ we have*

$$|A \cap \operatorname{excl}(f)| < |A \cap \operatorname{dupl}(f)| \Rightarrow \operatorname{exc}_f(A) > 0. \tag{3.21}$$

Proof. 1: As $f^{-1}(A) = \biguplus_{y \in A} f^{-1}(y)$, we have $|f^{-1}(A)| = \sum_{y \in A} |f^{-1}(y)|$, which gives Equation (3.19) with $|A| = \sum_{y \in A} 1$. Splitting the index set A in the sum from Equation (3.19) into $A \cap \operatorname{excl}(f)$, $A \cap (f(X) \setminus \operatorname{dupl}(f))$, and $A \cap \operatorname{dupl}(f)$, whose summands are -1, 0, and those of $\operatorname{exc}_f(A \cap \operatorname{dupl}(f))$ by Equation (3.16), gives Equation (3.20).

2: This holds as the index sets A and B in Equation (3.19) are disjoint.

3: This follows since for $A \subseteq \operatorname{dupl}(f)$ every summand of Equation (3.19) is at least one by Equation (3.16).

[3] By convention, the summation over an empty index set equals zero.

4: By Item 2 we have $\operatorname{exc}_f(B) = \operatorname{exc}_f(A) + \operatorname{exc}_f(B \setminus A)$. Since $B \setminus A \subseteq f(X)$, Equation (3.20) gives $\operatorname{exc}_f(B \setminus A) = \operatorname{exc}_f((B \setminus A) \cap \operatorname{dupl}(f))$, which is at least $|(B \setminus A) \cap \operatorname{dupl}(f)|$ by Item 3 and equals zero if $(B \setminus A) \cap \operatorname{dupl}(f) = \emptyset$.

5: Equation (3.20) for $A = f(X)$ gives $\operatorname{exc}_f(f(X)) = \operatorname{exc}_f(\operatorname{dupl}(f))$, as $\operatorname{dupl}(f) \subseteq f(X)$. As $f^{-1}(f(X)) = X$, we have $\operatorname{exc}_f(f(X)) = |X| - \operatorname{rank}(f)$. Since the non-empty inverse images are the sets $f^{-1}(y)$ for $y \in f(X)$, the last equality is Equation (3.19) for $A = f(X)$.

6: By Item 3 the assumption gives $|A \cap \operatorname{excl}(f)| < |A \cap \operatorname{dupl}(f)| \leq \operatorname{exc}_f(A \cap \operatorname{dupl}(f))$, so Equation (3.20) yields $\operatorname{exc}_f(A) > 0$. □

For functions between sets of equal size, the excess of the duplicate set is precisely the defect.

Proposition 3.14 *Let $f\colon X \to Y$ with X, Y finite and $|X| = |Y|$. Then the following holds:*

1. $\operatorname{def}(f) = \operatorname{exc}_f(\operatorname{dupl}(f)) = |f^{-1}(\operatorname{dupl}(f))| - |\operatorname{dupl}(f)|$,
2. $\operatorname{def}(f) \geq \operatorname{exc}_f(A)$ *for every* $A \subseteq Y$,
3. $\operatorname{def}(f) \geq |\operatorname{dupl}(f)|$.

Proof. 1: By Item 5 of Theorem 3.13, Equation (3.15), and $|X| = |Y|$, we have $\operatorname{exc}_f(\operatorname{dupl}(f)) = |X| - \operatorname{rank}(f) = |Y| - \operatorname{rank}(f) = \operatorname{def}(f)$.

2: For $A \subseteq Y$, Equation (3.20) gives $\operatorname{exc}_f(A) \leq \operatorname{exc}_f(A \cap \operatorname{dupl}(f))$, which is at most $\operatorname{exc}_f(\operatorname{dupl}(f)) = \operatorname{def}(f)$ by Item 4 of Theorem 3.13 and Item 1.

3: Combine Item 1 with Item 3 of Theorem 3.13 for $A = \operatorname{dupl}(f)$. □

Corollary 3.15 *Let $f\colon X \to Y$ with $|X| = |Y|$ and $\operatorname{def}(f) = 1$. Then $|\operatorname{dupl}(f)| = 1$ and $|f^{-1}(\operatorname{dupl}(f))| = 2$.*

Proof. By Item 3 of Theorem 3.14 we have $|\operatorname{dupl}(f)| \leq 1$, and $\operatorname{dupl}(f) \neq \emptyset$, as otherwise Item 1 of Theorem 3.14 would give $\operatorname{def}(f) = \operatorname{exc}_f(\emptyset) = 0$. So $|\operatorname{dupl}(f)| = 1$, and then the same item yields $|f^{-1}(\operatorname{dupl}(f))| = \operatorname{def}(f) + |\operatorname{dupl}(f)| = 2$. □

3.3 Transformations

A *transformation (on X)*, or *endofunction*, is a function $t\colon X \to X$ of a set X into itself, i.e., a function whose domain and codomain are the same. Let $i \geq 0$. The *i-fold composition*, or *i-th power*, is defined by $t^i = \mathrm{id}_X$ for $i = 0$ and $t^i = t^{i-1}t$ for $i > 0$.

In the context of transformations, the elements of X are also called *points*. We call $x \in X$ a *cycle point* if $t^p(x) = x$ for some $p > 0$. Clearly, cycle points are contained in the image $t(X)$. A point that is not a cycle point is called a *tail point*. A *root point* is a cycle point contained in $\operatorname{dupl}(t)$. We set $\operatorname{fix}(t) = \{x \in X \mid t(x) = x\}$ and $\operatorname{move}(t) = X \setminus \operatorname{fix}(t) = \{x \in X \mid t(x) \neq x\}$. A point $x \in X$ is said to be *fixed* (resp. *moved*) if $x \in \operatorname{fix}(t)$ (resp. $x \in \operatorname{move}(t)$). A fixed point is also called a *fixpoint*.

Let $t, h\colon X \to X$ and $A \subseteq X$. The following formulas are easy to verify.

$$\operatorname{fix}(t) \cap t(\operatorname{move}(t)) \subseteq \operatorname{dupl}(t) \subseteq t(\operatorname{move}(t)) \tag{3.22}$$

$$\operatorname{move}(t) \cap t^{-1}(\operatorname{fix}(t)) \subseteq \operatorname{dupl}(t) \tag{3.23}$$

$$\operatorname{fix}(t) \subseteq t(X) \tag{3.24}$$

$$\operatorname{excl}(t) \subseteq \operatorname{move}(t) \tag{3.25}$$

$$\operatorname{fix}(t) \cap \operatorname{fix}(h) = \operatorname{fix}(t) \cap \operatorname{fix}(ht) \tag{3.26}$$

$$t^{-1}(A) = (\operatorname{move}(t) \cap t^{-1}(A)) \cup (\operatorname{fix}(t) \cap A) \tag{3.27}$$

3.4 Relations

A *relation* R is a tuple $(R_G, X_1, \ldots, X_n)$ such that $R_G \subseteq X_1 \times \cdots \times X_n$. For $n = 2$, we call R a *binary relation* and in this case, for $(x_1, x_2) \in X_1 \times X_2$ we write $x_1 R x_2$ if and only if $(x_1, x_2) \in R_G$.

For $X = X_1 = X_2$ we say that R is a relation over X (such relations are also called *homogeneous*). If R is a relation over X, then it is called

reflexive if for all $x \in X$ we have xRx,
symmetric if for all $x, y \in X$ we have that xRy implies yRx,
antisymmetric if for all $x, y \in X$ we have that xRy and yRx imply $x = y$,
transitive if for all $x, y, z \in X$ we have that xRy and yRz imply xRz,
equivalence relation if it is reflexive, symmetric and transitive.

If $\sim$ denotes an equivalence relation[4] on X and $x \in X$, the *equivalence class of* x is

$$[x]_\sim = \{y \in X \mid y \sim x\}.$$

It is $[x]_\sim = [y]_\sim$ if and only if $x \sim y$ for $x, y \in X$. We write $X/\sim = \{[x]_\sim \mid x \in X\}$ for the set of equivalence classes. The surjective map $\pi_\sim \colon X \to X/\sim$ given by

$$\pi_\sim(x) = [x]_\sim \tag{3.28}$$

for $x \in X$ is called the *canonical projection map (for* $\sim$*)*.

The equivalence classes partition the set X. Conversely, given a partition $\{X_i \mid i \in I\}$ of X for some index set I, the *equivalence relation induced by it* is given by setting $x, y \in X$ to be equivalent if $\{x, y\} \subseteq X_i$ for some $i \in I$.

For a function $f \colon X \to Y$ the non-empty preimages $f^{-1}(y)$, $y \in Y$, partition X. The equivalence relation $\sim_f$ induced by this partition is called the *kernel relation (of* f*)*. We have $x \sim_f y$ if and only if $f(x) = f(y)$. The kernel relation of the canonical projection $\pi_\sim$ from above is $\sim$.

We say that an equivalence relation $\sim$ on X *saturates* a subset $A \subseteq X$ if and only if $x \sim y$ and $x \in A$ imply $y \in A$, or equivalently A is union of equivalence classes.

Proposition 3.16 *Let* $f \colon X \to Y$ *and* $A \subseteq X$*. Then the following are equivalent:*

[4] Using symbols like $\sim$ for equivalence relations is more common than denoting them by R.

1. *The kernel relation $\sim_f$ saturates A,*
2. $f^{-1}(f(A)) \subseteq A$,
3. $f^{-1}(f(A)) = A$.

Proof. 1 $\Rightarrow$ 2: That $\sim_f$ saturates A is equivalent to $f^{-1}(f(x)) \subseteq A$ for each $x \in A$. So $f^{-1}(f(A)) = f^{-1}(f(\bigcup_{x\in A}\{x\})) \subseteq A$.

2 $\Rightarrow$ 3: This follows from Equation (3.8).

3 $\Rightarrow$ 1: If $x \sim_f y$ for some $x \in A$ and $y \in X$, then $y \in f^{-1}(f(x)) \subseteq f^{-1}(f(A)) = A$. □

The *trivial equivalence relations* are $X \times X$ (the *universal relation*) and $\{(x, x) \mid x \in X\}$ (the *equality relation*).

For two equivalence relations $\sim$ and $\approx$ on X such that $x \sim y$ implies $x \approx y$ for all x, y, the relation $\sim$ (resp. $\approx$) is said to be *finer* (resp. *coarser*) than $\approx$ (resp. $\sim$). The *equivalence relation generated* by a given relation R over X is the finest equivalence relation $\sim$ such that xRy implies $x \sim y$ for all $x, y \in X$, i.e., the unique relation such that

1. xRy implies $x \sim y$ for all $x, y \in X$ and
2. for each equivalence relation $\equiv$ over X such that xRy implies $x \equiv y$ for all $x, y \in X$ we have that $x \sim y$ implies $x \equiv y$ for all $x, y \in X$.

Remark 3.17 In the literature, a relation R from X to Y is often just defined as a subset of $X \times Y$. However, as pointed out in [194, p. 2], under the usual identification of $(X \times Y) \times Z$ and $X \times (Y \times Z)$, a subset of $X \times Y \times Z$ can be interpreted as a relation from $X \times Y$ to Z or from X to $Y \times Z$, and both interpretations may yield relations with quite different properties. Another reason is that there might be (type- or category-theoretical) contexts when relations $(R, \emptyset, X)$ and $(R', \emptyset, Y)$ for $X \neq Y$ should be treated differently. Note that for homogeneous relations, these issues do not arise and the relation can indeed be identified with its set of tuples.

Remark 3.18 In [194] a relation from X to Y is modeled as a function $f\colon X \to \mathcal{P}(Y)$, which can be identified with functions $f\colon \mathcal{P}(X) \to \mathcal{P}(Y)$ with the property $f(\bigcup_{i\in I} S_i) = \bigcup_{i\in I} f(S_i)$ for $S_i \in \mathcal{P}(X)$ with $i \in I$. Note that, by the identities from Section 3.2.1, the extension of a function $X \to Y$ to subsets fulfills this equation.

3.5 Algebraic Notions

Here, in Section 3.5.1, we first introduce semigroups, monoids, groups, semirings and related notions. Section 3.5.2 introduced matrices.

3.5.1 Semigroups, Monoids, Groups, and Semirings

Here, we introduce the algebraic structures of relevance to us.

1. A *semigroup* is a tuple $(S, \cdot)$ where S is a set and $\cdot\colon S \times S \to S$ a binary operation, called a *product* and written as $x \cdot y$ (infix notation) for $x, y \in S$, that is *associative*, i.e., $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ for $x, y, z \in S$. Often, when it is clear from context what operation $S \times S \to S$ is meant, we simply call the set S a semigroup, i.e., omit the tuple notation, and we write xy instead of $x \cdot y$, i.e., we omit the operation symbol.
2. A *monoid* is a tuple $(M, \cdot, 1)$ where $(M, \cdot)$ is a semigroup and $1 \in M$ is an *identity element*, i.e., $1x = x1 = x$ for all $x \in M$. The identity element is unique, for if $1, 1' \in M$ are identity elements, then $1 = 1 \cdot 1' = 1'$.
3. A *group* is a tuple $(G, \cdot, ^{-1}, 1)$ where $(G, \cdot, 1)$ is a monoid and $^{-1}\colon G \to G$ is a unary operation, the *inverse operation*, written as g^{-1} instead of $^{-1}(g)$ for $g \in G$, such that $gg^{-1} = g^{-1}g = 1$ for all $g \in G$. We call g^{-1} the *inverse element* of $g \in G$, which is the unique element $h \in G$ such that $hg = gh = 1$.
4. A *semiring* is a tuple $(R, +, \cdot, 0, 1)$ where $(R, +, 0)$ and $(R, \cdot, 1)$ are monoids and the following additional equations are true:

   a. $x + y = y + x$ (*commutativity*),
   b. $x \cdot 0 = 0 \cdot x = 0$ (*annihilation law* for 0 in R),
   c. $x \cdot (y + z) = x \cdot y + x \cdot z$ (*left-distributivity*) and $(y + z) \cdot x = y \cdot x + z \cdot x$ (*right-distributivity*).

   The operation $+$ is called *addition* in R and $\cdot$ is called *multiplication* in R.

Due to associativity, we omit parentheses in expressions over semigroups.

Example 3.19 1. $(\Sigma^+, \cdot)$ for some alphabet Σ is a semigroup.
2. $(\Sigma^*, \cdot, \varepsilon)$ for some alphabet Σ is a monoid.
3. $(\mathbb{N}_0, +, 0)$ and $(\mathbb{N}_0, \cdot, 1)$ are monoids with the usual addition and multiplication operations.
4. $([n], +_n, 0)$ and $([n], \cdot_n, 1)$ are monoids for $n \geq 0$ with $x +_n y = \min(x + y, n - 1)$ and $x \cdot_n y = \min(x \cdot y, n - 1)$ for $x, y \in [n]$.
5. The integers $\mathbb{Z}$ with the usual addition, inversion $x \mapsto -x$ and identity element $0 \in \mathbb{Z}$ form a group.
6. The *Boolean semiring* is the semiring $(\{0, 1\}, \wedge, \vee, 0, 1)$.
7. $(\mathbb{N}_0, +, \cdot, 0, 1)$ and $([n], +_n, \cdot_n, 0, 1)$ for $n \geq 1$ are semirings. For $n = 2$ the latter semiring equals the Boolean semiring.

Let $(S_1, \cdot_1), (S_2, \cdot_2)$ be two semigroups. A *(semigroup) morphism* is a map $\varphi\colon S_1 \to S_2$ such that $\varphi(xy) = \varphi(x)\varphi(y)$ for all $x, y \in S_1$. We call φ an *isomorphism* if there exists a semigroup morphism $\psi\colon S_2 \to S_1$ such that $\psi\varphi = \mathrm{id}_{S_1}$ and $\varphi\psi = \mathrm{id}_{S_2}$. If φ is bijective, for its inverse function $\varphi^{-1}\colon S_2 \to S_1$ we have $\varphi^{-1}(\varphi(x)\varphi(y)) = \varphi^{-1}(\varphi(xy)) = xy = \varphi^{-1}(\varphi(x))\varphi^{-1}(\varphi(y))$ for $x, y \in S_1$. This yields that φ^{-1} is a semigroup morphism. With Theorem 3.3 we find that the semigroup isomorphisms are precisely the bijective morphisms. We say that S_1 is a *subsemigroup* of S_2, written $S_1 \leq_S S_2$, if $S_1 \subseteq S_2$ and, for all $x, y \in S_1$, $x \cdot_2 y \in S_1$ and $x \cdot_1 y = x \cdot_2 y$. For $i > 0$ and $x \in S_1$ we set $x^i = x \cdots x$ (i times). Let $A, B \subseteq S_1$. The *product on subsets of* S_1 (also called *complex product* in [393]) is $A \cdot B = \{ab \mid a \in A, b \in B\}$, written AB for short. This gives a semigroup $(\mathcal{P}(S_1), \cdot)$. Also, we set $aA = \{a\}A$ and $Aa = A\{a\}$ for $a \in S_1$. We call S_1

commutative if $xy = yx$ for all $x, y \in S_1$. An element $x \in S_1$ is called *idempotent* if $x^2 = x$.

Lemma 3.20 *Let S be a finite semigroup. Then for each $s \in S$ there exists $i > 0$ such that s^i is idempotent.*

Proof. By finiteness, we have $s^{j+p} = s^j$ for some $j \geq 0$ and $p > 0$. Let $i \geq 0$ be such that $j \leq i < j + p$ and $i \equiv 0 \pmod{p}$. Then $(s^i)^2 = s^{i+i} = s^i$. □

Remark 3.21 Let $(S, \cdot)$ be a semigroup. By adding some element $1 \notin S$ and setting $1 \cdot 1 = 1$ and $1 \cdot s = s \cdot 1 = s$ for all $s \in S$, we obtain a monoid $S \cup \{1\}$.

Let M_1, M_2 be two monoids with identity elements 1_1 and 1_2, respectively. We set $x^0 = 1_1$ for $x \in M_1$. A *monoid morphism* (resp. *monoid isomorphism*) is a semigroup morphism (resp. semigroup isomorphism) $\varphi\colon M_1 \to M_2$ with $\varphi(1_1) = 1_2$. When the context is clear, we simply say morphism (resp. isomorphism) instead of semigroup or monoid morphism (resp. isomorphism). We call M_1 a *submonoid* of M_2, written $M_1 \leq_M M_2$, if M_1 is a subsemigroup of M_2 with $1_2 \in M_1$. The *group of units* of M_1 is

$$\{g \in M_1 \mid \exists h \in M_1 : gh = hg = 1\}.$$

The group of units is in fact a group. Further, for $A \subseteq M_1$, we set $A^0 = \{1\}$, $A^{\leq n} = A^0 \cup A \cup A^2 \cup \ldots \cup A^n$ for $n \geq 0$, $A^{<0} = \emptyset$ and $A^{<n} = A^{\leq n-1}$ for $n > 0$. For $a \in M_1$ we also write $a^{<n}$ and $a^{\leq n}$ for $\{a\}^{<n}$ and $\{a\}^{\leq n}$, respectively.

Remark 3.22 In monoids, there might exist subsemigroups that are monoids in itself, but not submonoids. For example, in $\{1, s, s^2\}$ with $s^2 = s^3$ and $s1 = 1s = s$, $\{s^2\}$ is a subsemigroup that is a monoid, but it is not a submonoid.

Example 3.23 Let M be a monoid with identity $1 \in M$. Then we obtain a semiring structure $(\mathcal{P}(M), \cup, \cdot, \emptyset, 1)$ from the product on subsets and set union.

Let G_1, G_2 be two groups and $1 \in G_1$ be the identity in G_1. We call G_1 *trivial* if $G_1 = \{1\}$. A *group morphism* $\varphi\colon G_1 \to G_2$ is a monoid morphism such that $\varphi(g^{-1}) = \varphi(g)^{-1}$ for all $g \in G_1$. Now, for every semigroup morphism $\psi\colon G_1 \to G_2$ we have $\psi(1) = \psi(1 \cdot 1) = \psi(1)\psi(1)$, which implies that $\psi(1)$ is the identity in G_2, and $\psi(g)\psi(g^{-1}) = \psi(gg^{-1}) = \psi(1)$, so that $\psi(g)^{-1} = \psi(g^{-1})$. Thus, the group morphisms are precisely the semigroup morphisms and, technically, the definition of group morphisms can be omitted (we included it for conceptual reasons). We say that G_1 is a *subgroup* of G_2, written $G_1 \leq_G G_2$, if G_1 is a submonoid of G_2 and the inverse operation in G_1 agrees with the inverse operation in G_2 for each element of G_1, i.e., inversion in G_1 equals the inversion in G_2 restricted to G_1.[5] Next, assume $G_1 \leq_G G_2$.

1. A subset of the form $G_1 g$ (resp. gG_1) for some $g \in G_2$ is called a *right coset* (resp. *left coset*) of G_1 in G_2. A right coset is also just called coset. The set of right cosets (resp. left cosets) of G_1 forms a partition of G_2. If G_2 is finite, then each right coset (resp. left coset) has size $|G_1|$.

[5] Here, we use the term subgroup only w.r.t. groups. In the literature, for example [449], subgroups are also defined for semigroups as subsemigroups having an identity element (that does not have to be an identity for the whole semigroup) and containing inverses w.r.t. this identity.

2. We say that G_1 is a *maximal subgroup* of G_2 if $G_1 \neq G_2$ and for every subgroup H of G_2 with $G_1 \subseteq H$ we have $H = G_1$ or $H = G_2$.
3. We say that G_1 is a *normal subgroup* of G_2 if $gG_1g^{-1} \subseteq G_1$ for all $g \in G_1$.

A group G is said to be *simple* if the only normal subgroups of G are $\{1\}$ and G. The *center* of G is

$$Z(G) = \{g \in G \mid \forall h \in G\colon hg = gh\}.$$

We call G *cyclic* if $G = \langle\{g\}\rangle_G$ for some $g \in G$. A commutative group is also often called an *abelian group* in the literature. For a finite group G, $|G|$ is also called the *order* of the group.

For a subset T of a semigroup (resp. monoid, group) we denote by $\langle T\rangle_S$ (resp. $\langle T\rangle_M$, $\langle T\rangle_G$) the *subsemigroup* (resp. *submonoid, subgroup*) *generated by* T, i.e., the smallest subsemigroup (resp. submonoid, subgroup) containing T in the semigroup (resp. monoid, group) under consideration. In this regard, the elements in T are called the *generators* of the semigroup $\langle T\rangle_S$ (resp. monoid $\langle T\rangle_M$, group $\langle T\rangle_G$). We have $\langle T\rangle_S = \bigcup_{i\geq 1} T^i$, $\langle T\rangle_M = \bigcup_{i\geq 0} T^i$, $\langle\emptyset\rangle_S = \emptyset$ and $\langle\emptyset\rangle_M = \langle\emptyset\rangle_G = \{1\}$. Suppose $(G, \cdot, ^{-1}, 1)$ is a finite group. Then for $g \in G$ there exists $n \geq 1$ such that $g^n = 1$ and so $g^{-1} = g^{n-1}$. This implies that the subsemigroups in the semigroup $(G, \cdot)$ are precisely the subgroups of $(G, \cdot, ^{-1}, 1)$ and, for non-empty $T \subseteq G$, we have $\langle T\rangle_G = \langle T\rangle_M = \langle T\rangle_S$. So, for finite groups and $T \neq \emptyset$, we can omit S, M or G in the index and just write unambiguously $\langle T\rangle$.

Let S_1, S_2 be semigroups. Then $S_1 \times S_2$ with the operation

$$(s, t)(s', t') = (ss', tt')$$

for $(s, t), (s', t') \in S_1 \times S_2$ is called the *direct product* semigroup. If S_1, S_2 are monoids (resp. groups) with identities $1 \in S_1$ and $1' \in S_2$, respectively, then $S_1 \times S_2$ is a monoid with identity $(1, 1)$ (resp. group with identity $(1, 1)$ and inversion $(s, t)^{-1} = (s^{-1}, t^{-1})$). Let I be some set. We obtain a semigroup $(S_1^I, \odot)$ with the *(point-wise* or *component-wise) product* $\odot$ given by $(f \odot g)(x) = f(x) \cdot g(x)$ for $f, g \in S_1^I$ and all $x \in I$. If S_1 is a monoid with identity $1 \in S_1$, then S_1^I is a monoid where the identity is the constant function $\mathbf{1} \in S_1^I$ mapping everything to 1.

Example 3.24 Consider $(\mathbb{N}_0, +, 0)$, $(\mathbb{N}_0, \cdot, 1)$, $([n], +_n, 0)$ and $([n], \cdot_n, 1)$ from Example 3.19 and an alphabet Σ. We obtain monoid structures $(\mathbb{N}_0^\Sigma, \oplus, \mathbf{0})$ and $(\mathbb{N}_0^\Sigma, \odot, \mathbf{1})$ from $(\mathbb{N}_0, +, 0)$ and $(\mathbb{N}_0, \cdot, 1)$, respectively, by the construction given above where $\oplus$ is also called the *component-wise addition* on $\mathbb{N}_0^\Sigma$. Similarly, we obtain monoid structures $([n]^\Sigma, \oplus_n, \mathbf{0})$ and $([n]^\Sigma, \odot_n, \mathbf{1})$ on $[n]^\Sigma$. In the following, unless stated otherwise, we assume these monoid structures on $\mathbb{N}_0^\Sigma$ and $[n]^\Sigma$, and we write $\oplus$ as $+$ and $\oplus_n$ as $+_n$, since there is no danger of confusion.

3.5.2 Matrices

Let R be a semiring and $m, n \geq 0$. An $m \times n$ *matrix* A (over R) is a mapping $A\colon \{1, \ldots, m\} \times \{1, \ldots, n\} \to R$, also written as $A = (a_{ij})_{i\in\{1,\ldots,m\}, j\in\{1,\ldots,n\}}$

with $a_{ij} = a(i)(j)$. When the ranges of i and j are clear from the context, we also write $A = (a_{ij})$ for short. The elements $a_{i1}, \ldots, a_{in}$ for some $i \in \{1, \ldots, m\}$ constitute the i-th *row* of A and the elements $a_{1j} \ldots, a_{mj}$ for some $j \in \{1, \ldots, n\}$ constitute the j-th *column* of A. Correspondingly, the first index (here i) is called the *row index* and the second index (here j) is called the *column index*. See Figure 3.3 for a schematic depiction of an $m \times n$ matrix $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$.

$$A = \overbrace{\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}}^{n \text{ columns}} \Bigg\} m \text{ rows}$$

Fig. 3.3: Scheme of an $m \times n$ matrix $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$.

Let $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ be an $m \times n$ matrix and $B = (b_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,l\}}$ be an $n \times l$ matrix where $l, m, n \geq 1$. The *(matrix) product*, denoted by $A \cdot B$ or simply AB, is the $m \times l$ matrix C, i.e, $C = AB$, where $C = (c_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,l\}}$ is given by

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}$$

for $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, l\}$. The *(matrix) sum* of two $m \times n$ matrices $A = (a_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ and $B = (b_{ij})_{i \in \{1,\ldots,m\}, j \in \{1,\ldots,n\}}$ is the $m \times n$ matrix $C = (c_{ij})_{i \in \{1,\ldots,n\}, j \in \{1,\ldots,m\}}$, written $C = A + B$, given by $c_{ij} = a_{ij} + b_{ij}$ for $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, n\}$.

Let $n \geq 1$. The $n \times n$ *identity matrix (over R)* is $I_n = (a_{ij})$ defined by $a_{ij} = 1$ if $i = j$ and $a_{ij} = 0$ if $i \neq j$. The $n \times n$ *zero matrix* (over R) is $O_n = (o_{i,j})$ defined by $o_{ij} = 0$ for all $i, j \in \{1, \ldots, n\}$.

The set of $n \times n$ matrices over a semiring forms a semiring with the product and sum operations, and the identity and zero matrices.

A *Boolean matrix* is a matrix over the Boolean semiring $(\{0, 1\}, \wedge, \vee, 0, 1)$.

Next, we show how to describe a transformation $t \colon X \to X$ on a finite set of size n by a matrix. Fix some order $X = \{x_1, \ldots, x_n\}$ (where we take $X = [n]$ with $x_i = i - 1$ for $i \in [n]$ if nothing else is said). Then, to t we associate a *(Boolean) transition matrix* $A = (a_{ij})_{i,j \in \{1,\ldots,n\}}$ by setting

$$a_{ij} = \begin{cases} 1 & \text{if } t(x_i) = x_j, \\ 0 & \text{otherwise .} \end{cases}$$

Intuitively, point x_i is mapped to x_j if and only if in the i-th row of A the unique one appears at position j, i.e., in the j-th column of A. When the specific ordering of the states is not important, we assume some order to be taken without explicitly mentioning it.

3.5.3 Transformation Semigroups

The set of all transformations on a set X with function composition as operation forms a semigroup $(X^X, \circ)$, called the *full transformation semigroup on* X. This set also forms a monoid with identity id_X. We also denote $(X^X, \circ)$ by $\mathcal{T}_X$. For $X = [n]$, we set $\mathcal{T}_n = \mathcal{T}_X$. A *transformation semigroup* (resp. *transformation monoid*) on the *domain* X is a subsemigroup (resp. submonoid) of $\mathcal{T}_X$.

Let $S \leq_S \mathcal{T}_X$ be a transformation semigroup. A *fixpoint* of S is a point fixed by all elements from S. An *S-congruence* is an equivalence relation $\sim$ on X such that, for all $x, y \in X$ and $t \in S$, we have

$$x \sim y \Rightarrow t(x) \sim t(y). \tag{3.29}$$

We also say that x and y are *congruent* if $x \sim y$. Equivalence classes of $\sim$ are also called *congruence classes* in this case. The universal and equality relations on X are S-congruences, called the *trivial S-congruences*.

Recall from Section 3.2 that the elements of X are also called *points* in the context of transformations.

Remark 3.25 Let $(M, \cdot, 1)$ be a monoid. For $m \in M$ let $\hat{m}\colon M \to M$ be given by $\hat{m}(x) = xm$ for $x \in M$. Then $\hat{M} = \{\hat{m} \mid m \in M\}$ is a transformation monoid on the domain M. The map $m \mapsto \hat{m}$ is injective since $\hat{m} = \hat{n}$ for $m, n \in M$ implies $m = \hat{m}(1) = \hat{n}(1) = n$, and it is a monoid morphism. Thus, $\hat{M}$ represents the monoid M as a transformation monoid on the domain M. The M-congruences of $\hat{M}$ are also called right-congruences of M in the literature, see, for example, [325].

Remark 3.26 Let S be a semigroup. By adjoining an identity as in Theorem 3.21 we obtain a monoid $M = S \cup \{1\}$. Consider the transformation monoid constructed from it as in Theorem 3.25 via the map $m \mapsto \hat{m}$ for $m \in M$. Then $\{\hat{s} \mid s \in S\}$ is a transformation semigroup and the map $s \mapsto \hat{s}$ for $s \in S$ is an injective semigroup morphism. Thus, the semigroup S is represented as a transformation semigroup.

The embedding of an arbitrary semigroup in a transformation semigroup as in Theorem 3.26 is also known as Cayley's theorem, for example, see [236, Thm. 2.4.3]. However, Cayley only gave it for groups [137].[6] The extension to semigroups is due to Suschkewitsch [588].

3.6 Order Theory

Here, we collect the order-theoretical notions that we need such as quasiorders, partial orders, the induced equivalence relation, upper sets, etc. Section 3.6.1 give definitions

[6] Cayley did not give an explicit statement, but only implicitly assumed it, since the notion of an abstract group was not yet invented. He investigated a formal calculus of operations with an identity element and satisfying associativity (implicitly assuming inverses) and used it to investigate groups.

and some basic equations. In Section 3.6.2, we investigate upper and lower sets and the induced equivalence relation. Maximal and minimal elements are investigated in Section 3.6.3, and the set of maximal elements in Section 3.6.4. Section 3.6.5 collects results on sequences of upper sets. We later need all these notions when we introduce a special quasiorder on automata.

3.6.1 Definitions and Basic Facts

1. A *quasiorder* (or *preorder*) on a set X is a reflexive and transitive relation. We also write $(X, \lesssim)$ for X together with the quasiorder $\lesssim$.
2. For a quasiordered set $(X, \lesssim)$ we call two elements $x, y \in X$ *incomparable* if neither $x \lesssim y$ nor $y \lesssim x$ is true.
3. Let $\lesssim$ be a quasiorder on X and $S \subseteq X$.
   a. We call $x \in X$ a *minimal element* (resp. *maximal element*) of S if $x \in S$ and for all $y \in S$, if $y \lesssim x$ (resp. $x \lesssim y$), then $x \lesssim y$ (resp. $y \lesssim x$).
   b. We call $x \in X$ a *minimum*, or *least element*, (resp. *maximum*, or *greatest element*) of S if $x \in S$ and for all $y \in S$ we have $x \lesssim y$ (resp. $y \lesssim x$).
   c. The set S is a *lower set* (resp. an *upper set*) if $x \in S$ and $y \lesssim x$ (resp. $x \lesssim y$) imply $y \in S$ for $x, y \in X$.
   d. The *smallest lower set* (resp. *upper set*) *containing S (in X)* is $\downarrow S = \{y \in X \mid \exists x \in S \colon y \lesssim x\}$ (resp. $\uparrow S = \{y \in X \mid \exists x \in S \colon x \lesssim y\}$).

   > The notions of maximal/maximum and minimal/minimum are usually only defined for partially ordered sets, but here we use them for quasiordered sets as well.

4. Given quasiordered sets $(X, \lesssim)$ and $(Y, \lessapprox)$ an *(order) morphism* (or *monotone function*) is a function $\varphi \colon X \to Y$ such that, for all $x, y \in X$, we have

   $$x \lesssim y \Rightarrow \varphi(x) \lessapprox \varphi(y).$$

   Additionally, we call $\varphi \colon X \to Y$ an *(order) isomorphism* if there exists another order morphism $\psi \colon Y \to X$ such that $\varphi\psi = \mathrm{id}_Y$ and $\psi\varphi = \mathrm{id}_X$.
5. A *partial order* on a set X is a quasiorder that is antisymmetric.
6. A *total* (or *linear*) *order* is a partial order $\leq$ on X such that, for all $x, y \in X$, we have $x \leq y$ or $y \leq x$.
7. A *topological sorting* (also *topological order* or *linear extension*) of a partial order $\leq$ on X is a total order $\leq^*$ on X such that the identity transformation $\mathrm{id} \colon X \to X$ is a morphism from $(X, \leq)$ to $(X, \leq^*)$.[7]
   For finite X, a topological sorting can be identified with an indexing $x_1, \ldots, x_n$ of all the elements from X such that $x_i \leq x_j$ implies $i \leq j$.

[7] According to Szpilrajn's extension theorem every partial order can be topologically sorted [591].

8. Given a quasiorder $\lesssim$ on a set X, the *induced equivalence relation (by $\lesssim$)* is

$$x \simeq y \Leftrightarrow x \lesssim y \wedge y \lesssim x, \tag{3.30}$$

and the well-defined *induced partial order* $\leq$ on the equivalence classes $X/\simeq$ is defined by setting

$$[x]_\simeq \leq [y]_\simeq \Leftrightarrow x \lesssim y. \tag{3.31}$$

By Equation (3.31) the function $x \mapsto [x]_\simeq$ is a surjective order morphism. When the context is clear, we also denote the induced partial order on the equivalence classes by the same symbol $\lesssim$.

A minimum (resp. maximum) is also a minimal (resp. maximal) element. If not said otherwise, we mean minimal, maximal, minimum, and maximum w.r.t. $S = X$. In a partially ordered set, a minimum (resp. maximum) equals a unique minimal (resp. maximal) element. For $p \lesssim q$, we also say that p is below q, q is above p, and that these relations hold in a strict sense if additionally $q \not\lesssim p$.

Example 3.27 Let $(X, \lesssim)$ be a quasiordered set and I be some set. Then the set of functions X^I becomes a quasiordered set $(X^I, \lesssim^*)$ by setting $f \lesssim^* g$ if and only if $f(x) \lesssim g(x)$ for all $x \in I$ and $f, g \in X^I$. If $\lesssim$ is a partial order, then $\lesssim^*$ is also a partial order, called the *component-wise order*. Often, without special mentioning, we simply denote this quasiorder on X^I by $\lesssim$ as well.

Example 3.28 For some alphabet Σ, the sets of functions $\mathbb{N}_0^\Sigma$ and $[n]^\Sigma$ are partially ordered by the component-wise order from Theorem 3.27 derived from the usual order on $\mathbb{N}_0$.

Let $(X, \lesssim)$ and $(Y, \lessapprox)$ be quasiordered sets with induced equivalence relations $\simeq$ and $\approx$, respectively, and let $\varphi\colon X \to Y$ be an order morphism.

Proposition 3.29 *For $x \in X$ we have $\varphi([x]_\simeq) \subseteq [\varphi(x)]_\approx$, i.e., the morphism φ maps an equivalence class into an equivalence class. Furthermore, if φ is an order isomorphism, we also have $\varphi([x]_\simeq) = [\varphi(x)]_\approx$.*

Proof. Let $y \in [x]_\simeq$, i.e., $x \simeq y$. Clearly, we have $\varphi(x) \approx \varphi(y)$. Thus $\varphi(y) \in [\varphi(x)]_\approx$.

Next, suppose φ is an order isomorphism. Then $\psi\varphi = \mathrm{id}_X$ for some order morphism $\psi\colon Y \to X$. Let $y \in [\varphi(x)]_\approx$. By Theorem 3.3, φ is surjective. Thus there exist $z \in X$ such that $y = \varphi(z)$. Since $\varphi(z) \approx \varphi(x)$, we find $z = \psi(\varphi(z)) \approx \psi(\varphi(x)) = x$. Thus $z \in [x]_\simeq$ and so $y \in \varphi([x]_\simeq)$. □

Proposition 3.30 *The morphism φ induces a well-defined order morphism $\overline{\varphi}\colon X/\simeq \to Y/\approx$ w.r.t. the induced partial orders given by $\overline{\varphi}([x]_\simeq) = [\varphi(x)]_\approx$.*

Proof. Since $x \simeq y$ implies $\varphi(x) \simeq \varphi(y)$, $\overline{\varphi}$ is well-defined. By Equation (3.31) it is a morphism. □

Proposition 3.31 *$\varphi(S) \cap T \neq \emptyset$ implies $\varphi(S) \subseteq T$ for equivalence classes $S \subseteq X$ and $T \subseteq Y$ of the induced equivalence relation.*

Proof. Suppose $\varphi(S) \cap T \neq \emptyset$. Since all elements in $\varphi(S)$ are equivalent w.r.t. $\cong$, and T, as an equivalence class, is a maximal set whose element are all equivalent, we find $\varphi(S) \subseteq T$. □

The following example gives a bijective (order) morphism that is not an isomorphism.

Example 3.32 Let $X = \{0, 1\}$ and $\leq$ be the partial order with $0 \leq 1$. Then id_X is a bijective morphism of X ordered by equality to X ordered by $\leq$ that is not an isomorphism.

In the following statements, $(X, \lesssim)$ is a quasiordered set with induced equivalence relation $\simeq$ and induced partial order $\leq$ on $X/\simeq$.

For $x, y \in X$, the following hold:

$$x \lesssim y \Leftrightarrow y \in \uparrow\{x\} \Leftrightarrow \uparrow\{y\} \subseteq \uparrow\{x\} \tag{3.32}$$

$$x \simeq y \Leftrightarrow \uparrow\{x\} = \uparrow\{y\} \tag{3.33}$$

$$\tag{3.34}$$

For a morphism $\varphi\colon X \to Y$ and $S \subseteq X$ we have:

$$\varphi(\uparrow S) \subseteq \uparrow\varphi(S) \tag{3.35}$$

$$S \text{ is an upper set} \Rightarrow \varphi^{-1}(S) \text{ is an upper set} \tag{3.36}$$

For $S, T \subseteq X$ we have:

$$S, T \text{ are upper sets} \Rightarrow S \cup T \text{ and } S \cap T \text{ are upper sets} \tag{3.37}$$

$$S, T \text{ are disjoint upper sets} \Rightarrow \forall x \in S\ \forall y \in T\colon x \not\leq y \wedge y \not\leq x \tag{3.38}$$

$$S \text{ is an upper set} \Leftrightarrow S = \bigcup_{x \in S} \uparrow\{x\} \tag{3.39}$$

$$S \text{ is an upper set} \Leftrightarrow X \setminus S \text{ is a lower set} \tag{3.40}$$

Similar statements to Equations (3.32), (3.33), (3.35), and (3.37) to (3.39) also hold for lower sets.

Remark 3.33 The inclusion from Equation (3.35) can be proper. For example, let $X = \{x_1, x_2\}$ where $x_1 \lesssim x_2$. Then with the morphism $\varphi\colon X \to X$ defined by $\varphi(x_1) = \varphi(x_2) = x_1$ we have $\varphi(\uparrow\{x_1\}) = \{x_1\}$ and $\uparrow\{\varphi(x_1)\} = \{x_1, x_2\}$. This also shows that a morphism does not necessarily map upper sets to upper sets. More generally, if all elements are comparable in an arbitrary set X with at least two elements, then any constant map is an order morphism such that the inclusion from Equation (3.35) is proper.

Remark 3.34 Let $x \in X$. Then $\uparrow\{x\} \setminus [x]_\simeq$ is a upper set and $\downarrow\{x\} \setminus [x]_\simeq$ is a lower set.

From Equation (3.40) we obtain the next statement.

Proposition 3.35 *$S \subseteq X$ is a maximal proper upper set w.r.t. set inclusion if and only if $X \setminus S$ is a minimal non-empty lower set w.r.t. set inclusion.*

Remark 3.36 In most of the remaining statements of the present section that make a claim on upper and lower sets, we only give the proofs for upper sets, since the corresponding statement for lower sets follows similarly.

3.6.2 Equivalence Classes and Upper and Lower Sets

Here, Theorem 3.37 relates upper and lower sets to equivalence classes, and we list consequences in Theorems 3.38 and 3.39. Theorems 3.40 to 3.42 give relations between maximal and minimal upper and lower sets and equivalence classes.

In this section, let $(X, \lesssim)$ be a quasiorder and let $\simeq$ be its induced equivalence relation.

Proposition 3.37 *For $S \subseteq X$, the following are equivalent.*

1. *S is contained in an equivalence class for $\simeq$.*
2. *For every upper set $T \subseteq X$ we have $S \subseteq T$ or $S \cap T = \emptyset$.*
3. *For every lower set $T \subseteq X$ we have $S \subseteq T$ or $S \cap T = \emptyset$.*

Proof. $1 \Rightarrow 2$: Let $T \subseteq X$ be an upper set. Suppose we have $x \in S \cap T$. Since T is an upper set, we have $\uparrow\{x\} \subseteq T$. By Equation (3.32), we find $[x]_\simeq \subseteq \uparrow\{x\}$. Thus $S \subseteq T$ since $S \subseteq [x]_\simeq$ by assumption.

$2 \Rightarrow 3$: Let $T \subseteq X$ be a lower set. Since $X \setminus T$ is an upper set by Equation (3.40), by Item 2, S is either contained in $X \setminus T$ or T.

$3 \Rightarrow 1$: Let $x, y \in S$. By assumption, we find $S \subseteq \downarrow\{y\}$ and $S \subseteq \downarrow\{x\}$. This gives $x \lesssim y$ and $y \lesssim x$, respectively. Thus $x \simeq y$ and so S is contained in an equivalence class. □

Corollary 3.38 *Every upper or lower set $S \subseteq X$ is a union of equivalence classes.*

Proof. Clearly, $S \subseteq \bigcup_{y \in S} [y]_\simeq$. By Theorem 3.37, we have $\bigcup_{y \in S} [y]_\simeq \subseteq S$. □

Corollary 3.39 *Let $S, T \subseteq X$ be both upper or lower sets. If $T \setminus S$ is non-empty and contained in an equivalence class, then it equals this equivalence class.*

Proof. Without loss of generality, suppose S and T are upper sets. Let $x \in T \setminus S$. By assumption, $T \setminus S \subseteq [x]_\simeq$. By Theorem 3.37, $[x]_\simeq \subseteq T$ and $[x]_\simeq \cap S = \emptyset$. Therefore, $[x]_\simeq \subseteq T \setminus S$. □

Lemma 3.40 *Let $S, T \subseteq X$. If $T \setminus S$ is contained in an equivalence class, then for all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*

Proof. Let R be an upper set with $S \cap T \subseteq R \subseteq T$. By Theorem 3.37 either we have $(T \setminus S) \subseteq R$, which gives $T = (S \cap T) \cup (T \setminus S) \subseteq R$ and so $R = T$, or $(T \setminus S) \cap R = \emptyset$, which gives $R = R \cap T = R \cap ((S \cap T) \cup (T \setminus S)) = (R \cap (S \cap T)) \cup (R \cap (T \setminus S)) = R \cap (S \cap T) = S \cap T$. □

Lemma 3.41 *Let $S, T \subseteq X$. Then the following are equivalent.*

1. *For all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*
2. *For all lower sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$, we have $R = S \cap T$ or $R = T$.*

Proof. Let R be a lower (resp. an upper) set with $S \cap T \subseteq R \subseteq T$. Then $X \setminus T \subseteq X \setminus R \subseteq X \setminus (S \cap T) = (X \setminus S) \cup (X \setminus T)$. Since $X \setminus R$, $X \setminus S$, $X \setminus T$, and $(X \setminus T) \cup (X \setminus T)$ are upper (resp. lower) sets by Equations (3.37) and (3.40), from Item 1 we obtain $X \setminus R = X \setminus T$ or $X \setminus R = X \setminus (S \cap T)$, and so $R = T$ or $R = S \cap T$. This proves Item 2. The converse implication follows similarly. □

An analogous equivalence to Theorem 3.41, between upper and lower sets but with $S \subseteq R \subseteq S \cup T$ in place of $S \cap T \subseteq R \subseteq T$, can be established. We omit such a statement here.

Proposition 3.42 *Let $S, T \subseteq X$ be both upper sets or both lower sets. Then the following are equivalent.*

1. *$T \setminus S$ is contained in an equivalence class.*
2. *For all upper sets $R \subseteq X$ with $S \cap T \subseteq R \subseteq T$ we have $R = S \cap T$ or $R = T$.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 3.40 (note that this statement applies to any sets, not just upper/lower sets).

$2 \Rightarrow 1$: First, suppose S and T are upper sets. Suppose $T \setminus S$ is not contained in an equivalence class. Then we have $x \not\lesssim y$ for some $x, y \in T \setminus S$. Let $R = \uparrow\{x\}$. We have $y \notin R$. By Equation (3.37), $(S \cap T) \cup (R \cap T)$ is an upper set. This upper set properly contains $S \cap T$ and is properly contained in T.

Next, if S and T are lower sets, we use the same reasoning but with Theorem 3.41, Item 2 and lower set instead of upper sets.[8] □

By Theorems 3.39 and 3.42 for upper sets S, T, if $T \setminus S$ is non-empty and contained in an equivalence class, then $T = (S \cap T) \cup [x]_{\simeq}$ for some $x \in T \setminus S$.

3.6.3 Maximal and Minimal Elements

Theorem 3.43 characterizes upper and lower sets that are equivalence classes. In Theorem 3.47 we characterize uniqueness of a proper non-empty upper or lower set. Theorems 3.48 and 3.49 gives quantitative statements on the number of disjoint upper and lower sets, as well as on incomparable maximal and minimal elements in finite sets. Lastly, in Theorems 3.50 to 3.52 we list results concerning on maxima and minima.

In this section, let $(X, \lesssim)$ be a quasiorder and let $\simeq$ be its induced equivalence relation.

[8] Alternatively, for a lower set $R \subseteq X$ consider the lower set $K = (S \cup T) \cap (R \cup S)$. Since $S \subseteq K \subseteq S \cup T$, we have $(X \setminus S) \cap (X \setminus T) \subseteq X \setminus K \subseteq X \setminus S$ and as these sets are upper sets by Equation (3.37), from the assumption we get $X \setminus K = X \setminus (S \cup T)$ or $X \setminus K = X \setminus S$, equivalently $K = S \cup T$ or $K = S$. In the former case, $T \setminus S \subseteq R$, and in the latter case, $(T \setminus S) \cap R = \emptyset$. So, from Theorem 3.37, we get that $T \setminus S$ is contained in an equivalence class.

Proposition 3.43 *The following are equivalent for $S \subseteq X$:*

1. *S is a minimal non-empty upper (resp. lower) set w.r.t. set inclusion.*
2. *S is an equivalence class that is an upper (resp. a lower) set.*
3. *S is non-empty such that $S = \bigcap_{x \in S} \uparrow\{x\}$ (resp. $S = \bigcap_{x \in S} \downarrow\{x\}$).*
4. *S is non-empty such that $S = \uparrow\{x\}$ (resp. $S = \downarrow\{x\}$) for each $x \in S$.*
5. *Each $x \in S$ is maximal (resp. minimal) and $S = [x]_{\simeq}$.*
6. *We have $S = [x]_{\simeq}$ for some maximal (resp. minimal) $x \in X$.*
7. *S is a maximal (resp. minimal) equivalence class in $(X/\simeq, \leq)$.*

Proof. As mentioned in Theorem 3.36, we prove the result only for upper sets.

$1 \Rightarrow 2$: By minimality, there exists no upper set $R \subseteq X$ with $R \subseteq S$ such that $R \neq \emptyset$ and $R \neq S$. Hence, by Theorem 3.42 applied with $\emptyset$ and S, S is contained in an equivalence class. Since $S \neq \emptyset$, by Theorem 3.39 (or by Theorem 3.38), it equals an equivalence class.

$2 \Rightarrow 3$ Let $x \in S$. Since S is an equivalence class we have $S \subseteq \uparrow\{x\}$ by Theorem 3.38. Thus $S \subseteq \bigcap_{x \in S} \uparrow\{x\}$. Since S is an upper set, we have $\uparrow\{x\} \subseteq S$, Thus $\bigcap_{x \in S} \uparrow\{x\} \subseteq S$. Since S is an equivalence class it is non-empty.

$3 \Rightarrow 4$ Let $x \in S$. Then $S = \bigcap_{y \in S} \uparrow\{y\} \subseteq \uparrow\{x\}$. It is easy to see that $S = \bigcap_{y \in S} \uparrow\{y\}$ is an upper set. Thus we have $\uparrow\{x\} \subseteq S$ for $x \in S$ since $\uparrow\{x\}$ is the smallest upper set generated by x.

$4 \Rightarrow 5$: Let $x \in S$ and assume $x \lesssim y$ for some $y \in X$. Then $y \in \uparrow\{x\} = S$. By assumption, $\uparrow\{y\} = S = \uparrow\{x\}$. Thus, with Equation (3.33), we get $x \simeq y$. So x is maximal. Next, let $z \in [x]_{\simeq}$. Then $z \in \uparrow\{x\} = S$. Thus we have $[x]_{\simeq} \subseteq S$. Now, suppose $z \in S$. Then $\uparrow\{z\} = S = \uparrow\{x\}$ and so $z \simeq x$ by Equation (3.33). This gives $[x]_{\simeq} \subseteq S$.

$5 \Rightarrow 6$: This is clear.

$6 \Rightarrow 7$: Let $S = [x]_{\simeq}$ with x maximal in X. Suppose $[x]_{\simeq} \leq [y]_{\simeq}$ for some $y \in X$. Then we have $x \lesssim y$. Since x is maximal, we find $x \simeq y$ and so $[x]_{\simeq} = [y]_{\simeq}$. Thus $[x]_{\simeq}$ is maximal in $(X/\simeq, \leq)$.

$7 \Rightarrow 1$: Let $x \in S$ and $y \in X$ with $x \lesssim y$. Since S is maximal, we find $x \simeq y$ and so $x \in S$. Hence S is an upper set. Next, let $R \subseteq X$ be a non-empty upper set with $R \subseteq S$. Since S is an equivalence class, Theorem 3.38 (or Theorem 3.40) implies that $S \subseteq R$. Therefore, S is minimal w.r.t. set inclusion among all non-empty upper sets. □

Corollary 3.44 *An element $x \in X$ is maximal if and only if $[x]_{\simeq}$ is an upper set.*

Corollary 3.45 *If X is finite, then every upper set contains an upper upper set forming a maximal equivalence class in $(X/\simeq, \leq)$.*

Corollary 3.46 *An upper set is a maximal proper set w.r.t. set inclusion if and only if its complement is a minimal equivalence class for the induced partial order.*

Proof. This follows from directly from Theorems 3.35 and 3.43. □

Proposition 3.47 *Let $S \subseteq X$ be a non-empty and proper upper (resp. lower) set. Then S and X are the only non-empty upper (resp. lower) sets if and only if S and $X \setminus S$ are equivalence classes and $X \setminus S$ is not an upper (resp. a lower) set.*

Proof. Suppose S and X are the only non-empty upper sets. By considering $T = X$ and S in Theorem 3.42, we find that $X \setminus S$ is contained in an equivalence class. From Theorem 3.39 we get that it equals that equivalence class. Next, for $x \in S$, since $\uparrow\{x\} \subseteq S$, we get $\uparrow\{x\} = S$ and so S is an equivalence class by Theorem 3.43. By assumption, $X \setminus S$ is not an upper set.

Conversely, suppose S and $X \setminus S$ are equivalence classes. Since all upper sets are unions of equivalence classes and X is non-empty, S, $X \setminus S$, and X are all possible non-empty upper sets. So, if $X \setminus S$ is not an upper set, only S and X remain as upper sets. □

Proposition 3.48 *Suppose X is finite. Then the following numbers are equal:*

1. *The maximal number of pairwise disjoint non-empty upper (resp. lower) sets.*
2. *The number of maximal (resp. minimal) elements in $(X/\simeq, \leq)$.*
3. *The maximal number of incomparable maximal (resp. minimal) element in X.*

Proof. Let m_1, m_2, m_3 be the maximal number of pairwise disjoint non-empty upper sets, of distinct maximal elements in $(X/\simeq, \leq)$, and the maximal number of incomparable maximal elements in X, respectively.

By Theorem 3.43 maximal elements in $X/\simeq$ are upper sets that are equivalence classes, hence pairwise disjoint. This gives $m_1 \geq m_2$.

By the definition of the induced partial order, incomparable maximal elements induce distinct maximal elements in $X/\simeq$, which gives $m_2 \geq m_3$.

By Equation (3.38), we have $m_3 \geq m_1$. □

Corollary 3.49 *For finite X, there are at most k distinct maximal (resp. minimal) elements in the partially ordered set $(X/\simeq, \leq)$ if and only if, among $k + 1$ upper (resp. lower) sets, two have a non-empty intersection.*

The following statements are easy to verify and we omit their proofs.

Proposition 3.50 *The following are equivalent for $x \in X$:*

1. *$x \in X$ is a maximum (resp. minimum) for X.*
2. *$[x]_\simeq$ is a maximum (resp. minimum) in $(X/\simeq, \leq)$.*
3. *$[x]_\simeq \subseteq S$ for every upper (resp. lower) set $S \subseteq X$.*

Proposition 3.51 *The following are equivalent:*

1. *There exists a maximum (resp. minimum) in X for $\lesssim$.*
2. *For $x, y \in X$ there exists $z \in X$ with $x \lesssim z$ and $y \lesssim z$ (resp. $z \lesssim x$ and $z \lesssim y$).*
3. *For two maximal (resp. minimal) $x, y \in X$ we have $x \simeq y$.*
4. *For every two upper (resp. lower) sets $S, T \subseteq X$ we have $S \cap T \neq \emptyset$.*

Proposition 3.52 *Let $S \subseteq X$ be an equivalence class. If S is a maximum (resp. minimum) in $X/\simeq$, then it is the only upper (resp. lower) set among the equivalence classes. Furthermore, if X is finite and S the only upper (resp. lower) set that is an equivalence class, then it is a maximum (resp. minimum) in $X/\simeq$.*

Proof. We only give the proof for upper sets and maxima. First, suppose S is an equivalence class that is a maximum in $X/\simeq$. Let $T \subseteq X$ be an equivalence class that is an upper set. By Theorem 3.50, we have $S \subseteq T$. Since S and T are equivalence classes, we find $S = T$. By Theorem 3.43, S is an upper set, and so a unique equivalence class that is an upper set.

Conversely, suppose X is finite and S is a unique upper set that is an equivalence class. By Theorem 3.43, S is maximal in $X/\simeq$ and unique with this property. By finiteness, for each $x \in X$ there exists a maximal $y \in X$ such that $x \leq y$. Since S is the unique maximal element, we find $y \in S$. This yields that S is a maximum in $X/\simeq$. □

To show that the converse implication from Theorem 3.52 might fail for infinite X, consider $X = \mathbb{N}_0 \cup \{a\}$ with the usual order on $\mathbb{N}_0$ and a being incomparable to any other element. Then $\{a\}$ is a unique upper set that is an equivalence class, but it is not a maximum w.r.t. the induced partial order on the equivalence classes (which results in the same order structure since every equivalence class is a singleton).

3.6.4 The Set of Maximal Elements

In this section, let $(X, \lesssim)$ and

$$S = \{x \in X \mid x \text{ is maximal}\}$$

be the set of maximal elements. We denote the induced equivalence with $\simeq$.

Proposition 3.53 *The set of maximal elements S is an upper set and we have*

$$S = \bigcup_{x \text{ is maximal}} [x]_{\simeq}. \tag{3.41}$$

Proof. Equation (3.41) is obvious, and together with Theorem 3.43 we obtain that S is an upper set. □

Next, we investigate S for finite X. Similar statements could be obtained for the set of minimal elements. But since we only need the set of maximal elements later, we do not formulate corresponding statements for this set.

Lemma 3.54 *If X is finite, then for each $x \in X$ there exists $y \in S$ such that $x \lesssim y$.*

Proof. Assume there exists $x \in X$ for which this condition is false. Set $x_0 = x$ and, inductively, for x_i with $i \geq 0$ there exists $x_{i+1} \in X$ such that $x_i \lesssim x_{i+1}$ but $x_{i+1} \not\lesssim x_i$, since otherwise x_i would be a maximum in $\{x_0, x_1, \ldots, x_i\}$ and so also maximal such that $x \lesssim x_i$. Now, in this way, we obtain an infinite and strictly increasing sequence, which is not possible for finite X. □

Proposition 3.55 *Suppose X is finite. Then the set of maximal elements is the minimal non-empty set w.r.t. set inclusion that has non-empty intersection with each non-empty upper set.*

Proof. Let T be a minimal non-empty upper set that has non-empty intersection with each non-empty upper set.

Consider $x \in S$. By assumption, we have $y \in T \cap \uparrow\{x\}$ for some $y \in X$. Since x is maximal, this gives $y \simeq x$. Then $x \in T$ since T is an upper set. Hence $S \subseteq T$.

Next, let R be an upper set and $x \in R$. By Lemma 3.54, there exists $y \in S$ such that $x \lesssim y$. Since R is an upper set, we have $y \in R \cap S$. Hence, by minimality of T, we find $T \subseteq S$. □

Corollary 3.56 *Let X be finite and $S \subseteq X$ be a non-empty upper set. Then S contains an upper set that is an equivalence class.*

Proof. By Theorem 3.55 we have $q \in S$ for some maximal element $q \in X$. Then $[q]_\simeq$ is an upper set by Theorem 3.43 and we have $[q]_\simeq \subseteq S$. □

3.6.5 Sequences of Upper and Lower Sets

Here, we investigate sequences of increasing upper and lower sets under certain finiteness conditions. We only give statements for upper sets, since corresponding statements for lower sets follow analogously, and we actually only need them for upper sets in later chapters.

Proposition 3.57 *Let $S \subseteq Q$ be a proper upper set and $X \setminus S$ be finite. Then there exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = X$$

with $S_1 = S$ and such that $S_i \setminus S_{i-1}$ for $i \in \{2, \dots, m\}$ are all the equivalence classes in $X \setminus S$.

Proof. By finiteness, there exists a sequence of upper sets $S_1 \subsetneq \cdots \subsetneq S_m$ with $S_1 = S$ and m maximal. Since X is an upper set, we have $S_m = X$.

First, assume there exists $i \in \{2, \dots, m\}$ such that $S_i \setminus S_{i-1}$ is not contained in an equivalence class. By Theorem 3.37 there exists an upper set $T \subseteq X$ with $(S_i \setminus S_{i-1}) \cap T \neq \emptyset$ and $(S_i \setminus S_{i-1}) \not\subseteq T$. Thus, for the upper set $S_{i-1} \cup (T \cap S_i)$ we have $S_{i-1} \subsetneq S_{i-1} \cup (T \cap S_i) \subsetneq S_i$, which contradicts the maximal choice of m.

Next, let $x \in X \setminus S$. Then there exists a minimal $i \in \{2, \dots, m\}$ such that $x \in S_i$, so that $x \in S_i \setminus S_{i-1}$. Since $S_i \setminus S_{i-1}$ is contained in an equivalence class, we have $S_i \setminus S_{i-1} \subseteq [x]_\simeq$. By Theorem 3.39 we find $S_i \setminus S_{i-1} = [x]_\simeq$. □

Remark 3.58 Theorem 3.57 can also be proved by picking a proper upper set $S \subseteq S_{m-1}$ such that $|X \setminus S_{m-1}|$ has minimal size. Then S_{m-1} is a proper maximal upper set containing S and by Theorem 3.42 and Theorem 3.39 the difference $X \setminus S_{m-1}$ equals an equivalence class. If $S \neq S_{m-1}$, inductively, there exists a sequence of upper sets $S_1 \subsetneq \cdots \subseteq S_{m-1}$ such that $S_i \setminus S_{i-1}$ for $i \in \{1, \dots, m-1\}$ are all the equivalence classes in $S_{m-1} \setminus S$. Now for each equivalence class $R \subseteq X$ we either have $R \subseteq S_{m-1}$ or $R \cap S_{m-1} = \emptyset$. Since the latter case implies $R \subseteq X \setminus S_{m-1}$, and then $R = X \setminus S_{m-1}$ since $X \setminus S_{m-1}$ is an equivalence class, with $S_m = X$ we also obtain a sequence as claimed.

For finite X, by picking for S the empty set or the set of maximal elements we obtain the following corollary to Theorem 3.57.

Corollary 3.59 *Suppose X is finite.*

1. *There exists a sequence of upper sets*

$$\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = X$$

   such that $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the equivalence classes in X.
2. *There exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = X$$

   such that S_0 equals the set of maximal elements and each equivalence class of a non-maximal element has the form $S_i \setminus S_{i-1}$ for some $i \in \{1, \ldots, m\}$.

The sequences from Theorem 3.59, Item 1 are related to topological sortings in the following way. Given a topological sorting $[x_1]_{\simeq}, \ldots, [x_m]_{\simeq}$ of the elements from $X/{\simeq}$ for finite X, we obtain a sequence as in Theorem 3.59, Item 1 by setting $S_i = \uparrow\{x_1, \ldots, x_i\}$ and $S_0 = \emptyset$. Conversely, given a sequence as in Theorem 3.59, Item 1, the equivalence classes $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ give a topological sorting of $X/{\simeq}$ since $x_i \lesssim x_j$ for $x_i \in S_i \setminus S_{i-1}$ and $x_j \in S_j \setminus S_{j-1}$ with $i \neq j$ implies $\uparrow\{x_i\} \subsetneq \uparrow\{x_j\}$ by Equation (3.32). Thus, $\uparrow\{x_i\} \subseteq S_i$ and $\uparrow\{x_j\} \subseteq S_j$ give $S_i \subsetneq S_j$ and so $i \leq j$.

3.7 Formal Language Theory

Formal language theory is the study of Σ^* and its subsets as a means of representing and processing information. Besides the notions from formal language theory already introduced in Section 1.1, here, in Section 3.7.1, we introduce additional notions. In Section 3.7.2 we introduce regular expressions as a formalism to describe languages.

3.7.1 More Basic Notions

In Section 1.1 we introduced the $*$-operation that maps a set to the monoid of finite sequences over this set with concatenation as the binary operation. Despite being introduced only for finite sets Σ (here Σ always denotes a finite set), this operation can be applied to any set X and obeys the following property.

Proposition 3.60 (Universal Mapping Property) *Suppose X is a set. Then for every function $\varphi\colon X \to M$ into a monoid M, there exists a unique morphism $\hat{\varphi}\colon X^* \to M$*

(the evaluation morphism) such that $\varphi(x) = \hat{\varphi}(\iota(x))$ for all $x \in X$ where $\iota\colon X \to X^*$ is the embedding that maps each element from X to a sequence of length one in X^*.[9]

Proof. We set $\hat{\varphi}(x_1 \cdots x_n) = \varphi(x_1) \cdots \varphi(x_n)$ for $x_1, \ldots, x_n \in X$ and $\hat{\varphi}(\varepsilon)$ equal to the identity element of M. It is easy to verify that this gives a morphism as required and that it is in fact uniquely determined. □

The monoid X^* for some set X is called the *free monoid over* X. For a generating set X of M, $\hat{\varphi}\colon X^* \to M$ is surjective. So, every monoid is the surjective image of some free monoid.

Recall that, in Section 1.1, we have identified sequences of length one with X, i.e., x with $\iota(x)$ for $x \in X$ (which made $X \subseteq X^*$ into a meaningful statement). So, we can write $\varphi(x) = \hat{\varphi}(x)$ for $x \in X$. In the following, we denote the morphism $\hat{\varphi}$ by φ as well.

Theorem 3.60 implies that φ is fully specified by it images on the elements from X. Thus, to specify a morphism from X^* into some monoid M, it is sufficient to give the images of the element from X.

Now, let $L \subseteq X^*$. Applying Theorem 3.60 to $i\colon L \to X^*$ given by $i(u) = u$ for $u \in \Sigma^*$, we obtain a morphism $\hat{\imath}\colon L^* \to X^*$. We then have $\hat{\imath}(u_1 \cdots u_n) = u_1 \cdots u_n$ for $u_1, \ldots, u_n \in L$ where on the left hand side, concatenation in L^* is meant and on the right hand side concatenation in X^*. Thus, we find $\hat{\imath}(L^*) = \bigcup_{i \geq 0} L^i$. Then, by identifying L^* with $\hat{\imath}(L^*)$, the statement $L^* \subseteq X^*$ is meaningful and we can regard the $*$-operation as an operation of signature $\mathcal{P}(X^*) \to \mathcal{P}(X^*)$ mapping L to $\bigcup_{i \geq 0} L^i$.

Intuitively, the identification of L^* with $\hat{\imath}(L^*)$ corresponds to interpreting sequences whose entries are sequences over a set X as sequences over X. However, note that $\hat{\imath}$ need not be injective in general and in this case it is not an isomorphism. Thus, the submonoid $\hat{\imath}(L^*)$ of X^* might have different properties than L^* considered as the free monoid over L. Nevertheless, the identification of L^* and $\hat{\imath}(L^*)$ is common practice and does not pose any problems for our applications. So, we use this convention in the following without special mention.

Mostly, in our context, X equals some finite alphabet Σ. In this case, for $L \subseteq \Sigma^*$, we call L^*, with $L^* = \bigcup_{i \geq 0} L^i$ via our identification, the *Kleene star*.[10] Furthermore, the *Kleene plus* is $L^+ = \bigcup_{i \geq 1} L^i$.

Next, we state some rules for manipulating language equations. The two equations for the Kleene star follow with the identification discussed above.

Proposition 3.61 *Let $U, V, W \subseteq \Sigma^*$. Then the following holds.*

1. *$U(V \cap W) \subseteq UV \cap UW$ and $(V \cap W)U \subseteq VU \cap WU$,*
2. *$U(V \cup W) = UV \cup UW$ and $(V \cup W)U = VU \cup WU$,*

[9] This property, in fact, determines X^* up to isomorphism. First, note that $\mathrm{id}_{X^*}\colon X^* \to X^*$ fulfills $\iota(x) = \mathrm{id}_{X^*}(\iota(x))$ for all $x \in X$ and so, by the property, it is the unique morphism fulfilling this equation (and thus the unique extension of ι). Next. suppose $F(X)$ is another object fulfilling this property with embedding $\iota_{F(X)}\colon X \to F(X)$. Then there exist morphisms $\hat{\iota}_{F(X)}\colon X^* \to F(X)$ and $\hat{\iota}\colon F(X) \to X^*$ such that $\iota_{F(X)}(x) = \hat{\iota}_{F(X)}(\iota(x))$ and $\iota(x) = \hat{\iota}(\iota_{F(X)}(x))$ for $x \in X$. Thus $\iota(x) = \hat{\iota}(\hat{\iota}_{F(X)}(\iota(x)))$ for $x \in x$, and so $\hat{\iota}\hat{\iota}_{F(X)} = \mathrm{id}_{X^*}$. Similarly, we find $\hat{\iota}_{F(X)}\hat{\iota} = \mathrm{id}_{F(X)}$ and so $\hat{\iota}$ and $\hat{\iota}_{F(X)}$ are isomorphisms.

[10] In the literature, the Kleene star is often directly defined as $\bigcup_{i \geq 0} L^i$ and not as a mapping of sets into a monoid of sequences that yields this set under a suitable interpretation of sequences of sequences.

3. $(U \cup V)^* = (U^*V)^*U^*$.
4. $(U^*)^* = U^*$.

A language L is said to be *unary* (resp. *binary* or *ternary*) if there exists an alphabet Σ with $|\Sigma| = 1$ (resp. $|\Sigma| = 2$ or $|\Sigma| = 3$) such that $L \subseteq \Sigma^*$.

Let $L \subseteq \Sigma^*$. For $u \in \Sigma^*$, the *(left) quotient (of L by u)* is $u^{-1}L = \{v \in \Sigma^* \mid uv \in L\}$. Language quotients were introduced by Brzozowski [106] (called *derivatives* therein). Next, set $\mathrm{Pref}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x \in \Sigma^* \colon v = ux\}$, $\mathrm{Suff}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x \in \Sigma^* \colon v = xu\}$, and $\mathrm{Fact}(L) = \{u \in \Sigma^* \mid \exists v \in L\ \exists x, y \in \Sigma^* \colon v = xuy\}$. For $u \in \Sigma^*$, we set $\mathrm{Pref}(u) = \mathrm{Pref}(\{u\})$, $\mathrm{Suff}(u) = \mathrm{Suff}(\{u\})$, and $\mathrm{Fact}(u) = \mathrm{Fact}(\{u\})$.[11] We call $u \in \Sigma^*$ a *prefix* (resp. *suffix*, *factor*) of $v \in \Sigma^*$ if $u \in \mathrm{Pref}(v)$ (resp. $u \in \mathrm{Suff}(v)$, $u \in \mathrm{Fact}(v)$). A prefix (resp. suffix, factor) u of v is called *proper* if $u \neq v$, i.e., if $v \in u\Sigma^+$ (resp. $v \in \Sigma^+u$, $v \in \Sigma^*u\Sigma^+ \cup \Sigma^+u\Sigma^*$).

A *left ideal language* (resp. *right ideal language*) is a language $I \subseteq \Sigma^*$ such that $\Sigma^*I \subseteq I$ (resp. $I\Sigma^* \subseteq I$). Left (resp. right) ideals are precisely the languages of the form $\Sigma^* \setminus \mathrm{Suff}(L)$ (resp. $\Sigma^* \setminus \mathrm{Pref}(L)$) for some $L \subseteq \Sigma^*$. An *ideal language* is a language that is simultaneously a left and a right ideal language.

We need the following result to solve language equations.

Lemma 3.62 (Arden's rule (see [322])) *Let $U, V \subseteq \Sigma^*$ with $\varepsilon \notin U$. Then the language equation $X = XU \cup V$ (resp. $X = UX \cup V$) has the unique solution $X = VU^*$ (resp. $X = U^*V$).*

Example 3.63 Let $L \subseteq \Sigma^+$. We have $\Sigma^* = L\Sigma^* \cup (\Sigma^* \setminus L\Sigma^*)$. By Theorem 3.62 this gives $\Sigma^* = L^*(\Sigma^* \setminus L\Sigma^*)$. Note that $L^*(\Sigma^* \setminus L\Sigma^*) = \Sigma^* \setminus L\Sigma^* \uplus L^+(\Sigma^* \setminus L\Sigma^*)$.

For $L \subseteq \Sigma^*$, $u \in \Sigma^*$ and $n \geq 1$ we also need the following formulas that are easy to verify:

$$u \in \mathrm{Pref}(L) \Leftrightarrow \mathrm{Pref}(u) \subseteq \mathrm{Pref}(L) \tag{3.42}$$

$$\mathrm{Pref}(L^*) \cap \Sigma^{\leq n} = \mathrm{Pref}(L^n) \cap \Sigma^{\leq n} \tag{3.43}$$

$$\mathrm{Pref}(L^n) = L^{<n}(\mathrm{Pref}(L) \setminus L) \cup L^n \tag{3.44}$$

$$\mathrm{Pref}(L^*) = L^* \mathrm{Pref}(L) = L^+ \cup L^*(\mathrm{Pref}(L) \setminus L) \tag{3.45}$$

$$\mathrm{Pref}(L^*) = L^*(\mathrm{Pref}(L) \setminus L) \text{ if } \varepsilon \notin L \tag{3.46}$$

$$\Sigma^*v\Sigma^* \cap L \neq \emptyset \Leftrightarrow \Sigma^*v \cap \mathrm{Pref}(L) \neq \emptyset \tag{3.47}$$

Note that, if $\varepsilon \in L$, then $\varepsilon \notin L^*(\mathrm{Pref}(L) \setminus L)$, so that Equation (3.46) does not hold in this case. For a concrete example let $L = \{\varepsilon, aa\}$. Then $\mathrm{Pref}(L^*) = a^*$, $\mathrm{Pref}(L) \setminus L = \{a\}$ and $L^* = (aa)^*$ so that Equation (3.46) fails.

Let $\Gamma \subseteq \Sigma$. The *projection morphism* $\pi_\Gamma \colon \Sigma^* \to \Gamma^*$ onto Γ^* is given by $\pi_\Gamma(x) = x$ for $x \in \Gamma$ and $\pi_\Gamma(x) = \varepsilon$ for $x \notin \Gamma$. As the name suggests, π_Γ is a monoid morphism.

[11] Thus we obtain functions $\Sigma^* \to \mathcal{P}(\Sigma^*)$. Note that the extension of these functions to subsets would actually give functions $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\mathcal{P}(\Sigma^*))$, which creates some ambiguity with their original definitions as function with signature $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$. Our usage follows the standard convention and when applied to languages, we always mean the functions $\mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$ as originally defined, which should be clear from context so that no confusion arises.

We set $|u|_a = |\pi_{\{a\}}(u)|$ for $u \in \Sigma^*$ and $a \in \Sigma$. Then $\pi_{\{a\}}(u) = a^{|u|_a}$ and $|u|_a$ equals the number of times the letter a occurs in the word u. The *Parikh map* [474] (or *Parikh morphism*) is $\psi\colon \Sigma^* \to \mathbb{N}_0^\Sigma$ where $\psi(u) = (|u|_a)_{a\in\Sigma}$ for $u \in \Sigma^*$. A language $L \subseteq \Sigma^*$ is *commutative* if $\psi^{-1}(\psi(L)) = L$. Equivalently, if $uabv \in L$ implies $ubav \in L$ for all $u, v \in \Sigma^*$ and $a, b \in \Sigma$.

In expressions on languages using the operations concatenation $\cdot$ (as a product on subsets), Kleene star *, Kleene plus $^+$ and the set operations, when writing a single word u we mean to interpret it as the language $\{u\}$ and we omit parentheses according to the following precedence rules: * and $^+$ have the strongest binding, then $\cdot$ and lastly the set operations due to the usual convention where set difference is evaluated before union and intersection. For example, $abc \cup b^*$ denotes $\{abc, \varepsilon, b, b^2, b^3, \ldots\}$. We also omit parenthesis for associative operations such as union and intersection. For cases not covered by these rules, as, for example, combinations of union and intersection, parenthesis are used to resolve any ambiguity.

3.7.2 Regular Expressions

Let Σ be a finite alphabet and set $\hat{\Sigma} = \{\hat{a} \mid a \in \Sigma\} \cup \{\hat{\emptyset}\} \cup \{^*, \cdot, +, (,)\}$. A *regular expression* (also known as a *rational expression*)[12] is a word over $\hat{\Sigma}$ inductively defined in the following way: $\hat{a}$ for $a \in \Sigma$ and $\hat{\emptyset}$ are regular expressions, and if R, S are regular expressions, then $(R \cdot S)$ (or (RS) for short), $(R + T)$, and (R^*) are regular expressions. By $\mathcal{RE}(\Sigma) \subseteq \hat{\Sigma}^*$ we denote the *set of regular expressions*.

Each regular expression defines a language over Σ via the (interpretation or evaluation) function $L\colon \mathcal{RE}(\Sigma) \to \mathcal{P}(\Sigma^*)$ inductively defined by $L(\hat{a}) = \{a\}$, $L(\hat{\emptyset}) = \emptyset$, $L((RS)) = L(R)L(S)$, $L((R + S)) = L(R) \cup L(S)$ and $L((R^*)) = L(R)^*$.

When writing regular expressions, we leave out the hats, i.e., identify $\hat{a}$ with a for $a \in \Sigma$ and $\hat{\emptyset}$ with $\emptyset$, while still having the distinction between formal expressions and their interpretation in mind. Furthermore, we leave out parentheses due to the usual conventions, i.e., we arbitrarily add parentheses to expressions of the form RST and, with the other operations, according to the following operation precedence: * (strongest binding), $\cdot$ and then $+$. For example, $L(abc + ab^*) = L(((a(bc)) + (a(b^*))))$. If there is no danger of confusion, we also omit the L-function and simply write, for example, $abc + ab^*$ for the language $L(abc + ab^*)$.

At the end of Section 3.7.1 we discussed expressions over languages and precedence rules. Every regular expression can be readily translated to an expression over languages, the only difference being rather superficial in the usage of $\cup$ instead of $+$. So, the reader might wonder what we have gained by this extra level of abstraction. The reason is that in computer science we are often concerned with questions of expressibility (what can be described) and succinctness (complexity of descriptions) of some formal system or

[12] Eilenberg [194] suggested the renaming to rational expressions and there are good reasons to use the term rational instead of regular when considering more general monoids than the free monoid or generalizing the theory to formal power series instead of languages. We refer to the literature for more information [194, 195]. Here we are not concerned with these generalizations, and so we prefer the term regular expression which was introduced, historically, before.

description mechanism and in proving results with regard to these two questions we need to be precise and separate the syntax from its semantic. In Section 6.7 we will characterize the class of languages describable by regular expressions using automata.

3.8 Graph Theory

Here, in Section 3.8.1, we introduce undirected and directed graphs, and related notions. Section 3.8.2 investigates different connectedness notions, paths, and cycles. In Section 3.8.3 we introduce the graph of a function and of a homogeneous relation.

3.8.1 Basic Notions

An *undirected graph*, or just *graph*, (resp. a *directed graph*, or *digraph*) is a tuple $G = (V, E)$ where V is the set of *vertices* (or *nodes*) and $E \subseteq \{\{u, v\} \mid u, v \in V\}$ (resp. $E \subseteq V \times V$) the set of *edges*.[13] For an edge $\{u, v\} \in E$ (resp. $(u, v) \in E$) we call u and v its *endpoints*. In case of digraphs, (u, v) is called an *outgoing edge w.r.t. u* (and u is called the *starting/initial/source vertex*) and an *incoming edge w.r.t. v* (and v the *ending/final/terminal vertex*). For finite V and $v \in V$ the *outdegree* (resp. *indegree*) of v is $|\{u \in V : (v, u) \in E\}|$ (resp. $|\{u \in V : (u, v) \in E\}|$).

A *subgraph* of an undirected (resp. directed) graph $G = (V, E)$ is an undirected (resp. directed) graph $G = (V', E')$ such that $V' \subseteq V$ and $E' \subseteq E$. A *(graph) morphism* of two undirected (resp. directed) graphs $G = (V, E)$ and $G' = (V', E')$ is a map $\varphi\colon V \to V'$ such that, for all $u, v \in V$,

$$\{u, v\} \in E \Rightarrow \{\varphi(u), \varphi(v)\} \in E' \quad (\text{resp. } (u, v) \in E \Rightarrow (\varphi(u), \varphi(v)) \in E').$$

A *(graph) automorphism* (for undirected and directed graphs) is a morphism $\varphi\colon V \to V$ of a graph onto itself such that there exists a morphism $\psi : V \to V$ with $\psi\varphi = \varphi\psi = \mathrm{id}_V$. These are precisely those morphisms $\varphi\colon V \to V$ such that

$$\{u, v\} \in E \Leftrightarrow \{\varphi(u), \varphi(v)\} \in E' \quad (\text{resp. } (u, v) \in E \Leftrightarrow (\varphi(u), \varphi(v)) \in E').$$

The set of all automorphisms forms a permutation group, called the *automorphism group* $\mathrm{Aut}(G)$ of the graph.

Automorphisms of finite graphs are precisely the bijective morphisms from V onto V since in this case $\varphi(E) \subseteq E$ and $|\varphi(E)| = |E|$ imply $\varphi(E) = E$. However, this fails for infinite graphs, as shown by the following example.

Example 3.64 $G = (\mathbb{Z}, E)$ with $E = \{\{u, v\} \mid \{n, n+1\}$ and $n > 0\}$. The map $\varphi\colon \mathbb{Z} \to \mathbb{Z}$ with $\varphi(n) = n+1$ for $n \in \mathbb{Z}$ is a morphism, but it is not an automorphism since $\{0, 1\} \notin E$ and $\varphi(\{0, 1\}) = \{1, 2\} \in E$.

[13] Here, $u = v$ (a self-loop) is permitted, which some authors exclude from the definition.

Let $G = (V, E)$ be a digraph. We call G *symmetric* (resp. *oriented*) if $(u, v) \in E$ implies $(v, u) \in E$ (resp. $(v, u) \notin E$) for $u \neq v$.

We often give a graph (resp. digraph) in graphical form by drawing the vertices as circles and the edges as connecting lines (resp. connecting arrows) between them.

Groups are also often denoted by G, but the context should make it clear whether we refer to a graph or a group.

3.8.2 Connectedness Notions

Here, we introduce various notions related to connectedness in graphs. Let $k \geq 1$.

1. A *walk* (of length $n \geq 1$)[14] in an undirected (resp. directed) graph $G = (V, E)$ from $u \in V$ (*start*) to $v \in V$ (*end*) is a sequence of edges $e_1, \ldots, e_n$ with $e_i = \{v_{i-1}, v_i\}$ (resp. $e_i = (v_{i-1}, v_i)$) for all $i \in \{1, \ldots, n\}$ such that $v_0 = u$ and $v_n = v$. We call the walk a *path* (resp. *cycle* (at u)) if $v_0, v_1, \ldots, v_n$ are distinct (resp. $v_0, v_1, \ldots, v_{n-1}$ are distinct and $v_0 = v_n$).[15] Often, when the specific edges involved do not matter, we refer to a walk/path/cycle by its sequence of vertices $v_0, v_1, \ldots, v_n \in V$.[16]
2. A directed or undirected graph G is called *acyclic* if there exists no cycle in G. Equivalently, every walk is a path. For short, a directed acyclic graph is called a *DAG*. An undirected acyclic graph is also called a *forest*.
3. An undirected (resp. directed) graph $G = (V, E)$ is *k-connected* (resp. *strongly k-connected*) if for distinct vertices $u, v \in V$ and every $(k-1)$-set $A \subseteq V$ there exists a path $u_0, u_1, \ldots, u_n$ from u to v such that $\{u_1, \ldots, u_{n-1}\} \cap A = \emptyset$. For $k = 1$ we also call G just *connected* (resp. *strongly connected*).
4. We call the digraph $G = (V, E)$ *strongly $\frac{3}{2}$-connected* if it is strongly connected and for every path $v_0, v_1, \ldots, v_n$ and $0 < i < n$ there exists a path $u_0, u_1, \ldots, u_m$ with $\{u_0, u_m\} = \{v_0, v_n\}$ and $v_i \notin \{u_1, \ldots, u_{m-1}\}$. Equivalently, G is strongly connected and for every three distinct $u, v, w \in V$ there exists a path from u to v or from v to u not traversing w, i.e., we find a path avoiding w.
5. A digraph $G = (V, E)$ is called *weakly k-connected* if the undirected graph with vertex set V and edge set $\{\{u, v\} \mid (u, v) \in E\}$ is k-connected. A weakly 1-connected graph is simply called *weakly connected*.
6. A digraph $G = (V, E)$ is called *unilaterally connected* (or *semiconnected*) if for distinct vertices $u, v \in V$ there exists a path from u to v or from v to u.
7. In an undirected (resp. directed) graph $G = (V, E)$ a *connected component* (resp. *strongly connected component*) is a subset $S \subseteq V$ maximal for subset inclusion such that there exists a path from u to v for distinct $u, v \in S$. A subset of a

[14] There exist no walks of length zero according to our definition.

[15] The terminology varies in the literature: some authors use path where we use walk, and reserve simple path for a walk whose vertices are all distinct.

[16] Paths and cycles can also be identified with graphs and digraphs such that a graph or digraph contains a path if and only if it contains these as subgraphs. However, this does not work for walks since they can have repeated vertices or edges.

digraph $G = (V, E)$ is a *weakly connected component* if it is a connected component of the undirected graph with vertex set V and edge set $\{\{u, v\} \mid (u, v) \in E\}$.

Remark 3.65 The *connectivity* of a finite graph (resp. digraph) with n vertices in which not all vertices are joined by an edge is the minimal number of vertices whose deletion results in a graph (resp. digraph) that is not connected (resp. not strongly connected). This also equals the largest k such that the graph (resp. digraph) is k-connected (resp. strongly k-connected).

If all vertices are joined by an edge, deletion of any number of vertices never destroys connectedness or strong connectedness and it is k-connected or strongly k-connected for all $k \geq 1$. In this case, the connectivity is defined to be $n - 1$.

In a DAG $G = (V, E)$, when putting $u, v \in V$ in relation if there exists a path from u to v, we obtain a partial order. When saying we *topologically sort the DAG* we refer to a topological sorting of the vertices w.r.t. this partial order.

The following classical result was proved by Menger for undirected graphs [451] and extended to digraphs by Dirac [178]. A short proof for digraphs is given in [258].

Theorem 3.66 (Menger-Dirac [178, 451]) *An undirected (resp. directed) graph $G = (V, E)$ is* k-connected *(resp.* strongly k-connected*) if for distinct $u, v \in V$ there exist at least k paths from u to v such that for two such paths $v_0, v_1, \ldots, v_n$ and $u_0, u_1, \ldots, u_m$ with $v_0 = u_0 = u$, $v_n = u_m = v$ we have $\{v_1, \ldots, v_{n-1}\} \cap \{u_1, \ldots, u_{m-1}\} = \emptyset$.*

In the context of the present work, only strong $\frac{3}{2}$-connectivity and (strong) k-connectivity with $k \leq 2$ will be of interest. The implications from Figure 3.4 are easily seen. Figure 3.5 shows that the converse implications do not hold. Furthermore, it also gives an example of a strongly connected digraph that is weakly 2-connected but not strongly $\frac{3}{2}$-connected; thus strong connectivity and weak 2-connectivity do not imply strong $\frac{3}{2}$-connectivity.

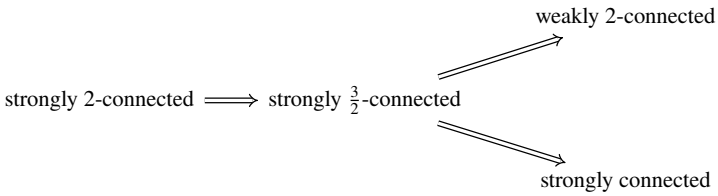
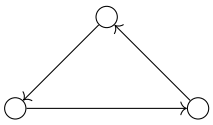


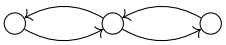
Fig. 3.4: Implications for strong 2-connectivity, strong $\frac{3}{2}$-connectivity, strong and weak connectivity. The converse implications do not hold, see Figure 3.5.

Lastly, we state a characterization of strong connectivity, which was given as a definition in [255, Def. 2].

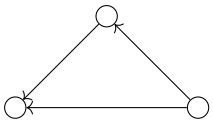
Lemma 3.67 *A digraph $G = (V, E)$ is strongly connected if and only if for every proper and non-empty $B \subsetneq V$ there exists an edge $(u, v) \in E$ such that $u \notin B$ and $v \in B$.*



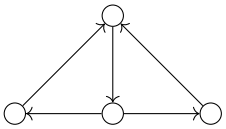
(a) Strongly $\frac{3}{2}$-connected, but not strongly 2-connected.



(b) Strongly connected, but not weakly 2-connected.



(c) Weakly 2-connected, but not strongly connected.



(d) Strongly connected and weakly 2-connected, but not strongly $\frac{3}{2}$-connected.

Fig. 3.5: Some relations among the introduced notions of connectedness.

Proof. First, suppose G is strongly connected. Let $B \subsetneq V$ with $u \in V \setminus B$ and $v \in B$. By assumption there exists a path $v_0, v_1, \ldots, v_n$ with $v_0 = u$ and $v_n = v$ in G. Let $i \in \{1, \ldots, n\}$ be a largest index such that $v_i \notin B$. We have $i < n$ since $v_n \in B$, and $v_{i+1} \in B$ by the maximal choice of i. So, the edge $(v_i, v_{i+1}) \in E$ fulfills the claim.

Conversely, suppose the condition is true. Let $u, v \in V$ be distinct vertices. Set $P = \{w \in V \mid \text{there exists a path from } u \text{ to } w\} \cup \{u\}$. If $V \neq P$, then, by applying the assumption to $B = V \setminus P$, there exists $(u', v') \in E$ with $u' \in P$ and $v' \notin P$. As there exists a path from u to u', by adjoining (u', v') we obtain a path from u to v'. However, as $v' \notin P$ this is a contradiction. So, we must have $P = V$ and there exists a path from u to v. Hence, G is strongly connected. □

Corollary 3.68 *An undirected graph $G = (V, E)$ is connected if and only if for every proper and non-empty $B \subseteq V$ there exists $e \in E$ with $|e \cap (V \setminus B)| = |e \cap B| = 1$.*

Proof. Consider the digraph $(V, \{(u, v), (v, u) \mid \{u, v\} \in E\})$. Since G is connected, it is strongly connected. By Theorem 3.67 it has an edge (u, v) such that $u \notin B$ and $v \in B$. By definition $\{u, v\} \in E$. Then the claim holds for the edge $e = \{u, v\}$. □

3.8.3 Graphs for Transformations and Homogeneous Relations

The *functional graph* of a transformation $t \colon X \to X$ is the digraph $G_t = (X, E)$ with

$$E = \{(x, t(x)) \mid x \in X\}.$$

In G_t, the vertices in $\mathrm{excl}(t)$ are precisely those with indegree zero, and those in $\mathrm{dupl}(t)$ are precisely the vertices with indegree at least two. A digraph is the functional graph of some transformation if and only if every vertex has outdegree one. These graphs are also called maximal directed pseudoforests [233], or sun-structures in [215].

Furthermore, to every homogeneous relation R over X we associate a digraph $G_R = (X, E)$ where $E = \{(a, b) \mid aRb\}$.

Remark 3.69 The cycle points of a transformation are precisely those contained in cycles of the functional graph. The cycle points can be partitioned into points on individual cycles. The tail points are connected to the cycle points in a "tree-like" fashion whose roots are precisely the root points. The edges on the tails point towards the root. See Figure 3.6 where a functional graphs with two cycles and three tails is depicted. The roots are indicated by solid black circles.

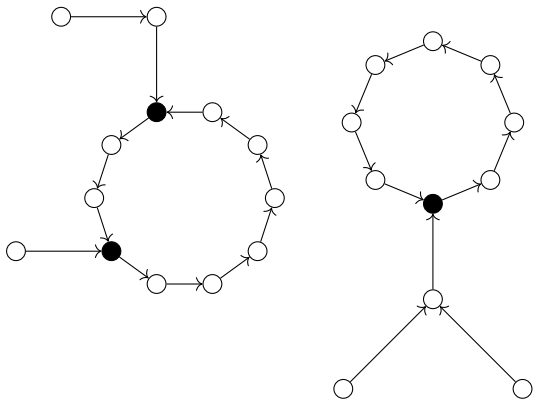


Fig. 3.6: Example of a functionl graph with root points marked, see Theorem 3.69.

Proposition 3.70 *Suppose R is a homogeneous relation over X. Then the weakly connected components of G_R are precisely the equivalence classes of the equivalence relation generated by R.*

Proof. For $a, b \in X$ we set $a \sim b$ if a and b are contained in the same weakly connected component of G_R. Then $\sim$ is an equivalence relation such that, for $a, b \in X$, we have

$$aRb \Rightarrow a \sim b. \tag{3.48}$$

Now, let E be any equivalence relation over X such that aRb implies aEb for all $a, b \in X$. Suppose $a \sim b$ for $a, b \in X$. Then there exist $v_0, v_1, \ldots, v_n \in X$ forming a path from a to b in the undirected graph resulting from G_R by forgetting about the direction of edges, i.e., $v_0 = a$, $v_n = b$ and for each $i \in \{1, \ldots, n\}$ we have $v_{i-1}Rv_i$ or v_iRv_{i-1}. Hence, by assumption and symmetry of E, we find $v_{i-1}Ev_i$ for all $i \in \{1, \ldots, n\}$. By transitivity of E, we get aEb. Thus, $\sim$ refines E.

So, by Equation (3.48) and as $\sim$ refines every equivalence relation E where aRb implies aEb, it equals the equivalence relation generated by R. □

For a more algebraic description of the equivalence relation generated by R, see [325, Prop. 1.4.9 & 1.4.10].

Remark 3.71 In the literature, the set $R_G \subseteq X \times Y$ for a relation $R = (R_G, X, Y)$ as defined in Section 3.4 is often called the graph of the relation. However, since this is not a graph as defined here, we avoid this terminology.

3.9 Computational Complexity Theory

Here, resource usage of computing processes and algorithms is measured in time and in space w.r.t. some model of computation. We introduce three models, the random access machine (RAM) model in Section 3.9.1, classical sequential Turing machines and transducers in Section 3.9.2, and circuits in Section 3.9.3. The RAM model is solely for practical purposes as a model close to existing computers in which to measure the runtime of algorithms. Turing machines and circuits serve a more theoretical purpose. Further cornerstones of our theoretical approach are decision problems, introduced in Section 3.9.4, and reductions and complexity classes, introduced in Section 3.9.5, which give a framework for the classification of problems based on their inherent computational complexity.

When giving time or space upper bounds, to abstract from some implementation specific details, only the behavior up to some constant is of interest. To express this, we use the big O notation [53] (see also [374]). For a function $g\colon \mathbb{N}_0 \to \mathbb{R}$ we set

$$O(g) = \{f\colon \mathbb{N}_0 \to \mathbb{R} \mid \exists c > 0\ \exists n_0 \in \mathbb{N}_0\ \forall n \geq n_0 : |f(n)| \leq c \cdot |g(n)|\},$$
$$o(g) = \{f\colon \mathbb{N}_0 \to \mathbb{R} \mid \forall c > 0\ \exists n_0 \in \mathbb{N}_0\ \forall n \geq n_0 : |f(n)| \leq c \cdot g(n))\}.$$

As is usually done, we identify a function with a defining expression and write, for example, $O(n^2)$ for $O(f)$ with $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ given by $f(n) = n^2$. We also use $O(g)$ and $o(g)$ in equations with the meaning that there exists a function from these sets fulfilling the equation. For example, Theorem 3.1 is equivalent to $p_n = n\log(n)(1 + o(1))$.

3.9.1 The Random Access Machine Model

A *random access machine (RAM)* (or *register machine*) consists of a single generic processor having access to an (idealized) infinitely long memory indexed by natural numbers. Each memory cell is called a *register*. In this model, instructions are executed one after another, with no concurrent operations. The instruction set consists of arithmetic (such as add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), and control operations (conditional branch, goto, subroutine call and return) each taking a constant amount of time. This model is most often used in textbooks on algorithms [163]. Hence, we do not go into more detail about the syntax and semantics of these machines, but refer to the literature [163, 566]. We assume our RAMs operate solely with non-negative integers.

When stating (asymptotic) runtime and space bounds and nothing else is said, we refer to the RAM model.

3.9.2 Turing Machines and Transducers

Here, we introduce Turing machines and transducers in an intuitive manner. For a more formal treatment of Turing machines, and also different but equivalent formalizations, we refer to the literature, for example [322].

A *nondeterministic Turing machine* (just called *Turing machine* for short) consists of a two-way read-only input tape and a two-way read/write work tape. A Turing machine is said to be *deterministic*, if in each configuration at most one configuration can follow in a single computational step.[17] A *(Turing) transducer* is a deterministic Turing machine that always halts and is additionally equipped with a write-only output tape on which the head never moves left. We say that a Turing machine *accepts* an input $u \in \Sigma^*$ if there exists a computational run starting from u that ends in a final state. We say a Turing machine accepts a language $L \subseteq \Sigma^*$ if it accepts precisely the words from L, i.e., for each input $u \in \Sigma^*$, u is accepted if and only if $u \in L$. A Turing machine *decides* a language if it accepts it and every computational run terminates.[18]

A transducer computes a function $f\colon \Sigma^* \to \Sigma^*$ if it produces, when supplied with the input u on the input tape, the value $f(u)$ on its output tape.

Let $f\colon \mathbb{N}_0 \to \mathbb{N}_0$. Following the convention of [322, p. 288], a Turing machine M is said to be $f(n)$-*time bounded* (resp. $f(n)$-*space bounded*) if all computational runs terminate and lie within the prescribed time (resp. space) bounds.[19] Similarly, a Turing transducer is $f(n)$-time (resp. $f(n)$-space) bounded if it takes at most $f(n)$ steps (resp. uses at most $f(n)$ work tape cells) to compute the output for an input of size n. As a shorthand, we also say a Turing machine or transducer is log-space bounded if it is $f(n)$-spaced bounded for some $f \in O(\log)$. This notion transfers to classes of function. For example, we say that a Turing machine is polynomial-time bounded if it is $p(n)$-time bounded for some polynomial function $p(n)$.

By the *linear speed up* and *tape compression* theorems (see [322, Sect. 12.2]) general claim on time or space requirements are only meaningful up to a multiplicative constant, which justifies using the O-notation in these contexts as well.

As a shorthand, for $f = \log$ we also say log-space bounded for $\log(n)$-space bounded machines or transducers, and we say that a machine or transducer is polynomial time/space bounded if it is $p(n)$-time/-space bounded for some polynomial p.

---

[17] This is the standard terminology, but it is somewhat unfortunate, since a deterministic Turing machine is also nondeterministic, i.e., the prefix "non" does not exclude it from being deterministic.

[18] In the literature (see, for example, [41, Sec. 2.1.2]), another convention appears that only requires that, for $u \in L$, at least one computational run terminates and accepts, and that, for $u \notin L$, all computational runs terminate and reject. For the notion of decidability, this makes no difference, since under both conventions a deterministic Turing machine deciding L can be constructed, and for deterministic Turing machines both conventions coincide. Our definition, in which all computational runs must terminate, is also used in Sipser's book [575, Def. 3.6].

[19] Other definitions of resource-bounded computations, and conditions for their equivalence, are discussed in Section 3.12.

3.9.3 Circuits

A *(Boolean) circuit* is a DAG with at least one vertex and the following additional conventions for its vertices:

1. each vertex of outdegree zero has indegree one and is called an *output*,
2. each vertex with indegree zero is called an *input*,
3. all other vertices with non-zero in- and outdegree are called *gates* and are labeled by a Boolean function $f\colon \{0,1\}^n \to \{0,1\}$ such that each input bit of f corresponds to a definite incoming edge (and so its indegree equals the arity n of f).

We also allow the constants 0 or 1 as inputs to input vertices.[20] The indegree (resp. outdegree) of a vertex is also called its *fan-in* (resp. *fan-out*). There is no constraint on the fan-out of non-output vertices. The *size* is the number of vertices in the circuit. The *depth* of the circuit is the length of a longest path.

Consider a circuit with n input and m output vertices, where we have a fixed order on the input and on the output vertices. This circuit computes a vector-valued Boolean function $\{0,1\}^n \to \{0,1\}^m$ by evaluating it for the input $(x_1,\dots,x_n) \in \{0,1\}^n$ in the following way: Assume some topological ordering of the vertices where the inputs appear first in their fixed order. The i-th input vertex is marked with the value of x_i. Then, going forward in the topological order, consider a non-input vertex g where the source vertices of the incoming edges are marked with $z_1,\dots,z_l$, $l \geq 1$. If g is a gate labeled with $f\colon \{0,1\}^l \to \{0,1\}$, where we assume $z_1,\dots,z_l$ are ordered in accordance with the input bits to f, then mark g with $f(z_1,\dots,z_l)$. Otherwise, g is an output vertex. In this case, it is $l = 1$ and we mark g with z_1. Finally, the results of the output vertices $(y_1,\dots,y_m)$ constitute the result of the evaluation of the circuit for the input $(x_1,\dots,x_n)$.

Let $k \geq 0$. We say that a family of circuits $C_1, C_2, \dots$ with strictly increasing number of input bits is in

NC^k if there exists a number $c > 0$ such that each circuit C_i with n input bits has size $O(n^c)$, depth $O(\log^k(n))$, and each gate in C_i is either a NOT gate, an AND gate of fan-in two, or an OR gate of fan-in two,

AC^k if there exists a number $c > 0$ such that each circuit C_i with n input bits has size $O(n^c)$, depth $O(\log^k(n))$, and each gate in C_i is either a NOT gate, an AND gate of unbounded fan-in, or an OR gate of unbounded fan-in.

In the following, we will use the obvious identification of Boolean formulas with circuits and often specify a circuit by only giving a formula (but note that in a circuit, the output of parts might be used several times, so a circuit might be smaller than suggested by a specifying formula).

Let $g, h\colon \{1,2,3,\dots\} \to \{1,2,3,\dots\}$ be such that g is monotone strictly increasing and $g \in O(n^c)$ for some constant c. Let $C_1, C_2, \dots$ be a family of circuits where C_n has $g(n)$ input vertices and $h(n)$ output vertices.

[20] This is also done in [102]. In [619, Def. 1.6] constants are modeled as 0-ary Boolean function with which input vertices can also be labeled.

Following the convention of [159, 160], we say the family of circuits *computes* a function $f\colon D \to \{0,1\}^+$ for $D = \bigcup_{n\geq 1}\{0,1\}^{g(n)}$ if the circuit C_n for $n \geq 1$ computes the function $f_{|\{0,1\}^{g(n)}}\colon \{0,1\}^{g(n)} \to \{0,1\}^{h(n)}$.

Suppose we have an appropriate binary encoding of the words in Σ^* where each word is encoded by a some binary word of length $g(n)$ for some $n \geq 1$. The circuit family *decides* a language $L \subseteq \Sigma^*$ if $h(n) = 1$ for all $n \geq 1$ and for $u \in \Sigma^*$ with binary encoding $x_1 \cdots x_{g(n)}$ we have $u \in L$ if and only if C_n evaluates to 1 when its i-th input vertex is initialized with the i-bit x_i of the binary encoding of u.

In our applications and problems the existence of some encoding is evident. Thus, in the following, to simplify our discussions, we often abstract away from the details of a specific encoding choice and accordingly omit g and h when giving circuit families.

With these definitions, even languages not decidable by Turing machines are decidable by circuits. For example, let $L \subseteq \{a\}^+$ be some unary language not decidable by Turing machines. Then $\{w \in \{0,1\}^+ : |w| \in L\}$ is not decidable by any Turing machine, but it is decidable by the family of circuits $C_1, C_2, \ldots$ in NC^0 [586, p. 142] given by

$$C_n = \begin{cases} x_1 \vee \neg x_1 & \text{if } a^n \in L, \\ x_1 \wedge \neg x_1 & \text{otherwise.} \end{cases}$$

To obtain only circuit families that compute functions, or decide languages, that are also computable, or decidable, by Turing machines, we incorporate a uniform computation mechanism for the construction of individual circuits into our circuit families.

Let $C_1, C_2, \ldots$ be a circuit family where C_n has $g(n)$ input bits. We call this family *L-uniform* [157] if the function $1^{g(n)} \mapsto \overline{C_n}$ w.r.t. some encoding scheme $\overline{C_n}$ for C_n is computable by a deterministic log-space Turing transducer. Thus, the family is L-uniform precisely if a deterministic log-space Turing transducer can produce each circuit of this family when supplied only with the size of the input to the circuit. With this computability condition, we obtain a computationally meaningful notion of decidability of languages and computability of functions by L-uniform circuit families.

Example 3.72 Multiplication of a fixed number of Boolean matrices can be realized by a family of L-uniform AC^0 circuits. Let $M_r = (m_{ij}^{(r)})_{i,j\in\{1,\ldots,n\}}$ for $r \in \{1,\ldots,k\}$ be k input matrices encoded with kn^2 bits. For $i, j \in \{1,\ldots,n\}$ and $r \in \{1,\ldots,k\}$ we define variables $z_{ij}^{(r)}$ by

$$z_{ij}^{(r)} = \begin{cases} m_{ij}^{(k)} & \text{if } r = 1, \\ \bigvee_{l=1}^{n} \left(m_{il}^{(k-r+1)} \wedge z_{lj}^{(r-1)} \right) & \text{if } r \geq 2. \end{cases}$$

Then $\left(z_{ij}^{(k)}\right)_{i,j\in\{1,\ldots,n\}}$ equals $M_1 \cdots M_k$. The formulas directly yield a circuit of depth $2(k-1)$ using $n^3 \cdot (k-1)$ AND gates of fan-in two and $n^2 \cdot (k-1)$ OR gates of fan-in n. We can encode the circuit (with the gates listed topologically sorted) by first listing n^2 OR gates with n inputs, each taking the result of n AND gates for $z_{ij}^{(r)} = \bigvee_{l=1}^{n} \left(m_{lj}^{(k-r+1)} \wedge z_{il}^{(r-1)} \right)$, $r = k$, and continue with this encoding for r going

from k to 2. The encoding can be generated in deterministic log-space by using counters to ensure we generate the correct number of gates with the corresponding inputs.

Remark 3.73 It is natural to ask what share of computational power of our L-uniform circuit families is due to the uniformity mechanism and what is genuinely due to circuitry. Next, we give a class of functions for which most of the computation is carried out by the log-space machine constructing the circuits and the circuitry itself is not very complex.

It is known that transformations computable by L-uniform NC^1 circuits are computable by deterministic log-space Turing transducers [102].

Next, let $f\colon \Sigma^+ \to \Sigma^+$ be such that, for some $\Gamma \subseteq \Sigma$, we have

1. $f(u) = f(v)$ for all $u, v \in \Gamma^*$ with $|u| = |v|$,
2. $f(u) = f(v)$ for all $u, v \in \Sigma^* \setminus \Gamma^*$ with $|u| = |v|$, and
3. for $n \geq 1$ we have $f(\Sigma^n) \subseteq \Sigma^m$ for some $m \geq 1$.

Suppose f is computable by a deterministic log-space Turing transducer M. We show that it can also be computed by a L-uniform AC^0 circuit family. First, we assume Γ is a proper and non-empty subset of Σ. Suppose the words from Σ are encoded by bitstrings of fixed length $r \geq 1$.

Next, we describe a L-uniform circuit family C_n in AC^0 computing f. We can assume the input has size n. Let M' be a Turing transducer operating in the following way for the input 1^n: First compute $f(a^n)$ for some $a \in \Gamma$ and $f(b^n)$ for some $b \in \Sigma \setminus \Gamma$ by simulating M. Let $y_1 \cdots y_m$ and $z_1 \cdots z_m$ be the bits of the encoding of $f(a^n)$ and $f(b^n)$, respectively. Then let M' output the description of the following circuit:

1. There exists a fixed NC^0 circuit C detecting the encoding of a word from $\Sigma \setminus \Gamma$. We can assume the input length n is a multiple of r. Consider the circuit that results from n/r copies of C where the first copy reads the first r bits, the second the next r bits and so on until the last one reads the last r bits. Then connect the outputs of these circuits to a common OR-gate and let x be the output of the resulting circuit. We have $x = 1$ if and only if the input contains a letter from $\Sigma \setminus \Gamma$.
2. Let x_1 be the first input bit to C_n. The i-th output bit of C_n equals the following formulas, from which a circuit can be derived:

$$\begin{cases} x_1 \vee \neg x_1 & \text{if } y_i = 1 \text{ and } z_i = 1, \\ x_1 \wedge \neg x_1 & \text{if } y_i = 0 \text{ and } z_i = 0, \\ x & \text{if } y_i = 0 \text{ and } z_i = 1, \\ \neg x & \text{if } y_i = 1 \text{ and } z_i = 0. \end{cases}$$

For $\Gamma = \Sigma$ or $\Gamma = \emptyset$, we do not need the circuitry to detect letters from Γ or its complement. Thus, in this case, the value of f only depends on the length of the input and we can even give a L-uniform NC^0 family computing f.

The next claims follows from the above construction:

1. Let $L \subseteq \{0,1\}^+$ be in L and suppose for $u, v \in \{0,1\}^+$ with $|u| = |v|$ we have $u \in L$ if and only if $v \in L$, i.e., membership depends only on the length of a word. Then L is also decidable by a L-uniform family of circuits $C_1, C_2, \ldots$ in NC^0

2. Let $L \subseteq 1^+$ over $\Sigma = \{0, 1\}$ be L. Then L is decidable by a L-uniform circuit family $C_1, C_2, \ldots$ in AC^0.
3. Since the binary encoding of $|u|$ from the input u can be computed by a deterministic log-space Turing transducer, it can also be computed by a L-uniform circuit family in NC^0.

Proposition 3.74 *The composition of two L-uniform AC^0 circuit families is computable by a L-uniform AC^0 circuit family.*

Proof. Let M and M' be deterministic log-space Turing transducers computing AC^0-circuits. Let M'' be a deterministic Turing transducer that parses the description of a circuit and outputs 1^m where m denotes the number of output vertices of the circuit. For an appropriate description, M'' can operate in deterministic log-space. Since the description that M produces can be assumed to have size polynomial in the input to M, the machine M'' runs in log-space w.r.t. the input to M. Next, M' can construct a circuit with input 1^m. Since m is polynomial in the input to M, M' runs in log-space w.r.t. the input of M. As shown in [322, p. 323], the composition of deterministic log-space Turing transducers can be realized by a deterministic log-space Turing transducer. Thus the composition of M, M'' and M' can be realized by a deterministic log-space Turing transducer. Thus, we can compute the above circuits in deterministic log-space from the input to M. Then connecting the output of the first circuit to the inputs of the second circuit can also be realized in deterministic log-space. This yields that all these steps can be realized by a single deterministic log-space Turing transducer that outputs a description of the circuit resulting by feeding the second circuit with the input of the first, which is a simple modification to the outputs of M and of the composition of M, M'' and M'. □

3.9.4 Decision Problems

A *decision problem* classifies a set of inputs, encoded as a language (usually over $\{0, 1\}$), into YES- and NO-instances. In Sections 3.9.2 and 3.9.3 we already defined what it means for a Turing machine or a family of circuits to decide a language. More generally, a computational mechanism (e.g., a Turing machine or a family of uniform circuits) *decides/solves/computes* a decision problem if for each input it terminates after a finite number of steps and returns YES (accepts the input) if the input encodes a YES-instance or NO (rejects the input) if it encodes a NO-instance. To every decision problem there corresponds a language encoding the YES-instances. Conversely, to every language $L \subseteq \Sigma^*$ there corresponds a decision problem (called the *membership problem*[21] *of* L) where the YES-instances are precisely those words $w \in \Sigma^*$ for which $w \in L$, and the

[21] In the literature, a more refined terminology is used, where a distinction is drawn between the *non-uniform/fixed* and the *uniform/general membership problem* for classes of languages [321]. In the former problem the language is fixed and the input is a single word, and in the latter problem a pair consisting of a language (from some class) *and* a word forms the input. In this sense, the membership problem for L as defined here is the non-uniform membership problem.

NO-instance those with $w \notin L$. So, languages realize and encode decision problems and often we identify a language with its decision problem and vice versa without special mention.

We give a decision problem by listing its name, the possible inputs, and the property we want to check on the input, where we use a notational way introduced by Karp [361] and popularized by Garey and Johnson [238]. In each case, it is obvious that an appropriate encoding as a language is possible. In proofs, a specific input is written as a tuple listing all the parameters and called an *input instance* to the (decision) problem under consideration.

Next, we list decision problems from the literature that are of relevance to the present study.

Of importance is the satisfiability problem SAT for a set $X = \{x_1, \ldots, x_n\}$ of *Boolean variables* that we formalize in the following way as in [257, p. 314]: a *literal* is an element of $X \times \{0, 1\}$, a subset of $X \times \{0, 1\}$ is a *clause* over X and an *assignment* (of values to the variables) is a function $\alpha\colon X \to \{0, 1\}$ (also written as an n-tuple $(\alpha(x_1), \ldots, \alpha(x_n))$). We say that α *satisfies* a set of clauses C if for each clause $c \in C$ there exists $x \in V$ such that $(x, \alpha(x)) \in c$. Intuitively, see also [238, 322, Sec. 2.6], literals can be seen as variables or their negations, and we often write x and $\neg x$ (or $\overline{x}$) for $(x, 1)$ and $(x, 0)$, respectively, a clause as a disjunction of literals and a set of clauses as a conjunction of clauses. Then an assignment satisfies a set of clauses if and only if if this formula evaluates to true for the assignment. For example, the formula $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$ is represented by the set of clauses $\{c_1, c_2\}$ where $c_1 = \{(x_1, 1), (x_2, 1)\}$ and $c_2 = \{(x_2, 0), (x_3, 1)\}$.

SAT (Cook [156])
Input: A set X of Boolean variables and a set C clauses over X.
Question: Does there exist a satisfying assignment for the input?

DAG (Jones [345], *Directed Acyclic Graph Accessibility Problem*)
Input: DAG $G = (V, E)$ and $s, t \in V$.
Question: Does there exist a path in G from s (start) to t (target)?

StronglyConn (Jones [345, Cor. 24])
Input: Digraph $G = (V, E)$.
Question: Is G strongly connected?

SingleCycle (Cook and McKenzie [161])
Input: Permutation presented pointwise.
Question: Does the permutation consist of a single cycle,[22] i.e., cyclically permutes its domain?

[22] Note that having only a single cycle is not the same as being a cyclic permutation. A cyclic permutation has at most one cycle of length greater than one, but might have many cycles of length one; we introduced the term cyclically permuting its domain in Section 4.2 to express the fact that the permutation has a single cycle.

OrderBetweenVertices (Etessami [203, Def. 4.1])
Input: Digraph $G = (V, E)$ where $V = \{v_1, \ldots, v_n\}$ and $E = \{(v_i, v_{i+1}) \mid i \in \{1, \ldots, n-1\}\}$ (encoded as a usual graph, without additional information pertaining to the assumed total order of the vertices)[23] and two vertices $v_i, v_j \in V$.
Question: Is $i < j$?

We also need the following variants of the above problems. 3SAT is a well-known variant of SAT [238, 361]. Let $k \geq 1$.

3SAT (Karp [361])
Input: A set X of Boolean variables and a set C of clauses over X each containing at most three literals.
Question: Does there exist a satisfying assignment for the input?

k-DAG
Input: DAG $G = (V, E)$ where the outdegree of every vertex is at most k and $s, t \in V$.
Question: Does there exist a path in G from s to t?

k-DAG-ST
Input: DAG $G = (V, E)$ and $s, t \in V$ where s has no incoming edges and precisely one outgoing edge, t has no outgoing edges and the outdegree of every other vertex is at most k.
Question: Does there exist a path in G from s to t?

3.9.5 Complexity Classes and Reductions

A *complexity class* is a class of decision problems, encoded by languages, which can be decided by some computational mechanism with imposed resource constraints. We will consider the following complexity classes (we list the resource bounded computational mechanism next to each class).

L-uniform NC^k	L-uniform circuit families in NC^k for $k \geq 0$.
L-uniform AC^k	L-uniform circuit families in AC^k for $k \geq 0$.
L	Log-space bounded deterministic Turing machines.
NL	Log-space bounded nondeterministic Turing machines.
NC	$\mathsf{NC} = \bigcup_{k \geq 0}$ L-uniform NC^k (Nick's class [160]).
P	Polynomial-time bounded deterministic Turing machines.
NP	Polynomial-time bounded nondeterministic Turing machines.
PSPACE	Polynomial-space bounded deterministic Turing machines.
NPSPACE	Polynomial-space bounded nondeterministic Turing machines.

[23] For example, encoding v_i by the number i would render the problem solvable in AC^0 as we simply have to compare two positive integers in this case. Equivalently, we can think of the input as a set together with a successor relation.

In the following, we always understand that NC^k (resp. AC^k) means L-uniform NC^k (resp. AC^k) and omit this modifier.

P can equivalently be defined by polynomial time bounded RAMs, as we can simulate a RAM with a Turing machine and vice versa with a polynomial overhead [162]. So, polynomial time bounds for RAMs imply containment in P.

Intuitively, for L and NL, a machine can, on inputs of size n, use a finite number of counters storing binary encoded values that do not exceed $O(n^c)$ for some fixed $c > 0$ and memory cells with a logarithmic number of bits to store intermediate results.

By Theorem 3.72, the decision problem to decide whether the product of k given matrices equals a given matrix is in AC^0.

Let $k \geq 1$. Most of the following inclusions are well-known (see for example [56, 57, 102, 322, 575, 619]):

$$\begin{aligned}\mathsf{AC}^0 \subseteq \mathsf{NC}^1 \subseteq \mathsf{L} \subseteq \mathsf{NL} \subseteq \mathsf{AC}^k \subseteq \mathsf{NC}^{k+1} \subseteq \mathsf{AC}^{k+1} \\ \subseteq \mathsf{NC} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} = \mathsf{NPSPACE}. \quad (3.49)\end{aligned}$$

The equality $\mathsf{PSPACE} = \mathsf{NPSPACE}$ follows from Savitch's theorem [550]. By the above inclusions we have $\mathsf{NC} = \bigcup_{k\geq 0} \mathsf{AC}^k$.

Next, we introduce several notions of reduction. An *oracle gate* for a language $L \subseteq \Sigma^*$ in a circuit is a gate that returns one if and only if the input encodes a word in L. Let $L, L' \subseteq \Sigma^*$.

1. We say L is *log-space* (resp. *polynomial-time,* L*-uniform* NC^1*,* L*-uniform* AC^0*) many-one reducible* to L', denoted by $L \leq_m^{\log} L'$ (resp. $L \leq_m^{\mathsf{P}} L'$, $L \leq_m^{\mathsf{NC}^1} L'$, $L \leq_m^{\mathsf{AC}^0} L'$), if there exists a log-space Turing transducer (resp. polynomial-time bounded Turing transducer, L-uniform circuit family in NC^1, L-uniform circuit family in AC^0) that, given input $u \in \Sigma^*$, produces an output $v \in \Sigma^*$ such that $v \in L'$ if and only if $u \in L$. For short, we call such a mapping a $\leq_m^{\log}$- (resp. $\leq_m^{\mathsf{P}}$-, $\leq_m^{\mathsf{NC}^1}$-, $\leq_m^{\mathsf{AC}^0}$-)reduction.
2. We say L is L*-uniform* NC^1 *oracle* (or *Turing*) *reducible* to L', denoted by $L \leq_T^{\mathsf{NC}^1} L'$, if there exists an L-uniform circuit family in NC^1 deciding L that can additionally use oracle gates for L' . We call such a family an $\leq_T^{\mathsf{NC}^1}$-reduction.

If we have $L \leq_m^{\log} L'$ and $L' \leq_m^{\log} L$, then we write $L \equiv_m^{\log} L'$ and call both decision problems encoded by these languages $\equiv_m^{\log}$*-equivalent*.

The language L is *hard* for a complexity class w.r.t. $\leq_m^{\log}$-reductions if for every language L'' from the complexity class we have $L'' \leq_m^{\log} L$. It is *complete* for a complexity class w.r.t. $\leq_m^{\log}$-reductions if it is hard w.r.t. $\leq_m^{\log}$-reductions and contained in the complexity class. The same definitions apply w.r.t. $\leq_m^{\mathsf{P}}$-, $\leq_m^{\mathsf{AC}^0}$-, $\leq_m^{\mathsf{NC}^1}$- and $\leq_T^{\mathsf{NC}^1}$-reductions.

Theorem 3.75 *1.* $\leq_m^{\mathsf{AC}^0}$, $\leq_m^{\log}$, $\leq_m^{\mathsf{P}}$, $\leq_T^{\mathsf{NC}^1}$, *and* $\leq_m^{\mathsf{NC}^1}$ *are quasiorders on languages.*

2. Let $\mathcal{X}$ *be any of the complexity classes* L, NL, P, NP, *or* PSPACE *and* L' *be a language in* $\mathcal{X}$*. Suppose* L *is* $\leq_m^{\mathsf{AC}^0}$-/$\leq_m^{\log}$-/$\leq_T^{\mathsf{NC}^1}$-/$\leq_m^{\mathsf{NC}^1}$*-reducible to* L'*. Then* L *is*

also in $\mathcal{X}$. For $\leq_m^{\mathsf{P}}$ this hold for $\mathcal{X}$ equal to P, NP, *or* PSPACE, *and for $\leq_m^{\mathsf{AC}^0}$-reductions this holds for all complexity classes introduced.*

Proof. The first claim is shown in [619, Lem. 4.81] for $\leq_m^{\log}$, $\leq_T^{\mathsf{NC}^1}$, and $\leq_m^{\mathsf{NC}^1}$, and in [322, p. 322 & 323] for $\leq_m^{\log}$ and $\leq_m^{\mathsf{P}}$. For $\leq_m^{\mathsf{AC}^0}$, the first claim follows from Theorem 3.74.

The second claim follows from the first claim and the inclusions given in Equation (3.49). □

By Theorem 3.75, a complexity class equals an equivalence class for the induced equivalence relation w.r.t. some notion of reduction, represented via complete problems.

The following statement collects complexity-theoretical results from the literature for the problems introduced in Section 3.9.4.

Theorem 3.76 *1. The problems* SAT *(Cook [156])*[24] *and* 3SAT *[361] are* NP*-complete for $\leq_m^{\log}$-reductions.*

2. DAG *(Jones [345, Cor. 25]) and* StronglyConn *([345, Cor. 24]) are* NL*-complete for $\leq_m^{\log}$-reductions.*[25]
3. SingleCycle *(Cook and McKenzie [161]),* OrderBetweenVertices *(Etessami [203, Thm. 4.1]),*[26] *and* 1-DAG *(Cook [158, p. 110] and Cook and McKenzie [161, p. 390]) are* L*-complete for $\leq_T^{\mathsf{NC}^1}$-reductions.*[27]

For the variants of k-DAG, we have the following w.r.t. $\leq_m^{\mathsf{AC}^0}$-reductions.

Proposition 3.77 *Let $k \geq 1$ be fixed. Then we have k-DAG $\equiv_m^{\mathsf{AC}^0}$ k-DAG-ST. Furthermore, for $k \geq 2$, both problems are also $\equiv_m^{\mathsf{AC}^0}$-equivalent to* DAG.

Proof. Clearly, k-DAG-ST $\leq_m^{\mathsf{AC}^0}$ k-DAG $\leq_m^{\mathsf{AC}^0}$ DAG.

Next, we show k-DAG $\leq_m^{\mathsf{AC}^0}$ k-DAG-ST. Suppose (G, s, t) with $G = (V, E)$ is an instance of k-DAG and let $s' \notin V$. Set

$$G' = (V \cup \{s'\}, (E \setminus (\{t\} \times V)) \cup \{(s', s)\}).$$

Then G' is a valid input for k-DAG-ST that admits a path from s' to t if and only if G admits a path from s to t.

Lastly, we prove DAG $\leq_m^{\mathsf{AC}^0}$ k-DAG for $k \geq 2$, which establishes all claims. Intuitively, given an arbitrary DAG with outdegree at most l, we use k-ary branching together with auxiliary vertices to simulate the edges of the input DAG in a DAG whose outdegree is at most k.

---

[24] Originally, Cook [156] used reductions realized by deterministic polynomial-time oracle Turing machines. Later, Jones [345] and Meyer & Stockmeyer [584] introduced $\leq_m^{\log}$-reductions and noted that the reductions given by Cook in [156] can be realized to give $\leq_m^{\log}$-reductions.

[25] In [345] NL-completeness is even shown for a finer notion of rudimentary reductions.

[26] Actually, for OrderBetweenVertices completeness was established for a weak form of first order reducibility that implies AC^0-reducibility.

[27] L-completeness of 1-DAG for other notions of reduction was shown in [282, Thm. 2.1] and [345, Cor. 25].

Let $G = (V, E)$ be some input DAG and $s, t \in V$. Suppose l is the largest outdegree of some node in G. Let $m \geq 1$ be the smallest integer such that $k^m \geq l$ and let $\Sigma = \{a_1, \ldots, a_k\}$ be an alphabet.

In the following, for $v \in V$ and $x \in \mathrm{Pref}(\Sigma^{m-1})$ we write v_x for (v, x). Let $v \in V$. Set $T_v = \{u_\varepsilon \mid (v, u) \in E\}$. Since $k^m \geq l$, there exists a surjection $f_v \colon \Sigma^m \to T_v$. Let

$$E_v = \{(v_x, v_{xy}) \mid x \in \Sigma^{<m-1} \wedge y \in \Sigma\} \cup \{(v_x, f(xy)) \mid x \in \Sigma^{m-1} \wedge y \in \Sigma\}.$$

Then $G' = (V \times \mathrm{Pref}(\Sigma^{m-1}), \bigcup_{v \in V} E_v)$ gives a DAG where every vertex has outdegree at most k and there exists a path from s to t in G if and only if there exists a path from s_ε to t_ε in G'. □

Corollary 3.78 *Let $k \geq 2$. Then k-DAG and k-DAG-ST are* NL*-complete.*

Proof. This follows from Theorems 3.76 and 3.77. □

For reference, we state the following folklore fact.

Proposition 3.79 *For two numbers given in unary encoding, their greatest common divisor can be computed in deterministic log-space. As a consequence, testing coprimality of two unary numbers is in* L*.*

Proof. We use the classical Euclidean algorithm that stores two variables.

Let a, b be given in unary, so the input length is $N = a + b + 1$ (counting a separator). Maintain two variables x, y, initialized to a and b and stored in binary. Repeatedly test whether $y = 0$ and, while $y \neq 0$, replace the pair (x, y) by $(y,\ x \bmod y)$, i.e., overwrite x with the old value of y and overwrite y with the remainder of x divided by y. Comparison against 0 and division with remainder of two $O(\log N)$-bit numbers run in $O(\log N)$ space (indeed in L-uniform TC^0 [147, 289, 290]).[28] The loop needs only the registers x, y, one temporary of $O(\log N)$ bits to form the remainder, plus $O(1)$ space for control. After every two iterations the larger operand drops by at least a factor of 2, so the loop makes $O(\log N)$ steps. When $y = 0$, the remaining value x is $\gcd(a, b)$. For coprimality, accept precisely if this value is 1.

Throughout this procedure, we have $0 \leq x, y \leq \max(a, b) \leq N$, since each remainder $x \bmod y$ is strictly smaller than the current y. So each variable occupies at most $\lceil \log_2(N + 1) \rceil \in O(\log N)$ bits, and the total work space is $O(\log N)$. □

For two binary-encoded integers, we can compute their greatest common divisor in polynomial time via the Euclidean algorithm as given in the proof of Theorem 3.79. However, the precise computational complexity for binary-encoded inputs is unknown. This problem is not known to be either in NC or P-complete [578]. For testing coprimality, the preprint [415, Thm. 1] claimed this problem to be in NC, but a fatal flaw was spotted in its proof later [571]. and the precise complexity of this problem remains open [555].

We note that finding L-uniform circuit families computing the division of two integers was an open problem for quite some time, see the survey [7].

Lastly, we show that testing strong $\frac{3}{2}$-connectivity of digraphs is NL-complete.

[28] TC^0 refers to circuits of constant depth and polynomial size that use only NOT gates and majority, AND, and OR gates of unbounded fan-in.

Proposition 3.80 *Deciding whether a given digraph is strongly $\frac{3}{2}$-connected is* NL*-complete.*

Proof. The problem is in NL, as for a digraph $G = (V, E)$ and $u, v, w \in V$, we can first guess whether a path should go from u to v or from v to u and then, successively and vertex-by-vertex, guess a path (of length at most $|V|$) from the start to the end vertex not using the vertex w. If an appropriate path is found, we have a YES-instance; otherwise a NO-instance.

Next, we give a reduction from StronglyConn, which is NL-complete by Theorem 3.76. Let $G = (V, E)$ be a given digraph as input. Set $G' = (V', E')$ with $V' = V \times \{0, 1\}$ and edge set E' equal to

$$\{((u, i), (v, i)) \mid (u, v) \in E, i \in \{0, 1\}\} \cup \{((u, 0), (u, 1)), ((u, 1), (u, 0)) \mid u \in V\}.$$

Next, we show that the following properties are equivalent, where the equivalence between 1 and 3 implies that we obtain a valid reduction:

1. G is strongly connected,
2. G' is strongly 2-connected,
3. G' is strongly $\frac{3}{2}$-connected,
4. G' is strongly connected.

For the proof, suppose G is strongly connected and let $(u, i), (v, j), (w, k) \in V'$. Without loss of generality, suppose $j = 1$. There exists a path $v_0, \ldots, v_n \in V$ from u to v in G. Then, depending on the values of i and k, we obtain a path from (u, i) to (v, j) not traversing (w, k) in the following way:

$$\begin{cases} (v_0, 0), (v_0, 1), (v_1, 1), \ldots, (v_n, 1) & \text{if } i = 0, k = 0, \\ (v_0, 1), (v_1, 1), \ldots, (v_n, 1) & \text{if } i = 1, k = 0, \\ (v_0, 0), (v_1, 0), \ldots, (v_n, 0), (v, 1) & \text{if } i = 0, k = 1, \\ (v_0, 1), (v_0, 0), (v_1, 0), \ldots, (v_n, 0), (v, 1) & \text{if } i = 1, k = 1. \end{cases}$$

So, we obtain that G' is strongly 2-connected. This implies strong $\frac{3}{2}$-connectivity, which implies strong connectivity, see Figure 3.4. Finally, suppose G' is strongly connected and let $u, v \in V$. Then, let $(v_0, 0), (v_1, i_1), \ldots, (v_{n-1}, i_{n-1}), (v_n, 0)$ be a path in G' with $i_1, \ldots, i_{n-1} \in \{0, 1\}$, $v_0 = u$, and $v_n = v$. We obtain a path from u to v in G by considering the vertices $v_1, v_1, \ldots, v_{n-1}, v_n$ and replacing parts of the form $v_i, v_{i+1}, \ldots, v_j$, $i < j$, with $v_i = v_j$ by v_i (where the order in which these parts are selected does not matter) until every vertex appears precisely once.[29] □

[29] More precisely, for each $i \in \{0, 1\}$, there exists a path from u to v in G if and only if there exists a path from (u, i) to (v, i) in G' using only vertices in $V \times \{i\}$.

3.9.6 Circuits and Encodings

Here, we discuss how we encode numbers, transformations, and, lastly, automata as inputs for circuits.

3.9.6.1 Numbers

Besides the binary encoding of a number here we also consider two types of unary encodings. In the *one-hot* (resp. *thermometer*) *unary encoding* [452] using n bits a number $0 \leq m < n$ is encoded by $0^m 10^{n-m-1}$ (resp. $1^{m+1}0^{n-m-1}$). In both representations, the value of the number increases as the rightmost 1 (when read from left to right) shifts further to the right. As a technicality, note that in both encodings, zero is represented by 10^{n-1} and not by 0^n.

Proposition 3.81 *1. There exists an L-uniform circuit family $C_1, C_2, \ldots$ in AC^0 where C_n converts a number in one-hot unary encoding using n bits to the binary encoding with $k = \lfloor \log_2(n) \rfloor + 1$ bits.*

2. There exists an L-uniform circuit family $C_1, C_2, \ldots$ in AC^0 where C_n converts the thermometer unary encoding using n bits to the one-hot unary encoding using n bits and vice versa.

3. There exists an AC^0 circuit of size $O(2^n)$ to convert an n-bit binary number to a one-hot unary encoding using 2^n bits.

Proof. 1: Let $x_0 x_1 \cdots x_n$ be the one-hot encoding of a number x. Then, for $i \in \{0, 1, \ldots, n\}$, we have $x_i = 1$ if and only if i equals x. Let $y_m \cdots y_1$ be the binary encoding of x with $m = \lfloor \log_2(n) \rfloor + 1$ and where the least significant bit is the rightmost bit y_1. Then, for $j \in \{1, \ldots, m\}$, we have $y_j = 1$ if and only if

$$k2^j + 2^{j-1} \leq x < (k+1)2^j$$

for some $k \geq 0$. Thus, for $j \in \{1, \ldots, m\}$, we obtain

$$y_j = \bigvee_{\exists k \colon k2^j + 2^{j-1} \leq i < (k+1)2^j} x_i,$$

i.e., y_j is the OR over all x_i with $i \in \{0, 1, \ldots, n\}$ where the j-th bit of i is one. These formulas can be translated into AC^0 circuits that are computable by deterministic log-space Turing transducers. [30]

2: The one-hot encoding $x_0, x_1, \ldots, x_{n-1}$ and the thermometer encoding $y_0, y_1, \ldots, y_{n-1}$ of a number are related by $y_i = x_i \vee \cdots \vee x_{n-1}$ and $x_i = y_i \wedge \neg y_{i+1}$ where we set $y_n = 0$ for $i \in \{0, \ldots, n-1\}$. These formulas can be implemented by L-uniform AC^0 circuit families.

[30] This is called an n-to-k binary encoder with $k = \lfloor \log_2 n \rfloor + 1$. A different circuit commonly used for this conversion is the priority encoder described in [375, p. 117], which also allows "don't-cares".

3: Suppose $x_{n-1}, \ldots, x_1, x_0$ are the bits encoding a binary number with x_0 being the least significant bit. Let $y_0, y_1, \ldots, y_{2^n-1}$ be the bits for the output in that order. Then, for $i \in \{0, 1, \ldots, 2^n - 1\}$, we have $y_i = z_{i,n} \wedge \cdots \wedge z_{i,1}$ where, for $j \in \{1, \ldots, n\}$,

$$z_{i,j} = \begin{cases} x_{j-1} & \text{if the } j\text{-th bit of the binary encoding of } i \text{ is one,} \\ \neg x_{j-1} & \text{if the } j\text{-th bit of the binary encoding of } i \text{ is zero.} \end{cases}$$

Intuitively, the bit pattern $x_{n-1}, \ldots, x_1, x_0$ is matched against the binary encoding of i via a minterm.[31] We have

$$y_i = 1 \Leftrightarrow \text{the bit pattern } x_{n-1}, \ldots, x_1, x_0 \text{ encodes } i.$$

Now, each y_i uses a single AND gate of fan-in n and at most n NOT gates. So, this is an AC^0 circuit of size $O(2^n)$. □

By Theorem 3.74 and Item 2 from Theorem 3.81 for the unary encoding we can chose the one-hot or thermometer encoding, whatever is more convenient, up to AC^0-reductions.

The input to our problems is often some numerical parameter together with other data. An additional parameter can be encoded in the input along with other data in the following ways.

1. Most of the time the parameter can be assumed to be bounded by some monotone function $f(n)$ of the size of the data n. For example, if the parameter bounds a shortest synchronizing word of an input automaton, then it can be assumed to be bounded by a cubic polynomial, see Table 1.1. In this case, the total length of the input is the sum of n and $f(n)$ (or $\log_2(f(n))$ for a binary encoding) and n can be extracted from the input length. Thus, the parameter is deducible from the total length of the input. If this is possible in deterministic log-space, then for our L-uniform circuit families we can assume the input is split into designated bits for the parameter and for the rest of the data.
2. If the parameter can be arbitrarily large even if the remaining data is fixed, then the parameter cannot be encoded in the length of the input. In this case, it has to be encoded into the input in such a way that it can be separated from the remaining data. For example, by encoding the total input in the form $1^m 0 v$ where 1^m encodes the parameter and v the rest of the input. Then the circuits have to use additional circuitry to parse the input and separate the parameter from the remaining data.

In reductions, when we assume a unary encoding for a parameter we assume it to have the form 1^m for some $m \geq 0$.[32]

---

[31] This is a conjunction of variables and negated variables that evaluates to one for precisely one specific assignment to the variables.

[32] However, when the input is a single unary number without additional data, this assumption makes no sense since then we always just feed 1^n as input to the circuits. This is precisely the input the Turing machine constructing the circuit sees and thus for NC^1 and below, the computation can already be performed by the Turing machine and the circuitry does not give more computational power. So, in this case, the number of bits and the specific value encoded in the input bitstring must be distinguished.

3.9.6.2 Transformations

A transformation $t\colon [n] \to [n]$ can be represented by a transformation matrix consisting of n^2 binary values or by the *point-wise representation*, which lists the values $t(0), t(1), \ldots, t(n-1)$ where we assume each $t(i)$ uses the same number of bits. With this convention for the point-wise representation, the parameter n can be easily extracted solely from the length of the input, which has size n^2 if we use a unary encoding for each $t(i)$ or $n\lfloor \log_2(n) + 1 \rfloor$ if we use a binary encoding.

For the transformation $t\colon [n] \to [n]$, for $i \in [n]$, $t(i)$ in one-hot encoding equals the i-th row of the transition matrix of t w.r.t. the natural order on $[n]$. Thus, both representations are essentially the same, and we can also easily obtain the point-wise representation in terms of the thermometer unary encoding from this representation.

Next, we show that we can also convert the point-wise representation using a binary encoding and the representation by transition matrices into each other by L-uniform AC^0-circuits.

Theorem 3.82 *There exists a family of L-uniform AC^0 circuits computing the conversion of the point-wise representation of a transformation with a binary encoding into the representation by transition matrices and vice versa.*

Proof. Let $t\colon [n] \to [n]$. We can assume $n \geq 1$. First, suppose t is given by a point-wise representation $t(0), \ldots, t(n-1)$ where $t(i)$ for $i \in [n]$ is encoded in binary. By our convention, each $t(i)$ for $i \in [n]$ uses $\lfloor \log_2(n) + 1 \rfloor$ bits, so that the encoding uses $n\lfloor \log_2(n) + 1 \rfloor$ bits in total.

The AC^0 circuits for the conversion of the binary encoding of $t(i)$ to a one-hot unary encoding from Theorem 3.81, Item 3 have size $O(n)$ and they produce, for $t(i)$, the bits $y_{i+1,0}, y_{i+1,1}, \ldots, y_{i+1,2^m-1}$ storing the one-hot encoding of $t(i)$ as a word $y_{i+1,0}y_{i+1,1}\cdots y_{i+1,2^m-1} \in \{0,1\}^+$. We have $2^{m-1} \leq n \leq 2^m - 1$. Thus, since $t(i) \in [n]$, we find $y_{i+11,n+1} = \cdots = y_{i+1,2^m-1} = 0$. Then the matrix $(y_{i,j})_{i,j\in\{1,\ldots,n\}}$ equals the transition matrix of t. Inspection of the proof of Theorem 3.81, Item 3 gives that the above circuits can be generated by a deterministic log-space Turing transducer on input $1^{\lfloor \log_2(n)+1 \rfloor}$ since it can easily compute n from the input length and use this parameter in the construction of the circuit.

Conversely, suppose the transition matrix $y = (y_{ij})_{i,j\in\{1,\ldots,n\}}$ of t is given. The i-th row $y_{i1}, \ldots, y_{in}$ corresponds to a unary one-hot encoding of $t(i-1)$. So, by Theorem 3.81, Item 1, we can compute binary encodings of $t(0), t(1), \ldots, t(n-1)$ by L-uniform AC^0 circuits. □

3.9.6.3 Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton. A reasonable idea is to represent $\mathcal{A}$ as a list of transition matrices for each letter.

A list of transition matrices can be specified by $|\Sigma|n^2$ bits where the first n^2 bits correspond to the first letter, the next n^2 bits to the next letter and so on for some ordering of the alphabet.

For a fixed alphabet, a deterministic log-space Turing machine can easily separate the input into the individual transition matrices, i.e., identify the first n^2 bits, then the next n^2 bits and so on, and it just needs the total length $|\Sigma|n^2$ of the input to perform this task.

However, in our problems, the alphabet is also part of the input. In this case, the input length alone does not determine the size of the alphabet or the number of states, since multiple possibilities exist for each given length.

Hence, a machine that constructs circuits using the length of the input as parameter cannot create a circuit with fixed input gates for each transition matrix. The circuit must determine from the input which bits constitute the individual transition matrices. For the circuit to do this, we need some way to supply this information in the input, for example, by separating the individual transition matrices.

There are several schemes to achieve this. For example, we can encode every 0 by 00, every 1 by 11 and use 01 and 10 as separating sequences between transition matrices. There are also more clever schemes using so-called self-delimiting descriptions that encode the length of the individual parts (here the transition matrices) into the input, for example, see [412, Sect. 1.4] or [411, Sect. 2.2]. However, it is not immediately evident how an AC^0-circuit can parse such inputs to identify the individual transition matrices.

For this reason, we use another encoding that is easy for AC^0 circuits to process, but uses $(n + |\Sigma|)^3$ bits (and therefore, uses more space than the schemes discussed above). Let R_δ be the ternary relation over the universe $\Sigma \cup Q$ such that $R_\delta(x, y, z)$ holds if and only if x represents a letter a_x, y represents a state q_y, z represents a state q_z, and $\delta(q_y, a_x) = q_z$. Then an automaton is supplied to a circuit as the bitstring of length $(n + |\Sigma|)^3$ encoding R_δ.

It is well-known that properties that are testable by families of AC^0 circuits are precisely those checkable by first order formulas [333, Thm. 5.22]. We use this equivalence by giving first order sentences to show containment in AC^0. The only technicality left is to argue that this is also possible with our notion of L-uniformity. We argue next that, given a first order formula, a deterministic log-space Turing transducer can construct an AC^0 circuit family where each circuit evaluates to true on a given input of size $(n+|\Sigma|)^3$ if and only if the first order formula holds for the automaton encoded in the input.

Suppose a first order formula quantifying over the elements in $Q \cup \Sigma$ and using only the predicate R_δ is given. A Turing transducer, given $1^{(n+|\Sigma|)^3}$ as input, can compute the size of the universe $n + |\Sigma|$. Furthermore, for numbers $i, j, k \in \{1, \ldots, n + |\Sigma|\}$ it can also determine the bit that encodes $R_\delta(i, j, k)$. With this information, an all- or exists-quantifier can be translated into an unbounded AND or OR-gate for the input of size $(n + |\Sigma|)^3$, respectively. Using this, by induction over the structure of the formula, the transducer can construct an AC^0-circuit.

Next, we show that, when giving formulas to show containment in AC^0, we can, without loss of generality, quantify only over letters or states, and can access entries of the transition matrices of $\mathcal{A}$, thus giving us some "syntactic sugar" in comparison to our plain formulas where quantification is always over $Q \cup \Sigma$ and that only uses the relation R_δ.

Consider the two unary relations R_Q and R_Σ where $R_Q(x)$ (resp. $R_\Sigma(x)$) is true precisely if x is a state (resp. a letter). The relations R_Σ and R_Q can be specified by

the first order formulas $\bigvee_{z_1} \bigvee_{z_2} R_\delta(x, z_1, z_2)$ and $\bigwedge_{z_1} \bigwedge_{z_2} \neg R_\delta(x, z_1, z_2)$, respectively. Thus, they can be used in first order formulas. Furthermore, we can use them to quantify only over the letters or only the states. For example, the following formula, querying whether an automaton has a trap state,

$$\bigvee_{a \in \Sigma} \left(\bigwedge_{q \in Q} R_\delta(a, q, q) \right),$$

can be translated into

$$\bigvee_{x \in U} \left(R_\Sigma(x) \rightarrow \left(\bigwedge_{y \in U} R_Q(y) \rightarrow R_\delta(x, y, y) \right) \right).$$

This gives us the mean to write simpler formulas. Note that if x represents a letter a and y, z states, then $R_\delta(x, y, z)$ corresponds to an entry in the transition matrix of a. Thus, we can directly reference entries of the transition matrices of a letter in first order formulas. We will often do so, but using a simpler notation, for example denoting the transition matrices by $(m_{i,j})_{i,j \in \{1,\dots,n\}}$ for $a \in \Sigma$ and using these expressions in first order formulas instead of R_δ.

Remark 3.83 All of the representations discussed above, and the representation by giving the transformation of the letters in point-wise representation in binary with some mechanism of separation between the letters, are readily convertible into each other by deterministic log-space Turing machines. Thus, when using $\leq_m^{\log}$-reductions, we may assume either of these representations.

Remark 3.84 The representation of structures by relations and constants over a finite universe, and their relation to logic and computation, is explored in descriptive complexity, see [333]. In [333, Def. 2.1] the encoding of these structures by binary sequences is discussed. Here, we encoded letters and states in a common universe. In this respect, we also refer to the modeling of SAT in [333, Ex. 2.18] where the elements of the universe each model variables and clauses.

3.10 Remarks on Different Notions of Reduction

The notion of $\leq_{\log m}$-reduction is too strong to classify problems inside L, since the reduction itself already has the computational power of L. For this reason, we consider weaker (and finer) notions such as $\leq_{NC^1}^T$- and $\leq_{AC^0}^m$-reductions.[33]

[33] There does not seem to be a universal convention on when to call one form of reduction weaker or stronger. Our convention appears to be the prevalent one, although the opposite convention, where the roles are reversed, is also used in the literature, for example in [144].

Clearly, a $\leq_m^{\mathsf{AC}^0}$-reduction is also an $\leq_T^{\mathsf{NC}^1}$-reduction and a $\leq_m^{\log}$-reduction.[34] Most of the time, the reductions we present here are $\leq_m^{\mathsf{AC}^0}$-reductions. But often, concluding results are formulated for $\leq_T^{\mathsf{NC}^1}$- or $\leq_m^{\log}$-reductions only. The reason is that we often use results from the literature that are only stated for these stronger forms of reduction.

To make statements regarding complexity classes more concise, we omit mentioning the type of reduction and, when nothing else is said, adopt the following conventions.

1. When talking about complete or hard problems for NL, P, NP or PSPACE, we mean this with respect to $\leq_m^{\log}$-reductions.
2. When talking about complete or hard problems in L, we mean this with respect to $\leq_T^{\mathsf{NC}^1}$-reductions.
3. When talking about circuit complexity classes and families of circuits, as already said on page 72, when nothing else is said, we always mean the L-uniform variants.

Regarding the first two assumptions, when citing results from the literature we occasionally say explicitly what kind of reduction is involved. Also, complexity classification results stated in Part IV involve different reducibility notions. For example, NP- and L-completeness statements are w.r.t. $\leq_m^{\log}$- and $\leq_T^{\mathsf{NC}^1}$-reducibility, respectively. However, having a single (weak) form of reduction is preferable, as it would imply stronger separation results, see [10, Sect. 23.6.1] for a short discussion. We reserve the question whether our complexity classification results hold for a common notion of reduction for future work.

Remark 3.85 Most of our reductions can be expressed by simple operations on input automata that are computable by AC^0 circuits. For example, these include adding letters according to a fixed transition scheme, adding auxiliary states with transitions derived from the given ones, taking Cartesian products, or rewiring the transition structure of an input automaton. However, we often omit the full bit-level details of these reductions, as this would lead to lengthy and technical proofs.

3.11 Remarks on the Order of Function Composition

Our convention for function composition deviates from the one adopted in permutation group theory or in some approaches to the algebraic theory of automata. As this can be confusing at times, we feel the urge to justify and comment on that. The notation and terminology introduced in the present section will not be included in the notation index or the index of terminology, as it is only used here.

When applying a function $f\colon X \to Y$ to some $x \in X$, we can write the argument to the right, i.e., $f(x)$, and f acts on the left (left action), or we can write the argument to the left, i.e., $(x)f$ (xf for short), and f acts from the right (right action). In the present work, we use the left-action convention (introduced on page 3). Furthermore,

[34] However, $\leq_T^{\mathsf{NC}^1}$-reductions are *not* known to be $\leq_m^{\log}$-reductions. This is because of the difference between oracle and many-one reductions. For example, the former allows complementation of the results of oracle queries, see also the discussion in [10, Sect. 23.6].

when defining the composition of two functions $f\colon X \to Y$ and $g\colon Y \to Z$, either the function on the right-hand side can be applied first (right-most first, as defined here, see page 31), also called the *Leibniz order*, or the function on the left-hand side, i.e., define a composition operator by $(f;g)(x) = g(f(x))$ for $x \in X$ (left-most first), also called the *anti-Leibniz* or *diagrammatic order*.

In [468] also the notation f^*g for $g \circ f$ was mentioned as pulling back g along f to the composite. Here, the asterik operator makes $f\colon X \to Y$ into a function $f^*\colon Z^Y \to Z^X$ defined by $(f^*(g))(x) = g(f(x))$ for $x \in X$. So, conceptually, rather than regarding composition as a binary operation on functions, here we rather consider each function as an operator acting via composition (this is reminiscent of letting each element of a monoid act via multiplication from left or right, see [491, Prop. 4.25]). In the wild we have never seen anyone using this notation solely for the purpose of having a notation for function composition, but either in rather specialized contexts as operators on special function spaces as, for example, in differential geometry [579, p. 77 & p. 89], as the composition operator in functional analysis [166], or in the context of Koopman Operator Theory [180].

Now, the following combinations of conventions are conceivable.

1. Right-action and left-most first (also called "postfix convention").
   With these conventions, we have $x(f;g) = (xf)g$. This combination of conventions was in wide use in the early days of the theory of automata, for example in the books [194, 195, 247, 489]. It has the advantage of being in harmony with reading a diagram $X \xrightarrow{f} Y \xrightarrow{g} Z$ for the composition from left to right. In [194, p. 1], it is even noted that $x \in X$ can be interpreted as a function from some singleton set into X with value x and then application of f to x can be written as function composition and modeled by diagrams depicting compositions. However, these conventions seemed to be out of fashion in automata theory at least since the mid-nineties. In permutation group theory, this combination of conventions is often used [126, 179, 638], even taken a step further by using the *exponential notation* for function application, i.e., writing x^f for f applied to x, see for example [393].
2. Left-action and right-most first.
   This is the combination of conventions we adopted (see pages 3 and 31). Outside of specialized areas of mathematics such as the algebraic theory of automata or permutation group theory (or other branches of algebra as for example [217]), this seems to be the prevalent notion. For example, three famous textbooks from diverse fields: Lang's *Algebra* [405], Spivak's *Calculus on Manifolds* [579] or Königsberger's *Analysis I* [379] (the latter being a standard text for first year math students in Germany, at least it was during the author's time) all use these conventions.
3. Left-action and and left-most first.
   The textbook by Almeida [11] from 1995 uses this convention, the survey article by Pin [490] and, for example, a randomly sampled article by Diekert et al. [175]. Interestingly, function composition is not explicitly defined in these works. However, they have to use this convention, for otherwise, some cited results from this field have to be formulated differently. For example, partially ordered automata are precisely those automata having an $\mathcal{R}$-trivial transition monoid [111] (we refer to

the cited literature for definitions). But this result is only valid, due to the definition of the $\mathcal{R}$-relation via right ideals, when the left-most function is applied first in a composition (otherwise, it would correspond to $\mathcal{L}$-triviality).
A recent (work-in-progress, available online) textbook by Pin [491] solved this by using two composition operations. To cite [491, Sect. II.4.2]:

> [...] the reader should be warned that the product $\alpha\beta$ is not equal to $\alpha \circ \beta$, but to $\beta \circ \alpha$. In other word, the operation in $\mathcal{T}(P)$ is reverse composition. [...]

Another reason to adopt this convention is that it uses the common way of writing functions on the left, i.e., $f(x)$, while retaining compatibility with the *composition of relations* when defined in the following way: for a relation R from X to Y and a relation S from Y to Z, the composition is the relation RS defined, for all $x \in X$ and $z \in Z$, by

$$x(RS)z \Leftrightarrow \exists y \in Y \colon xRy \wedge ySz.$$

Now, when associating with $f \colon X \to Y$ the relation R_f from X to Y by $xR_f y$ if and only if $f(x) = y$ we have $R_{(f;g)} = R_f R_g$ for functions $f \colon X \to Y$, $g \colon Y \to Z$. This is, in particular, better suited when functions are defined as relations (which we have not done in the present work since we introduced function before as the more primitive notion as in [194]), or when representing relations by matrices, as this convention fits with matrix multiplication. For example, in teaching first-year computer science students at Trier University [208] these conventions are adopted, as relations are introduced as a more primitive concept and (partial) functions are defined as special relations.
Of course, one can define the composition of relations by reversing the order (this is done in [491, Sect. I.1.2]). However, in this case compatibility with matrix multiplication mentioned in the previous paragraph is lost (and, to the best of our knowledge, matrix multiplication is defined in the same way in all of mathematics) and a correct formula would involve additional matrix transpositions.

4. Right-action and right-most first. We have never seen this combination in use.

Furthermore, given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$, some authors choose to suppress mentioning δ explicitly in the specification and application of the transition function, writing $q.u$ for $\delta(q, u)$ with $q \in Q$ and $u \in \Sigma^*$. This dot-notation actually fits well with the right-action and left-most first conventions, as then $q.u = (q)\delta_u = (q)(\delta_v; \delta_w) = ((q)\delta_v)\delta_w$ for $u = vw$, $u, v, w \in \Sigma^*$, $q \in Q$. However, we have opted against this dot-notation mainly because in Part IV often two transition functions (of a constraint automaton and of an input automaton) appear in the same context. Hence we felt the need to distinguish them notation-wise. Then, in our opinion, when applying a transition function $\delta \colon Q \times \Sigma^* \to Q$ explicitly, the left action convention seems to fit better. Also, our convention adopted for composition was motivated by what we feel is most often used in all of theoretical computer science and formal language theory (apart from specialized areas such as the algebraic theory of automata).

Note that "operators" such as $\mathrm{Syn}(\mathcal{A})$ or $Z(G)$, which are never composed with other "operators", are almost always written as acting from the left, resulting in a mix of both conventions in some works as for example in [29, 393].

Since Part III of the present thesis involves permutation group theory, we were faced with the choice what convention to adopt. In face of Part IV, which does not involve any permutation group theory at all and is a more classical formal language theory topic, we opted to the left-action convention that is more common in formal language theory. This convention has implications on how we define conjugation later. For more discussion on the two conventions of composition, see [468] (the notation $f;g$ is taken from this article).

3.12 Discussion

The primary purpose of the present chapter was to fix notation and introduce results from the literature. For more (well-known) properties of functions and compositions, the first chapter from [491] gives a nice overview.

Our algebraic notions are standard. The textbooks by Clifford and Preston [152, 153] and Ljapin [419] are two standard references from the early days of semigroup theory. More recent books are written by Howie [325] or Higgins [291]. For more information on algebra in general, we refer to the textbook [405]. The book by Lallement discusses semigroup theory while pointing out connections to formal language theory [401]. Semirings find application in generalizations of automata theory and formal language theory to weighted automata and formal power series, see [391, 88]. Idempotent elements were introduced in [478] and also appear widely throughout ring and module theory, for example, in the study of algebras [478, 402, 27], and in particular the proof of the Wedderburn-Artin classification theorem for semisimple algebras [44, 402, 630], and in semigroup theory in connection with Green's relations, see [152, 153, 325]. Idempotency is also studied in the context of analysis, see [378]. In geometrical settings or linear algebra, idempotent linear transformations are also called projections.

The textbook by Ganyushkin & Mazorchuk [236] is devoted to transformations semigroups. The first chapter of [195] recaps some folklore results from transformation semigroup theory and develops this theory further. Clifford & Preston [152, 153] also discuss transformation semigroups.

The textbooks [194, 322, 388, 422, 491, 575] contain content on formal language theory. Classes of languages with effectively computable semilinear Parikh images, as for example context-free languages [149, 474] or languages accepted by reversal-bounded multicounter machines [326], have applications in formal verification and model checking since this property yields decidability of the emptiness problem for these languages. See the introduction of [314] for details. The earliest work that investigated commutative languages that we were able to trace back is [399].

An introduction to order theory is [169]. In Theorem 3.32 we gave a bijective order morphism that is not an isomorphism. Furthermore, not every bijective endomorphism is an automorphism, for example, see [392]. (We note that this holds for finite sets, since then some power of the endomorphism yields the identity.)

Our notions and results from graph theory are standard as can be found in any textbook, for example, as [177, 278], except the notion of strong $\frac{3}{2}$-connectivity, which is new.

The systematic study of computational complexity was initiated in [284]. Turing machines were introduced in [608], where also nondeterministic machines were mentioned (called choice machines therein), but not considered further. Nondeterminism as a computational mechanism was introduced later in [502]. There are a wealth of textbooks available on computational complexity theory, for example [41, 56, 57, 322, 575, 388].

Below we discuss some more issues and literature related to computational complexity in more detail. By doing so, often new notions are mentioned in passing (for example, alternating Turing machines) without formally introducing them. In such instances, we always refer to appropriate literature where the interested reader can find definitions.

Running Time, Complexity Theory, and Machine Models.

We have introduced the following machine models: the random access machine (RAM), (sequential) Turing machines and transducers, and circuits. In theoretical computer science, Turing machines and transducers are the most traditional model, circuits serve to classify problems with low complexity or to describe problems solvable by employing parallelism. The RAM model is often used for claims about runtime and space that can be more directly translated to bounds for real-life implementations. The other two models are used for theoretical investigations into the inherent difficulty of problems and for a finer classification not possible with the RAM model by the, rather powerful, constant time assumption on the basic (arithmetic) operations. Clearly, the practical and theoretical perspectives are strongly related. We cannot expect problems inherently difficult in the theoretical sense to have exact algorithms that run practically fast in all instances. For more discussion on the practical implications of computational complexity theory, see [6]. RAMs and Turing machines can both be used to define P [162]. Furthermore, there are also theoretical investigations on the RAM model, see [26, 280, 364, 423, 479, 566, 587] to name a few, and it has been adapted to capture low-level complexities [26, 270, 362] or investigated when it is universal, i.e., capable of executing other programs, called the Random Access Stored-Program Machine (RASP) [280, 162]. Lastly, we note that there exist complexity classes that appear reasonable, but are very sensitive to the machine model. For example, the class of problems solvable in linear time, i.e., $O(n)$-bounded time, see [270]. Hence, runtime claims made for the RAM model do not necessarily translate into natural complexity classes.

In classical complexity theory, various types of reduction have been considered. To name a few more than the ones introduced in Section 3.9.5:

1. Polynomial-time Turing reductions [156].
2. Polynomial-time many-one reductions [361].
3. Log-space many-one reductions [345, 584], log-space linear many one reductions [584], and rudimentary reductions [345] (the latter notion has been used to characterize DLOGTIME-uniform AC^0 [8]).
4. One-way log-space reductions [282].
5. First-order reductions [331, 167] (these reductions equal DLOGTIME-uniform AC^0-reductions, see [333])

6. Projections (non-uniform, each bit of the output depends on at most one bit of the input) [576].
7. AC^0 truth-table reductions [144, 232].
8. NC^1 Turing reductions [160],
9. First order translations/projections [332], which equal DLOGTIME-uniform NC^0-reductions [2].

More information on notions of reduction can be found in [9].

Resource-Bounded Computations.

In Section 3.9.2 we defined resource-bounded (time or space) Turing machines by demanding that all computational runs stay within the resource bounds. This convention is also used in [322, p. 288], [345, Sec.2], [575, Def. 7.9], and [41, Sec. 2.1.2]. Following conventions for alternating Turing machines from [143, 394], in [636, p. 5], for resource-bounded computations it is only required that, for accepted inputs, at least one accepting computational run stays within the resource bounds, and no constraints are imposed on computational runs that do not accept. This convention is also used in [57, Def. 2.11]. We note that in [86] yet another convention was used, where for accepted inputs, at least one accepting computational runs is required to stay within the resource bounds, and for not accepted inputs all terminating computational runs must stay within the resource bounds.

It is well known that, for reasonable resource-bounding functions commonly considered in computational complexity theory, all of the above definitions of resource-bounded machines are equivalent. Next, we will show this for multitape Turing machines and the classes of *fully space-constructible* [322, p. 297]) and *fully time-constructible* [322, p. 299] functions, which are functions $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ such that there exists a deterministic one-tape Turing machine that, for each input u, visits exactly $f(|u|)$ tape cell and makes exactly $f(|u|)$ moves, respectively. For example, the polynomial functions and the standard exponential functions are both fully time-constructible and fully space-constructible. Logarithms are not fully time-constructible since we need at least linear time to read the input, but they are fully space-constructible by using the b-adic representation of numbers.[35]

Let $s\colon \mathbb{N}_0 \to \mathbb{N}_0$ with $s(n) \geq \log_2(n)$ be fully space-constructible witnessed by a deterministic Turing machine M_s that uses exactly $s(n)$ tape cells for each input u with $n = |u|$. Let M be a Turing machine that, for an accepted input u, has at least one accepting computational run that takes up at most $s(n)$ space. Let m and t be the number of states and tape symbols, respectively, of M. If M accepts, it does so by a sequence of at most $(n+2)ms(n)t^{s(n)}$ moves (input head position, state, working head position, tape content), since otherwise some configuration repeats. We construct a new Turing machine accepting precisely the same inputs and for which all computational runs terminate and stay within the space bounds given by s in the following way:

[35] In [57, Def. 2.18], these functions are simply called time- and space-constructible, whereas in [322] the same terms refer to weaker notions.

1. For an input u, we mark out $s(n)$ tape cells by using M_s.
2. We simulate M where we halt and reject when it leaves those marked cells.
3. We maintain a counter on a separate tape that counts each step in base $4mt$. When this counter reaches $(4mt)^{s(n)}$ we halt and reject.

Since $(4mt)^{s(n)} \geq (n+2)ms(n)t^{s(n)}$, if the counter is reached, a configuration of the machine repeats. Thus, the counter prevents that the machine gets stuck in a loop. The overhead created by the counter is logarithmic and inside the space bounds demanded by $s(n)$ since $s(n) \geq \log_2(n)$.[36] The machine can also be simulated by a one-tape Turing machine without any additional space overhead [322, Proof of Thm. 7.2].

Next, let $t\colon \mathbb{N}_0 \to \mathbb{N}_0$ be fully time-constructible witnessed by a deterministic Turing machine M_t. Consider a nondeterministic Turing machine M that has, for accepted words u, at least one accepting computational run using at most $t(|u|)$ moves. Construct a new Turing machine that simulates M and also performs one step of M_t for each simulated time step of M using a separate work tape, and that rejects inputs on which M_t halts and M has not reached a final state. This gives a factor of two overhead in time. By the linear speed-up theorem [322, Thm. 12.3], we obtain a multitape Turing machine deciding the language and halting on each computational run for input u in at most $t(|u|)$ steps. From this, we can also obtain a one-tape Turing machine halting on each run in at most $t^2(|u|)$ steps [322, p. 292].

We note that in the seminal paper [284] that started the systematic study of the computational complexity of algorithms, time and space measuring functions are only required to be computable.

Circuits.

Circuits and Boolean algebra entered into computing with Shannon's master thesis [559] (see also the paper [560] based on it) where he showed how to use Boolean algebra for the synthesis and analysis of switching circuits and relays. One of the first papers investigating the relation of circuits to languages seems to be [157]. Pippenger [492] observed that size and[37] depth bounds for families of circuits correspond to resource bounds (see also [102, 523, 585] expanding these relations). He introduced the class NC in [492] to formalize this observation (the name NC is due to Cook [157, p. 338], see also [160], as an abbreviation for "Nick's class").

Uniformity is imposed on circuit families in order to define classes of languages that correspond to machine-based classes. L-uniformity (also called log-space uniformity) was introduced by Cook [157, p. 338] (see also [619, p. 47] or [56, Def. 4.2]), based on

[36] A related results is similarly proved in [322, Lem. 12.1]. However, in this book, resource-bounded machines are defined to always stay within the bounds, not just in case of acceptance. With this assumption, the proof from [322, Lem. 12.1] works for any space bounding function (not just the fully space-constructible ones). The trick is the dynamically enlarge the space used for the counter along with the number of cell i used by the simulated machine, and test whether it reaches $(4mt)^i$.

[37] Both parameters have to be considered for circuits as, by the conjunctive or disjunctive normal forms, every Boolean function can be realized by a logarithmic depth circuit using only gates of fan-in two. However, such a circuit might have exponential size.

a similar definition from [102, p. 738] (see also some related discussion in [159, 160]). We defined L-uniformity by demanding a machine that works in logarithmic space in the number of input bits. This was also done in [157, p. 338] and [7, p. 4].[38] In [619, p. 47] and [56, Def. 4.2] it was demanded that the Turing transducer constructing the circuits works in logarithmic space in the size of the circuits, but this makes no difference for our circuit families in NC. We note that in [7, 157, 619, 56], for circuit families $C_1, C_2, \ldots$ it was also assumed C_n has n input bits, where here we only demand the family to have a strictly increasing number of input bits.

In the present work, we only use L-uniformity as our form of uniformity, as it is easy to use and captures complexity classes below L. Apart from L-uniformity, other notions of uniformity have been studied in the literature, for example:

1. Ruzzo [523] introduced U_D-uniformity, U_E-uniformity, and U_{E^*}-uniformity for a circuit family $C_1, C_2, \ldots$ by requiring not that the circuits C_n itself are computable, but that a circuit connection language is decidable by, respectively, an $O(s_n)$-time deterministic Turing machine, an $O(s_n)$-time and $O(\log s_n)$-space deterministic Turing machine, and an $O(d_n)$-time and $O(\log s_n)$-space alternating Turing machine, where s_n and d_n denote the size and depth of C_n.
2. Barrington et al. [62] introduced DLOGTIME-uniformity where a random-access[39] deterministic Turing machine can decide queries about n-bit encodings of circuits in time $O(\log n)$.
3. P-uniformity [72] requiring that $1^n \mapsto C_n$ (n number of input bits here) is computable by a deterministic polynomial time Turing machine (see also [5] for a systematic study).
4. Borodin [102] introduced uniformity in depth $S(n)$ that requires that there exists a deterministic $S(n)$-space Turing transducer computing $1^n \mapsto C_n$ (n number of input bits here) and that the resulting circuit C_n has depth $O(S(n))$ (This is also called U_B-uniformity in [523, 634]).

Languages definable by first order formulas are precisely those decidable by nonuniform AC^0 circuit families [61, Thm. 4] and [332, Thm. 6.2]. DLOGTIME-uniform AC^0 corresponds to first order logic with a bit-query predicate, equality, and order [62], [61, Thm. 5], [333, Thm. 5.22]. More generally, for a class C of functions containing the DLOGTIME-computable functions and closed under composition, C-uniform AC^0 has the same expressibility as first order logic using predicates computable by functions in

[38] Originally, in [157] Cook demanded that the function $1^n \mapsto C_n$ for circuits of size $s(n)$ is computable by a transducer Turing machine in $O(\log s(n))$. (This is also called U_{BC}-uniformity in [523, 634].) For circuits in NC this equals our notion of computability by a log-space transducer measured in the number of input bits.

[39] We have introduced Turing machines as sequential devices. However, when studying sublinear-time computation, as in [62], this model is inadequate as we cannot read the whole input. A random-access Turing machine is appropriate for studying sublinear time and consists of a read-only input tape, a fixed number of sequential read-write work tapes, and a special sequential read-write *address tape*. At each computation step, the machine can read the input bit whose position is encoded (in binary) on the address tape, or determine that the encoded address exceeds the input length.

Sequential Turing machines can simulate these machines with only polynomial time overhead and logarithmic space overhead. Hence, for defining the classical complexity classes L, NL, P, NP, and PSPACE, it makes no difference whether one uses the sequential or the random-access model.

C [61, Cor. 6]. This yields that log-space uniformity is equivalent to first order theory using only predicates computable by deterministic log-space Turing machines.

The nonuniform class AC^0 was introduced in [232] and it was shown that the parity function is not computable by an AC^0 circuit family. Furthermore, the same paper introduced AC^0-Turing reductions and reduced the computation of majority, multiplication and the transitive closure to the computation of the parity function. In [3] it was shown that no first order formula with equality and a finite number of predicates evaluates to true precisely on an even subset of the universe. By the mentioned correspondence between AC^0 and first order formula this yields another proof that the parity function is not in AC^0. These results hold for non-uniform AC^0.

We remark that it is unknown whether the containment $\mathsf{NC}^1 \subseteq \mathsf{L}$ is strict, but this is generally believed. For example [168] investigates the complexity landscape between NC^1 and L. Also, it is not known whether the inclusion $\mathsf{NL} \subseteq \mathsf{NC}^2$ is strict, see also the discussion in [523].

A standard reference on circuits is the book by Wegener [634] (the "blue book"). It also features an extensive chapter on the Parallel RAM (PRAM) machine model of parallel computation. However, it contains very little material on complexity classes related to circuits and only a single short section (without proofs) mentioning uniformity notions and relations to alternating Turing machines. The book by Vollmer [619] offers more on these topics and treats uniformity thoroughly. The PRAM is also described in [225, 341]. Clote and Kranakis's book [154] covers circuit complexity classes but without much discussion on uniformity issues. But it offers additional material on randomness and proof systems. More advanced (and recent) material can be found in the book by Jukna [347]. Greenlaw et al. [262] have written a book on P-completeness that also contains much background material on NC and parallel computation. Circuit complexity classes also have strong ties to logic and descriptive complexity. We refer to the book by Immerman [333], which is a standard reference for these topics, for more information on this connection. The book by Straubing [586] focuses on the relation between logic, circuits up to NC^1 and finite automata, but does not include material beyond that, e.g., on the whole of NC. The relation between automata and circuits from NC^1 was first observed by Chandra et al. [142] and developed further by Barrington [60], Barrington and Thérien [63] and Barrington et al. [64]. The two volumes [56, 57] by Balcázar, Díaz, and Gabarró give a good coverage of circuits in complexity theory, whereas the first volume covers the non-uniform setting and the second volume the uniform setting. A selection of several contributions to a symposium on Boolean function complexity can be found in Paterson's book [475].

Chapter 4
Classes of Transformations

Here, we investigate classes of transformations. Sections 4.1, 4.2, 4.3, 4.4, and 4.5 introduce and investigate permutations, cycles, one-point singulars, and idempotent transformations, respectively. In Section 4.6, we investigate idempotent one-singulars. We will present different characterizations. Later, the transformations that arise in the study of automata theory result from compositions of transformations induced by letters. For this reason, we also investigate resulting transformations when composing different types of transformations.

4.1 Permutations and Quasipermutations

A *permutation* is a bijective transformation. Recall that, in this case, as discussed in Section 3.2.1, there exists an *inverse transformation* $t^{-1}\colon X \to X$ such that $t^{-1}t = tt^{-1} = \mathrm{id}_X$.

For an arbitrary transformation $t\colon X \to X$ and $S \subseteq X$ we say that *t permutes a subset $S \subseteq X$* if $t_{|S}$ is a permutation on S. We call $t\colon X \to X$ a *quasipermutation* [257] if it permutes its image $t(X)$.

First, in Theorem 4.1 we collect several well-known characterizations of permutations on finite sets. Then, the remaining statements investigate quasipermutations.

Proposition 4.1 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a permutation.*
2. $\mathrm{dupl}(t) = \emptyset$*, i.e., t is injective.*
3. $t^{-1}(\mathrm{dupl}(t)) \cap t([n])$ *is empty.*
4. *t has no root point.*
5. $\mathrm{def}(t) = 0$.
6. *Every state is a cycle state.*
7. *There exists $i > 0$ such that $t^i = \mathrm{id}_{[n]}$.*
8. $\mathrm{rank}(t) = n$*, i.e., t is surjective.*

Proof. $1 \Rightarrow 2$: This follows as a permutation is injective by definition.

$2 \Rightarrow 3$: This is obvious.

$3 \Rightarrow 4$: Let $y \in [n]$ be a root point. Then $y \in \operatorname{dupl}(t)$ and $t^p(y) = y$ for some $p > 0$. This gives $t^{p-1}(y) \in t^{-1}(\operatorname{dupl}(t)) \cap t([n])$.

$4 \Rightarrow 5$: Suppose $\operatorname{def}(t) > 0$. Then there exists $x \in \operatorname{excl}(t)$. By finiteness, there exists a minimal $i \geq 0$ such that $t^i(x) = t^{i+p}(x)$ for some $p > 0$, where, since $x \in \operatorname{excl}(t)$, we must have $i > 0$. Set $y = t^i(x)$. We have $t^p(y) = y$. By the minimal choice of i, we have $t^{i-1}(x) \neq t^{i+p-1}(x)$ and both map to the same point y. Hence we have $y \in \operatorname{dupl}(t)$. Thus y is a root point.

$5 \Rightarrow 6$: By Item 3 from Theorem 3.14, we have $|\operatorname{dupl}(t)| \leq \operatorname{def}(t) = 0$, i.e. t is injective. By finiteness there exist $i \geq 0$ and $p > 0$ with i minimal such that $t^i(x) = t^{i+p}(x)$. For $i > 0$, by injectivity, we obtain $t^{i-1}(x) = t^{i-1+p}(x)$, contradicting the minimal choice of i. So $i = 0$ and $x = t^p(x)$. Thus x is a cycle state.

$6 \Rightarrow 7$: By assumption for each $x \in [n]$ there exists $i_x > 0$ such that $t^{i_x}(x) = x$. Then $t^{\operatorname{lcm}\{i_x \mid x \in [n]\}} = \mathrm{id}_{[n]}$.

$7 \Rightarrow 8$: By assumption we have $x = t(t^{i-1}(x)) \in t([n])$ for each $x \in [n]$. So $t([n]) = [n]$, which gives $\operatorname{rank}(t) = n$.

$8 \Rightarrow 1$: By Equation (3.15) from Theorem 3.12 we have $n = \operatorname{rank}(t) = n + \operatorname{def}(t)$. Thus $\operatorname{def}(t) = 0$. By Item 3 from Theorem 3.14, we have $|\operatorname{dupl}(t)| \leq \operatorname{def}(t) = 0$, i.e. t is injective. □

Corollary 4.2 *Let $t\colon X \to X$ and $S \subseteq X$ be finite. Then t permutes S if and only if $t(S) = S$.*

Proposition 4.3 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a quasipermutation.*
2. *For each $x \in X$, $t(x)$ is a cycle point.*
3. *For each $x \in \operatorname{excl}(t)$, $t(x)$ is a cycle point.*
4. *The set of tail points equals $\operatorname{excl}(t)$.*
5. *$\operatorname{dupl}(t_{|t([n])}) = \emptyset$.*

Proof. $1 \Rightarrow 2$: By assumption, $t_{|t(X)}$ is a permtuation. Then the claim follows from Item 6 of Theorem 4.1 applied to the permutation $t_{|t(X)}$.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 4$: Clearly, states in $\operatorname{excl}(t)$ are tail points. For a tail point x we have $x = t^i(z)$ for an excluded point z and some $i \geq 0$. By assumption, $i = 0$ and so $x = z$. Thus $\operatorname{excl}(t)$ equals precisely the tail points.

$4 \Rightarrow 5$: Assume $t_{|t(X)}$ is not injective. Then $t(t(x)) = t(t(z))$ for $x, z \in [n]$ with $t(x) \neq t(z)$. Since a cycle point has precisely one predecessor on its cycle, one of both points $t(x)$ or $t(z)$ must be a tail point, and this point is $t(X)$.

$5 \Rightarrow 1$: We have $t(t([n])) \subseteq t([n])$. By assumption, t acts injectively on $t(X)$. This yields $|t(t([n]))| = |t([n])|$ and thus we obtain $t(t([n])) = t([n])$. □

Lemma 4.4 *Let $t\colon [n] \to [n]$ be a quasipermutation. Then*

$$|t^{-1}(y)| = |t^{-1}(y) \cap \operatorname{excl}(t)| + 1 \quad \textit{for } y \in t([n]),$$

and $t^{-1}(y) = \emptyset$ otherwise. In particular $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$.

Proof. As $t(t([n])) = t([n])$ and $t([n])$ is finite, $t_{|t([n])}$ is a permutation of $t([n])$. Hence every $y \in t([n])$ has exactly one preimage in $t([n])$, and the remaining preimages lie in $\operatorname{excl}(t) = [n] \setminus t([n])$, which gives the displayed equality; points outside $t([n])$ have no preimage. Consequently $|t^{-1}(y)| \geq 2$ if and only if $t^{-1}(y)$ meets $\operatorname{excl}(t)$, that is, if and only if $y \in t(\operatorname{excl}(t))$. □

Proposition 4.5 *Let $t\colon [n] \to [n]$ be of defect at most two. If $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$, then t is a quasipermutation.*

Proof. If there exists $x \in \operatorname{dupl}(t)$ not contained in a cycle, then $x = t^i(y) = t^j(z)$ for two distinct excluded points $y, z \in \operatorname{excl}(t)$ and $i, j \geq 1$. By Item 4 of Theorem 4.1 there exists another point in $\operatorname{dupl}(t)$ contained in a cycle, which must be the image of yet another excluded point by assumption. Then t would have defect at least three. Thus every duplicate point is contained in a cycle. Then, with $\operatorname{dupl}(t) = t(\operatorname{excl}(t))$, the image of every excluded point is a cycle point. Thus, with Theorem 4.3, t is a quasipermutation. □

4.2 Cycles and Cycles Induced by a Transformation

Let X be some set and $a_0, a_1, \ldots, a_{m-1} \in X$ be $m \geq 1$ distinct elements. Then the permutation $(a_0\ a_1\ \cdots\ a_{m-1})_X\colon X \to X$ given by

$$(a_0\ a_1\ \cdots\ a_{m-1})_X(x) = \begin{cases} a_{(j+1) \bmod m} & \text{if } x = a_j \text{ for some } j \in \{0, 1, \ldots, m-1\}, \\ x & \text{otherwise,} \end{cases}$$

is called an *m-cycle* or *cycle* of length m.[1]

Equivalently, $t\colon X \to X$ is an m-cycle if and only if $m \leq 1$ and $t = \mathrm{id}_X$ or $m = |\operatorname{move}(t)|$ and for $x, x' \in \operatorname{move}(t)$ there exists $i \geq 0$ such that $t^i(x) = x'$. When the context is clear, we often omit the index X and simply write $(a_0\ a_1\ \cdots\ a_{m-1})$. We do not use commas in the cycle notation between points, to distinguish it from an ordinary m-tuple. When writing a $1 \times n$ matrix as in Figure 3.3 we obtain the same notation as for a cycle, but in each case the context should make it clear whether we refer to a cycle or a $1 \times n$ matrix. For each $a \in X$, we have $(a) = () = \mathrm{id}_X$. The following equations hold:

$$(a_0\ a_1\ \cdots\ a_{m-2}\ a_{m-1}) = (a_1\ a_2\ \cdots\ a_{m-1}\ a_0) \tag{4.1}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_1\ \cdots\ a_{m-1})(a_0\ a_{m-1}) = (a_0\ a_2\ \cdots\ a_{m-1})(a_0\ a_1) \tag{4.2}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_{m-2}\ a_{m-1})(a_0\ a_1\ \cdots\ a_{m-3}\ a_{m-1}) \tag{4.3}$$

$$(a_0\ a_1\ \cdots\ a_{m-1}) = (a_0\ a_1)(a_1\ \cdots\ a_{m-1}) \tag{4.4}$$

$$(a_0\ a_1\ \cdots\ a_{m-2}\ a_{m-1}) = (a_{m-2}\ a_0)(a_{m-2}\ a_{m-1})(a_0\ \cdots\ a_{m-3}) \tag{4.5}$$

[1] Note that for $X \neq \{a_0, a_1, \ldots, a_m\}$ the cycle has fixpoints. Some authors exclude this possibility and stipulate $X = \{a_0, a_1, \ldots, a_m\}$.

And for $0 \leq i < j < m$ we have:

$$(a_0\ a_1\ \cdots\ a_{m-1})(a_i\ a_j) = (a_0\ a_1\ \cdots\ a_i\ a_{j+1}\ a_{j+1} \cdots a_{m-1})(a_{i+1}\ \cdots\ a_j)$$
$$(a_i\ a_j)(a_0\ a_1\ \cdots\ a_{m-1}) = (a_0\ a_1\ \cdots\ a_{i-1}\ a_j\ a_{j+1}\ \cdots\ a_m)(a_i\ \cdots\ a_{j-1})$$

A *transposition* is a 2-cycle. Two cycles $t, h\colon X \to X$ are *disjoint* if $\mathrm{move}(t) \cap \mathrm{move}(h) = \emptyset$, in this case we have $th = ht$. Let $t\colon X \to X$ be some transformation. For finite $S \subseteq X$, we say that t *cyclically permutes* S if t permutes S and $t_{|S}$ is an $|S|$-cycle. Then $t^{|S|}(x) = x$ for all $x \in S$. A permutation on a set of size at least two cyclically permutes its domain if and only if it is a cycle with no fixpoints. If $t_{|S} = (a_0\ a_1\ \ldots\ a_{m-1})$ we also say that t *induces the cycle*, or *contains the cycle*, $(a_0\ a_1\ \cdots\ a_{m-1})$. Equivalently, $t(a_j) = a_{(j+1) \bmod m}$ for all $j \in \{0, 1, \ldots, m-1\}$, i.e., $(x\ t^2(x)\ \cdots\ t^{m-1}(x)) = (a_0\ a_1\ \ldots\ a_{m-1})$. For $x \in X$ and $m \geq 1$, the points $x, t(x), \ldots, t^{m-1}(x)$ form a cycle induced by t if and only if m is minimal such that $t^m(x) = x$. Two distinct cycles induced by t are disjoint.

Proposition 4.6 *A permutation on a finite set equals the composition (in any order) of its induced cycles.*

Proof. Let $t\colon [n] \to [n]$ be a permutation with induced cycles $t_1, \ldots, t_m$. For $x \in [n]$, we have $t_i = (x\ t(x) \cdots t^{k-1}(x))$ for some minimal $k \geq 1$ with $t^k(x) = x$ and unique $i \in \{1, \ldots, m\}$. Furthermore, for $j \neq i$, we also have $t_j(x) = x$. Thus, the result of any composition of $t_1, \ldots, t_m$ maps x to $t_i(x) = t(x)$, and so this composition equals t. □

Corollary 4.7 *A permutation on a finite set can be written as a composition of transpositions.*

Proof. This follows from Equation (4.2) and Theorem 4.6. □

4.3 Tail Transformations, Simple Idempotents, and Cycles

Here, we introduce tail transformations and simple idempotents. We then classify the transformations obtained by composing a simple idempotent with a cycle. Theorem 4.13 bounds the length of cycles induced in the composition of a simple idempotent with a cycle.

Let $a_1, \ldots, a_m \in X$ be distinct numbers with $m \geq 2$. The *tail transformation (of* $a_1, \ldots a_m$*)* is the transformation $[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]_X\colon X \to X$ given by

$$[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]_X(x) = \begin{cases} a_{i+1} & \text{if } x = a_i \text{ for } i \in \{1, \ldots, m-1\}, \\ x & \text{otherwise.} \end{cases}$$

When the context is clear, we often omit X and simply write $[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m]$.

Proposition 4.8 *Let $a_1, \ldots, a_m \in X$ be pairwise distinct and $1 < i < m$. Then*

$$[a_1 \mapsto a_2 \mapsto \cdots \mapsto a_m] = [a_1 \mapsto a_2 \mapsto \cdots \mapsto a_i][a_i \mapsto a_{i+1} \mapsto \cdots \mapsto a_m]$$
$$t[a_i \mapsto a_{i-1}] = [a_i \mapsto a_{i+1}]t = [a_i \mapsto a_{i+1} \mapsto \cdots \mapsto a_m][a_1 \mapsto a_2 \mapsto \cdots \mapsto a_i]$$

where $t = [a_1 \mapsto a_2 \mapsto \cdots a_{i-1} \mapsto a_{i+1} \mapsto \cdots \mapsto a_m]$.

A *simple idempotent* is a tail transformation involving two distinct numbers, i.e., where $m = 2$.[2] So, a simple idempotent is written as $[a \mapsto b]$ for distinct $a, b \in X$. In [128] a similar notation was used. A transformation $t\colon X \to X$ is a simple idempotent if and only if $\mathrm{move}(t)$ is a singleton set.

Proposition 4.9 *For pairwise distinct* $a, b, c, d \in X$*, the following equations hold:*

$$[a \mapsto b][a \mapsto b] = [a \mapsto b] = [a \mapsto b][b \mapsto a] \tag{4.6}$$
$$[c \mapsto d][a \mapsto b] = [a \mapsto b][c \mapsto d] \tag{4.7}$$
$$[a \mapsto c][a \mapsto b] = [a \mapsto b] \tag{4.8}$$
$$[c \mapsto b][a \mapsto b] = [a \mapsto b][c \mapsto b] = [a \mapsto b][c \mapsto a] \tag{4.9}$$
$$[c \mapsto a][a \mapsto b][b \mapsto c] = [c \mapsto b][b \mapsto a][a \mapsto c][c \mapsto b] \tag{4.10}$$
$$[d \mapsto c][c \mapsto a][a \mapsto b][b \mapsto c] = [d \mapsto a][a \mapsto b][b \mapsto d][d \mapsto c][c \mapsto b] \tag{4.11}$$

Equation (4.9) also follows from Theorem 4.8 for $m = 3$, but is kept for reference. The equations from Theorem 4.9 yield a presentation in terms of generators and relations for the set of all non-permutations on a finite set [188].

Corollary 4.10 *Let* $a, b, c, d \in X$ *with* $a \neq b$ *and* $c \neq d$ *and* $[a \mapsto b] \neq [c \mapsto d]$*. Then we have* $[c \mapsto d][a \mapsto b] = [a \mapsto b][c \mapsto d]$ *if and only if* $a \neq c$ *and* $b \neq c$ *and* $d \neq a$*, i.e., both simple idempotents commute if and only if* a, b, c, d *are all distinct or* $b = d$ *and* a, b, c *are all distinct.*

Proof. For $a = c$, since $[a \mapsto b] \neq [c \mapsto d]$ we have $d \neq b$ and both simple idempotents do not commute by Equation (4.8). Next, suppose $b = c$. For $a = d$, both do not commute by Equation (4.6). For $a \neq d$, we have $[a \mapsto c][c \mapsto d] = [a \mapsto c \mapsto d] \neq [c \mapsto d][a \mapsto c]$.

Conversely, suppose $a \neq c, b \neq c, d \neq a$, i.e., $a \notin \{c, d\}$ and $b \neq c$. If $b \neq d$, i.e., all points are pairwise distinct, we have $[a \mapsto b][c \mapsto d] = [c \mapsto d][a \mapsto b]$ by Equation (4.7). For $b = d$, we have $[a \mapsto b][c \mapsto b] = [c \mapsto b][a \mapsto b]$ by Equation (4.9). □

Let $t = [a \mapsto b]$ for $a \neq b$. Then $\mathrm{dupl}(t) = \{b\}$, $\mathrm{excl}(t) = \{a\}$, and for finite $S \subseteq X$ we have

$$|t(S)| < |S| \Leftrightarrow \{a, b\} \subseteq S \Leftrightarrow S = t(S) \cup \{a\}, \tag{4.12}$$
$$|t(S)| = |S| \Leftrightarrow t(S) = S \vee (b \notin S \wedge a \in S \wedge t(S) = (S \cup \{b\}) \setminus \{a\}). \tag{4.13}$$

[2] Also called a *singular* transformation in [107].

The following equations encompass, up to a renaming of the points, all combinations to compose a simple idempotent with a cycle. Equations (4.14) and (4.19) are illustrated in Figure 4.1.

Proposition 4.11 *Let $m \geq 2$, $\{0, 1, \ldots, m\} \subseteq X$, and $0 < d \leq m$. Then we have*

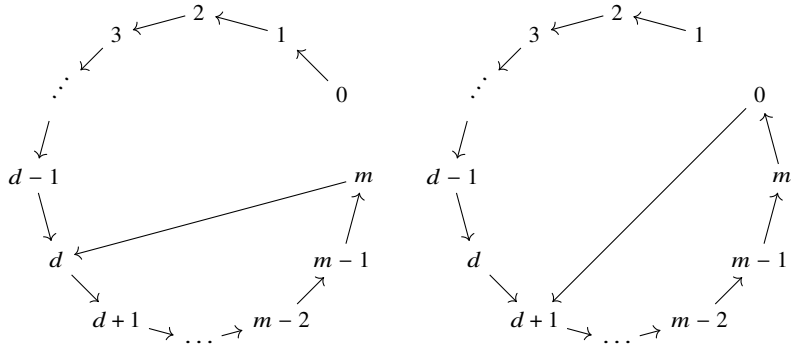
$$\begin{aligned}
[0 \mapsto d](0\ 1\ \cdots\ m) &= [0 \mapsto \cdots \mapsto d-1 \mapsto d](d\ d+1\ \cdots\ m) && (4.14)\\
&= (d\ d+1\ \cdots\ m)[0 \mapsto \cdots \mapsto d-1 \mapsto m] && (4.15)\\
&= (0\ 1\ \cdots\ m)[m \mapsto d] && (4.16)\\
&= [m \mapsto d][0 \mapsto 1 \mapsto \cdots \mapsto m] && (4.17)\\
&= [0 \mapsto 1 \mapsto \cdots \mapsto m][m \mapsto d-1] && (4.18)
\end{aligned}$$

and for distinct $a, b \in X \setminus \{0, 1, \ldots, m\}$ we have

$$\begin{aligned}
(0\ 1\ \cdots\ m)[0 \mapsto d] &= [1 \mapsto 2 \mapsto \cdots \mapsto d+1](0\ d+1\ \cdots\ m) && (4.19)\\
(0\ 1\ \cdots\ m)[0 \mapsto a] &= [1 \mapsto \cdots \mapsto m \mapsto 0 \mapsto a] && (4.20)\\
[0 \mapsto a](0\ 1\ \cdots\ m) &= [0 \mapsto 1 \mapsto \cdots \mapsto m \mapsto a] && (4.21)\\
(0\ 1\ \cdots\ m)[a \mapsto b] &= [a \mapsto b](0\ 1\ \cdots\ m) && (4.22)
\end{aligned}$$

Corollary 4.12 *For $m \geq 2$ we have the following:*

$$[0 \mapsto 1](0\,1\,\cdots\,m) = [0 \mapsto 1](1\,2\,\cdots\,m) = (1\,2\,\cdots\,m)[0 \mapsto m] = (0\,1\,\cdots\,m)[m \mapsto 0]$$



(a) $[0 \mapsto d](0\ 1\ \cdots\ m)$. (b) $(0\ 1\ \cdots\ m)[0 \mapsto d]$.

Fig. 4.1: Functional graphs resulting from the two possible compositions of $(0\ 1\ \cdots\ m)$ with $[0 \mapsto d]$ where $0 < d \leq m$, see Equations (4.14) and (4.19).

Corollary 4.13 *Let $t\colon X \to X$ be an m-cycle and $h\colon X \to X$ be a simple idempotent. Then there exist $0 \leq i, j \leq m$ and $0 < p, q < m$ such that $(ht)^i = (ht)^{i+p}$ and $(th)^j = (th)^{j+q}$.*

Proof. We can assume $m \geq 2$. This follows from the equations given in Theorem 4.11, since the results of both compositions move at most $m + 2$ points, contain at most one cycle moving more than one point, and such a cycle moves at most m points, or it equals a tail transformation moving at most $m + 1$ points. □

4.4 One-Point Singular Transformations

A transformation $t\colon X \to X$ is said to be *one-point singular* [524] if $\mathrm{dupl}(t)$ is a singleton. As a slight abuse of language, using an adjective as a noun, we call a one-point singular transformation simply a one-point singular.

To give a few examples, simple idempotents and, by Theorem 3.14, Item 3, defect-one transformations on finite sets are one-point singulars. Figure 4.2 depicts the functional graph of an example one-point singular.

Theorems 4.14 and 4.15 provide characterizations of transformations that are injective or one-point singulars, the latter investigating the case of a finite domain. Theorem 4.16 gives a characterization in the special case of quasipermutations. Theorem 4.18 characterizes when the composition of two transformations gives a one-point singular. The special case when both transformations are one-point singulars is given in Theorem 4.23.

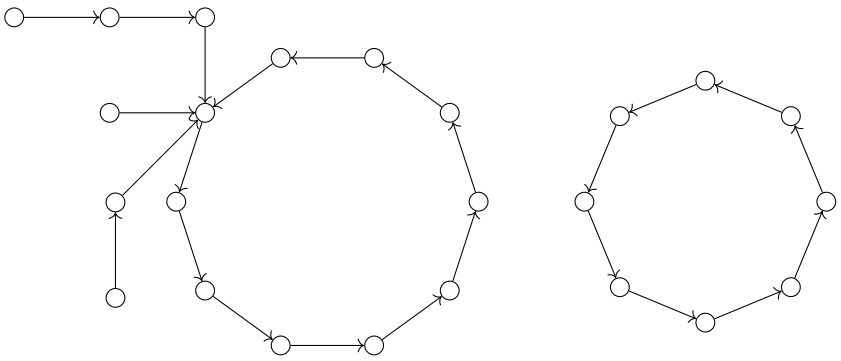


Fig. 4.2: Functional graph of a one-point singular transformation.

Proposition 4.14 *Let $t\colon X \to X$. Then the following are equivalent.*

1. *t is injective or a one-point singular.*
2. *for all $x, x', z, z' \in X$, if $x \neq x'$, $z \neq z'$, $t(x) = t(x')$, and $t(z) = t(z')$, then $t(x) = t(z)$.*

Proof. This follows directly from the definitions. □

Proposition 4.15 *Let $t\colon [n] \to [n]$. Then the following are equivalent.*

1. *t is a permutation or a one-point singular.*
2. $|t^{-1}(\mathrm{dupl}(t))| \leq \mathrm{def}(t) + 1$.
3. *t is a permutation or* $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$.
4. *There exists $y \in [n]$ such that* $|t^{-1}(y)| = \mathrm{def}(t) + 1$.
5. *t has kernel type* $(\mathrm{def}(t) + 1, 1, \ldots, 1)$.
6. *t has kernel type* $(k, 1, \ldots, 1)$ *for some* $k \geq 1$.

Proof. $1 \Rightarrow 2$: By Theorem 3.14, Item 1 we have $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + |\mathrm{dupl}(t)|$. Since, by assumption, $|\mathrm{dupl}(t)| \leq 1$, we obtain the claim.

$2 \Rightarrow 3$: We have $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$ or $|t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t)$. In the latter case, from Theorem 3.14, Item 1, we get $|\mathrm{dupl}(t)| = 0$. So t is a permutation.

$3 \Rightarrow 4$: Clearly, the implication is true for a permutation. Otherwise, by Theorem 3.14, Item 1 we have $\mathrm{def}(t) + |\mathrm{dupl}(t)| = |t^{-1}(\mathrm{dupl}(t))| = \mathrm{def}(t) + 1$. So, $|\mathrm{dupl}(t)| = 1$ and $|t^{-1}(y)| = \mathrm{def}(t) + 1$ for the unique $y \in \mathrm{dupl}(t)$.

$4 \Rightarrow 5$: Let $y' \in [n] \setminus \{y\}$. Then we have

$$\begin{aligned}
\mathrm{def}(t) &\geq |t^{-1}(\{y, y'\})| - 2 && \text{(Theorem 3.14, Item 2)}\\
&= |t^{-1}(y)| + |t^{-1}(y')| - 2 && (|t^{-1}(\{y, y'\})| = |t^{-1}(y)| + |t^{-1}(y')|)\\
&= \mathrm{def}(t) + |t^{-1}(y')| - 1. && (|t^{-1}(y)| = \mathrm{def}(t) + 1)
\end{aligned}$$

Hence $|t^{-1}(y')| \leq 1$.

$5 \Rightarrow 6$ & $6 \Rightarrow 1$: These implications are obvious. □

Proposition 4.16 *Let $t\colon [n] \to [n]$ be a quasipermutation. Then t is a permutation or a one-point singular if and only if there exists $y \in [n]$ with* $\mathrm{excl}(t) \subseteq t^{-1}(y)$.

Proof. By Theorem 4.4 we have $\mathrm{dupl}(t) = t(\mathrm{excl}(t))$. By assumption, we have $|\mathrm{dupl}(t)| \leq 1$. Now $|t(\mathrm{excl}(t))| \leq 1$ holds if and only if $t(\mathrm{excl}(t)) \subseteq \{y\}$ for some $y \in [n]$, which is $\mathrm{excl}(t) \subseteq t^{-1}(y)$. □

Remark 4.17 Theorem 4.16 might fail if t is not a quasipermutation. For example, let $t\colon \{1, 2, 3, 4\} \to \{1, 2, 3, 4\}$ with $t(1) = t(3) = 2$ and $t(2) = t(4) = 4$. Then $\mathrm{excl}(t) = \{1, 3\} \subseteq t^{-1}(2)$, but t has kernel type $(2, 2)$.

In the remainder of this section let $t, h\colon X \to X$ and put $Z = t(X) \setminus \mathrm{dupl}(t)$. Note that a restriction $h_{|Y}$ with $Y \subseteq t(X)$ need not be a transformation, as its image need not be contained in Y. Moreover, as $\mathrm{dupl}(t) \subseteq t(X)$, injectivity of $h_{|t(X)}$ implies $|h(\mathrm{dupl}(t))| = |\mathrm{dupl}(t)|$; in particular, $h(\mathrm{dupl}(t))$ is then a singleton set if and only if t is a one-point singular.

By Equation (3.11), it is easy to see that ht is a one-point singular precisely if $h(\mathrm{dupl}(t)) \subseteq \mathrm{dupl}(h_{|Z})$ where $\mathrm{dupl}(h_{|Z})$ is a singleton set, or $\mathrm{dupl}(h_{|Z}) \subseteq h(\mathrm{dupl}(t))$ where $h(\mathrm{dupl}(t))$ is a singleton set. By Equation (3.11), a similar statement holds with $t(X)$ in place of Z. In fact, since $\mathrm{dupl}(h_{|Z}) \subseteq \mathrm{dupl}(h_{|Y}) \subseteq \mathrm{dupl}(h_{|t(X)})$ and $\mathrm{dupl}(h_{|Y \setminus Z}) \subseteq h(\mathrm{dupl}(t))$ for $Z \subseteq Y \subseteq t(X)$ as $Y \setminus Z \subseteq \mathrm{dupl}(t)$, we can substitute any set Y with $Z \subseteq Y \subseteq t(X)$ for Z and obtain a valid characterization for ht being a one-point singular. Here, we replaced Z in each disjunct by the same set and the question might arise, under what circumstances can we replace Z in each disjunct independently,

or whether we can find similar statement where this is possible. For example, it can be shown that, for each $Z \subseteq Y_1, Y_2 \subseteq t(X)$, ht is a one-point singular if and only if t is injective and $\mathrm{dupl}(h_{|Z_1})$ a singleton, or $\mathrm{dupl}(h_{|Z_2}) \subseteq h(\mathrm{dupl}(t))$ where $h(\mathrm{dupl}(t))$ is a singleton. In the following we give a more detailed analysis driven by this question.

Proposition 4.18 *Let $t, h\colon X \to X$ and $Z \subseteq Y \subseteq t(X)$. Set*

$$A = h(\mathrm{dupl}(t)), \qquad D_S = \mathrm{dupl}(h_{|S}) \text{ for } S \subseteq t(X), \qquad B_Y = h(Z) \cap h(Y \setminus Z).$$

Then ht is a one-point singular if and only if exactly one of the following cases occurs.

1. *t is injective and D_Z is a singleton set.*
2. *$h_{|t(X)}$ is injective and t is a one-point singular.*

3_Y. *$B_Y \neq \emptyset$, $h_{|Z}$ is injective, and t is a one-point singular.*[3]

$3_{\neg Y}$. *$h_{|t(X)}$ is not injective, $B_Y = \emptyset$, $h_{|Z}$ is injective, and t is a one-point singular.*

4_Y. *$B_Y \neq \emptyset$ or $D_{Y\setminus Z} \neq \emptyset$, $h_{|Z}$ is injective, $|A| = 1$, and t is not a one-point singular.*

$4_{\neg Y}$. *$B_Y = D_{Y\setminus Z} = \emptyset$, $h_{|Z}$ is injective, $|A| = 1$, and t is not a one-point singular.*

5. *$D_Z = A$ and this set is a singleton set.*

Which cases can occur depends on Y. For $Y = t(X)$, where $Y \setminus Z = \mathrm{dupl}(t)$, the cases $3_{\neg Y}$ and $4_{\neg Y}$ cannot occur; writing 3 and 4 for 3_Y and 4_Y, we are left with the five mutually exclusive cases 1, 2, 3, 4, 5. For $Y = Z$ the cases 3_Y and 4_Y cannot occur, and for $Z \neq Y \neq t(X)$ only the cases 4_Y, $4_{\neg Y}$ and 5 can occur.

See Table 4.1 for a list of the cases with the above sets.

Proof. That the listed cases are mutually exclusive is easy to see. Next, we use $\mathrm{dupl}(ht) = A \cup D_Z$ from Equation (3.11) and

$$D_Y = D_Z \cup D_{Y\setminus Z} \cup B_Y \tag{4.23}$$

from Equation (3.4). As $Y \setminus Z \subseteq \mathrm{dupl}(t)$, we have $D_{Y\setminus Z} \subseteq A$ and $B_Y \subseteq A$.

Let ht be a one-point singular, that is, $|A \cup D_Z| = 1$. If t is injective, then $A = \emptyset$, so D_Z is a singleton set: case 1. Otherwise $A \neq \emptyset$, whence $|A| = 1$ and $D_Z \subseteq A$. If $D_Z = A$, we are in case 5; so let $D_Z = \emptyset$, that is, $h_{|Z}$ be injective. If t is a one-point singular, then injectivity of $h_{|t(X)}$ gives case 2, and otherwise we are in case 3_Y or $3_{\neg Y}$ according to whether $B_Y \neq \emptyset$ or not. If t is not a one-point singular, we are in case 4_Y or $4_{\neg Y}$ according to whether $B_Y \neq \emptyset$ or $D_{Y\setminus Z} \neq \emptyset$ holds or not.

Conversely, each case gives $|A \cup D_Z| = 1$: in case 1 we have $A = \emptyset$ with D_Z a singleton set, in case 5 we have $A \cup D_Z = A$, and in the other five we have $D_Z = \emptyset$ and $|A| = 1$, the latter in case 2 since t is a one-point singular.

For $Y = t(X)$ we have $Y \setminus Z = \mathrm{dupl}(t)$. In case $3_{\neg Y}$ then Equation (4.23) reads $D_{t(X)} = B_Y$, as $D_Z = \emptyset$ and $D_{\mathrm{dupl}(t)} = \emptyset$ for a singleton set $\mathrm{dupl}(t)$; but the left-hand side is non-empty as $h_{|t(X)}$ is not injective. In case $4_{\neg Y}$ we have $|\mathrm{dupl}(t)| \geq 2$, as $A \neq \emptyset$ and t is not a one-point singular, so h identifies the points of $\mathrm{dupl}(t)$ as $h(\mathrm{dupl}(t)) = A$ is a singleton set, giving $D_{Y\setminus Z} = A$. Both contradict the respective case hypotheses. For $Y = Z$ we have $Y \setminus Z = \emptyset$ and hence $B_Y = D_{Y\setminus Z} = \emptyset$, excluding 3_Y and 4_Y. For

[3] Note that $D_{Y\setminus Z} = \emptyset$ if t is a one-point singular, as then $Y \setminus Z$ is contained in the singleton set $\mathrm{dupl}(t)$.

$Z \neq Y \neq t(X)$, such a Y exists only if $\mathrm{dupl}(t)$ has at least two points, so t is neither injective nor a one-point singular, excluding the cases 1, 2, 3_Y and $3_{\neg Y}$. □

Table 4.1: The cases from Theorem 4.18 for a fixed set Y with $Z \subseteq Y \subseteq t(X)$; an entry A means equality with A. We have $D_Z \subseteq D_Y \subseteq D_{t(X)}$ and, if t is injective, these sets are all equal, see Theorem 4.19, and contained in A if $A \neq \emptyset$. With Equations (3.10) and (3.11) $A, D_Z, D_Y, D_{t(X)}$ each can contain at most one point. For $|\mathrm{dupl}(t)| \geq 2$ we have $D_{\mathrm{dupl}(t)} = A$, as h then identifies the points of $\mathrm{dupl}(t)$ since $|A| = 1$.

Case	$\lvert\mathrm{dupl}(t)\rvert$	A	D_Z	D_Y	$D_{t(X)}$
1	0	$\emptyset$	singleton	singleton	singleton
2	1	singleton	$\emptyset$	$\emptyset$	$\emptyset$
3_Y	1	singleton	$\emptyset$	A	A
$3_{\neg Y}$	1	singleton	$\emptyset$	$\emptyset$	A
4_Y	≥ 2	singleton	$\emptyset$	A	A
$4_{\neg Y}$	≥ 2	singleton	$\emptyset$	$\emptyset$	A
5	≥ 1	singleton	A	A	A

Table 4.2 lists further conditions, each equivalent to the disjunction of a subset of the cases from Theorem 4.18. Hence the disjunction of any set of rows whose marks cover every column yields a characterization for ht being a one-point singular.

Remark 4.19 We have

$$t \text{ injective and } D_Z \text{ a singleton} \Leftrightarrow t \text{ injective and } D_Y \text{ a singleton},$$
$$D_Z \subseteq A \text{ and } A \text{ a singleton} \Leftrightarrow D_Y \subseteq A \text{ and } A \text{ a singleton},$$
$$h_{|t(X)} \text{ injective and } A \text{ a singleton} \Leftrightarrow h_{|t(X)} \text{ injective and } t \text{ a one-point singular}.$$

For the first equivalence, note that t is injective if and only if $\mathrm{dupl}(t) = \emptyset$, which by $\mathrm{dupl}(t) \subseteq t(X)$ is equivalent to $Z = t(X)$; and then $Y = t(X)$ is the only admissible choice. The second follows since, by Equation (3.4), we have $\mathrm{dupl}(h_{|Y}) \setminus \mathrm{dupl}(h_{|Z}) \subseteq h(\mathrm{dupl}(t))$. The third is the last statement of the paragraph preceding Theorem 4.18.

In Table 4.2 the marks of the rows *t injective, D_Z singleton* and *$D_Z \subseteq A$, A singleton* cover every column. Hence ht is a one-point singular if and only if t is injective and D_Z is a singleton set, or $D_Z \subseteq A$ and A is a singleton set; and by the above equivalences Z can be replaced by any set Y between Z and $t(X)$, independently in each disjunct, since each holds for some such Y if and only if it holds for every such Y.

Next, consider the two conditions

1. $A \subseteq D_{t(X)}$ and $D_{t(X)}$ a singleton, or $h_{|Y}$ injective and t a one-point singular;
2. $A \subseteq D_Z$ and D_Z a singleton, or $h_{|Y}$ injective and t a one-point singular.

The first is equivalent to ht being a one-point singular, but the second is not, as by Table 4.2 the cases $3_Y, 4_Y$ and $4_{\neg Y}$ are not covered by it. For a concrete example, let $X = \{1, \ldots, 5\}$ with $t(1) = t(2) = 1, t(3) = t(4) = 2, t(5) = 5$ and $h(1) = h(2) = h(3) = 3$,

Table 4.2: The columns show the cases from Theorem 4.18, where "o.p.s." abbreviates "one-point singular". Each row gives a condition equivalent to the disjunction of the cases marked in it.

			Case 3		Case 4		
	1	2	3_Y	$3_{\neg Y}$	4_Y	$4_{\neg Y}$	5
t injective, D_Z singleton	x						
$h_{\mid Z}$ injective, A singleton		x	x	x	x	x	
$h_{\mid Y}$ injective, A singleton		x		x		x	
$h_{\mid t(X)}$ injective, A singleton		x					
$h_{\mid Z}$ injective, t o.p.s.		x	x	x			
$h_{\mid Y}$ injective, t o.p.s.		x		x			
$h_{\mid t(X)}$ injective, t o.p.s.		x					
$D_Y \subseteq A$, A singleton		x	x	x	x	x	x
$D_Z \subseteq A$, A singleton		x	x	x	x	x	x
$D_{t(X)} \subseteq A$, A singleton		x	x	x	x	x	x
$A \subseteq D_Z$, D_Z singleton	x						x
$A \subseteq D_Y$, D_Y singleton	x		x		x		x
$A \subseteq D_{t(X)}$, $D_{t(X)}$ singleton	x		x	x	x	x	x
$A = D_Z$, this set singleton							x
$A = D_Y$, this set singleton			x		x		x
$A = D_{t(X)}$, this set singleton			x	x	x	x	x

$h(4) = 4$, $h(5) = 5$. Then $\mathrm{dupl}(t) = \{1, 2\}$, $A = \{3\}$, $D_Z = \emptyset$, $D_{t(X)} = \{3\}$, and ht maps $1, 2, 3, 4$ to 3 and fixes 5. So $\mathrm{dupl}(ht) = \{3\}$ and ht is a one-point singular. Here $Z = \{5\}$ and $\mathrm{dupl}(t)$ has two points, so we are in case 4_Y for $Y = t(X)$ and in case $4_{\neg Y}$ for every other admissible Y. Since t is not a one-point singular and $A = \{3\} \not\subseteq \emptyset = D_Z$, both disjuncts of the second condition fail for every admissible Y.

Remark 4.20 If X is finite, then ht is a one-point singular if t is injective and h a one-point singular, since then $t(X) = X$ and so, with Equation (3.10), we have $\mathrm{dupl}(ht) = \mathrm{dupl}(h_{\mid t(X)}) = \mathrm{dupl}(h)$. However, this might fail for infinite X. For $X = \mathbb{N}$ with $t(n) = n + 1$, and $h(1) = h(2) = 1$ and $h(n) = n$ for $n \geq 3$, the transformation h is one-point singular with $\mathrm{dupl}(h) = \{1\}$ and $h(\mathrm{dupl}(t)) = \emptyset$ as t is injective; but ht maps 1 to 1 and n to $n + 1$ for $n \geq 2$ and hence is injective.

Remark 4.21 By Theorem 4.20, if h is a one-point singular, for finite X, we have $\mathrm{dupl}(ht) = \mathrm{dupl}(h)$ precisely if $h(\mathrm{dupl}(t)) \subseteq \mathrm{dupl}(h)$. The example from Theorem 4.20 also shows that this equivalence might fail for infinite X.

Remark 4.22 Let $\mathrm{dupl}(h) = \{c\}$ and $S \subseteq X$. Note that then $h_{\mid S}$ is injective if and only if $|S \cap h^{-1}(\mathrm{dupl}(h))| \leq 1$, as every pair of points identified by h is mapped to c.

Finally we spell out the case in which both of the transformations are assumed to be one-point singulars.

Corollary 4.23 *Let $t, h \colon X \to X$ be one-point singulars. Then the following are equivalent.*

1. *ht is a one-point singular.*
2. $h(\operatorname{dupl}(t)) = \operatorname{dupl}(h)$ *or* $h_{|t(X)}$ *is injective.*
3. *Exactly one of the following holds:*
   a. $h_{|t(X)}$ *is injective.*
   b. $h(Z) \cap h(\operatorname{dupl}(t)) \neq \emptyset$ *and* $h_{|Z}$ *is injective.*
   c. $\operatorname{dupl}(h_{|Z}) = h(\operatorname{dupl}(t))$.

Proof. Here $h(\operatorname{dupl}(t))$ is a singleton set, $\operatorname{dupl}(h_{|\operatorname{dupl}(t)}) = \emptyset$, and t is not injective. By Equation (3.10) and $\operatorname{dupl}(h_{|t(X)}) \subseteq \operatorname{dupl}(h)$, the set $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t)) \cup \operatorname{dupl}(h_{|t(X)})$ is a singleton set precisely if $\operatorname{dupl}(h_{|t(X)}) = \emptyset$ or $h(\operatorname{dupl}(t)) = \operatorname{dupl}(h)$, which gives 2. For 3 apply Theorem 4.18 for $Y = t(X)$: only case 2,3 and 5 apply and reduce to the stated conditions. □

In Theorem 4.23, Item 3 the former two conditions can be combined into the single condition that $h_{|Z}$ is injective, and the latter two conditions can be combined into $\operatorname{dupl}(h_{|t(X)}) = h(\operatorname{dupl}(t))$.

Figure 3.2 shows some cases where the composition of two one-point singulars results in a one-point singular.

4.5 Idempotent Transformations

We call $t\colon X \to X$ an *idempotent (transformation)* if it is idempotent w.r.t. functional composition, i.e., we have $t^2 = t$. For example, the idempotent transformations of defect one are precisely the simple idempotents introduced in Section 4.3.

First, in Theorem 4.24, we list several equivalent conditions to idempotency, and Theorem 4.25 collects some consequences. Then Theorem 4.26 characterizes idempotency of the composition, and Theorems 4.27, 4.29, and 4.30 give characterizations under additional assumptions concerning the idempotency of the given functions. Theorem 4.32 gives a formula for the inverse image of the duplicate set of the composition involving at least one idempotent, stating that it equals a union of inverse images of the individual transformations.

Proposition 4.24 *Let* $t\colon X \to X$. *Then the following are equivalent.*

1. *t is idempotent.*
2. $t(x) \in t^{-1}(t(x))$ *for all* $x \in X$.
3. $t^{-1}(y) \neq \emptyset$ *implies* $y \in t^{-1}(y)$ *for* $y \in X$.
4. $t(X) \subseteq \operatorname{fix}(t)$.
5. $\operatorname{move}(t) \subseteq \operatorname{excl}(t)$.
6. $t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t)$.

Proof. The equivalences $1 \Leftrightarrow 2 \Leftrightarrow 3 \Leftrightarrow 4$ follow readily.
$4 \Rightarrow 5$: This follows as $\operatorname{move}(t) = X \setminus \operatorname{fix}(t) \subseteq X \setminus t(X) = \operatorname{excl}(t)$.
$5 \Rightarrow 6$: Here $t(\operatorname{move}(t)) \subseteq t(X) = X \setminus \operatorname{excl}(t) \subseteq X \setminus \operatorname{move}(t) = \operatorname{fix}(t)$.
$6 \Rightarrow 4$: As $t(\operatorname{fix}(t)) = \operatorname{fix}(t)$ and, by assumption, $t(\operatorname{move}(t)) \subseteq \operatorname{fix}(t)$, we find

$$t(X) = t(\mathrm{fix}(t) \cup \mathrm{move}(t)) = t(\mathrm{fix}(t)) \cup t(\mathrm{move}(t)) \subseteq \mathrm{fix}(t).$$

□

Corollary 4.25 *Suppose $t\colon X \to X$ is idempotent. Then we have the following:*

1. $t(X) = \mathrm{fix}(t)$,
2. $\mathrm{move}(t) = \mathrm{excl}(t)$,
3. $\mathrm{dupl}(t) = t(\mathrm{move}(t))$ *(equivalently, $t(x) \notin \mathrm{dupl}(t)$ implies $t(x) = x$),*
4. $\mathrm{dupl}(t) \subseteq \mathrm{fix}(t)$ *so that* $\mathrm{dupl}(t) = t(\mathrm{dupl}(t))$,
5. $t^{-1}(\mathrm{dupl}(t)) = \mathrm{move}(t) \uplus \mathrm{dupl}(t)$,
6. $t(A) \subseteq A$ *for* $A \subseteq X$ *with* $\mathrm{dupl}(t) \subseteq A$.

Proof. 1: This follows from Item 4 of Theorem 4.24 and Equation (3.24).

2: This follows from Item 5 of Theorem 4.24 and Equation (3.25).

3: By Item 6 of Theorem 4.24 and Equation (3.22) we have the following inclusions, which give the claim:

$$t(\mathrm{move}(t)) \subseteq \mathrm{fix}(t) \cap t(\mathrm{move}(t)) \subseteq \mathrm{dupl}(t) \subseteq t(\mathrm{move}(t)).$$

4: This follows as $\mathrm{dupl}(t) \subseteq t(X) \subseteq \mathrm{fix}(t)$ by Item 4 of Theorem 4.24.

5: By Item 3 we have $\mathrm{move}(t) \subseteq t^{-1}(\mathrm{dupl}(t))$, and $\mathrm{dupl}(t) \subseteq t^{-1}(\mathrm{dupl}(t))$ by Item 4. Conversely, let $x \in t^{-1}(\mathrm{dupl}(t))$ with $x \notin \mathrm{move}(t)$. Then $x = t(x) \in \mathrm{dupl}(t)$. The union is disjoint as $\mathrm{dupl}(t) \subseteq \mathrm{fix}(t)$.

6: For $x \in A$ we have $t(x) = x \in A$ or, by Item 3, $t(x) \in \mathrm{dupl}(t) \subseteq A$. □

Next, we investigate when the composition of two transformations gives an idempotent transformation.

Proposition 4.26 *Let $t, h\colon X \to X$. Then the following are equivalent.*

1. *ht is idempotent.*
2. $\mathrm{move}(t) \cap h(t(X)) \subseteq \mathrm{fix}(ht)$ *and* $\mathrm{fix}(t) \cap h(t(X)) \subseteq \mathrm{fix}(h)$.
3. $t(y) \in h^{-1}(y)$ *for all* $y \in h(t(X))$.
4. $h(\mathrm{move}(h) \cap t(X)) \subseteq \mathrm{fix}(ht)$ *and* $\mathrm{fix}(h) \cap t(X) \subseteq \mathrm{fix}(ht)$.

Proof. 1 $\Rightarrow$ 2: By Theorem 4.24 we have $h(t(X)) \subseteq \mathrm{fix}(ht)$, which gives the first inclusion and, together with Equation (3.26), also the second.

2 $\Rightarrow$ 3: Let $y \in h(t(X))$. If $y \in \mathrm{move}(t)$, then $h(t(y)) = y$ by the first assumption. Otherwise $y \in \mathrm{fix}(t)$, so $h(y) = y$ by the second assumption and hence $h(t(y)) = h(y) = y$.

3 $\Rightarrow$ 4: Let $x \in X$. Applying 3 to $y = h(t(x))$ gives $h(t(x)) \in \mathrm{fix}(ht)$, and applying it to $y = t(x)$ in case $t(x) \in h(t(X))$ gives $t(x) \in \mathrm{fix}(ht)$. The first covers $h(\mathrm{move}(h) \cap t(X))$, the second $\mathrm{fix}(h) \cap t(X)$, as $t(x) \in \mathrm{fix}(h)$ implies $t(x) = h(t(x)) \in h(t(X))$.

4 $\Rightarrow$ 1: With $t(X) = (\mathrm{move}(h) \cap t(X)) \uplus (\mathrm{fix}(h) \cap t(X))$ we get

$$h(t(X)) = h(\mathrm{move}(h) \cap t(X)) \cup (\mathrm{fix}(h) \cap t(X)) \subseteq \mathrm{fix}(ht),$$

so ht is idempotent by Theorem 4.24. □

Corollary 4.27 *Let $t, h\colon X \to X$.*

1. *For idempotent h, ht is idempotent if and only if* $\operatorname{move}(t) \cap h(t(X)) \subseteq \operatorname{fix}(ht)$.
2. *For idempotent t, ht is idempotent if and only if* $h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$.

Proof. 1: For idempotent h we have $h(t(X)) \subseteq h(X) \subseteq \operatorname{fix}(h)$ by Theorem 4.24, so the second condition of Item 2 of Theorem 4.26 always holds.

2: For idempotent t we have $t(X) \subseteq \operatorname{fix}(t)$ by Theorem 4.24, so $\operatorname{fix}(h) \cap t(X) \subseteq \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$ by Equation (3.26), and the second condition of Item 4 of Theorem 4.26 always holds. □

Remark 4.28 By Item 1 of Theorem 4.27, for idempotent h the composition ht is idempotent if $h(t(X)) \subseteq \operatorname{fix}(t)$, as then $\operatorname{move}(t) \cap h(t(X)) = \emptyset$. This also follows directly with Item 4 of Theorem 4.24 since we have $h(t(X)) \subseteq \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$. The condition is not necessary: if $h(x) = c$ for all $x \in X$ and $t(c) \neq c$, then h and ht are idempotent and $h(t(X)) \cap \operatorname{fix}(t) = \emptyset$.

Corollary 4.29 *Let $t, h\colon X \to X$ be idempotent. Then ht is idempotent if and only if* $\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$.[4]

Proof. If ht is idempotent, then $h(t(X)) \subseteq \operatorname{fix}(ht)$ by Theorem 4.24, and the set in question is contained in $h(t(X))$.

Conversely, as t is idempotent we have $\operatorname{fix}(h) \cap t(X) = \operatorname{fix}(h) \cap \operatorname{fix}(t) \subseteq \operatorname{fix}(ht)$ by Equation (3.26), and as h is idempotent we have $\operatorname{fix}(t) \cap h(t(X)) \subseteq \operatorname{fix}(t) \cap \operatorname{fix}(h) \subseteq \operatorname{fix}(ht)$. With $t(X) = (\operatorname{move}(h) \cap t(X)) \uplus (\operatorname{fix}(h) \cap t(X))$ and $X = \operatorname{move}(t) \uplus \operatorname{fix}(t)$ we obtain

$$\begin{aligned} h(t(X)) &= (\operatorname{move}(t) \cap h(t(X))) \cup (\operatorname{fix}(t) \cap h(t(X))) \\ &= (\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X))) \\ &\quad \cup (\operatorname{move}(t) \cap (\operatorname{fix}(h) \cap t(X))) \\ &\quad \cup (\operatorname{fix}(t) \cap h(t(X))). \end{aligned}$$

where we used $h(\operatorname{fix}(h) \cap t(X)) = \operatorname{fix}(h) \cap t(X)$. All three sets are contained in $\operatorname{fix}(ht)$, the first by assumption, so ht is idempotent by Theorem 4.24. □

Corollary 4.30 *Let $t, h\colon X \to X$ be idempotent. Then ht is idempotent if one of the following is true:*

1. $\operatorname{dupl}(h) \subseteq \operatorname{fix}(ht)$*, i.e., $h(t(y)) = y$ for all $y \in \operatorname{dupl}(h)$,*
2. $\operatorname{excl}(t) \subseteq \operatorname{fix}(ht)$*,*
3. *$ht = th$.*

Proof. 1: As h is idempotent, we have $h(\operatorname{move}(h) \cap t(X)) \subseteq h(\operatorname{move}(h)) = \operatorname{dupl}(h)$ by Item 3 of Theorem 4.25, which is contained in $\operatorname{fix}(ht)$ by assumption. Thus the claim follows from Item 2 of Theorem 4.27.

2: With Theorem 4.24 we find $\operatorname{move}(t) \subseteq \operatorname{excl}(t) \subseteq \operatorname{fix}(ht)$, so Theorem 4.29 gives the claim.

3: We have $h(t(h(t(x)))) = h(h(t(t(x)))) = h(t(x))$ for $x \in X$. □

[4] Equivalently, for $x \in X$, $h(t(x)) \neq t(x)$ and $t(h(t(x))) \neq h(t(x))$ imply $h(t(x)) = h(t(h(t(x))))$.

Proposition 4.31 *Let $t, h\colon X \to X$ be idempotent. Then the following are equivalent.*

1. $ht = t$.
2. $t(X) \subseteq \mathrm{fix}(h)$.
3. $t(X) \cap h^{-1}(\mathrm{dupl}(h)) \subseteq \mathrm{dupl}(h)$.

Proof. $1 \Rightarrow 2$: By assumption we have $h(t(x)) = t(x)$ for all $x \in X$.

$2 \Rightarrow 3$: For $t(x) \in h^{-1}(\mathrm{dupl}(h))$ we have $t(x) = h(t(x)) \in \mathrm{dupl}(h)$.

$3 \Rightarrow 1$: Suppose $h(t(x)) \neq t(x)$ for some $x \in X$. Then $h(t(x)) \in \mathrm{dupl}(h)$ by Item 3 of Theorem 4.25, so $t(x) \in t(X) \cap h^{-1}(\mathrm{dupl}(h))$, but $t(x) \notin \mathrm{dupl}(h)$ as h fixes every element of $\mathrm{dupl}(h)$ by Item 4 of Theorem 4.25. Thus Item 3 fails. □

Next, we refine Theorem 3.6 for the case that one transformation is idempotent.

Proposition 4.32 *Let $t\colon X \to X$ be idempotent and $h\colon X \to Y$. Set $A = t^{-1}(\mathrm{dupl}(t))$, $B = h^{-1}(\mathrm{dupl}(h_{|t(X)}))$ and $C = h^{-1}(\mathrm{dupl}(ht))$. Then*

$$\begin{aligned}(ht)^{-1}(\mathrm{dupl}(ht)) &= A \cup B = A \cup C = \mathrm{move}(t) \cup C \\ &= A \cup (B \cap t(X)) = A \cup (C \cap t(X)) = \mathrm{move}(t) \uplus (C \cap t(X)).\end{aligned}$$

Proof. Let D denote the left-hand side, and note that $D = t^{-1}(C)$. By Items 1, 2, and 5 of Theorem 4.25, we have $X \setminus t(X) = \mathrm{move}(t) \subseteq A$, and t fixes every point in $t(X)$, so

$$t^{-1}(S) \cap t(X) = S \cap t(X) \quad \text{for all } S \subseteq X. \tag{$*$}$$

Hence

$$\begin{aligned}
A \cup (B \cap t(X)) &= A \cup B && (X \setminus t(X) \subseteq A) \\
&= A \cup t^{-1}(B) && ((*) \text{ and } X \setminus t(X) \subseteq A) \\
&= D && (\text{Theorem 3.6}) \\
&= \mathrm{move}(t) \cup t^{-1}(C) && (\mathrm{move}(t) \subseteq A \subseteq D = t^{-1}(C)) \\
&= \mathrm{move}(t) \uplus (C \cap t(X)) && ((*) \text{ and } X \setminus t(X) = \mathrm{move}(t)) \\
&= \mathrm{move}(t) \cup C && (X \setminus t(X) = \mathrm{move}(t)) \\
&= A \cup C && (\mathrm{move}(t) \subseteq A \subseteq \mathrm{move}(t) \cup C) \\
&= A \cup (C \cap t(X)). && (X \setminus t(X) \subseteq A)
\end{aligned}$$

As the chain passes through D, this shows the claim. □

In Theorem 4.32, $\mathrm{dupl}(h_{|t(X)})$ cannot be replaced with $\mathrm{dupl}(h)$, see Theorem 3.7, where f is idempotent.

4.6 Idempotent One-Point Singulars

Theorem 4.33 lists several characterizations for idempotents whose duplicate set contains at most one point, and Theorem 4.34 provides an additional characterization for

finite domains. Then, Theorem 4.35, which we need in Part III, gives conditions when the composition of two idempotent one-point singulars yields an idempotent one-point singular. As a consequence, we obtain that if the composition of two simple idempotents yields a simple idempotent, then it equals one of the two original simple idempotents.

A simple idempotent is an idempotent one-point singular. Rystsov [535] introduced simple idempotents and showed [535, Lem. 1] that a simple idempotent $t\colon [n] \to [n]$ on a finite set fulfills $t^{-1}(A) \subseteq A$ or $A \subseteq t^{-1}(A)$ for every $A \subseteq [n]$. In Theorem 4.33, Item 10 we extend this result by characterizing the transformations that fulfill this equation as precisely the idempotents whose duplicate set contains at most one point.

Theorem 4.33 *Let $t\colon X \to X$ with X non-empty. Then the following are equivalent.*

1. *t is an idempotent one-point singular or equals the identity transformation.*
2. *$t = [a_{m-1} \mapsto a_m] \cdots [a_1 \mapsto a_m]$ for $\operatorname{move}(t) = \{a_1, \ldots, a_m\}$ and $\operatorname{dupl}(t) = \{a_m\}$ or $t = \mathrm{id}_X$.*
3. *$\operatorname{dupl}(t)$ contains at most one point and t is idempotent.*
4. *$\operatorname{dupl}(t)$ contains at most one point and $|t^{-1}(z)| = 1$ implies $t^{-1}(z) = \{z\}$ for all $z \in X$.*
5. *There exists $y \in X$ such that for all $z \in X$ we have $t^{-1}(z) = \{z\}$ or $z \in t^{-1}(y)$.*
6. *There exists $y \in X$ such that $x \in t^{-1}(z)$ implies $t(x) = x$ for $z, x \in X$ with $z \neq y$.*
7. *For all $x, z \in X$ with $t(x) \neq x$ and $t(z) \neq z$ we have $t(x) = t(z)$, i.e., $\operatorname{move}(t) \subseteq t^{-1}(y)$ for some $y \in X$.*
8. *There exists $y \in X$ such that for all $z \in X$ we have $t^{-1}(z) \subseteq \{z\}$ or $z \in t^{-1}(y)$.*
9. *There exists $y \in X$ such that, for all $z \in t(X)$, we have*

$$t^{-1}(z) = \begin{cases} \operatorname{excl}(t) \cup \{y\} & \text{if } z = y, \\ \{z\} & \text{otherwise.} \end{cases}$$

10. *For all $A \subseteq X$ we have $A \subseteq t^{-1}(A)$ or $t^{-1}(A) \subseteq A$.*

Proof. $1 \Rightarrow 2$ For $t \neq \mathrm{id}_X$, let $\operatorname{dupl}(t) = \{a_m\}$ and $\operatorname{move}(t) = \{a_1, \ldots, a_{m-1}\}$. Then we have $t = [a_{m-1} \mapsto a_m] \cdots [a_1 \mapsto a_m]$.

$2 \Rightarrow 3$: This is clear.

$3 \Rightarrow 4$: By assumption, $\operatorname{dupl}(t)$ contains at most one point. By idempotency, the rest of the claim follows from Theorem 4.24, Item 3.

$4 \Rightarrow 5$: For $\operatorname{dupl}(t) \neq \emptyset$, let y be the unique point in $\operatorname{dupl}(t)$, otherwise, let $y \in X$ be any point. Next, let $z \in X$ with $t(z) \neq y$. Then as $t(z) \notin \operatorname{dupl}(t)$ we have $t^{-1}(t(z)) = \{z\}$. By assumption $|t^{-1}(t(z))| = 1$ gives $t^{-1}(t(z)) = \{t(z)\}$. Thus $t(z) = z$, which gives $t^{-1}(z) = \{z\}$.

$5 \Rightarrow 6$: Let $x, z \in X$ with $z \neq y$ and suppose $x \in t^{-1}(z)$. Since $t^{-1}(z) = \{z\}$, we get $x = z$ and so $t(x) = x$.

$6 \Rightarrow 7$: Suppose $t(x) \neq x$ and $t(z) \neq z$ for $x, z \in X$. Since $x \in t^{-1}(t(x))$ and $z \in t^{-1}(t(z))$, we find $t(x) = t(z) = y$.

$7 \Rightarrow 8$: We can assume $t(x) \neq x$ for some $x \in X$, as otherwise t equals the identity transformation and the claim obviously holds. Then set $y = t(x)$. Next, consider $z \in X$ with $z \notin t^{-1}(y)$. By assumption, $t(z) = z$. Now, assume $t(z) = t(z')$ for some $z' \in X$

with $z \neq z'$. Then $t(z') \neq z'$ yields $t(z') = y$. So, this is not possible and we have $t^{-1}(z) = \{z\}$.

$6 \Rightarrow 8$: Let $z \in X$ with $z \neq y$ and assume $z \notin t^{-1}(y)$. Now, for $x \in X$ with $x \in t^{-1}(z)$, by assumption, we get $z = t(x) = x$ and so $t^{-1}(z) \subseteq \{z\}$.

$8 \Rightarrow 9$: Let $z \in t(X)$ and suppose $z \neq y$. We have $z = t(x)$ for some $x \in X$. Thus $x \in t^{-1}(z)$. Since $z \neq y$, by assumption $t^{-1}(z) \subseteq \{z\}$ and so $z = x$. This gives $t^{-1}(z) = \{z\}$. Since $t^{-1}(z) = \{z\}$ for $z \in t(X)$ with $z \neq y$, and the union of the inverse images equals X, we have that the remaining points $\operatorname{excl}(t) \cup \{y\}$ must be in the inverse image of $t^{-1}(y)$, and since this union is pairwise disjoint, no other point is in $t^{-1}(y)$ and so $t^{-1}(y) = \operatorname{excl}(t) \cup \{y\}$.

$9 \Rightarrow 10$: Let $A \subseteq X$. Then

$$t^{-1}(A) = t^{-1}(A \cap t(X)) = \begin{cases} A \cap t(X) & \text{if } y \notin A, \\ \operatorname{excl}(t) \cup A & \text{if } y \in A. \end{cases}$$

So, $t^{-1}(A) \subseteq A$ for $y \notin A$ and $A \subseteq t^{-1}(A)$ for $y \in A$.

$10 \Rightarrow 1$: For $z \in X$, consider $A = \{t(z)\}$. Then $A \subseteq t^{-1}(A)$ or $t^{-1}(A) \subseteq A$ and as $t^{-1}(A) \neq \emptyset$, we get $t(z) \in t^{-1}(t(z))$ for both cases. Hence, by Theorem 4.24, Item 2, t is idempotent. Then, for $\operatorname{dupl}(t) = \emptyset$ we have $t = \mathrm{id}_X$. Next, for $\operatorname{dupl}(t) \neq \emptyset$, by Theorem 4.25, Item 5 there exists $y \in \operatorname{dupl}(t)$ such that $x \in t^{-1}(y) \cap \operatorname{excl}(t)$ for some $x \in X$. Let $y' \in \operatorname{dupl}(t)$ and set $A = \{x, y'\}$. Then $t^{-1}(A) = t^{-1}(y')$ and, as $t^{-1}(y')$ contains at least two points, for $A \subseteq t^{-1}(y')$ or $t^{-1}(y') \subseteq A$ we get $x \in t^{-1}(y')$ in both cases. Now $x \in t^{-1}(y) \cap t^{-1}(y')$ gives $y = y'$ and so t is a one-point singular. □

Regarding Theorem 4.33, Item 2, inductively, from Equation (4.9) (but it is also easy to verify directly) we get the following identity:

$$[a_{m-1} \mapsto a_m] \cdots [a_2 \mapsto a_3][a_1 \mapsto a_2] = [a_{m-1} \mapsto a_m] \cdots [a_2 \mapsto a_m][a_1 \mapsto a_m]. \tag{4.24}$$

Next, we state a new characterization of idempotents on finite sets whose duplicate set contains at most one point.

Proposition 4.34 *Let $t\colon [n] \to [n]$. Then $t = \mathrm{id}_{[n]}$ or t is an idempotent one-point singular if and only if for all $A \subseteq [n]$ we have that $|t^{-1}(A)| = |A|$ implies $t^{-1}(A) = A$.*

Proof. We can assume $t \neq \mathrm{id}_{[n]}$. First, suppose t is an idempotent one-point singular. Let $A \subseteq [n]$ be such that $|t^{-1}(A)| = |A|$. By Theorem 4.33, Item 10 we have $t^{-1}(A) \subseteq A$ or $A \subseteq t^{-1}(A)$. Finiteness implies $t^{-1}(A) = A$ in both cases.

Conversely, consider a set $A \subseteq [n]$. Since $\operatorname{def}(t) + |A \cap t([n])| \geq |t^{-1}(A \cap t([n]))|$ by Theorem 3.14, Item 2, we can choose $B \subseteq \operatorname{excl}(t)$ such that $|B| + |A \cap t([n])| = |t^{-1}(A \cap t([n]))|$ where we let $B \subseteq A \cap \operatorname{excl}(t)$ if $|B| \leq |A \cap \operatorname{excl}(t)|$ and $A \cap \operatorname{excl}(t) \subseteq B$ otherwise. Since $B \subseteq \operatorname{excl}(t)$ we have $|B| + |A \cap t([n])| = |B \cup (A \cap t([n]))|$ and $t^{-1}(A \cap t([n])) = t^{-1}((A \cap t([n])) \cup B)$. Thus $|t^{-1}((A \cap t([n])) \cup B)| = |(A \cap t([n])) \cup B)|$ and so $t^{-1}((A \cap t([n])) \cup B) = (A \cap t([n])) \cup B$. Since $t^{-1}(A) = t^{-1}(A \cap t([n]))$ and $A = (A \cap t([n])) \cup (A \cap \operatorname{excl}(t))$, we find $t^{-1}(A) \subseteq A$ for $B \subseteq A \cap \operatorname{excl}(t)$ and $A \subseteq t^{-1}(A)$ for $A \cap \operatorname{excl}(t) \subseteq B$. Therefore, by Theorem 4.33, Item 10, it follows that t is an idempotent one-point singular. □

Proposition 4.35 *Let $t, h\colon X \to X$ be idempotent one-point singulars. Then the following are equivalent.*

1. *ht is an idempotent one-point singular.*
2. $\operatorname{move}(t) \cap h(\operatorname{move}(h) \cap t(X)) \subseteq \operatorname{fix}(ht)$ *and* $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$.[5]
3. $h(\operatorname{move}(h) \cap t(X)) \subseteq h(\operatorname{dupl}(t))$.
4. $t(X) \subseteq \operatorname{fix}(h)$ *or* $\operatorname{dupl}(h) = h(\operatorname{dupl}(t))$.[6]
5. *We have $ht = t$ or with* $\operatorname{move}(t) \cup \operatorname{move}(h) = \{a_1, \dots, a_{m-1}\}$ *and* $\operatorname{dupl}(h) = \{a_m\}$ *for distinct $a_1, \dots, a_m \in X$ we have*

$$ht = [a_1 \mapsto a_m] \cdots [a_{m-1} \mapsto a_m].$$

Proof. 1 $\Rightarrow$ 2: The first inclusion follows from Theorem 4.29. Since ht and t are one-point singulars, we must have $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$.

2 $\Rightarrow$ 3: Let $x \in X$ be such that $h(t(x)) \neq t(x)$. If $t(h(t(x))) \neq h(t(x))$, then, by assumption, $h(t(h(t(x)))) = h(t(x))$ and thus $h(t(x)) \in \operatorname{dupl}(h_{|t(X)})$. If $t(h(t(x))) = h(t(x))$, then $h(t(x)) \in t(X)$. By idempotency of h, we find $h(h(t(x))) = h(t(x))$. Thus $h(t(x)) \in \operatorname{dupl}(h_{|t(X)})$. Since $\operatorname{dupl}(h_{|t(X)}) \subseteq \operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$ we obtain the claim.

3 $\Rightarrow$ 4: Let $t(x) \in \operatorname{move}(h)$. Since h is an idempotent, we have $h(\operatorname{move}(h)) = \operatorname{dupl}(t)$. Since it is a one-point singular, we find $\operatorname{dupl}(h) = \{h(t(x))\}$. By assumption, we have $h(t(x)) \in h(\operatorname{dupl}(t))$. Since t is a one-point singular we get $h(\operatorname{dupl}(t)) = \{h(t(x))\}$. Thus $\operatorname{dupl}(h) = h(\operatorname{dupl}(t))$.

4 $\Rightarrow$ 5: Suppose $ht \neq t$, i.e., $t(X) \not\subseteq \operatorname{fix}(h)$. Then, by Item 4, we have $\operatorname{dupl}(t) \subseteq h^{-1}(a_m)$. Let $x \in \operatorname{move}(t)$. Then $t(x) \in \operatorname{dupl}(t)$ since t is idempotent and so $h(t(x)) = a_m$. Next, let $x \in \operatorname{fix}(t)$ and $x \in \operatorname{move}(h)$. Then $h(t(x)) = h(x) = a_m$ since h is idempotent. Next, for $x \in \operatorname{fix}(t) \cap \operatorname{fix}(h)$ we have $h(t(x)) = x$. Thus, precisely the points from $\operatorname{move}(t) \cup \operatorname{move}(h)$ are mapped to a_m and all other points are fixed. This is precisely the transformation described by $[a_1 \mapsto a_m] \cdots [a_{m-1} \mapsto a_m]$.[7]

5 $\Rightarrow$ 1: Clearly, in each case, ht is an idempotent one-point singular. □

Corollary 4.36 *Let $t, h\colon X \to X$ be idempotent one-point singulars such that ht is an idempotent one-point singular. Then*

$$(ht)^{-1}(\operatorname{dupl}(ht)) = \begin{cases} t^{-1}(\operatorname{dupl}(t)) & \text{if } t(X) \subseteq \operatorname{fix}(h), \\ t^{-1}(\operatorname{dupl}(t)) \cup h^{-1}(\operatorname{dupl}(h)) & \text{otherwise,} \end{cases}$$

where we also have $t^{-1}(\operatorname{dupl}(t)) \cup h^{-1}(\operatorname{dupl}(h)) = t^{-1}(\operatorname{dupl}(t)) \cup (t(X) \cap h^{-1}(\operatorname{dupl}(h)))$.

Proof. First, suppose $t(X) \subseteq \operatorname{fix}(h)$. Then $ht = t$ and $\operatorname{dupl}(ht) = \operatorname{dupl}(t)$, which gives the equality for this case.

---

[5] Recall that $\operatorname{dupl}(h_{|t(X)}) \subseteq h(\operatorname{dupl}(t))$ is equivalent to $\operatorname{dupl}(ht) = h(\operatorname{dupl}(t))$ by Equation (3.10).

[6] This is equivalent to the statement that $\operatorname{move}(h) \cap t(X) \neq \emptyset$ implies $\operatorname{dupl}(h) = h(\operatorname{dupl}(t))$.

[7] Alternatively, the equation for ht follows by deriving $\operatorname{dupl}(ht) = \operatorname{dupl}(h)$, substituting this into Theorem 4.32, and noting that all points outside of $t^{-1}(\operatorname{dupl}(t)) \cup h^{-1}(\operatorname{dupl}(h)) = \operatorname{move}(t) \cup \operatorname{move}(h) \cup \{a_m\}$ are fixed by ht.

Next, suppose $t(X) \not\subseteq \operatorname{fix}(h)$. By Theorem 4.35, we have $\operatorname{dupl}(h) = \operatorname{dupl}(ht)$. Then the claim follows from Theorem 4.32. □

Remark 4.37 Figure 3.2 shows two idempotent one-point singulars whose composition results in an idempotent one-point singular. The first two pictures on the left show the case $t(X) \subseteq \operatorname{fix}(h)$ (see also Theorem 4.31), the rightmost picture shows the case $\operatorname{dupl}(h) = h(\operatorname{dupl}(t))$.

Corollary 4.38 *Let $t, h\colon X \to X$ be two simple idempotents. If ht is a simple idempotent, then $ht \in \{t, h\}$.*

Proof. By Theorem 4.35, Item 5, if ht is a simple idempotent, we must have $ht = t$ or $\operatorname{move}(t) \cup \operatorname{move}(h) = \{a\}$ and $\operatorname{dupl}(h) = \{b\}$, which gives $h = [a \mapsto b]$, and $ht = [a \mapsto b]$. □

Remark 4.39 Several simple idempotents composed can result in new simple idempotents, a result that fails for two simple idempotents by Theorem 4.38. For example, consider $t_1 = [1 \mapsto 2]$, $t_2 = [2 \mapsto 3]$, and $t_3 = [3 \mapsto 1]$. Then $t_1t_2t_3t_1t_2t_3 = [1 \mapsto 3]$ where $t_1t_2t_3t_1t_2 = [1 \mapsto 3 \mapsto 2]$.

Remark 4.40 Let $t, h\colon X \to X$ such that t (resp. h) is a simple idempotent moving a to the distinct point b. Then a simple case yields that ht is a simple idempotent if and only if $\operatorname{move}(h) \subseteq \{a, b\}$ and $h(b) \in \{a, b\}$ (resp. $\operatorname{move}(t) \subseteq \{a, b\}$ and $t(b) \in \{a, b\}$). In this case we have that ht equals $[a \mapsto b]$ or $[b \mapsto a]$ (resp. $[a \mapsto b]$ for $t(a) \in \{a, b\}$ and $[a \mapsto t(a)]$ for $t(a) \notin \{a, b\}$). The next example illustrates some cases.

Example 4.41 Let $t, h\colon X \to X$ for $X = \{1, 2, 3\}$ with $t(1) = t(2) = 2$, $t(3) = 1$, and $h(1) = h(3) = 3$, $h(2) = 1$. Then we have

1. $[1 \mapsto 3]t = [1 \mapsto 2]$.
2. $h[1 \mapsto 2] = [2 \mapsto 1]$.
3. $[1 \mapsto 3][1 \mapsto 2] = [1 \mapsto 2]$.

4.7 Discussion

Most of the present chapter recaps known results. However, the results on one-point singulars, for example, Theorems 4.15, 4.33, and 4.34, and those on compositions of our classes of transformations, are, to the best of our knowledge, new. One-point singulars were introduced in [524], but this type of transformations has been investigated before, for example in Araújo & Cameron [34] on finite sets where they are referred to via their kernel type $(k, 1, \ldots, 1)$ with $k \geq 2$. We are also not aware of any studies that investigated when the composition of two transformations results in a one-point singular, or an idempotent. Idempotent transformations are discussed in [236, Sec. 2.7] where Theorem 4.24, Item 4 is stated and it is shown that $\mathcal{T}_n$ contains

$$\sum_{k=1}^{n} \binom{n}{k} k^{n-k}$$

idempotent transformations. In [75] algorithmic and complexity theoretical questions of deciding whether all transformations of a given transformation semigroup are idempotent were investigated.

A classical result by Howie [323] states that the set of all simple idempotents on a finite set generates the transformation semigroup consisting of all non-permutations. Gomes and Howie [254] proved that $\binom{n}{2}$ idempotents are sufficient and necessary to generate this set.

Chapter 5
Permutation Groups

"It was at this point that Wigner, Hund, Heitler, and Weyl entered the picture with their 'Gruppenpest': the pest of the group theory... The authors of the 'Gruppenpest' wrote papers which were incomprehensible to those like me who had not studied group theory, in which they applied these theoretical results to the study of the many electron problem. The practical consequences appeared to be negligible, but everyone felt that to be in the mainstream one had to learn about it. Yet there were no good texts from which one could learn group theory. It was a frustrating experience, worthy of the name of a pest."

—John Slater (1900–1976)

The set $\mathcal{S}_X$ of all permutations on a set X with function composition and inversion given by $t^{-1}\colon X \to X$ for $t \in \mathcal{S}_X$ is a group, called the *(full) symmetric group on* X. A *permutation group* G (over the *(permutation) domain* X) is a subgroup of $\mathcal{S}_X$. Here, we investigate permutation groups.

Section 5.1 introduces basic notions. In Section 5.2 we collect statements related to transitive permutation groups. The relation between permutation groups and (abstract) groups is discussed in Section 5.3. Section 5.4 investigates the group of units in transformation semigroups. Lastly, Section 5.5 points out some relations between permutation groups and graphs.

5.1 Basic Notions

Let $G \leq_G \mathcal{S}_X$ be a permutation group.

1. If X is finite, we call $|X|$ the *degree (of* G*)*. For $n \geq 0$, we set $\mathcal{S}_n = \mathcal{S}_{[n]}$.
2. A non-empty subset $\Delta \subseteq X$ is a *block*[1] if and only if, for all $g \in G$, the condition $g(\Delta) \cap \Delta \neq \emptyset$ implies $g(\Delta) = \Delta$. A collection of blocks that forms a partition of X

[1] To distinguish them from ordinary sets, blocks for G are denoted by capital Greek letters. Also, alphabets are denoted by capital Greek letters, but the context should always make it clear whether a specific capital Greek letter denotes a block or an alphabet.

is called a *system of blocks*. Blocks are precisely the congruence classes of some G-congruence on X. A block is a *trivial block* if it is a singleton or equals X.

3. A permutation group G over X is *primitive* if the only G-congruences on X are the trivial G-congruences. As the blocks are precisely the G-congruence classes for some G-congruence, G is primitive if and only if it only admits the trivial blocks. If G is transitive and not primitive, then we call it *imprimitive*.
4. The *G-orbit* (or just *orbit*) of $a \in X$ in G is $\mathrm{Orb}_G(a) = \{g(a) \mid g \in G\}$. The orbits $\mathrm{Orb}_G(a)$ for $a \in X$ form a system of blocks, and the union of several orbits is a block.
5. Let $k \geq 1$. Then G is called *k-homogeneous* (resp. *k-transitive*) if for every two k-sets $A, B \subseteq X$ (resp. k-tuples $(a_1, \ldots, a_k)$, $(b_1, \ldots, b_k) \in X^k$) there exists $g \in G$ such that $B = g(A)$ (resp. $(b_1, \ldots, b_k) = (g(a_1), \ldots, g(a_k))$). Every k-transitive group is k-homogeneous. The symmetric group $\mathcal{S}_X$ is k-transitive for each $k \geq 1$. A 1-homogeneous permutation group is also called *transitive*. Equivalently, G is transitive if and only if there exists $a \in X$ such that $X = \mathrm{Orb}_G(a)$. For $A \subseteq X$ we also say that G is *transitive on A*, or *acts transitively on A*, if $\mathrm{Orb}_G(a) = A$ for some (and so each) $a \in A$. We call G *intransitive* if it is not transitive.
6. Let $a \in X$. The *point stabilizer subgroup (of a in G)* is $G_a = \{g \in G \mid g(a) = a\}$. For $g \in G$ we have
$$gG_a = \{h \in G \mid h(a) = g(a)\}. \tag{5.1}$$
7. For any transformation $t\colon X \to X$ and permutation $g\colon X \to X$ the *conjugate of t by g* is $t^g\colon X \to X$ given by gtg^{-1}. For $a, b \in X$, we have[2]
$$t(a) = b \Leftrightarrow t^g(g(a)) = g(b). \tag{5.2}$$
This equation implies, for $a \in X$ and $g \in G$,
$$G_{g(a)} = gG_ag^{-1}. \tag{5.3}$$
By Equation (5.2) every permutation $g\colon X \to X$ is a graph morphism of the functional graphs of t and t^g for a transformation $t\colon X \to X$.
8. We call G *semiregular* if G_a is trivial for each $a \in X$. This is equivalent to the condition that $g(a) = h(a)$ for $g, h \in G$ and some $a \in X$ implies $g = h$ since $g(a) = h(a)$ yields $h^{-1}(g(a)) = a$. By semiregularity, we find $h^{-1}(g) = \mathrm{id}_X$, giving $g = h$.
9. We call G *regular* if it is semiregular and transitive. Equivalently, G is transitive and there exists $a \in X$ such that $G_a = \{\mathrm{id}_X\}$.
10. The *alternating group* (for a given n) is the unique subgroup $\mathcal{A}_n \leq_G \mathcal{S}_n$ such that $|\mathcal{A}_n| = |\mathcal{S}_n|/2$. See [126] for more information.

There should be no danger of confusion of $\mathcal{A}_n$ or $\mathcal{S}_n$ with the notation for a point stabilizer, where also n as a point might appear in the lower index, or with an automaton from an automaton family indexed by n. For the former, this should not arise since

[2] Note that by our convention of reading functional composition from right to left, the inverse has to occur on the right hand side in the defining equation of t^g to make Equation (5.2) valid. In the literature, for example in [393], conjugation of t by g is also often defined as $g^{-1}tg$ instead.

groups are not denoted by calligraphic letters in the present work, and for the latter the context should make it clear whether we refer to an automaton or the alternating group.

Proposition 5.1 *A primitive permutation group G on a domain containing at least three points is transitive.*

Proof. Suppose G is not transitive. Since X contains at least three points, there exist (not necessarily distinct) $a, b \in X$ such that the set $\Delta = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$ is a proper subset of X and contains at least two points. Now, Δ is a block, and so G is not primitive.[3] □

As defined here, every permutation group on $[2]$ is primitive, even the intransitive trivial permutation group. To exclude this special case, some authors prefer to include transitivity into the definition of primitivity. By Theorem 5.1 this makes no difference for permutation groups on domains containing at least three points.

Example 5.2
1. By Theorem 5.4, for prime n and transitive $G \leq_G \mathcal{S}_n$, each block is trivial and so G is primitive.
2. The symmetric group $\mathcal{S}_n$ is primitive. For suppose $A \subseteq [n]$ is any subset such that $a, b \in A$ and $c \notin A$ for distinct $a, b, c \in [n]$. Then, for $t = (b\ c)$, we have $a \in t(A) \cap A$ and $c \in t(A) \setminus A$, which shows that A is not a block.

If $\sim$ is a G-congruence, then we have the following equivalence for all $g \in G$ and $a, b \in X$:

$$a \sim b \Leftrightarrow g(a) \sim g(b). \tag{5.4}$$

Here, we are exclusively concerned with finite permutation domains. So, from now on, in the remaining examples and results of the present work we always take $X = [n]$.

Example 5.3 The *Klein four-group* $G = \{(), (0\ 1)(2\ 3), (0\ 2)(1\ 3), (0\ 3)(1\ 2)\}$ is a regular subgroup of $\mathcal{S}_4$ where $\{0, 1\}$ and $\{2, 3\}$ form a system of blocks.

Example 5.4 Let $G \leq_G \mathcal{S}_n$ be transitive and $\Delta \subseteq [n]$ be some block. Then the sets $g(\Delta)$ for $g \in G$ are equal-sized blocks partitioning $[n]$. So, n is a multiple of $|\Delta|$.

First, we characterize the G-congruences generated by a permutation cyclically permuting its domains. A related result appears in [287, Lem. 2].

Lemma 5.5 *Set $g = (0\ 1\ 2\ \cdots\ n-1)_{[n]}$ and $G = \langle\{g\}\rangle_G$. Then for every G-congruence $\sim$ there exists a divisor r of n such that, for all $a, b \in [n]$,*

$$a \sim b \Leftrightarrow a \equiv b \pmod{r}. \tag{5.5}$$

Conversely, each divisor r of n defines a G-congruence on $[n]$ by Equation (5.5)*.*

[3] Alternatively, for the trivial group every subset of $[n]$ is a block. So, for $n \geq 3$, a primitive G must be non-trivial. Hence, there exist $a \in [n]$ and $g \in G$ with $g(a) \neq a$. Then $|\mathrm{Orb}_G(a)| > 1$ and as $\mathrm{Orb}_G(a)$ is a block we find $\mathrm{Orb}_G(a) = [n]$, i.e., G is transitive.

Proof. First, suppose $\sim$ is a G-congruence. Let $0 < r \leq n$ be the smallest number such that $g^r(0) \sim 0$. Inductively, we find $0 \sim g^{r \cdot i}(0)$ for all $i \geq 0$. Since $g^i(0) \equiv i \pmod{n}$ for $i \geq 0$, it is $\{x \in [n] \mid x \equiv 0 \pmod{r}\} \subseteq [0]_\sim$. Assume $0 \sim g^{r \cdot i + d}(0)$ for some $i \geq 0$ and $0 \leq d < r$. Since $0 \sim g^{r \cdot i}(0)$ we find $g^d(0) \sim g^{r \cdot i + d}(0)$. So, $g^d(0) \sim 0$, which implies $d = 0$ by the minimal choice of r. Hence, $[0]_\sim \subseteq \{x \in [n] \mid x \equiv 0 \pmod{r}\}$. Now, for $0 \leq a < b < n$ we get

$$\begin{aligned} a \equiv b \pmod{r} &\Leftrightarrow 0 \equiv b - a \pmod{r} \\ &\Leftrightarrow 0 \sim b - a \Leftrightarrow g^a(0) \sim g^a(b - a) \Leftrightarrow a \sim b, \end{aligned}$$

which shows Equation (5.5). Conversely, as $g(x) = (x + 1) \bmod n$ for $x \in [n]$ the relation $a \equiv b \pmod{r}$ yields a G-congruence when n is a multiple of r. □

Next, we collect some properties of semiregular permutation groups.

Proposition 5.6 *Let $G \leq \mathcal{S}_n$ be semiregular. Then $|G| \leq n$, and we have $|G| = n$ if and only if G is regular.*

Proof. Pick any point $a \in [n]$. Consider the map $\varphi\colon G \to [n]$ given by $\varphi(g) = g(a)$. Since G is semiregular, this map is injective. Thus $|G| \leq n$. Furthermore, φ is surjective if and only if G is transitive, and φ is surjective precisely if $|G| = n$. This yields that $|G| = n$ if and only if G is regular. □

Next, we characterize semiregularity by cycle lengths.

Proposition 5.7 *Let $G \leq_G \mathcal{S}_n$. Then G is semiregular if and only if for each $g \in G$ all cycles contained in g have the same length.*

Proof. First, suppose G is not semiregular. Then there exist $a \in [n]$ and $g \in G_a \setminus \{\mathrm{id}_{[n]}\}$. As $a \in \mathrm{fix}(g)$, g contains a cycle of length one and, as $g \neq \mathrm{id}_{[n]}$, it contains a cycle of length greater than one. Conversely, suppose there exists $g \in G$ containing two cycles of different lengths. Then $g^i(a) = a$ and $g^j(b) = b$ with $g(a) \neq a$ and $g(b) \neq b$ for minimal $0 < i < j < n$ and some $a, b \in [n]$, giving $g^i \in G_a$ but $g^i \neq \mathrm{id}_{[n]}$ as $g^i(b) \neq b$. So, G is not semiregular. □

For a semiregular $G \leq_G \mathcal{S}_X$ and $a, b \in X$, since $g(a) = g'(a)$ implies $g(b) = g'(b)$ for $g \in G$, we can define a bijection $h\colon \mathrm{Orb}_G(a) \to \mathrm{Orb}_G(b)$ by setting $h(c) = g(b)$ for $g \in G$ with $g(a) = c$, i.e., the edges of the orbital digraph $\{(g(a), g(b)) \mid g \in G\}$ describe a bijection. If G is transitive, h is a permutation on X. In this case, since, by Equation (5.2), a permutation mapping a to b maps $g(a)$ to $g(b)$ if and only if $h^g = h$, the permutation described by the tuples $\{(g(a), g(b)) \mid g \in G\}$ is characterized as the permutation h with $h(a) = b$ and $hg = gh$ for all $g \in G$. Thus, if G is regular and commutative, it contains a permutation described by the tuples $\{(g(a), g(b)) \mid g \in G\}$.

Example 5.8 Let $G = \langle\{t\}\rangle_G$ for some permutation $t\colon [n] \to [n]$. Then,

1. The orbits of G are the sets $\{t^i(x) \mid i \geq 0\}$, and the non-singleton sets of this form equal the points moved by some non-identity cycle induced by t.
2. t is an cycle if and only if G has at most one non-singleton orbit,
3. G is semiregular if and only if every cycle in t has the same length,
4. G is regular $\Leftrightarrow$ G is transitive $\Leftrightarrow$ t cyclically permutes $[n]$.

5.2 Transitive Permutation Groups

The following three subsections collect results on permutation groups related to transitivity. Section 5.2.1 treats separation of subsets by a transitive group and shows which idempotents arise in the transformation semigroup generated by a transitive group together with a one-point singular. Section 5.2.2 establishes a correspondence between the subgroups containing the stabilizer of a point a and the blocks containing a, under which cosets correspond to images of blocks. Section 5.2.3 relates conditions on how a group maps 2-sets to one another to the size of orbits, to transitivity, and to 2-homogeneity.

5.2.1 Separating Sets and Idempotents

In transitive permutation groups we can separate subsets in the following way (see [36, p. 130] for related discussion).

Theorem 5.9 ([36, 91, 464]) *Let $G \leq_G \mathcal{S}_n$ be transitive and let $A, B \subseteq [n]$ with $|A| \cdot |B| < n$. Then there exists $g \in G$ such that $g(A) \cap B = \emptyset$.*

Our next results let us construct certain idempotent one-point singulars.

Proposition 5.10 *Let $t\colon [n] \to [n]$ be one-point singular with $\mathrm{dupl}(t) = \{c\}$. Then there exists an idempotent $h \in \langle\{t\}\rangle_S$ and we have the following:*

1. *If $t(c) = c$, then h is one-point singular with $\mathrm{dupl}(h) = \{c\}$ and $t^{-1}(c) \subseteq h^{-1}(c)$.*
2. *If t has defect one and the point excluded by t lies in $t^{-1}(c)$, say $\mathrm{excl}(t) = \{a\}$ and $t^{-1}(c) = \{a, b\}$, then h is the simple idempotent mapping a to b.*

Proof. By Theorem 3.20 such an h exists; write $h = t^i$ with $i > 0$.

1: We show $\mathrm{dupl}(t^j) = \{c\}$ for $j \geq 1$ by induction, the case $j = 1$ being clear. For $j \geq 2$ write $t^j = tt^{j-1}$, so that Equation (3.10) yields $\mathrm{dupl}(t^j) = \mathrm{dupl}(t_{|t^{j-1}([n])}) \cup t(\mathrm{dupl}(t^{j-1}))$, where the first set is contained in $\mathrm{dupl}(t) = \{c\}$ and the second equals $t(\{c\}) = \{c\}$. Hence $\mathrm{dupl}(h) = \{c\}$. Moreover $t(x) = c$ gives $h(x) = t^{i-1}(c) = c$, so $t^{-1}(c) \subseteq h^{-1}(c)$.

2: We have $t([n]) = [n] \setminus \{a\}$, and $t(a) = t(b) = c$ with $b \neq a$ gives $t([n] \setminus \{a\}) = t([n]) = [n] \setminus \{a\}$. So t maps the finite set $[n] \setminus \{a\}$ onto itself and thus permutes it. Hence h restricts to an idempotent permutation of $[n] \setminus \{a\}$, that is, to the identity, and $h(a) = t^{i-1}(t(a)) = t^{i-1}(t(b)) = h(b) = b$. □

Remark 5.11 The hypotheses of the two items differ in which point is required to lie in $t^{-1}(c)$: in Item 1 it is the duplicated point c itself, and in Item 2 it is the excluded point a. This accounts for the stronger conclusion in the second case. There, t maps $[n] \setminus \{a\}$ onto itself, so no point outside $t^{-1}(c)$ is ever collapsed by a power of t and we obtain $h^{-1}(c') = t^{-1}(c)$ for the point c' with $\mathrm{dupl}(h) = \{c'\}$. Under Item 1 alone, the inclusion $t^{-1}(c) \subseteq h^{-1}(c)$ can be strict, as passing to powers may collapse further points: for $n = 4$ and t given by $t(0) = t(1) = t(2) = 0$ and $t(3) = 2$ we have $c = 0$ and $t^{-1}(0) = \{0, 1, 2\}$, whereas t^2 is the constant map at 0, so that $h^{-1}(0) = [4]$.

Corollary 5.12 *Let $G \leq_G \mathcal{S}_n$ be a transitive permutation group and let $t\colon [n] \to [n]$ be one-point singular. Then for each $a \in t^{-1}(\mathrm{dupl}(t))$ there exists an idempotent one-point singular $h \in \langle G \cup \{t\}\rangle_S$ with $\mathrm{dupl}(h) = \{a\}$ and $t^{-1}(\mathrm{dupl}(t)) \subseteq h^{-1}(a)$.*

Proof. Write $\mathrm{dupl}(t) = \{c\}$ and pick $g \in G$ with $g(c) = a$, which is possible as G is transitive. As g is a bijection, gt is one-point singular with $\mathrm{dupl}(gt) = \{a\}$ and $(gt)^{-1}(a) = t^{-1}(\mathrm{dupl}(t))$. The latter contains a, so gt fixes a and Item 1 of Theorem 5.10, applied to gt, gives an idempotent $h \in \langle\{gt\}\rangle_S \subseteq \langle G \cup \{t\}\rangle_S$ as claimed. □

Corollary 5.13 *Let $G \leq_G \mathcal{S}_n$ be a transitive permutation group and let $t\colon [n] \to [n]$ be of defect one with $t(a) = t(b)$ for distinct $a, b \in [n]$. Then there exists a simple idempotent $h \in \langle G \cup \{t\}\rangle_S$ with $h(a) = h(b) = b$.*

Proof. Write $\mathrm{excl}(t) = \{e\}$ and pick $g \in G$ with $g(e) = a$, which is possible as G is transitive. As g is a bijection, gt has defect one with $\mathrm{excl}(gt) = \{a\}$ and $(gt)^{-1}(\mathrm{dupl}(gt)) = t^{-1}(\mathrm{dupl}(t)) = \{a, b\}$. Now Item 2 of Theorem 5.10, applied to gt, gives an idempotent $h \in \langle\{gt\}\rangle_S \subseteq \langle G \cup \{t\}\rangle_S$ mapping a to b. □

Applying Theorem 5.13 to the pair (a, b) and to the pair (b, a) shows that for t of defect one with $t(a) = t(b)$ the semigroup $\langle G \cup \{t\}\rangle_S$ contains *both* simple idempotents identifying a and b.

Example 5.14 Let $g = (0\ 1\ 2\ 3)$ and let $t\colon [4] \to [4]$ be given by $t(0) = 1$, $t(2) = 3$ and $t(1) = t(3) = 2$. Then gt permutes $\{0, 2, 3\}$ and g^3t permutes $\{0, 1, 2\}$. Here g^3t is already the simple idempotent mapping 3 to 1, and $(gt)^2$ is the simple idempotent mapping 1 to 3.

5.2.2 Blocks and Point Stabilizers

We next establish a correspondence between the subgroups containing the stabilizer of a point a and the blocks containing a in Item 1 of Theorem 5.15. This also appears in [179, Thm. 1.5A].

Theorem 5.15 *Let $G \leq_G \mathcal{S}_n$ be transitive and $a \in [n]$. Then the following hold.*

1. *The maps*

$$H \mapsto \mathrm{Orb}_H(a) \qquad \text{and} \qquad \Delta \mapsto \{g \in G \mid g(\Delta) = \Delta\}$$

   are mutually inverse bijections between the subgroups H with $G_a \leq_G H \leq_G G$ and the blocks Δ of G with $a \in \Delta$.
2. *For $G_a \leq_G H \leq_G G$, $\Delta = \mathrm{Orb}_H(a)$ and $g_1, g_2 \in G$ we have*

$$g_1H = g_2H \iff g_1(\Delta) = g_2(\Delta),$$

   and hence $gH = \{g' \in G \mid g'(\Delta) = g(\Delta)\}$.

Proof. 1: Let $G_a \leq_G H \leq_G G$ and $\Delta = \mathrm{Orb}_H(a)$. We show

$$\{g \in G \mid g(\Delta) \cap \Delta \neq \emptyset\} = H. \tag{5.6}$$

If $g \in H$ then $g(\Delta) = \Delta$, since Δ is an H-orbit; in particular the left-hand side contains H. Conversely, suppose $g(h_2(a)) = h_1(a)$ for some $h_1, h_2 \in H$. Then $h_1^{-1} g h_2 \in G_a \leq_G H$ and therefore $g \in h_1 H h_2^{-1} = H$. This proves (5.6), and, since $g \in H$ implies $g(\Delta) = \Delta$, it shows both that Δ is a block and that the second map returns H when applied to Δ.

Now let Δ be a block with $a \in \Delta$ and set $H = \{g \in G \mid g(\Delta) = \Delta\}$, which is a subgroup. If $g \in G_a$ then $a \in g(\Delta) \cap \Delta$, so $g(\Delta) = \Delta$ and $g \in H$; hence $G_a \leq_G H$. The inclusion $\mathrm{Orb}_H(a) \subseteq \Delta$ is immediate from the definition of H. Conversely let $b \in \Delta$. By transitivity $b = g(a)$ for some $g \in G$, so $b \in g(\Delta) \cap \Delta$ and thus $g(\Delta) = \Delta$, i.e. $g \in H$ and $b \in \mathrm{Orb}_H(a)$. Hence $\Delta = \mathrm{Orb}_H(a)$, so the first map returns Δ, and the two maps are mutually inverse.

2: By (5.6) we have $g \in H$ if and only if $g(\Delta) = \Delta$. Thus

$$g_1 H = g_2 H \iff g_2^{-1} g_1 \in H \iff g_2^{-1} g_1(\Delta) = \Delta \iff g_1(\Delta) = g_2(\Delta).$$

That $gH = \{g' \in G \mid g'(\Delta) = g(\Delta)\}$ follows since $g' \in gH$ precisely if $g'H = gH$. □

Remark 5.16 By Item 2 of Theorem 5.15 the map $\varphi(gH) = g(\Delta)$ is a well-defined bijection from the cosets of H onto the block system $\{g(\Delta) \mid g \in G\}$ generated by $\Delta = \mathrm{Orb}_H(a)$, and $\varphi(h \cdot gH) = h(\varphi(gH))$ for $h \in G$. Thus subgroups containing G_a together with their cosets correspond to blocks containing a together with their block systems, and, for $g \in G$ and $h \in G$, we have

$$h \in gH \iff h(a) \in g(\Delta). \tag{5.7}$$

As an example, let G be as in Theorem 5.5. Then G can be identified with the additive group of residues modulo n via $g^i \mapsto [i]_{\equiv_n}$. Since every subgroup of that group is the group of multiples of some divisor r of n [393, Sec. 1.4], Equation (5.7) also implies Theorem 5.5.

Remark 5.17 Theorem 5.15 is an instance of the standard correspondence between transitive G-actions and actions on the cosets of a subgroup: a system of blocks induces a G-action for which H is a point stabilizer, and the theorem states that the actions arising in this way are precisely those on the cosets of a subgroup containing a point stabilizer. We refer to [126] for the formalisation via G-actions, which we do not introduce here, and note that by our conventions permutations act on the left, whereas in [126] they act on the right; accordingly, our statements concern left cosets and multiplication from the left.

Remark 5.18 Specialising Theorem 5.15, Item 2 to $H = G_a$, the left cosets of G_a are in bijection with the points of $[n]$. As the cosets partition G and all have size $|G_a|$, we obtain $|G| = n \cdot |G_a|$, the *orbit-stabilizer property* [126, 179, 393, 638]. This implies that a transitive $G \leq_G \mathcal{S}_n$ is regular if and only if $|G| = n$.

Corollary 5.19 *Let $G \leq_G \mathcal{S}_n$ be transitive, $a \in [n]$ and $G_a \leq_G H \leq_G G$. Set $\Delta = \mathrm{Orb}_H(a)$. Then:*

1. *For $g \in G$ we have $g \notin H$ if and only if $a \notin g(\Delta)$.*
2. *If $H \neq G$ then H does not act transitively.*

Proof. 1: Since Δ is a block by Theorem 5.15, $a \in g(\Delta)$ is equivalent to $g(\Delta) = \Delta$ for $g \in G$. Since $H = \{g \in G \mid g(\Delta) = \Delta\}$ by Item 1 of Theorem 5.15, $g \in G \setminus H$ if and only if $a \notin g(\Delta)$.

2: By Theorem 5.15, Item 1 the assignment $H \mapsto \mathrm{Orb}_H(a)$ is injective, and $\mathrm{Orb}_G(a) = [n]$ by transitivity of G. Hence $\mathrm{Orb}_H(a) = [n]$ forces $H = G$. □

5.2.3 Mapping 2-Sets, Orbit Sizes, and Transitivity

We now relate mapping properties of 2-sets to orbit sizes and transitivity. Two technical statements do most of the work. Theorem 5.22 treats images of the form $\{a, y\} \mapsto \{b, y\}$, where a point of the source is retained, and Theorem 5.25 settles, for a single target 2-set, when it is the image of a 2-set containing a fixed point a under a map subject to a side condition. Quantifying the latter over all targets yields Theorem 5.26, which relates several such mapping properties to the size of $\mathrm{Orb}_G(a)$, to transitivity, and to the fixed points of the point stabiliser of a. From these we obtain characterisations of transitivity in Theorem 5.27 and sufficient conditions in Theorem 5.28. Finally, Theorem 5.31 determines which mapping properties force 2-homogeneity, i.e., that any two 2-sets can be mapped to one another. Before that we recall a result of Livingstone and Wagner [418] relating k-homogeneity to k-transitivity.

Theorem 5.20 (Livingstone and Wagner [418]) *Let $G \leq_G \mathcal{S}_n$ be k-homogeneous with $k \geq 2$ and $2k \leq n$. Then G is $(k-1)$-transitive, and for $k \geq 5$ even k-transitive.*

The argument from the following lemma is taken from [445, p. 533], where it is used to deduce that a 2-homogeneous group is transitive.

Lemma 5.21 *Let $G \leq_G \mathcal{S}_n$ and $a, b, c \in [n]$. If $g(\{a, b\}) = \{c, b\}$ for some $g \in G$, then $c \in \mathrm{Orb}_G(a)$.*

Proof. We have $g(a) = c$ or $g(a) = b$. In the former case, $c \in \mathrm{Orb}_G(a)$. In the latter case, we have $g(b) = c$ and thus $g^2(a) = c$. So also $c \in \mathrm{Orb}_G(a)$. □

Proposition 5.22 *Let $G \leq_G \mathcal{S}_n$, let $a \in [n]$ and let $A \subseteq [n]$. Suppose that for every $b \in A$ there are $y \in [n]$ and $g \in G$ with $g(\{a, y\}) = \{b, y\}$. Then*

$$A \cup \{a\} \subseteq \mathrm{Orb}_G(a).$$

Proof. By Theorem 5.21 we have $b \in \mathrm{Orb}_G(a)$ for every $b \in A$, and $a \in \mathrm{Orb}_G(a)$ holds anyway. □

Remark 5.23 The method of proof of Theorem 5.21 actually established $c \in B$ for

$$B = \{g^2(a) \mid g \in G\} \cup \{g(a) \mid g \in G \text{ with } g(b) = b \text{ for some } b \neq a\}.$$

Thus A from Theorem 5.22 is also contained in this set. For $A = [n]$ this yields that, for fixed $a \in [n]$, for each $c \in [n]$ there exist $b \in [n]$ and $g \in G$ such that $g(\{a, b\}) = \{c, b\}$ if and only if $[n] = B$. As $B \subseteq \mathrm{Orb}_G(a)$, this forces G to be transitive. For example, we have $[n] = B$ for transitive groups of odd order. Indeed, on a finite group the squaring map $g \mapsto g^2$ is surjective if and only if it is injective, and it fails to be injective precisely when there is an involution, i.e., a non-identity element whose square equals the identity. By Cauchy's theorem an involution exists whenever $|G|$ is even, and conversely an involution has order 2, so that 2 divides $|G|$. Hence for odd $|G|$ we get $\{g^2 \mid g \in G\} = G$ and therefore $\{g^2(a) \mid g \in G\} = \mathrm{Orb}_G(a) = [n]$. Note also that for transitive G the condition does not depend on the choice of a: conjugation by an element $h \in G$ with $h(a) = a'$ carries the two sets for a onto those for a'.

Remark 5.24 We do not need this fact, but it is worth noting that, for fixed $a, b \in [n]$, if, for every $c \in [n]$, we have $g(\{a, b\}) = \{a, c\}$ or $g(\{a, b\}) = \{b, c\}$ for some $g \in G$, then $[n] = \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$. This follows since then $c \in \mathrm{Orb}_G(a) \cup \mathrm{Orb}_G(b)$ for each $c \in [n]$.

Proposition 5.25 *Let $G \leq_G \mathcal{S}_n$, let $a \in [n]$ and let $\{c, d\} \subseteq [n]$ be a 2-set.*

1. *For $T \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(a) \in T$ if and only if $\{c, d\} \cap \mathrm{Orb}_G(a) \cap T \neq \emptyset$.*
2. *There are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(b) \neq b$ if and only if there is $g \in G$ with $g(a) = c$ and $g(d) \neq d$, or with $g(a) = d$ and $g(c) \neq c$.*

Proof. 1: Given such g and b, we have $g(a) \in g(\{a, b\}) = \{c, d\}$ as well as $g(a) \in \mathrm{Orb}_G(a)$ and $g(a) \in T$. Conversely, for $x \in \{c, d\} \cap \mathrm{Orb}_G(a) \cap T$ and y the other element of $\{c, d\}$, choose $g \in G$ with $g(a) = x$ and put $b = g^{-1}(y)$; then $g(a) \in T$ and $g(\{a, b\}) = \{x, y\} = \{c, d\}$.

2: Given such g and b, we have $g(a) \in \{c, d\}$; say $g(a) = c$, so that $g(b) = d$ and $b = g^{-1}(d)$, and $g(b) \neq b$ gives $d \neq g^{-1}(d)$, i.e., $g(d) \neq d$. Conversely, for $g(a) = c$ and $g(d) \neq d$ put $b = g^{-1}(d)$; then $g(\{a, b\}) = \{c, d\}$ and $g(b) = d \neq b$, as $b = d$ would give $g(d) = d$. Interchanging c and d gives the remaining cases. □

Corollary 5.26 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$.*

1. *For every 2-set $\{c, d\} \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ if and only if $|\mathrm{Orb}_G(a)| \geq n - 1$.*
2. *For every 2-set $\{c, d\} \subseteq [n]$ there are $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(a) \neq a$ if and only if G is transitive.*
3. *For every $b \in [n] \setminus \{a\}$ there are $g \in G$ and $c \in [n]$ with $g(\{a, c\}) = \{a, b\}$ and $g(a) \neq a$ if and only if G is transitive.*
4. *For every $b \in [n] \setminus \{a\}$ there are $g \in G$ and $c \in [n]$ with $g(\{a, c\}) = \{a, b\}$ and $g(c) \neq c$ if and only if every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$.*

Proof. 1: By Item 1 of Theorem 5.25 with $T = [n]$ the condition states that every 2-set meets $\mathrm{Orb}_G(a)$, which holds if and only if $|[n] \setminus \mathrm{Orb}_G(a)| \leq 1$, as a 2-set contained in that complement exists precisely when it has at least two elements.

2 and 3: By Item 1 of Theorem 5.25 with $T = [n] \setminus \{a\}$, both conditions state that the 2-sets in question meet $\mathrm{Orb}_G(a) \cap T = \mathrm{Orb}_G(a) \setminus \{a\}$.

For a target $\{a, b\}$ with $b \neq a$ this says that $b \in \mathrm{Orb}_G(a)$, so Item 3 implies that G is transitive (For a direct argument, for the target set $\{a, b\}$ we have $g(a) \in \{a, b\}$ with $g(a) \neq a$, which gives $g(a) = b$.). If G is transitive, then $\mathrm{Orb}_G(a) \setminus \{a\} = [n] \setminus \{a\}$, which is met by every 2-set, giving Item 2. Finally, Item 2 implies Item 3, as the latter only concerns the targets $\{a, b\}$. Hence Item 2, Item 3 and transitivity of G are equivalent, which proves both stated equivalences.

4: Consider some $b \in [n] \setminus \{a\}$. By Item 2 of Theorem 5.25 applied to $\{a, b\}$, such g and c exist if and only if there is $g \in G$ with $g(a) = a$ and $g(b) \neq b$, or with $g(a) = b$, i.e., if and only if some element of G_a moves b or $b \in \mathrm{Orb}_G(a)$. This is equivalent to the condition that if the considered b is fixed by all elements from G_a, then it must be in $\mathrm{Orb}_G(a)$. Demanding this for every $b \in [n] \setminus \{a\}$ says precisely that a point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$. □

Theorem 5.27 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$. Then the following are equivalent.*

1. *G is transitive.*
2. *For distinct $c, d \in [n]$ there is $g \in G$ with $g(a) = c$ and $g(d) \neq d$, or $g(a) = d$ and $g(c) \neq c$.*
3. *Every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$, and every point of $[n] \setminus \mathrm{Orb}_G(a)$ is fixed by all of G.*

Proof. $1 \Rightarrow 2$: Choose $g_0, g_1 \in G$ with $g_0(a) = c$ and $g_1(a) = d$. If $g_0(d) \neq d$ or $g_1(c) \neq c$, we are done. Otherwise $h = g_1^{-1} g_0$ satisfies $h(a) = g_1^{-1}(c) = c$ and $h(d) = g_1^{-1}(d) = a \neq d$, the latter since $a = d$ would give $c = g_0(a) = g_0(d) = d$.

$2 \Rightarrow 3$: By Item 2 of Theorem 5.25, condition 2 gives, for every 2-set $\{c, d\}$, some $g \in G$ and $b \in [n]$ with $g(\{a, b\}) = \{c, d\}$ and $g(b) \neq b$. Omitting $g(b) \neq b$ gives $|\mathrm{Orb}_G(a)| \geq n - 1$ by Item 1 of Theorem 5.26, so at most one point lies outside $\mathrm{Orb}_G(a)$, and it forms an orbit and is thus fixed by all of G. Restricting to the targets $\{a, b\}$ gives the remaining condition by Item 4 of Theorem 5.26.

$3 \Rightarrow 1$: Let $b \in [n] \setminus \mathrm{Orb}_G(a)$. Then b is fixed by all of G, hence by all of G_a, and so lies in $\mathrm{Orb}_G(a)$, a contradiction. Thus $\mathrm{Orb}_G(a) = [n]$. □

Finally, we give some sufficient conditions.

Proposition 5.28 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 2$ and $a \in [n]$. Then G is transitive if one of the following holds:*

1. *$|\mathrm{Orb}_G(a)| \geq n - 1$ and for $b \in [n] \setminus \{a\}$ we have $g(\{a, c\}) = \{a, b\}$ for some $c \in [n] \setminus \{b\}$ and $g \in G$.*
2. *For distinct $c, d \in [n]$ we have $g(\{a, b\}) = \{c, d\}$ for some $g \in G$ and $b \in [n] \setminus \{c\}$.*
3. *For every $c \in [n]$ we have $g(\{a, b\}) = \{c, b\}$ for some $b \in [n]$ and $g \in G$.*

Proof. 1: Such a g maps c to a or to b with $c \notin \{a, b\}$, so $g(c) \neq c$, and Item 4 of Theorem 5.26 gives that every point fixed by all of G_a lies in $\mathrm{Orb}_G(a)$. As $|\mathrm{Orb}_G(a)| \geq n - 1$ leaves at most one point outside $\mathrm{Orb}_G(a)$, which forms an orbit and is fixed by all of G, Theorem 5.27 applies.

2: By Item 1 of Theorem 5.26 we have $|\mathrm{Orb}_G(a)| \geq n - 1$, and the pair b, a gives $g(\{a, c\}) = \{a, b\}$ for some $c \neq b$, which is the condition from Item 1.

3: This follows from Theorem 5.22 applied with $A = [n]$. □

Remark 5.29 None of the conditions of Theorem 5.28 is necessary for transitivity: for $n = 4$ the group $G = \langle (0\ 1\ 2\ 3) \rangle_G$ is transitive, but at $a = 0$ and $b = 2$ the equality $g(\{0, c\}) = \{0, 2\}$ forces $c = 2$, so that Item 1 fails, and applying Item 2 to the pair $2, 0$ would give exactly such a $c \neq 2$. Likewise Item 3 fails for odd c: in $g(\{0, b\}) = \{c, b\}$ we have $c \neq b$, so either $g(0) = c$ and $g(b) = b$, which for a shift forces g to be the identity and hence $c = 0$, or $b = g(0)$ and $c = g(b) = g(g(0)) = 2k$, and both values are even.

Remark 5.30 The ordered reading matters in Item 2 of Theorem 5.28. Demanding instead a single witness with $b \notin \{c, d\}$ is strictly stronger and is unsatisfiable for $n = 3$: for the 2-set $[n] \setminus \{a\}$ there is no b with $b \neq a$ and $b \notin [n] \setminus \{a\}$. By contrast, $\mathcal{S}_3$ satisfies the hypothesis of Item 2 from Theorem 5.28. For $n = 4$ the transitive group $G = \langle (0\ 1\ 2\ 3) \rangle_G$ violates it at $a = 0$ and the 2-set $\{0, 2\}$: a witness would need $b \in \{1, 3\}$, whereas $g(\{0, b\}) = \{0, 2\}$ forces $b = 2$.

Proposition 5.31 *Let $G \leq_G \mathcal{S}_n$ with $n \geq 3$. Then the following are equivalent.*

1. *G is 2-homogeneous.*
2. *There exist distinct $a, b \in [n]$ such that for all $c, d \in [n]$ with $\{c, d\} \neq \{a, b\}$ there is $g \in G$ with $g(\{a, c\}) = \{a, d\}$ or $g(\{b, c\}) = \{b, d\}$.*
3. *There exists $a \in [n]$ such that*
   a. *for all $b, c \in [n] \setminus \{a\}$ there is $g \in G$ with $g(\{a, b\}) = \{a, c\}$, and*
   b. *there are $b \in [n] \setminus \{a\}$ and $g \in G$ with $a \notin g(\{a, b\})$.*

Proof. 1⇒2: Let a, b be any two distinct points and consider $\{c, d\} \neq \{a, b\}$. Then $a \notin \{c, d\}$ or $b \notin \{c, d\}$. As the two conditions in Item 2 arise from one another by interchanging a and b, we may assume $a \notin \{c, d\}$. Then $\{a, c\}$ and $\{a, d\}$ are 2-sets and 2-homogeneity gives $g(\{a, c\}) = \{a, d\}$ for some $g \in G$.

2⇒3: Let a, b be as in Item 2 and let O be the orbit of $\{a, b\}$. Note that $g(\{x, x\})$ is a singleton and hence never a 2-set. For $c \neq a$ applying Item 2 to b, c, which is admissible as $\{b, c\} \neq \{a, b\}$, thus gives $g(\{a, b\}) = \{a, c\}$, so that $\{a, c\} \in O$. Hence any two 2-sets containing a lie in O and are therefore images of one another. Finally, picking $e \notin \{a, b\}$, possible as $n \geq 3$, and applying 2 to a, e, admissible as $\{a, e\} \neq \{a, b\}$, gives $g(\{b, a\}) = \{b, e\}$ with $a \notin \{b, e\}$.

3⇒1: By assumption, there exist $b \in [n] \setminus \{a\}$ and $g \in G$ such that $a \notin g(\{a, b\})$. Let O be the orbit of $\{a, b\}$ for the action of G on the 2-sets. By assumption, every 2-set containing a lies in O. This also implies that every 2-set containing a point from $\mathrm{Orb}_G(a)$ is in O since each such 2-set can be mapped to one containing a.

Put $d = g(a)$, so that $d \neq a$. For $z \neq a$ the 2-sets $\{a, z\}$ and $\{a, d\}$ lie in O. So $h(\{a, d\}) = \{a, z\}$ for some $h \in G$. Whence $z = h(a)$ or $z = h(d) = h(g(a))$ and thus $z \in \mathrm{Orb}_G(a)$. So $\mathrm{Orb}_G(a) = [n]$, and by the above every 2-set lies in O. □

Remark 5.32 Condition 3b cannot be omitted from Item 3 of Theorem 5.31: for $G = \langle (1\ 2\ 3) \rangle_G$ on $\{0, 1, 2, 3\}$ and $a = 0$ the three 2-sets containing 0 are images of one another, so that 3a holds, yet G is not 2-homogeneous, and not even transitive.

The following is immediate but stated for later reference.

Proposition 5.33 *Let G be a permutation group on $[n]$ with $n \geq 2$ that is not transitive. Then there exists a proper subset $A \subsetneq [n]$ with $|A| \geq \lceil \frac{n}{2} \rceil$ such that $g(A) = A$ for all $g \in G$.*

Proof. Let O be an orbit of G. Both O and $[n] \setminus O$ are G-invariant, and one of them has size at least $\lceil \frac{n}{2} \rceil$. As G is not transitive, both are proper subsets of $[n]$. □

5.3 Permutation Groups and Abstract Groups

Often groups are given by some representation. For example, by sets of permutations as permutation groups. We call a group an *abstract group*[4] if we want to emphasize that we do not assume some internal structure or a (re-) presentation beyond the axiomatic definition of a group.

Now, let $(G, \cdot, ^{-1}, 1)$ be an abstract group. Next, we describe one possibility on how to represent G as a permutation group. For each $g \in G$ let $\lambda_g \colon G \to G$ be the permutation defined by $\lambda_g(h) = gh$ for $h \in G$. The map $\Pi \colon G \to \mathcal{S}_G$ given by $\Pi(g) = \lambda_g$ is injective, and we have $\Pi(gg') = \lambda_{gg'} = \lambda_g \lambda_{g'}$, $\Pi(1) = \mathrm{id}_G$, and $\Pi(g^{-1}) = \lambda_{g^{-1}} = \lambda_g^{-1}$ for $g, g' \in G$, i.e., it is a group morphism. This yields that G can be uniquely identified with the permutation group $\{\lambda_g \mid g \in G\} \leq_G \mathcal{S}_G$, giving a permutation representation of G that is called the *left regular (Cayley) representation*. This is not the only way to represent an abstract group as a permutation group, the interested reader can find more information in the discussion (Section 5.6).

Remark 5.34 Every regular permutation group arises in the form of the left regular representation on itself in the following way. Let $G \leq \mathcal{S}_n$ be a regular permutation group and $a \in [n]$. The map $\varphi \colon [n] \to G$ given by $\varphi(b) = g$ for the unique $g \in G$ with $g(a) = b$ is a bijection. Since for $h, g \in G$ and $b \in [n]$ with $g(a) = b$ we have $(h \circ g)(a) = h(g(a)) = h(b)$, we find

$$\varphi(h(b)) = h \circ \varphi(b) = \lambda_h(\varphi(b))$$

where $\lambda_h \colon G \to G$ is the left regular representation introduced above. Thus $h = \varphi^{-1} \lambda_h \varphi$ and h maps b to c for $b, c \in [n]$ if and only if λ_h maps $\varphi(b)$ to $\varphi(c)$. Hence the map

[4] In the same vein, we can talk about abstract monoids or abstract semigroups when solely referring to their axiomatic properties.

φ identifies the permutation domains $[n]$ and G in a way so that G and $\overline{G}$ act in the same way on each. As an example, applying this construction to the right regular representation $\overline{G} = \{\rho_g \mid g \in G\}$, which is a subgroup of $\mathcal{S}_G$, and some $g \in G$, we obtain $\varphi\colon G \to \overline{G}$ such that for $h \in G$ we have $\varphi(h) = \rho_{h^{-1}g}$ since $\rho_{h^{-1}g}(g) = g(h^{-1}g)^{-1} = h$.

Next, we show how to obtain a representation of $G \times G$ as a subgroup of $\mathcal{S}_G$ for abstract groups G with trivial center, called the *diagonal action (on G)*.[5]

Proposition 5.35 *Let G be an abstract group with identity $1 \in G$ and $Z(G) = \{1\}$. For $x, y \in G$ let $\delta_{x,y}\colon G \to G$ be given by $\delta_{x,y}(g) = xgy^{-1}$.*

1. *The map $(x, y) \mapsto \delta_{x,y}$, $x, y \in G$, is an injective morphism from $G \times G$ into S_G.*
2. *$G^D = \{\delta_{x,y} \mid (x, y) \in G\}$ is a transitive permutation group (the representation of $G \times G$ by the diagonal action) on G that is primitive if and only if G is simple.*

Proof. 1 Let $x_1x_2, y_1, y_2 \in G$. As

$$\delta_{x_1x_2,y_1y_2}(g) = (x_1x_2)g(y_1y_2)^{-1} = x_1(x_2gy_2^{-1})y_1^{-1} = (\delta_{x_1,y_1}\delta_{x_2,y_2})(g),$$

for $g \in G$ we obtain a morphism. For injectivity, suppose $\delta_{x_1,y_1} = \delta_{x_2,y_2}$. Then $x_2^{-1}x_1gy_1^{-1}y_2 = g$ for all $g \in G$. For $g = 1$, we get $x_2^{-1}x_1y_1^{-1}y_2 = 1$, which yields $x_2^{-1}x_1 = y_2^{-1}y_1$. Now, $x_2^{-1}x_1g(x_2^{-1}x_1)^{-1} = g$ for all $g \in G$ if and only if $x_2^{-1}x_1 \in Z(G)$. As $Z(G) = \{1\}$, $x_1 = x_2$ and $y_1 = y_2$.

2 As $\delta_{hg^{-1},1}(g) = hg^{-1}g = h$ for $g, h \in G$, we find that G^D is transitive. Next, we show that blocks of G^D containing 1 are precisely the normal subgroups of G, which gives the claim. Let $\Delta \subseteq G$ be a block of G^D with $1 \in \Delta$ and $x, y \in \Delta$. As $x = xyy^{-1}$ we have $x \in \Delta \cap x\Delta y^{-1}$. So $x\Delta y^{-1} = \Delta$. As $1 \in \Delta$ this yields $xy^{-1} \in \Delta$. Hence Δ is a subgroup of G. Now, let $g \in G$. Then $1 \in \Delta \cap g\Delta g^{-1}$. So $\Delta = g\Delta g^{-1}$ and Δ is a normal subgroup. Conversely, let N be a normal subgroup of G. Suppose $x, y \in G$. We have $\delta_{x,y}(N) = xNy^{-1} = xy^{-1}yNy^{-1} = xy^{-1}N$, which is a left coset. Since these cosets partition G, $\delta_{x,y}(N) \cap N \neq \emptyset$ implies $\delta_{x,y}(N) = N$. So, N is a block of G^D. □

Next, more generally, suppose we have given a subgroup H of G such that [6]

$$\bigcap_{g \in G} gHg^{-1} = \{1\} \tag{5.8}$$

Then we can construct a permutation group on the left cosets $X = \{hH \mid h \in G\}$ by $\overline{G} = \{\lambda_g \mid g \in G\}$ where $\lambda_g\colon X \to X$ is a permutation on X given by $\lambda_g(hH) = (gh)H$. Equation (5.8) ensures that $\lambda_g \neq \lambda_{g'}$ if and only if $g \neq g'$. Further, we have $\lambda_{gg'} = \lambda_g\lambda_{g'}$ for $g, g' \in G$. So, $g \mapsto \lambda_g$ is an isomorphism of abstract groups between G and $\overline{G}$. In this way, we have realized G as a permutation group on X. For $H = \{1\}$ (in this case we identify X and G) we obtain the left regular representation. There is also a

[5] This is actually a more general construction related to so-called group actions, see [36, Example 2.6] or [126, Exercise 1.6].

[6] In general, $\bigcap_{g \in G} gHg^{-1}$ is called the *core* of G and H is called *core-free* if Equation (5.8) holds.

corresponding construction on the right cosets by considering the mappings $\rho_g : Y \to Y$ given by $\rho_g(Hh) = H(hg^{-1})$ for $Y = \{Hh \mid h \in H\}$. The inversion entered by our choice on the order of function composition, so that the equation $\rho_{gg'} = \rho_g \rho_{g'}$ is valid (see the related discussion in Section 3.11).[7] The permutation representation obtained here for $H = \{1\}$ is also called the *right regular (Cayley) representation*.

By considering those subgroups H that fulfill Equation (5.8) we, in fact, obtain all permutation representations of G. This follows since H precisely equals the point stabilizer of the permutation group constructed above, and that, by Theorem 5.15, every permutation group can be realized as a permutation group on the left cosets for some point stabilizer subgroup. A form of these representations without Equation (5.8) seems to have been first observed by Burnside in [118], where he also refers to Dyck (without citing a source).

5.4 Permutation Groups and Non-Permutations

Here, we collect results about transformation semigroups generated by a permutation group G and a set $T \subseteq \mathcal{T}_n$ of non-permutations.

Proposition 5.36 *Let $G \leq_G \mathcal{S}_n$ and $T \subseteq \mathcal{T}_n \setminus S_n$. Then G is the group of units of $\langle G \cup T \rangle_S$.*

Proof. By Theorem 3.12, Item 3.18, for $t \in \mathcal{T}_n \setminus \mathcal{S}_n$ or $h \in \mathcal{T}_n$ we have $ht, th \in \mathcal{T}_n \setminus S_n$, which gives the claim. □

By successively using the formula $gt = t^g g$ for a transformation t and permutation g, we obtain the next statement.

Proposition 5.37 *Let $G \leq_G \mathcal{S}_n$ be a permutation group and $T \subseteq \mathcal{T}_n$. Set $S = \langle G \cup T \rangle_S$. Let $K = \{t^g \mid t \in T, g \in G\}$ be the set of conjugates of elements from T. Then $S = \langle K \rangle_M G = G \langle K \rangle_M$.*

Proof. As $K \subseteq S$ and $G \subseteq S$, we have $\langle K \rangle_M \subseteq S$ and so also $\langle K \rangle_M G \subseteq S$. Next, suppose $s \in S$. Then we have $s = g_0 t_1 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n$ and

$$g_0 t_1 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n = t_1^{g_0} t_2^{g_0 g_1} \cdots t_{n-1}^{g_0 g_1 \cdots g_{n-2}} t_n^{g_0 g_1 \cdots g_{n-1}} g_0 g_1 \cdots g_n. \tag{5.9}$$

The above formula can be seen by induction on the number of t_i's in the product. If s is a product of permutations, then the formula clearly holds. Otherwise, we have

$$g_0 t_1 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n = t_1^{g_0} g_0 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n$$

and, by induction hypothesis,

$$g_0 g_1 t_1 g_2 \cdots t_{n-1} g_{n-1} t_n g_n = t_1^{g_0 g_1} \cdots t_{n-1}^{g_0 g_1 \cdots g_{n-2}} t_n^{g_0 g_1 \cdots g_{n-1}} g_0 g_1 \cdots g_n,$$

[7] Note that when the convention is adopted that in a function composition the left-most function is applied first, we have to use an inversion for the left regular action instead, see, for example, [126].

which yields Equation (5.9). Since $t_1^{g_0} t_2^{g_0 g_1} \cdots t_{n-1}^{g_0 g_1 \cdots g_{n-2}} t_n^{g_0 g_1 \cdots g_{n-1}} \in \langle K \rangle_M$ and $g_1 g_2 \cdots g_n \in G$ we obtain $S \subseteq \langle K \rangle_M G$. So $S = \langle K \rangle_M G$. Similarly, we can show

$$g_0 t_1 g_1 t_2 g_2 \cdots t_{n-1} g_{n-1} t_n g_n = g_0 g_1 \cdots g_n t_1^{(g_1 g_2 \cdots g_n)^{-1}} t_2^{(g_2 \cdots g_n)^{-1}} \cdots t_{n-1}^{(g_{n-1} g_n)^{-1}} t_n^{g_n^{-1}},$$

which then yields $S = G\langle K \rangle_M$. □

The following corollary was noted in [29, Sec. 2]).

Corollary 5.38 *Suppose $G \leq_G \mathcal{S}_n$ is a permutation group and $t \in \mathcal{T}_n$. If $\langle G \cup \{t\} \rangle_S$ contains a constant, then some constant is a composition of conjugates of t.*

Next, we note a special case when t is idempotent.

Corollary 5.39 *Suppose $G \leq_G \mathcal{S}_n$ is a permutation group and $t \in \mathcal{T}_n$ is an idempotent. Set $S = \langle G \cup \{t\} \rangle_S$. Let E be the set of idempotents of S. Then $S = \langle E \rangle_M G$.*

Proof. Let $K = \{t^g \mid g \in G\}$. By Equation (5.2) the mappings in K are idempotents, so $K \subseteq E$. With Proposition 5.37 we find $S = \langle K \rangle_M G \subseteq \langle E \rangle_M G$. As $G \subseteq S$ and $\langle E \rangle_M \subseteq S$, and so $\langle E \rangle_M G \subseteq S$, we conclude $S = \langle E \rangle_M G$. □

As $t_1 \cdots t_n g^{-1} = g^{-1} t_1^g \cdots t_n^g$ for a permutation g and transformations $t_1, \ldots, t_n$ we have, in Theorem 5.37 and Theorem 5.39,

$$\langle K \rangle_M G = G\langle K \rangle_M \text{ and } \langle E \rangle_M G = G\langle E \rangle_M.$$

Further, as G contains the identity transformation, we have $\langle G \cup \{t\} \rangle_S = \langle G \cup \{t\} \rangle_M$. However, for a non-permutation t, $\langle K \rangle_S G \neq \langle K \rangle_M G$, as $\mathrm{id}_{[n]} \notin \langle K \rangle_S$.

5.5 Graphs and Permutation Groups

Permutation group theory enters graph theory via the automorphism group in a natural way. Here, we introduce some further notions connecting graphs and permutation groups. Section 5.5.1 collects results on generating sets for $\mathcal{S}_n$. For example, we give a graph-theoretical characterization when a set of transpositions generate $\mathcal{S}_n$. Section 5.5.2 introduces vertex-transitive graphs and investigates the connectivity of vertex-transitive graphs. Lastly, Section 5.5.3 introduces orbital graphs.

We often denote graphs and groups with the capital letter G, but the context should make it clear whether we mean a graph or a group.

5.5.1 Generating Sets for the Full Symmetric Group

It is a well-known result that permutation cyclically permuting a finite set and a transposition on this set generate the full symmetric group. Isaacs and Zieschang [338, Thm. B] gave the following generalization.

Theorem 5.40 (Isaacs and Zieschang [338, Thm. B]) *Let* $x = (0\ 1\ 2\ \cdots\ n-1)_{[n]}$ *and* $y = (0\ 1\ 2\ \cdots\ m-1)_{[n]}$ *for some* $1 < m < n$. *Then* $\langle\{x, y\}\rangle_G$ *equals the full symmetric group on* $[n]$ *unless both n and m are odd, in which case* $\langle\{x, y\}\rangle_G$ *equals* $\mathcal{A}_n$.

Isaacs and Zieschang [338, Thm. A] also proved that for $n \neq 4$ and $x \in \mathcal{S}_n$ there exists $y \in \mathcal{S}_n$ such that $\langle\{x, y\}\rangle_G = \mathcal{S}_n$ (the exception being any element x in the Klein four-group from Theorem 5.3). Actually, this result was first proved by Sophie Piccard [482, pp. 80–86] in 1946, but got lost in limbo[8] and was reproved by Isaacs & Zieschang [338] in 1996.

By Theorem 4.7 the full symmetric group on a finite set is generated by all transpositions. Next, we state a refined result, needed in **??**, that characterizes the sets of transpositions generating $\mathcal{S}_n$ in terms of an associated graph. For the convenience of the reader, we give a proof (which can also be found in [249, Lem. 3.10.1]; compare also the proof of [34, Lem. 6(a)] which uses a similar argument).

Proposition 5.41 *Let* $T \subseteq \mathcal{S}_n$ *be a set of transpositions and consider the undirected graph* $G = ([n], \{\{a, b\} \mid (a\ b) \in T\})$. *Then G is connected if and only if* $\langle T\rangle_G = \mathcal{S}_n$.

Proof. Define a relation $\sim$ on $[n]$ by $a \sim b$ if and only if $a = b$ or $(a\ b) \in \langle T\rangle_G$. Clearly, $\sim$ is reflexive and symmetric. Further, for $a \sim b$ and $b \sim c$ with distinct $a, b, c \in [n]$ it is $(a\ b)^{(b\ c)} = (a\ c) \in \langle T\rangle_G$; thus $\sim$ is transitive and an equivalence relation. As the graph G results from the digraph $G_\sim$ by forgetting about the direction of the edges, from Theorem 3.70 we obtain that the connected components equal the equivalence classes of $\sim$. So, if G is connected, then all points are pairwise comparable for $\sim$, and $\langle T\rangle_G$ contains all transpositions. By Fact 4.7, we find $\langle T\rangle_G = \mathcal{S}_n$. Otherwise, suppose G is not connected. Then, we can pick a, b with $a \nsim b$, i.e., $(a\ b) \notin \langle T\rangle_G$ and so $\langle T\rangle_G \neq \mathcal{S}_n$. □

Remark 5.42 In the proof of Jordan's theorem from [338], it is also shown that for a set of 3-cycles T, $\langle T\rangle_G$ contains the alternating group if and only if the graph on $[n]$ where $a, b \in [n]$ are joined by an edge if and only if $a, b \in \mathrm{move}(t)$ for some $t \in T$ is connected.

In [373] it was shown that a k-transitive group of degree $n > k$ is generated by all elements fixing precisely i points if $i \leq k - 3$. Since the full symmetric group on a set of size n is k-transitive for each $1 \leq k \leq n$, by this result we obtain more generating sets for the full symmetric group of a finite set.

5.5.2 Vertex-Transitive Graphs

A (directed or undirected) graph is *vertex-transitive* if its automorphism group acts transitively on its vertices. If $G = (V, E)$ is vertex-transitive, then every vertex has the same outdegree and we call the common value the *outdegree of* G.

[8] We were not able to obtain a copy of Piccards original work [482] but learned this through the work of Salomaa [545].

A proof of the next statement can be found in [126, Thm 1.10], [36, Thm 1.1], [249, Chap. 2.6], and in [536] (in the latter work formulated with slightly different notions but easily seen to give the result).

Theorem 5.43 *Let $G = (V, E)$ be a finite vertex-transitive digraph. If G is weakly connected, then G is strongly connected.*

Next, we link the outdegree of a vertex-transitive digraph to its connectivity. The result appears in [275, Prop. 3.4] (compare also [48, Thm 3.7 (a)] where it is formulated in terms of the connectivity from Theorem 3.65).

Theorem 5.44 (Hamidoune [275]) *If $G = (V, E)$ is a finite strongly connected vertex-transitive digraph of outdegree d without edges of the form (v, v) for $v \in V$, then G is strongly $\left(\left\lfloor \frac{d}{2} \right\rfloor + 1\right)$-connected.*

Proof. We can assume $E \neq V \times V$. Since G is finite, there exists a largest number k such that G is strongly k-connected, but not strongly $(k+1)$-connected. For $U \subseteq V$ set $N^+(U) = \{v \in V \setminus U \mid \exists u \in U : (u, v) \in E\}$. We have $k \leq |N^+(\{u\})| = d$ for each $u \in V$. An *atom* [273] $A \subseteq V$ is a set of minimal cardinality with $|N^+(A)| = k$ and $A \cup N^+(A) \neq V$.[9] In a strongly connected digraph the atoms partition V [273] (see also [274, 275, Thm. 2.1 in both]). As an automorphism maps atoms to atoms, the atoms induce an $\mathrm{Aut}(G)$-congruence on the vertices. Hamidoune proved that $|A| \leq k$ [275, Lem. 3.3]. So, for $u \in A$, as $N^+(\{u\}) \subseteq A \setminus \{u\} \cup N^+(A)$, we get $d \leq |A \setminus \{u\}| + |N^+(A)| < |A| + k \leq 2k$. □

In the context of the present work, we actually only need the following implication.

Corollary 5.45 *If $G = (V, E)$ is a finite strongly connected vertex-transitive graph without edges of the form (v, v) for $v \in V$, then G is strongly $\frac{3}{2}$-connected.*

Proof. Let d be the outdegree of G. For $d = 1$, G forms a single cycle, i.e., a strongly connected digraph where every vertex has outdegree one, which is clearly strongly $\frac{3}{2}$-connected. For $d \geq 2$, by Theorem 5.44, G is strongly 2-connected, hence strongly $\frac{3}{2}$-connected. □

Note that, by Theorem 5.45, the conclusion of Theorem 5.44 is valid for $d = 1$ even after removal of the rounding, which somehow explains our choice of naming for strong $\frac{3}{2}$-connectivity.

5.5.3 Orbital Graphs

Suppose $G \leq_G \mathcal{S}_X$ is a permutation group. Let $a, b \in X$. Then we can associate two graphs with the tuple (a, b) in a natural way.

[9] For undirected graphs, atoms were introduced in [427, 428, 627].

1. The *directed orbital graph* of the tuple (a, b) is the graph with vertex set X and edge set $\{(g(a), g(b)) \mid g \in G\}$.
2. The *undirected orbital graph* of the tuple (a, b) (or the 2-set $\{a, b\}$) is the graph with vertex set X and edge set $\{\{g(a), g(b)\} \mid g \in G\}$.

Example 5.46 Figure 5.1 shows the directed orbital graphs of the group $G = \langle\{(0\,1\,2\,3)\}\rangle_G$. The undirected orbital graphs are obtained from these by forgetting about the directions on the edges. Since G contains no element swapping two points, the orbital graphs are oriented.

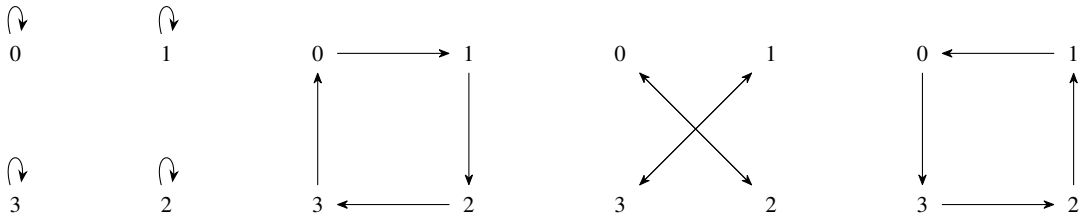


Fig. 5.1: The directed orbital graphs of $G = \langle\{(0\,1\,2\,3)\}\rangle_G$.

Example 5.47 Figure 5.2 shows the directed orbital graphs of the group $G = \langle\{(0\,1)(2\,3), (0\,2)(1\,3)\}\rangle_G$. All of them except the left-most one are symmetric.

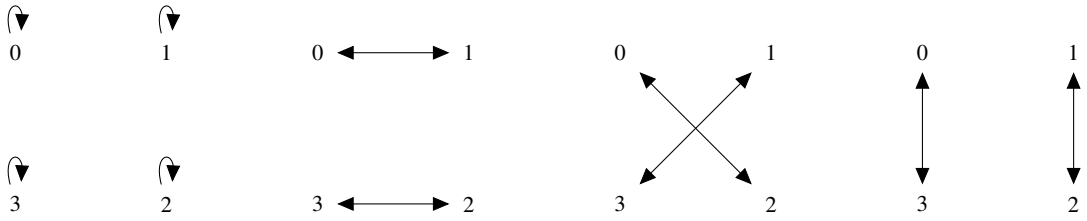


Fig. 5.2: The directed orbital graphs of $G = \langle\{(0\,1)(2\,3), (0\,2)(1\,3)\}\rangle_G$.

The edge set of the directed orbital graph of (a, b) is contained in $\mathrm{Orb}_G(a) \times \mathrm{Orb}_G(b)$. The directed orbital graph of (a, b) is either symmetric or oriented, and the former case applies precisely if G contains a permutation swapping a and b [249, Lem. 2.4.2]. Also, G is intransitive if and only if in the (directed or undirected) orbital graph exist vertices that are not part of any edge. Furthermore, G is a subgroup of the automorphism group of each (directed or undirected) orbital graph. So, by Theorem 5.43 the weakly connected components of a finite directed orbital graph are precisely the strongly connected components. Finally, a (directed or undirected) graph is vertex-transitive if and only if it is a union of orbital graphs w.r.t. its automorphism group.

Lemma 5.48 *Let $G \leq_G \mathcal{S}_n$ and $a, b \in [n]$. Then $\mathrm{Orb}_G(a) \neq \mathrm{Orb}_G(b)$ if and only if in the undirected orbital graph of $\{a, b\}$ (resp. directed orbital graph of (a, b)) there exists no edge with both endpoints in $\mathrm{Orb}_G(a)$ or both endpoints in $\mathrm{Orb}_G(b)$ (resp. the starting vertex of each edge is in $\mathrm{Orb}_G(a)$ and its ending vertex is in $\mathrm{Orb}_G(b)$).*

Proof. Since $\mathrm{Orb}_G(a)$ and $\mathrm{Orb}_G(b)$ are orbits, we have $\mathrm{Orb}_G(a) = \mathrm{Orb}_G(b)$ if and only if there exists $g \in G$ such that $g(a) \in \mathrm{Orb}_G(b)$ or $g(b) \in \mathrm{Orb}_G(a)$. So, $\{g(a), g(b)\} \subseteq \mathrm{Orb}_G(a)$ or $\{g(a), g(b)\} \subseteq \mathrm{Orb}_G(b)$ for some $g \in G$ if and only if G is transitive, which gives the claim for the undirected orbital graph. Furthermore, since the edge set of the directed orbital graph is in $\mathrm{Orb}_G(a) \times \mathrm{Orb}_G(b)$, if G is intransitive every edge of the directed orbital graph starts in $\mathrm{Orb}_G(a)$ and ends in $\mathrm{Orb}_G(b)$. □

5.6 Discussion

Jordan [346] wrote the first textbook on permutation group theory (but with an antique notation and terminology). Prior to this, permutation groups in the context of polynomial equations (as permutations of their root) were investigated by Lagrange, Galois, Abel and Ruffini, see [370, 466, 641]. At the end of the 19th century, Burnside published his classical textbook [119] (1st edition 1897) which collected and contained most results from 19th century group theory with a focus on permutation groups. Wielandt [638] wrote a standard textbook on permutation group theory which established some standard terminology. Other standard introductions are the textbooks by Dixon & Mortimer [179] and by Cameron [126]. Interestingly, permutation group theory, in its historical development, preceded semigroup theory, see [318].

Before the middle of the 19th century, groups were synonymous to permutation groups on a finite set and were simply thought of as non-empty subsets of permutations closed under function composition (called substitution groups back then). The concept of an abstract group slowly emerged. Cayley [137, 138, 139] was the first who viewed groups more abstract solely in terms of laws, and not in some permutation representation. However, his conceptualization was still far away from our modern view and still relied on implicit assumptions. Weber gave an axiom system for finite groups [629] and for general groups [628]. During the 19th century many further famous mathematician besides the ones already mentioned were involved into forming the abstract theory of groups as, for example, Cauchy, Kronecker, Dyck, Frobenius, and Hölder to name a few. The first definition of an abstract group in its modern form appears in van der Waerden's textbook [625], which is based on lectures by Artin and Noether from the beginning of the 20th century. For more information on the history and genesis of abstract group theory, see [370, 461, 641].

Not all authors use the term G-congruence (some just call them equivalence relations preserved by the group), but it is a notion that appears in the literature, for example in [117]. The generalization of it to transformation semigroups is obvious, but I have not seen in the literature. In [367], in the more general context of semigroup actions, the authors introduce the term S-act congruences, and G-congruences are also used in the more general context of group actions, also called G-sets, for example, in [467].

The books [556, 120, 319] contains information on algorithms for groups and permutation groups. Beaudry [75] investigated the computational complexity of deciding whether all transformations in a given transformation monoid are idempotent.

Vertex-transitive graphs seemed to be introduced by Sabidussi [540], at least this is the first reference that we were able to find where this term was introduced. Vertex-transitive graphs are also called *point-symmetric* in [278]. The study of symmetry in graphs was initiated by Foster [226]. More information on vertex-transitive graphs can be found in [48, 90, 127, 498].

Chapter 6
Notions From Automata Theory

Recall the definition of a (semi-)automaton and the set of synchronizing words from Sections 1.1 and 1.2. Here, we give additional background on automata. Section 6.1 collects more basic notions. The reachability quasiorder and related notions are introduced in Section 6.3. Section 6.2 introduces the transformation semigroup of an automaton and a representation by transition matrices. Section 6.4 investigates morphisms on automata. We introduce endo-, iso- and automorphisms, and collect several properties, facts, and formulas; in particular how morphisms act w.r.t. the reachability order, strongly connected components, and upper sets. Then a sequence of sections follows devoted to various constructions. Specifically, Section 6.5.1, 6.5.2, 6.5.3, 6.5.4, 6.5.5, and 6.5.6 introduce the sub- and restricted, quotient, Rees quotient, product, power, and pair automaton, respectively. Section 6.6 discusses the set of maximal states and shows ways to decompose an automaton into a sequence of upper sets. In Section 6.7 we connect automata to formal languages. We introduce regular languages as languages accepted by automata when adjoined with a start state and a set of final states (which we then call automata as language acceptors). Section 6.7.2 investigates the minimal automaton accepting a given regular language.

6.1 More Basic Notions

Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton. For $u, v \in \Sigma^*$, $S \subseteq Q$ and $q \in Q$, we set $\delta^{-1}(S, u) = \{p \in Q \mid \delta(p, u) \in S\}$ and $\delta^{-1}(q, u) = \delta^{-1}(\{q\}, u)$. We have the following formula (related to Equation (3.5)):

$$\delta^{-1}(S, uv) = \delta^{-1}(\delta^{-1}(S, v), u). \tag{6.1}$$

The *automaton (di-)graph* corresponding to $\mathcal{A}$ is the digraph $G(\mathcal{A}) = (Q, E)$ with $E = \{(s, t) \mid \exists x \in \Sigma : \delta(s, x) = t\}$.

We say $\mathcal{A}$ has a *self-loop* at $q \in Q$ w.r.t. $x \in \Sigma$ (or x induces a self-loop at q) if $\delta(q, x) = q$. The state $q \in Q$ is called a *trap* (or *sink*) *state* if $\delta(q, x) = q$ for all $x \in \Sigma$, i.e., every letter induces a self-loop at q.[1]

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called a *unary* (resp. *binary*, *ternary*) *automaton* if $|\Sigma| = 1$ (resp. $|\Sigma| = 2$, $|\Sigma| = 3$). The automata from the Černý family (Figure 1.7) are binary.

We call $\mathcal{A}$ *primitive* if the trivial $\mathcal{T}_S(\mathcal{A})$-congruences are the only $\mathcal{T}_S(\mathcal{A})$-congruences.

A subset $S \subseteq Q$ is said to be a *block* in $\mathcal{A}$ if it is non-empty and $\delta(S, x) = S$ or $\delta(S, x) \cap S = \emptyset$ for all $x \in \Sigma$. This extends the notion of a block from permutation group theory introduced in Chapter 5 (see 111) to automata. The context should make it clear whether we are referring to a block in a permutation group or in an automaton, so no confusion should arise.

6.2 The Transformation Semigroup of an Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. Each word $u \in \Sigma^*$ *induces a transformation* $\delta_u \colon Q \to Q$ by $\delta_u(q) = \delta(q, u)$ for all $q \in Q$. For $u, v \in \Sigma^*$ we have $\delta_{uv} = \delta_v \delta_u$ and $\delta_{uv}^{-1} = \delta_u^{-1} \delta_v^{-1}$. Compare the latter equation with Equation (6.1) wheres u and v are "swapped" in the expression on the right hand side. This follows as in the δ-notation the left-most word is applied first whereas in the composition of two functions, by our definition, the right-most function is applied first.

We call a letter $x \in \Sigma$ (resp. a word $u \in \Sigma^*$) a *permutational / idempotent / simple idempotent* letter (resp. word) if δ_x (resp. δ_u) is a permutation/idempotent/simple idempotent. The *rank* (resp. *defect*, *excluded set*, *duplicate set*) of a word $u \in \Sigma^*$, denoted by $\operatorname{rank}(u)$ (resp. $\operatorname{def}(u)$, $\operatorname{excl}(u)$, $\operatorname{dupl}(u)$) is the rank (resp. defect, excluded set, duplicate set) of δ_u. Furthermore, when using terminology from functions, transformations, and permutations, as cyclically permuting the states, the image of a set, transition matrix, being a cycle, having a fixed point etc., in the context of a word (or letter) $u \in \Sigma^*$, we refer to the induced transformation δ_u. We also say that δ_u is the *action of* u on the states or that u *acts as* δ_u. The same terminology transfers when speaking about properties of δ_u, for example, we say that u acts as a cycle if δ_u is a cycle.

For $u \in \Sigma^*$, we write $[u]_{\mathcal{A}}$ (or just $[u]$ when there is no ambiguity to which automaton we refer) for the transition matrix of δ_u. Figure 6.1 depicts transition matrices for the Černý automata C_n from Section 1.4 w.r.t. the natural order of its state set $[n]$. When the specific ordering of the states is not important, we assume some order to be taken without explicitly mentioning it.[2]

The *transformation semigroup of* $\mathcal{A}$ is

[1] A sink state is sometimes defined to be a trap state reachable from every other state, but here it is just another name for a trap state.

[2] The maps $u \mapsto \delta_u$ and $u \mapsto [u]$ are morphisms from Σ^* into $\mathcal{T}_Q$ and the set of $|Q| \times |Q|$ matrices, respectively. Thus, by Theorem 3.60, they are uniquely determined by their images on the letters Σ.

$$[b]_{C_n} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{pmatrix} \qquad [a]_{C_n} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \\ 0 & 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$$

Fig. 6.1: The transition matrices for the Černý automata C_n from Section 1.4 (Figure 1.7) where the states are ordered w.r.t. the natural order on $[n]$.

$$\mathcal{T}_S(\mathcal{A}) = \{\delta_u \mid u \in \Sigma^+\}$$

and the *transformation monoid* is $\mathcal{T}_M(\mathcal{A}) = \{\delta_u \mid u \in \Sigma^*\}$.

Proposition 6.1 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and δ_u be a permutation for $u \in \Sigma^*$. Then $\delta_u^{-1} \in \mathcal{T}_M(\mathcal{A})$, i.e., the inverse permutation is induced by some word over Σ^*.*

Proof. By finiteness, we have $\delta_{u^i} = \mathrm{id}_Q$ for some $i > 0$. Then $\delta_u^{-1} = \delta_{u^{i-1}} \in \mathcal{T}_M(\mathcal{A})$. □

Next, we determine the group of units in $\mathcal{T}_M(\mathcal{A})$.

Proposition 6.2 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\Gamma = \{x \in \Sigma \mid x \text{ is a permutational letter}\}$. Then for the group of units G of $\mathcal{T}_M(\mathcal{A})$ we have*

$$G = \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\},$$

and if G is non-trivial, equivalently $\Gamma \neq \emptyset$, we also have $G = \{\delta_u \mid u \in \Gamma^+\}$.

Proof. We have $G \subseteq \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\}$ as every transformation in G, having an inverse, is a permutation. Next, by Theorem 3.12, Item 3.18 the composition of a non-permutation with a transformation gives a non-permutation. So, for $u \in \Sigma^*$, δ_u is a permutation if and only if $u \in \Gamma^*$, i.e.,

$$\{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\}.$$

By Theorem 6.1, for $u \in \Gamma^*$ we have $\delta_u^{-1} = \delta_v$ for some $v \in \Sigma^*$, i.e., δ_u has an inverse in $\mathcal{T}_M(\mathcal{A})$ and so $\delta_u \in G$. This gives $\{\delta_u \mid u \in \Gamma^*\} \subseteq G$. Combining everything, we find $G = \{\delta_u \mid \delta_u \text{ is a permutation and } u \in \Sigma^*\} = \{\delta_u \mid u \in \Gamma^*\}$.

Lastly, if $\Gamma \neq \emptyset$, then δ_u equals the identity transformation for some $u \in \Gamma^+$. This gives $\{\delta_u \mid u \in \Gamma^*\} = \{\delta_u \mid u \in \Gamma^+\}$. □

Thus the group of units of $\mathcal{T}_M(\mathcal{A})$ is precisely the set of permutations in $\mathcal{T}_M(\mathcal{A})$ and every permutation group in $\mathcal{T}_S(\mathcal{A})$ is contained in the group of units of $\mathcal{T}_S(\mathcal{A})$ (but not every subset $S \subseteq \mathcal{T}_M(\mathcal{A})$ that forms a group in itself; for example for every non-permutational word u permuting its image $\delta(Q, u)$ the set $\{\delta_{u^i} \mid i \geq 1\}$ forms a group, but contains no permutation).

6.3 The Reachability Order

First, in Section 6.3.1 we introduce the reachability order. Then, Section 6.3.2 investigates connectivity notions and the notion of an upper set derived from the reachability order.

6.3.1 Definition and a Characterization

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $s, t \in Q$. We define the *reachability (quasi)order*

$$s \preceq_R^{\mathcal{A}} t \Leftrightarrow \exists u \in \Sigma^* \colon \delta(s, u) = t.$$

We say that t is *reachable* from s (in $\mathcal{A}$) if $s \preceq_R^{\mathcal{A}} t$. When the automaton we refer to is clear from context, we simply write $\preceq_R$. The reachability order is characterized as the finest quasiorder such that $s \preceq_R \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$, as we formally state next.

Proposition 6.3 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We have $s \preceq_R \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$. Further, if $\lesssim$ is a quasiorder on Q such that $s \lesssim \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$, then $s \preceq_R t$ implies $s \lesssim t$ for all $s, t \in Q$.*

Proof. Clearly, we have $s \preceq_R \delta(s, u)$ for $s \in S$ and $u \in \Sigma^*$. Next, suppose $\lesssim$ is a quasiorder such that $s \lesssim \delta(s, u)$ for all $s \in Q$ and $u \in \Sigma^*$. Now, for $s, t \in Q$, $s \preceq_R t$ is equivalent to $\delta(s, u) = t$ for some $u \in \Sigma^*$. So $s \lesssim \delta(s, u) = t$. □

By $\simeq_R$ we denote the *(reachability) equivalence relation* induced by $\preceq_R$ according to Equation (3.30).

6.3.2 Connectivity and Upper Sets

Let Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$. If nothing else is said, order-theoretical notions in the context of automata are meant w.r.t. the reachability order. For example, by calling S an *upper set*, we mean it to be an upper set w.r.t. the reachability quasiorder. This is equivalent to $\delta(S, x) \subseteq S$ for all $x \in \Sigma$. Note that the empty set $\emptyset$ is an upper set in each automaton. Since then $\delta(S, u) \subseteq \delta(Q, u) \cap S$ for all $u \in \Sigma^*$, we obtain the following equation:

$$S \text{ is an upper set} \Rightarrow \forall u \in \Sigma^* \colon \delta(Q, u) \cap S \neq \emptyset. \tag{6.2}$$

The automaton $\mathcal{A} = (\{q_1, q_2\}, \{a\}, \delta)$ with $\delta(q_1, a) = q_2$ and $\delta(q_2, a) = q_1$ shows, by considering $S = \{q_1\}$, that the condition on the right hand side can hold for sets that are not upper sets. A state $t \in Q$ is a trap state if and only if $\{t\}$ is an upper set. We also have

$$S \text{ is upper set and block} \Leftrightarrow \forall x \in \Sigma \colon \delta(S, x) = S \Leftrightarrow \forall u \in \Sigma^* \colon \delta(S, u) = S. \tag{6.3}$$

A subset $S \subseteq Q$ is called *strongly connected* (resp. *unilaterally connected*) if, for all $s, t \in S$, we have $s \preceq_R t$ (resp. $s \preceq_R t$ or $t \preceq_R s$). A maximal strongly connected subset is called a *strongly connected component* and equals an equivalence class w.r.t. $\simeq_R$. A strongly connected component that is an upper set is called a *sink (strongly connected) component*. Equivalently, it is an upper set that is strongly connected. By Theorem 3.43, a sink component is precisely a minimal non-empty upper set w.r.t. set inclusion. In the automaton graph, the sink components are the strongly connected components without transitions leading out of it.

Proposition 6.4 *In an automaton, the minimal possible rank of a word is bounded from below by the number of distinct sink components.*

Proof. This follows from Equation (6.2) and the fact that distinct sink components are disjoint. □

Proposition 6.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then S is a union of sink components if and only if $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$.*

Proof. First, suppose $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$. Consider some $q \in S$. By assumption $\uparrow\{q\} \subseteq [q]_{\simeq_R}$, and so $[q]_{\simeq_R}$ is a sink component by Theorem 3.43. Since S is an upper set, we get $[q]_{\simeq_R} \subseteq S$ and so S equals the union of all $[q]_{\simeq_R}$ with $q \in S$.

Conversely, suppose S is a union of sink components. Let $q \in S$. Then q is contained in some sink component, which yields that $[q]_{\simeq_R}$ is a sink component and so $\delta(q, u) \simeq_R q$. □

Corollary 6.6 *A set permuted by each letter equals a union of sink components.*

Proof. Let $S \subseteq Q$ be a set in $\mathcal{A} = (Q, \Sigma, \delta)$ such that $\delta(S, a) = S$ for each $a \in \Sigma$. Then S is an upper set. Consider $q \in S$ and $u \in \Sigma^*$. Since the word u permutes S, there exists $i > 0$ such that $\delta(p, u^i) = p$ for all $p \in Q$. Then $\delta(q, uu^{i-1}) = q$ gives $\delta(q, u) \preceq_R q$ and since $q \preceq_R \delta(q, u)$ the claim follows from Theorem 6.5. □

We call $\mathcal{A} = (Q, \Sigma, \delta)$ *strongly connected* (resp. *unilaterally connected*) if Q is strongly connected (resp. unilaterally connected). For $Q \neq \emptyset$, $\mathcal{A}$ is strongly connected if and only if Q is a strongly connected component. We call $\mathcal{A}$ *cyclic* [339] if $Q = \uparrow\{q\}$ for some $q \in Q$.

Proposition 6.7 *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is strongly connected if and only if there exists no proper non-empty upper set, i.e., $\emptyset$ and Q are the only upper sets.*

Proof. This follows from Theorem 3.42. □

Proposition 6.8 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with a trap state $t \in Q$ and $|Q| \geq 2$. Then $Q \setminus \{t\}$ is a strongly connected component if and only if Q and $\{t\}$ are the only non-empty upper sets in $\mathcal{A}$.*

Proof. This follows from Theorem 3.42 with $S = \{t\}$. □

Proposition 6.9 *An unilaterally connected automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is cyclic.*

Proof. Let $q \in Q$ be such that $\uparrow\{q\}$ has maximal size. Assume $p \notin \uparrow\{q\}$ for some $p \in Q$. Since $\mathcal{A}$ is unilaterally connected, we have $q \in \uparrow\{p\}$ and so $\uparrow\{q\} \subsetneq \uparrow\{p\}$, contradicting the maximal choice of q. □

The converse of Theorem 6.9 does not hold. For example, for $\mathcal{A} = (\{q_1, q_2, q_3\}, \{a, b\}, \delta)$ with $\delta(q_1, a) = q_2$, $\delta(q_1, b) = q_3$, and $\delta(q_i, x) = q_i$ for $i \in \{2, 3\}$ and $x \in \{a, b\}$ we have $\uparrow\{q_1\} = \{q_1, q_2, q_3\}$, but it is not unilaterally connected.

By the results from Section 3.6, a subset of states is a strongly connected component if and only if for each upper set it is contained in this set or its complement (Theorem 3.37), every upper set is a union of strongly connected components (Theorem 3.38), and for the set difference $T \setminus S$ of two upper sets S, T the following are equivalent: strong connectivity, being a strongly connected component, and maximality relative to T and set inclusion (Theorems 3.39 and 3.42).

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is said to be *strongly trap-connected* if there exists a trap state $t \in Q$ such that for all $p \in Q \setminus \{t\}$ and $q \in Q$ we have $p \precsim_R q$. Clearly, a strongly trap-connected automaton has a unique trap-state, is synchronizing, and unilaterally connected.

Proposition 6.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$. Then the following are equivalent.*

1. *$\mathcal{A}$ is strongly trap-connected.*
2. *$Q = S \cup \{t\}$ for a trap state $t \in Q$ and a non-sink strongly connected component S.*
3. *There exists $t \in Q$ such that $\{t\}$ is the unique proper and non-empty upper set.*
4. *$\mathcal{A}$ is not strongly connected and has no proper upper set of size at least two.*

Proof. 1 $\Rightarrow$ 2: Let t be the unique trap state. Suppose R is a non-empty upper set distinct from $\{t\}$. Then there exists $q \in R \setminus \{t\}$. Since $\mathcal{A}$ is strongly trap-connected, we have $\uparrow\{q\} = Q$. As an upper set, it is $\uparrow\{q\} \subseteq R$ and so $R = Q$. Thus Q and $\{t\}$ are the only non-empty upper sets. Since $\{t\}$ is a sink component, Theorem 3.47 gives the claim.

2 $\Rightarrow$ 3: This follows from Theorem 3.47.

3 $\Rightarrow$ 4: Since $\mathcal{A}$ has at least two states and an upper set $\{t\}$ of size one, by Theorem 6.7 it is not strongly connected. By assumption $\{t\}$ is a unique proper and non-empty upper set, so that $\mathcal{A}$ has no proper upper set of size at least two.

4 $\Rightarrow$ 1: Since $\mathcal{A}$ is not strongly connected, by Theorem 6.7 there exist some proper and non-empty upper set $S \subseteq Q$. By assumption, $S = \{t\}$ for some $t \in Q$. Let $q \in Q \setminus \{t\}$. Then $\uparrow\{q\} \cup \{t\}$ is an upper set of size at least two. Hence, we have $\uparrow\{q\} \cup \{t\} = Q$. This gives $|\uparrow\{q\}| \geq 2$. Now, we have $t \in \uparrow\{q\}$ as otherwise, $\uparrow\{q\}$ is a proper upper set of size at least two, and so $\uparrow\{q\} = Q$. □

Corollary 6.11 *In automata with at least three states, there exists no proper upper set of size at least two in $\mathcal{A}$ if and only if $\mathcal{A}$ is strongly connected or strongly trap-connected.*

Proof. By Theorems 6.7 and 6.10, a strongly connected or strongly trap-connected automaton has no proper upper set of size at least two. Conversely, the automaton is either strongly connected or, by Theorem 6.10, Item 4 it has no proper upper set of size at least two. □

A two-state automaton where every letter induces a self-loop also has no proper upper set of size at least two, but is neither strongly connected nor strongly trap-connected.

6.4 Morphisms

Here, we introduce morphisms, endomorphisms, isomorphisms, and automorphisms of automata and collect basic facts about them.

6.4.1 Definitions and Basic Facts

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ be two semiautomata.

1. A *(semiautomaton) (homo-)morphism* from $\mathcal{A}$ to $\mathcal{B}$ (or between $\mathcal{A}$ and $\mathcal{B}$) is a function $\varphi\colon Q \to S$ such that for each $q \in Q$ and $x \in \Sigma$ we have

   $$\varphi(\delta(q, x)) = \mu(\varphi(q), x).$$

   Equivalently, φ is a morphism if and only if $\varphi\delta_x = \mu_x\varphi$ for all $x \in \Sigma$. Thus, regardless of which of the two paths we follow in the diagram below, we end up at the same state.

   $$\begin{array}{ccc} Q & \xrightarrow{\delta(\cdot,x)} & Q \\ {\scriptstyle\varphi}\downarrow & & \downarrow{\scriptstyle\varphi} \\ S & \xrightarrow[\mu(\cdot,x)]{} & S \end{array}$$

   If φ is surjective, we say that it maps $\mathcal{A}$ *onto* $\mathcal{B}$.
2. A *(semiautomaton) endomorphism* is a morphism with $\mathcal{A} = \mathcal{B}$.
3. A *(semiautomaton) isomorphism* is a morphism $\varphi\colon Q \to S$ such that there exists a morphism $\psi\colon S \to Q$ with $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$. The automata $\mathcal{A}$ and $\mathcal{B}$ are called *isomorphic*, written

   $$\mathcal{A} \cong \mathcal{B},$$

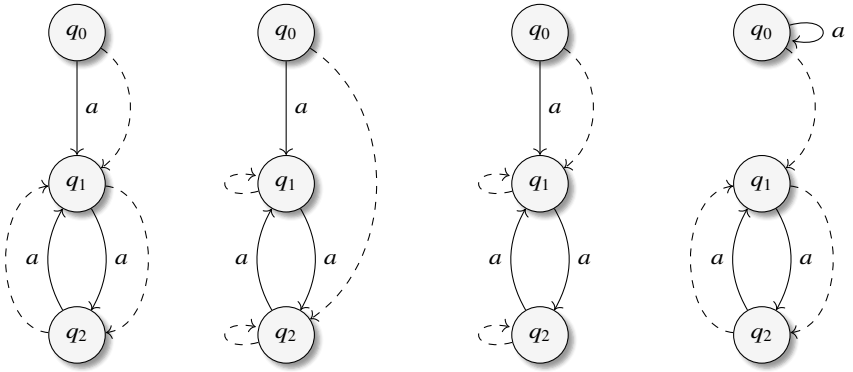
   if there exists an isomorphism between them.
4. A *(semiautomaton) automorphism* is an endomorphism that is an isomorphism.

The set of endomorphisms of a given automaton forms a transformation semigroup and the set of automorphisms forms a permutation group.

Example 6.12 Consider the automata from Figure 6.2, where the dashed lines indicate a mapping of their state sets.

In the automaton from Figure 6.2a the mapping is an endomorphism that does not map the strongly connected component $\{q_0\}$ to a strongly connected component, but to $\{q_1\}$. Another endomorphism of the same automaton is given in Figure 6.2b. Together with the identity transformation, this automaton has exactly three endomorphisms.

In Figures 6.2c and 6.2d, the mapping φ is not a morphism since in both cases we have $\varphi(\delta(q_0, a)) = q_1 \neq q_2 = \delta(\varphi(q_0), a)$.



(a) An endomorphism. (b) An endomorphism. (c) Not a morphism. (d) Not a morphism.

Fig. 6.2: Automata and mappings of their states indicated by dashed lines, see Theorem 6.12.

Example 6.13 Let $\mathcal{A} = (\{q_1, q_2\}, \{a_1, a_2\}, \delta)$ with $\delta(q_1, a_i) = \delta(q_2, a_i) = q_i$ for $i \in \{1, 2\}$, see Figure 6.3. If $\varphi\colon Q \to Q$ is an endomorphism, then $\varphi(q_i) = \varphi(\delta(q_i, a_i)) = \delta(\varphi(q_i), a_i) = q_i$ for $i \in \{1, 2\}$. Thus the identity transformation is the only endomorphism of $\mathcal{A}$.

Later, in Theorem 6.45, we will show that, in general, a strongly connected and synchronizing automaton only admits the identity transformation as an endomorphism.

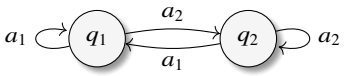


Fig. 6.3: Automaton not admitting any non-identity endomorphism, see Theorem 6.13.

Next, we give some alternative characterizations for morphisms.

Proposition 6.14 *Let $\varphi\colon Q \to S$ be a function. Then the following are equivalent.*

1. *φ is a morphism.*
2. *For all $p \in Q$ and all $u \in \Sigma^*$ we have*

$$\varphi(\delta(p, u)) = \mu(\varphi(p), u)$$

3. *There exists $T \subseteq Q$ with $Q = \bigcup_{p \in T} \uparrow\{p\}$ such that for all $p \in T$ and all $u \in \Sigma^*$ we have*

$$\varphi(\delta(p, u)) = \mu(\varphi(p), u)$$

Proof. $1 \Rightarrow 2$: This clearly holds for $u = \varepsilon$. Next, let $u = va$ for some $v \in \Sigma^*$ and $a \in \Sigma$. Inductively, we have $\varphi(\delta(p, v)) = \mu(\varphi(p), v)$. Using that φ is a morphism, this gives

$\varphi(\delta(p,u)) = \varphi(\delta(\delta(p,v),a)) = \mu(\varphi(\delta(p,v)),a) = \mu(\mu(\varphi(p),v),a) = \mu(\varphi(p),u)$. Then the claim follows by induction.

$2 \Rightarrow 3$: By assumption, this holds for $T = Q$.

$3 \Rightarrow 1$: Let $a \in \Sigma$ and $q \in Q$. By assumption, there exists $u \in \Sigma^*$ such that $q = \delta(p,u)$ for some $p \in T$. Set $s = \varphi(p)$. Then we have

$$\begin{aligned}
\varphi(\delta(q,a)) &= \varphi(\delta(p,ua)) && (q = \delta(p,u)) \\
&= \mu(s,ua) && (\varphi \text{ morphism}) \\
&= \mu(\mu(s,u),a) && (\text{Def. transition function}) \\
&= \mu(\varphi(\delta(p,u)),a) && (\varphi(\delta(p,u)) = \mu(s,u)) \\
&= \mu(\varphi(q),a). && (q = \delta(p,u))
\end{aligned}$$

Thus, φ is a morphism. □

By Item 3 of Theorem 6.14, a morphism is completely specified by the images of $\varphi(p_1), \ldots, \varphi(p_m)$ for some $p_1, \ldots, p_m \in Q$ such that $Q = \uparrow\{p_1\} \cup \cdots \cup \uparrow\{p_m\}$.

Corollary 6.15 *If $\mathcal{A}$ is cyclic, there are at most $|S|$ morphisms from $\mathcal{A}$ into $\mathcal{B}$. In particular, there are at most $|Q|$ endomorphisms of $\mathcal{A}$.*

Proof. By Theorem 6.24 two morphisms having the same image for a state p with $Q = \uparrow\{p\}$ are equal. Since we have at most $|S|$ distinct images for this state, the claim follows. □

Example 6.16 The might be strictly less than $|Q|$ endomorphisms in a cyclic automaton. For example, the automaton from Theorem 6.13 is cyclic and admits no non-identity morphism.

In the rest of this section, let $\varphi\colon Q \to S$ be a morphism. For $T \subseteq Q$, we call $\varphi(T)$ the *morphic image (of T)*.

By our next statement, isomorphisms are precisely the bijective morphisms. Note that a similar statement does not hold for order morphisms, see Theorem 3.32.

Proposition 6.17 *The morphism $\varphi\colon Q \to S$ is an isomorphism if and only if it is bijective.*

Proof. By Theorem 3.3, existence of a map $\psi\colon S \to Q$ such that $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$ is equivalent to bijectivity of φ. So, an isomorphism is bijective.

Next, assume φ is bijective. Since φ is surjective, we have $\varphi\psi = \mathrm{id}_S$ for some $\psi\colon S \to Q$ by Theorem 3.3. Let $s \in S$ and $x \in \Sigma$. Then we find

$$\begin{aligned}
\varphi(\delta(\psi(s),x)) &= \mu(\varphi(\psi(s)),x) && (\varphi \text{ morphism}) \\
&= \mu(s,x) && (\varphi\psi = \mathrm{id}_S) \\
&= \varphi(\psi(\mu(s,x))). && (\varphi\psi = \mathrm{id}_S)
\end{aligned}$$

Since φ is injective, $\varphi(\delta(\psi(s),x)) = \varphi(\psi(\mu(s,x)))$ implies $\delta(\psi(s),x) = \psi(\mu(s,x))$. Thus ψ is a morphism. □

Remark 6.18 By Theorem 3.3 a morphism $\varphi\colon Q \to S$ is injective (resp. surjective) if there exists a morphim $\psi\colon S \to Q$ such that $\psi\varphi$ (resp. $\varphi\psi$). However, unlike for functions, the converse implication does not hold in general.

To show this, for $i \in \{1,2,3\}$, let $\mathcal{A}_i = (Q_i, \{a\}, \delta_i)$ where $Q_1 = \{q_1, q_2\}$, $Q_2 = \{q_1, q_2, t\}$, $Q_3 = \{t\}$, and where the letter a swaps the states q_1 and q_2 in $\mathcal{A}_1$ and $\mathcal{A}_2$ and fixes the state t in $\mathcal{A}_2$ and $\mathcal{A}_3$. Define $\varphi\colon Q_1 \to Q_2$ by $\varphi(q_1) = q_1$, $\varphi(q_2) = q_2$, and $\psi\colon Q_1 \to Q_3$ by $\psi(q_1) = \psi(q_2) = t$. Then φ is injective and ψ is surjective and both maps are morphisms. However, there exists no morphism from $\mathcal{A}_2$ to $\mathcal{A}_1$, and no morphism from $\mathcal{A}_3$ to $\mathcal{A}_1$.

Proposition 6.19 *If $t \in Q$ is a trap state, then $\varphi(t)$ is a trap state.*

Proof. Let $x \in \Sigma$. Then $\mu(\varphi(t), x) = \varphi(\delta(t, x)) = \varphi(t)$ and so $\varphi(t)$ is a trap state. □

Proposition 6.20 *If φ is surjective, the image of a synchronizing state is a synchronizing state.*

Proof. Let $\delta(Q, u) = \{q\}$ for $u \in \Sigma^*$ and $q \in Q$. Then $\mu(S, u) = \mu(\varphi(Q), u) = \varphi(\delta(Q, u)) = \{\varphi(q)\}$ □

Next, a morphism of automata induces a morphism of the underlying digraphs.

Proposition 6.21 *$\varphi\colon Q \to S$ is also a graph morphism of the automaton digraphs of $\mathcal{A}$ and $\mathcal{B}$.*

Proof. For $p \in Q$ and $x \in \Sigma$, the edge $(p, \delta(p, x))$ is mapped to the edge $(\varphi(p), \mu(\varphi(p), x))$ by φ in the respective automaton digraphs, which gives the claim. □

The following two examples gives mappings that are graph morphisms of the automaton digraphs, but not automaton morphisms.

Example 6.22 Let $\mathcal{A} = (\{p, q\}, \{a, b\}, \delta)$ with $\delta(p, x) = q$ for $x \in \{a, b\}$, $\delta(q, a) = p$, and $\delta(q, b) = q$. Then the transformation φ that maps all states to q is a graph morphism of the automaton digraph of $\mathcal{A}$, but it is not an endomorphism since $\delta(\varphi(q), a) = p$ and $\varphi(\delta(q, a)) = q$.

Example 6.23 Let $\mathcal{A} = (\{q_1, \ldots, q_n\}, \Sigma, \delta)$ with $n \geq 2$ be such that for each pair of distinct states q_i, q_j there exists a letter a_{ij} inducing a simple idempotent mapping q_i to q_j. Every permutation of $\{q_1, \ldots, q_n\}$ is a graph automorphism of the automaton digraph of $\mathcal{A}$. However, $\mathcal{A}$ has no non-identity automaton morphism since if $\varphi(q_i) = q_j$ with $i \neq j$, then $\varphi(\delta(q_i, a_{ji})) = q_j$, but $\delta(\varphi(q_i), a_{ji}) = q_i$.

6.4.2 Morphisms and the Reachability Order

In this section, let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ be two automata.

First, we show that two morphisms agree on an upper set of the form $\uparrow\{p\}$ if and only if they agree at p.

Proposition 6.24 *For morphisms $\varphi, \psi : Q \to S$ and $p \in Q$, we have*

$$\varphi(p) = \psi(p) \Rightarrow \forall q \in \uparrow\{p\} : \varphi(q) = \psi(q) \tag{6.4}$$

In particular, two morphism φ, ψ agree on a strongly connected component or have different images for each state of the strongly connected component.

Proof. Let $q \in \uparrow\{p\}$. Then we have $q = \delta(p, u)$ for some $u \in \Sigma^*$. By Theorem 6.14 we find $\varphi(\delta(p, u)) = \mu(\varphi(p), u) = \mu(\psi(p), u) = \psi(\delta(p, u))$. □

For the remainder of this section, suppose $\varphi : Q \to S$ is a morphism. Let $p, q \in Q$. Then φ is an order morphism with respect to $\preceq_R$. Indeed, if $p \preceq_R q$, then $q = \delta(p, u)$ for some $u \in \Sigma^*$. Consequently, $\varphi(q) = \varphi(\delta(p, u)) = \mu(\varphi(p), u)$, so $\varphi(p) \preceq_R \varphi(q)$. Thus, we have shown that[3]

$$p \preceq_R q \Rightarrow \varphi(p) \preceq_R \varphi(q). \tag{6.5}$$

Proposition 6.25 *Let $\varphi : Q \to S$ be a morphism.*

1. *Let $T \subseteq Q$. Then we have*
$$\varphi(\uparrow T) = \uparrow\varphi(T) \tag{6.6}$$
2. *$\varphi(T)$ is an upper set (resp. unilaterally connected, strongly connected, a sink component) for an upper set (resp. unilaterally connected, strongly connected, sink component) $T \subseteq Q$.*
3. *For two strongly connected components $T \subseteq Q$ and $R \subseteq S$, we have $\varphi(T) \cap R = \emptyset$ or $\varphi(T) \subseteq R$.*
4. *Let $T \subseteq Q$ be an upper set. Then $\varphi^{-1}(T)$ is also an upper set.*

Proof. 1: We have

$$\begin{aligned}\varphi(\uparrow T) &= \varphi(\{\delta(p, u) \mid p \in T \wedge u \in \Sigma^*\}) \\ &= \{\varphi(\delta(p, u)) \mid p \in T \wedge u \in \Sigma^*\} \\ &= \{\mu(\varphi(p), u) \mid p \in T \wedge u \in \Sigma^*\} \\ &= \uparrow\varphi(T).\end{aligned}$$

2: Let $T \subseteq Q$ be an upper set. Then $T = \uparrow T$ and, with Item 1, we find $\varphi(T) = \varphi(\uparrow T) = \uparrow\varphi(T)$. Since $\uparrow\varphi(T) \subseteq \varphi(T)$, $\varphi(T)$ is an upper set.

For unilaterally connected T, consider $\varphi(p), \varphi(q)$ for $p, q \in T$. Since $p \preceq_R q$ or $q \preceq_R p$, we have $\varphi(p) \preceq_R \varphi(q)$ or $\varphi(q) \preceq_R \varphi(p)$ and so $\varphi(T)$ is unilaterally connected.

By Theorem 3.29, $\varphi(T)$ is strongly connected if T is strongly connected.

Lastly, suppose T is a sink component. Then $\varphi(T)$ is strongly connected and an upper set since both properties are preserved as shown above.

3: This follows from Theorem 3.31. □

For general order morphisms, Equation (6.6) may fail and the images of upper sets need not be upper sets, see Theorem 3.33.

[3] This also follows from Theorem 6.21 since a morphism maps paths to paths in the automaton digraphs.

Corollary 6.26 *Let $T \subseteq Q$. If $\varphi(T) \subseteq R$ for a sink component R, then $\varphi(\uparrow T) = R$.*

Proof. Since $\varphi(T) \subseteq R$ we have $\uparrow\varphi(T) \subseteq \uparrow R = R$. Since R is strongly connected and $\uparrow\varphi(T) \cap R \neq \emptyset$, we have $R \subseteq \uparrow\varphi(T)$. Thus, with Theorem 6.25, we find $\varphi(\uparrow T) = \uparrow\varphi(T) = R$. □

Remark 6.27 Consider the automata in Figures 6.2a and 6.2b from Theorem 6.12. The image of $\{q_0\}$ is $\{q_1\}$, which is strictly contained in the sink component $\{q_1, q_2\}$, and the image of $\uparrow\{q_0\}$ equals $\{q_1, q_2\}$. Note that $\{q_0\}$ is a strongly connected component, but its image is not a strongly connected component in both automata.

In Figure 6.2d, $\{q_0\}$ is a sink component and its image is not a sink component. Thus, in this case, we also obtain that the mapping is not a morphism from Theorem 6.26.

In general, bijective order morphisms do not have to be order isomorphisms, see Theorem 3.32. By the next statement, with Theorem 6.17, bijective order morphisms arising from automata morphisms are in fact order isomorphisms.

Proposition 6.28 *If $\varphi\colon Q \to S$ is an isomorphism, then it is also an order isomorphism.*

Proof. By assumption, there exists a morphism $\psi\colon S \to Q$ such that $\psi\varphi = \mathrm{id}_Q$ and $\varphi\psi = \mathrm{id}_S$. By Equation (6.5), ψ is also an order morphism. Thus, φ is an order isomorphism. □

Corollary 6.29 *Suppose $\varphi\colon Q \to S$ is an isomorphism. If $T \subseteq Q$ is a strongly connected component, then $\varphi(T)$ is a strongly connected component.*

Proof. This follows from Theorems 3.29 and 6.28. □

Proposition 6.30
1. *If $Q \neq \emptyset$ and $\mathcal{B}$ is strongly connected, or strongly trap-connected where $\varphi(Q)$ has size at least two, then φ is surjective, i.e., $\varphi(Q) = S$.*
2. *If $\mathcal{A}$ is strongly connected (resp. strongly trap-connected) and φ is surjective, then $\mathcal{B}$ is strongly connected (resp. strongly trap-connected).*

Proof. 1: By Item 2 of Theorem 6.25, $\varphi(Q)$ is an upper set. First, suppose $\mathcal{B}$ is strongly connected. By Theorem 6.7, a strongly connected automaton has no proper non-empty upper set. Thus $\varphi(Q) = S$. Next, suppose $\mathcal{B}$ is strongly trap-connected and $|\varphi(Q)| \geq 2$. By Theorem 6.10, $\mathcal{B}$ has no proper upper set of size at least two. Thus $\varphi(Q) = S$.

2: We can assume $|Q|, |\varphi(Q)| \geq 2$.

If $\mathcal{A}$ is strongly connected, then Q is a sink component. Thus, by Item 2 from Theorem 6.25, $\varphi(Q)$ is also a sink component. By surjectivity, $\mathcal{B}$ is strongly connected.

Next, suppose $\mathcal{A}$ is strongly trap-connected. Let $T \subseteq S$ be an upper set in $\mathcal{B}$ of size at least two. By Equation (3.36), $\varphi^{-1}(T)$ is an upper set. Since φ is surjective, $|\varphi^{-1}(T)| \geq 2$. By Theorem 6.10 we find $Q = \varphi^{-1}(T)$. Thus $S = \varphi(Q) \subseteq T$. Since the trap state in $\mathcal{A}$ is mapped to a trap state in $\mathcal{B}$, $\mathcal{B}$ is not strongly connected. Then, Theorem 6.10 gives that $\mathcal{B}$ is strongly trap-connected. □

Corollary 6.31 *If $|Q| = |S|$ and $\mathcal{B}$ is strongly connected, or strongly trap-connected with $|\varphi(Q)| \geq 2$, then φ is an isomorphism (and so $\mathcal{A}$ is also strongly connected or strongly trap-connected).*

Proof. By Item 1 from Theorem 6.30, we have $\varphi(Q) = S$. Since $\operatorname{rank}(\varphi) = |Q|$, by Equation (3.15) from Theorem 3.12 we find $\operatorname{def}(\varphi) = \emptyset$. Thus, φ is bijective. By Theorem 6.17, φ is an isomorphism.

The inverse map of φ is an isomorphism. Since $\mathcal{B}$ is strongly connected or strongly trap-connected, by Item 2 from Theorem 6.30, we find that $\mathcal{A}$ is strongly connected or strongly trap-connected. □

6.4.3 Endomorphisms

By the first result of this section, Theorem 6.32, an endomorphism cannot map a state to one that strictly precedes it in the reachability order. This property does not hold for general quasiorders, see Theorem 3.33. We derive several consequences of this fact. For example, Theorem 6.34, showing that an endomorphism permutes a strongly connected component T if and only if it permutes the upper set generated by T, or Theorem 6.35 showing that an endomorphism either permutes a strongly connected component or maps all of its states to states outside the component. Lastly, we give several results on endomorphisms of strongly connected and synchronizing automata.

In this section, let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and $\varphi\colon Q \to Q$ be an endomorphism.

Proposition 6.32 *Let $p \in Q$. Then $\varphi(p) \preceq_R p$ implies $p \preceq_R \varphi(p)$.*

Proof. Suppose $\varphi(p) \preceq_R p$. By Equation (3.32) we have $\uparrow\{p\} \subseteq \uparrow\{\varphi(p)\}$. By Equation (6.6) we have $\uparrow\{\varphi(p)\} = \varphi(\uparrow\{p\})$. Thus $\uparrow\{p\} \subseteq \varphi(\uparrow\{p\})$. By finiteness, this gives $\uparrow\{p\} = \varphi(\uparrow\{p\}) = \uparrow\{\varphi(p)\}$. Equation (3.33) we find $p \simeq_R \varphi(p)$. □

Corollary 6.33 *Either p and $\varphi(p)$ are incomparable or we have $p \preceq_R \varphi(p)$*

Corollary 6.34 *For a strongly connected component $T \subseteq Q$, the following are equivalent:*

1. $T \subseteq \varphi(\uparrow T)$,
2. $\varphi(T) = T$,
3. $\varphi(T) \cap T \neq \emptyset$,
4. $\varphi(\uparrow T) = \uparrow T$.

Proof. 1 ⇒ 2: Let $p \in T$. By assumption, we have $p = \varphi(q)$ for some $q \in \uparrow T$. Since T is a strongly connected component, we have $\uparrow T = \uparrow\{p\}$. Thus, there exists $u \in \Sigma^*$ such that $q = \delta(p, u)$. This gives $p = \varphi(\delta(p, u))$. By Theorem 6.3 we have $p \preceq_R \delta(p, u)$. Hence, $\varphi(\delta(p, u)) \preceq_R \delta(p, u)$. By Theorem 6.32 this gives $\varphi(\delta(p, u)) \simeq_R \delta(p, u)$. Thus $p \simeq_R \delta(p, u)$ and so $q \in T$. Hence $p \in \varphi(T)$. This gives $T \subseteq \varphi(T)$. Since $|\varphi(T)| \leq |T|$ by Equation (3.17), we find $T = \varphi(T)$.

2 ⇒ 3: This is clear.

3 ⇒ 4: Let $p \in \varphi(T) \cap T$. By Theorem 3.31 we have $\varphi(T) \subseteq T$. Thus $\varphi(p) \in T$. Since T is a strongly connected component, we have $\uparrow T = \uparrow\{p\} = \uparrow\{\varphi(p)\}$. So

$$\varphi(\uparrow T) = \varphi(\uparrow\{p\}) = \uparrow\{\varphi(p)\} = \uparrow T.$$

This gives the claim.[4]

4 $\Rightarrow$ 1: This is clear. □

Corollary 6.35 *For a strongly connected component $T \subseteq Q$, we have $\varphi(T) \cap T = \emptyset$ or $\varphi(T) = T$.*

Proof. This directly follows from Items 2 and 3 of the previous corollary. □

Remark 6.36 In general, for a sink component $T \subseteq Q$, a morphism φ need not induce an isomorphism between $\mathcal{A}_{|T}$ and $\mathcal{A}_{|\varphi(T)}$. For example, this fails when $|\varphi(T)| = 1$ and $|T| > 1$. However, by Theorem 6.35, if $\varphi(T) \cap T \neq \emptyset$, then $\varphi(T) = T$. Consequently, the restriction $\varphi_{|T}$ is an automorphism of $\mathcal{A}_{|T}$.

Corollary 6.37 *An endomorphism of a strongly connected finite automaton is an automorphism.*

Proof. By Theorem 6.35, it follows that every endomorphism is bijective, and so an automorphism by Theorem 6.17.

Alternatively, this also follows from Item 1 of Theorem 6.30. □

There exist automata that are not strongly connected and whose non-identity endomorphisms are all automorphisms. For example, a unary automaton in which the single letter induces two cycles of coprime lengths.

Corollary 6.38 *Let $S \subseteq Q$ be strongly connected. Then $\varphi(S) \subseteq S$ implies $\varphi(S) = S$.*

Proof. We have $S \subseteq T$ for some strongly connected component $T \subseteq Q$. Let $p \in S$. By assumption, we have $\varphi(p) \in T$. Thus $\varphi(p) \in T \cap \varphi(T)$. By Theorem 6.35 we find $\varphi(T) = T$. Hence φ permutes the states in T and so $\varphi_{|T}$ is injective. Then $\varphi(S) \subseteq S$ gives $\varphi(S) = S$. □

Corollary 6.39 *Let $\psi\colon Q \to Q$ be another endomorphism and $S \subseteq Q$ be strongly connected. If $\psi(\varphi(S)) \subseteq \varphi(S)$, then $\psi(\varphi(S)) = \varphi(S)$.*

Proof. By Item 3 of Theorem 6.25 the set $\varphi(S)$ is strongly connected. Then the claim follows from Theorem 6.38. □

Corollary 6.40 *For $\varphi(S) \subseteq S$ and $\delta(S, u) \subseteq T$ with strongly connected components $S, T \subseteq Q$ and $u \in \Sigma^*$ we have $\varphi(T) = T$.*

Proof. Let $q \in S$. Then $\varphi(q) \in S$. Since $\delta(S, u) \subseteq T$, we find $\delta(q, u) \in T$ and $\varphi(\delta(q, u)) = \delta(\varphi(q), u) \in T$. Thus $\varphi(\delta(q, u)) \in \varphi(T) \cap T$. By Theorem 6.34, this gives $\varphi(T) = T$. □

[4] Alternatively, since $\varphi(\uparrow T)$ is an upper set having non-empty intersection with T, we have $T \subseteq \varphi(\uparrow T)$. As $\uparrow T$ is the smallest upper set containing T, this gives $\uparrow T \subseteq \varphi(\uparrow T)$. By finiteness, we then obtain $\uparrow T = \varphi(\uparrow T)$.

By Theorem 6.34, for $\varphi(\uparrow\{p\}) = \uparrow\{p\}$, φ permutes the strongly connected component $[p]_{\simeq_R}$. However, this does not imply that φ permutes each strongly connected component in $\uparrow\{p\}$, as Theorem 6.41 shows next.

Example 6.41 Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ with $Q = \{q_1, q_2, q_3, q_4\}$ and

$$\delta(q_i, a) = \begin{cases} q_4 & \text{if } i = 1, \\ q_3 & \text{if } i = 2, \\ q_i & \text{otherwise,} \end{cases} \qquad \delta(q_i, b) = \begin{cases} q_1 & \text{if } i = 2, \\ q_2 & \text{if } i = 1, \\ q_i & \text{otherwise.} \end{cases}$$

The strongly connected components are $\{q_1, q_2\}$, $\{q_3\}$ and $\{q_4\}$. We have $\uparrow\{q_1\} = Q$. The map φ given by $\varphi(q_1) = q_2, \varphi(q_2) = q_1, \varphi(q_3) = q_4$ and $\varphi(q_4) = q_3$ is a morphism that maps $\{q_1, q_2\}$ to itself and interchanges $\{q_3\}$ and $\{q_4\}$.

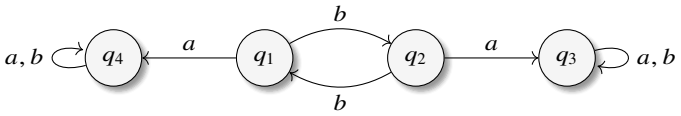


Fig. 6.4: Automaton admitting a morphism that leaves $\{q_1, q_2\}$ invariant and swaps $\{q_4\}$ and $\{q_3\}$, see Theorem 6.41.

Proposition 6.42 *For a strongly connected component $T \subseteq Q$, either we have $\varphi(p) = p$ for all $p \in T$ or $\varphi(p) \neq p$ for all $p \in T$.*

Proof. Suppose we have $\varphi(p) = p$ for some $p \in T$. Since $T \subseteq \uparrow\{p\}$, from Theorem 6.24 applied to φ and the identity transformation id_Q we get $\varphi(q) = q$ for all $q \in \uparrow\{p\}$, in particular φ fixes all states of T. □

Corollary 6.43 *The automorphism group of a strongly connected automaton is semiregular (and so has size at most $|Q|$).*

Proof. By Theorem 6.42, every non-identity isomorphism fixes no state. Thus, the automorphism group is semiregular. By Theorem 5.6, its size is at most $|Q|$. □

Proposition 6.44 *$\varphi\colon Q \to Q$ maps a synchronizing state to itself.*

Proof. Let $u \in \Sigma^*$ be such that $\delta(Q, u) = \{q\}$. Then we have $\varphi(q) = \varphi(\delta(q, u)) = \delta(\varphi(q), u) = q$. □

Corollary 6.45 *The only endomorphism of a strongly connected synchronizing automaton is the identity transformation.*

Proof. By Theorem 6.44 an endomorphism fixes the synchronizing state. Thus, by Theorem 6.42, it equals the identity transformation.[5] □

[5] Alternatively, since in this case every state is a synchronizing state as we show later in Theorem 7.14, from Theorem 6.44 we also get that the endomorphism fixes every state.

Corollary 6.46 *For a strongly connected synchronizing automaton having the same number of states as $\mathcal{A}$, there exists at most one morphism from $\mathcal{A}$ to that automaton.*

Proof. Let $\mathcal{B} = (S, \Sigma, \mu)$ be strongly connected and synchronizing with $|Q| = |S|$. Suppose $\psi, \xi \colon Q \to S$ are two morphisms. Since $\mathcal{B}$ is strongly connected, by Theorem 6.31, they are both isomorphisms. Let $\psi^{-1} \colon S \to Q$ be the inverse morphism of ψ. Then $\xi\psi^{-1}$ is an endomorphism of $\mathcal{B}$. By Theorem 6.45 it equals the identity transformation, which implies $\psi = \xi$. □

6.4.4 Cyclic Unary Automata

As a simple example, here we classify all morphisms between cyclic unary automata. First, Theorems 6.47 and 6.50 are preliminary statements, and Theorem 6.51 gives the classification.

Lemma 6.47 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ and $\mathcal{B} = (S, \{a\}, \mu)$ where $Q = \uparrow\{p\}$ for some $p \in Q$. Suppose $\varphi \colon Q \to S$ is a morphism. Let $i_2 \geq 0$ be the least number such that, with a corresponding least number $p_2 > 0$, we have $\mu(\delta(p), a^{i_2+p_2}) = \mu(\delta(p), a^{i_2})$. Then for all $i_1 \geq 0$ and $p_1 > 0$ with $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$ we have $i_2 \leq i_1$ and that p_1 is a multiple of p_2.*

Proof. First, $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$ implies $\mu(\varphi(p), a^{i_1+p_1}) = \varphi(\delta(p, a^{i_1+p_1})) = \varphi(\delta(p, a^{i_1})) = \mu(\varphi(p), a^{i_1})$. Hence, by the minimal choice of i_2 and p_2, we have $i_2 \leq i_1$ and $p_2 \leq p_1$. Next, write $p_1 = lp_2 + r$ for some $l \geq 0$ and $0 \leq r < p_2$. Then we have $\mu(\delta(p), a^{i_2+p_1}) = \mu(\delta(p), a^{i_2+r})$ and, by minimality, we must have $r = 0$, i.e., p_1 is a multiple of p_2. □

Example 6.48 Let $\mathcal{A} = (Q, \{a\}, \delta)$ consist of two disjoint cycles for a of lengths p and q for two distinct prime numbers. Then endomorphisms mapping some state from one cycle to the other cycle are not possible by Theorem 6.47. With Theorem 6.52, each endomorphism shifts each state on the cycle of length p and on the cycle of length q by some amount $0 \leq i < p$ and $0 \leq j < q$, respectively. So, $\mathcal{A}$ has pq endomorphisms.

Corollary 6.49 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ where $Q = \uparrow\{p\}$ for some $p \in Q$ and $i_1 \geq 0$ be the least number with a corresponding least number $p_1 > 0$ such that $\delta(p, a^{i_1+p_1}) = \delta(p, a^{i_1})$. Then $\delta(p, a^{i_2+p_2}) = \delta(p, a^{i_2})$ implies $i_1 \leq i_2$ and that p_2 is a multiple of p_1.*

Proof. Apply Theorem 6.47 with the identity transformation, which is a morphism. □

Proposition 6.50 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ with $Q = \uparrow\{q\}$ for some $q \in Q$ and $\mathcal{B} = (S, \{a\}, \mu)$. Then $\varphi \colon Q \to S$ is a morphism if and only if $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some $0 \leq i_2 \leq i_1$ with p_1 being a multiple of p_2 and $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for $i \in \{0, 1, \ldots, n-1\}$ with $n = |Q|$.*

Proof. First, let φ be a morphism. By finiteness, we have $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some minimal $i_2 \geq 0$ and some $p_2 > 0$. By Theorem 6.47, we find $i_2 \leq i_1$ and that

p_1 is a multiple of p_2. Since φ is a morphism, we have $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for all $i \geq 0$.

Next, suppose we have $\mu(\varphi(q), a^{i_2+p_2}) = \mu(\varphi(q), a^{i_2})$ for some $0 \leq i_2 \leq i_1$ with p_1 being a multiple of p_2 and $\varphi(\delta(q, a^i)) = \mu(\varphi(q), a^i)$ for $i \in \{0, 1, \ldots, n-1\}$. We have $n = i_1 + p_1$. By assumption, there exists $k \geq 1$ such that $p_1 = k \cdot p_2$. Since $i_2 \leq i_1$ we have $\mu(\varphi(q), a^{i_1}) = \mu(\varphi(q), a^{i_1+k\cdot p_2}) = \mu(\varphi(q), a^n)$. Then we find

$$\varphi(\delta(q, a^{n-1}), a) = \varphi(\delta(q, a^{i_1})) = \mu(\varphi(q), a^{i_1}) = \mu(\varphi(q), a^n) = \mu(\delta(q, a^{n-1}), a).$$

Since $Q = \{q, \delta(q, a), \ldots, \delta(q, a^{n-1})\}$, we have $\varphi(\delta(p, a)) = \delta(\varphi(p), a)$ for all $p \in Q$. □

Corollary 6.51 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ and $\mathcal{B} = (S, \{a\}, \mu)$ where $Q = \uparrow\{p\}$ and $S = \uparrow\{s\}$ for some $q \in Q$ and $s \in S$. Let $i_1, i_2 \geq 0$ be the least numbers with corresponding least numbers $p_1, p_2 > 0$ such that $\delta(q, a^{i_1+p_1}) = \delta(q, a^{i_1})$ and $\mu(s, a^{i_2+p_2}) = \mu(s, a^{i_2})$. Then there exist $\min\{i_1, i_2\} + p_2$ morphisms from $\mathcal{A}$ to $\mathcal{B}$ if and only if p_1 is a multiple of p_2, and there exists no morphism otherwise.*

Proof. If p_1 is not a multiple of p_2, there exists no morphism by Theorem 6.51. Next, suppose p_1 is a multiple of p_2. Then, for $i_2 - \min\{i_1, i_2\} \leq i < |S|$, setting $\varphi(\delta(q, a^j)) = \mu(s, a^{i+j})$, which gives $\varphi(\delta(q, a^j)) = \mu(\varphi(q), a^j)$, for $j \in \{0, 1, \ldots, |Q| - 1\}$ yields a morphism by Theorem 6.51 where $\mu(\varphi(q), a^{\min\{i_2-i,0\}+p_2}) = \mu(\varphi(q), a^{\min\{i_2-i,0\}})$ and $\min\{i_2 - i, 0\} \leq \min\{i_1, i_2\} \leq i_1$.

Next, let $\varphi\colon Q \to S$ be a morphism and $0 \leq i < |S|$ such that $\varphi(q) = \mu(s, a^i)$. We have $\mu(s, a^{i+j}) = \mu(s, a^{i+j+p_2})$ for $j = \min\{i_2 - i, 0\}$. By Theorem 6.51 we find $j \leq i_1$. Thus, for $i_2 > i_1$ we must have $i_2 - i_1 \leq i$ since otherwise $i + j < i_2$. This gives $i_2 - \min\{i_1, i_2\} \leq i < |S|$.

So, each morphism is uniquely given by its image $\varphi(q) = \mu(s, a^i)$ for $i_2 - \min\{i_1, i_2\} \leq i < |S|$. Thus for $i_2 < i_1$ we have $|S| = i_2 + p_2 = \min\{i_1, i_2\} + p_2$ possible morphisms. For $i_2 \geq i_1$ we have $|\{i \mid i_2 - i_1 \leq i < i_2 + p_2\}| = i_1 + p_2 = \min\{i_1, i_2\} + p_2$ possible morphisms. □

Corollary 6.52 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ with $Q = \uparrow\{q\}$ for some $q \in Q$. Then $\varphi\colon Q \to Q$ is an endomorphism if and only if there exists $0 \leq m < |Q|$ such that $\varphi(p) = \delta(p, a^m)$ for all $p \in Q$.*

Proof. Let $m \geq 0$ such that $\varphi(q) = \delta(q, a^m)$. By Theorem 6.51, φ is a morphism if and only if $\varphi(\delta(q, a^i)) = \delta(\varphi(q), a^i)$ for $0 \leq i < |Q|$. This is equivalent to $\varphi(p) = \delta(p, a^m)$ for all $p \in Q$ since $\delta(\varphi(q), a^i) = \delta(\delta(q, a^i), a^m)$. □

Since in unary automata, strongly connected components are cycles, by Theorem 6.47, the size of a strongly connected component in the image of a morphism divides the size of a strongly connected component that maps to it. This might fail for non-unary automata, as shown next.

Example 6.53 Let $\mathcal{A} = (\{q_0, q_1, q_2\}, \{a, b\}, \delta)$ and $\mathcal{B} = (\{p_0, p_1\}, \{a, b\}, \mu)$ with $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_2$, $\delta(q_1, b) = q_0$, $\delta(q_2, b) = q_0$, $\mu(p_0, a) = p_1$, $\delta(p_1, b) = p_0$ and self-loops for all the remaining transitions, see Figure 6.5. The map φ with

$\varphi(q_0) = p_0$, $\varphi(q_1) = p_1$, $\varphi(q_2) = p_1$ is a morphism since $\mu(\varphi(q), b) = p_0 = \varphi(q_0) = \varphi(\delta(q, a))$ for each $q \in \{q_0, q_1, q_2\}$ and

$$\begin{aligned}
\mu(\varphi(q_0), a) &= \mu(p_0, a) = p_1 = \varphi(q_1) = \varphi(\delta(q_0, a)),\\
\mu(\varphi(q_1), a) &= \mu(p_1, a) = p_1 = \varphi(q_2) = \varphi(\delta(q_1, a)),\\
\mu(\varphi(q_2), a) &= \mu(p_1, a) = p_1 = \varphi(q_2) = \varphi(\delta(q_2, a)).
\end{aligned}$$

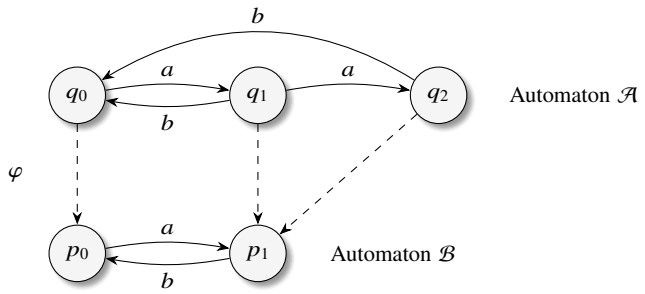


Fig. 6.5: Morphism φ between two automata $\mathcal{A}$ and $\mathcal{B}$, see Theorem 6.53.

6.5 Operations on Automata

6.5.1 Subautomata and Restricted Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\Gamma \subseteq \Sigma$.

1. The *restriction of $\mathcal{A}$ to* Γ is $\mathcal{A}_{|\Gamma} = (Q, \Gamma, \delta_{|\Gamma})$ where $\delta_{|\Gamma} \colon Q \times \Gamma \to Q$ is defined by $\delta_{|\Gamma}(q, x) = \delta(q, x)$ for all $q \in Q$ and all $x \in \Gamma$.
2. Let $S \subseteq Q$ be an upper set in $\mathcal{A}$. The *restriction of $\mathcal{A}$ to the upper set S* is the deterministic and complete semiautomaton $\mathcal{A}_{|S} = (S, \Sigma, \delta_{|S})$ with $\delta_{|S} \colon S \times \Sigma \to S$ given by $\delta_{|S}(q, x) = \delta(q, x)$ for $q \in S$ and $x \in \Sigma$. We also call $\mathcal{A}_{|S}$ the *sub(semi-)automaton* corresponding to (or induced/defined by) the upper set $S \subseteq Q$. We often just denote $\delta_{|S}$ by δ in this context.
3. If $S \subseteq Q$ is an upper set in $\mathcal{A}_{|\Gamma}$, then by $\mathcal{A}_{|\Gamma,S} = (S, \Gamma, \delta_{|\Gamma,S})$ we denote the *restriction of $\mathcal{A}$ to the the alphabet Γ and the subset S*, i.e., $(\mathcal{A}_{|\Gamma})_{|S}$.

Upper sets can be identified with subautomata. Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be upper sets. The following facts are readily seen. We have

$$(\mathcal{A}_{|S})_{|T} \cong \mathcal{A}_{|T} \text{ for } T \subseteq S \tag{6.7}$$

Clearly, $\mathcal{A}_{|S}$ is strongly connected if and only if S is a sink component.

Proposition 6.54 *The extension of the inclusion map $i\colon S \to Q$ given by $i(q) = q$ for $q \in S$ to subsets induces a bijection between the strongly connected components (resp. upper sets) in $\mathcal{A}_{|S}$ and the strongly connected components (resp. upper sets) R in $\mathcal{A}$ with $R \subseteq S$.*

6.5.2 The Quotient Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. Suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence. Equivalently, for every $p, q \in Q$ and $x \in \Sigma$, we have

$$p \sim q \Rightarrow \delta(p, x) \sim \delta(q, x). \tag{6.8}$$

The *quotient automaton* is $\mathcal{A}/\!\sim = (Q/\!\sim, \Sigma, \delta_\sim)$ with the well-defined transition function $\delta_\sim([q]_\sim, x) = [\delta(q, x)]_\sim$ for all $x \in \Sigma$ and $q \in Q$.

The next two statements connect $\mathcal{T}_S(\mathcal{A})$-congruences, quotient automata, the kernel relation, and morphisms in the following way: $\mathcal{T}_S(\mathcal{A})$-congruences correspond to surjective morphisms onto quotient automata via the canonical morphism (Theorem 6.55), and surjective morphisms correspond to $\mathcal{T}_S(\mathcal{A})$-congruences via the kernel relation (Theorem 6.56).

In view of Theorem 6.55, in this context we also call $\pi_\sim$ the *canonical morphism (for $\sim$)* from $\mathcal{A}$ onto $\mathcal{A}/\!\sim$.

Theorem 6.55 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on $\mathcal{A}$. Then the canonical projection map $\pi_\sim\colon Q \to Q/\!\sim$ from Equation* (3.28) *is a surjective morphism whose kernel relation equals $\sim$.*

Proof. Clearly, $\pi_\sim$ is surjective and its kernel relation equals $\sim$. It is a morphism since by definition of $\mathcal{A}/\!\sim$ we have $\delta_\sim(\pi_\sim(q), x) = \pi_\sim(\delta(q, x))$ for $q \in Q$ and $x \in \Sigma$. □

Theorem 6.56 *Let $\varphi\colon Q \to S$ be a morphism between the automata $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Then the following is true:*

1. *$\varphi(Q)$ is an upper set in $\mathcal{B}$,*
2. *the kernel relation $\sim_\varphi$ is a $\mathcal{T}_S(\mathcal{A})$-congruence,*
3. *$[q]_{\sim_\varphi} \mapsto \varphi(q)$, $q \in Q$, is a well-defined isomorphism between $\mathcal{A}/\!\sim_\varphi$ and $\mathcal{B}_{|\varphi(Q)}$.*

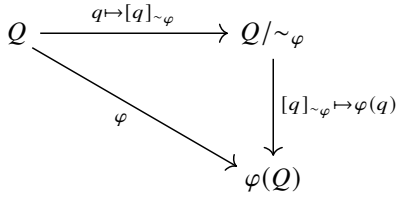
Proof. 1: This follows from Theorem 6.25, Item 2.

2: Suppose $p \sim_\varphi q$ for $p, q \in Q$. Then we have $\varphi(p) = \varphi(q)$. Since φ is a morphism, we find $\varphi(\delta(p, x)) = \mu(\varphi(p), x) = \mu(\varphi(q), x) = \varphi(\delta(q, x))$, and so $\delta(p, x) \sim_\varphi \delta(q, x)$, i.e., $\sim_\varphi$ is a $\mathcal{T}_S(\mathcal{A})$-congruence.

3: Well-definedness follows from the definition of the kernel relation. Now $\delta_{\sim_\varphi}([q]_{\sim_\varphi}, x) = [\delta(q, x)]_{\sim_\varphi}$ is mapped to $\varphi(\delta(q, x))$ and $[q]_{\sim_\varphi}$ is mapped to $\varphi(q)$. Since $\mu(\varphi(q), x) = \varphi(\delta(q, x))$, we obtain that the mapping is a morphism as we arrive at the same result when applying first a letter and then the mapping, or when first applying the mapping and then a letter. The mapping clearly maps onto $\mathcal{B}_{|\varphi(Q)}$. □

A similar statement as Theorem 6.56 in the theory of groups is often called the homomorphism theorem, for example, see [393, Sect. 1.2.5]. Theorem 6.56 states that

morphisms composed along paths in the following diagram ending at the same point are equal.



Corollary 6.57 *Let $\varphi\colon Q \to S$ be a morphism between the automata $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ with $Q \neq \emptyset$. If $\mathcal{B}$ is strongly connected, then $\varphi(Q) = S$, i.e., φ is surjective. So $\mathcal{A}/{\sim_\varphi} \cong \mathcal{B}$. In particular, for $\mathcal{A} = \mathcal{B}$, by finiteness, φ is an isomorphism.*

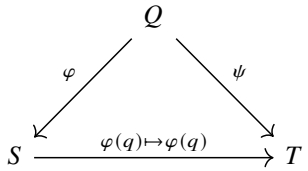
Corollary 6.58 *Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. Then every morphism from $\mathcal{A}$ into another automaton is either injective or constant.*

Proof. For a morphism into another automaton, its kernel relation is a $\mathcal{T}_S(\mathcal{A})$-congruence by Theorem 6.56, Item 2. So it either equals the universal relation, and so the morphism is a constant, or the equality relation, and so the morphism is injective. □

Next, we show a condition to obtain a morphism from two morphisms from the same automaton. The proof is straightforward and omitted.

Theorem 6.59 *Let $\varphi\colon Q \to S$ and $\psi\colon Q \to T$ be morphisms from $\mathcal{A} = (Q, \Sigma, \delta)$ to $\mathcal{B} = (S, \Sigma, \mu)$ and $\mathcal{C} = (T, \Sigma, \xi)$, respectively. Suppose ${\sim_\varphi}$ refines ${\sim_\psi}$. Then $\varphi(q) \mapsto \psi(q)$ is a well-defined surjective morphisms from $\mathcal{B}_{|\varphi(Q)}$ onto $\mathcal{C}_{|\psi(Q)}$.*

By Theorem 6.56, Item 3, $\mathcal{A}/{\sim_\varphi} \cong \mathcal{B}_{|\varphi(Q)}$ and $\mathcal{A}/{\sim_\psi} \cong \mathcal{C}_{|\psi(Q)}$ in Theorem 6.59. In the same way as for Theorem 6.56, the situation from Theorem 6.59 is illustrated in the following diagram.



Corollary 6.60 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$ with ${\sim_\mathcal{A}}$ and ${\sim_\mathcal{B}}$ being $\mathcal{T}_S(\mathcal{A})$- and $\mathcal{T}_S(\mathcal{B})$-congruences, respectively. If ${\sim_\mathcal{A}}$ refines ${\sim_\mathcal{B}}$, then $[q]_{\sim_\mathcal{A}} \mapsto [q]_{\sim_\mathcal{B}}$ is a morphism from $\mathcal{A}/{\sim_\mathcal{A}}$ into $\mathcal{B}/{\sim_\mathcal{B}}$.*

6.5.3 The Rees Quotient Automaton

Here, we introduce a special kind of quotient automata whose construction resembles a similar one from the theory of semigroups introduced by Rees [505].

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. The *Rees congruence* is the $\mathcal{T}_S(\mathcal{A})$-congruence on Q given, for $p, q \in Q$, by

$$p \sim_S q \Leftrightarrow p, q \in S \vee (p = q).$$

We call $\mathcal{A}/\sim_S$ the *Rees quotient (semi-)automaton* w.r.t. the upper set $S \subseteq Q$. For $S \neq \emptyset$, intuitively, $\mathcal{A}/\sim_S$ results from $\mathcal{A}$ by collapsing the states in S to a single trap state in $\mathcal{A}/\sim_S$.

Remark 6.61 For $S \neq \emptyset$, unraveling the definition, $\mathcal{A}/\sim_S$ has the state set $\{\{p\} \mid p \in Q \setminus S\} \cup \{S\}$, the transition function fulfills $\delta_{\sim_S}(S, x) = S$ and, for all $p \notin S$ and all $x \in \Sigma$,

$$\delta_{\sim_S}(\{p\}, x) = \begin{cases} \{\delta(p, x)\} & \text{if } \delta(p, x) \notin S, \\ S & \text{otherwise.} \end{cases} \tag{6.9}$$

So, $\mathcal{A}/\sim_S$ has $|Q| - |S| + 1$ states. By Theorem 3.37, as a strongly connected component is either contained in S or disjoint from S, for $p, q \in Q$ and $x \in \Sigma$ with $p \simeq_R q$ we have

$$p = \delta(q, x) \Leftrightarrow \begin{cases} p = \delta_{|S}(q, x) & \text{if } p \in S, \\ \pi_{\sim_S}(p) = \delta_{\sim_S}(\pi_{\sim_S}(q), x) & \text{if } p \notin S. \end{cases} \tag{6.10}$$

For $T \subseteq Q$, we have

$$\pi_{\sim_S}(T) = \begin{cases} \{S\} \cup \{\{q\} \mid q \in T \setminus S\} & \text{if } S \cap T \neq \emptyset, \\ \{\{q\} \mid q \in T\} & \text{if } S \cap T = \emptyset. \end{cases} \tag{6.11}$$

As stated next, the projection maps induce bijections between certain strongly connected components and upper sets in $\mathcal{A}$ and $\mathcal{A}/\sim_S$

Proposition 6.62 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and let $S \subseteq Q$ be an upper set.*

1. *Let P be a strongly connected component of $\mathcal{A}/\sim_S$, distinct from $\{S\}$. Then $\pi_{\sim_S}^{-1}(P)$ is a strongly connected component of $\mathcal{A}$ contained in $Q \setminus S$, and*

   $$\pi_{\sim_S}\left(\pi_{\sim_S}^{-1}(P)\right) = P.$$

   Conversely, let $R \subseteq Q \setminus S$ be a strongly connected component of $\mathcal{A}$. Then $\pi_{\sim_S}(R)$ is a strongly connected component of $\mathcal{A}/\sim_S$ distinct from $\{S\}$, and

   $$\pi_{\sim_S}^{-1}\left(\pi_{\sim_S}(R)\right) = R.$$

2. *Let P be an upper set of $\mathcal{A}/\sim_S$ containing S. Then $\pi_{\sim_S}^{-1}(P)$ is an upper set of $\mathcal{A}$ containing S, and*

   $$\pi_{\sim_S}\left(\pi_{\sim_S}^{-1}(P)\right) = P.$$

   Conversely, let R be an upper set of $\mathcal{A}$ containing S. Then $\pi_{\sim_S}(R)$ is an upper set of $\mathcal{A}/\sim_S$ containing S, and

   $$\pi_{\sim_S}^{-1}\left(\pi_{\sim_S}(R)\right) = R.$$

Proof. This follows from Theorem 6.61, in particular Equation (6.9), which yield that for $p, q \in Q \setminus S$ and $u \in \Sigma^*$ we have $\delta(p, u) = q$ if and only if $\delta_{\sim_S}(\pi_{\sim_S}(p), u) = \pi_{\sim_S}(q)$. □

Corollary 6.63 *The number of strongly connected components in $\mathcal{A}/{\sim_S}$ is precisely the number of strongly connected components not in S plus one accounting for $\{S\}$.*

Corollary 6.64 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be a proper upper set. Then $Q \setminus S$ is a strongly connected component in $\mathcal{A}$ if and only if $(Q/{\sim_S}) \setminus \{S\}$ is a strongly connected component in $\mathcal{A}/{\sim_S}$.*

Corollary 6.65 *For $S \neq \emptyset$ and $S \neq Q$, $\mathcal{A}/{\sim_S}$ is strongly trap-connected if and only if $Q \setminus S$ is strongly connected and $\delta(Q \setminus S, x) \cap S \neq \emptyset$ for some $x \in \Sigma$.*

Proof. By Theorem 6.62, $Q \setminus S$ is strongly connected if and only if $\pi_{\sim_S}(Q \setminus S)$ and $\pi_{\sim_S}(S)$ are the only strongly connected components in $\mathcal{A}/{\sim_S}$. Since $\pi_{\sim_S}(Q \setminus S)$ is not a sink component, the claim follows from Item 2 of Theorem 6.10. □

Next, we state isomorphisms between Rees quotient automata.

Proposition 6.66 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be upper sets. Then we have:*

$$\mathcal{A} \cong \mathcal{A}/{\sim_S} \qquad \text{for } |S| \leq 1 \tag{6.12}$$

$$(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)} \tag{6.13}$$

$$(\mathcal{A}_{|S\cup T})/{\sim_T} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)} \qquad \text{for } S \cap T \neq \emptyset \tag{6.14}$$

$$(\mathcal{A}/{\sim_S})/{\sim_{\pi_{\sim_S}(T)}} \cong \mathcal{A}/{\sim_{S\cup T}} \tag{6.15}$$

$$(\mathcal{A}/{\sim_S})/{\sim_{\pi_{\sim_S}(T)}} \cong (\mathcal{A}/{\sim_T})/{\sim_{\pi_{\sim_T}(S)}} \tag{6.16}$$

Proof. Equation (6.12): For $|S| \leq 1$ the Rees congruence $\sim_S$ equals the equality relation and thus $\mathcal{A} \cong \mathcal{A}/{\sim_S}$.
Equation (6.13): The map $(\pi_{\sim_T})_{|S} \colon S \to Q/{\sim_T}$ is a morphism from $\mathcal{A}_{|S}$ to $\mathcal{A}/{\sim_T}$ with kernel $\sim_{S\cap T}$. Thus the stated isomorphism follows from Theorem 6.56.[6]
Equation (6.14): Set $S' = S \cup T$. Then S' is an upper set and, by Equation (6.13), we find

$$(\mathcal{A}_{|S'})/{\sim_{S'\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S')}.$$

Then the claim follows since $S' \cap T = T$ and for $S \cap T \neq \emptyset$ we have $\pi_{\sim_T}(S) = \pi_{\sim_T}(S')$.
Equation (6.15): The map $\pi_{\sim_R}\pi_{\sim_S} \colon Q \to (Q/{\sim_S})/{\sim_R}$ with $R = \pi_{\sim_S}(T)$ is a morphism with kernel $\sim_{S\cup T}$. Thus the stated isomorphism follows from Theorem 6.56.[7]

[6] Alternatively, by Theorem 6.59, the map $\pi_{\sim_{S\cap T}}(q) \mapsto \pi_{\sim_T}(q)$ for $q \in S$ is a well-defined surjective morphism from $(\mathcal{A}_{|S})/{\sim_{S\cap T}}$ onto $(\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)}$. Since

$$\pi_{\sim_{S\cap T}}(p) = \pi_{\sim_{S\cap T}}(q) \iff \pi_{\sim_T}(p) = \pi_{\sim_T}(q)$$

for all $p, q \in S$, this morphism is an isomorphism. Thus we have $(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_T})_{|\pi_{\sim_T}(S)}$.

[7] Alternatively, since $\sim_S$ refines $\sim_{S\cup T}$, the map $\pi_{\sim_S}(p) \mapsto \pi_{\sim_{S\cup T}}(p)$ for $q \in Q$ is a well-defined surjective morphism by Theorem 6.59. Since it has kernel $\pi_{\sim_S}(T)$ the stated isomorphism follows from Theorem 6.56.

Equation (6.16): By Equation (6.15) both automata involving two Rees quotients are isomorphic to $\mathcal{A}/{\sim_{S\cup T}}$. Thus they are isomorphic to each other. □

Note that Equation (6.13) also implies $(\mathcal{A}_{|S})/{\sim_{S\cap T}} \cong (\mathcal{A}/{\sim_{S\cap T}})_{|\pi_{\sim_{S\cap T}}(S)}$ by applying it to the upper sets S and $S \cap T$ instead of S and T.

For a morphism and an upper set, we obtain a morphism between corresponding Rees quotient automata.

Proposition 6.67 *Let $\varphi\colon Q \to S$ be a morphism from $\mathcal{A} = (Q, \Sigma, \delta)$ to $\mathcal{B} = (S, \Sigma, \mu)$ and $T \subseteq Q$ be an upper set. Then $\varphi(T)$ is an upper set and*

$$[q]_{\sim_T} \mapsto [\varphi(q)]_{\sim_{\varphi(T)}}$$

for $q \in Q$ is a well-defined surjective morphism from $\mathcal{A}/{\sim_T}$ onto $(\mathcal{B}/{\sim_{\varphi(T)}})_{|\pi_{\sim_{\varphi(T)}}(\varphi(Q))}$.

Proof. For $p, q \in Q$, we have

$$\begin{aligned} p \sim_T q &\Rightarrow p = q \vee \{p, q\} \subseteq T \\ &\Rightarrow \varphi(p) = \varphi(q) \vee \{\varphi(p), \varphi(q)\} \subseteq \varphi(T) \\ &\Rightarrow \varphi(p) \sim_{\varphi(T)} \varphi(q). \end{aligned}$$

Hence, Theorem 6.59 gives the claim. In fact, we have $\varphi([q]_{\sim_T}) = [\varphi(q)]_{\sim_{\varphi(T)}}$ and so φ induces this morphism on the equivalence classes of $\sim_T$. □

Lastly, using the Rees quotient automaton construction we can derive a necessary condition, originally due to Thierrin [599, Prop. 8], for primitivity of automata (see also [51, Cor. 3] or, in a more algebraic context, [581, Prop. 5.1]).

Proposition 6.68 (Thierrin [599, Prop. 8]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a primivite automaton with $|Q| \geq 3$. Then $\mathcal{A}$ is strongly connected or strongly trap-connected with trap state $t \in Q$ such that $\delta(p, a) = t$ and $\delta(q, a) \neq t$ for some $p, q \in Q$ and $a \in \Sigma$.*

Proof. There exists no proper upper set $S \subseteq Q$ of size at least two, as otherwise $\sim_S$ would be a non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. Hence, by Theorem 6.11, $\mathcal{A}$ is either strongly connected or strongly trap-connected with a trap state $t \in Q$. In the latter case, since $Q \setminus \{t\}$ and $\{t\}$ form a partition that does not induce a $\mathcal{T}_S(\mathcal{A})$-congruence, there exist $p, q \in Q$ and $a \in \Sigma$ such that $\delta(p, a) = t$ and $\delta(q, a) \neq t$. □

The following corollary has been noted in [525, Sect. 5.1] and [610, Prop. 3.1].

Corollary 6.69 *A primitive automaton $\mathcal{A}$ with at least three states is either strongly connected or synchronizing.*

Proof. This follows from Theorem 6.68 since a strongly trap-connected automaton is synchronizing. □

Remark 6.70 Theorem 6.68 generalizes Theorem 5.1 (via the correspondence from Section 6.8), since a trap state in a permutation automaton is not reachable from any other state. Thus, by Theorem 6.68, a permutation automata whose transformation monoid is a primitive group G is strongly connected and so G is transitive.

6.5.4 The Product Automaton

Given $\mathcal{A}_i = (Q_i, \Sigma, \delta_i)$ for $i \in \{1, \ldots, m\}$, their *product (semi-)automaton* [322, 502] is $\mathcal{A}_1 \times \ldots \times \mathcal{A}_m = (Q_1 \times \ldots \times Q_m, \Sigma, \delta_{\mathcal{A}_1 \times \ldots \times \mathcal{A}_m})$ with

$$\delta((q_1, \ldots, q_m), x) = (\delta_1(q_1, x), \ldots, \delta_m(q_m, x))$$

for $q_1 \in Q_1, \ldots, q_m \in Q_m$ and $x \in \Sigma$. If $\mathcal{A}_1 = \cdots = \mathcal{A}_m$, then we also write $\mathcal{A}_1^m$ for $\mathcal{A}_1 \times \cdots \times \mathcal{A}_m$. For the set of synchronizing words we have

$$\mathrm{Syn}(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m) = \bigcap_{i=1}^{m} \mathrm{Syn}(\mathcal{A}_i). \tag{6.17}$$

For upper sets $S_1 \subseteq Q_1, \ldots, S_m \subseteq Q_m$, the set $S = S_1 \times \cdots \times S_m$ is an upper set and since $(q_1, \ldots, q_m) \in S$ if and only if $q_1 \in S_1, \ldots, q_m \in S_m$ we have

$$(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{|S} \cong (\mathcal{A}_1)_{|S_1} \times \cdots \times (\mathcal{A}_m)_{|S_m} \tag{6.18}$$

$$(\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)/{\sim_S} \cong (\mathcal{A}_1/{\sim_{S_1}}) \times \cdots \times (\mathcal{A}_m/{\sim_{S_m}}) \tag{6.19}$$

6.5.5 The Power Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$. The *power (semi-)automaton* is $\mathcal{P}(\mathcal{A}) = (\mathcal{P}(Q), \Sigma, \delta)$ where $\delta\colon \mathcal{P}(Q) \times \Sigma \to \Sigma$ is the extension of $\delta\colon Q \times \Sigma \to Q$ to subsets $\delta(S, x) = \{\delta(q, x) \mid q \in Q\}$ for $S \subseteq Q$ and $x \in \Sigma$ (see also Section 1.1). Figure 6.6 shows the power automaton of a four-state automaton. Also, part of the power automaton of the Černý automaton C_4 is depicted in **??**.

In this section, we present properties of powers of a single word in the power automaton. These can also be interpreted as properties of transformations on subsets of a finite set; however, since our focus is on automata, we express them in automata-theoretic terms.

Proposition 6.71 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $n = |Q|$ and $u \in \Sigma^*$. Then for $i \geq 0$ we have $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ and for $P = \delta(Q, u^{n-1})$ the following is true:*

1. *$P = \delta(Q, u^{n-|P|})$,*
2. *$\delta(P, u) = P$ (u permutes P) where P is the largest set with this property, and $P = \delta(Q, u^i)$ for all $i \geq n - |P|$,*
3. *$P \subseteq \delta(Q, u^i)$ for all $i \geq 0$.*

Proof. For the first claim, since $\delta(Q, u) \subseteq Q$ and $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ implies

$$\delta(\delta(Q, u^{i+1}), u) \subseteq \delta(\delta(Q, u^i), u),$$

by induction we find that $\delta(Q, u^{i+1}) \subseteq \delta(Q, u^i)$ for all $i \geq 0$.

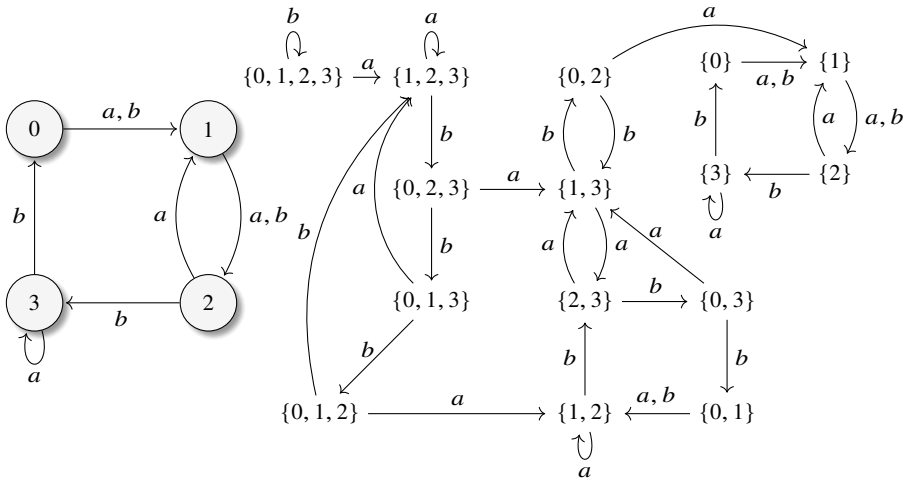


Fig. 6.6: Left: An automaton. Right: Its power automaton (empty set omitted).

Next, by finiteness, there exists a minimal $j \geq 0$ such that $\delta(Q, u^{j+1}) = \delta(Q, u^j)$. Then we have $\delta(Q, u^k) = \delta(Q, u^j)$ for all $k \geq j$ and we get the following chain of proper inclusions

$$\delta(Q, u^j) \subsetneq \delta(Q, u^{j-1}) \subsetneq \cdots \subsetneq \delta(Q, u) \subsetneq Q. \tag{6.20}$$

This yields $\delta(Q, u^j) \subseteq \delta(Q, u^i)$ for every $i \geq 0$. Since $|\delta(Q, w)| \geq 1$ for all $w \in \Sigma^*$, the above chain of proper inclusions has length at most $n - |\delta(Q, u^j)|$, i.e., $j \leq n - |\delta(Q, u^j)|$. Hence we find

$$P = \delta(Q, u^{n-1}) = \delta(Q, u^{n-|\delta(Q,u^j)|}) = \delta(Q, u^j) = \delta(Q, u^n) = \delta(P, u),$$

and so $P = \delta(Q, u^{n-|P|})$, $P \subseteq \delta(Q, u^i)$ for all $i \geq 0$, and P is permuted by u. This also gives $P = \delta(Q, u^i)$ for $i \geq n - |P|$ by induction. Furthermore, if $\delta(S, u) = S$ for $S \subseteq Q$, then $S = \delta(S, u^i) \subseteq \delta(Q, u^i)$ for all $i \geq 0$. In particular, we obtain $S \subseteq \delta(Q, u^{n-1}) = P$ and so P is the largest set permuted by u. This shows all claims. □

The set P from Theorem 6.71 is precisely the set of cycle states described in Theorem 3.69. By finiteness, after applying a word at most $n - 1$ times to any state we end up at a cycle state.

Note that since $\delta(Q, u) \subseteq Q$ for $u \in \Sigma^*$, we have $\delta(Q, uv) \subseteq \delta(Q, v)$ for each $v \in \Sigma^*$.

Corollary 6.72 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ with $n = |Q|$ and $u \in \Sigma^*$. Then we have $\delta(Q, u^i) = \delta(Q, u^{\min\{n-1,i\}})$ for $i \geq 0$.*

Proof. By Theorem 6.71, the word u permutes the set $\delta(Q, u^{n-1})$. Thus, for $i \geq n - 1$ we find $\delta(Q, u^i) = \delta(Q, u^{n-1})$, which gives the claim. □

In [501, Lem. 1 & 2] it was also proved that P is the largest set permuted by u in Theorem 6.71, together with the fact that we have to apply u at most $n - |P|$ times to Q to obtain P.

Note that for the properties collected in Theorem 6.71 to hold it is essential that we start from the whole state set before applying powers of u.

Proposition 6.73 *Let $\mathcal{A} = (Q, \Sigma, \delta)$, $n = |Q|$, $u \in \Sigma^*$, $S \subseteq Q$, and $i \geq 0$.*

1. *$\delta(S, u^i) = \delta(S, u^j)$ for some $0 \leq j < 2^n - 1$.*
2. *$|\delta(S, u^i)| = |\delta(S, u^j)|$ for some $0 \leq j \leq n - |\delta(S, u^i)|$.*
3. *$|\delta(S, u^i)| = |\delta(S, u^{\min\{n-1,i\}})|$.*

Proof. 1: We can assume $S \neq \emptyset$. Let $j \geq 0$ be a minimal number such that $\delta(S, u^i) = \delta(S, u^j)$. Assume $j \geq 2^n - 1$. We have $\delta(S, u^k) \neq \emptyset$ for all $k \geq 0$. Since there are at most $2^n - 1$ non-empty subsets, and the sequence $\delta(S, u^0), \delta(S, u), \ldots, \delta(S, u^{2^n-1})$ has length 2^n, there exist $0 \leq k < k' \leq 2^n - 1$ such that $\delta(S, u^k) = \delta(S, u^{k'})$. Then $\delta(S, u^{j-(k'-k)}) = \delta(S, u^j)$, contradicting the minimal choice of j. So, we have $j < 2^n - 1$.

2: We can assume $i > n - |\delta(S, u^i)|$. For the minimal $j \geq 0$ with $\delta(Q, u^{j+1}) = \delta(Q, u^j)$, we have a proper chain of inclusions as in Equation (6.20). Thus, for $k \in \{0, 1, \ldots, j\}$, we get $|\delta(Q, u^k)| \leq n-k$. Since $\delta(S, u^j) \subseteq \delta(Q, u^j)$, we find $|\delta(S, u^j)| \leq n - j$, and so $j \leq n - |\delta(S, u^j)|$. Set $P = \delta(Q, u^{n-1})$. With Theorem 6.71 we get $\delta(Q, u^j) = P$ and that u permutes P, and so u acts injectively on subsets of P. Thus, from $\delta(S, u^j) \subseteq P$ we obtain $|\delta(S, u^k)| = |\delta(S, u^j)|$ for all $k \geq j$. In particular, since $i > j$, we find $|\delta(S, u^i)| = |\delta(S, u^j)|$.

3: We can assume $i \geq n$. By Item 2 we have $|\delta(S, u^i)| = |\delta(S, u^j)|$ for some $0 \leq j < n$. With Equation (3.17) we find

1. $|\delta(S, u^{n-1})| = |\delta(\delta(S, u^j), u^{n-1-j})| \leq |\delta(S, u^j)|$ and
2. $|\delta(S, u^i)| = |\delta(\delta(S, u^{n-1}), u^{i-n+1})| \leq |\delta(S, u^{n-1})|$.

Thus we obtain $|\delta(S, u^i)| = |\delta(S, u^{n-1})|$. □

Corollary 6.74 *Let $\mathcal{A} = (Q, \Sigma, \delta)$, $u \in \Sigma^*$, and $S \subseteq Q$. Then $|\delta(S, u^i)| = 1$ for some $i \geq 0$ if and only if $|\delta(S, u^{n-1})| = 1$ for $n = |Q|$.*

Proof. First, assume $|\delta(S, u^i)| = 1$ for some $i \geq 0$. Since $|\delta(S, u^i)| = |\delta(S, u^{\min\{n-1,i\}})|$ by Theorem 6.73, Item 3 and $|\delta(S, u^{n-1})| \leq |\delta(S, u^{\min\{n-1,i\}})|$, we find $|\delta(S, u^{n-1})| = 1$. The converse implication is obvious. □

Remark 6.75 To obtain Theorem 6.73, Item 1, we have basically used the fact that in any n-state automaton we enter a cycle when we read a word more than $n-1$ times. Now, as u permutes P in Theorem 6.71, the smallest $i \geq 0$ such that $\delta(q, u^i) = q$ for all $q \in P$ is asymptotically equal to $\exp((1 + o(1))\sqrt{(|P| \ln |P|)}$ by a result from Landau [403]. This can be used to improve the bound from Theorem 6.73 asymptotically. However, as the bound given is sufficient for our applications, we do not bother to state a more precise bound.

6.5.6 The Pair Automaton

Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S = \{\{p, q\} \mid p, q \in Q\}$. Then S is an upper set in $\mathcal{P}(\mathcal{A})$. We denote $\mathcal{P}(\mathcal{A})_{|S}$ also by $\mathcal{A}^{\leq 2} = (S, \Sigma, \delta^{\leq 2})$. We have $\delta^{\leq 2}(\{p, q\}, x) = \delta(\{p, q\}, x)$ for $x \in \Sigma$. The set $F = \{\{q\} \mid q \in Q\}$ is an upper set in $\mathcal{A}^{\leq 2}$ and the Rees quotient automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ is called the *pair (semi-)automaton*. The state set of the pair automaton is $\{\{\{p, q\}\} \mid p \neq q \text{ and } p, q \in Q\} \cup \{F\}$. For ease of notation, for distinct $p, q \in Q$ we identify a state $\{\{p, q\}\}$ with $\{p, q\}$. The pair automaton has $\binom{n}{2} + 1$ states.

The map $\varphi \colon Q \times Q \to \{\{p, q\} \mid p, q \in Q\}$ given by $\varphi(p, q) = \{p, q\}$ is a surjective morphism of $\mathcal{A} \times \mathcal{A}$ onto $\mathcal{A}^{\leq 2}$. Thus, by Theorem 6.55, $[(p, q)]_{\sim_\varphi} \mapsto \{p, q\}$ is an isomorphism from $(\mathcal{A} \times \mathcal{A})/{\sim_\varphi}$ to $\mathcal{A}^{\leq 2}$. Note that $(p, q) \sim_\varphi (s, t)$ if and only if $\{p, q\} = \{s, t\}$ and so $\sim_\varphi$ is the $\mathcal{T}_M(\mathcal{A} \times \mathcal{A})$-congruence that precisely identifies (p, q) with (q, p) for $p, q \in Q$.

6.6 Sequences of Upper Sets and Maximal States

In this section, we investigate sequences of upper sets as constructed in Section 3.6.5 for general quasiorders in the special case of automata and the reachability order. More specifically, we relate these sequences to subautomata and Rees quotient automata as introduced previously in Sections 6.5.1 and 6.5.3.

Proposition 6.76 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. There exists a sequence of upper sets*

$$\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$$

such that the sets $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the strongly connected components in $\mathcal{A}$, $\mathcal{A}_{|S_1}$ is strongly connected and, for $i \in \{2, \ldots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected or $S_i \setminus S_{i-1}$ is a sink component in $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ (so that $S_i \setminus S_{i-1}$ and $\{S_{i-1}\}$ are the only sink components in this case).

Proof. Consider a sequence $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 3.59, Item 1, where, according to the statement, the sets $S_i \setminus S_{i-1}$ for $i \in \{1, \ldots, m\}$ are all the strongly connected components. Since S_1 is a strongly connected component, $\mathcal{A}_{S_1}$ is strongly connected. Next, let $i \in \{2, \ldots, m\}$. Now, suppose $S_i \setminus S_{i-1}$ is not a sink component. Since the state set of $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is $S_i \setminus S_{i-1} \cup \{S_{i-1}\}$ with $S_{i-1} \neq \emptyset$, if $S_i \setminus S_{i-1}$ is not a sink component, by Theorem 6.10, Item 2 the automaton is strongly trap-connected; otherwise $S_i \setminus S_{i-1}$ is a sink component and the claim is proved. □

In a sequence as in Theorem 6.76, for $p \in S_i$ and $q \in S_j$ with $p \preceq_R q$ we have $i \leq j$.

Proposition 6.77 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. There exists a sequence of upper sets*

$$S_1 \subsetneq \cdots \subsetneq S_m = Q$$

such that S_1 equals the set of maximal states in $\mathcal{A}$, and for $i \in \{2, \dots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected and $S_i \setminus S_{i-1}$ are all the strongly connected components in $\mathcal{A}$ that are not sink components.

Proof. Consider a sequence $S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 3.59, Item 2 where S_0 equals the set of maximal states and $S_i \setminus S_{i-1}$ for $i \in \{1, \dots, m\}$ are all the non-sink strongly connected components. Since the state set of $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{1, \dots, m\}$ is $S_i \setminus S_{i-1} \cup \{S_{i-1}\}$ with $S_{i-1} \neq \emptyset$ and $S_i \setminus S_{i-1}$ is not a sink component, it is strongly trap-connected by Theorem 6.10, Item 2. □

Remark 6.78 The strongly trap-connected quotient automata that arise from a series as in Theorem 6.76 are always the same up to isomorphism. However, the remaining quotient automata and the subautomaton $\mathcal{A}_{S_1}$ might be different for different series, where they differ by an isolated, i.e., not reachable, trap state. However, the length of such a series is always the same. A series as in Theorem 6.77 always has the same length for a given automaton and the resulting quotient automata $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{2, \dots, m\}$ are always the same up to isomorphism. However, $\mathcal{A}_{S_1}/{\sim_\emptyset}$ might admit further subautomata. For synchronizing automata, series obtained according to Theorem 6.76 or Theorem 6.77 have the same length, isomorphic quotient automata, and $\mathcal{A}_{|S_1}$ admits no subautomata. Thus, in this case, we obtain an analog of the Jordan-Hölder theorem from algebra [405].

Corollary 6.79 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then Q equals the set of maximal states or there exists a proper and non-empty upper set $S \subseteq Q$ such that $\mathcal{A}/{\sim_S}$ is strongly trap-connected.*

Proof. Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 6.77. For $m = 1$, Q equals the set of maximal states. Otherwise, the claim is fulfilled with the upper set $S = S_{m-1}$. □

Proposition 6.80 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states. Then we have the following.*

1. *For all $q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(Q, u) \subseteq S$.*
2. *$[q]_{\sim_S} \preceq_R^{\mathcal{A}/{\sim_S}} S$ in $\mathcal{A}/{\sim_S}$ for all $q \in Q$.*

Proof. 1 & 2: By finiteness, for each $q \in Q$ there exists a maximal $p \in S$ such that $q \preceq_R p$. By the definition of the reachability order, this yields $\delta(q, u) = p$ for some $u \in \Sigma^*$, which proves Item 1. Next for the canonical morphism $\pi_{\sim_S} \colon Q \to Q/{\sim_S}$ from Theorem 6.55 we have $\pi_{\sim_S}(q) \preceq_R^{\mathcal{A}/{\sim_S}} \pi_{\sim_S}(p)$ by Equation (6.5) where $\pi_{\sim_S}(q) = [q]_{\simeq_R}$ and $\pi_{\sim_S}(p) = S$ since $p \in S$. □

Proposition 6.81 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. The set S of maximal states is strongly connected if and only if there exists a unique sink component in $\mathcal{A}$ (which then equals S).*

Proof. First, suppose S is strongly connected. Since two sink component consist of incomparable elements, it follows that S equals a sink component, which is the only sink component in $\mathcal{A}$.

Conversely, suppose there exists a unique sink component. Since S is the union of all sink components by Equation (3.41) this unique sink component equals S and so S is strongly connected. □

6.7 Automata as Language Acceptors

6.7.1 Regular Languages

Chapter 1 introduced semiautomata as mere finite-state transition systems. Here we associate languages with automata. A *(deterministic and complete) finite automaton*[8] *(as a language acceptor)* is a 5-tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ where (Q, Σ, δ) is a semiautomaton as defined before, q_0 is the *start* (or *initial) state* and $F \subseteq Q$ is the *set of final states*. The language *accepted*, or *recognized*, by $\mathcal{A}$ is

$$L(\mathcal{A}) = \{u \in \Sigma^* \mid \delta(q_0, u) \in F\},$$

i.e., the set of labels of paths going from the start state to some final state. We also say a word $u \in \Sigma^*$ is *accepted* by $\mathcal{A}$ if $u \in L(\mathcal{A})$.[9] A language $L \subseteq \Sigma^*$ is called *regular* if there exists an automaton $\mathcal{A}$ such that $L = L(\mathcal{A})$.[10]

Kleene [369] has shown that regular expressions denote precisely the set of regular languages. A proof (in contemporary notation) can be found in any textbook on formal language theory, e.g. [322, 491].

Theorem 6.82 (Kleene's theorem [369]) *The set of regular languages is precisely the set of languages describable by regular expressions, i.e., for every $\mathcal{A} = (Q, \Sigma, \delta, q, F)$ there exists a regular expression R such that $L(\mathcal{A}) = L(R)$ and vice versa.*

Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ and $q \in Q$. The state q is called *accessible* if it is reachable from the start state, i.e., there exists a word $u \in \Sigma^*$ such that $\delta(q_0, u) = q$. It is called *coaccessible* if there exists a word $u \in \Sigma^*$ such that $\delta(q, u) \in F$. We call $\mathcal{A}$ *accessible* (or *initially connected*) if all states are accessible. Obviously, for every regular language there exists an accessible automaton accepting it.

For a semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$, $q \in Q$ and $S \subseteq Q$, by

$$\mathcal{A}_{q,S} = (Q, \Sigma, \delta, q, S)$$

we denote the language acceptor automaton that results from $\mathcal{A}$ by choosing q as a start state and S as a set of final states.

There should be no confusion by calling both types of automata simply automata, as it should be clear from the context whether we mean automata as language acceptors, or whether we are talking about semiautomata. Furthermore, both types are specified by a different sets of data, a 3-tuple versus a 5-tuple, which distinguishes them notationwise. In case any ambiguity might arise, and it is necessary to differentiate between

[8] In the literature, most of the time, automata are meant to be language acceptors without a special term and automata without a start state and a final state are called semiautomata.

[9] There should be no problem with using the modifier "regular" in the context of languages and in the context of permutation groups since both domains are sufficiently different so that no confusion should arise.

[10] Eilenberg [194] called languages acceptable by automata recognizable, which makes sense when considering more general monoids than the free monoid. Since we do not consider the more general case, we do not use this terminology here. See also the related footnote on page 58.

both notions, we revert to using the proper names semiautomaton and automaton as language acceptor.

Every automaton as a language acceptor is also a semiautomaton when forgetting about the start state and the set of final states. So, all semiautomata notions transfer to automata as language acceptors and we use them also for language acceptor automata in this sense.

Given automata $\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$ where $i \in \{1, \ldots, m\}$ and $m \geq 2$, for the product automaton $\mathcal{A}_1 \times \ldots \times \mathcal{A}_m$ we have the following:

$$L(\mathcal{A}_1) \setminus L(\mathcal{A}_2) = L((\mathcal{A}_1 \times \mathcal{A}_2)_{(q_1,q_2),(F_1\times Q)\setminus(Q\times F_2)}), \tag{6.21}$$

$$\bigcap_{i=1}^{m} L(\mathcal{A}_i) = L((\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{(q_1,\ldots,q_m),F_1\times\cdots\times F_m}), \tag{6.22}$$

$$\bigcup_{i=1}^{m} L(\mathcal{A}_i) = L((\mathcal{A}_1 \times \cdots \times \mathcal{A}_m)_{(q_1,\ldots,q_m),E}), \tag{6.23}$$

where $E = \bigcup_{i=1}^{n} Q \times \cdots \times \underbrace{F_i}_{i\text{-th position}} \times \cdots \times Q$.

Next, we state two results that relate the accepted language to morphic images and congruences.

Proposition 6.83 *Let $\varphi\colon Q \to S$ be a morphism between $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Then $L(\mathcal{A}_{q_0,F}) \subseteq L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$ for $q_0 \in Q$ and $F \subseteq Q$. Furthermore, if $\varphi^{-1}(\varphi(F)) = F$, then $L(\mathcal{B}_{\varphi(q_0),\varphi(F)}) \subseteq L(\mathcal{A}_{q_0,F})$.*

Proof. Let $u \in \Sigma^*$. For $\delta(q_0, u) \in F$, we have $\mu(\varphi(q_0), u) = \varphi(\delta(q_0, u)) \in \varphi(F)$ and so $u \in L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$. Next, if $\varphi^{-1}(\varphi(F)) = F$ and $u \in L(\mathcal{B}_{\varphi(q_0),\varphi(F)})$, then we get $\varphi(\delta(q_0, u)) = \mu(\varphi(q_0), u) \in \varphi(F)$ and so $\delta(q_0, u) \in F$. □

Theorem 6.55 gives the following corollary to Theorem 6.83.

Corollary 6.84 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton and $\sim$ be a $\mathcal{T}_S(\mathcal{A})$-congruence. Then for each $q \in Q$ and $F \subseteq Q$ we have $L(\mathcal{A}_{q,F}) \subseteq L((\mathcal{A}/\sim)_{[q]_\sim,[F]_\sim})$ and if F saturates $\sim$, then $L(\mathcal{A}_{q,F}) = L((\mathcal{A}/\sim)_{[q]_\sim,[F]_\sim})$ (note that $[F]_\sim = \{[q]_\sim \mid q \in F\}$).*

Morphisms as introduced here are a concept defined for semiautomata. Theorem 6.83 gives a way to extend this notions to morphisms for automata as language acceptors by also stipulating that the start state is preserved and the final state set saturates the kernel relation. We do not use such an extended notion of morphisms for automata as language acceptors and thus can use the term morphism alone without any ambiguity here.

6.7.2 The Minimal Automaton

Here, we introduce the Myhill-Nerode relation, the minimal automaton of a given language, the canonical map of an accessible automaton onto the minimal automaton,

and the notion of distinguishability of states. Theorem 6.86 gives a characterization of regularity in terms of the Myhill-Nerode relation and shows that the canonical map is a surjective morphism. Then, in Theorem 6.88, we characterize those automata with the least number of states accepting a given regular language. Theorem 6.91 characterizes those automata having the least number of states for every distribution of a start state and some non-empty and proper set of final states.

Let $L \subseteq \Sigma^*$ be a language.

1. For $u, v \in \Sigma^*$ define the *Myhill-Nerode relation* w.r.t. L [456, 459] by

$$u \equiv_L v \Leftrightarrow u^{-1}L = v^{-1}L.$$

   The relation $\equiv_L$ is an equivalence relation that saturates L and such that, for all $u, v, w \in \Sigma^*$, we have[11]

$$u \equiv_L v \Rightarrow uw \equiv_L vw. \tag{6.24}$$

2. The *minimal automaton (of L)* is the (here possibly infinite) acceptor automaton $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$ with

$$\begin{aligned} Q_L &= \Sigma^*/\equiv_L, \\ \delta_L([u]_{\equiv_L}, v) &= [uv]_{\equiv_L}, \\ F_L &= \{[u]_{\equiv_L} \mid u \in L\} \end{aligned}$$

   where δ_L is well-defined by Equation (6.24). Clearly, $\mathrm{MinAut}(L)$ is accessible.
3. Suppose L is regular and $L = L(\mathcal{A})$ for some accessible automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$. Then, for $u, v \in \Sigma^*$,

$$\delta(q_0, u) = \delta(q_0, v) \Rightarrow u \equiv_L v. \tag{6.25}$$

   Let $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$. By Equation (6.25), the map $\pi_{\mathcal{A}} \colon Q \to Q_L$, called the *canonical map of $\mathcal{A}$ onto* $\mathrm{MinAut}(L)$, given by

$$\pi_{\mathcal{A}}(q) = [u]_{\equiv_L}$$

   for some word $u \in \Sigma^*$ such that $\delta(q_0, u) = q$ is well-defined. So, $\pi_{\mathcal{A}}(\delta(q_0, u)) = [u]_{\equiv_L}$ and $\pi_{\mathcal{A}}$ is surjective.
4. Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be an automaton. Two states $p, q \in Q$ are *distinguishable* (in $\mathcal{A}$) if there exists $u \in \Sigma^*$ such that

$$(\delta(p, u) \in F \wedge \delta(q, u) \notin F) \vee (\delta(p, u) \notin F \wedge \delta(q, u) \in F). \tag{6.26}$$

   We call p and q *indistinguishable* if they are not distinguishable. The indistinguishable relation is a quasiorder.

[11] Every word $w \in \Sigma^*$ can be associated to a unique transformation $\sigma_w \colon \Sigma^* \to \Sigma^*$ given by $\sigma_w(u) = uw$ for $u \in \Sigma^*$. Then $\hat{\Sigma}^* = \{\sigma_w \mid w \in \Sigma^*\}$ gives a transformation monoid on Σ^* and Equation (6.24) states that $\equiv_L$ is a $\hat{\Sigma}^*$-congruence.

Suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence that saturates $F \subseteq Q$ for $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$. Then the following implication holds for $p, q \in Q$:

$$p \sim q \Rightarrow p \text{ and } q \text{ are indistinguishable,} \tag{6.27}$$

i.e., the indistinguishability relation is the coarsest $\mathcal{T}_S(\mathcal{A})$-congruence saturating F. Note that, by Equation (6.26), two states p, q are indistinguishable in $\mathcal{A}$ if and only if

$$\forall u \in \Sigma^* \colon \delta(p, u) \in F \Leftrightarrow \delta(q, u) \in F.$$

We aim to characterize minimality in terms of $\equiv_L$, congruences, morphisms, and the distinguishability relation in Theorem 6.88. But before, we list relations, needed in the proof of Theorem 6.88, between $\equiv_L$, accessible automata $\mathcal{A}$ and $\pi_{\mathcal{A}}$. The next lemma follows readily from the definitions.

Lemma 6.85 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be accessible and $L = L(\mathcal{A})$. Then, for $u, v \in \Sigma^*$, the following are equivalent:*

1. $\pi_{\mathcal{A}}(\delta(q_0, u)) = \pi_{\mathcal{A}}(\delta(q_0, v))$,
2. $u \equiv_L v$,
3. $\delta(q_0, u)$ *and* $\delta(q_0, v)$ *are indistinguishable.*

Next, we summarize the important properties of $\mathrm{MinAut}(L)$ and $\pi_{\mathcal{A}}$.

Theorem 6.86 *Let $L \subseteq \Sigma^*$ and $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$. Then L is regular if and only if Q_L is finite. Further, if L is regular, then for every accessible $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with $L = L(\mathcal{A})$, the map $\pi_{\mathcal{A}} \colon Q \to Q_L$ is a surjective morphism such that $\pi_{\mathcal{A}}(q_0) = [\varepsilon]_{\equiv_L}$, $\pi_{\mathcal{A}}^{-1}(F_L) = F$ and $\mathrm{MinAut}(L)$ accepts L.*

Proof. First, suppose Q_L is finite. As $\equiv_L$ saturates L, for $u \in \Sigma^*$, we get

$$u \in L \Leftrightarrow \exists v \in L \colon u \equiv_L v \Leftrightarrow [u]_{\equiv_L} \in F_L \Leftrightarrow \delta_L([\varepsilon]_{\equiv_L}, u) \in F_L.$$

So, $\mathrm{MinAut}(L)$ is a finite automaton accepting L. Hence, L is regular. Conversely, suppose L is regular. Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton with $L = L(\mathcal{A})$, which we can assume to be accessible. As $\pi_{\mathcal{A}}$ is surjective, we find that Q_L is finite. Next, we show the remaining claims w.r.t. $\mathcal{A}$. Let $q \in Q$ and $u \in \Sigma^*$ with $\delta(q_0, u) = q$. Then, by the definitions of $\pi_{\mathcal{A}}$ and δ_L, we have, for $v \in \Sigma^*$,

$$\pi_{\mathcal{A}}(\delta(q, v)) = \pi_{\mathcal{A}}(\delta(q_0, uv)) = [uv]_{\equiv_L} = \delta_L([u]_{\equiv_L}, v) = \delta_L(\pi_{\mathcal{A}}(q), v).$$

So, $\pi_{\mathcal{A}}$ is a morphism from $\mathcal{A}$ onto $\mathrm{MinAut}(L)$. Lastly, by the definition of F_L, $\pi_{\mathcal{A}}$ and as $\equiv_L$ saturates L, we have

$$\begin{aligned}
\pi_{\mathcal{A}}(q) \in F_L &\Leftrightarrow \exists u \in L\ \exists v \in \Sigma^* \colon \delta(q_0, v) = q \wedge v \equiv_L u && \text{(Def. } \pi_{\mathcal{A}} \text{ \& } F_L\text{)} \\
&\Leftrightarrow \exists u \in L\ \exists v \in L \colon \delta(q_0, v) = q \wedge v \equiv_L u && (\equiv_L \text{ saturates } L) \\
&\Leftrightarrow \exists v \in L \colon \delta(q_0, v) = q \\
&\Leftrightarrow q \in F.
\end{aligned}$$

Therefore, $\pi_{\mathcal{A}}^{-1}(F_L) = F$. By Theorem 6.83, we find $L(\mathcal{A}) = L(\mathrm{MinAut}(L))$. □

Brzozowski [106] has shown that a language is regular if and only if the set of its left quotients is finite, which follows from Theorem 6.86.

Corollary 6.87 (Brzozowski [106]) *A language $L \subseteq \Sigma^*$ is regular if and only if $\{u^{-1}L \mid u \in \Sigma^*\}$ is finite.*

Proof. By the defining equation of the Myhill-Nerode relation, $[u]_{\equiv} \mapsto u^{-1}L$ is a bijection between Q_L and the different left quotients. Thus, the claim follows from Theorem 6.86. □

Next, we state various characterizations for an automaton to have the least number of states for its accepted language.

Theorem 6.88 *Let $L \subseteq \Sigma^*$ be regular,* $\mathrm{MinAut}(L) = (Q_L, \Sigma, \delta_L, [\varepsilon]_{\equiv_L}, F_L)$ *and $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with $L = L(\mathcal{A})$. Then the following are equivalent.*

1. *$\mathcal{A}$ has the least number of states such that $L = L(\mathcal{A})$.*
2. *$\mathcal{A}$ is accessible and there exists no automaton $\mathcal{B} = (S, \Sigma, \mu, s_0, E)$ and a morphism $\varphi\colon Q \to S$ with $\varphi(q_0) = s_0$ and $\varphi^{-1}(E) = F$ such that $|\varphi(Q)| < |Q|$.*
3. *$\mathcal{A}$ is accessible and $\pi_{\mathcal{A}}\colon Q \to Q_L$ is an isomorphism of $\mathcal{A}$ onto* $\mathrm{MinAut}(L)$.
4. *$\mathcal{A}$ is accessible and all pairs of distinct states are distinguishable.*
5. *$\mathcal{A}$ is accessible and the equality relation is the only $\mathcal{T}_S(\mathcal{A})$-congruence saturating F.*

Proof. 1 ⇒ 2: Set $T = \uparrow\{q_0\}$. The automaton $(\mathcal{A}_{|T})_{q_0, T\cap F}$ is accessible and we have $L((\mathcal{A}_{|T})_{q_0,T\cap F}) = L(\mathcal{A})$. By assumption, we find $T = Q$. So, $\mathcal{A}$ is accessible. Next, let $\mathcal{B}$ and φ be as in the statement. We can assume φ is surjective, i.e., $S = \varphi(Q)$. So, $\varphi(F) = \varphi(\varphi^{-1}(E)) = E$. Furthermore, we have

$$\varphi^{-1}(\varphi(F)) = \varphi^{-1}(\varphi(\varphi^{-1}(E))) = \varphi^{-1}(E) = F.$$

Then, by Theorem 6.83, we find $L(\mathcal{A}) = L(\mathcal{B})$. Since $\mathcal{A}$ has the least number of states among all automata accepting $L(\mathcal{A})$, we get $|S| = |Q|$.

2 ⇒ 3: By Theorem 6.86 we have $\pi_{\mathcal{A}}(q_0) = [\varepsilon]_{\equiv_L}$, $\pi_{\mathcal{A}}^{-1}(F_L) = F$ and $\pi_{\mathcal{A}}$ is surjective so that $|Q_L| \leq |Q|$. By assumption, we get $|Q| = |Q_L|$ and so $\pi_{\mathcal{A}}$ is bijective.

3 ⇒ 4: The automaton $\mathrm{MinAut}(L)$ is accessible and as $\mathrm{MinAut}(L) \cong \mathcal{A}$, also $\mathcal{A}$ is accessible. Let $p, q \in Q$ be two distinct states. As $\pi_{\mathcal{A}}$ is injective, we have $\pi_{\mathcal{A}}(p) \neq \pi_{\mathcal{A}}(q)$. By Theorem 6.85, it follows that p and q are distinguishable.

4 ⇒ 5: Suppose $\sim$ is some $\mathcal{T}_S(\mathcal{A})$-congruence that saturates F. As any pair of distinct states $p, q \in Q$ is distinguishable, we find, by Formula (6.27), that $p \nsim q$. So $\sim$ equals the equality relation.

5 ⇒ 1 : Suppose there exists an accessible automaton $\mathcal{B} = (S, \Sigma, \mu, s_0, E)$ with $|S| < |Q|$ and $L = L(\mathcal{B})$. By Theorem 6.86, as $\pi_{\mathcal{B}}$ is surjective, we have $|Q_L| \leq |S|$. Thus $|Q_L| < |Q|$ and so $\pi_{\mathcal{A}}$ maps to distinct states to one state in $\mathrm{MinAut}(L)$, giving that $\sim_{\pi_{\mathcal{A}}}$ does not equal the equality relation. By Theorems 6.56 and 6.86, $\sim_{\pi_{\mathcal{A}}}$ is a $\mathcal{T}_M(\mathcal{A})$-congruence, which, as $\pi_{\mathcal{A}}^{-1}(F_L) = F$, saturates F. □

For regular $L \subseteq \Sigma^*$, we call $\min\{|Q| : \mathcal{A} = (Q, \Sigma, \delta, q_0, F) \text{ with } L = L(\mathcal{A})\}$ the *state complexity* of L, which equals, by Theorem 6.88, Item 3, the number of states of $\mathrm{MinAut}(L)$. Furthermore, all automata with the least number of states accepting L are isomorphic by Theorem 6.88, Item 3, which justifies calling $\mathrm{MinAut}(L)$ the minimal automaton. In view of this, we also call $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ *minimal* if $\pi_{\mathcal{A}}$ is an isomorphism. Note that two non-coaccessible states are indistinguishable. So, a minimal automaton has at most one state that is not coaccessible and this state must be a trap state.

Remark 6.89 Since the indistinguishability relation is the coarsest $\mathcal{T}_S(\mathcal{A})$-congruence saturating F, by Theorem 6.88, Item 5 the automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is minimal if and only if it is accessible and the indistinguishability relation equals the equality relation.

Remark 6.90 Accessibility in Items 2,4 and 5 of Theorem 6.88 is crucial. For example, in an automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ whose state set forms two cycles of size at least two induced by different letters a and b that induce self-loops on the other cycle and with one final state on each cycle, all pairs of distinct states are distinguishable, equality is the only $\mathcal{T}_S(\mathcal{A})$-congruence saturating F, and there exists no morphism as stated in Item 2. This follows by considering $\mathcal{A}_{|\{a\}}$ it either permutes the states on the a-cycle or maps all those states to a single state outside the a-cycle. But since this is also a morphism of $\mathcal{A}_{|\{b\}}$, it must permute the full b-cycle if it maps outside the a-cycle. The same argument for the b-cycle gives the it has to permute the a-cycle and the b-cycle. Thus it is an automorphism.

Davies [171, Lem. 3.7] (see also [172, Lem. 4.2.8]) proved that an automaton with a non-empty and proper final state set is minimal if and only if it admits no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence saturating the final state set. This readily follows from Theorem 6.88, Item 5, as the final state set is empty or equals the whole state set if and only if the accepted language is empty or equals Σ^*, respectively. Further, she also characterized strongly connected automata that are minimal for every start state and proper non-empty set of final states by the absence of non-trivial $\mathcal{T}_S(\mathcal{A})$-congruences [171, Prop. 4.2.3] (or [172, Prop. 3.2]). Next, we state this together with other characterizations as a corollary to Theorem 6.88.

Note that for strongly connected $\mathcal{A} = (Q, \Sigma, \delta)$ with at least two states, $L(\mathcal{A}_{q,F})$ is a non-empty proper subset of Σ^* for $q \in Q$ and non-empty and proper $F \subsetneq Q$ since $\delta(q, u) \in F$ and $\delta(q, v) \notin F$ for some $u, v \in \Sigma^*$ by accessibility and strong connectedness.

Corollary 6.91 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}_{q,F}$ is minimal for each $q \in Q$ and non-empty and proper $F \subsetneq Q$.*
2. *$\mathcal{A}$ is strongly connected and all pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$ for all $q \in Q$ and $\emptyset \neq F \subsetneq Q$.*
3. *$\mathcal{A}$ is strongly connected and there exists no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence.*
4. *$\mathcal{A}$ is strongly connected and there exists no $\mathcal{B} = (S, \Sigma, \mu)$ with a morphism $\varphi\colon Q \to S$ such that $1 < |\varphi(Q)| < |Q|$.*

Proof. $1 \Rightarrow 2$: Suppose $\mathcal{A}_{q,F}$ is minimal for each $q \in Q$ and $\emptyset \neq F \subsetneq Q$. By Theorem 6.88, Item 4, all pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$. Fix some $\emptyset \neq F \subsetneq Q$. By Theorem 6.88, Item 4, $\mathcal{A}_{q,F}$ is accessible w.r.t. each state q. Now, by Theorem 3.43, Item 4, $\mathcal{A}_{q,F}$ being accessible for any $q \in Q$ is equivalent to $\mathcal{A}$ being strongly connected. Alternatively, let $S \subseteq Q$ be a sink component and $q \in S$. As the states in $Q \setminus S$ are not accessible in $\mathcal{A}_{q,S}$, we must have $Q = S$ since minimality implies that all states are accessible by Theorem 6.88. So $\mathcal{A}$ is strongly connected.

$2 \Rightarrow 3$: This follows from Equation (6.27).

$3 \Rightarrow 4$: This follows from Theorem 6.56, Item 2.

$4 \Rightarrow 1$: Let $q \in Q$ and $\emptyset \neq F \subsetneq Q$. Since $\mathcal{A}$ is strongly connected, $\mathcal{A}_{q,F}$ is accessible. As there exists no morphism onto an automaton with less states, by Theorem 6.88, Item 2, $\mathcal{A}_{q,F}$ is minimal. □

Note that the equivalences from Theorem 6.91 also hold for $|Q| = 1$ vacuously since then there exists no proper and non-empty final state set and no morphism φ with $1 < |\varphi(Q)| < |Q|$. A one-state automaton only accepts $\emptyset$ or Σ^*.

Theorem 6.68 stated that an automaton admitting no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence is either strongly connected or the state set is partitioned into a trap state and a strongly connected component. Theorem 6.91 gives more details in case the automaton is strongly connected and admits no non-trivial $\mathcal{T}_M(\mathcal{A})$-congruence.

Remark 6.92 Strong connectivity is necessary in Items 2,3, and 4 of Theorem 6.91. For example, consider $\mathcal{A} = (\{0, 1, \ldots, n\}, \{a, b\}, \delta)$ with $\delta_a = (0)(1\ 2\ \cdots n)$ and $\delta_b = [1 \mapsto 0]$ for some number $n > 1$. Then $\mathcal{A}_{0,\{0\}}$ is not minimal. However, we have the following:

1. All pairs of distinct states are distinguishable in $\mathcal{A}_{q,F}$ for each $q \in Q$ and non-empty and proper $F \subseteq Q$, for we can move one state to 0 while the other remains on the cycle, and then can be moved to a state that is final if $0 \notin F$ or not final if $0 \in F$.
2. The automaton admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence. For suppose $\sim$ is a $\mathcal{T}_S(\mathcal{A})$-congruence such that $p \sim q$ for $p, q \in Q$ with $p < q$. Then $\delta(p, b^{n-p+1}a) = 0$, $\delta(q, b^{n-p+1}a) \neq 0$ and thus $\delta(q, b^{n-p+1}a) \sim 0$. This yields $\delta(q, b^{n-p+1}ab^i) \sim 0$ for each $i \geq 0$, and since $p = \delta(q, b^{n-p+1}ab^i)$ for some $i \geq 0$ for each $p \in Q \setminus \{0\}$, we find that all pairs of states are congruent w.r.t. $\sim$.
3. There exists no morphism to a strictly smaller automaton with at least two states.

So, each condition except strong connectivity is fulfilled.

Note that any automaton $\mathcal{A}$ with at most two states, even those that are not strongly connected, admits no non-trivial $\mathcal{T}_S(\mathcal{A})$-congruence, no non-trivial morphic images, and has the property that distinct states are indistinguishable in $\mathcal{A}_{q,F}$ for $q \in Q$ and $\emptyset \neq F \subsetneq Q$.

Restivo & Vaglica [508, 509] also investigated automata that are minimal for all k-sets of final states for some $k \geq 1$ or for all non-empty and proper subsets of final states.

6.8 Remarks on Transformation Semigroups and Automata

To every (complete and deterministic) semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ corresponds its transformation semigroup $\mathcal{T}_S(\mathcal{A})$ with generating set $\{\delta_a \colon Q \to Q \mid a \in \Sigma\}$. Conversely, let $S \leq_S \mathcal{T}_n$ be a transformation semigroup generated by $g_1, \ldots, g_m$. Then for $\Sigma = \{g_1, \ldots, g_n\}$ (viewing the generating sequence as a set of abstract symbols) we obtain a semiautomaton $\mathcal{A} = ([n], \Sigma, \delta)$ with $\mathcal{T}_S(\mathcal{A}) = S$ by setting $\delta(q, g_i) = g_i(q)$ for $q \in [n]$ and $i \in \{1, \ldots, m\}$.[12] Thus, a semiautomaton may be viewed as a transformation semigroup together with a distinguished generating sequence.

Example 6.93 The choice of a generating set for a given transformation semigroup $S \leq_S \mathcal{T}_n$ can lead to quite distinct automata. For example, consider the permutation group $G = \langle g \rangle_G$ with $g = (0\,1\,\cdots\,5)$. When constructing the automaton with generating set $\{g\}$ we obtain an automaton that cyclically permutes its states. Another generating set is given by $g^2 = (0\ 2\ 4)(1\ 3\ 5)$ and $g^3 = (0\ 3)(1\ 4)(2\ 5)$ since $g^2 g^2 g^3 = g$. The automaton we obtain from this generating set is depicted in Figure 6.7.

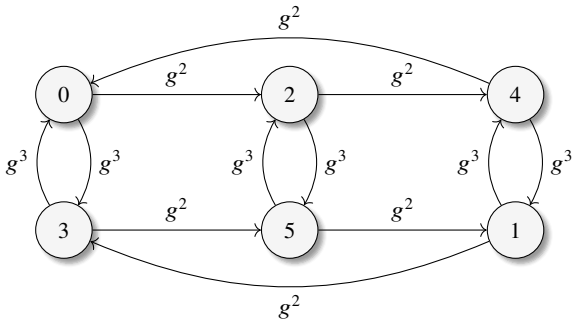


Fig. 6.7: Automaton constructed from permutation group, see Theorem 6.93.

Many of our notions and results can be formulated either for automata or for transformation semigroups. For example, $\mathcal{A}$ is synchronizing if and only if $\mathcal{T}_S(\mathcal{A})$ contains a constant transformation.

Our results are motivated by automata-theoretical problems, but properties only related to subset mapping properties do not depend on the choice of some generating set and are actually properties of transformation semigroups. So, these properties are better expressed in an algebraic setting with transformation semigroups, as done in Part III. The corresponding automata-theoretic reformulations are usually straightforward and are therefore omitted.

A semiautomaton is a machine model that processes, or reacts to, inputs. In this setting, the alphabet is crucial, as it is the means by which the machine interacts with its environment. Since inputs correspond to physical actions and an operator may want

[12] A transformation g_i is allowed to appear more than once among $g_1, \ldots, g_m$.

to minimize the effort to achieve a given goal, it is natural to ask for the minimum number of such actions required to do this. Thus, subset mapping properties with length bounds on shortest words realizing some property are naturally expressed in an automata-theoretic formulation.

Later in Part IV, we use automata-theoretic notation exclusively.

We note that there is a generalization of the Černý conjecture for transformation monoids [22, 524]. Shortest words in some generating sets have also been investigated in the context of the diameter of a Cayley graph depends [256].

6.9 Discussion

Some material on automata is presented differently here or is not covered in standard texts, such as [322, 339, 388]. Specifically, the various facts on semiautomata morphisms from Section 6.4 appear to be scattered in the literature or, as far as we know, not stated explicitly in the form presented here. Moreover, the isomorphisms in Theorem 6.66 concerning Rees quotient automata and Theorem 6.67 are, to the best of our knowledge, new. Automata can also be viewed as unary algebras with finitely many operations on a finite domain. Results from automata theory and formal language theory have been extended to unary algebras in [480]; see also [617, p. 835] and [97] for connections between unary algebras, their identities, and synchronizing automata.

Overview on Automata Theory.

The theory of finite automata as an independent field of study is generally considered to have started with the seminal book *Automata Studies* [564]. Medvedev translated this book into Russian and added his own article [450], in which he introduced semiautomata (therefore sometimes referred to as *Medvedev automata*, see [547, 617]). Early work in automata theory was primarily concerned with the processing of events, signals, and languages by automata rather than with automata viewed merely as finite transition systems. While models of computation were initially developed in the 1930s to investigate the limits of computability, the focus later shifted to modeling the operation of nervous systems using electronic components. See, for example, [460, 563] for surveys of the work and motivations of the 1940s and 1950s (where automata is also used in a broader sense than here). Synchronization is a concept tied to the transition structure of an automaton, or more generally a machine, and thus is a semiautomaton concept.[13]

Fleck [218, 219] introduced morphisms on automata and initiated a structure theory for automata centered on their automorphism groups. For example, among the results obtained in this line of research are the following.

[13] In [320] the notion of synchronization was transferred to languages via the minimal automaton by calling a language synchronizing if there exists a word always ending in the same state in the (possibly infinite) minimal automaton of the language.

1. The automorphism group of a strongly connected automaton is a semiregular permutation group [219, 631] (here stated in Theorem 6.43).[14]
2. Let $\mathcal{A} = (\{q_1, \ldots, q_n\}, \Sigma, \delta)$ be strongly connected. In [631], for $i \in \{1, \ldots, n\}$, an equivalence relation $\sim_i$ on Σ^* was introduced where, for $u, v \in \Sigma^*$,

$$u \sim_i v \quad \text{if and only if} \quad \delta(q_i, u) = \delta(q_i, v).$$

   Furthermore, in [631, Def. 5], the quotient $\Sigma^*/\sim_i$ was equipped with a semigroup structure $(\Sigma^*/\sim_i, *_i)$ by setting $[u]_{\sim_i} *_i [v]_{\sim_i} = [w]_{\sim_i}$ precisely if $\delta(q_i, u'v') = \delta(q_i, w)$ for all $u', v' \in \Sigma^*$ with $u' \sim_i u$ and $v' \sim_i v$. Barnes [58, 59] used these semigroups to characterize the automorphism group of $\mathcal{A}$. For instance, this semigroup structure is isomorphic to the automorphism group whenever the latter is regular, i.e. transitive [58, Cor. 1 and Thms. 3 & 4].

The investigation of automata via their automorphism groups was continued in the following works: [58, 59, 66, 67, 219, 220, 266, 267, 268, 443, 469, 632, 633].

A *monoid action* or *(right) M-act* [367] is a tuple (X, M, μ) where X is some set, M a monoid with identity $1 \in M$, and $\mu\colon X \times M \to X$ a function such that

1. $\mu(x, 1) = x$ and
2. $\mu(x, st) = \mu(\mu(x, s), t)$ for $x \in X$ and $s, t \in M$.

A morphism of two M-acts (X, M, μ) and (Y, M, ρ) is a map $\varphi\colon X \to Y$ such that $\varphi(\mu(x, s)) = \rho(\varphi(x), s)$ for all $x \in X$ and $s \in M$. The morphism is called an endomorphism of the M-act if $X = Y$. Via their extensions to Σ^*, semiautomata are precisely the Σ^*-acts, and morphisms of semiautomata coincide with the morphisms of these Σ^*-acts (see also [367, Prop. 4.5]). Hence, the study of semiautomata may be viewed as the study of finite Σ^*-acts. For a group G, a G-act is also called a group action, G-action, or a G-set [126, 179]. Isomorphisms of G-acts are called representation equivalences in [179, p. 21]. The notion of G-acts also arises in algebra, topology, and geometry [467].

Remark 6.94 An M-act can also be identified with a monoid morphism from M to $\mathcal{T}_X$ equipped with the left-most-first composition operation ; from Section 3.11 as its operation, in the following way. Given an M-act (X, M, μ), the map $s \mapsto \mu_s$ for $s \in M$ with $\mu_s(x) = \mu(x, s)$ for $x \in X$ gives such a morphism. Conversely, given a morphism $\varphi\colon M \to \mathcal{T}_X$ of $(M, \cdot)$ to $(T_X, ;)$, defining $\mu\colon X \times M$ by $\mu(x, s) = \varphi(s)(x)$ yields an M-act (X, M, μ). For example, every transformation monoid $M \leq_M \mathcal{T}_X$ can be considered as an M-act by applying this construction to the identity transformation on M.

In this setting, semiautomata can be seen as morphisms $\Sigma^* \to \mathcal{T}_Q$. The image of such a morphism is the transformation monoid. This viewpoint was used in [534]. Since morphisms from Σ^* are uniquely determined by their images on Σ, it suffices to specify a semiautomaton by a map $\Sigma \to \mathcal{T}_Q$ and Theorem 3.60 then yields the extension to all of Σ^*.

[14] In [219, 631], semiregularity is referred to simply as "regular". We follow the terminology that is now standard [126, 179, 638].

The notion of an $\mathcal{T}_S(\mathcal{A})$-congruence was first considered by Hartmanis [279] (therein called partitions with the substitution property) in connection with series-parallel decompositions of automata. Slightly earlier, Rabin and Scott [502] had considered the closely related concept of right-invariant congruences on Σ^*. The notion of a quotient automaton is implicit in Hartmanis's work, but was formally introduced for the first time by Bayer [68]. However, he defined it [68, Def. 9] only when $\mathcal{T}_S(\mathcal{A})$ is a group. One year later, Bayer [69] gave the general definition and related the lattice of $\mathcal{T}_S(\mathcal{A})$-congruences to endomorphisms of general automata. Thierrin [599] and Babcsányi [51] investigated primitive automata $\mathcal{A}$ (therein called *simple automata*). For example, Theorem 6.68 is due to Thierrin [599, Prop. 8]. Primitivity has been linked to minimality conditions for automata in [171, 172, 508, 509].

In the context of coding theory and code parsing, Schützenberger [551, Sect. 5 & 6] introduced the syntactic semigroup of a code (which he called the *fundamental semigroup*) and represented its elements using matrices. His construction in fact yields the transition matrices of the minimal automaton of the code. Although automata are not mentioned explicitly, this relation to the minimal automaton shows that his work can be viewed as the first investigation of the transformation semigroup of an automaton. An early work that explicitly introduced transition matrices of automata is the work by Burks and Wang [116]. Closely related stochastic transition matrices had, however, already been used decades earlier in the theory of Markov chains.

The congruences $\sim_i$ from Weeg [631, Def. 5], discussed above, are related to the transformation semigroup as follows. Let $\sim$ be the equivalence relation on Σ^* defined by

$$u \sim v \quad \Longleftrightarrow \quad u \sim_i v \text{ for all } i \in \{1, \ldots, n\}.$$

Then $u \sim v$ if and only if $\delta(q_i, u) = \delta(q_i, v)$ for all $i \in \{1, \ldots, n\}$. Moreover, if $u \sim v$, then $wu \sim wv$ and $uw \sim vw$ for every $w \in \Sigma^*$. Thus, for the product on subsets we have

$$[u]_\sim \, [v]_\sim = [uv]_\sim,$$

and so we obtain a well-defined operation on $\Sigma^*/\!\sim$, making it into a semigroup. Now the map $[u]_\sim \mapsto \delta_u$ is an isomorphism onto the transformation semigroup.

In the early days, automata were considered as *(sequential) transducers* (or *automata with output*), see [322, 548] for details. A notion related to synchronization in this setting is that of a *homing sequence* for a transducer automaton model. Such a sequence is an input word whose produced outputs uniquely determine the state reached by the automaton. For more information, we refer to the survey by Sandberg [548], which discusses homing sequences and their relation to synchronizing sequences.

The seminal paper by Rabin and Scott [502] is widely regarded as the first to systematically consider finite automata as language acceptors.[15] They also introduced nondeterministic automata and the power-set construction and established much of the standard terminology and notation. Nevertheless, it was not the first paper to relate finite automata to language acceptance. Shannon [561] investigated them as language-generating devices in the context of coding theory and Chomsky [148, 150] studied them

[15] Behind the Iron Curtain, however, automata had already been defined as language acceptors by Medvedev [450].

in the context of the processing, understanding, and production of natural languages by humans.[16] Furthermore, building on the ideas of McCulloch and Pitts [446], several authors, for example Burks and Wang [115, 116] and Kleene [369], investigated finite automata as models of nerve nets. Kleene [369] introduced regular expressions in this context.

The characterization of minimal automata and the Myhill–Nerode relation go back to Nerode [459] (input-output version) and Myhill [456] (automata/semigroup version). Regular languages have been classified by algebraic properties of the transformation semigroups of minimal accepting automata via the theory of (pseudo-)varieties of languages and semigroups [195, 196, 489]. Seminal results in this direction include the characterization of star-free languages by counter-free automata [449] and aperiodic monoids [552], see [194, 195, 489, 490, 491] for more information.

Krohn and Rhodes [389] developed the *Krohn–Rhodes decomposition theory*, a decomposition theory for automata. In this theory, automata are represented as homomorphic images of certain basic permutation automata and aperiodic automata related to the given automaton. The theory identifies basic building blocks from which every automaton can be simulated by an appropriate product of these basic automata. There are several formulations of this theory:

1. The original semigroup-theoretic formulation from [389] and [39, Chap. 5] uses the wreath product. Other formulations based on the same notions were given in [400] and [201]. Textbook expositions close to the original formulation can be found in [511, 512]. The monograph [511] also discusses broader applications to biology, physics, and psychology, together with some philosophical reflections.
2. A formulation in terms of the cascade product was developed in [247, 645, 646] and [39, Chap. 4]. In this formulation, every automaton is covered[17] by a cascade product of certain permutation-reset automata. Ginzburg [247] gives a correction to the proof in [645, 646]. We also refer to the expository article [649].
3. Meyer and Thompson's [121] formulation in terms of cascade products connects the theory to an earlier decomposition result of Hartmanis and Stearns [285].
4. Based on Zeiger's approach [645, 646], Eilenberg [195] gave an elegant formulation in the language of transformation semigroups, termed the *holonomy decomposition*, see also [317]. Egri-Nagy and Nehaniv [181, 190, 191, 192, 193] gave an automata-theoretic reformulation of the holonomy decomposition and studied its complexity.
5. Diekert, Kufleitner, and Steinberg [176] gave an algebraic formulation utilizing the notion of local divisors.

In the special case of groups, the Krohn–Rhodes theorem reduces to the *(Krasner–Kaloujnine) universal embedding theorem* [352], see [179, Sect. 2.6] for a textbook exposition of this special case.

Maler and Pnueli [431] carried out an in-depth study of the size of the Krohn–Rhodes decomposition and used their results to obtain an improved algorithm for translating counter-free automata into past temporal logic formulae. A corresponding bound for

[16] Chomsky concluded that regular languages are not sufficiently expressive to model natural languages adequately and therefore introduced more powerful grammar formalisms in [148].

[17] Cover maps are related to morphisms.

future linear temporal logic (LTL) was later established in [98]. We refer to [430, 432] for further discussion and pointers to the literature. Results on the relationship between temporal logics and automata can be found in [155].

The survey by Glushkov [248] also gives a good overview of the early developments in automata theory, and includes also work carried out behind the Iron Curtain.

More on Automata Morphisms.

In the transformation semigroup viewpoint, the endomorphism semigroup corresponds to the centralizer of a transformation semigroup, that is, the set of transformations commuting with every transformation in the semigroup. The centralizer of a single transformation has been characterized in [37]; we also refer to this work for further pointers to the literature. For permutation groups, the centralizer of a single permutation in the full symmetric group is described in [589, pp. 295], [637, Thm. 15.2], and [365], see also the generalization in [414]. It is a folklore result in permutation group theory that the centralizer in the symmetric group of a transitive (resp. semiregular) permutation group is semiregular (resp. transitive). Consequently, a regular permutation group on a finite set has the same order as its centralizer and coincides with its centralizer when it is abelian. That the centralizer of a transitive permutation group is semiregular also follows from Theorem 6.43.

Theorems 6.30 and 6.31 state that morphisms between strongly trap-connected automata are either full isomorphisms or map all states to a single trap state. This is somehow reminiscent of the well-known Schur's lemma from ring theory [405], see also a related remark from [534, p. 275].

In our definition of morphisms, the alphabet was fixed. We can generalize this by including a mapping $\alpha\colon \Sigma \to \Gamma$ of alphabets in the definition and let $\varphi\colon Q \to S$ be a morphism between $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Gamma, \mu)$ if

$$\varphi(\delta(q, x)) = \mu(\varphi(q), \alpha(x))$$

for all $q \in Q$ and $x \in \Sigma$ (a definition in this direction was given in [637]).

Given such a morphism that is surjective, let $F\colon \mathcal{T}_M(\mathcal{A}) \to \mathcal{T}_M(\mathcal{B})$ be given by $F(\delta_u) = \mu_{\alpha(u)}$ for $u \in \Sigma^*$ with α extended to a mapping (also denoted by α) from Σ^* into Γ^* via Theorem 3.60. This is well-defined since for $u, v \in \Sigma^*$ with $\delta_u = \delta_v$, by choosing for $s \in S$ some $q \in Q$ with $s = \varphi(q)$ we have

$$\mu_{\alpha(u)}(s) = \mu(\varphi(q), \alpha(u)) = \varphi(\delta(q, u)) = \varphi(\delta(q, v)) = \mu(\varphi(q), \alpha(v)) = \mu_{\alpha(v)}(s)$$

Then we have the following:

1. F is a monoid morphism since

$$F(\delta_v \delta_u) = F(\delta_{uv}) = \mu_{\alpha uv} = \mu_{\alpha(v)} \mu_{\alpha(u)} = F(\delta_v) F(\delta_u)$$

2. For $t \in \mathcal{T}_M(\mathcal{A})$ with $t = \delta_u$ for $u \in \Sigma^*$ we have

$$F(t)(\varphi(q)) = F(\delta_u)(\varphi(q)) = \mu(\varphi(q), u) = \varphi(\delta(q, u)) = \varphi(\delta_u(q)) = \varphi(t(q))$$

for $q \in Q$.

This is related to the notion of permutation isomorphism from [179, p. 17] which is defined between two permutation groups $G \leq_G \mathcal{S}_X$ and $H \leq_G \mathcal{S}_Y$ as a bijection $\lambda\colon X \to Y$ with a group isomorphism $\psi\colon G \to H$ such that $\lambda(g(p)) = \psi(g)(\lambda(p))$ for all $p \in X$ and $g \in G$.

Reachability Quasiorder.

The reachability relation appears to have been introduced independently by Meyer and Thompson [121], Brzozowski and Fich [111], and Letichevsky [408]; it was later used in [340]. Brzozowski and Fich [111] proved that a language is accepted by a partially ordered automaton if and only if Green's $\mathcal{R}$-relation [261] on its transition semigroup is the equality relation (this was also proved independently in [155, Thm. 5.1]). We obtain the $\mathcal{R}$-relation as the equivalence relation induced by the quasiorder $\leq_{\mathcal{R}}$ on $\mathcal{T}_S(\mathcal{A})$ given by

$$\delta_u \leq_{\mathcal{R}} \delta_v \Leftrightarrow \exists w \in \Sigma^*\ \forall q \in Q : \delta(q, u) = \delta(q, vw)$$

for $\mathcal{A} = (Q, \Sigma, \delta)$ and $u, v \in \Sigma^*$. Clearly, if v is a prefix of u, then $\delta_u \leq_{\mathcal{R}} \delta_v$. Furthermore, $\delta_u \leq_{\mathcal{R}} \delta_v$ implies $\delta(q, v) \preceq_R \delta(q, u)$ for every $q \in Q$. Consequently,

$$\delta_u \mathcal{R} \delta_v \Rightarrow \forall q \in Q\colon \delta(q, u) \simeq_R \delta(q, v). \tag{6.28}$$

It follows from Equation (6.28) that, for a partially ordered automaton, the $\mathcal{R}$-relation is the equality relation. Conversely, let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be such that the $\mathcal{R}$-relation on $\mathcal{T}_S(\mathcal{A})$ is the equality relation. Then the automaton $\mathcal{B} = (\mathcal{T}_S(\mathcal{A}), \Sigma, \mu, \delta_\varepsilon, E)$, where $\mu(\delta_u, a) = \delta_{ua}$ and $E = \{\delta_u \mid \delta_u(q_0) \in F\}$, is partially ordered and satisfies $L(\mathcal{A}) = L(\mathcal{B})$.[18] This gives a short proof of the characterization due to Brzozowski and Fich [111].

The notion of an upper set (for the reachability quasiorder) appears under the name *subautomaton* in [534]. It also appears in other contexts, for example in the notion of coverage of two automata introduced in [113]. As mentioned in the main text, the Rees congruence goes back to a related congruence introduced by Rees [505] for semigroups.

The notion of a sink component (and a corresponding dual notion) can actually be defined for any quasiordered set, and the results from Section 6.3 transfer to this more general setting. Furthermore, reachability is a property of the underlying automaton digraph, so the reachability order can also be defined in the context of digraphs. Similarly, the notions of strong and unilateral connectivity in digraphs correspond to these notions in automata, and a strongly connected component in an automaton is precisely a strongly connected component of its underlying automaton digraph. However, as we need these

[18] Alternatively, let $p, q \in Q$ satisfy $\delta(p, u) = q$ and $\delta(q, v) = p$ for some $u, v \in \Sigma^*$. Set $w = uv$. Since $\mathcal{A}$ is finite, $\delta_{w^i} = \delta_{w^{i+p}}$ for some $i \geq 0$ and $p > 0$. This implies $\delta_{w^i u} \mathcal{R} \delta_{w^i}$. By assumption, $\delta_{w^i u} = \delta_{w^i}$. Therefore, $q = \delta(p, (uv)^i u) = \delta(p, (uv)^i) = p$, showing that $\mathcal{A}$ is partially ordered. In fact, this proves that $\leq_{\mathcal{R}}$ is a partial order if and only if $\preceq_R$ is a partial order.

concepts only in the context of automata, we introduce them here for automata and rather than for general quasiorders and digraphs.

Chapter 7
Classes of Automata

Here, we discuss the following classes of automata that appear throughout the present work and collect results of relevance to us:

1. synchronizing automata (Section 7.1) and synchronizing automata with a trap state (Section 7.1.3),
2. unary automata (Section 7.2),
3. automata with one-point singular idempotents (OI-automata) and automata with simple idempotents (SI-automata) (Section 7.3),
4. permutation-reset automata (Section 7.4),
5. monotonic automata (Section 7.5),
6. automata whose strongly connected components are blocks (Section 7.6),
7. partially ordered automata (Section 7.7),
8. ρ-automata (Section 7.8),
9. commutative automata (Section 7.9),
10. solvable automata (Section 7.10), which generalize commutative and partially ordered automata.

See Figure 7.1 for a Hasse diagram of the inclusions that hold among these classes. The following content is new and did not appear before in the literature:

1. the novel class of ρ-automata,
2. the results on automata whose strongly connected components are blocks,
3. the structural results on solvable automata and the result that a solvable automaton admitting a word of rank r has a word of rank at most r and of length at most $n - r$ and
4. new structural results on solvable automata.

The remaining results presented are known and taken from the literature (maybe presented a little differently).

Other classes of automata, for example circular automata, completely reachable, completely distinguishable or sync-maximal automata are only used in Part III and so not introduced here but separately in Part III.

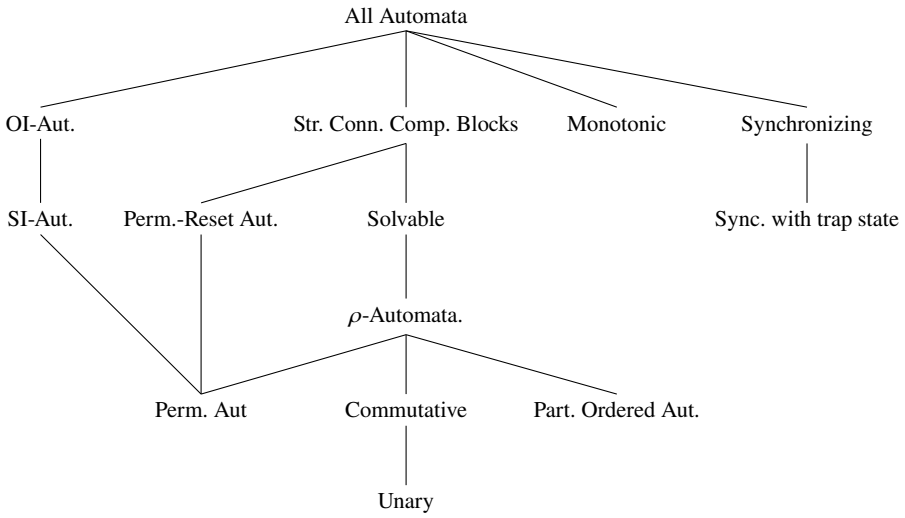


Fig. 7.1: Inclusion relations among the classes of automata from the present section.

7.1 Synchronizing Automata

Synchronizing automata were already defined in Section 1.1. Here, we collect further properties that we need in later chapters. Section 7.1.1 states that the set of synchronizing words is an ideal language. Section 7.1.3 investigates synchronizing automata with a trap state. Section 7.1.2 relates synchronizing words to subautomata, morphic images, quotient automata and vice versa. Section 7.1.4 investigates the set of maximal states in a synchronizing automaton. Section 7.1.5 expresses synchronization in terms of pairs of states. Lastly, Section 7.1.6 collects algorithmic results related to practical implementations (results from a more theoretical perspective in the framework of computational complexity theory are collected in Chapter 8 later).

7.1.1 Sets of Synchronizing Words and Ideal Languages

The set of synchronizing words of a semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$ is a regular ideal language: it is an ideal, since $\delta(Q, uwv) \subseteq \delta(\delta(Q, w), v)$ is a singleton whenever $\delta(Q, w)$ is, and it is regular, since $\mathrm{Syn}(\mathcal{A}) = L(\mathcal{P}(\mathcal{A})_{Q,\mathcal{F}})$ with $\mathcal{F} = \{\{q\} \mid q \in Q\}$.

Next, suppose $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is minimal and accepts a non-empty ideal language. Then, as observed by Maslennikova [440, 441],

$$L(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}). \tag{7.1}$$

The assumption $L(\mathcal{A}) \neq \emptyset$ cannot be dropped: the minimal automaton for $\emptyset$ is $(\{q\}, \Sigma, \delta, q, \emptyset)$ with $\delta(q, x) = q$, which is synchronizing with $\mathrm{Syn}(\mathcal{A}) = \Sigma^*$. Of course, $\emptyset$ is still the set of synchronizing words of any non-synchronizing semiautomaton, and so we obtain the following characterization.

Theorem 7.1 *Let $I \subseteq \Sigma^*$. Then I is a regular ideal language if and only if $I = \mathrm{Syn}(\mathcal{A})$ for some finite semiautomaton $\mathcal{A} = (Q, \Sigma, \delta)$.*

By Theorem 7.1, investigating acceptors for sets of synchronizing words, as we do in Part III, amounts to investigating acceptors for regular ideal languages. In the following, we use the fact that $\mathrm{Syn}(\mathcal{A})$ is an ideal language without further mention.

As a shortest word in a regular language accepted by an n-state automaton has length at most $n - 1$, Equation (7.1) yields the following bound.

Proposition 7.2 *If $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ is a minimal n-state automaton accepting an ideal language, then $\mathcal{A}$ is synchronizing with reset length at most $n - 1$.*

For $i \geq 1$, the minimal automaton for $\Sigma^* a^i \Sigma^*$ has $i + 1$ states and reset threshold i, so the bound is sharp for every $n \geq 1$.

7.1.2 Subautomata, Morphic Images, and Quotient Automata

Theorem 7.3 collects statements on how the set of synchronizing words related to subautomata, surjective morphic images, and quotient automata. Theorem 7.3, Item 4 is due to Rystsov [534, Prop. 2.3].

Proposition 7.3 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following is true.*

1. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{B})$ *if we have a surjective morphism $\varphi\colon Q \to S$ onto $\mathcal{B} = (S, \Sigma, \mu)$.*
2. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{A}/\!\sim)$ *for a $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on $\mathcal{A}$.*
3. $\mathrm{Syn}(\mathcal{A}) \subseteq \mathrm{Syn}(\mathcal{A}_{|S})$ *for an upper set S. So, if $\mathcal{A}$ is synchronizing, then $\mathcal{A}_{|S}$ is synchronizing.*
4. $\mathrm{Syn}(\mathcal{A}/\!\sim_S)\,\mathrm{Syn}(\mathcal{A}_{|S}) \subseteq \mathrm{Syn}(\mathcal{A})$ *for an upper set S (Rystsov [534, Prop. 2.3]).*

Proof. 1 Suppose $|\delta(Q, u)| = 1$ for $u \in \Sigma^*$. We have $\mu(S, u) = \mu(\varphi(Q), u) = \varphi(\delta(Q, u))$. So $|\mu(S, u)| = 1$.

2 Applying 1 to the canonical morphism for $\sim$ from Theorem 6.55 gives the claim.

3 The equation $\delta_{|S}(S, u) \subseteq \delta(Q, u)$ for $u \in \Sigma^*$ gives the claim.

4 Let $u \in \mathrm{Syn}(\mathcal{A}/\!\sim_S)$. The set S is a trap state in $\mathcal{A}/\!\sim_S$. So, this state must be the synchronizing state in $\mathcal{A}/\!\sim_S$, i.e., we have $\delta_{\sim_S}([q]_{\sim_S}, u) = S$ for all $q \in Q$, or equivalently $\delta(Q, u) \subseteq S$. Then for $v \in \mathrm{Syn}(\mathcal{A}_{|S})$ we get $\delta(Q, uv) = \delta(\delta(Q, u), v) \subseteq \delta(S, v)$ where $|\delta(S, v)| = 1$. Hence $uv \in \mathrm{Syn}(\mathcal{A})$. □

Corollary 7.4 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with a trap state and a $\mathcal{T}_S(\mathcal{A})$-congruence $\sim$ on its states such that $[t]_\sim = \{t\}$ for each trap state t. Then we have $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}/\!\sim)$.*

Proof. By Theorem 7.3, Item 2 we only have to prove that $\mathrm{Syn}(\mathcal{A}/\sim)$ is contained in $\mathrm{Syn}(\mathcal{A})$. So, let $u \in \mathrm{Syn}(\mathcal{A}/\sim)$. Let t be some trap state in $\mathcal{A}$. By Theorem 6.19, also $[t]_\sim$ is a trap state in $\mathcal{A}/\sim$. By Theorem 7.6, we have $\delta_\sim([q]_\sim, u) = [t]_\sim$ for all $q \in Q$, i.e., $\delta(q,u) \sim t$ for all $q \in Q$. Since $[t]_\sim = \{t\}$ this yields $\delta(q,u) = t$ for all $q \in Q$ and so $u \in \mathrm{Syn}(\mathcal{A})$. □

Corollary 7.5 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing n-state automaton all whose synchronizing words have length at least $n-1$ and $S \subseteq Q$ be an upper set. Then all synchronizing words of $\mathcal{A}_{|S}$ and $\mathcal{A}/\sim_S$ have length at least $|S|-1$ and $n-|S|$, respectively.*

Proof. If $\mathcal{A}_{|S}$ or $\mathcal{A}/\sim_S$ have a synchronizing word of shorter length, then by Theorem 7.3, Item 4 also $\mathcal{A}$ would have a synchronizing word of shorter length. □

7.1.3 Synchronizing Automata with a Trap State

Several classes of automata we investigate later have the property that a synchronizing state must be a trap state. Here, we collect statements about synchronizing automata with a trap state. First, Theorem 7.6 characterizes synchronizability for automata with a trap state. It also yields a sharp bound on the reset length of synchronizing automata with a trap state, first stated by Rystsov [526] (see also [534, Thm. 6.1]) and Babcsányi [52, Cor. 3]; we state this bound, together with a family of automata attaining it, in Theorem 7.7. Then, in Theorem 7.9, we state some consequences. For example, we characterize the synchronizing automata with a trap state as those automata possessing a unique maximal state with respect to the reachability order.

Theorem 7.6 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton with a trap state $t \in Q$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}$ is synchronizing with unique synchronizing state t.*
3. *$\forall q \in Q\ \exists u \in \Sigma^* \colon \delta(q,u) = t$, i.e., $q \preceq_R t$ for all $q \in Q$.*
4. *$\exists u \in \Sigma^* \colon |u| \leq \frac{n(n-1)}{2} \wedge (\forall q \in Q \colon \delta(q,u) = t)$.*

Proof. 1 $\Rightarrow$ 2: Let $\delta(Q,u) = \{q\}$ for some $u \in \Sigma^*$ and $q \in Q$. As t is a trap state, $t \in \delta(Q,u)$, so $t = q$. Hence $\{t\}$ is the set of synchronizing states.

2 $\Rightarrow$ 3: By assumption there exists $u \in \Sigma^*$ such that $\delta(q,u) = t$ for all $q \in Q$, i.e., $q \preceq_R t$ for all $q \in Q$.

3 $\Rightarrow$ 4: For $n = 1$ we have $Q = \{t\}$ and ε is a synchronizing word. Thus, assume $n > 1$ in the following.

We construct words u_i and sets $Q = Q_0, Q_1, \ldots$ containing t with strictly decreasing cardinalities as follows. Set $Q_0 = Q$. Given Q_i with $t \in Q_i$, stop if $Q_i = \{t\}$. Otherwise, since t is reachable from each state by Item 3, we can pick a shortest word u_{i+1} with $\delta(q_i, u_{i+1}) = t$ for some $q_i \in Q_i \setminus \{t\}$. Then set $Q_{i+1} = \delta(Q_i, u_{i+1})$.

Since t is a trap state, we have $\delta(t, u_{i+1}) = t = \delta(q_i, u_{i+1})$. So $t \in Q_{i+1}$ and $|Q_{i+1}| < |Q_i|$. Thus there exists $m \leq n-1$ such that $Q_m = \{t\}$.

Let $i \in \{0, 1, \ldots, m-1\}$ and let $q_i = p_0, p_1, \ldots, p_\ell = t$ be the path induced by u_{i+1}, where $\ell = |u_{i+1}|$. By the minimal choice of u_{i+1}, no p_r with $r \geq 1$ lies in $Q_i \setminus \{t\}$ and no p_r with $r < \ell$ equals t, and the p_r are pairwise distinct; otherwise a proper suffix, a proper prefix, or the word obtained by deleting a factor of u_{i+1} would take a state of $Q_i \setminus \{t\}$ to t. As $p_1, \ldots, p_{\ell-1}$ are thus $\ell - 1$ distinct states outside Q_i, of which there are exactly $n - |Q_i|$, we get $|u_{i+1}| \leq n - |Q_i| + 1$.

Set $u = u_1 \cdots u_m$. We have $\delta(Q, u) = \{t\}$. As $n = |Q_0| > |Q_1| > \cdots > |Q_{m-1}| \geq 2$, the numbers $n-|Q_{i-1}|+1$ for $i \in \{1, \ldots, m\}$ are pairwise distinct and lie in $\{1, \ldots, n-1\}$. Thus, we have

$$|u| = |u_1| + \cdots + |u_m| \leq \sum_{i=1}^{m} \big(n - |Q_{i-1}| + 1\big) \leq \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}.$$

$4 \Rightarrow 1$: This is obvious. □

Remark 7.7 By Theorem 7.6, every synchronizing n-state automaton with a trap state has reset length at most $\frac{n(n-1)}{2}$, and this bound is sharp: the automata from Figure 7.2 have reset length exactly $\frac{n(n-1)}{2}$.

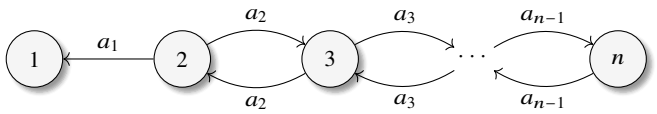


Fig. 7.2: Trap-state automata from [534] with reset length $\frac{n(n-1)}{2}$. The letter a_1 maps 2 to the trap state 1, and a_i interchanges i and $i+1$ for $i \geq 2$; self-loops are omitted. A shortest synchronizing word is $u = w_1 w_2 \cdots w_{n-1}$ where $w_i = a_i a_{i-1} \cdots a_1$ for $i \in \{1, \ldots, n-1\}$. Note that $\Sigma = \{a_1, a_2, \ldots, a_{n-1}\}$ grows with n.

Item 2 of Theorem 7.6 states that in a synchronizing automaton, a trap state is the unique synchronizing state.

Corollary 7.8 *In a synchronizing automaton, there exists at most one trap state.*

Proof. Let t, t' be trap states. By Item 2 of Theorem 7.6, each of them is the unique synchronizing state, so $t = t'$. □

Thus, by Theorem 7.6, an automaton with a trap state is synchronizing if and only if that trap state is reachable from every state. The trap state is then unique by Theorem 7.8. Note, however, that it is easy to find automata with a unique trap state that are not synchronizing.

Corollary 7.9 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing with a trap state.*
2. *There exists a trap state $t \in Q$ such that $q \preceq_R t$ for all $q \in Q$.*

3. *$Q/{\simeq_R}$ has a maximum, and this maximum is a singleton $\{t\}$ for some $t \in Q$.*
4. *$\mathcal{A}$ has a unique maximal state.*
5. *$\mathcal{A}$ has a unique synchronizing state.*

Proof. 1 $\Rightarrow$ 2: This follows from Item 3 of Theorem 7.6.

2 $\Rightarrow$ 3: Since t is a maximum, $[t]_{\simeq_R}$ is a maximum in $Q/{\simeq_R}$. Since t is a trap state, we have $[t]_{\simeq_R} = \{t\}$.

3 $\Rightarrow$ 4: Since $\{t\}$ is a maximum in $Q/{\simeq_R}$, t is maximal. Let $q \in Q$ be a maximal state. By assumption, $q \precsim_R t$. By maximality, $t \precsim_R q$. Thus $q \simeq_R t$, which gives $q = t$ by assumption.

4 $\Rightarrow$ 5: Let t be the unique maximal state. By Theorem 3.53, the set of maximal states $\{t\}$ is an upper set. Thus, t is a trap state. Since every state is below some maximal state by finiteness, and there is only one maximal state, we have $q \precsim_R t$ for each $q \in Q$. Thus, by Theorem 7.6, t is a unique synchronizing state.

5 $\Rightarrow$ 1: Let t be the unique synchronizing state. Then $\delta(Q, u) = \{t\}$ for some $u \in \Sigma^*$. Let $a \in \Sigma$. We have $|\delta(Q, ua)| = 1$. By assumption, $\delta(Q, ua) = \{t\}$. Thus $\delta(t, a) \in \delta(Q, ua) = \{t\}$, and so t is a trap state. □

7.1.4 Synchronizing Automata and the Reachability Order

Here, in Theorem 7.10, we first prove that the quotient automaton w.r.t. the set of maximal states is synchronizing. Then, in Theorem 7.11, we prove that an automaton is synchronizing if and only if the subautomaton on the set of maximal states is synchronizing and strongly connected. Further, in this case, the set of maximal states equals the set of synchronizing states. As an application, in Theorem 7.15, we show that if the Černý conjecture holds for strongly connected automata, then it holds for all automata.

Proposition 7.10 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states for $\precsim_R$. Then S is an upper set and $\mathcal{A}/{\sim_S}$ is synchronizing with trap state S.*

Proof. By Theorem 3.53, S is an upper set forming a trap state in $\mathcal{A}/{\sim_S}$. By Theorem 6.80, this trap state is reachable from every other state, and so the claim follows with Theorem 7.9, Item 2. □

Theorem 7.11 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be the set of maximal states for $\precsim_R$. Then S is an upper set and the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A}_{|S}$ is synchronizing.*
3. *$\mathcal{A}_{|S}$ is synchronizing and strongly connected (so that S is a sink component).*
4. *$\mathcal{A}$ is synchronizing and S equals the set of synchronizing states of $\mathcal{A}$.*

Proof. By Theorem 3.53, S is an upper set. Next, we show the stated equivalences.

1 $\Rightarrow$ 2: By Theorem 7.3, Item 3, if $\mathcal{A}$ is synchronizing, then $\mathcal{A}_{|S}$ is synchronizing.

$2 \Rightarrow 3$: By assumption $\delta(S,u) = \{q\}$ for some $q \in Q$ and $u \in \Sigma^*$. So $p \preceq_R q$ for all $p \in S$, which gives, as p is a maximal state, $p \simeq_R q$ for all $p \in S$. Hence S is strongly connected and as it is an upper set it is a sink component.

$3 \Rightarrow 4$: By Theorem 7.10 the automaton $\mathcal{A}/\sim_S$ is synchronizing. So, from Theorem 7.3, Item 4 we get that $\mathcal{A}$ is synchronizing. Let $q \in Q$ be such that $\delta(Q,u) = \{q\}$ for some $u \in \Sigma^*$. Then $p \preceq_R q$ for all $p \in Q$. So, q is maximal and we have $q \in S$. Furthermore, as S is a sink component, for $p \in S$ there exists $v \in \Sigma^*$ such that $\delta(q,v) = p$. Then $\delta(Q,uv) = \{p\}$ so that p is a synchronizing state. So we obtain that S equals the set of synchronizing states.

$4 \Rightarrow 1$: This is obvious. □

Corollary 7.12 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ and $q \in Q$. Then the following are equivalent.*

1. *q is synchronizing.*
2. *$\mathcal{A}$ is synchronizing and q is maximal w.r.t. $\preceq_R$.*
3. *$\mathcal{A}$ is synchronizing and q is a maximum w.r.t. $\preceq_R$.*

Theorem 7.11, Item 3 and Theorem 6.81 give the next corollary.

Corollary 7.13 *A synchronizing automaton has a unique sink component.*

Furthermore, Theorem 7.11 yields the following corollary (but this also follows from Theorem 6.4 stated earlier).

Corollary 7.14 *In an automaton every state is a synchronizing state if and only if it is synchronizing and strongly connected.*

As a nice application of Theorem 7.3 and Theorem 7.11, we prove that confirmation of the Černý conjecture for strongly connected automata implies it for all automata. This result appears in the literature in [487, Thm. 10], [130, Prop. 2], [530, Thm. 2], and in slightly more general form in [614, Prop. 2.1] and [52, Thm. 6].

Proposition 7.15 *The Černý conjecture is true for all automata if it is true for all strongly connected automata.*

Proof. Assume the Černý conjecture is true for strongly connected automata. Let $\mathcal{A} = (Q,\Sigma,\delta)$ be a synchronizing n-state automaton with set of synchronizing states $S \subseteq Q$. By Theorem 7.11 the set S equals the upper set of all maximal states and $\mathcal{A}_{|S}$ is synchronizing and strongly connected. So, by assumption, $\mathcal{A}_{|S}$ has a synchronizing word $v \in \Sigma^*$ of length at most $(|S|-1)^2$. By Theorem 7.10, $\mathcal{A}/\sim_S$ is synchronizing with trap state S. As $\mathcal{A}/\sim_S$ has $n - |S| + 1$ states, from Theorem 7.6, Item 4 we obtain that $\mathcal{A}/\sim_S$ has a synchronizing word $u \in \Sigma^*$ of length at most $\frac{(n-|S|+1)(n-|S|)}{2}$. By Theorem 7.3, Item 4, the word uv synchronizes $\mathcal{A}$ and

$$|uv| \leq \frac{(n-|S|+1)(n-|S|)}{2} + (|S|-1)^2 \leq (n-|S|)^2 + (|S|-1)^2$$
$$\leq (n-|S|+|S|-1)^2 = (n-1)^2. \square$$

Volkov [614, Prop. 2.1] (see also [21, Prop. 1]) gave the following generalization of the previous argument. Suppose we have a class of automata fulfilling the following two properties:

1. it is closed under taking subautomata and Rees quotient automata, and
2. the reset length of every n-state synchronizing automaton from this class that is strongly connected or has a trap state is bounded by $f(n)$ for some function $f\colon \mathbb{N}_0 \to \mathbb{N}_0$ with $f(n) + f(m) \leq f(n+m-1)$ for all $n, m \geq 1$.

Then, with the notation as in the proof of Theorem 7.15, $|uv| \leq f(n-|S|+1)+f(|S|) \leq f(n)$, and we obtain that every n-state synchronizing automaton in this class has a synchronizing word of length at most $f(n)$. Specifically, for the class of all automata, the function $f(n) = (n-1)^2$ fulfills both conditions.

7.1.5 The Pair Criterion

The pair criterion (Theorem 7.18), due to Černý [140, Thm. 2], states that an automaton is synchronizing if and only if all pairs of states can be mapped to a single state. Here, we deduce this important criterion from Theorem 7.6 and the observation, stated next, that the pair automaton from Section 6.5.6 is an automaton with a trap state having the same set of synchronizing states.

Proposition 7.16 (Rystsov [534, Prop. 5.2]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then we have $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$ where $F = \{\{q\} \mid q \in Q\}$. So, $\mathcal{A}$ is synchronizing if and only if its pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ is synchronizing.*

Proof. Suppose $u \in \mathrm{Syn}(\mathcal{A})$. Then $\delta(Q,u) = \{q\}$ for some $q \in Q$. Hence $\delta(\{p_1,p_2\},u) = \{q\}$ for all $p_1, p_2 \in Q$. So, u maps all states of $\mathcal{A}^{\leq 2}$ into F, giving $u \in \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$.

Conversely, let $u \in \mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right)$. Pick some $q \in Q$. Consider a state $p \in Q$. Since F is a trap state, it must be the unique synchronizing state and so $\delta_{\leq 2}(\{p,q\},u) \in F$. Thus $\delta(p,u) = \delta(q,u)$ and since p was chosen arbitrarily, we find $\delta(Q,u) = \{\delta(q,u)\}$ and so u synchronizes $\mathcal{A}$. □

We remark that it also easily follows that $\mathrm{Syn}(\mathcal{A}^{\leq 2}) = \mathrm{Syn}(\mathcal{A})$.

Remark 7.17 Since the pair automaton has $\binom{n}{2} + 1$ states, by Theorem 7.16, the length of a shortest synchronizing words is quadratically bounded in the number of states if and only if it is linearly bounded in the number of states of the pair automaton.

The pair automaton has F as a trap state. By Theorem 7.16 and by applying the quantifier-swapping for trap-state synchronizing automata stated in Theorem 7.6 3 and 4 to $\mathcal{A}^{\leq 2}/{\sim_F}$, we obtain the pair criterion, firstly stated by Černý [140, Thm. 2].

Theorem 7.18 (Pair Criterion, Černý [140, Thm. 2]) *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with non-empty Q is synchronizing if and only if for all pairs of states $p, q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(p,u) = \delta(q,u)$.*

Proof. By Theorem 7.16 we have $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$ if and only if $\mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right) \neq \emptyset$ where $F = \{\{q\} \mid q \in Q\}$. The pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ has F as a trap state. So, by Theorem 7.6, Item 3, we have $\mathrm{Syn}\left(\mathcal{A}^{\leq 2}/{\sim_F}\right) \neq \emptyset$ if and only if the trap state is reachable from every other state, i.e., for $p, q \in Q$ there exists $u \in \Sigma^*$ such that $\delta(\{p, q\}, u) \in F$, or equivalently $\delta(p, u) = \delta(q, u)$. □

Remark 7.19 In the language of transformation semigroup as discussed in Section 6.8, the pair criterion states that a transformation semigroup S on a finite set contains a constant map if and only if every two points can be mapped to a common point by some transformation from S. The pair automaton has a trap state, but note that in general, we have a quadratic lower bound for trap-state automata, see Figure 7.2.

The following equivalences follow readily from Theorem 7.18.

Corollary 7.20 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is synchronizing.*
2. *$\mathcal{A} \times \mathcal{A}$ is synchronizing.*
3. *$\mathcal{A}^{\leq 2}$ is synchronizing.*
4. *$\mathcal{A}^{\leq 2}/{\sim_F}$ with $F = \{\{q\} \mid q \in Q\}$ is synchronizing.*
5. *$(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $D = \{(q, q) \mid q \in Q\}$ is synchronizing.*

The pair criterion can be expressed in the following form using the *stability relation* $\sim_s$ introduced in [329, 358] where we set for $p, q \in Q$

$$p \sim_s q \Leftrightarrow \forall u \in \Sigma^* \; \exists v \in \Sigma^* \colon \delta(p, uv) = \delta(q, uv),$$

This is in fact a $\mathcal{T}_S(\mathcal{A})$-congruence.

Corollary 7.21 (Kari [358, 329]) *An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q \neq \emptyset$ is synchronizing if and only if the stability relation $\sim_s$ equals the universal relation.*

Later, we will also use a transformation semigroup variant of this relation. In this case, for a transformation semigroup S on a domain X and $a, b \in X$ we set $a \sim_s b$ if and only if for all $t \in S$ there exists $h \in S$ such that $h(t(a)) = h(t(b))$. Then S contains a constant transformation if and only if $\sim_s$ equals the universal relation (see also Section 6.8 for a discussion of the relation of transformation semigroups and automata).

7.1.6 Algorithmic Questions

First, in Theorems 7.22 and 7.23, we investigate the running time of the problem of deciding synchronizability for automata with a trap state. Then, Theorems 7.25 and 7.26 make algorithmic statements for general automata. Lastly, in Theorem 7.27, we prove that for synchronizing automata we can compute in linear time the set of synchronizing states. As a consequence, stated in Theorem 7.28, for general automata the set of synchronizing states can be computed in quadratic time.

Later, in Section 8.1, we will introduce a decision variant of the problem of computing a shortest synchronizing word. In general, this problem is NP-complete even for partially ordered binary automata, see Section 8.4.

Proposition 7.22 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O(|Q|)$ and space $O(1)$ whether $\mathcal{A}$ has a trap state, and also whether it has a unique trap state and compute this state if it exists.*

Proof. Iterating through all states and checking whether we find precisely one trap state can be done in the mentioned time and space bounds. □

By Theorem 7.6, Item 3 an automaton with a trap state is synchronizing precisely if this trap state is reachable from every other state, which gives the following time and space bounds to test synchronizability for automata with a trap state that were also noted by Martyugin [436, Prop. 3.2].

Theorem 7.23 (Martyugin [436, Prop. 3.2]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton with a trap state. Then we can test in time $O(|Q| + |Q| \cdot |\Sigma|)$ and space $O(|Q|)$ whether it is synchronizing.*

Proof. By Theorem 7.22 we can test whether the automaton has a unique trap, and if so compute it, in time $O(|Q|)$ and space $O(1)$. If this is the case, then checking whether there exists a path from every other state to this trap state can be done in time $O(|Q| + |\Sigma||Q|)$ and space $O(|Q|)$ with a breadth-first search by following the edges in the automaton graph in reverse order starting from the trap state (see, for example, the textbook [163] for an explanation of this algorithm and a statement of its resource requirements). □

Corollary 7.24 *For a class of automata with the property that a synchronizing state is a trap state, we can test whether an automaton from this class is synchronizing in time $O(|Q| + |Q| \cdot |\Sigma|)$ and space $O(|Q|)$.*

Proof. First test in linear time and constant space whether an input automaton has a trap state, which can be done in the stated time and space bounds by Theorem 7.22. If this is not the case, then it is not synchronizing by assumption Otherwise, by Theorem 7.23, we can test in the given time and space bounds whether it is synchronizing. □

By Theorem 7.16, $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}^{\leq 2}/{\sim_F}$ with $F = \{\{q\} \mid q \in Q\}$ is synchronizing. As the pair automaton $\mathcal{A}^{\leq 2}/{\sim_F}$ has the trap state F, $\frac{|Q|(|Q|-1)}{2} + 1$ states, and $\left(\frac{|Q|(|Q|-1)}{2} + 1\right) \cdot |\Sigma|$ transitions, to test synchronizability of $\mathcal{A}$ we obtain the following time and space bounds from Theorem 7.23.

Theorem 7.25 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O\left(|Q|^2 + |Q|^2 \cdot |\Sigma|\right)$ and space $O\left(|Q|^2\right)$ whether $\mathcal{A}$ is synchronizing.*

However, the algorithm behind Theorem 7.25 does not give us a synchronizing word. Natarajan [457, p. 138] has given a greedy algorithm to compute some synchronizing word of length at most $O(|Q|^3 \cdot |\Sigma|)$ in time $O(|Q|^4|\Sigma|)$. Eppstein [198, Thm. 5, 6 &

7] analyzed this algorithm further, optimized it to run in $O(|Q|^3 + |Q|^2 \cdot |\Sigma|)$ time and $O(|Q|^2 + |Q| \cdot |\Sigma|)$ space, and proved that some synchronizing word of length at most $O(|Q|^3)$ can be computed by it (see also the surveys by Sandberg [548, Thm. 1.15], Volkov [613, p. 16] and Kari & Volkov [360, p. 531]). Later, in Section 8.3.1, we will also prove that the deciding whether there exists a synchronizing word bounded by some given length-parameter is NP-complete.

Theorem 7.26 (Eppstein [198], Natarajan [457]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can test in time $O(|Q|^3 + |Q|^2 \cdot |\Sigma|)$ and (not counting the space for the output) working space $O(|Q|^2 + |Q| \cdot |\Sigma|)$ whether $\mathcal{A}$ is synchronizing and compute a synchronizing word of length $O(|Q|^3)$.*

Now, for a synchronizing automaton, we can compute the set of synchronizing states in linear time.

Theorem 7.27 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing automaton. Then we can compute in time $O(|Q| + |Q||\Sigma|)$ and space $O(|Q|)$ the set of synchronizing states.*

Proof. First, compute the strongly connected components, which can be done in the mentioned time and space bounds [163, 596]. Since $\mathcal{A}$ is synchronizing, it follows from Theorem 7.11 and Theorem 6.81 there exists a unique sink component, i.e., a strongly connected component with no outgoing transitions, which can be easily identified and coincides with the set of synchronizing states.

Alternatively, we can pick some state and start a breath-first search or a depth-first search in the automaton graph from this state. The first state q we encounter with no unvisited neighbors is a maximal state. Since $\mathcal{A}$ is synchronizing, $[q]_{\simeq_R}$ equals the set of synchronizing states by Theorem 7.11. □

For a synchronizing automaton with a trap state t, the set of synchronizing states equals $\{t\}$ and this set can be computed in linear time by Theorem 7.22.

Combining Theorem 7.25 and 7.27 gives the next result.

Corollary 7.28 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. We can compute in time $O\left(|Q|^2 \cdot |\Sigma|\right)$ and space $O\left(|Q|^2\right)$ the set of synchronizing states of $\mathcal{A}$.*

Proof. First, test whether $\mathcal{A}$ is synchronizing. If $\mathcal{A}$ is not synchronizing, then return the empty set. Otherwise, compute the set of synchronizing states and return this set. Combining the time and space bounds from Theorem 7.25 and 7.27 for these computations gives the claim. □

7.2 Unary Automata

The following tight bound on shortest words of rank r in unary automata is probably folklore. For completeness, we give a short proof.

Proposition 7.29 *Let $\mathcal{A} = (Q, \{a\}, \delta)$ be an n-state unary automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ with rank equal to r, and this bound is tight even in unary automata where r is the minimal rank among all words.*

Proof. Suppose we have $r = |\delta(Q, a^i)|$ for some $i \geq 0$. Then, by Theorem 6.73, there exists $0 \leq j \leq n - r$ such that $r = |\delta(Q, a^j)|$. Tightness for $1 \leq r < n$ is shown by $\mathcal{A}_n = (\{q_1, \ldots, q_n\}, \{a\}, \delta)$ with $\delta(q_i, a) = q_{\min\{i+1, n-r\}}$ for $i \in \{1, \ldots, n - r\}$ and $\delta(q_i, a) = q_i$ for $i \in \{n - r + 1, \ldots, n\}$ (see Figure 7.3). Alternatively, the automata family $\mathcal{B}_n = (\{1 \ldots, n\}, \{a\}, \delta)$ with $\delta(i, a) = \min\{i + 1, n\}$ has a^{n-r} as the shortest word of rank at most r with $\delta(\{1, \ldots, n\}, a^{n-r}) = \{n - r + 1, n - r + 2, \ldots, n\}$. □

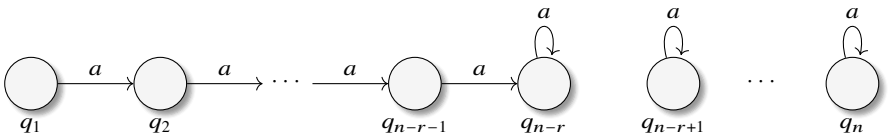


Fig. 7.3: Unary automaton realizing the lower bound from Proposition 7.29.

Remark 7.30 The unary automata $C_n = (\{1 \ldots, n\}, \{a\}, \delta)$ with $\delta(i, a) = \min\{i + 1, n\}$ would also show tightness of the bound from Theorem 7.29 for general unary automata since the shortest word of rank at most r in C_n is a^{n-r} with $\delta(\{1, \ldots, n\}, a^{n-r}) = \{n - r + 1, n - r + 2, \ldots, n\}$. Here r is not the minimal rank among all words for $r > 1$.

7.3 OI- and SI-Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$. We call $\mathcal{A}$ an *automaton with one-point singular idempotents (OI-automaton)* if every letter is either permutational or induces a one-point singular idempotent. We call $\mathcal{A}$ an *automaton with simple idempotents (SI-automaton)* if every letter is either permutational or a simple idempotent. Clearly, every SI-automaton is an OI-automaton.

For example, the Černý automata from Section 1.2 (Figure 1.7) are SI-automata. Rystsov [535] introduced SI-automata and proved that every synchronizing SI-automaton with n states admits a synchronizing word of length at most $2(n - 1)^2$. In **??**, we extend this bound to OI-automata.

For an OI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$, we set

$$\Sigma_I = \{a \in \Sigma \mid a \text{ induces a one-point singular idempotent }\},$$
$$\Sigma_G = \{a \in \Sigma \mid a \text{ is a permutational letter }\}.$$

Then $\Sigma = \Sigma_I \cup \Sigma_G$.

First, in Theorem 7.31, we classify synchronizing binary SI-automata, a result originally due to Martyugin [436, Prop. 6.2]. Theorem 7.32 gives synchronizing words for binary SI-automata. Theorem 7.33 establishes a lower bound on the rank of a reachable subset, and Theorem 7.34 establishes a lower bound on the length of a word of a given rank. Theorem 7.35 determines the length of shortest words reaching a given subset in SI-automata.

Proposition 7.31 (Martyugin [436, Prop. 6.2]) *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be an n-state binary synchronizing SI-automaton. Then we have $n \leq 2$ or one of the following holds:*

1. *$n = 3$ and $\mathcal{A}$ is, up to renaming of the letters, one of the automata in Figures 7.4a and 7.4b, or*
2. *one letter, say a, is a simple idempotent with $\delta(s, a) = t$ for distinct $s, t \in Q$, the letter b permutes the states with $Q = \{\delta(s, b^i) \mid i \geq 0\} \cup \{t\}$, and either*
   a. *t is a trap state (see Figure 7.4c), or*
   b. *$\delta(s, b^d) = t$ for some $0 < d < n$ with $\gcd(d, n) = 1$ (see Figure 7.4d).*[1]

Proof. Clearly, one letter, say a, must be a simple idempotent.

First, suppose a and b are both simple idempotents. If $n \geq 4$, then, since each letter moves only one state, there exist two distinct states $p, q \in Q$ that are fixed by both letters. Consequently, every word fixes them, and so no word synchronizes the automaton. Thus, we have $n \leq 3$. It is clear that for $n \leq 2$, $\mathcal{A}$ is always synchronizing. For $n = 3$, a simple case analysis yields that $\mathcal{A}$ is synchronizing if and only if it equals, up to renaming of the letters, one of the automata from Figures 7.4a and 7.4b.

Next, let b be a permutational letter. Let $s, t \in Q$ be distinct with $\delta(s, a) = t$. Set $S = \{\delta(s, b^i) \mid i \geq 0\}$ and $T = \{\delta(t, b^i) \mid i \geq 0\}$. As $\delta(S \cup T, x) \subseteq S \cup T$ and $\delta(Q \setminus (S \cup T), x) \subseteq Q \setminus (S \cup T)$ for $x \in \{a, b\}$, we have $Q = S \cup T$. If $s \notin T$, then $\delta(T, x) = T$ for all $x \in \Sigma$, which gives $T = \{t\}$ and that t is a trap state. Otherwise, $s \in T$ implies $S = T$. Then $\delta(s, b^d) = t$ for some $0 < d < n$. Set $r = \gcd(d, n)$ and, for $p, q \in Q$, write $p \sim q$ if there exists $i \geq 0$ such that $\delta(p, b^{ir}) = q$. For all $x \in \{a, b\}$ and $p, q \in Q$ we have

$$p \sim q \Leftrightarrow \delta(p, x) \sim \delta(q, x).$$

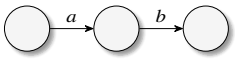
So $|\delta(\{p, q\}, u)| = 2$ for all $p, q \in Q$ with $p \nsim q$. Hence, as $\mathcal{A}$ is synchronizing, all states must be equivalent w.r.t. $\sim$, which implies $r = 1$. □

Next, we give synchronizing words for each of the two types of automata from Theorem 7.31.

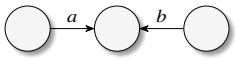
Proposition 7.32 *Let $n \geq 2$.*

1. *The word $(ab)^{n-2}a$ synchronizes the n-state SI-automaton from Figure 7.4c.*
2. *The word $(ab^{n-d})^{n-2}a$ synchronizes the n-state SI-automaton from Figure 7.4d.*

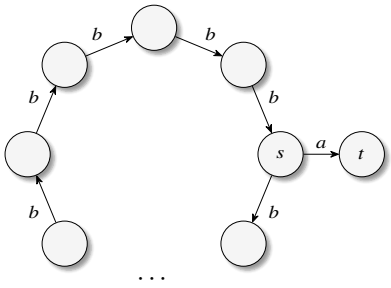
[1] In [436, Prop. 6.2] a different naming of the states is used, with a parameter $2 \leq p \leq n$ such that $p = n - d + 1$. There appears to be a minor mistake in [436, Prop. 6.2]. It is claimed that the automaton is not synchronizing whenever $\gcd(n, p) > 1$. However, if $p = n$ (which results in the Černý automaton), then $\gcd(n, p) = n$. The correct condition is $\gcd(n, n - p + 1) > 1$.



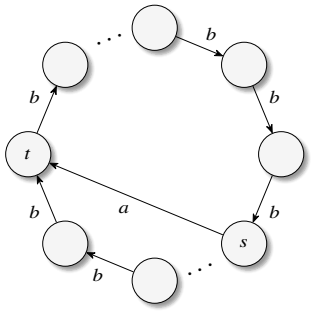
(a) Three states, with a trap state.



(b) Three states, both letters merging.



(c) Binary synchronizing SI-automaton with a trap state.



(d) Strongly connected binary synchronizing SI-automaton where $\delta(s, b^d) = t$.

Fig. 7.4: Synchronizing binary SI-automata (self-loops omitted) as listed in Theorem 7.31. Note that, in Figure 7.4d, for $d = 1$ we obtain the Černý family.

Proof. We use the notation from Theorem 7.31.

1: For $i \in \{1, \ldots, n-1\}$ we have

$$t = \delta(s, b^i(ab)^{(n-1)-i}a) = \delta(s, b^i(ab)^{n-2}a).$$

Since the state set is $\{s, \delta(s,b), \ldots, \delta(s, b^{n-2})\} \cup \{t\}$ and t is a trap state, we find $\delta(Q, (ab)^{n-2}a) = \{t\}$.

2: We can assume $n \geq 3$. For $i \in \{0, 1, \ldots, n-1\}$ set $Q_i = \{ s, \delta(s, b^d), \ldots, \delta(s, b^{id}) \}$. Since d and n are coprime, the smallest $x > 0$ with $xd \equiv 0 \pmod{n}$ is $x = n$. Hence the states $\delta(s, b^{jd})$ for $j \in \{0, 1, \ldots, n-1\}$ are pairwise distinct, so $|Q_i| = i + 1$ and $Q_{n-1} = Q$.

We show by induction on i that $Q_i \subseteq \delta^{-1}(s, (ab^{n-d})^i)$. This is easy to see for $i = 1$. Next, suppose $i > 1$. By induction hypothesis, we have $Q_{i-1} \subseteq \delta^{-1}(s, (ab^{n-d})^{i-1})$. For $j \in \{0, 1, \ldots, i-1\}$, since d and n are coprime, we have $(j+1)d \not\equiv 0 \pmod{n}$ and thus $\delta(s, b^{(j+1)d}) \neq s$. This gives $\delta(s, b^{(j+1)d}ab^{n-d}) = \delta(s, b^{jd})$. With $\delta(s, ab^{n-d}) = s$ we find $\delta(Q_i, ab^{n-d}) \subseteq Q_{i-1}$. Thus $Q_i \subseteq \delta^{-1}(s, (ab^{n-d})^i)$. This concludes our induction argument.

Now, set $q = \delta(s, b^{(n-1)d})$. We have $\delta(q, (ab^{n-d})^{n-2}) = \delta(s, b^d)$ and so

$$\delta(Q, (ab^{n-d})^{n-2}) = \delta(Q_{n-2} \cup \{q\}, (ab^{n-d})^{n-2}) = \{s, \delta(s, b^d)\}.$$

Thus, $\delta(Q, (ab^{n-d})^{n-2}a) = \{\delta(s, b^d)\}$. □

Further synchronizing words can be constructed. For example, for the automata from Figure 7.4c, all words in

$$\Sigma^* a^+ (b((ba^*)^{n-1})^*)a^+)^{n-2}\Sigma^*$$

synchronize it, or $abbabbabbabba$ synchronizes this automaton for $n = 6$. For the automata from Figure 7.4d all words in $\Sigma^* a^+((ba^*)^{n-p}(ba^*)^n a^+)^{n-2}\Sigma^*$ synchronize it.

Lemma 7.33 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton. Then for $S \subseteq Q$ and $u \in \Sigma^*$ we have $|S| - \sum_{a\in\Sigma_I} |u|_a \leq |\delta(S, u)|$.*

Proof. Clearly, the claim holds for $u = \varepsilon$. Next, let $u = wx$ with $x \in \Sigma$ and $w \in \Sigma^*$. Set $T = \delta(S, w)$. Inductively, we have

$$|S| - \sum_{a\in\Sigma_I} |w|_a \leq |T|. \tag{7.2}$$

For $x \in \Sigma_I$, since x maps only two states to one state, we have $|T| - 1 \leq |\delta(T, x)|$. Thus, with $\delta(S, u) = \delta(T, x)$ and Equation (7.2), we find

$$|S| - \sum_{a\in\Sigma_I} |u|_a = |S| - \left(\sum_{a\in\Sigma_I} |w|_a\right) - 1 \leq |T| - 1 \leq |\delta(S, u)|.$$

For $x \in \Sigma_G$, we have $|\delta(T, x)| = |T|$ and $\sum_{a\in\Sigma_I} |u|_a = \sum_{a\in\Sigma_I} |w|_a$ and substituting these values into Equation (7.2) we get the claim. □

Corollary 7.34 *In an n-state SI-automaton $\mathcal{A} = (Q, \Sigma, \delta)$ for a word u of rank m we have $n - m \leq |u|$.*

Proof. By Theorem 7.33 we have $n - \sum_{a\in\Sigma_I} |u|_a \leq m$. This gives $n - m \leq \sum_{a\in\Sigma_I} |u|_a \leq |u|$. □

Lastly, we note a bound on shortest words reaching a given subset of states in SI-automata without permutational letters.

Proposition 7.35 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an SI-automaton without permutational letters. Then for a shortest word $u \in \Sigma^*$ such that $\delta(Q, u) = S$ we have $|u| = |Q| - |S|$.*

Proof. We can assume $S \neq Q$. Let $u_1, \ldots, u_m \in \Sigma$ and $p_1, \ldots, p_m \in Q$ be such that $u = u_1 \cdots u_m$ and $\mathrm{excl}(u_i) = \{p_i\}$ for $i \in \{1, \ldots, m\}$. Next, by induction, we show that for $i \in \{1, \ldots, m\}$ we have

1. $\delta(Q, u_1 \cdots u_i) = Q \setminus \{p_1, \ldots, p_i\}$ and
2. the states $p_1, \ldots, p_i$ are pairwise distinct.

The claim is clear for $i = 1$. Now let $i > 1$. By the minimal choice of u, we have $p_i \in \delta(Q, u_1 \cdots u_{i-1})$ since otherwise the letter u_i fixes every state in $\delta(Q, u_1 \cdots u_{i-1})$ and thus can be omitted. Since, inductively, $\delta(Q, u_1 \cdots u_{i-1}) = Q \setminus \{p_1, \ldots, p_{i-1}\}$, we find $\delta(Q, u_1 \cdots u_i) = Q \setminus \{p_1, \ldots, p_i\}$ and $p_i \notin \{p_1, \ldots, p_{i-1}\}$. Since $p_1, \ldots, p_{i-1}$ are pairwise distinct by induction hypothesis, the states $p_1, \ldots, p_i$ are pairwise distinct.

Finally, for $i = m$ we find $|S| = |\delta(Q, u)| = |Q| - m$. Thus $m = |Q| - |S|$. □

7.4 Permutation-Reset Automata

Let $\mathcal{A} = (Q, \Sigma, \delta)$. We call $\mathcal{A}$ a *permutation-reset automaton* if $\delta(Q, a) \neq Q$ implies $|\delta(Q, a)| = 1$ for every $a \in \Sigma$, a *permutation automaton* if every letter is a permutational letter, and a *reset automaton* if every letter synchronizes the automaton. The automaton $\mathcal{A}$ is a permutation automaton if and only if $\mathcal{T}_S(\mathcal{A})$ is a permutation group on Q. Permutation automata are SI-automata without idempotent letters. Clearly, if $\mathcal{A}$ is a permutation-reset automaton, then we have

$$\mathrm{Syn}(\mathcal{A}) = \Sigma^* \cdot \{a \in \Sigma : |\delta(Q, a)| = 1\} \cdot \Sigma^* \tag{7.3}$$

Proposition 7.36 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then the following are equivalent.*

1. *$\mathcal{A}/{\sim_S}$ is a permutation-reset automaton*
2. *$\delta(Q \setminus S, a) \neq Q \setminus S$ implies $\delta(Q, a) \subseteq S$ for each $a \in \Sigma$.*
3. *$Q \setminus S$ is a block.*
4. *$\mathcal{A}_{|Q\setminus S,\Gamma}$ for $\Gamma = \{a \in \Sigma \mid \delta(Q \setminus S, a) \subseteq Q \setminus S\}$ is a permutation automaton and $a \in \Sigma \setminus \Gamma$ implies $\delta(Q \setminus S, a) \subseteq S$.*

Proof. $1 \Rightarrow 2$: By Theorem 6.61, the state set $Q/{\sim_S}$ is permuted by a letter $a \in \Sigma$ if and only if $\delta(Q \setminus S, a) = Q \setminus S$ and $\delta_{\sim_S}(S, a) = S$. So, $\delta(Q \setminus S, a) \neq Q \setminus S$ implies $|\delta_{\sim_S}(Q/{\sim_S}, a)| = 1$. Since S is a trap state in $\mathcal{A}/{\sim_S}$, this yields $\delta_{\sim_S}(Q/{\sim_S}, a) = \{S\}$, which is equivalent to $\delta(Q, a) \subseteq S$.

$2 \Rightarrow 3$: Since S is an upper set, $\delta(Q, a) \subseteq S$ is equivalent to $\delta(Q \setminus S, a) \subseteq S$. So, by assumption, we have $\delta(Q \setminus S, a) = Q \setminus S$ or $\delta(Q \setminus S, a) \subseteq S$, and the latter condition is equivalent to $\delta(Q \setminus S, a) \cap (Q \setminus S) = \emptyset$. Thus, $Q \setminus S$ is a block.

$3 \Rightarrow 4$: Since $Q \setminus S$ is a block, $\delta(Q \setminus S, a) \subseteq Q \setminus S$ implies $\delta(Q \setminus S, a) = Q \setminus S$, and so $\mathcal{A}_{|Q\setminus S,\Gamma}$ is a permutation automaton. For $a \in \Sigma \setminus \Gamma$, we have $\delta(Q \setminus S, a) \cap (Q \setminus S) = \emptyset$, which implies $\delta(Q \setminus S, a) \subseteq S$.

$4 \Rightarrow 1$: The assumption yields that $\delta(Q \setminus S, a) = Q \setminus S$ or $\delta(Q \setminus S, a) \subseteq S$ for each $a \in \Sigma$. Since S is an upper set, the latter condition is equivalent to $\delta(Q, a) \subseteq S$. From Theorem 6.61 we get that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. □

Remark 7.37 By Theorem 7.36, a strongly connected component $S \subseteq Q$ in $\mathcal{A} = (Q, \Sigma, \delta)$ is a block if and only if $(\mathcal{A}_{|T})/{\sim_{T\setminus S}}$ for $T = \uparrow S$ is a permutation-reset automaton.

By Theorem 7.36 in a permutation-reset automaton with a trap state t, by considering $S = \{t\}$, every non-permutational letter maps everything to the trap state.

The equivalences from Theorem 7.36 fail for $S = \emptyset$ in case of permutation-reset automata $\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$ and a letter $a \in \Sigma$ such that $|\delta(Q, a)| = 1$.

Proposition 7.38 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a permutation automaton. Then $\delta(q, u) \simeq_R q$ for all $q \in Q$ and $u \in \Sigma^*$.*

Proof. This follows from Theorem 6.6 since the whole state set is permuted by each letter. □

Corollary 7.39 *In a permutation automaton each strongly connected component is a sink component.*

By Theorem 7.38, the strongly connected components in a permutation automaton are precisely the orbits of the group $\mathcal{T}_S(\mathcal{A})$. Furthermore, for a permutation automaton the following are equivalent: strong connectivity, transitivity of the group of units of the transformation monoid, univalent connectedness, and the property of being cyclic.

Next, using Theorem 6.88, we prove the following minimality condition for permutation automata (the special cases $|F| = 1$ or $|F| = |Q| - 1$ were mentioned in [171, Cor. 3.8]).

Proposition 7.40 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a permutation automaton with $1 \leq |F| < |Q|$ such that $\gcd(|F|, |Q|) = 1$. Then $\mathcal{A}$ is minimal if and only if $\mathcal{A}$ is accessible.*

Proof. A non-accessible automaton is not minimal by Theorem 6.88. Next, assume $\mathcal{A}$ is accessible. Set $G = \mathcal{T}_S(\mathcal{A})$. We show that $\mathcal{A}$ admits no non-trivial G-congruence that saturates F, which gives minimality of $\mathcal{A}$ by Theorem 6.88, Item 5. Assume $\sim$ is some G-congruence that saturates F. Then F is a union of blocks. Let $\Delta \subseteq F$ be a block. As $\mathcal{A}$ is accessible, G is transitive. Then, by Theorem 5.4, F and Q are unions of equal-sized sets of the form $g(\Delta)$ for $g \in G$. Thus $|\Delta|$ divides $|F|$ and $|Q|$, and so it divides $\gcd(|F|, |Q|)$. By coprimality, we find $|\Delta| = 1$. So Δ is a trivial block and the only G-congruence saturating F is the equality relation. □

Corollary 7.41 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a permutation automaton with $1 \leq |F| < |Q|$ such that $\gcd(|F|, |Q|) = 1$. If $\mathcal{T}_S(\mathcal{A})$ contains a transitive permutation group G, then $\mathcal{A}$ is minimal.*

Proof. Let $\Gamma = \{x \in \Sigma \mid x \text{ is a permutational letter}\}$. Since $G \subseteq \mathcal{T}_S(\mathcal{A}_{|\Gamma})$, $\mathcal{T}_S(\mathcal{A}_{|\Gamma})$ is a transitive permutation group, which yields that $\mathcal{A}_{|\Gamma}$ is an accessible permutation automaton. So, by Theorem 7.40, $\mathcal{A}_{|\Gamma}$ is minimal. Since accessibility and distinguishability of pairs of states extend from $\mathcal{A}_{|\Gamma}$ to $\mathcal{A}$, also $\mathcal{A}$ is minimal by Theorem 6.88, Item 4. □

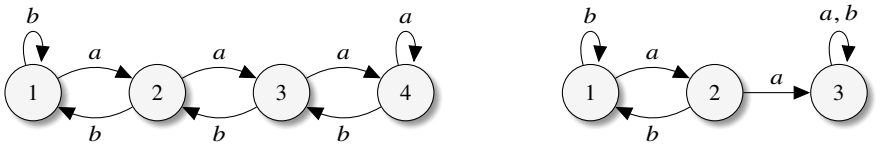
7.5 Monotonic Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *monotonic* if there exists a linear order $\leq$ on the states such that $p \leq q$ implies $\delta(p, x) \leq \delta(q, x)$ for all $p, q \in Q$ and $x \in \Sigma$. See Figure 7.5 for two example monotonic automata.

Such an order is generally not unique. For example, if all transitions are self-loops in an n-state automaton, $n!$ orderings are possible, and for each state, there exists an ordering in which it is either the minimal or the maximal element

Suppose $q = \delta(q, u^i)$ for $q \in Q$, $i \geq 1$ and $u \in \Sigma^*$ in a monotonic $\mathcal{A} = (Q, \Sigma, \delta)$. We have $q \leq \delta(q, u)$ or $\delta(q, u) \leq q$. In the first case, we get $q \leq \delta(q, u) \leq \delta(q, u^2) \leq \ldots \leq \delta(q, u^{i-1}) \leq q$, so that $q = \delta(q, u)$. A similar argument yields $q = \delta(q, u)$ for

$\delta(q, u) \leq q$. So, no word can permute a subset of states non-trivially.[2] In particular, no permutation automata is monotonic.



(a) Linear order on states is $1 < 2 < 3 < 4$

(b) Linear order on states is $1 < 2 < 3$

Fig. 7.5: Two synchronizing monotonic automata.

With respect to synchronizability Ananichev and Volkov [23] have established the following tight bound (tightness is shown by the automata from Figure 7.3, which are monotonic).

Proposition 7.42 (Ananichev and Volkov [23]) *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state monotonic automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ whose rank is at most r and this bound is tight.*

7.6 Automata Whose Strongly Connected Components are Blocks

In the present section, we investigate automata all of whose strongly connected components are blocks. These automata arose from the study of solvable automata, which we introduce later in Section 7.10. We observed that several properties of solvable automata extend to this more general class.

First, Theorem 7.43 gives a bound on the length of the shortest words of rank r. Theorem 7.45 gives bounds on the length of the shortest words mapping the entire state set into an upper set. Theorem 7.46 gives words ending in certain maximal states. We derive several consequences in Theorems 7.47 to 7.49. Theorem 7.50 characterizes maximal states. Lastly, in Theorem 7.51, we show that every synchronizing state is a trap state.

Theorem 7.43 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state automaton all of whose strongly connected components are blocks that has a word of rank r. Then there exists a word of length at most $n - r$ whose rank is at most r, and this bound is tight.*

Furthermore, let $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$. Then u^{n-r} has rank at most r, and $\delta(Q, u^{n-1})$ is the union of all sink components of $\mathcal{A}$.

Proof. Suppose $\mathcal{A}$ admits a word of rank r. We can assume $n > r$. By Proposition 6.77 there exists a sequence of upper sets

[2] Automata with this property are called counter-free or aperiodic [449].

$$S_1 \subsetneq \cdots \subsetneq S_m = Q \tag{7.4}$$

where S_1 equals the union of all sink components and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ are all the strongly connected components that are not sink components. Let $i \in \{2, \ldots, m\}$ and set $S = S_i \setminus S_{i-1}$. As S is not a sink component, we have $\delta(S, x_i) \setminus S \neq \emptyset$ for some $x_i \in \Sigma$, which gives $\delta(S, x_i) \cap S = \emptyset$ as S is a block. As S_i is an upper set, we have $\delta(S, x_i) \subseteq S_i = S_{i-1} \cup S$ and so $\delta(S, x_i) \subseteq S_{i-1}$. Hence, we find

$$\delta(S_i, x_i) = \delta(S_{i-1} \cup S, x_i) = \delta(S_{i-1}, x_i) \cup \delta(S, x_i) \subseteq S_{i-1}.$$

This also gives $\delta(S_i, u) \subseteq S_{i-1}$ for all $i \in \{2, \ldots, m\}$ since $\delta(S_i, a) \subseteq S_i$ and $\delta(S_{i-1}, a) \subseteq S_{i-1}$ for all $a \in \Sigma$. Now, inductively, from $\delta(S_i, x_i) \subseteq S_{i-1}$ and $\delta(S_i, u) \subseteq S_{i-1}$ for all $i \in \{2, \ldots, m\}$, we obtain that for all $i \in \{2, \ldots, m\}$ we have

$$\delta(Q, x_m \cdots x_i) \subseteq S_{i-1} \quad \text{and} \quad \delta(Q, u^{m-(i-1)}) \subseteq S_{i-1}.$$

Since the sequence from Equation (7.4) strictly decreases, we get $|S_i| \leq n - (m - i)$ for $i \in \{1, \ldots, m\}$. As S_1 is a union of sink components that are blocks, from Equation (6.3) we get $S_1 = \delta(S_1, u) \subseteq \delta(Q, u)$ for each $u \in \Sigma^*$, which gives $S_1 \subseteq \delta(Q, u^{n-1})$ and $|S_1| \leq r$. As $|S_1| \leq r$ there exists a maximal $j \in \{1, \ldots, m\}$ such that $|S_j| \leq r$. We have $j \neq m$ and $r < |S_{j+1}| \leq n - (m - (j + 1))$. So we get $m - j - 1 < n - r$. Now, as $\delta(Q, x_m \cdots x_{j+1}) \subseteq S_j$ and $\delta(Q, u^{m-j}) \subseteq S_j$ the words $x_m \cdots x_{j+1}$ and u^{m-j} of length $m - j \leq n - r$ and $|\Sigma|(m - j)$, respectively, have rank at most r, and the same follows for u^{n-r}. Lastly, $\delta(Q, u^{m-1}) \subseteq S_1$ gives $\delta(Q, u^{n-1}) \subseteq S_1$ as $m - 1 \leq n - 1$ and so $\delta(Q, u^{n-1}) = S_1$.

For tightness, consider the automata from the proof of Theorem 7.29. □

Remark 7.44 Theorem 6.79 can be used to give an inductive proof of Theorem 7.43. We can assume $\mathcal{A}$ admits a word $u \in \Sigma^*$ of rank r with $r < n$ for $n = |Q|$. If Q equals the set of maximal states, then Q equals a union of sink components. By assumption, these sink components are blocks. By Equation (6.3) each letter permutes each sink component. Thus, $\mathcal{A}$ is a permutation automaton. Now, for $r < |Q|$, this cannot be the case. Then Theorem 6.79 yields existence of a proper and non-empty upper set $S \subseteq Q$ such that $\mathcal{A}/{\sim_S}$ is strongly trap-connected. Since $|\delta(S, u)| \leq r$, inductively, there exists $v \in \Sigma^*$ with $|v| \leq \max\{|S| - r, 0\}$ such that $|\delta(S, v)| \leq r$. Furthermore, we have $\delta(Q \setminus S, x) \cap S \neq \emptyset$ for some $x \in \Sigma$ since $Q \setminus S$ is not a sink component by Theorem 6.65. Thus, as $Q \setminus S$ is a block, we find $\delta(Q \setminus S, x) \subseteq S$. Hence $\delta(Q, x) \subseteq S$. Then $|\delta(Q, xv)| \leq r$ and $|xv| \leq 1 + \max\{|S| - r, 0\} \leq n - r$. For $w \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$, we can also inductively suppose $|\delta(S, w^{\max\{|S|-r,0\}})| \leq r$ and, similarly as above, conclude $|\delta(Q, w^{n-r})| \leq r$.

Proposition 7.45 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton all of whose strongly connected components are blocks and $S \subseteq Q$ be an upper set. Suppose $\delta(Q, u) \subseteq S$ for some $u \in \Sigma^*$. Then there exists $v \in \Sigma^*$ with $\delta(Q, v) \subseteq S$ and $|v| \leq |Q| - |S|$, and this bound is tight.*

Proof. We may assume $Q \neq \emptyset$; then $\emptyset \neq \delta(Q, u) \subseteq S$, so $S \neq \emptyset$. We induct on $|Q|$.

If $S = Q$, then $v = \varepsilon$ works. So assume $S \subsetneq Q$. Then S is a non-empty proper upper set, so we may choose an inclusion-maximal proper non-empty upper set $T \subseteq Q$ and put $C := Q \setminus T$. By Theorems 3.39 and 3.42, C is a strongly connected component, hence a block by assumption.

As T is an upper set, $\mathcal{A}_{|T}$ is well defined and its strongly connected components are strongly connected components of $\mathcal{A}$, hence blocks. Moreover $T \cap S$ is an upper set of $\mathcal{A}_{|T}$ and $\delta(T, u) \subseteq T \cap \delta(Q, u) \subseteq T \cap S$. Since $|T| < |Q|$, induction gives $w \in \Sigma^*$ with $\delta(T, w) \subseteq T \cap S$ and $|w| \leq |T| - |T \cap S| = |T \setminus S|$.

Case 1: $C \subseteq S$. Then $Q \setminus S \subseteq T$, so $|w| \leq |T \setminus S| = |Q \setminus S| = |Q| - |S|$. As S is an upper set, $\delta(C, w) \subseteq S$, whence $\delta(Q, w) = \delta(C, w) \cup \delta(T, w) \subseteq S$. Take $v = w$.

Case 2: $C \not\subseteq S$. First, $\delta(C, a) \cap T \neq \emptyset$ for some $a \in \Sigma$: otherwise $\delta(C, a) \subseteq C$ for all $a \in \Sigma$, so C and hence $C \cap S$ are upper sets; the latter is non-empty because $\emptyset \neq \delta(C, u) \subseteq C \cap S$, and it is a proper subset of C, so $T \subsetneq T \cup (C \cap S) \subsetneq Q$ contradicts the maximality of T. Fix such an a. Since C is a block, $\delta(C, a) \cap C = \emptyset$, i.e. $\delta(C, a) \subseteq T$, and $\delta(T, a) \subseteq T$; thus $\delta(Q, a) \subseteq T$ and $\delta(Q, aw) \subseteq \delta(T, w) \subseteq S$. Finally $Q \setminus (S \cup T) = C \setminus S \neq \emptyset$, so

$$|Q| - |S| = |Q \setminus (S \cup T)| + |T \setminus S| \geq 1 + |T \setminus S| \geq 1 + |w| = |aw|.$$

Take $v = aw$.

Tightness follows from the automata in Figure 7.3 with $S = \{q_{n-r}, \ldots, q_n\}$. □

Theorem 7.45 might fail if S is not an upper set. For example, in the automaton from Theorem 7.30, the shortest word mapping all states into $\{1, n\}$ is a^{n-1}.

Proposition 7.46 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an automaton all of whose strongly connected components are blocks and $S \subseteq Q$. Then, for $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$ and $n \geq |Q| - 1$ we have*

$$\delta(S, u^n) \subseteq \{q \in \uparrow S \mid q \text{ is a maximal state } \}.$$

Proof. Let $q \in S$. Clearly, $\delta(q, u^n) \in \uparrow S$. By finiteness, we have $\delta(q, u^{i+p}) = \delta(q, u^i)$ for some $0 \leq i < |Q|$ and $p > 0$. Let T be the strongly connected component T with $\delta(q, u^i) \in T$. We have

$$\delta(q, u^{i+p}) \simeq_R \delta(q, u^{i+p-1}) \simeq_R \cdots \simeq_R \delta(q, u^i)$$

Thus $\delta(T, u) \cap T \neq \emptyset$. Since T is a block, we find $\delta(T, u) = T$. Because u contains each letter, this yields $\delta(T, a) = T$ for each $a \in \Sigma$ and so T is a sink component. Hence, $\delta(q, u^i)$ is maximal, and so $\delta(q, u^n)$ is maximal. □

The containment in Theorem 7.46 can be proper. For example, this is the case for $S = \{q_1\}$ and $u = ab$ in the automaton whose strongly connected components are blocks from Figure 7.7a

Corollary 7.47 *Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is an automaton all of whose strongly connected components are blocks. Let $S \subseteq Q$ and $R = \{q \in \uparrow S \mid q \text{ is a maximal state}\}$. Suppose $R \subseteq S$.*

1. *For $T \subseteq Q$ we have $\delta(S, u) \subseteq T$ for some $u \in \Sigma^*$ if and only if $R \subseteq T$.*

2. *For* $u \in \bigcap_{a\in\Sigma} \Sigma^* a \Sigma^*$ *and* $n \geq |Q| - 1$ *we have* $\delta(S, u^n) = R$.

Proof. 1: First, let $\delta(S, u) \subseteq T$ for some $u \in \Sigma^*$. For $R \subseteq S$, the set R is the union of the sink components contained in S. By Equation (6.3), $\delta(S, u)$ also contains these sink components. Thus R is contained in $\delta(S, u)$ and therefore in T.

Conversely, if $R \subseteq T$, by Theorem 7.46 there exists a word mapping Q into R, and thus into T.

2: Set $T = \delta(S, u^n)$. We have $\delta(S, u^n) \subseteq T$. Thus, by Item 1, we get $R \subseteq T$. By Theorem 7.46 we have $T \subseteq R$. □

Note that $R \subseteq S$ holds in Theorem 7.47 if S is an upper set.

Theorem 7.47, Item 1 generalizes the statement from Theorem 7.43 that words which contain each letter applied $|Q| - 1$ times map the whole state set onto the set of maximal states.

Corollary 7.48 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks. For* $S \subseteq Q$ *there exists* $u \in \Sigma^*$ *such that* $\delta(Q, u) \subseteq S$ *if and only if* $T \subseteq S$ *for each sink component* $T \subseteq Q$ *of* $\mathcal{A}$.

Proof. This follows from Theorem 7.47 applied to Q and S (since in this case, R equals the union of all sink components). □

Corollary 7.49 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks and* $S \subseteq Q$. *Suppose* $\delta(Q, u) \subseteq S$ *for some* $u \in \Sigma^*$. *Then there exists* $v \in \Sigma^*$ *with* $\delta(Q, v) \subseteq S$ *and* $|v| \leq |Q| - 1$.

Proof. Let R be the union of all sink components in $\mathcal{A}$. By Theorem 7.46 there exists a word mapping Q into R. Thus, by Theorem 7.45, we have $\delta(Q, u) \subseteq R$ for some $u \in \Sigma^*$ with $|u| \leq |Q| - |R| \leq |Q| - 1$. By Theorem 7.48, we have $R \subseteq S$ and so u maps Q into S. □

Next, we characterize the maximal states.

Proposition 7.50 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose strongly connected components are blocks. A state* $q \in Q$ *is contained in a sink component if and only if for all* $x \in \Sigma$ *we have* $\delta(q, x^i) = q$ *for some* $i \in \{1, \ldots, |Q|\}$.

Proof. Let $S \subseteq Q$ be the strongly connected component with $q \in S$. First, assume S is a sink component. By Equation (6.3), every letter $x \in \Sigma$ permutes S and so $\delta(q, x^i) = q$ for some $i \in \{1, \ldots, |S|\}$. Next, assume S is not a sink component. Then, as S is a block, we have $\delta(S, x) \cap S = \emptyset$ for some $x \in \Sigma$. As S is a strongly connected component, $\delta(q, x) \notin S$ yields $\delta(q, x^i) \notin S$, and so $\delta(q, x^i) \neq q$, for all $i \geq 1$. □

Proposition 7.51 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *be an automaton all of whose sink components are blocks. Then each synchronizing state is a trap state.*

Proof. By Theorem 7.11, the set of synchronizing states S is a sink component. Thus $\mathcal{A}_{|S}$ is a synchronizing permutation automaton. This implies $|S| = 1$. □

So, by Theorem 7.51 an automaton in which all sink components are blocks is synchronizable if and only if it has a unique trap state that is reachable from every other state.

7.7 Partially Ordered Automata

The automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *partially ordered* if $q \simeq_R p$ implies $p = q$, i.e., the reachability quasiorder is a partial order. This is equivalent to the fact that all cycles are induced by self-loops, i.e., $\delta(q, u) = q$ implies $\delta(q, v) = q$ for every factor v of u, in particular for the letters $v \in \Sigma$. Equivalently, the strongly connected components are singletons (and so also blocks). Hence, the subset mapping properties from Section 7.6 hold for partially ordered automata.

Here, we collect results for partially ordered automata that we need later and, in Theorem 7.58, characterizes synchronizable partially ordered automata as those with a unique trap state.

Example 7.52 Figure 7.6 depicts a partially ordered automaton with four states. This automaton is not monotonic. For assume $\leq$ is a linear order on Q. If $q_3 \leq q_4$ (resp. $q_4 \leq q_3$), then $q_2 = \delta(q_3, aab) \leq \delta(q_4, aab) = q_1$ (resp. $q_1 \leq q_2$) and $q_1 = \delta(q_3, bba) \leq \delta(q_4, bba) = q_2$ (resp. $q_2 \leq q_1$), so $q_1 = q_2$. However, since $q_1 \neq q_2$, $\mathcal{A}$ is not monotonic.

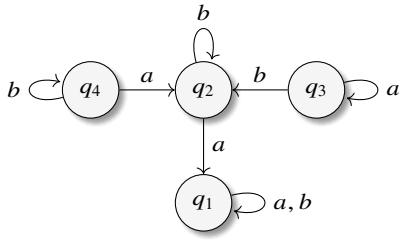


Fig. 7.6: Partially ordered automaton

Proposition 7.53 *In a partially ordered automaton $\mathcal{A} = (Q, \Sigma, \delta)$ a state is maximal for $\precsim_R$ if and only if it is a trap state.*[3] *In particular, there exists at least one trap state, and every upper set contains at least one trap state.*

Proof. Clearly, a trap state is maximal. Furthermore, for a maximal state $q \in Q$ and $a \in \Sigma$ we have $\delta(q, a) \simeq_R q$, so that $\delta(q, a) = q$. Hence, q is a trap state. □

Since trap states equal maximal states, the subset mapping conditions in terms of maximal states from Section 7.6 involve sets of trap states. For example, the following conditions follows from Theorem 7.48.

Proposition 7.54 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton and $S \subseteq Q$. Then $\delta(Q, u) \subseteq S$ for some $u \in \Sigma^*$ if and only if S contains all trap states of $\mathcal{A}$.*

[3] We note that this is only true for the model of complete automata considered in the first half of the present work, but not for the more general model of PDFAs introduced later in **??**.

Proposition 7.55 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be partially ordered. Then for $q \in Q$, $i \geq 0$, and $u \in \Sigma^*$ we have*

$$\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}}). \tag{7.5}$$

Proof. By finiteness, we have $\delta(q, u^{j+p}) = \delta(q, u^j)$ for some $0 \leq j < |Q|$ and some minimal $p > 0$. Since $\mathcal{A}$ is partially ordered, it is $p = 1$. Hence, for $i > |Q| - 1 \geq j$, we find $\delta(q, u^i) = \delta(q, u^{|Q|-1})$, which gives Equation (7.5). □

Proposition 7.56 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be partially ordered and $u \in \bigcap_{a \in \Sigma} \Sigma^* a \Sigma^*$. Then $\delta(q, u^{|Q|-1})$ is a trap state.*

Proof. This follows from Theorems 7.46 and 7.53. □

Proposition 7.57 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton and $q \in Q$. Then the following are equivalent.*

1. *The state q is a synchronizing state.*
2. *The state q is a unique trap state.*
3. *The state q is a maximum state in the partial order.*

Proof. $1 \Rightarrow 2$: By Theorem 7.9, q is a trap state. By Theorem 7.8, q is a unique trap state in $\mathcal{A}$.

$2 \Rightarrow 3$: By Theorem 7.53 the state q is the unique maximal state, i.e., is a maximum.

$3 \Rightarrow 1$: This follows from Theorem 7.9, Item 4. □

By Theorem 7.57, a state is a synchronizing state if and only if it is a unique trap state, giving the following corollary.

Corollary 7.58 *A partially ordered automaton is synchronizing if and only if it has a unique trap state.*

Comparing Theorem 7.58 with Theorem 7.9, we see that we do not have to state that the unique trap state is reachable from every other state, since this is automatically fulfilled.

By Theorem 7.53 the set of maximal states S equals the set of trap states in partially ordered $\mathcal{A}$. So, $\mathcal{A}_{|S}$ is synchronizing if and only if $|S| = 1$. As, by Theorem 7.11, Item 2, $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}_{|S}$ is synchronizing, we obtain another proof for Theorem 7.58.

Next, by Theorem 7.58 together with Theorem 7.22 we obtain the following time and space bounds on checking synchronizability, since we simply have to check existence of a unique trap state.

Proposition 7.59 *For n-state partially ordered automata we can test in time $O(n)$ and space $O(1)$ whether it is synchronizing.*

Later, in **??**, we will prove a statement implying that for a shortest $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$ we have $|u| \leq |S|(|Q| - 1)$.

Lemma 7.60 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered automaton. Then, for $q \in Q$, $i \geq 0$, and $u \in \Sigma^*$ we have $\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}})$.*

Proof. By finiteness, there exists such that $\delta(q, u^j) = \delta(q, u^l)$ for some $0 \leq j < |Q|$. We have $l \leq |Q|$. Now, set $p = l - j$. Then $\{\delta(q, u^j), \delta(q, u^{j+1}), \ldots, \delta(q, u^{j+p-1})\}$ is strongly connected, and so must be a singleton as $\mathcal{A}$ is partially ordered. Hence, by the minimal choice of l, we have $p = 1$. This gives $\delta(q, u^r) = \delta(q, u^{|Q|-1})$ for all $r \geq |Q| - 1$. So, we find $\delta(q, u^i) = \delta(q, u^{\min\{|Q|-1, i\}})$. □

Corollary 7.61 *Let $\Sigma = \{a_1, \ldots, a_k\}$ with $k = |\Sigma|$. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is a partially ordered commutative automaton. Then, for $n \geq |Q| - 1$, $q \in Q$, and $u \in \Sigma^*$ we have*

$$\delta(q, u) = \delta(q, a_1^{\min\{n, |u|_{a_1}\}} \cdots a_k^{\min\{n, |u|_{a_k}\}}). \tag{7.6}$$

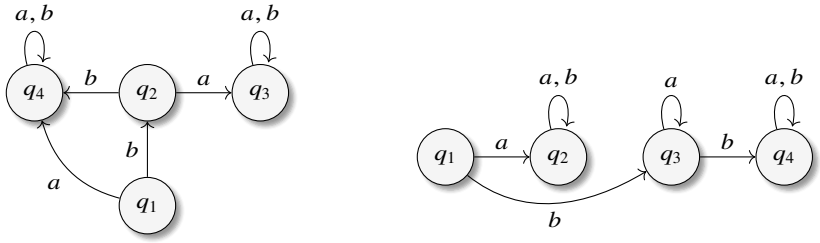
Proof. By commutativity, we have $\delta(q, u) = \delta(q, a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}})$. Then, for $i \in \{1, \ldots, k\}$, with Theorem 7.60, we find

$$\begin{aligned}
\delta(q, a_1^{|u|_{a_1}} \cdots a_k^{|u|_{a_k}}) &= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}} a_i^{|u|_{a_i}}) \\
&= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}} a_i^{\min\{n, |u|_{a_i}\}}) \\
&= \delta(q, a_1^{|u|_{a_1}} \cdots a_{i-1}^{|u|_{a_{i-1}}} a_i^{\min\{n, |u|_{a_i}\}} a_{i+1}^{|u|_{a_{i+1}}} \cdots a_k^{|u|_{a_k}}).
\end{aligned}$$

By applying the above reasoning successively to each $i \in \{1, \ldots, k\}$ we obtain the claim. □

7.8 ρ-Automata

In general, the equivalence relation $\simeq_R$ induced by the reachability quasiorder is not a $\mathcal{T}_S(\mathcal{A})$-congruence. For example, consider the automaton from Figure 7.5b where $1 \simeq_R 2$, but $\delta(1, a) \not\simeq_R \delta(2, a)$. An automaton $\mathcal{A}$ is said to be a *ρ-automaton* if $\simeq_R$ is a $\mathcal{T}_S(\mathcal{A})$-congruence. In this case, the strongly connected components are congruence classes. Trivially, every strongly connected automaton is a ρ-automaton.



(a) Here $q_1 \precsim_R q_2$, but $\delta(q_1, a) \not\precsim_R \delta(q_2, a)$. (b) Here $q_1 \precsim_R q_2$, but $\delta(q_1, b) \not\precsim_R \delta(q_2, b)$.

Fig. 7.7: Two ρ-automata where Equation (7.7) is not true and $\delta(q_1, a) \not\precsim_R q_1$.

First, we give two sufficient conditions to identify ρ-automata.

Proposition 7.62 *Let $\mathcal{A} = (Q, \Sigma, \delta)$.*

1. *Suppose, for all $p, q \in Q$ and $a \in \Sigma$, we have*

$$p \preceq_R q \Rightarrow \delta(p, a) \preceq_R \delta(q, a). \tag{7.7}$$

   Then $\mathcal{A}$ is a ρ-automaton.
2. *Suppose, for all $q \in Q$ and $a \in \Sigma$, we have $\delta(q, a) \preceq_R q$. Then $\mathcal{A}$ fulfills Equation* (7.7) *and so it is a ρ-automaton.*

Proof. 1: Let $a \in \Sigma$ and $p, q \in Q$. If $p \simeq_R q$, then $\delta(p, a) \simeq_R \delta(q, a)$. Hence Equation (6.8) holds and so $\simeq_R$ is a $\mathcal{T}_S(\mathcal{A})$-congruence.

2: Let $p, q \in Q$ with $p \preceq_R q$ and $a \in \Sigma$. Then, by assumption and Theorem 6.3, we have $\delta(p, a) \preceq_R p \preceq_R q \preceq_R \delta(q, a)$. Whence Equation (7.7) is true and the claim follows from Item 1. □

Figure 7.7 shows two ρ-automata where both conditions from Theorem 7.62 do not hold, so that they give proper subclasses of ρ-automata. Also, the classes of partially ordered and permutation automata form proper subclasses of ρ-automata.

Proposition 7.63 *Every partially ordered automaton is a ρ-automaton and every permutation automaton is a ρ-automaton.*

Proof. For a partially ordered automaton $\mathcal{A}$, the relation $\simeq_R$ equals the equality relation, which is a trivial $\mathcal{T}_S(\mathcal{A})$-congruence. By Theorem 7.38 and Item 2 of Theorem 7.62 we find that permutation automata are ρ-automata. □

The reachability order on $\mathcal{A}/\simeq_R$ equals the order induced on the $\simeq_R$-classes by the reachability order on $\mathcal{A}$. Since this is a partial order, we find that $\mathcal{A}/\simeq_R$ is a partially ordered automaton. See Figure 7.8 for an example.

Moreover, constructing $\mathcal{A}/\simeq_R$ amounts to contracting the strongly connected components of $\mathcal{A}$.[4] With Tarjan's algorithm [596] this can be done in time $O(|Q| + |\Sigma||Q|)$. We thus obtain the following statement.

Proposition 7.64 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a ρ-automaton. Then $\mathcal{A}/\simeq_R$ is a partially ordered automaton computable in $O(|Q| + |\Sigma||Q|)$.*

Next, we show that for ρ-automata with a trap state, the set of synchronizing words is preserved when considering the quotient automaton $\mathcal{A}/\simeq_R$.

Proposition 7.65 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a ρ-automaton with a trap state. Then* $\mathrm{Syn}(\mathcal{A}) = \mathrm{Syn}(\mathcal{A}/\simeq_R)$.

Proof. For the reachability order, we have $[t]_{\simeq_R} = \{t\}$ for each trap state t. Thus the claim follows from Theorem 7.4. □

The assumption that $\mathcal{A}$ has a trap state is crucial. For example, for the Černý family C_n (Figure 1.7) we have $\mathrm{Syn}(C_n) \neq \Sigma^*$ but $\mathrm{Syn}(C_n/\simeq_R) = \Sigma^*$.

[4] In graph theory, the graph obtained by contracting each strongly connected component to a single vertex is called the *condensation* of the graph.

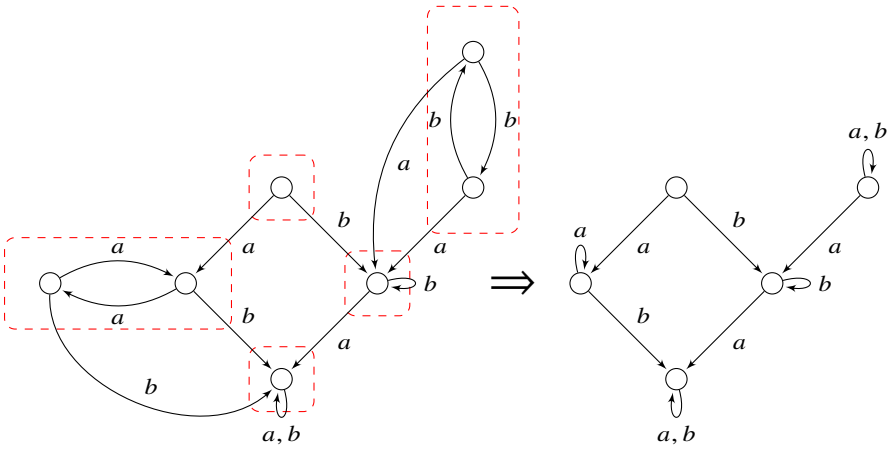


Fig. 7.8: Left: A ρ-automaton $\mathcal{A}$ (that is commutative and synchronizing). The strongly connected components of $\mathcal{A}$ are surrounded by boxes. Right: The automaton $\mathcal{A}/\simeq_R$.

Corollary 7.66 *Every synchronizing ρ-automaton $\mathcal{A}$ with n states and a trap state has a synchronizing word of length at most $n-1$, and this bound is tight.*

Proof. Since $\mathcal{A}/\simeq_R$ is partially ordered, its strongly connected components form blocks. Hence, by Theorem 7.43 with $r = 1$, there exists a synchronizing word of length at most $n-1$ in $\mathrm{Syn}(\mathcal{A}/\simeq_R)$. The result now follows from $\mathrm{Syn}(\mathcal{A}/\simeq_R) = \mathrm{Syn}(\mathcal{A})$, which holds by Theorem 7.65. □

Corollary 7.67 *A synchronizing n-state ρ-automaton $\mathcal{A}$ with a trap state all of whose synchronizing words have length at least $n-1$ is partially ordered.*

Proof. If it is not partially ordered, then $\mathcal{A}/\simeq_R$ has strictly less states. Since it has a trap state, it has a synchronizing word of length less than $n-1$ by Theorem 7.66, which also synchronizes $\mathcal{A}$ by Theorem 7.65. Thus, $\mathcal{A}$ must be partially ordered. □

7.9 Commutative Automata

An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *commutative* if $\delta(q, ab) = \delta(q, ba)$ for all $q \in Q$ and $a, b \in \Sigma$. Said differently, the mapping $q \mapsto \delta(q, x)$ is a morphism for $x \in \Sigma$ and so an automaton is commutative precisely if its transformation semigroup is contained in its endomorphism semigroup. Thus, the letters fulfill the properties from Section 6.4.3. For example, by Equation (6.5), we find that Equation (7.7) holds, and so commutative automata are ρ-automata, $q = \delta(q, u)$ for some $q \in Q$ and $u \in \Sigma^*$ implies $\delta(q, v) = \delta(\delta(q, v), u)$ for all $v \in \Sigma^*$, and, by Theorem 6.35, every strongly connected component is a block. Every unary automaton is commutative. See Figure 7.8 for two commutative

example automata. Clearly, for commutative $\mathcal{A}$, $L(\mathcal{A}_{q,F})$ is commutative for each $q \in Q$ and $F \subseteq Q$.

First, Section 7.9.1 we characterize the commutativity of Rees quotient automata. Then in Section 7.9.2 we investigate strongly connected commutative automata and give structural results on them and their endomorphisms. Section 7.9.3 gives some results on subset mapping and synchronization properties in commutative automata. Lastly, Section 7.9.4 characterizes those commutative automata with longest possible synchronizing words.

7.9.1 Commutativity of Rees Quotients

Proposition 7.68 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then the following are equivalent.*

1. *$\mathcal{A}/{\sim_S}$ is commutative.*
2. *$\delta(q, ab) = \delta(q, ba)$ for each $q \in Q \setminus S$ and $a, b \in \Sigma$ with $\delta(q, ab) \in Q \setminus S$.*
3. *$\delta(q, ab) \neq \delta(q, ba)$ implies $\{\delta(q, ab), \delta(q, ba)\} \subseteq S$ for all $q \in Q$ and $a, b \in \Sigma$.*

Proof. $1 \Rightarrow 2$: This directly follows from Theorem 6.61.

$2 \Rightarrow 3$: By Item 1, for $\{\delta(q, ab), \delta(q, ba)\} \not\subseteq S$, we have $\delta(q, ab) = \delta(q, ba)$, which gives the claim.

$3 \Rightarrow 1$: Since S is a trap state in $\mathcal{A}/{\sim_S}$, from Item 3 and Theorem 6.61 we obtain $\delta_{\sim_S}(\pi_{\sim_S}(p), ab) = \delta_{\sim_S}(\pi_{\sim_S}(q), ba)$ for all $q \in Q$ and $a, b \in \Sigma$, i.e., $\mathcal{A}/{\sim_S}$ is commutative. □

Corollary 7.69 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is a commutative permutation-reset automaton if and only if $\mathcal{A}_{|\Gamma, Q\setminus S}$ for $\Gamma = \{a \in \Sigma \mid \delta(Q \setminus S, a) = Q \setminus S\}$ is commutative and $\delta(Q \setminus S, a) \subseteq S$ for $a \in \Sigma \setminus \Gamma$.*

Proof. First, suppose $\mathcal{A}/{\sim_S}$ is a commutative permutation-reset automaton. Then clearly $\mathcal{A}_{|\Gamma, Q\setminus S}$ is commutative. By Theorem 7.36 we have $\delta(Q \setminus S, a) \subseteq S$ for $a \in \Sigma \setminus \Gamma$. Conversely, since each letter in Γ permutes $Q \setminus S$, $\mathcal{A}_{|\Gamma, Q\setminus S}$ is a permutation automaton. Thus, with Theorem 7.36, we find that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. Next, let $q \in Q \setminus S$ and $a, b \in \Sigma$. For $a, b \in \Gamma$, we have $\delta(q, ab) = \delta(q, ba)$ since $\mathcal{A}_{|\Gamma, Q\setminus S}$ is commutative. Next, without loss of generality, suppose $a \notin \Gamma$. Then $\delta(Q, a) \subseteq S$ and this gives $\{\delta(q, ab), \delta(q, ba)\} \subseteq S$. Thus, by Theorem 7.68, $\mathcal{A}/{\sim_S}$ is commutative. □

Corollary 7.70 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set such that $\mathcal{A}/{\sim_S}$ is commutative and $Q \setminus S$ is strongly connected. Then $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton (resp. permutation automaton for $S = \emptyset$).*

Proof. Since S is an upper set, $Q \setminus S$ is a strongly connected component. By Theorem 6.62, $\pi_{\sim_S}(Q \setminus S)$ is a strongly connected component in $\mathcal{A}/{\sim_S}$. Thus, $\pi_{\sim_S}(Q \setminus S)$ is a block by Theorem 6.35. Now, a set $R \subseteq Q \setminus S$ is a block in $\mathcal{A}$ if and only if $\pi_{\sim_S}(R)$ is a block in $\mathcal{A}/{\sim_S}$. Thus, $Q \setminus S$ is also a block in $\mathcal{A}$. For $S = \emptyset$, this yields that $\mathcal{A}/{\sim_S}$ is a permutation automaton and, by Theorem 7.36, we find that $\mathcal{A}/{\sim_S}$ is a permutation-reset automaton. □

7.9.2 Strongly Connected Automata

Proposition 7.71 *A commutative automaton that is strongly connected (resp. strongly trap-connected) is a permutation automaton (resp. permutation-reset automaton with a trap state).*

Proof. This follows from Theorem 7.70 applied with $S = \emptyset$ or with $S = \{t\}$ for a trap state. □

Corollary 7.72 *A strongly connected commutative automaton with at least two states is not synchronizing.*

Proof. This follows from Theorem 7.71 since a synchronizing permutation automaton has a single state. □

Proposition 7.73 *In a cyclic commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$, the endomorphism semigroup of $\mathcal{A}$ equals $\mathcal{T}_M(\mathcal{A})$.*

Proof. In a commutative automaton, every element of its transformation monoid is an endomorphism. By assumption, we have $Q = \uparrow\{q\}$ for some $q \in Q$. Now, let $\varphi\colon Q \to Q$ be an endomorphism. Then $\varphi(q) = \delta(q, u)$ for some $u \in \Sigma^*$. Consider some state $p \in Q$. We have $p = \delta(q, v)$ for some $v \in \Sigma^*$. This gives $\varphi(p) = \varphi(\delta(q, v)) = \delta(\varphi(q), v) = \delta(q, uv) = \delta(\delta(q, v), u) = \delta(p, u)$ and so $\varphi = \delta_u$. □

Corollary 7.74 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a strongly connected commutative automaton. Then $\mathcal{T}_M(\mathcal{A})$ is a regular permutation group and thus has size $|Q|$.*

Proof. Since $\mathcal{A}$ is strongly connected, its endomorphism semigroup equals it automorphism group by Theorem 6.37. By Theorem 6.43 the automorphism group is semiregular. By Theorem 7.73 it equals $\mathcal{T}_M(\mathcal{A})$. Since $\mathcal{A}$ is strongly connected, $\mathcal{T}_M(\mathcal{A})$ is transitive. Thus, $\mathcal{T}_M(\mathcal{A})$ is regular. □

Next, we give examples of commutative automata where the transformation monoid is properly contained in its endomorphism monoid.

Example 7.75 Let $\mathcal{A} = (Q, \{a_1, a_2\}, \delta)$ with $Q = \{q_{1,1}, q_{1,2}, q_{2,1}, q_{2,2}\}$ where, for $i \in \{1, 2\}$, we set $\delta(q_{i,1}, a_i) = \delta(q_{i,2}, a_i) = q_{i,2}$ and the remaining transitions are self-loops. Then $\mathcal{A}$ is commutative and $\varphi\colon Q \to Q$ with $\varphi(q_{i,1}) = \varphi(q_{i,2}) = q_{i,2}$ for $i \in \{1, 2\}$ is a morphism that is not contained in $\mathcal{T}_M(\mathcal{A})$.

Example 7.76 Let $\mathcal{A} = (\{q_{i,j} \mid i \in \{0, 1\},\ j \in [n]\}, \{a\}, \delta)$ be the semiautomaton with

$$\delta(q_{i,j}, a) = q_{i,(j+1) \bmod n}$$

for $i \in \{0, 1\}$ and $j \in [n]$. For every $0 \le s, t < n$, the map φ defined by

$$\varphi(q_{0,j}) = q_{1,(j+s) \bmod n}, \qquad \varphi(q_{1,j}) = q_{0,(j+t) \bmod n}$$

is an endomorphism since

$$\delta(\varphi(q_{0,j}), a) = q_{1,(j+s+1) \bmod n} = \varphi(q_{0,(j+1) \bmod n}) = \varphi(\delta(q_{0,j}, a)),$$

and similarly for the states $q_{1,j}$. None of these endomorphisms is induced by a word and thus none belongs to the transformation monoid.

Next, we determine the structure of binary commutative strongly connected automata. Let $n, m \geq 1$ and $r \in [n]$. Define automata $\mathcal{B}_{n,m,r} = ([n] \times [m], \{a, b\}, \mu_{n,m,r})$ where $\mu_{n,m,r}((i, j), a) = ((i + 1) \bmod n, j)$ and

$$\mu_{n,m,r}((i, j), b) = \begin{cases} (i, j + 1) & \text{if } 0 \leq j \leq m - 2, \\ ((i + r) \bmod n, 0) & \text{if } j = m - 1. \end{cases}$$

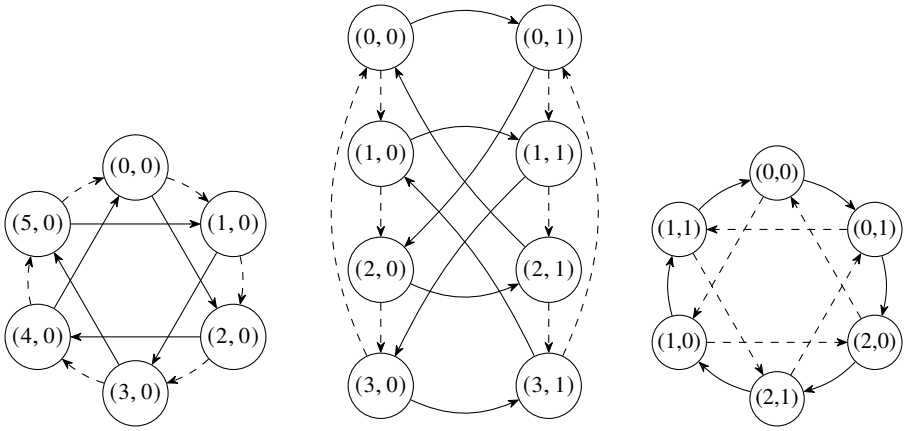
for $(i, j) \in [n] \times [m]$. These automata are commutative. See Figure 7.9 for some examples.

Remark 7.77 For $n, m \geq 2$, the automaton $\mathcal{B}_{n,m,1}$ looks like a grid where each component counts modulo n and m the number of a's and b's, respectively, and each state is uniquely determined by these numbers.

Remark 7.78 In $\mathcal{B}_{n,1,r}$ the letter a induces a single n-cycle and we have

$$\mu_{n,1,r}((i, 1), b) = \mu_{n,1,r}((i, 1), a^r), \quad i \in [n],$$

i.e., the letter b "wraps" around this cycle by skipping r steps on the cycle.



$\mathcal{B}_{6,1,2}$ $\mathcal{B}_{4,2,2}$ $\mathcal{B}_{3,2,2}$

Fig. 7.9: Automata $\mathcal{B}_{n,m,r}$ with a-transitions drawn dashed and b-transitions solid.

Remark 7.79 Suppose r is coprime to n, and let $q = (0, 0)$. Then there exists an integer l with $1 \leq l < n$ such that $rl \equiv 1 \pmod{n}$, i.e., l is the multiplicative inverse of r modulo n. We have the following properties:

1. The states
$$q, \delta(q, b), \ldots, \delta(q, b^{nm})$$
are pairwise distinct and form a single cycle.
2. Since $\delta(q, b^m) = \delta(q, a^r)$, we obtain
$$\delta(q, a) = \delta(q, a^{rl}) = \delta(q, b^{ml}).$$
Consequently, we have
$$\delta(q, b^i a) = \delta(q, ab^i) = \delta(q, b^{i+ml})$$
for all $i \geq 0$. In other words, the letter a acts on the b-cycle as a shift by ml positions.

By these properties, the automaton has the same structure as $\mathcal{B}_{nm,1,ml}$, with the roles of a and b interchanged.

Proposition 7.80 *Let $\mathcal{A} = (Q, \{a, b\}, \delta)$ be a commutative and strongly connected. Then $\mathcal{A}$ is isomorphic to $\mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$.*

Proof. By Theorem 7.71, $\mathcal{A}$ is a permutation automaton. Let $q \in Q$ and $n \geq 0$ be minimal such that $\delta(q, a^n) = q$. Then the set
$$S = \{q, \delta(q, a), \ldots, \delta(q, a^{n-1})\} \tag{7.8}$$
forms a strongly connected component in $\mathcal{A}_{\{a\}}$. The letter b acts as an isomorphism on this automaton. By Theorem 6.29 from Theorem 6.28 it permutes the strongly connected components of $\mathcal{A}_{\{a\}}$. Thus, we have $\delta(S, b^m) = S$ for some minimal $m \geq 1$.

Next, we show that
$$Q = S \cup \delta(S, b) \cup \cdots \cup \delta(S, b^{m-1}). \tag{7.9}$$
Let $p \in Q$. Since $\mathcal{A}$ is commutative and strongly connected, every state is reachable from q by some word from a^*b^*. Thus $p = \delta(q, a^i b^j)$ for some $i, j \geq 0$ where these numbers are picked such that j is minimal. Assume $j > m$. We have $\delta(q, a^i b^m) \in S$. By Equation (7.8), this yields $\delta(q, a^i b^m) = \delta(q, a^l)$ for some $0 \leq l < n$. Hence $p = \delta(q, a^l b^{j-m})$, contradicting the minimal choice of j. Thus, we have $j < m$. So, $p \in \delta(S, b^j)$ with $0 \leq j < m$.

Now, consider $\varphi\colon [n] \times [m] \to Q$ given by $\varphi(i, j) = \delta(q, a^i b^j)$ for $(i, j) \in [n] \times [m]$. By Equations (7.8) and (7.9) this map is surjective. Next, let $\varphi(i, j) = \varphi(i', j')$ for $(i, j), (i', j') \in [n] \times [m]$. Then $\varphi(i', j') \in \delta(S, b^j)$. Since the sets $S, \delta(S, b), \ldots, \delta(S, b^{m-1})$ are pairwise disjoint, we must have $j' = j$. Since b^j induces a permutation on the state set, we get $\delta(q, a^i) = \delta(q, a^{i'})$. Since $i, i' \in [n]$ and $q, \delta(q, a), \ldots, \delta(q, a^{n-1})$ are pairwise distinct, we find $i = i'$. Thus, φ is also injective.

Since $\delta(S, b^m) = S$ and $q \in S$, by Equation (7.8) there exists $r \in [n]$ such that
$$\delta(q, b^m) = \delta(q, a^r). \tag{7.10}$$
Next, we show that φ is a morphism of $\mathcal{A}$ and $\mathcal{B}_{n,m,r} = ([n] \times [m], \{a, b\}, \mu_{n,m,r})$. Let $(i, j) \in [n] \times [m]$. We have

$$\begin{aligned}
\delta(\varphi(i,j),a) &= \delta(\delta(q,a^i b^j),a) = \delta(q,a^{i+1}b) && \text{(Def. } \varphi \text{ and } \delta\text{)}\\
&= \varphi((i+1) \bmod n, j) && \text{(Eqn. (7.8) and Def. } \varphi\text{)}\\
&= \varphi(\mu_{n,m,r}((i,j),a)). && \text{(Def. } \mu_{n,m,r}\text{)}
\end{aligned}$$

For $j \leq m-2$, we similarly get $\delta(\varphi(i,j),b) = \varphi(\mu_{n,m,r}((i,j),b))$. Lastly, we have

$$\begin{aligned}
\delta(\varphi(i,m-1),b) &= \delta(q,a^i b^m) = \delta(q,b^m a^i) && \text{(Def. } \varphi \text{ and } \mathcal{A} \text{ comm.)}\\
&= \delta(q,a^{i+r}) && \text{(Eqn. (7.10))}\\
&= \varphi((i+r) \bmod n, 0) && \text{(Eqn. (7.8) and Def. } \varphi\text{)}\\
&= \varphi(\mu_{n,m,r}((i,m-1),b)). && \text{(Def. } \mu_{n,m,r}\text{)}
\end{aligned}$$

Thus φ is an isomorphism between $\mathcal{A}$ and $\mathcal{B}_{n,m,r}$. □

Corollary 7.81 *Let $\mathcal{A} = (Q, \{a,b\}, \delta)$ be a commutative permutation automaton and $S \subseteq Q$ be a strongly connected component. Then S is a sink component and $\mathcal{A}_{|S} \cong \mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$.*

Proof. By Theorem 7.39, S is a sink component. By Theorem 7.80, $\mathcal{A}_{|S}$ isomorphic to $\mathcal{B}_{n,m,r}$ for some $n, m \geq 1$ and $r \in [n]$. □

Remark 7.82 Let $\varphi\colon [n] \times [m] \to [n] \times [m]$ be an automorphism of $\mathcal{B}_{n,m,r}$. Then $\varphi((0,0)) = (s,t) = \mu_{n,m,r}((0,0), a^s b^t)$ for some $(s,t) \in [n] \times [m]$ and this value determines φ uniquely by Theorem 6.14. Thus, the automorphism group of $\mathcal{B}_{n,m,r}$ consists of all mappings $(i,j) \mapsto \mu((i,j), a^s b^t)$ for $(s,t) \in [n] \times [m]$. We see that this group equals $\mathcal{T}_M(\mathcal{B}_{n,m,r})$ and is regular. With Theorem 7.80 this gives another proof of Theorem 7.74

7.9.3 Subset Mapping Properties

We begin with some statements on synchronizing commutative automata.

Proposition 7.83 (Fernau and Hoffmann [212, Lem. 20]) *In a commutative automaton, a synchronizing state is a trap state.*

Proof. This follows from Theorem 7.51 since the sink components are blocks. □

Corollary 7.84 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative permutation-reset automaton that is not a permutation automaton. Then there exists a trap state $t \in Q$ such that for each $a \in \Sigma$ that is not a permutational letter we have $\delta(Q,a) = \{t\}$.*

Proof. By assumption, $\mathcal{A}$ has a letter mapping all states to the same state t. By Theorem 7.83, t is a trap state. Let $a \in \Sigma$ be some state that is not a permutational letter. Then $|\delta(Q,a)| = 1$ and, since t is a trap state, $t \in \delta(Q,a)$. This gives $\delta(Q,a) = \{t\}$. □

Proposition 7.85 *$\mathcal{A} = (Q, \Sigma, \delta)$ with $|Q| \geq 2$ is a synchronizing commutative permutation-reset automaton if and only if there exists a non-permutational letter, a trap state $t \in Q$, and $S = Q \setminus \{t\}$ is a block such that $\delta(q, ab) = \delta(q, ba)$ for all $q \in S$ and $a, b \in \Sigma$ with $\delta(q, ab) \in S$.*

Proof. First, suppose $\mathcal{A}$ is a synchronizing commutative permutation-reset automaton. By Theorem 7.84 we have a trap state t, a non-permutational letter, and for the upper set $S = \{t\}$ we have $\delta(Q, a) = Q$ or $\delta(Q, a) = \{t\}$ for each $a \in \Sigma$. Thus, S is a block. Since $\mathcal{A}$ is commutative, we have $\delta(q, ab) = \delta(q, ba)$ for all $q \in S$ and $a, b \in \Sigma$.

Conversely, by considering $\mathcal{A}/{\sim_{\{t\}}}$, which is isomorphic to $\mathcal{A}$, from Theorems 7.36 and 7.68 we find that it is a commutative permutation-reset automaton, which is synchronizing since it has a non-permutational letter mapping everything to t. □

Proposition 7.86 *A two-state automaton is commutative and synchronizing if and only if it has a trap state t, some non-identity letter, and every letter acts as the identity or maps both states to t.*

Proof. A synchronizing and commutative automaton has a trap state t by Theorem 7.83. Thus, each letter either fixes both states or maps both states to t. Conversely, an automaton of this form is easily seen to be synchronizing and commutative. □

Proposition 7.87 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative n-state automaton and $u \in \Sigma^*$.*

1. *We have $\delta(Q, u) = \delta(Q, v)$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Sigma$.*
2. *For $S \subseteq Q$, $|\delta(S, u)| = |\delta(S, v)|$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Sigma$.*

Proof. 1 Let $\Gamma \subseteq \Sigma$ be a largest alphabet such that, for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Gamma$ and $|v|_a = |u|_a$ for $a \in \Sigma \setminus \Gamma$, we have $\delta(Q, u) = \delta(Q, v)$. Since, by commutativity, $\Gamma = \emptyset$ fulfills this condition, such a set Γ exists.

Assume there exist $a \in \Sigma \setminus \Gamma$. Let $w \in \Sigma^*$ with $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, $|w|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$, and $|w|_a = 0$. By choice of Γ, we have $\delta(Q, u) = \delta(Q, a^{|u|_a} w)$. By Theorem 6.72 we find $\delta(Q, a^{|u|_a} w) = \delta(Q, a^{\min\{n-1, |u|_a\}} w)$. Set $w' = a^{\min\{n-1, |u|_a\}} w$. By commutativity, we have $\delta(Q, w') = \delta(Q, v)$ for all $v \in \Sigma^*$ with $|v|_b = |w'|_b$ for $b \in \Sigma$. Since $|w|_a = \min\{n-1, |u|_a\}$, $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, and $|w|_b = |u|_b$ otherwise, and $\delta(Q, u) = \delta(Q, v)$, this contradicts the maximal choice of Γ since the condition by which we selected it is also fulfilled for $\Gamma \cup \{a\}$. Thus, we must have $\Sigma = \Gamma$, which gives the claim.

2: Let $S \subseteq Q$. We show, by induction, that for each $\Gamma \subseteq \Sigma$ we have $|\delta(S, u)| = |\delta(S, v)|$ for all $v \in \Sigma^*$ with $|v|_a = \min\{n-1, |u|_a\}$ for $a \in \Gamma$ and $|v|_a = |u|_a$ for $a \in \Sigma \setminus \Gamma$. Then picking $\Gamma = \Sigma$ gives the statement we want to prove. The claim clearly holds for $\Gamma = \emptyset$.

Next, consider $a \in \Sigma \setminus \Gamma$. Let $v \in \Sigma^*$ with $|v|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma \cup \{a\}$ and $|v|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$. Choose $w \in \Sigma^*$ such that $|w|_b = \min\{n-1, |u|_b\}$ for $b \in \Gamma$, $|w|_b = |u|_b$ for $b \in \Sigma \setminus \Gamma$ with $b \neq a$ and $|w|_a = 0$. Then, by induction hypothesis, we have $|\delta(S, u)| = |\delta(S, wa^{|u|_a})|$. From Item 3 of Theorem 6.73, for $v' = wa^{\min\{n-1, |u|_a\}}$ we get

$$|\delta(S, wa^{|u|_a})| = |\delta(S, v')|.$$

We have $|v|_b = |v'|_b$ for all $b \in \Sigma$. Thus, by commutativity, $\delta(S, v) = \delta(S, v')$ and so $|\delta(S, u)| = |\delta(S, v)|$. This concludes the induction step. □

Corollary 7.88 *In a commutative n-state automaton, a shortest word reaching a given subset has length at most $|\Sigma|(n-1)$. The same bound holds for shortest words synchronizing a given subset in commutative automata.*

Note that Theorem 7.45 gives a better bound than $|\Sigma|(n-1)$ in commutative automata for words mapping the whole state set into a given subset.

Corollary 7.89 *Let $\Sigma = \{a_1, \ldots, a_k\}$. Suppose $\mathcal{A} = (Q, \Sigma, \delta)$ is a commutative n-state automaton and $u \in \Sigma^*$. Then we have*

1. $\delta(Q, a_1^{n-1} \cdots a_k^{n-1}) \subseteq \delta(Q, u)$ *and*
2. $|\delta(S, a_1^{n-1} \cdots a_k^{n-1})| \leq |\delta(S, u)|$ *for $S \subseteq Q$.*

Proof. Set $v = a_1^{\min\{n-1, |u|_{a_1}\}} \cdots a_k^{\min\{n-1, |u|_{a_k}\}}$ and $w = a_1^{n-1-|v|_{a_1}} \cdots a_k^{n-1-|v|_{a_k}}$. By Theorem 7.87 we have $\delta(Q, u) = \delta(Q, v)$ and $|\delta(S, u)| = |\delta(S, v)|$. Since $\delta(Q, w) \subseteq Q$ we have $\delta(Q, wv) \subseteq \delta(Q, v)$ and $|\delta(S, vw)| \leq |\delta(S, v)|$. By commutativity, we find $\delta(Q, wv) = \delta(Q, a_1^{n-1} \cdots a_k^{n-1})$ and $\delta(S, vw) = \delta(S, a_1^{n-1} \cdots a_k^{n-1})$, and this yields the claim. □

By Theorem 7.89, in a commutative n-state automaton we have $\delta(Q, u) \subseteq S$ (resp. $|\delta(S, u)| = 1$) for some $u \in \Sigma^*$ if and only if $\delta(Q, a_1^{n-1} \cdots a_k^{n-1}) \subseteq S$ (resp. $|\delta(S, a_1^{n-1} \cdots a_k^{n-1})| = 1$).

Because it easily follows from other results, we state a tight bound for shortest synchronizing words that was first obtained by Rystsov [533, Thm. 3.5] and [534, Thm. 3.2] (see also [212, Cor. 19] for another proof).

Proposition 7.90 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative and $n = |Q|$.*

1. $\mathrm{Syn}(\mathcal{A})$ *is commutative.*
2. $uwv \in \mathrm{Syn}(\mathcal{A})$ *for all $u, v, w \in \Sigma^*$ with $uv \in \mathrm{Syn}(\mathcal{A})$.*
3. *(Rystsov [533, 534]) A shortest word in* $\mathrm{Syn}(\mathcal{A})$ *has length at most $n-1$ and this bound is tight.*

Proof. 1: By commutativity, $\delta(Q, uabv) = \{q\} = \delta(Q, ubav)$ for all $u, v \in \Sigma^*$ and $a, b \in \Sigma$, which yields that $\mathrm{Syn}(\mathcal{A})$ is commutative.

2: Since $\mathrm{Syn}(\mathcal{A})$ is an ideal by Theorem 7.1, we have $uvw \in \mathrm{Syn}(\mathcal{A})$. By Item 1, we get $uwv \in \mathrm{Syn}(\mathcal{A})$.

3: Since in a commutative automaton the strongly connected components are blocks, this follows from Theorem 7.43 with $r = 1$, or from Theorem 7.45 applied $S = \{t\}$ where t is the synchronizing state, which must be a trap state so that S is an upper set. □

Next, in Theorem 7.94, we show that weak connectivity and existence of a trap state is sufficient for synchronizability in commutative automata. This follows from a couple of statements we give next.

Lemma 7.91 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative automaton with sink component $S \subseteq Q$ and $\Gamma \subseteq \Sigma$. Then the set $T = \{q \in Q \mid \exists u \in \Gamma^* \colon \delta(q, u) \in S\}$ is an upper set such that S is the only sink component having non-empty intersection with T.*

Proof. Let $q \in T$ and $b \in \Sigma$. We have $\delta(q, u) \in S$ for some $u \in \Gamma^*$. By commutativity, we find $\delta(q, bu) = \delta(q, ub)$. Since $\delta(q, u) \in S$ and S is a sink component we have $\delta(q, ub) \in S$. Thus, $\delta(q, bu) \in S$ and so $\delta(q, b) \in T$. Hence, T is an upper set.

Next, let $R \subseteq Q$ be a sink component with $R \cap T \neq \emptyset$. By the definition of T, we have $\delta(R, u) \cap S \neq \emptyset$ for some $u \in \Gamma^*$. Since R is a sink component, we have $\delta(R, u) \subseteq R$. Thus $R \cap S \neq \emptyset$. Since distinct sink components are disjoint, this gives $R = S$. □

Corollary 7.92 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative with sink component $S \subseteq Q$. Set $T = \{q \in Q \mid \exists u \in \Sigma^* \colon \delta(q, u) \in S\}$. Then T is a weakly connected component of the automaton graph of $\mathcal{A}$.*

Proof. Since the states in T are precisely those having a path leading into S, and S is strongly connected, T is weakly connected and thus contained in some weakly connected component. By the definition of T, there are no transitions leading into T since $\delta(q, a) \in T$ implies $q \in T$. Since T is an upper set by Theorem 7.91 there are also no transitions leading out of T since $\delta(q, a)$ with $q \in T$ implies $\delta(q, a) \in T$. Thus T equals the weakly connected component containing it, since otherwise there would either exist a transition going out of T or into T. □

Corollary 7.93 *A commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$ whose automaton graph is weakly connected has a unique sink component.*

Proof. Let $S \subseteq Q$ be a sink component and consider $T = \{q \in Q \mid \exists u \in \Sigma^* \colon \delta(q, u) \in S\}$. Since $\mathcal{A}$ is weakly connected, by Theorem 7.92, we find $Q = T$. From Theorem 7.91 we get that S is the only sink component in $\mathcal{A}$. □

Proposition 7.94 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative automaton. Then the following are equivalent.*

1. *$\mathcal{A}$ has a trap state and its automaton graph is weakly connected.*
2. *$\mathcal{A}$ has a trap state reachable from every other state.*
3. *$\mathcal{A}$ is synchronizing.*

Proof. 1 ⇒ 2: By Theorem 7.93 the trap state forms a unique sink component. Thus, it is reachable from every other state.

2 ⇒ 3: By Theorem 7.6, $\mathcal{A}$ is synchronizing.

3 ⇒ 1: By Theorem 7.112, a synchronizing state is a trap state. Clearly, a synchronizing automaton has an automaton graph that is weakly connected. □

Theorem 7.94 was stated in [210, Lem. 5.1].

7.9.4 Extremal Synchronizing Automata

Since commutative automata are ρ-automata, and every synchronizing commutative automaton has a trap state, Theorem 7.67 implies that every synchronizing commutative n-state automaton with reset length $n-1$ is partially ordered. We now determine the structure of those commutative automata that are extremal w.r.t. the reset length. Such automata were first classified in [212, Thm. 23]. Figure 7.10 shows a commutative automaton with the largest possible reset length.

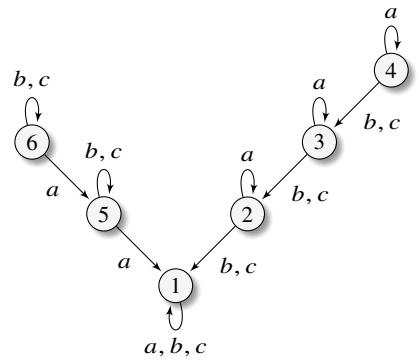


Fig. 7.10: Commutative six-state automaton with shortest synchronizing word $aabbb$.

Theorem 7.95 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a synchronizing commutative automaton with trap state t. Then the reset length of $\mathcal{A}$ equals $|Q| - 1$ if and only if we have a partition $\Gamma_1, \ldots, \Gamma_m$ of Σ and sets $T_{\Gamma_i} \subseteq Q$ such that*

1. *$T_{\Gamma_i} \cap T_{\Gamma_j} = \{t\}$ for $i \neq j$ and $Q = T_1 \cup \cdots \cup T_m$,*
2. *$T_{\Gamma_i} = \{q_{i,0}, \ldots, q_{i,m_i}\}$ for some $m_i \geq 0$,*
3. *$\delta(q_{i,j}, x) = q_{i,\min\{j+1,m_i\}}$ for $x \in \Gamma_i$, and*
4. *$\delta(q_{i,j}, x) = q_{i,j}$ for $x \in \Sigma \setminus \Gamma_i$.*

Proof. That automata of this form have reset length $|Q| - 1$ is clear.

Conversely, we do the proof only for a binary alphabet $\Sigma = \{a_1, a_2\}$, since for larger alphabets the arguments are essentially the same, but involve more indices and more complicated notation.

By Theorem 7.67, $\mathcal{A}$ is partially ordered. By Theorem 3.59 there exists a sequence of upper sets

$$S_1 \subsetneq S_2 \subsetneq \cdots \subsetneq S_m$$

such that $S_1 = \{t\}$ and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ is a singleton set since $\mathcal{A}$ is partially ordered. We show that the claim holds for each $\mathcal{A}_{|S_i}$ by induction.

For $i = 1$, the claim clearly holds for the single-state automaton $\mathcal{A}_{|S_1}$.

Next, let $i \geq 2$. We have $S_i \setminus S_{i-1} = \{q\}$ for some $q \in Q$. Without loss of generality, let $\delta(q, a_1) \neq q$. By Theorem 7.5, $\mathcal{A}_{|S_{i-1}}$ and $\mathcal{A}_{|S_i}$ have reset lengths $|S_{i-1}| - 1$ and

$|S_i| - 1$, respectively. Thus, by induction hypothesis, we either have $S_{i-1} = \{q_0, \ldots, q_l\}$ with $q_l = t$ and $\delta(q_j, x) = q_{\min\{j+1,l\}}$ for $j \in \{0, 1, \ldots, l\}$ for $x \in \Sigma$, or $S_i = \{q_{1,0}, \ldots, q_{1,l_1}, q_{2,0}, \ldots, q_{2,l_2}\}$ such that

1. $q_{l_1} = q_{l_2} = t$,
2. $\delta(q_{j,r}, a_j) = q_{j,\min\{r+1,l_j\}}$ for $j \in \{1, 2\}$ and $r \in \{0, 1, \ldots, l_j\}$,
3. $\delta(q_{j,r}, x) = q_{j,r}$ for $x \in \Sigma \setminus \{a_j\}$, $j \in \{1, 2\}$ and $r \in \{0, 1, \ldots, l_j\}$.

First, suppose the case $S_{i-1} = \{q_0, \ldots, q_l\}$ applies. If $\delta(q, a_1) \in \{q_1, \ldots, q_l\}$, then a_1^l synchronizes $\mathcal{A}_{|S_i}$, which is not possible since $l = |S_{i-1}| - 1 < |S_i| - 1$. Thus, $\delta(q, a_1) = q_0$. We also have $\delta(q, a_2) \notin \{q_1, \ldots, q_l\}$ since otherwise a_2^l synchronizes $\mathcal{A}_{|S_i}$. Next, $\delta(q, a_2) = q$ is excluded, since then $a_2 a_1^l$ ends at the state q_{l-1} and $a_1^l a_2$ ends at q_l, which contradicts commutativity of $\mathcal{A}$. Thus also $\delta(q, a_2) = q_0$. Then $\mathcal{A}_{|S_i}$ has the form as claimed.

Now, suppose the case $S_i = \{q_{1,0}, \ldots, q_{1,l_1}, q_{2,0}, \ldots, q_{2,l_2}\}$ applies. Assume $\delta(q, a_1) \in \{q_{2,0}, \ldots, q_{2,l_2}\}$ or $\delta(q, a_1) \in \{q_{1,1}, \ldots, q_{1,l_1}\}$. Then the word $a_1^{l_1} a_2^{l_2}$ also synchronizes $\mathcal{A}_{|S_i}$. Thus, we must have $\delta(q, a_1) = q_{1,0}$. Assume $\delta(q, a_2) \neq q$. Then $\delta(q, a_2 a_1) \neq \delta(q, a_1 a_2) = \delta(q, a_1)$, contradicting the commutativity of $\mathcal{A}$. Thus $\delta(q, a_2) = q$ and $\mathcal{A}_{|S_i}$ has the form as claimed.

Thus, we obtain the statement by induction. □

7.10 Solvable Automata

We call $\mathcal{A}$ *solvable* if $\delta(q, ab) \preceq_R q$ implies $\delta(q, ab) = \delta(q, ba)$ for all $q \in Q$ and $a, b \in \Sigma$. Clearly, every commutative automaton is solvable. Since for partially ordered automata, $\delta(q, ab) \preceq_R q$ implies $\delta(q, a) = \delta(q, b) = q$, and so $\delta(q, ab) = \delta(q, ba) = q$, partially ordered automata are also solvable. Clearly, commutative automata are solvable as well.

Here, we investigate solvable automata. In Section 7.10.1, we show that the class of solvable automata is closed under various operations. Section 7.10.2 gives other characterizations of solvability. In Section 7.10.3, we prove bounds on shortest words whose rank is bounded, and we give a technical result on words mapping one subset into another subset. Solvable automata were introduced by Rystsov [534] with a slightly different, more restricted, definition, we discuss this in Section 7.10.4.

7.10.1 Operations

Here, we prove that solvability is preserved in subautomata and morphic images, Rees quotient, product, and pair automata in Theorem 7.96, Theorem 7.97, Theorem 7.98, Theorem 7.99, and Theorem 7.101, respectively. Theorem 7.100 proves that solvability of $\mathcal{A}$, $\mathcal{A} \times \mathcal{A}$, and $\mathcal{A}^{\leq 2}$ are equivalent properties, and in Theorem 7.101 that solvability of $\mathcal{A}$ implies solvability of $\mathcal{A}^{\leq 2}/\sim_F$ and $(\mathcal{A} \times \mathcal{A})/\sim_D$ with $F = \{\{q\} \mid q \in Q\}$ and

$D = \{(q,q) \mid q \in Q\}$. In Theorem 7.102 we show that for permutation automata, solvability of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ implies it for $\mathcal{A}$.

Proposition 7.96 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable automaton and $S \subseteq Q$ be an upper set. Then $\mathcal{A}_{|S}$ is solvable.*

Proof. Suppose $\delta_{|S}(q, ab) \preceq_R q$ for some $q \in S$ and $a, b \in \Sigma$. Then $\delta(q, ab) \preceq_R q$ and, by solvability of $\mathcal{A}$, we have $\delta_{|S}(q, ab) = \delta(q, ab) = \delta(q, ba) = \delta_{|S}(q, ba)$. Hence, $\mathcal{A}_{|S}$ is solvable. □

Proposition 7.97 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $\mathcal{B} = (S, \Sigma, \mu)$. Suppose there exists a surjective morphism $\varphi\colon Q \to S$. Then solvability of $\mathcal{A}$ implies solvability of $\mathcal{B}$.*

Proof. Suppose we have $\mu(\varphi(q), abu) = \varphi(q)$ for some $u \in \Sigma^*$. Then it is $\varphi(\delta(q, (abu)^i)) = \varphi(q)$ for all $i \geq 0$. By finiteness, there exist $i \geq 0$ and $p > 0$ such that $\delta(q, (abu)^{i+p}) = \delta(q, (abu)^i)$. Hence $\delta(q, (abu)^i ab) \preceq_R \delta(q, (abu)^i)$. Since $\mathcal{A}$ is solvable, we find $\delta(q, (abu)^i ab) = \delta(q, (abu)^i ba)$. Then

$$\begin{aligned}\mu(\varphi(q), ab) &= \mu(\varphi(\delta(q, (abu)^i)), ab) = \varphi(\delta(q, (abu)^i ab)) \\ &= \varphi(\delta(q, (abu)^i ba)) = \mu(\varphi(\delta(q, (abu)^i)), ba) = \mu(\varphi(q), ba).\end{aligned}$$

Hence, $\mathcal{B}$ is solvable. □

By Theorem 7.97 the class of solvable automata is closed under isomorphisms.

Corollary 7.98 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable automaton and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is solvable.*

Proof. The canonical morphism for $\sim_S$ from Theorem 6.55 is surjective and maps $\mathcal{A}$ onto $\mathcal{A}/{\sim_S}$. So the claim follows from Theorem 7.97. □

Proposition 7.99 *Let $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1)$ and $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2)$. Then $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable if and only if $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable.*

Proof. First, assume $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable. Since the maps $(p, q) \mapsto p$ and $(p, q) \mapsto q$ are surjective morphisms from $\mathcal{A}_1 \times \mathcal{A}_2$ to $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively, we find that $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable by Theorem 7.97.

Now, assume $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable. Let $(p, q) \in Q_1 \times Q_2$ and $a, b \in \Sigma$ be such that $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) \preceq_R (p, q)$. This gives $\delta_1(p, ab) \preceq_R p$ and $\delta_2(q, ab) \preceq_R q$.[5] Since $\mathcal{A}_1$ and $\mathcal{A}_2$ are solvable, we find $\delta_1(q, ab) = \delta_1(q, ba)$ and $\delta_2(p, ab) = \delta_2(q, ba)$. Thus, we get $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) = \delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ba)$ from the definition of $\mathcal{A}_1 \times \mathcal{A}_2$. □

Corollary 7.100 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $i \geq 2$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*

[5] Note that $\delta_1(p, ab) \preceq_R p$ and $\delta_2(q, ab) \preceq_R q$ not necessarily imply $\delta_{\mathcal{A}_1 \times \mathcal{A}_2}((p, q), ab) \preceq_R (p, q)$.

2. $\mathcal{A} \times \cdots \times \mathcal{A}$ *(i times) is solvable.*
3. $\mathcal{A} \times \mathcal{A}$ *is solvable.*
4. $\mathcal{A}^{\leq 2}$ *is solvable.*

Proof. $1 \Rightarrow 2$: This follows from Theorem 7.99 applied i times.

$2 \Rightarrow 3$: We can assume $i > 2$. Then

$$\mathcal{A} \times \cdots \times \mathcal{A} = \underbrace{(\mathcal{A} \times \cdots \times \mathcal{A})}_{i-2 \text{ times}} \times (\mathcal{A} \times \mathcal{A}).$$

By Theorem 7.99 we obtain that $\mathcal{A} \times \mathcal{A}$ is solvable.

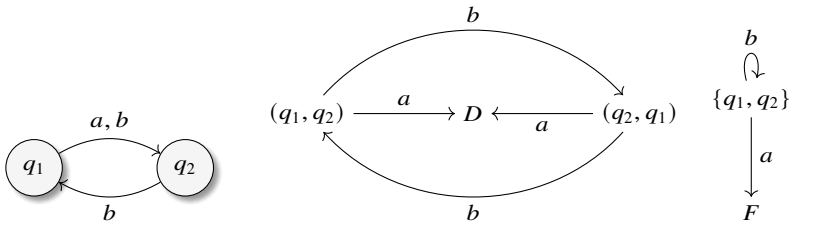
$3 \Rightarrow 4$: We have $\mathcal{A}^{\leq 2} \cong (\mathcal{A} \times \mathcal{A})/{\sim}$ for the $\mathcal{T}_S(\mathcal{A} \times \mathcal{A})$-congruence $\sim$ given by $(p_1, p_2) \sim (q_1, q_2)$ if and only if $\{p_1, p_2\} = \{q_1, q_2\}$. So, the claim follows from Theorem 7.98.

$4 \Rightarrow 1$: As $(\mathcal{A}^{\leq 2})_{|F} \cong \mathcal{A}$ with $F = \{\{q\} \mid q \in F\}$, the claim follows from Theorem 7.98. □

Corollary 7.101 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be solvable. Then the automata $\mathcal{A}^{\leq 2}/{\sim_F}$ and $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $F = \{\{q\} \mid q \in Q\}$ and $D = \{(q, q) \mid q \in Q\}$ are solvable.*

Proof. For solvable $\mathcal{A} = (Q, \Sigma, \delta)$, $\mathcal{A}^{\leq 2}$ and $\mathcal{A} \times \mathcal{A}$ are solvable by Theorem 7.100. So, Theorem 7.98 gives the claim. □

The converse statement to Theorem 7.101 does not hold in general, as the example from Figure 7.11 shows.



(a) The automaton $\mathcal{A}$. (b) $(\mathcal{A} \times \mathcal{A})/{\sim_D}$, which is solvable. (c) $\mathcal{A}^{\leq 2}/{\sim_F}$

Fig. 7.11: The automaton $\mathcal{A}$ is not solvable, but $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ and $\mathcal{A}^{\leq 2}/{\sim_F}$ with $D = \{(q, q) \mid q \in Q\}$ and $F = \{\{q\} \mid q \in Q\}$ are solvable. Self-loops omitted.

However, for permutation automata, a converse of Theorem 7.101 holds for $(\mathcal{A} \times \mathcal{A})/{\sim_D}$.

Proposition 7.102 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a permutation automaton. Then solvability of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ with $D = \{(q, q) \mid q \in Q\}$ implies that $\mathcal{A}$ is solvable.*

Proof. We can assume $|Q| \geq 2$. Clearly, $\mathcal{A}$ is solvable if each subautomaton of the form $\mathcal{A}_{|\uparrow\{q\}}$ for each $q \in Q$ is solvable. Thus, we can assume $Q = \uparrow\{q\}$ for some $q \in Q$ in

the following. Let $p \in Q$ be a state with $p \neq q$. Consider the upper set $S = \uparrow\{(p, q)\}$ in $\mathcal{A} \times \mathcal{A}$. Since $p \neq q$ and $\mathcal{A}$ is a permutation automaton, we have

$$(\mathcal{A} \times \mathcal{A})_{|S} \cong ((\mathcal{A} \times \mathcal{A})/{\sim_D})_{|\pi_{\sim_D}(S)} .$$

and the projection mapping restricted to S, i.e., $(\pi_2)_{|S} S \to Q$, is a surjective morphism from $(\mathcal{A}\times\mathcal{A})_{|S}$ onto $\mathcal{A}$. Thus, by Theorems 7.96 and 7.97 we find that $\mathcal{A}$ is solvable. □

The partition $\{\{(p, q), (q, p)\}\} \cup \{\{D\}\}$ of the state set of $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ induces a congruence whose quotient automaton is isomorphic to $\mathcal{A}^{\leq 2}/{\sim_F}$. Then by Theorems 6.55 and 7.97 it follows that $\mathcal{A}^{\leq 2}/{\sim_F}$ is solvable if $(\mathcal{A} \times \mathcal{A})/{\sim_D}$ is solvable. However, we do not know whether the converse implication holds.

7.10.2 Characterizations

Solvable automata enjoy a wealth of other characterizations as we show next. Theorem 7.103 collects various characterizations in terms of strongly connected components. Theorem 7.107 characterizes solvability by increasing sequences of upper sets, resembling one definition originally given by Rystsov [534] (see the discussion in Section 7.10.4). Theorem 7.108 gives definition in terms of a single upper sets, some of them are suitable for an inductive reasoning and construction of solvable automata, starting from commutative automata.

Theorem 7.103 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *For each $q \in Q$, the automaton*

   $$(\mathcal{A}_{|S})/{\sim_T}$$

   with $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$ is commutative.
3. *For each $q \in Q$, the automaton*

   $$(\mathcal{A}_{|S})/{\sim_T}$$

   with $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$ is a strongly trap-connected commutative permutation-reset automaton or a commutative permutation automaton (where the latter case includes $T = \emptyset$, in which case $\mathcal{A}$ is strongly connected).
4. *For all upper sets $S, T \subseteq Q$ such that $T \subseteq S$ and $S \setminus T$ is a strongly connected component, the automaton $(\mathcal{A}_{|S})/{\sim_T}$ is a commutative permutation-reset automaton.*
5. *For each strongly connected component $S \subseteq Q$ the automaton $\mathcal{A}_{|\Gamma_1, S}$ with $\Gamma_1 = \{a \in \Sigma \mid \delta(S, a) = S\}$ is a commutative permutation automaton and $\Sigma = \Gamma_1 \cup \Gamma_2$ for $\Gamma_2 = \{a \in \Sigma \mid \delta(S, a) \cap S = \emptyset\}$.*
6. *Each strongly connected component $S \subseteq Q$ is a block such that $\delta(q, ab) = \delta(q, ba)$ for all $q \in S$ and $a, b \in \Sigma$ with $\delta(S, a) = \delta(S, b) = S$.*

7. *For each strongly connected component* $S \subseteq Q$ *and* $a, b \in \Sigma$ *with* $\delta(S, a) \cap S \neq \emptyset$ *and* $\delta(S, b) \cap S \neq \emptyset$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$.
8. *For each strongly connected* $S \subseteq Q$ *and* $a, b \in \Sigma$ *with* $\delta(S, a) \cap S \neq \emptyset$ *and* $\delta(S, b) \cap S \neq \emptyset$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$.
9. *For each strongly connected* $S \subseteq Q$ *we have* $\delta(q, ab) = \delta(q, ba)$ *for all* $q \in S$ *and all* $a, b \in \Sigma$ *with* $\delta(q, ab) \in S$.
10. *For each* $q \in Q$ *and* $u \in \Sigma^*$, $\delta(q, u) = q$ *implies* $\delta(q, v) = \delta(q, u)$ *for all* $v \in \Sigma^*$ *such that* $|v|_x = |u|_x$ *for all* $x \in \Sigma$.
11. *For each* $q \in Q$ *and* $u \in \Sigma^*$, $\delta(q, u) = q$ *implies*

$$\delta(q, v) = \delta(q, v_{\pi(1)} \cdots v_{\pi(m)})$$

for each permutation $\pi \colon \{1, \ldots, m\} \to \{1, \ldots, m\}$ *and* $v \in \Sigma^*$ *with* $u \in v\Sigma^*$ *and* $v = v_1 \cdots v_m$ *for* $v_1, \ldots, v_m \in \Sigma$.

Proof. $1 \Rightarrow 2$: Let $p \in S$ with $p \notin T$. Assume $\delta_{|S}(p, ab) \neq \delta_{|S}(p, ab)$. Then $\delta(p, ab) \neq \delta(p, ba)$ and since $\mathcal{A}$ is solvable, we have $\delta(p, ab) \not\simeq_R p$ and $\delta(p, ab) \not\simeq_R p$. Since $p \simeq_R q$, we find $\delta(p, ab) \not\simeq_R q$ and $\delta(p, ba) \not\simeq_R q$ and so $\delta(p, ab), \delta(p, ba) \in T$ by Theorem 6.3 and Equation (3.32). Hence, in $(\mathcal{A}_{|S})/{\sim_T}$, the state p is mapped to T by both words ab and ba. This already establishes commutativity of $(\mathcal{A}_{|S})/{\sim_T}$ since if $\delta_{|S}(p, ab) = \delta_{|S}(p, ab)$, then also in $(\mathcal{A}_{|S})/{\sim_T}$ the state p is mapped to the same state by ab and ba.

$2 \Rightarrow 3$: By assumption $(\mathcal{A}_{|S})/{\sim_T}$ is commutative. Hence, each letter acts as a morphism on its state set, which is $[q]_{\simeq_R} \cup \{T\}$. By Theorem 6.35, each letter either permutes $[q]_{\simeq_R}$ or maps all states in it to T. So, $(\mathcal{A}_{|S})/{\sim_T}$ is a permutation-reset automaton. If there exists a letter mapping $[q]_{\simeq}$ to T, then it is not a sink component and by Theorem 6.10, Item 2 it is strongly trap-connected. Otherwise, since T is a trap state for $T \neq \emptyset$, each letter permutes all states.

Alternatively, this follows with Theorem 7.70.

$3 \Rightarrow 4$: First, assume S is a sink component. Then, by Item 3, $\mathcal{A}_{|S}$ is a commutative permutation automaton. So, $(\mathcal{A}_{|S})/{\sim_T}$ is a commutative permutation automaton, since for $T \neq \emptyset$ merely a trap state is added. Next, assume S is not a sink component. By assumption, we have $S \setminus T = [q]_{\simeq_R}$ for some $q \in Q$. Since S is an upper set, we have $\uparrow\{q\} \subseteq S \subseteq T$. Set $K = \uparrow\{q\}$ and $R = K \setminus [q]_{\simeq_R}$, where $R \neq \emptyset$ and $T \neq \emptyset$ as S is a sink component. Then, it is

$$(\mathcal{A}_{|S})/{\sim_T} \cong (\mathcal{A}_{|K})/{\sim_R}$$

since $\sim_T$ on S equals $[q]_{\simeq_R} \times [q]_{\simeq_R} \cup \{T\}$ and $\sim_R$ on K equals $[q]_{\simeq_R} \times [q]_{\simeq_R} \cup \{R\}$ and so both induce isomorphic Rees quotient automata. By Item 3, $(\mathcal{A}_{|K})/{\sim_R}$ is a commutative permutation-reset automaton.

$4 \Rightarrow 5$: Let $S \subseteq Q$ be a strongly connected component. By assumption, $(\mathcal{A}_{|R})/{\sim_T}$ with $R = \uparrow S$ and $T = R \setminus S$ is a permutation-reset automaton, which is equivalent to S being a block by Theorem 7.36, which gives $\Sigma = \Gamma_1 \cup \Gamma_2$, and by Theorem 7.68 it is commutative so that $\mathcal{A}_{|\Gamma_1, S}$ is a commutative permutation automaton.

$5 \Rightarrow 6$: This is clear.

$6 \Rightarrow 7$: Since a strongly connected component S is a block by assumption, $\delta(S,a) \cap S \neq \emptyset$ and $\delta(S,b) \cap S \neq \emptyset$ imply $\delta(S,a) = \delta(S,b) = S$ and then $\delta(q,ab) = \delta(q,ba)$ for all $q \in S$.

$7 \Rightarrow 8$: Let $S \subseteq Q$ be strongly connected, $q \in \delta(S,a) \cap S$, and $p \in \delta(S,b) \cap S$. Set $T = [q]_{\simeq}$. Then $S \subseteq T$, $q \in \delta(T,a) \cap T$, $p \in \delta(T,b) \cap T$, and so $\delta(r,ab) = \delta(r,ba)$ for all $r \in T$, in particular for those states from S.

$8 \Rightarrow 9$: Let S, $q \in S$ and $a, b \in \Sigma$ as in the statement. Since $\delta(q,ab) \in S$ and $q \in S$, the set $T = S \cup \{\delta(q,a)\}$ is also strongly connected. we have $\delta(q,a) \in T$ and $\delta(\delta(q,a),b) \in T$ so that, by Item 8, we find $\delta(q,ab) = \delta(q,ba)$.

$9 \Rightarrow 10$: Write $u = u_1 \cdots u_n$ with $u_i \in \Sigma$ for $i \in \{1,\ldots,n\}$. Let $i \in \{1,\ldots,n\}$. Since $\delta(q,u) = q$, we have

$$q \simeq_R \delta(q, u_1 \cdots u_{i-1}) \simeq_R \delta(q, u_1 \cdots u_{i-1}u_i) \simeq_R \delta(q, u_1 \cdots u_{i-1}u_iu_{i+1}).$$

So, the set $S = \{q, \delta(q,u_1\cdots u_{i-1}), \delta(q,u_1\cdots u_{i-1}u_i), \delta(q,u_1\cdots u_{i-1}u_iu_{i+1}), q\}$ is strongly connected. By assumption, we get $\delta(p,u_iu_{i+1}) = \delta(p,u_{i+1}u_i)$ for all $p \in S$, in particular we have

$$\delta(q, u_1 \cdots u_{i-1}u_iu_{i+1}) = \delta(q, u_1 \cdots u_{i-1}u_{i+1}u_i).$$

Since every permutation of the letters $u_1,\ldots,u_n$ can be realized by swapping adjacent letters, we obtain the claim.

$10 \Rightarrow 11$: Let $q \in Q$ and $u, v, w \in \Sigma^*$ be such that $\delta(q,u) = q$ and $u = vw$. Write $v = v_1 \cdots v_m$ for $v_1,\ldots,v_m \in \Sigma$ and consider a permutation $\pi\colon \{1,\ldots,m\} \to \{1,\ldots,m\}$. Since $\delta(q,vw) = q$ we have $\delta(q,v) = \delta(q,vwv)$ and $\delta(q, v_{\pi(1)}\cdots v_{\pi(m)}) = \delta(q, vwv_{\pi(1)}\cdots v_{\pi(m)})$. By assumption, the former equality gives $\delta(q, vwv_{\pi(1)}\cdots v_{\pi(m)}) = \delta(q,v)$. Hence, we find $\delta(q,v) = \delta(q, v_{\pi(1)}\cdots v_{\pi(m)})$. This gives the claim.

$11 \Rightarrow 1$: Let $p \in Q$ and $a, b \in \Sigma$ such that $\delta(q,ab) \precsim_R q$. Then $\delta(q,abu) = q$ for some $u \in \Sigma^*$ and, by assumption, we have $\delta(q,ab) = \delta(q,ba)$. □

Our first corollary extends Theorem 7.71.

Corollary 7.104 *A strongly trap-connected (resp. strongly connected) solvable automaton $\mathcal{A} = (Q,\Sigma,\delta)$ is a commutative permutation-reset (resp. commutative permutation) automaton.*

Proof. By Theorem 6.10, Item 2 we have $Q = [q]_{\simeq_R} \cup \{t\}$ for a non-maximal $q \in Q$ and a trap state $t \in Q$ (resp. $Q = [q]_{\simeq_R}$ for some $q \in Q$). Set $S = \uparrow\{q\}$ and $T = S \setminus [q]_{\simeq_R}$. Since $Q = S$ and $T = \{t\}$ (resp. $T = \emptyset$) from Theorem 7.103, Item 3 and Equation (6.12) we get that $\mathcal{A}$ is a commutative permutation-reset (resp. commutative permutation) automaton. □

Corollary 7.105 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ be solvable and $S \subseteq Q$ a non-empty union of sink components (as, for example, the set of maximal states). Then $\mathcal{A}_{|S}$ is a commutative permutation automaton.*

Proof. For each sink component $T \subseteq S$, $\mathcal{A}_{|T}$ is solvable and strongly connected, and so a commutative permutation automaton by Theorem 7.104, which gives the claim. □

Corollary 7.106 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$ be an upper set. Then $\mathcal{A}/{\sim_S}$ is solvable if and only if for each strongly connected component $T \subseteq Q$ we have $\delta(q, ab) = \delta(q, ba)$ for all $a, b \in \Sigma$ and $q \in T \setminus S$ with $\delta(q, ab) \in T$.*

Proof. This follows directly from Theorem 7.103, Item 9, the definition of the Rees quotient automaton, and the fact that a strongly connected set is either completely outside of S or completely inside of S. □

Proposition 7.107 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *There exists a sequence of upper sets $S_1 \subsetneq \cdots \subsetneq S_m = Q$ where S_1 equals the union of all sink components, $\mathcal{A}_{|S_1}$ is a commutative permutation automaton, and, for $i \in \{2, \ldots, m\}$, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a strongly trap-connected commutative permutation-reset automaton and $S_i \setminus S_{i-1}$ are all the strongly connected components that are not sink components.*
3. *There exists a sequence of upper sets $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ such that $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a commutative permutation-reset automaton and $S_i \setminus S_{i-1}$ are all the strongly connected components for $i \in \{1, \ldots, m\}$.*
4. *There exists a sequence of upper sets $\emptyset = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_m = Q$ such that $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a solvable automaton for each $i \in \{1, \ldots, m\}$.*

Proof. 1 $\Rightarrow$ 2: Consider a sequence of upper sets as in Theorem 3.59, Item 2. By Theorem 7.105, $\mathcal{A}_{|S_0}$ is a commutative permutation automaton. Next, each set S_i for $i \in \{1, \ldots, m\}$ has the form $S_i = [q]_{\simeq} \cup S_{i-1}$ for some $q \in S_i \setminus S_{i-1}$, where q is not maximal since S_1 equals the set of maximal states. By Theorem 7.103, Item 3, $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is a commutative permutation-reset automaton.[6] Since q is not a maximal state, $[q]_{\simeq}$ is not a sink component and so $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ is strongly trap-connected by Theorem 6.10.

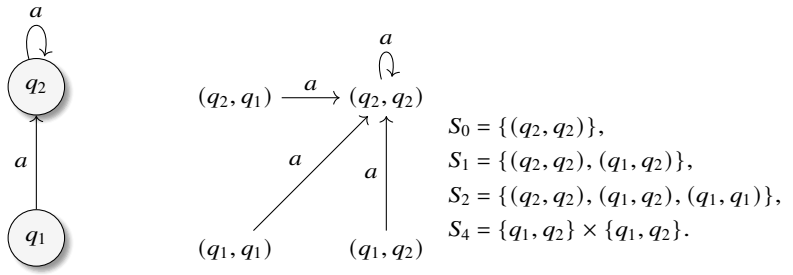
2 $\Rightarrow$ 3: Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Item 2. Applying Theorem 3.59, Item 1 to $\mathcal{A}_{|S_1}$ there exists a sequence $\emptyset = T_0 \subsetneq T_1 \subsetneq \cdots \subsetneq T_l$ such that $T_i \setminus T_{i-1}$ for $i \in \{1, \ldots, l-1\}$ are all the sink components. Since $\mathcal{A}_{|S_1}$ is a commutative permutation automaton, each $\mathcal{A}_{|T_i}$ is a commutative permutation automaton. Then each $(\mathcal{A}_{|T_i})/{\sim_{T_{i-1}}}$ is a commutative permutation automaton. Now, $T_0 \subsetneq T_1 \subsetneq \cdots \subsetneq T_l \subsetneq S_2 \subsetneq \cdots \subsetneq S_m$ yields a sequence as claimed.

3 $\Rightarrow$ 4: As a commutative automaton is solvable, we obtain the claim.

4 $\Rightarrow$ 1: Let $q \in Q$ and $a, b \in \Sigma$ be such that $\delta(q, ab) \precsim_R q$. Since, by Theorem 6.3, $\delta(q, ab)$ and q are contained in the same strongly connected component, by Theorem 3.37, we have $\{\delta(q, ab), q\} \subseteq S_i \setminus S_{i-1}$ for some $i \in \{1, \ldots, m\}$. Since $(\mathcal{A}_{|S_i})/{\sim_{S_i}}$ is solvable, by Theorem 6.61 we have $\{\delta_{|S_i}(q, ab)\} = \delta_{\sim_{S_i}}([q]_{\sim_{S_i}}, ab) = \delta_{\sim_{S_i}}([q]_{\sim_{S_i}}, ba) = \{\delta_{|S_i}(q, ba)\}$ where $[q]_{\sim_{S_i}} = \{q\}$, and so $\delta(q, ab) = \delta(q, ba)$. Hence, $\mathcal{A}$ is solvable. □

Figure 7.12 gives a commutative automaton with a sequence as in Theorem 7.107, Item 2.

[6] Alternatively, since the quotient automata $(\mathcal{A}_{|S_i})/{\sim_{S_{i-1}}}$ for $i \in \{2, \ldots, m\}$ are strongly trap-connected by Theorem 6.10 and solvable by Theorem 7.98, from Theorem 7.104 we obtain that they are commutative permutation-reset automata.



(a) An automaton $\mathcal{A}$. (b) The automaton $\mathcal{A} \times \mathcal{A}$. (c) Sequence of upper sets.

Fig. 7.12: A sequence of subsets $S_0 \subseteq S_1 \subseteq S_2 \subseteq S_3$ as stated in Theorem 7.107, Item 2 for a unary (and so solvable) automaton of the form $\mathcal{A} \times \mathcal{A}$.

Proposition 7.108 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. Then the following are equivalent.*

1. *$\mathcal{A}$ is solvable.*
2. *$\mathcal{A}$ is a commutative permutation-reset automaton or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ is solvable and $\mathcal{A}/{\sim_S}$ is a commutative and strongly trap-connected permutation-reset automaton.*
3. *$\mathcal{A}$ is a commutative permutation-reset automaton or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ is a commutative permutation-reset automaton and $\mathcal{A}/{\sim_S}$ is solvable.*
4. *$\mathcal{A}$ is commutative or there exists a proper upper set $S \subseteq Q$ with $|S| \geq 2$ such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable.*
5. *There exists an upper set $S \subseteq Q$ such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable.*

Proof. $1 \Rightarrow 2$ & $1 \Rightarrow 3$: Consider a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ as in Theorem 7.107, Item 2. For $m = 1$, Q equals the set of maximal states and $\mathcal{A}$ is a commutative permutation automaton and so Items 2 and 3 hold. Next, suppose $m \geq 2$. Then $\mathcal{A}/{\sim_{S_{m-1}}}$ is a commutative and strongly trap-connected permutation-reset automaton. For $|S_{m-1}| = 1$, we have $\mathcal{A} \cong \mathcal{A}/{\sim_{S_{m-1}}}$ and this shows Item 2. Otherwise, since $\mathcal{A}_{|S_{m-1}}$ is solvable by Theorem 7.96, Item 2 holds with $S = S_{m-1}$.

Next, if $|S_1| \geq 2$, then Item 3 is established since $\mathcal{A}_{|S_1}$ is a commutative permutation automaton and $\mathcal{A}/{\sim_{S_1}}$ is solvable by Theorem 7.98. For $|S_1| = 1$, we have $\mathcal{A}_{|S_2} \cong (\mathcal{A}_{|S_2})/{\sim_{S_1}}$, and the latter automaton is a commutative permutation-reset automaton by choice of the sequence. Thus for $m = 2$, $\mathcal{A}$ is a commutative permutation-reset automaton and Item 3 holds or Item 3 holds with the proper upper set of size at least two $S = S_2$.

$2 \Rightarrow 4$ & $3 \Rightarrow 4$ & $4 \Rightarrow 5$: These implications are clear.

$5 \Rightarrow 1$: Let $S \subseteq Q$ be an upper set such that $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are solvable. We can assume $0 < |S| < |Q|$. Then $\emptyset \subsetneq S \subsetneq Q$ gives a sequence as in Theorem 7.107, Item 4. So $\mathcal{A}$ is solvable. □

Theorem 7.108, Item 2 is similar to Theorem 6.79, but gives more information on automata-theoretical properties.

Remark 7.109 Theorem 7.108, Item 4 can be used to give an alternative inductive proof of the statement from Theorem 7.99 that $\mathcal{A}_1 \times \mathcal{A}_2$ is solvable for solvable $\mathcal{A}_1, \mathcal{A}_2$. Since, if $\mathcal{A}_1$ and $\mathcal{A}_2$ are commutative, then it is readily seen that $\mathcal{A}_1 \times \mathcal{A}_2$ is commutative. Next, if there exists, without loss of generality, an upper set $S_1 \subseteq Q_1$ with $2 \leq |S_1| < |Q_1|$ such that $(\mathcal{A}_1)_{|S_1}$ and $\mathcal{A}_1/{\sim_{S_1}}$ are solvable, then the set $T = S_1 \times Q_2$ is an upper set in $\mathcal{A}_1 \times \mathcal{A}_2$ and by Equations (6.18) and (6.19) we have

$$(\mathcal{A}_1 \times \mathcal{A}_2)_{|T} \cong (\mathcal{A}_1)_{|S_1} \times \mathcal{A}_2 \quad \text{and} \quad (\mathcal{A}_1 \times \mathcal{A}_2)/{\sim_T} \cong \mathcal{A}_1/{\sim_{S_1}}.$$

Inductively, $(\mathcal{A}_1)_{|S_1} \times \mathcal{A}_2$ is solvable. So we obtain solvability of $\mathcal{A}_1 \times \mathcal{A}_2$.

Remark 7.110 Theorem 7.108, Item 4 can be used to give an alternative proof for Theorem 7.96 using the various isomorphisms established in Section 6.5.3 as we shown next with the same notation as in Theorem 7.96. First, commutativity of $\mathcal{A}$ transfers to $\mathcal{A}_{|S}$. Next, suppose there exists an upper set $R \subseteq Q$ with $2 \leq |R| < |Q|$ such that $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable. By Equation (6.7) we have

$$(\mathcal{A}_{|S})_{|S\cap R} \cong \mathcal{A}_{|S\cap R} \cong (\mathcal{A}_{|R})_{|S\cap R},$$

and with the canonical projection map $\pi_{\sim_R} : Q \to Q/{\sim_R}$ introduced in Section 3.4, by Equation (6.13),

$$(\mathcal{A}_{|S})/{\sim_{S\cap R}} \cong (\mathcal{A}/{\sim_R})_{|\pi_{\sim_R}(S)}.$$

Inductively, $(\mathcal{A}_{|R})_{|S\cap R}$ and $(\mathcal{A}/{\sim_R})_{|\pi_{\sim_R}(S)}$ are solvable. So, by Theorem 7.108, Item 4 or by Equation (6.12), $\mathcal{A}_{|S}$ is solvable.

Remark 7.111 Theorem 7.108, Item 4 can be used to give an alternative proof for Theorem 7.97 using the various isomorphisms established in Section 6.5.3 as we shown next with the same notation as in Theorem 7.97. First, if $\mathcal{A}$ is commutative, then clearly $\mathcal{B}$ is commutative. Now, suppose there exists an upper set $R \subseteq Q$ with $2 \leq |R| < |Q|$ such that $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable. The set $\varphi(R)$ is an upper set in $\mathcal{B}$ by Theorem 6.25. Now, $\varphi_{|R}$ considered as a surjective function from R onto $\varphi(R)$ is a surjective morphism from $\mathcal{A}_{|R}$ onto $\mathcal{B}_{|\varphi(R)}$. By Theorem 6.67 there exists a surjective morphism from $\mathcal{A}/{\sim_R}$ onto $(\mathcal{B}/{\sim_{\varphi(R)}})_{|\pi_{\sim_{\varphi(R)}}(\varphi(Q))}$, where $(\mathcal{B}/{\sim_{\varphi(R)}})_{|\pi_{\sim_{\varphi(R)}}(\varphi(Q))} = \mathcal{B}/{\sim_{\varphi(R)}}$ as $\varphi(Q) = S$. By induction hypothesis, as $\mathcal{A}_{|R}$ and $\mathcal{A}/{\sim_R}$ are solvable, $\mathcal{B}_{|\varphi(R)}$ and $\mathcal{B}/{\sim_{\varphi(R)}}$ are solvable. So, by Theorem 7.108, Item 4, $\mathcal{B}$ is solvable.

7.10.3 Synchronization and Subset Mapping Properties

First, in Theorem 7.112, we state that in solvable automata, a synchronizing state is a trap state (and hence unique). Theorem 7.113 states the bound $n - r$ for shortest words of rank at most r in solvable automata that admit a word of rank r, thus extending Rystsov's bound for $r = 1$ [534, Cor. 4.2]. Proposition 7.114 gives words of a specific form to map one subset onto another subset in solvable automata. This result is rather technical, and we need it in Section 8.3.3 (Theorem 8.18) to prove that SetTrans is in NP for solvable automata.

Proposition 7.112 *A synchronizing state in a solvable automaton is a trap state.*

Proof. By Theorem 7.103, Item 6 in solvable automata the sink components are blocks. So, the claim follows from Theorem 7.51. □

Theorem 7.113 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state solvable automaton. If there exists a word of rank r in $\mathcal{A}$, then there exists a word of length at most $n - r$ whose rank is at most r and these bounds are tight even for unary automata where r is the minimal rank among all words. Furthermore, for $\Sigma = \{a_1, \ldots, a_k\}$, the word $(a_1 \cdots a_k)^{n-r}$ has rank at most r.*

Proof. By Theorem 7.103, Item 6 the strongly connected components in $\mathcal{A}$ are blocks. So, Theorem 7.43 gives the upper bound $n-r$ and that $(a_1 \cdots a_k)^{n-r}$ has rank at most r. Tightness as claimed follows from the automata given in the proof of Theorem 7.29. □

Proposition 7.114 *Let $\mathcal{A} = (Q, \Sigma, \delta)$ be solvable with n states, $S \subseteq Q$, and $w \in \Sigma^*$. Then for $\Sigma = \{a_1, \ldots, a_k\}$ there exists $v \in \Sigma^*$ with $|v| < 2^n$ and $\delta(S, v) = \delta(S, w)$ of the form*

$$v = v_0 x_1 \cdots v_{r-1} x_r v_r$$

for some $r \leq n^2$ where $x_1, \ldots, x_r \in \Sigma$ and $v_i = a_1^{r_{i,1}} \cdots a_k^{r_{i,k}}$ for numbers $0 \leq r_{i,j} < 2^n$ with $i \in \{0, 1, \ldots, r\}$ and $j \in \{1, \ldots, k\}$.

Proof. We can assume $S \neq \emptyset$. By Theorem 7.107, Item 2 there exist upper sets

$$S_1 \subsetneq \cdots \subsetneq S_m = Q \tag{7.11}$$

where S_1 equals the union of all sink components, $\mathcal{A}_{|S_1}$ is commutative, and $S_i \setminus S_{i-1}$ for $i \in \{2, \ldots, m\}$ are all the strongly connected components that are not sink components. Consider a word $u \in \Sigma^*$ of shortest possible length with $\delta(S, u) = \delta(S, w)$. Let $S = \{s_1, \ldots, s_l\}$ for $l = |S| \geq 1$ and set $P = \mathrm{Pref}(\{u\})$. Next, we define a function $f \colon P \to \{1, \ldots, m\}^l$ by setting $f(u') = (i_1, \ldots, i_l)$ for $u' \in P$ where $i_j \in \{1, \ldots, m\}$ equals the minimal number with $\delta(s_j, u') \in S_{i_j}$ for all $j \in \{1, \ldots, l\}$. Now, there exists a unique factorization $u = u_0 x_1 u_1 \cdots u_{r-1} x_r u_r$ with $u_0, u_1, \ldots, u_r \in \Sigma^*$ and $x_1, \ldots, x_r \in \Sigma$ for some $r \geq 0$ such that $f(u_0 x_1 \cdots u_{t-1}) \neq f(u_0 x_1 \cdots u_{t-1} x_t)$ for $t \in \{1, \ldots, r\}$ and

$$\begin{aligned} f(\varepsilon) &= f(u_0), \\ f(u_0 x_1) &= f(u_0 x_1 u_1), \\ &\vdots \\ f(u_0 x_1 \cdots u_{r-1} x_r) &= f(u_0 x_1 \cdots u_{r-1} x_r u_r). \end{aligned}$$

Since the S_i are upper sets, for $t \in \{1, \ldots, r\}$ with $f(u_0 x_1 \cdots u_{t-1}) = (i_1, \ldots, i_l)$ and

$$f(u_0 x_1 \cdots u_{t-1} x_t) = (i'_1, \ldots, i'_l)$$

we have $i_j \geq i'_j$ for all $j \in \{1, \ldots, l\}$ where $i_j > i'_j$ for at least one $j \in \{1, \ldots, l\}$. So, $i'_1 + \ldots + i'_l < i_1 + \ldots + i_l$. As these sums are between l and lm, they can decrease

at most $l(m-1)$ times and we find $r \leq l(m-1)$. As $l(m-1) \leq n^2$ we get $r \leq n^2$. Now, for $j \in \{1,\dots,l\}$ and $t \in \{0,1,\dots,r\}$ the states $\delta(s_j, u_0x_1\cdots u_{t-1}x_t)$ and $\delta(s_j, u_0x_1\cdots u_{t-1}x_tu_t)$ are both contained in the strongly connected component $S_{i_j} \setminus S_{i_j-1}$ for $i_j \geq 2$ or both states are contained in S_1.[7] By Theorem 7.103, Item 10 and as $\mathcal{A}_{|S_1}$ is commutative we can reorder the letters in u_t while still ending up at the same state and so we find

$$\delta(s_j, u_0x_1\cdots u_{t-1}x_tu_t) = \delta(s_j, u_0x_1\cdots u_{t-1}x_ta_1^{|u_t|_{a_1}}\cdots a_k^{|u_t|_{a_k}}).$$

Now, as we have chosen u of minimal length, for distinct $u', u'' \in P$ we have $\delta(S,u') \neq \delta(S,u'')$ as otherwise, for, without loss of generality, $|u'| < |u''|$ we have $\delta(S,u'u''') = \delta(S,u''u''')$ for $u''' \in \Sigma^*$ with $u = u''u'''$ where $|u'u'''| < |u|$, contradicting the minimal choice of u. So, each $u' \in P$ corresponds to a distinct nonempty subset of states $\delta(S,u')$ and we conclude $|u| \leq 2^n - 1$, which gives $|u_t|_{a_h} \leq 2^n - 1$ for each $h \in \{1,\dots,k\}$. Then for the word $v = v_0x_1\cdots v_{r-1}x_rv_r$ where $v_i = a_1^{|u_i|_{a_1}}\cdots a_k^{|u_i|_{a_k}}$ for $i \in \{0,1,\dots,r\}$ we have $\delta(S,u) = \delta(S,v)$ and the claim is proved with v. □

7.10.4 Comparison with Original Definition

Rystsov [533] originally defined an automaton to be solvable, if it is synchronizing and admits a sequence $S_1 \subsetneq \cdots \subsetneq S_m = Q$ of upper sets such that S_1 is a singleton set containing a trap state and $\mathcal{A}_{|S_i}/{\sim_{S_{i-1}}}$ for $i \in \{2,\dots,m\}$ are commutative. This is equivalent to being synchronizing and solvable since, with Theorem 7.112, it follows that S_1 is always a singleton for synchronizing and solvable automata. We call a synchronizing and solvable automaton also an R_1-*solvable* automaton in the following. Furthermore, we say that $\mathcal{A} = (Q,\Sigma,\delta)$ is R_2-*solvable* if $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ is R_1-solvable where $D = \{(q,q) \mid q \in Q\}$. This was another notion of solvability considered in [533]. Since synchronizability of $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ is equivalent to synchronizability of $\mathcal{A}$ by Theorem 7.16, an R_2-solvable automaton is synchronizing.[8]

Proposition 7.115 *Let $\mathcal{A} = (Q,\Sigma,\delta)$ and $S \subseteq Q$ be the set of maximal states in $\mathcal{A}$ w.r.t. $\preceq_R$. Then $\mathcal{A}$ is solvable if and only if $\mathcal{A}/{\sim_S}$ is R_1-solvable and $\mathcal{A}_{|S}$ is a commutative permutation automaton.*

Proof. First, suppose $\mathcal{A}$ is solvable. Then, by Theorem 7.107, Item 2, $\mathcal{A}_{|S}$ is a commutative permutation automaton. By Theorem 7.98, $\mathcal{A}/{\sim_S}$ is solvable and, by Theorem 7.10, it is synchronizing; and so R_1-solvable.

Conversely, suppose $\mathcal{A}_{|S}$ is a commutative permutation automaton and $\mathcal{A}/{\sim_S}$ is R_1-solvable, and so also solvable. Thus, by Theorem 7.108, Item 5, since also $\mathcal{A}_{|S}$ is solvable, $\mathcal{A}$ is solvable. □

Proposition 7.116 *An automaton $\mathcal{A} = (Q,\Sigma,\delta)$ is R_2-solvable if and only if it is synchronizing and $(\mathcal{A}\times\mathcal{A})/{\sim_D}$ with $D = \{(q,q) \mid q \in Q\}$ is solvable.*

[7] We set $u_0x_1\cdots u_{-1}x_0 = \varepsilon$.

[8] In the original definition from [533, p. 277] synchronizability of $\mathcal{A}$ was explicitly included.

Proof. If $\mathcal{A}$ is R_2-solvable, then $(\mathcal{A} \times \mathcal{A})/\sim_D$ is R_1-solvable, and so solvable and synchronizing. Synchronizability of $(\mathcal{A} \times \mathcal{A})/\sim_D$ implies synchronizability of $\mathcal{A}$ by Theorem 7.16.

Conversely, Theorem 7.16 implies synchronizability of $(\mathcal{A} \times \mathcal{A})/\sim_D$ and so it is R_1-solvable. Thus $\mathcal{A}$ is R_2-solvable. □

Proposition 7.117 *Let $\mathcal{A} = (Q, \Sigma, \delta)$. If $\mathcal{A}$ is R_1-solvable, then $\mathcal{A}$ is R_2-solvable.*

Proof. Since $\mathcal{A}$ is solvable, Theorem 7.101 gives solvability of $(\mathcal{A} \times \mathcal{A})/\sim_D$. Then, synchronizability of $\mathcal{A}$ yields that it is R_2-solvable. □

By the following corollary, the class of R_1-solvable automata also equals the intersection of the class of R_2-solvable with the class of solvable automata.

Corollary 7.118 *The automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is R_1-solvable if and only if it is R_2-solvable and solvable.*

Proof. An R_1-solvable automaton is solvable and, by Theorem 7.117, R_2-solvable. Conversely, since an R_2-solvable automaton is synchronizing, if it is solvable, it is also R_1-solvable. □

Figure 7.11 depicts an R_2-solvable automaton that is not solvable, and so also not R_1-solvable. By definition, every non-synchronizing solvable automaton is neither R_1- nor R_2-solvable. Figure 7.13 shows the inclusions relations between our notions of solvability that follow from the above considerations.

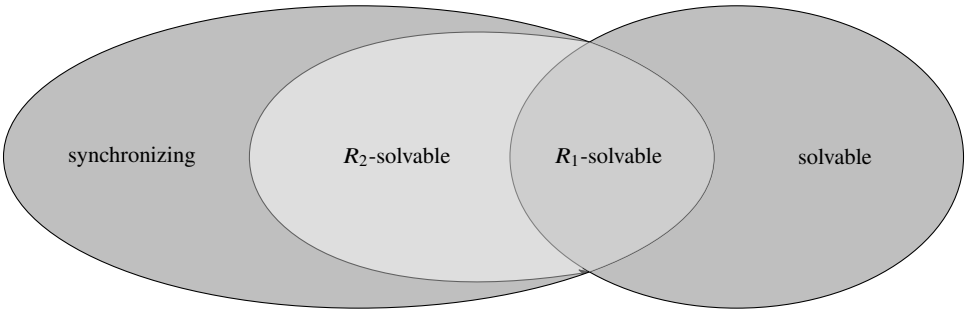


Fig. 7.13: Inclusion relations between the different solvability notions.

Remark 7.119 Yet another, more general notion, is obtained by considering those automata where $(\mathcal{A} \times \mathcal{A})/\sim_D$ is solvable. This class contains the R_2-solvable automata and, by Theorem 7.101, all solvable automata. By adding a trap state to the automaton $\mathcal{A}$ from Figure 7.11, we obtain a non-solvable automaton such that $(\mathcal{A} \times \mathcal{A})/\sim_D$ is solvable but not synchronizing. Thus this class properly contains the R_2-solvable automata.

7.11 Discussion

Here, we discuss the classes, and related literature, introduced in this chapter. Later, in **????** and Chapter 8, we investigate these subclasses w.r.t. to decision problems, automata for sets of synchronizing words, and the constrained synchronization problem, respectively. A discussion of the results from the these chapters can be found in **????** and Section 8.4. Here, we leave out a discussion of synchronizable automata, because they are the topic of the whole work and some related literature was already discussed in the introductory part (Chapter 1). The new notion of ρ-automata is introduced here for the first time.

SI-Automata.

Rystsov introduced SI-automata in [535] and proved the bound $2(n-1)^2$ for the reset length of an n-state SI-automata $\mathcal{A}$. In [536] the author showed the bound $n-1$ in case the group of units of $\mathcal{T}_S(\mathcal{A})$ is trivial and the bound $(n-1)^2$ in case the group of units of $\mathcal{T}_S(\mathcal{A})$ is a regular permutation group [536]. We deduce both bounds in **????** in the case of OI-automata.

Permutation-Reset Automata.

Permutation-reset automata were firstly mentioned by Zeiger in the conference contribution [644] (see [645] for the extended version) in connection with the Krohn-Rhodes decomposition theory [389] (see Section 3.12 for some background on this theory). McNaughton [448] and Thierrin [598] investigated permutation automata as language acceptors, see also [195, 489, 490] for information on the class of languages accepted by permutation automata, the so-called *variety of group languages*.

Commutative Automata.

Commutative automata were introduced, to the best of our knowledge, by Laing and Wright [399]. They focused on commutative automata over a binary alphabet, gave normal form theorems for regular expressions modulo commutativity, and also considered infinite-state commutative automata and regularity conditions. Rystsov [533] gave (by an algebraic proof) the tight bound $n-1$ for a shortest synchronizing word of an n-state commutative automaton. Fernau and Hoffmann [212] gave a different (combinatorial) proof and a structure theorem for commutative automata that reach this bound. Fleck [219] determined the automorphism group of strongly connected and commutative automata. Cascade product decomposition results (see the discussion in Section 3.12 on automata theory for this notion) for commutative automata were given in [241, 242]. See also [55, 202, 334, 477] for further algebraic investigations. Question on automata sizes for commutative regular languages were investigated in [312, 311].

The languages accepted by different types of commutative automata were investigated in [13, 293, 302, 310, 312, 489, 490]. Algorithmic problems on commutative automata were investigated in [213], and the membership problem on commutative transformation semigroups in [73].

Partially Ordered Automata.

This type of automata has been considered in the literature under different names: *weakly acyclic* [537], *acyclic* [344], *extensive* [491] or *1-weak/very weak/linear* [173] (linear stems from the existence of a topological sorting of the states). Ryzhikov [537] investigated several synchronization and rank problems on partially ordered automata and obtained the tight bound $n - r$ for a shortest word of rank at most r. Bruchertseifer and Fernau [105] investigated the subclass of *series-parallel automata with loops* with respect to questions of synchronizability in the setting of parameterized complexity (see the part on parameterized complexity in Section 8.4 for more information). Brzozowski and Fich [111] obtained that partially ordered automata accept precisely those languages with $\mathcal{R}$-trivial syntactic monoids. Jirásková and Masopust [344] investigated the state complexity of the reverse operation for partially ordered automata, Dax and Klaedtke [173] investigated the translation of alternating partially ordered automata to nondeterministic automata in the setting of infinite words.

Monotonic Automata.

Ananichev and Volkov [23] introduced monotonic automata and proved Theorem 7.42.[9] However, transformation semigroups whose elements preserve some linear order were investigated before, for example, in [324]. The bound $n - 1$ for shortest synchronizing of monotonic automata was stated in [22] in the language of transformation semigroups. In [539] problems related to (subset) synchronization problems on monotonic partial automata were investigated. Checking whether a given complete automaton is monotonic is NP-complete [592]. Anachichev and Volkov [24] extended the tight bound $n - r$ on shortest words of rank at most r (Theorem 7.42) to so-called *generalized monotonic automata* (these automata where defined in levels that yield an infinite hierarchy even in the case of strongly strongly connected automata [12]) Apart from its rather simple definition, the behavior of monotonic automata as language acceptors remains rather elusive. Shcherbak [565] posed the *open problem* to give a combinatorial description (for example, by regular expressions) of the languages accepted by monotonic automata. In [382, 383] monotonic automata were used to describe certain winning conditions in infinite games. In [19] shortest words that are undefined on each state in

[9] The monotonic automata as introduced earlier by Eppstein [198] are more general, as they are only required to respect a cyclic order and were renamed *oriented* automata by Ananichev and Volkov [23], and *cyclically monotonic* by Martyugin [436]). Linearly and circularly monotonic might be another possible renaming for both concepts.

partially ordered monotonic automata were investigated, which correspond to shortest synchronizing words in the complete automaton resulting from adding a trap state.

Solvable Automata.

Rystsov [533] gave the tight bound $n-1$ for shortest synchronizing words of n-state R_1- and R_2-solvable automata. Here, in Theorem 7.113, for solvable automata, we gave the bound $n-r$ for shortest words of rank at most r. In personal communication, Mikhail V. Volkov noted that the transition monoid of a solvable automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is in the pseudovariety **DS** (in fact, of any automaton whose strongly connected components are blocks). Automata with this property admit a word of rank r and length at most $n-r$ if the minimal rank among its words is r [15, Thm. 2.6]. Theorem 7.113 is slightly more general since it applies to values of r that do not equal the minimal rank among all the ranks of the words. The containment of their transition monoids in **DS** follows from a characterization of **DS** as the set of finite semigroups that are semilattices over semigroups nilpotent over their kernel (see the equivalence (4c) $\Leftrightarrow$ (1b) in [567, Thm. 3] and [216, 567] for definitions of the mentioned notions). For let $P_1, \ldots, P_m$ be a topological sorting of the strongly connected components w.r.t. $\preceq_R$. Then set

$$Y = \{(x_1, \ldots, x_m) \in \{0,1\}^m \mid \exists w \in \Sigma^* \colon x_i = 1 \Leftrightarrow \delta(P_i, w) \subseteq P_i\}$$

and, for $(x_1, \ldots, x_m) \in Y$, set

$$S_{(x_1,\ldots,x_m)} = \{\delta_w \colon Q \to Q \mid \forall i \in \{1, \ldots, m\} \colon x_i = 1 \Leftrightarrow \delta(P_i, w) \subseteq P_i\}.$$

Then Y is a semilattice with the operation

$$(x_1, \ldots, x_m)(y_1, \ldots, y_m) = (\min\{x_1, y_1\}, \ldots, \min\{x_m, y_m\})$$

and $\{S_y\}_{y \in Y}$ forms a semilattice of semigroups where the S_y are groups, and so nilpotent over their kernel.

Chapter 8
Subset Mapping Problems

In this chapter, we investigate various subset mapping and synchronization problems on automata, which we introduce in Section 8.1. Section 8.2 explores reductions and relations between them. In Section 8.3 we investigate the computational complexity of these problems on general automata and the subclasses of solvable, commutative, and partially ordered automata. We give a classification of the (classical) complexity landscape and close some gaps from the literature.

8.1 The Problems

Here, we list the automata-theoretical decision problems of interest to us.

Recall that in Section 3.9.6 we discussed how we encode automata as inputs, and that here, automata have a totally defined transition function.

Sync (Černý [140])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$.
Question: Is $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$?

SyncTrap
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with trap state.
Question: Is $\mathrm{Syn}(\mathcal{A}) \neq \emptyset$?

SyncState
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and state $q \in Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) = \{q\}$?

SyncWords
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and finite list of words $u_1, \ldots, u_k \in \Sigma^*$.
Question: $\exists i \in \{1, \ldots, k\} : |\delta(Q, u_i)| = 1$?

SyncShort (Rystsov [528], Eppstein [198])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $l \geq 0$.
Question: Is $\mathrm{Syn}(\mathcal{A}) \cap \Sigma^{\leq l} \neq \emptyset$?

SyncFrom (Rystsov [529])[1]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $|\delta(S, u)| = 1$?

SyncInto (Rystsov [529])[2]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) \subseteq S$?

SetTrans (Rystsov [529], Luks and McKenzie [425], Blondin et al. [96])[3]
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$?

DisjSetTrans
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ with $S \cap T = \emptyset$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) \subseteq T$?

Intersection (Kozen [387])
Input: List of automata $\mathcal{A}_1, \ldots, \mathcal{A}_k$ as language acceptors.
Question: Do we have $\bigcap_{i=1}^{k} L(\mathcal{A}_i) \neq \emptyset$?

Reach (Rystsov [529], Bondar and Volkov [99])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(Q, u) = S$?

ReachFrom (Rystsov [529])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta(S, u) = T$?

8.2 Reductions

Here, in Theorem 8.2, we state various $\leq_m^{\mathsf{AC}^0}$-reductions that work for any fixed alphabet. Omitting the AC^0-equivalent problems, the problems and the reductions from Theorem 8.2 are depicted in the form of a Haase diagram in Figure 8.1. Theorems 8.3, 8.4, and 8.6 give some further reductions for classes of automata. Theorem 8.5 shows

[1] Called *(global) inclusion problem for weakly-initial automata with a single final state* in [529].
[2] Called *(global) inclusion problem for non-initial automata* in [529].
[3] Called *(global) inclusion problem for weakly initial automata* in [529].

that in SyncShort, it does not matter whether we encode the length parameter in unary or binary up to $\leq_m^{\mathsf{AC}^0}$-equivalence.

Remark 8.1 The reductions in Theorems 8.2 to 8.4 involving automata do not change the alphabet. So, they can be used when we investigate problem variants where the alphabet is fixed (as in some statements in Section 8.3). Furthermore, because they do not require that the alphabet is held fixed, they can also be used for complexity upper bounds when the alphabet is part of the input.

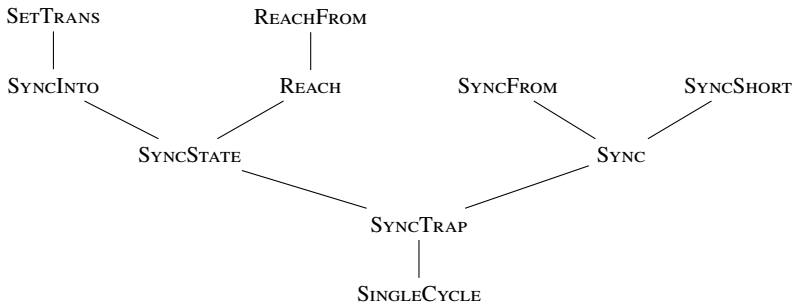


Fig. 8.1: Haase diagram of the reductions from Theorem 8.2; those equivalent to SetTrans omitted.

Proposition 8.2 *Fix a non-empty alphabet Σ. Then the following statements hold:*

1. SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap,
2. SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync,
3. SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState,
4. Sync $\leq_m^{\mathsf{AC}^0}$ SyncShort,
5. Sync $\leq_m^{\mathsf{AC}^0}$ SyncFrom,
6. SyncState $\leq_m^{\mathsf{AC}^0}$ SyncInto *and* SyncState $\leq_m^{\mathsf{AC}^0}$ Reach,
7. Reach $\leq_m^{\mathsf{AC}^0}$ ReachFrom,
8. SyncInto $\leq_m^{\mathsf{AC}^0}$ SetTrans,
9. Intersection $\equiv_m^{\mathsf{AC}^0}$ SetTrans,
10. DisjSetTrans $\equiv_m^{\mathsf{AC}^0}$ SetTrans.

Furthermore, besides preserving the alphabet, the reductions used in Items 2 to 10 transform a commutative (resp. partially ordered, solvable, permutation) automaton into a commutative (resp. partially ordered, solvable, permutation) automaton.

Proof. The following reductions given in Items 1 to 10 below can be easily implemented by L-uniform circuit families in AC^0. Further, for each reduction involving automata, the alphabet is not altered and it is readily seen that when it is applied to a commutative

(resp. partially ordered) automaton it yields a commutative (resp. partially ordered) automaton.

1: Let $g\colon [n] \to [n]$ be a input permutation to SingleCycle. Set $\mathcal{A} = ([n], \Sigma, \delta)$ where, for $x \in \Sigma$, $\delta(0, x) = 0$ and $\delta(i, x) = g(i)$ for $i \in [n] \setminus \{0\}$. Then $\mathcal{A}$ has 0 as a unique synchronizing trap state if and only if g has a single cycle.

Note that $\mathcal{A}$ has the form of a single tail with a trap state at the end if it is synchronizing. But this is not the case if g has multiple cycles, since then the automaton also has at least one cycle.

2: This is obvious.

3: We assume the n-state input $\mathcal{A}$ to SyncTrap is given as a list of transition matrices $(m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$ for each $a \in \Sigma$, and the input to SyncState is supplied as a list of transition matrices and a vector with a one indicating the state to which we want to synchronize and zero at all other positions. The state i is a trap state if and only if $m_{ii}^{(a)} = 1$ for all $a \in \Sigma$. Let

$$b_i = \left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)}\right) \wedge \left(\bigwedge_{j=1}^{i-1} \bigvee_{a\in\Sigma} \neg m_{jj}^{(a)}\right).$$

Each b_i can be computed by a circuit family in AC^0 by directly implementing the defining formula. We have $b_i = 1$ if and only if i is a trap state and the other states are not trap states. For $b_1 = \cdots = b_n = 0$ we feed a fixed non-synchronizing two-state automaton with some state to SyncState. Otherwise, $(b_1, \ldots, b_n)$ contains the entry "1" precisely once at a position corresponding to a unique trap state. Then, the transition matrices together with $(b_1, \ldots, b_n)$ are an encoding of an input instance for SyncState that is an YES-instance if and only if the original automaton is synchronizing.

4: Let $\mathcal{A}$ be an n-state input automaton to Sync. A shortest synchronizing word has length at most $(n^3 - n)/6$, see Table 1.1 in Section 1.4. So, $\mathcal{A}$ has a synchronizing word if and only if $(\mathcal{A}, n^3)$ is a YES-instance of SyncShort. The number of states n can be readily computed from the input $\mathcal{A}$ and n^3 can be easily computed by circuits constructible by deterministic log-space Turing transducers. Thus the input $(\mathcal{A}, n^3)$ can be computed by a L-uniform family of circuits in AC^0 (where the encoding of n^3 does not matter by Theorem 8.5).

5: This follows as an automaton $\mathcal{A}$ is synchronizing if and only if $(\mathcal{A}, Q)$ is a YES-instance of SyncFrom.

6: An input instance $(\mathcal{A}, q)$ of SyncState can be converted readily to an equivalent input $(\mathcal{A}, \{q\})$ to SyncInto, which is also an equivalent input for Reach.

8: The SyncInto-instance $(\mathcal{A}, S)$ is equivalent to the SetTrans-instance $(\mathcal{A}, Q, S)$, giving a reduction doable by an L-uniform circuit family in AC^0.

9: Let $\mathcal{A}_i = (Q_i, \Sigma, \delta_i)$, $q_i \in Q_i$, and $F_i \subseteq Q_i$ for $i \in \{1, \ldots, k\}$ be an input instance for Intersection. We can assume the state sets Q_i are pairwise disjoint. Set $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = Q_1 \cup \cdots \cup Q_k$ and $\delta(q, x) = \delta_i(q, x)$ if $q \in Q_i$. Then, for $u \in \Sigma^*$, we have $\delta(\{q_1, \ldots, q_k\}, u) \subseteq F_1 \cup \ldots \cup F_k$ if and only if $\delta_i(q_i, u) \in F_i$ for all $i \in \{1, \ldots, k\}$, giving a reduction preserving the alphabet and the mentioned automata types.

Next, let $(\mathcal{A}, S, T)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ be a SetTrans-instance. Write $S = \{s_1, \ldots, s_m\}$. Then, for $i \in \{1, \ldots, m\}$, with $\mathcal{A}_i = \mathcal{A}$, $q_i = s_i$, and $F_i = T$ we obtain an instance of Intersection that has a solution if and only if there exists a word in $\mathcal{A}$ mapping S into T.

10: It is sufficient to show SetTrans $\leq_m^{\mathsf{AC}^0}$ DisjSetTrans. Let $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$. We can assume $S \not\subseteq T$. Set $\mathcal{A} = (Q \cup \{0\} \times S, \Sigma, \overline{\delta})$ where, for $x \in \Sigma$, we let $\overline{\delta}((0, s), x) = \delta(s, x)$ for $s \in S$ and $\overline{\delta}(q, x) = \delta(q, x)$ for $q \in Q$. This construction can be implemented by a L-uniform circuit family in AC^0 as we only add states and transitions according to a fixed scheme. For a non-empty word $u \in \Sigma^+$ we have $\delta(S, u) \subseteq T$ if and only if $\overline{\delta}(\{0\} \times S, u) \subseteq T$ and as $\{0\} \times S$ and T are disjoint this gives a valid reduction. □

Proposition 8.3 *For automata whose synchronizing states are trap states as, for example, solvable automata, we have* Sync $\equiv_m^{\mathsf{AC}^0}$ SyncState *and* SyncState $\equiv_m^{\mathsf{AC}^0}$ SyncTrap.

Proof. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input automaton and $n = |Q|$. We assume $\mathcal{A}$ is supplied as a list of transition matrices $(m_{ij}^{(a)})_{i,j \in \{1,\ldots,n\}}$, $a \in \Sigma$, w.r.t. to an identification of the states with $\{1, \ldots, n\}$. By Theorem 8.2, Item 3, SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState. Thus, it is sufficient to show SyncState $\leq_m^{\mathsf{AC}^0}$ Sync $\leq_m^{\mathsf{AC}^0}$ SyncTrap.

SyncState $\leq_m^{\mathsf{AC}^0}$ Sync: Since a trap state must be a unique synchronizing state, with the assumption we obtain that a state q in $\mathcal{A}$ is synchronizing if and only if $\mathcal{A}$ is synchronizing and q a trap state. So, we first check whether q is a trap state by implementing the formula $\bigwedge_{a \in \Sigma} m_{qq}^{(a)}$ and, if q is a trap state, pass $\mathcal{A}$ to Sync. Otherwise, we pass some fixed NO-instance to Sync.

Sync $\leq_m^{\mathsf{AC}^0}$ SyncTrap: Since by assumption a synchronizing state is a trap state, $\mathcal{A}$ is synchronizing if and only if it has a trap state and is synchronizing. Existence of a trap state can be checked by the formula $\bigvee_{i=1}^{n} \bigwedge_{a \in \Sigma} m_{ii}^{(a)}$. If this check succeeds, we pass $\mathcal{A}$ to SyncTrap. Otherwise, we pass some fixed NO-instance to SyncTrap. □

Proposition 8.4 *For commutative input automata we have* Sync $\leq_m^{\mathsf{AC}^0}$ SyncWords.

Proof. Let $\Sigma = \{a_1, \ldots, a_k\}$ and $\mathcal{A} = (Q, \Sigma, \delta)$ be commutative. By Theorem 7.89, $\mathcal{A}$ is synchronizing if and only if $(\mathcal{A}, a_1^{n-1} \cdots a_k^{n-1})$ is a YES-instance of SyncWords. This yields an $\leq_m^{\mathsf{AC}^0}$-reduction. □

The length of a shortest synchronizing word is polynomially bounded in the number of states of the automaton, see Table 1.1 in Section 1.4. Hence, for an input instance $(\mathcal{A}, l)$ to SyncShort we can assume that l is polynomially bounded in the number of states of $\mathcal{A}$. As a consequence, as we show next, it does not matter, up to $\equiv_m^{\mathsf{AC}^0}$-equivalence, whether the length parameter is encoded in unary or binary.

Proposition 8.5 SyncShort *with length parameter encoded in thermometer or one-hot unary encoding is* $\equiv_m^{\mathsf{AC}^0}$*-equivalent to* SyncShort *with length parameter encoded in binary.*

Proof. Let $(\mathcal{A}, l)$ be an input to SyncShort with an n-state automaton $\mathcal{A}$. Suppose l is encoded in unary together with an encoding of $\mathcal{A}$. Whether it is encoded as 1^l, in one-hot, or thermometer representation with some delimiting bits to the rest of the input, similarly as done in in the proof of Theorem 3.81 we can convert it to a binary encoding of l by L-uniform AC^0-circuit families.

Next, suppose l is given in binary encoding. If $\mathcal{A}$ is synchronizing, then a shortest synchronizing word has length at most $\frac{n^3-n}{6}$, see Table 1.1 in Section 1.4. Hence, it is sufficient to consider only $O(\log(n))$ least-significant bits of the binary encoding of l. Then, by Theorem 3.81, Item 3, there exists an AC^0 circuit of size $O(n)$ computing the one-hot unary encoding of the number encoded by these bits. Inspection of the proof of Theorem 3.81, Item 3 yields that these circuits are constructible in deterministic log-space, so that we obtain a L-uniform circuit family in AC^0 converting the binary encoding to the one-hot unary encoding of l. □

By Theorem 8.5, we can use the most convenient encoding of the length parameter. In the following, we will often do this without special mentioning.

Lastly, we state an AC^0-equivalence that is valid for a fixed unary alphabet.

Proposition 8.6 *For unary input automata we have* SyncShort $\equiv_m^{\mathsf{AC}^0}$ SyncWords *which is true for the unary and the binary encoding of the length parameter in* SyncShort.

Proof. Let $\mathcal{A} = (Q, \{a\}, \delta)$ be a unary input automaton and $n = |Q|$. By Theorem 8.5, we can assume the length parameter is supplied in unary encoding.

SyncWords $\leq_m^{\mathsf{AC}^0}$ SyncShort: By Theorem 7.1, the unary automaton $\mathcal{A}$ has a synchronizing word of length at most L where $L = \max\{l_1, \ldots, l_m\}$ for $l_1, \ldots, l_m \geq 0$ if and only if it has a synchronizing word among the words $a^{l_1}, \ldots, a^{l_m}$. Thus, an input $(\mathcal{A}, a^{l_1}, \ldots, a^{l_m})$ to SyncWords is a YES-instance if and only if $(\mathcal{A}, L)$ is a YES-instance of the problem SyncShort for $L = \max\{l_1, \ldots, l_m\}$, giving an $\leq_m^{\mathsf{AC}^0}$-reduction.

SyncShort $\leq_m^{\mathsf{AC}^0}$ SyncWords: An input $(\mathcal{A}, l)$ for $l \geq 0$ is a YES-instance of SyncShort if and only if $(\mathcal{A}, a^l)$ is a YES-instance of SyncWords. If l is given in unary, we can easily transform l into the word a^l by an AC^0 circuit, which gives a reduction. □

8.3 Classes of Automata

Here, we collect results on the automata-theoretical problems introduced in Section 8.1 w.r.t. classes of automata. First, in Section 8.3.1, we recap known results from the literature and give an overview of those results from the present section that are new. Then, in Section 8.3.2, 8.3.5, and 8.3.4, we investigate general, partially ordered, and commutative automata, respectively.

We distinguish the case when Σ is part of the input and can be arbitrarily large (the case of an unbounded alphabet) from the case when the size of Σ is bounded, where in the latter case we can equivalently assume Σ is fixed and not part of the input.

8.3.1 Known Results and Overview

The following problems were proved to be PSPACE-complete for $\leq_m^{\log}$-reductions in the literature:

1. Intersection (Kozen [387]),
2. SetTrans (Rystsov [529]),
3. SyncFrom (Rystsov [529]),
4. SyncInto (Rystsov [529], Berlinkov et al. [84]),[4]
5. Reach (Rystsov [529], Bondar and Volkov [99]),
6. ReachFrom (Rystsov [529]).

Recall that, by Theorem 8.2, Intersection and SetTrans are equivalent w.r.t. our notion of AC^0-reduction.

Besides these results, the following results on classes of input automata investigated in the present thesis were obtained in the literature:

Intersection: Kozen [387, p. 261] introduced this problem and proved that it is PSPACE-complete for an unbounded alphabet; Fernau and Krebs [214, p. 2] mentioned PSPACE-completeness for a fixed binary alphabet. NP-completeness was proved for the following classes:

1. partially ordered automata even for a binary alphabet by Ryzhikov [537, Prop. 9],[5]
2. ternary partially ordered monotonic automata by Ryzhikov and Shemyakov [539, Thm. 4.1],
3. unary automata by Martens and Neven [434, Lem. 27] (using arguments that are implicitly used in [234, Thm. 2] and [584, Thm. 6.1]), Beaudry [74], and Fernau et al. [213, Thm. 4.1], where in the latter work also containment in NP was shown for commutative automata [213, Thm 4.3],
4. partially ordered commutative automata over an unbounded alphabet by Arrighi et al. [42, Lem. 17] (by a similar reduction as used by Martyugin [436, Prop. 5.3.] for SyncShort).[6]

[4] Berlinkov et al. [84] investigated the problem of totally extending a given subset and showed this problem to be PSPACE-complete. In fact, this problem is equivalent to SyncInto, see the discussion section (Section 8.4).

[5] He stated the result only for a fixed binary alphabets, but the given proof for containment in NP also works when the alphabet is part of the input.

[6] NP-hardness can be seen by the following reduction from SAT. A SAT-instance with clauses $C_1, \ldots, C_m$ over the variables $\{x_1, \ldots, x_n\}$ admits a satisfying assignment if and only if the following intersection of languages acceptable by partially ordered commutative automata is non-empty:

SyncFrom: Vorel [620, Thm. 12] proved PSPACE-completeness for binary strongly connected automata. Ryzhikov [537, Thm. 4] proved the problem to be NP-complete for binary partially ordered automata, where the proof for NP-membership also works for an unbounded alphabet.

SyncInto: Berlinkov et al. [84, Thm. 3 & 4] proved PSPACE-completeness for binary strongly connected automata.[7]

SyncShort: Eppstein [198, Thm. 8] and Rystsov [528] proved NP-completeness on binary partially ordered automata, Martyugin [436, Prop. 5.3] for partially ordered commutative automata over an unbounded alphabet by a reduction from SAT. Fernau, Heggernes and Villanger [211, Thm. 3] gave a reduction from HittingSet to SyncState that results in a partially ordered commutative automaton to show W[2]-hardness w.r.t. the length-bound parameter, but the reduction also yields NP-hardness for this class. Salomaa [546] and Goralčýk and Koubek [257] investigated more general problems and obtained NP-completeness for SyncShort as a special case. For related problems and more information, see the discussion in Section 8.4.

Sync: Holzer & Jacobi [320, Lem. 14] proved NL-completeness even for a ternary alphabet and trap-state automata. Sreejith (see Volkov [617, Prop. 2.2])[8] and Hoffmann [299, Thm. 5] gave proofs yielding NL-hardness even for a binary alphabet and trap-state automata (Theorems 8.10 and 8.13 present a simplified version of the proof from [299, Thm. 5], which is similar to the proof from [617, Prop. 2.2]); Fernau, Haase and Hoffmann [210, Thm. 5.4] stated (without proof) that Sync is L-complete for commutative automata (we will prove in Theorem 8.20 that it remains L-complete for solvable automata); L-membership was also proved in [299, Thm. 5] for unary automata.

Černý [140, Thm. 2] remarked that the pair automaton, via Theorem 7.18, can be used to decide Sync. Natarajan [457] and Eppstein [198] explicitly gave polynomial time algorithms running in cubic time, see Theorem 7.26.

We prove further results for solvable automata and some of their subclasses in Sections 8.3.2, 8.3.4, and 8.3.5. These results were first presented by the author in [299, 210, 313]. Recall that, by Items 9 and 10 from Theorem 8.2, the problems Intersection, SetTrans, and DisjSetTrans are equivalent under $\leq_m^{\mathsf{AC}^0}$-reductions. Moreover, these reductions preserve commutativity, the property of being partially ordered, and the alphabet.

Table 8.1 summarizes our results and those from the literature. We say that the alphabet is unbounded (or arbitrary) if it is part of the input. Note that every unary automaton is commutative. Moreover, hardness for a unary alphabet implies hardness

$$\left(\bigcap_{j=1}^{m} \bigcup_{y \in C_j} \Sigma^* y \Sigma^*\right) \cap \left(\bigcap_{i=1}^{n} \Sigma^* \setminus (\Sigma^* x_i \Sigma^* \overline{x}_i \Sigma^* \cup \Sigma^* x_i \Sigma^* \overline{x}_i \Sigma^*)\right)$$

[7] They formulated the problem differently in terms of totally extending a subset, see the discussion section, Section 8.4.

[8] As written by Volkov [617] the proof presented by him is originally due to the Indian computer scientist Sreejith, but was not published before and presented courtesy of the author.

for any non-empty alphabet, since additional letters can either be added as self-loops or made to act identically to the single letter.

Table 8.1: Complexity landscape for general, partially ordered (PO), commutative, and partially ordered commutative automata (POC). It is indicated whether the result holds for an unbounded (U), any fixed (F), including a unary, or any fixed alphabet containing at least two letters ($F^{\geq 2}$). Gray cells indicate known results, see Section 8.3.1.

	Automata Class						
	General	Solvable	PO	Commutative		POC	
Alphabet Σ	$F^{\geq 2}$ & U	$F^{\geq 2}$ & U	$F^{\geq 2}$ & U	F	U	F	U
Sync	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncState	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncTrap	NL-c [A]	L-c [B]	AC^0 [C]	L-c [B]	L-c [B]	AC^0 [C]	AC^0 [C]
SyncWords	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]	L-c [D]
SyncShort	NP-c [E]	NP-c [E]	NP-c	L-c [E]	NP-c [E]	L-c [E]	NP-c [E]
SyncFrom	PSPACE-c	NP-c [F]	NP-c [F]	L-c [G]	L-c [G]	L-c [G]	L-c [G]
SyncInto	PSPACE-c	L-c [B]	AC^0 [H]	L-c [B]	L-c [B]	AC^0 [H]	AC^0 [H]
SetTrans	PSPACE-c	NP-c [I]	NP-c [I]	NP-c [I]	NP-c [I]	L-c [G]	NP-c [I]
Reach	PSPACE-c	NP-c [J]	NP-c [J]	L-c [K]	NP-c [K]	L-c [K]	L-c [K]
ReachFrom	PSPACE-c	NP-c [L]	NP-c [L]	P [M]	NP-c [L]	L-c [N]	NP-c [L]

References. [A] Thm. 8.13; [B] Thm. 8.20; [C] Thm. 8.29; [D] Thm. 8.9; [E] Thm. 8.24; [F] Thm. 8.19; [G] Thm. 8.23; [H] Thm. 8.28; [I] Thm. 8.18; [J] Thm. 8.21; [K] Thm. 8.25; [L] Thm. 8.22; [M] Thm. 8.27; [N] Thm. 8.26.

8.3.2 General Automata

Here, we state and prove the following.

1. SyncWords is L-complete even for partially ordered unary input automata and a single input word (Theorems 8.7 and 8.9).
2. SyncTrap is NL-hard even for a fixed binary alphabet (Theorem 8.10). The proof is a simplified version of a construction given by Hoffmann [299, Thm. 5], which is similar to a construction from Holzer & Jacobi [320, Lem. 14] but uses a binary instead of a ternary alphabet. The proof from Volkov [617, Prop. 2.2, due to Sreejith] uses only a binary alphabet as well, and is also similar to the proof given here. In Theorems 8.11 and 8.12 we also give the related results that checking whether a state is a maximum or maximal for $\preceq_R$ in an automaton is NL-complete.
3. Sync, SyncTrap, and SyncState are all NL-complete, even for a fixed binary alphabet that is not part of the input (Theorem 8.13).
4. PSPACE-completeness of SyncFrom, SyncInto, Intersection, DisjSetTrans, SetTrans, Reach, and ReachFrom (Theorems 8.14 and 8.16).

Proposition 8.7 SyncWords *is L-hard even for a single word as parameter and unary partially ordered automata.*

Proof. We give a reduction from OrderBetweenVertices, which is L-complete by Theorem 3.76, Item 3. Let (G, v_i, v_j) be an input instance with $G = (V, E)$ and $v_i, v_j \in V$ where $V = \{v_1, \ldots, v_n\}$ and $E = \{(v_i, v_{i+1}) \mid i \in \{1, \ldots, n-1\}\}$. We can assume $i \neq j$ and $i, j \notin \{1, n\}$. Note that v_1 (resp. v_n) is easily computable as the unique vertex without any ingoing (resp. outgoing) vertices.

Construction: Set $\mathcal{A} = (V \cup \{1, \ldots, n\} \times \{v_1, v_i, v_j\}, \{a\}, \delta)$ where, for $v_r \in V$, $v_s \in \{v_1, v_i, v_j\}$, $m \in \{1, \ldots, n\}$,

$$\delta(v_r, a) = \begin{cases} v_{r+1} & \text{if } r \notin \{i, j, n\}, \\ (1, v_r) & \text{if } r \in \{i, j\}, \\ v_r & \text{if } r = n, \end{cases}$$

$$\delta((m, v_s), a) = \begin{cases} (m+1, v_s) & \text{if } m < n, \\ v_1 & \text{if } m = n \text{ and } s = 1, \\ v_i & \text{if } m = n \text{ and } s = j, \\ v_n & \text{if } m = n \text{ and } s = i. \end{cases}$$

See Figure 8.2 for a graphical depiction of $\mathcal{A}$ for both cases $i < j$ and $i > j$. Now, $\mathcal{A}$ is a unary partially ordered automaton with trap state v_n.

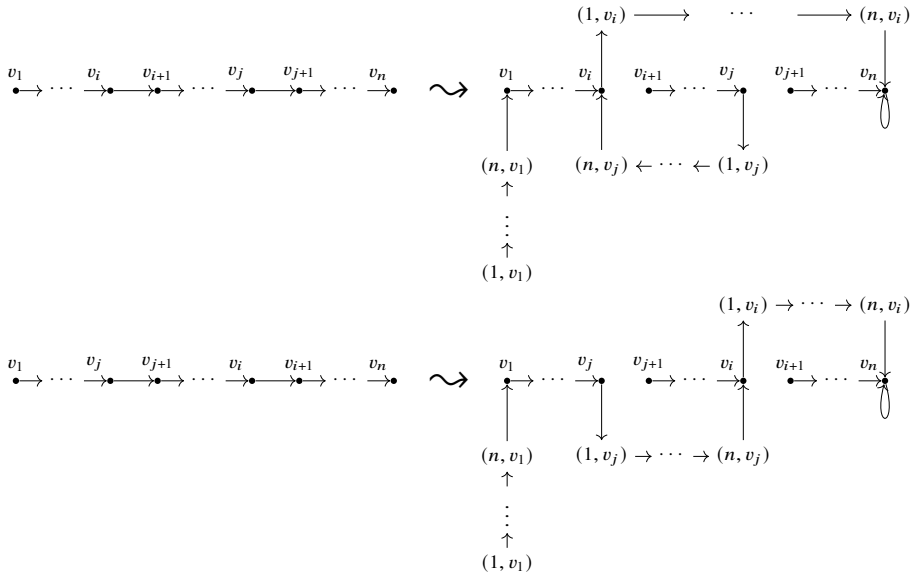


Fig. 8.2: Reduction of OrderBetweenVertices to SyncWords from the proof of Theorem 8.7 for the two cases $i < j$ (upper part), i.e., v_i appears before v_j, and $i > j$ (lower part), i.e., v_j appears before v_i, where the right hand side depicts the resulting unary automata over a single letter (which is omitted at the transitions).

Verification: We show that a^{3n-1} synchronizes $\mathcal{A}$ if and only if $i < j$. The state v_n is a trap state and so must be the synchronizing state. First, assume $j < i$. Then a shortest path in the automaton graph of $\mathcal{A}$ to v_n traverses the following vertices

$$(1, v_1), \ldots, (n, v_1), v_1, \ldots, v_j, (1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i), v_n,$$

which has length $3n + j + 1 > 3n$. So a^{3n-1} does not synchronize $\mathcal{A}$. Next, assume $i < j$. Then the following three sequences of vertices trace out three paths ending at v_n and contain all states:

$$\begin{aligned}
&(1, v_1), \ldots, (n, v_1), v_1, \ldots, v_i, (1, v_i), \ldots, (n, v_i), v_n,\\
&v_{i+1}, \ldots, v_j, (1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i), v_n,\\
&v_{j+1}, \ldots, v_n.
\end{aligned}$$

As each path has length strictly less than $3n$ the word a^{3n-1} synchronizes $\mathcal{A}$. □

Remark 8.8 We easily obtain L-hardness of SyncWords for unary automata since, by Theorems 8.2 and 8.4, we have SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync $\leq_m^{\mathsf{AC}^0}$ SyncWords, and SingleCycle is L-hard by Theorem 3.76. However, the reduction used to establish SingleCycle $\leq_m^{\mathsf{AC}^0}$ SyncTrap might result in an automaton that is not partially ordered.

It was proved that deciding whether a single word synchronizes a given automaton is in L in [299, Thm. 4]. By iterating through the given words, this also yields that SyncWords is in L. We give an independent proof of containment in L next for self-containment.

Theorem 8.9 SyncWords *is L-complete for general automata, where L-hardness holds even for a single word as parameter and commutative partially ordered automata as input over any fixed non-empty, even a unary, alphabet.*

Proof. Proposition 8.7 gives L-hardness even for a unary alphabet. Next, we describe a deterministic log-space bounded Turing machine that solves the problem for the inputs $\mathcal{A} = (Q, \Sigma, \delta)$ and $u_1, \ldots, u_r \in \Sigma^*$. Set $Q = \{q_1, \ldots, q_n\}$. The machine iterates through the words $u_1, \ldots, u_r$ and through the states, and checks whether $\delta(q, u_i)$ equals the same state for all $q \in Q$ and some $i \in \{1, \ldots, r\}$. This can be done using three counters and two memory cells in the following way. First, we iterate through $i \in \{1, \ldots, r\}$ and $j \in \{1, \ldots, n\}$ using two counters. While doing this, we compute $\delta(q_j, u_i)$ by using

1. a third counter to iterate letter-by-letter through the prefixes $u_{i,1} \cdots u_{i,l}$ for $l \in \{1, \ldots, m_i\}$ of $u_i = u_{i,1} \cdots u_{i,m_i}$ where $u_{i,1}, \ldots, u_{i,m_i} \in \Sigma$ and
2. one memory cell to store $\delta(q_j, u_{i,1}, \ldots, u_{i,l-1})$ for $l \geq 1$ that is initialized with q_j and updated for each letter $u_{i,l}$ to $\delta(q_j, u_{i,1}, \ldots, u_{i,l-1}u_{i,l})$.

Then the memory cell stores $\delta(q_j, u_i)$ when $l = m_i$. Furthermore, another memory cell stores $\delta(q_{j-1}, u_i)$ for $j \geq 2$ to check equality of $\delta(q_{j-1}, u_i)$ and $\delta(q_j, u_i)$ for each $j \in \{2, \ldots, n\}$. The word u_i synchronizes $\mathcal{A}$ if and only if all of the equality tests for $\delta(q_{j-1}, u_i)$ and $\delta(q_j, u_i)$ pass. The machine operates in logarithmic space by using a binary encoding for the counters. □

Proposition 8.10 *Let $\Sigma = \{a, b\}$ be a fixed binary alphabet. Then* SyncTrap *is* NL*-hard.*

Proof. We give a reduction from 2-DAG-ST.

Construction: Let (G, s, t) with $G = (V, E)$ be an instance of 2-DAG-ST. For each $q \in V \setminus \{s, t\}$, denote by $q^R, q^L \in V$ vertices such that, if $(\{q\} \times V) \cap E$ is non-empty, we have $(\{q\} \times V) \cap E = \{(q, q^R), (q, q^L)\}$, and $q^R = q^L = s$ for $(\{q\} \times V) \cap E = \emptyset$. Next, set $\mathcal{A} = (Q, \Sigma, \delta)$ with $Q = (V \setminus \{s, t\}) \times \{\varepsilon, b\} \cup \{(s, \varepsilon), (t, \varepsilon)\}$ where, for $(q, u) \in Q \setminus \{(s, \varepsilon), (t, \varepsilon)\}$ and $x \in \Sigma$,

$$\delta((q,u),x) = \begin{cases} (q^R, \varepsilon) & \text{if } ux = a, \\ (q^L, \varepsilon) & \text{if } ux = ba, \\ (q, b) & \text{if } ux = b, \\ (s, \varepsilon) & \text{otherwise,} \end{cases}$$

and $\delta((t, \varepsilon), x) = (t, \varepsilon)$ for all $x \in \Sigma$, $\delta((s, \varepsilon), a) = (q, \varepsilon)$ for the unique $q \in V$ with $(s, q) \in E$ and $\delta((s, \varepsilon), b) = (s, \varepsilon)$. See Figure 8.3 for an example.

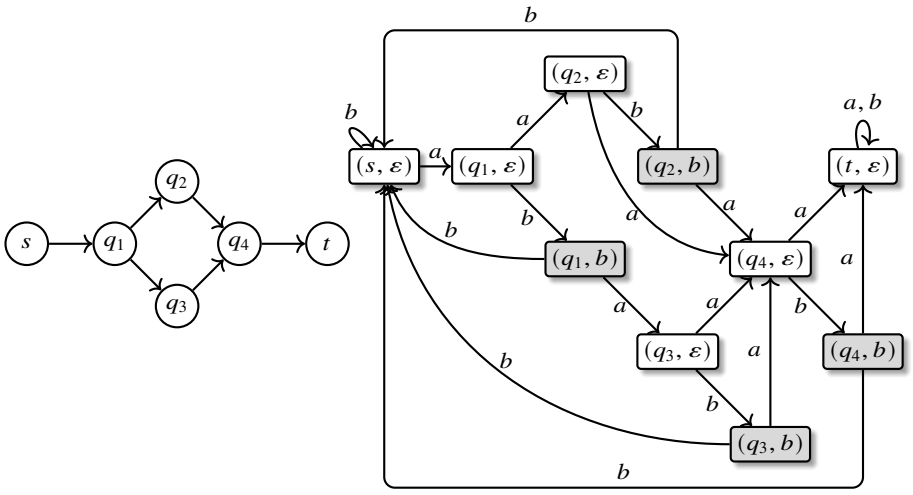


Fig. 8.3: Example for the reduction used in Theorem 8.10. Left: Input graph. Right: Transformation into an automaton having a synchronizing word if and only if there exists a path from s to t in the input graph (states from $V \times \{b\}$ are colored grey).

Properties: The following two properties are readily seen.

1. $\delta(Q \setminus \{(t, \varepsilon)\}, bb) = \{(s, \varepsilon)\}$ and $\delta((t, \varepsilon), bb) = (t, \varepsilon)$.
2. For $q \in V \setminus \{s, t\}$ we have $\delta((q, \varepsilon), a) = (q^R, \varepsilon)$ and $\delta((q, \varepsilon), ba) = (q^L, \varepsilon)$.

Verification: First, suppose there exists a path $q_0, q_1, \ldots, q_n$ with $q_0 = s$ and $q_n = t$ in G. Define a word $u = u_1 u_2 \cdots u_n$ where $u_1 = a$ and, for $i \in \{2, \ldots, n\}$,

$$u_i = \begin{cases} a & \text{if } q_i = q_{i-1}^R, \\ ba & \text{if } q_{i-1}^R \neq q_{i-1}^L \text{ and } q_i = q_{i-1}^L. \end{cases}$$

By Properties 1 and 2, $\delta(Q, bbu_1 \cdots u_i) = \{(q_i, \varepsilon), (t, \varepsilon)\}$. So $\delta(Q, bbu) = \{(t, \varepsilon)\}$.

Conversely, suppose the word $u \in \Sigma^*$ synchronizes $\mathcal{A}$. As (t, ε) is a trap state, we have $\delta(Q, u) = \{(t, \varepsilon)\}$. Let w be a shortest factor of u such that $\delta((s, \varepsilon), w) = (t, \varepsilon)$. By Property 1 we have $w \notin \Sigma^* bb\Sigma^*$, i.e., $w \in \{a, ba\}^* \cup \{a, ba\}^* b$. As we can enter the state (t, ε) from another state only via the letter a, we have $w \in \{a, ba\}^*$. Let $w = w_1 \cdots w_n$ with $w_1, \ldots, w_n \in \{a, ba\}^*$. Then, by Property 2, the vertices

$$(s, \varepsilon), \delta(s, w_1), \delta(s, w_1 w_2) \ldots \delta(s, w_1 w_2 \cdots w_{n-1}), (t, \varepsilon)$$

trace out a path in G from s to t. □

Proposition 8.11 *Given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ it is* NL*-complete to test whether q is a maximum w.r.t. the reachability quasiorder $\preceq_R$.*

Proof. First, we give an $\leq_m^{\mathsf{AC}^0}$-reduction from SyncTrap, which yields NL-hardness by Proposition 8.10. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance to SyncTrap. Suppose $t \in Q$ is the trap state, which can be computed by an AC^0-circuit as in the proof of Theorem 8.2, Item 3. So, the reduction produces $\mathcal{A}$ together with the trap state t and $(\mathcal{A}, t)$ is a Yes-instance of our problem if and only if $\mathcal{A}$ is synchronizing, since the state t is a maximum in $\mathcal{A}$ if and only if $\mathcal{A}$ is synchronizing by Theorem 7.6.

For containment in NL, suppose $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ are given. To test whether q is a maximum for $\preceq_R$, we guess, for each state $p \in Q$, a word $u \in \Sigma^{<|Q|}$ letter by letter and check whether $\delta(p, u) = q$; a property that characterizes that q is a maximum by Theorem 7.12. This can be done in nondeterministic log-space by storing a state and using two counters, one to iterate through all states $p \in Q$ and the other to make at most $|Q|$ guessing steps for each $p \in Q$. □

Proposition 8.12 *Given an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $q \in Q$ it is* NL*-complete to test whether q is maximal w.r.t. the reachability quasiorder $\preceq_R$.*

Proof. We give a reduction from the NL-complete problem 2-DAG-ST. Let $G = (V, E)$ and $s, t \in V$ be an input instance. Construct an automaton $\mathcal{A} = (Q, \Sigma, \delta)$ as in the proof of Theorem 8.10, but instead of making (t, ε) a trap state, let both letters a and b map back to the state (s, ε). Then t is a maximal state (and $\mathcal{A}$ is strongly connected) if and only if there exists a path from s to t in G.

To test maximality, we have to check that for each state $p \in Q$ with $\delta(q, u) = p$ for some $u \in \Sigma^*$ there exists $v \in \Sigma^*$ with $\delta(p, v) = q$. To verify this condition, we can pick words u, v of length at most $|Q|$ and proceed similarly as in the proof of Theorem 8.11 to obtain NL-membership. □

Similar procedures as in Theorems 8.11 and 8.12 can be used to check whether a state is minimal or a minimum, but we do not need to check these properties in the following and hence give no formal statement. We also note that we obtain L-completeness of these problems in case the input automata are unary automata since we only needed nondeterminism when guessing different letters in the above proofs.

Theorem 8.13 *The problems* Sync, SyncTrap *and* SyncState *are* NL*-complete, even for a fixed binary alphabet.*

Proof. By Proposition 8.2 we have SyncTrap $\leq_m^{\mathsf{AC}^0}$ SyncState and SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync. By Theorem 8.10, SyncTrap is NL-hard for an alphabet containing at least two letters. Sync is in NL, see Section 8.3.1 (page 232). For $\mathcal{A} = (Q, \Sigma, \delta)$, by Theorem 7.11, a state $q \in Q$ is a synchronizing state if and only if $\mathcal{A}$ is synchronizing and q is a maximum for the reachability order $\preceq_R$. The latter two properties can be checked in NL since Sync is in NL and by Theorem 8.11, respectively. So, SyncState is in NL. □

Proposition 8.14 SetTrans *is in* PSPACE.

Proof. Let $(\mathcal{A}, S, T)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ and $S, T \subseteq Q$ where $S = \{s_1, \ldots, s_m\}$ and $n = |Q|$. Store the values $s_1, \ldots, s_m$ in m memory cells and then repeat the following steps at most n^m times: If the memory cells store a subset of T, then abort and return YES, i.e., classify this input instance as a YES-instance of the decision problem. Otherwise guess a letter $a \in \Sigma$ and update the memory cells with $\delta(s_1, a), \ldots, \delta(s_m, a)$. If this procedure finishes without ever writing a subset of T into the memory cells, then return NO. The upper bound for the number of steps can be ensured by utilizing an $(m \cdot \log n)$-bit counter counting the number of steps in binary encoding. After at most n^m steps we have seen at least one memory cell configuration twice. So, if there exists a word mapping S into T, then there exists such a word of length at most n^m, which implies correctness of the algorithm. As it runs in nondeterministic polynomial space, and as PSPACE = NPSPACE by Savitch's theorem [550], the claim follows. □

The following lemma is a technical result that we need to reduce an arbitrary alphabet to a binary alphabet in our subset mapping problems. Another reduction to reduce the alphabet for synchronization problems has been given in [439, Sec. 5] and [438, Prop. 2].

Lemma 8.15 *Let* $\mathcal{A} = (Q, \Sigma, \delta)$ *with* $\Sigma = \{a_1, \ldots, a_k\}$. *Set* $C = \{0^k 10^i 1 \mid i \in \{0, 1, \ldots, k-1\}\}$ *and* $\mathcal{B} = (Q \times \mathrm{Pref}(C) \setminus C, \{a, b\}, \mu)$ *where, for* $q \in Q$, $u \in \mathrm{Pref}(C) \setminus C$, *and* $x \in \{a, b\}$, *we have*

$$\mu((q,u),x) = \begin{cases} (\delta(q,a_i),\varepsilon) & \text{if } ux = 0^k 10^{i-1}1 \text{ for some } i \in \{1,\ldots,k\}, \\ (q,ux) & \text{if } ux \in \mathrm{Pref}(C) \setminus C, \\ (q,ux) & \text{if } u \in \mathrm{Pref}(0^{k-1}) \text{ and } x = 0, \\ (q,0^k) & \text{if } u = 0^k \text{ and } x = 0, \\ (q,0^k) & \text{if } u = 0^k 10^{k-1} \text{ and } x = 0, \\ (q,\varepsilon) & \text{otherwise.} \end{cases}$$

Then $\mathcal{A}$ *is strongly connected if and only if* $\mathcal{B}$ *is strongly connected and, for* $i \in \{1, \ldots, k\}$, $q \in Q$, $u \in \mathrm{Pref}(C) \setminus C$, *we have*

$$\mu((q,u), 0^k 10^{i-1}1) = (\delta(q,a_i),\varepsilon), \tag{8.1}$$

and, for distinct $S, T \subseteq Q$, *we have* $\delta(S,u) = T$ *for some* $u \in \Sigma^*$ *if and only if* $\mu(S \times \mathrm{Pref}(C) \setminus C, v) = T \times \{\varepsilon\}$ *for some* $u \in \{a,b\}^*$.

Proof. That $\mathcal{A}$ is strongly connected if and only if $\mathcal{B}$ is strongly connected is easy to see. Equation (8.1) is also easy to verify.

Next, suppose $\delta(S, u) = T$ for some $u \in \Sigma^*$. Since $S \neq T$ we have $|u| > 0$. Let $v_j = 0^k 10^{i-1}1$ if the j-th symbol in u equals a_i for $i \in \{1, \ldots, k\}$. By Equation (8.1) we have $\mu(S \times \mathrm{Pref}(C) \setminus C, v_1 \cdots v_{|u|}) = (T, \varepsilon)$.

Conversely, suppose $\mu(S \times \mathrm{Pref}(C) \setminus C, v) = T \times \{\varepsilon\}$ for some $u \in \{a, b\}^*$. Let $q \in S$ and $u \in \mathrm{Pref}(C) \setminus C$. The language C has the property that there exist unique words $u_1, \ldots, u_m \in C$ and $v_0, \ldots, v_m \in \Sigma^*$, each v_j containing no factor from C, such that

$$uv = v_0 u_1 v_1 \cdots u_m v_m.$$

For each $j \in \{1, \ldots, m\}$, set $x_j = a_i$ if $u_j = 0^k 10^{i-1}1$ for some $i \in \{1, \ldots, k\}$. By induction, using Equation (8.1), we obtain

$$\mu((q, u), v) = (\delta(q, x_1 \cdots x_m), \varepsilon).$$

Hence $\delta(S, x_1 \cdots x_m) = T$. □

Theorem 8.16 *The following problems are* PSPACE*-complete w.r.t.* $\leq_m^{\log}$*-reductions:*

1. SyncFrom *and* SyncInto *even on binary strongly connected automata,*
2. Intersection, SetTrans, *and* DisjSetTrans *even on binary strongly connected automata,*
3. Reach *and* ReachFrom *even for binary automata.*

Proof. Membership of these problems in PSPACE was proved in the literature, see Section 8.3.1 (page 232). As an example, we included a proof for SetTrans in Theorem 8.14.

PSPACE-hardness of SyncFrom and SyncInto for binary strongly connected automata was also mentioned in Section 8.3.1 (page 232).

By Theorem 8.2 we have

$$\textsc{SyncInto} \leq_m^{\mathsf{AC}^0} \textsc{SetTrans} \leq_m^{\mathsf{AC}^0} \textsc{Intersection}$$

and the reductions from Theorem 8.2 preserve strong connectedness and the alphabet. Thus, the claim for SetTrans and Intersection follow.

The reduction showing $\textsc{SetTrans} \leq_m^{\mathsf{AC}^0} \textsc{Intersection}$ from Item 9 in Theorem 8.2 does not preserve strong connectedness and the alphabet. However, it can easily be adapted by adding a new letter that swaps two states q and $(q, 0)$ with $q \in S$ and induces a self-loop at every other state not in S. Then the resulting automaton is strongly connected and this gives a valid reduction. With Theorem 8.15 we then obtain PSPACE-hardness even for binary strongly connected automata. This proves the claim for Intersection, SetTrans, and DisjSetTrans.

Reach was shown to be PSPACE-hard in [529, Thm. 3.2] and [99, Thm. 3] by reductions from SyncInto and Intersection, respectively, that add a single new letter. Neither of these reductions preserves strong connectedness. By Theorem 8.15 we obtain a reduction to a binary alphabet. Since $\textsc{Reach} \leq_m^{\mathsf{AC}^0} \textsc{ReachFrom}$ by Theorem 8.2 where

the reduction preserves the alphabet, we obtain the hardness claims for Reach and ReachFrom. □

8.3.3 Solvable Automata

Here, we show the following:

1. SetTrans, DisjSetTrans, and Intersection are NP-complete for solvable input automata (Theorem 8.18).
2. SyncFrom is NP-complete for solvable automata (Theorem 8.19).
3. SyncTrap, SyncState, Sync, and SyncInto are L-complete for solvable automata even for a unary alphabet (Theorem 8.20).
4. ReachFrom and Reach are NP-complete for solvable automata in (Theorems 8.21 and 8.22).

For self-containment, we give a proof of NP-hardness of Intersection for unary permutation automata. The proof is similar to the one given in [434, Lem. 27], but uses automata instead of regular expressions.

Proposition 8.17 Intersection *is* NP*-hard for unary permutation automata.*

Proof. We give a reduction from 3SAT, which is NP-complete by Theorem 3.76. Let $C_1, \dots, C_m \subseteq \{x_1, \overline{x}_1, \dots, x_n, \overline{x}_n\}$ be the clauses of an 3SAT-instance over the variables $x_1, \dots, x_n$. We can assume every variable appears, as a positive or negative literal, in some clause.

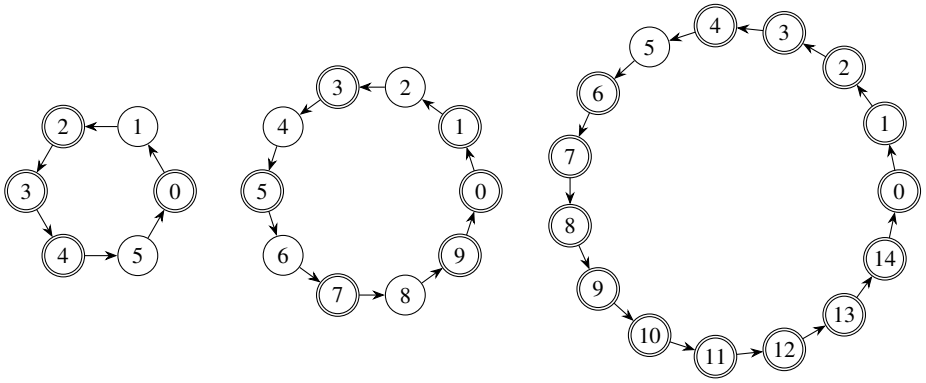
Construction: Let $p_1 < p_2 < \cdots < p_n$ be the first n prime numbers. For each clause C_j, $j \in \{1, \dots, m\}$, we construct $\mathcal{A}_j = (Q_j, \{a\}, \delta_j, q_j, F_j)$ with

$$\begin{aligned} Q_j &= \{0, 1, \dots, N_j - 1\}, \\ q_j &= 0, \\ F_j &= \{q \in Q_j \mid \exists i \in \{1, \dots, n\} : (x_i \in C_j \wedge q \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge q \not\equiv_{p_i} 0)\}, \\ \delta_j(q, a) &= (q + 1) \bmod N_j \end{aligned}$$

where $N_j = \left(\prod_{\substack{i \in \{1,\dots,n\} \\ x_i \in C_j \vee \overline{x}_i \in C_j}} p_i \right)$, i.e., we multiply those primes that correspond to the variables in C_j.

By Theorem 3.2 there exists $N \geq 1$ such that $p_n < n^2$ for all $n \geq N$. We can assume $p_n < n^2$ in the following. Since $|C_j| \leq 3$ we get $|Q_j| \leq n^6$. Then, as detecting primality for unary numbers is in L [281, Thm. 7], the automata $\mathcal{A}_j$ can be constructed in deterministic log-space.

Figure 8.4 shows the construction for the clauses $C_1 = \{x_1, x_2\}$, $C_2 = \{\overline{x_1}, x_3\}$, and $C_3 = \{x_2, \overline{x_3}\}$ (shown in their logical interpretation in the figure). If $x_1 = 1$, then $x_3 = 1$ to fulfill C_2 and thus $x_2 = 1$ to fulfill C_3. For $x_1 = 0$, we must have $x_2 = 1$ to fulfill C_1



$\mathcal{A}_1$ for $C_1 = x_1 \vee x_2$ $\mathcal{A}_2$ for $C_2 = \overline{x_1} \vee x_3$ $\mathcal{A}_3$ for $C_3 = x_2 \vee \overline{x_3}$

Fig. 8.4: Example for the proof of Theorem 8.17. States with label 0 are start states. Final states are indicated by double circles. The label a at the transitions is omitted.

and x_3 can be arbitrary. Thus we find three satisfying assignments that correspond to the words a^{30}, a^3, and a^{15}.

Properties: We have the following two properties:

1. $\delta_j(q_j, a^r) \equiv_{N_j} r$ for each $r \geq 0$.
2. $(b_1, \ldots, b_n)$ is a satisfying assignment if and only if $a^{\prod_{i=1}^n p_i^{b_i}} \in \bigcap_{j=1}^m L(\mathcal{A}_j)$.

Property 1 follows from the construction of $\mathcal{A}_j$ whose automaton graph forms a single cycle. Property 2 follows since for $(b_1, \ldots, b_n) \in \{0,1\}^n$, $l = \prod_{i=1}^n p_i^{b_i}$, $j \in \{1, \ldots, m\}$, and $s_j = \delta_j(q_j, a^l)$ we have $s_j \equiv l \pmod{N_j}$ and then

$$
\begin{aligned}
&\exists i \in I\colon ((x_i \in C_j \wedge b_i = 1) \vee (\overline{x}_i \in C_j \wedge b_i = 0)) \\
&\Leftrightarrow \exists i \in I\colon ((x_i \in C_j \wedge l \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge l \not\equiv_{p_i} 0)) && \text{(Def. } l\text{)} \\
&\Leftrightarrow \exists i \in I\colon ((x_i \in C_j \wedge s_j \equiv_{p_i} 0) \vee (\overline{x}_i \in C_j \wedge s_j \not\equiv_{p_i} 0)) && \text{(Eqn. (3.1), Property (1))} \\
&\Leftrightarrow s_j \in F_j && \text{(Def. } F_j\text{)}
\end{aligned}
$$

where $I = \{1, \ldots, n\}$.

Verification: By Property 2, if there exists a satisfying assignment, then we have $\bigcap_{j=1}^m L(\mathcal{A}_j) \neq \emptyset$. Conversely, suppose $a^l \in \bigcap_{j=1}^m L(\mathcal{A}_j)$ for some $l \geq 0$. Define an assignment $(b_1, \ldots, b_n)$ to the variables by setting, for $i \in \{1, \ldots, n\}$,

$$
b_i = \begin{cases} 1 & \text{if } p_i \text{ divides } l, \\ 0 & \text{if } p_i \text{ does not divide } l. \end{cases}
$$

Let $l' = \prod_{i=1}^n p_i^{b_i}$. Then $l' \equiv_{p_i} 0$ if and only if $l \equiv_{p_i} 0$ for $i \in \{1, \ldots, n\}$. So $a^{l'} \in L(\mathcal{A}_j)$ if and only if $a^l \in L(\mathcal{A}_j)$ for $j \in \{1, \ldots, m\}$. Hence, by Property 2, we obtain that $(b_1, \ldots, b_n)$ is a satisfying assignment. □

Theorem 8.18 SetTrans, DisjSetTrans, *and* Intersection *are* NP*-complete for solvable input automata, where* NP*-hardness holds even for the following classes of input automata:*

1. *binary partially ordered input automata,*
2. *ternary partially ordered monotonic automata,*
3. *commutative partially ordered automata over an unbounded alphabet,*
4. *unary permutation automata.*

Proof. That Intersection is NP-complete for the mentioned classes follows from results in the literature mentioned in Section 8.3.1, page 231. By Items 9 and 10 from Theorem 8.2 we obtain NP-hardness for all three problems, and by these statements it is sufficient to establish NP-membership for SetTrans, which we do in the following.

Let $\Sigma = \{a_1, \ldots, a_k\}$ and $\mathcal{A} = (Q, \Sigma, \delta)$ be a solvable input automaton to SetTrans with n states and input sets $S, T \subseteq Q$. If there exists a word mapping S into T, by Proposition 7.114 there exists a word of the form

$$v = v_0 x_1 \cdots v_{r-1} x_r v_r$$

for some $r \leq n^2$ where $x_1, \ldots, x_r \in \Sigma$ and $v_i = a_1^{r_{i,1}} \cdots a_k^{r_{i,k}}$ for numbers $0 \leq r_{i,j} < 2^n$ with $i \in \{0, 1, \ldots, r\}$ and $j \in \{1, \ldots, k\}$ mapping S into T. By using a binary encoding for the exponents $0 \leq r_{i,j} < 2^n$, a word of the above form can be stored using $O(|\Sigma|n \cdot (n^2+1) + \log(|\Sigma|)n^2)$ bits where each of the n^2+1 words v_i, $i \in \{0, 1, \ldots, r\}$, takes up $|\Sigma|n$ bits and each letter x_i, $i \in \{1, \ldots, r\}$, takes $\log(|\Sigma|)$ bits. So, using this encoding, it can be guessed in polynomial time. Next, the transition matrix of $a_j^{r_{i,j}}$ for $0 \leq r_{i,j} < 2^n$ can be computed in polynomial time by a square-and-multiply algorithm (also known as exponentiation by squaring) using the transition matrix of a_j. Hence, the transition matrix of v can be computed in polynomial time by multiplying the transition matrices of $v_0, x_1, \ldots, v_{r-1}, x_r$, and v_r in this order. Then v maps S into T if and only if each row in the transition matrix of v corresponding to a state in S has precisely one 1 at a column corresponding to a state in T. So, the above algorithm shows containment in NP. □

Theorem 8.19 SyncFrom *is* NP*-complete for solvable input automata even for binary partially ordered automata.*

Proof. Ryzhikov [537, Thm. 4] proved the problem to be NP-complete for partially ordered binary automata, see also page 232 in Section 8.3.1.

For NP-membership, we can guess a state $q \in Q$ in an n-state solvable input automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and then test whether there exists a word mapping Q to $\{q\}$. As the latter problem is in NP by Theorem 8.18, we obtain that SyncFrom is in NP. NP-hardness follows as SyncFrom is NP-complete for binary partially ordered automata as proved in [537, Thm. 4], see also Table 8.1. □

Theorem 8.20 SyncTrap, SyncState, Sync, *and* SyncInto *are* L*-complete for solvable input automata, even for a fixed unary alphabet.*

Proof. By Theorem 8.2 we have

$$\textsc{SingleCycle} \leq_m^{\mathsf{AC}^0} \textsc{SyncTrap} \leq_m^{\mathsf{AC}^0} \textsc{SyncState} \leq_m^{\mathsf{AC}^0} \textsc{SyncInto},$$

and $\textsc{SyncTrap} \leq_m^{\mathsf{AC}^0} \textsc{Sync}$, where the reductions of an automata-theoretical problem to another automata-theoretical problem given in the proof of Theorem 8.2 preserve the alphabet. Since SingleCycle is L-complete by Theorem 3.76, and the reduction to SyncTrap works even for unary automata, we obtain the L-hardness claims even for a fixed unary alphabet.

Since the reductions from Theorem 8.2 transforming automata preserve the property of being solvable, by the reductions from above, it is sufficient to prove L-membership for Sync and SyncInto to establish it for the other problems as well.

For Sync, to test synchronizability, by Theorem 7.113 we can simply test whether the word $(a_1 \cdots a_k)^{n-1}$ for $\Sigma = \{a_1, \ldots, a_k\}$ synchronizes the n-state solvable input automaton, which can be done in deterministic log-space by Theorem 8.9.

However, this is not sufficient since we cannot simply construct the word as this would take up more than logarithmic space, i.e., having a deterministic log-space Turing machine deciding whether a word synchronizes a given automaton, we cannot simply write $(a_1 \cdots a_k)^{n-1}$ and an encoding of the automaton on its input tape and then simulate it, as this would take up more than logarithmic space. But we can simulate the machine by letting it read the input automaton from the original input tape and feeding $(a_1 \cdots a_k)^{n-1}$ one symbol at a time to it in log-space without storing the full word similarly as closure of L is proved for log-space transducers, see [322, Lem. 13.2]; in fact, a log-space transducer that additionally produces $(a_1 \cdots a_k)^{n-1}$ can also be given to obtain the claim.

To solve SyncInto for $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$, by Theorem 7.103, Item 6 and Theorem 7.48 we have to test whether S contains all sink components of $\mathcal{A}$. We can do this by iterating through all states $q \in Q$, test whether it is contained in a sink component and if this is the case, test whether $q \in S$. By Theorem 7.50, a state $q \in Q$ is in a sink component if and only if for all $x \in \Sigma$ we have $\delta(q, x^i) = q$ for some $i \in \{1, \ldots, |Q|\}$. This can be checked in deterministic log-space by successively applying $|Q|$ times — using a counter that counts up to $|Q|$ and so takes up at most logarithmic space via a binary encoding — the letter x to a state stored in a memory cell initialized with q. Then it is easy to give an algorithm running in deterministic log-space. □

Theorem 8.21 Reach *is* NP*-complete for solvable automata even for partially ordered binary automata.*

Proof. The problem is NP-complete for partially ordered binary automata [537, Thm. 5]. Furthermore, as in the proof of Theorem 8.18 by using Proposition 7.114 we obtain NP-membership for solvable automata. □

Theorem 8.22 ReachFrom *is* NP*-complete for solvable automata even for commutative partially ordered automata, and also even for partially ordered binary automata.*

Proof. NP-hardness for partially ordered binary automata follows from Theorem 8.21 and Item 7 from Theorem 8.2. Next, in [73, Thm. 9] it has been shown that deciding for commutative partially ordered $\mathcal{A} = (Q, \Sigma, \delta)$ and $t \colon Q \to Q$ whether there exists

$u \in \Sigma^*$ such that $\delta(q, u) = t(q)$ for all $q \in Q$ is NP-complete.[9] This problem can be reduced to Intersection by letting $Q = \{q_1, \ldots, q_n\}$ and $\mathcal{A}_i = \mathcal{A}$ and $F_i = \{t(q_i)\}$ for $i \in \{1, \ldots, n\}$. Then when applying the reduction from Theorem 8.2, Item 9 to obtain an instance of SetTrans from this Intersection-instance the resulting automaton is a commutative partially ordered automaton $\mathcal{A}' = (Q', \Sigma, \delta')$ where $\delta'(\{q_1, \ldots, q_n\}, u) \subseteq F_1 \cup \ldots \cup F_n$ if and only if $\delta'(\{q_1, \ldots, q_n\}, u) = F_1 \cup \ldots \cup F_n$ for $u \in \Sigma^*$, i.e., we actually have a reduction to ReachFrom.

As in the proof of Theorem 8.18, by using Proposition 7.114, we obtain that ReachFrom is in NP for solvable automata. □

8.3.4 Commutative Automata

Here, we show the following:

1. Intersection, DisjSetTrans, and SetTrans are L-completeness for partially ordered commutative automata over a fixed non-empty alphabet (Theorem 8.23). Recall that in Section 8.3.3 we have shown that these problems are NP-complete for partially ordered commutative automata over an unbounded alphabet.
2. SyncFrom is L-complete for commutative automata (Theorem 8.23) even for unary partially ordered automata. Recall that this problem is NP-complete for solvable automata by Theorem 8.19.
3. SyncShort is L-complete for commutative automata over a fixed alphabet and NP-complete for an unbounded alphabet (Theorem 8.24). Hardness remains in both cases even for partially ordered commutative automata.

In this section, we assume an ordering $\Sigma = \{a_1, \ldots, a_k\}$ of each input alphabet Σ with $k \geq 1$ (the case $\Sigma = \emptyset$ is trivial and most problems are solvable in constant time).

Theorem 8.23 *1.* SyncFrom *is L-complete for commutative automata over an unbounded alphabet, where L-hardness holds even for unary partially ordered automata.*

2. SetTrans, DisjSetTrans, Intersection *are L-complete for partially ordered commutative automata over a fixed non-empty alphabet even for an alphabet of size one.*

Proof. By Theorem 8.2, where the reductions preserve the properties of commutativity, partial orderness, and the alphabet, it is sufficient to show the second claim for SetTrans.

For hardness, we give a reduction from 1-DAG-ST that works for SyncFrom and SetTrans, and also even if we fix k to be equal to one. For an 1-DAG-instance (G, s, t) with $G = (V, E)$, let $\mathcal{A} = (V, \Sigma, \delta)$ where, for $p \in V$ and $x \in \Sigma$,

[9] In [73] this problem was formulated as a problem on transformation monoids, where the commutative aperiodic transformation monoids used correspond precisely to the commutative partially ordered automata.

$$\delta(p,x) = \begin{cases} q & \text{if } (p,q) \in E \text{ for some } q \in V, \\ p & \text{otherwise.} \end{cases}$$

We have the following:

1. $\mathcal{A}$ is a well-defined deterministic and complete automaton as for every $p \in V$ there exists at most one $q \in Q$ such that $(p,q) \in E$.
2. $\mathcal{A}$ is a partially ordered commutative automaton.
3. t is a trap state in $\mathcal{A}$ since it has no outgoing transitions in G.

Now, the following are equivalent:

1. there exists a path from s to t in G,
2. $\delta(\{s\}, u) = \{t\}$ for some $u \in \Sigma^*$,
3. $|\delta(\{s,t\}, u)| = 1$ for some $u \in \Sigma^*$,

where the latter two properties are equivalent as t is a trap state and so must be the synchronizing state. By these equivalences, the reduction works for the problems SetTrans and SyncFrom. Thus these problems are L-hard even for $|\Sigma| = 1$ by Item 3 from Theorem 3.76.

Next, we show membership in L. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be a partially ordered commutative automaton with n states.

SyncFrom: Let $v = a_1^{n-1} \cdots a_k^{n-1}$. By Theorem 7.89, there exists $u \in \Sigma^*$ such that $|\delta(S,u)| = 1$ if and only if $|\delta(S,v)| = 1$.

To test whether $|\delta(S,v)| = 1$, we iterate through the states $S = \{s_1, \ldots, s_m\}$. For each state s_i, we compute $\delta(s_i, v)$ by processing the letters of v one by one and, for each letter a_j, applying it repeatedly $n-1$ times. We then check whether all computed values $\delta(s_i, v)$ are equal.

This procedure requires three counters: one to iterate over the states of S, one to iterate over the letters $a_1, \ldots, a_k$, and one to count the repetitions $0, \ldots, n-1$ for each letter. In addition, two memory cells suffice: one for the current state reached during the computation of $\delta(s_i, v)$, and one for storing the result of the previous iteration in order to compare $\delta(s_i, v)$ with $\delta(s_{i-1}, v)$ for $i > 1$.

Using binary encodings for the counters and for the stored states, the total space needed is logarithmic.

SetTrans: Let $S, T \subseteq Q$ and suppose, unlike in the proof of SyncFrom, that Σ is fixed. We iterate through all tuples $(n_1, \ldots, n_k)$ satisfying $0 \leq n_j < n$ for all $j \in \{1, \ldots, k\}$. Each tuple can be stored using k counters, which require $k \cdot \log n$ space when encoded in binary.

For each tuple $(n_1, \ldots, n_k)$, let $w = a_1^{n_1} \cdots a_k^{n_k}$. We then iterate through all states $q \in S$ and compute $\delta(q, w)$. If $\delta(q, w) \in T$ for every $q \in S$, we return YES. Otherwise, we continue with the next tuple. If the tuple $(n-1, \ldots, n-1)$ is reached without finding a word w such that $\delta(q, w) \in T$, we return NO.

The computation of $\delta(q, w)$ can be done in logarithmic space. Besides the k counters storing $(n_1, \ldots, n_k)$, we use a counter to iterate through the states in S, a memory cell storing the current state, a counter j going from 1 to k to keep track fo the current letter a_j, and a counter r to feed in the factor $a_j^{n_j}$ of $a_1^{n_1} \cdots a_k^{n_k}$. All of these require only $O(\log n)$ space.

Correctness follows from Equation (7.6) of Corollary 7.61, which implies that whenever there exists a word mapping S into T, there also exists a word of the form $a_1^{n_1} \cdots a_k^{n_k}$ with $0 \leq n_j < n$ for all $j \in \{1, \ldots, k\}$. □

Theorem 8.24 SyncShort *is* NP*-complete even for partially ordered commutative input automata. For a fixed non-empty alphabet, the problem is* L*-complete for commutative input automata even for unary partially ordered automata.*

Proof. NP-completeness: Martyugin [436, Prop. 5.3] proved NP-hardness for commutative automata over an unbounded alphabet, where the reduction resulted in a partially ordered commutative automaton. SyncShort is in NP for general input automata. See also the discussion in Section 8.3.1 (page 232).

L-hardness for fixed unary alphabet: By Theorem 8.6, we have SyncShort $\equiv_m^{\mathsf{AC}^0}$ SyncWords where the reductions preserve the automaton. Then, the claim follows as, by Theorem 8.9, SyncWords is L-complete for unary partially ordered automata.

L-membership for fixed alphabet: Let $(\mathcal{A}, l)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance. By Item 3 from Theorem 7.90, we can assume $l < |Q|$. Using k counters, iterate through all tuples $(n_1, \ldots, n_k)$ with $n_1 + \cdots + n_k \leq l$. By using a binary encoding, this can be done in $O(k \log l)$ space. Then, for each tuple, compute $\delta(q, a_1^{n_1} \cdots a_k^{n_k})$ for all $q \in Q$ and check whether always the same state results. If this is the case, then return YES. Otherwise, continue with the next tuple. If the iteration through all tuples ends and we have not found a synchronizing word, return NO.

This algorithm can be made to run in deterministic log-space similarly as in the proof of Theorem 8.23. □

Theorem 8.25 Reach *is* NP*-complete even for partially ordered commutative automata. For a fixed non-empty alphabet, the problem is* L*-complete even for unary partially ordered automata.*

Proof. Since commutative automata are solvable, by Theorem 8.21, the problem is in NP for commutative automata.

For hardness, we give a reduction from SyncShort for partially ordered commutative automata.

Let $(\mathcal{A}, l)$ with $\mathcal{A} = (Q, \Sigma, \delta)$ be an input instance for SyncShort where $\mathcal{A}$ is partially ordered and commutative. By Item 3 from Theorem 7.90, we can assume $l < |Q|$. we can assume $l < |Q| - 1$. By Theorem 7.83 if $\mathcal{A}$ is synchronizing, the synchronizing state is a unique trap state t. Existence of a unique trap state and its computation if it exists can be done by an AC^0-circuit as in the proof of Item 3 of Theorem 8.2. By Theorem 7.1, $\mathcal{A}$ has a synchronizing word of length at most l if and only if it has one of length exactly l. Let $\mathcal{A}' = (Q', \Sigma, \delta')$ with $Q' = Q \cup \{q_1, \ldots, q_l, q_{l+1}\}$ and, for $x \in \Sigma$, we set $\delta'(q_i, x) = q_{\min\{l+1, i+1\}}$ and $\delta'(q, x) = \delta(q, x)$ for $q \in Q$. Then $\mathcal{A}$ has a synchronizing word of length l if and only if $\delta'(Q', u) = \{t, q_l, q_{l+1}\}$ for some $u \in \Sigma^*$. Thus, we obtain a valid reduction to Reach.

Note that the alphabet is preserved and that $\mathcal{A}'$ is partially ordered and commutative. Thus, by Theorem 8.24, for an unbounded alphabet we obtain NP-hardness and for a

fixed non-empty alphabet we obtain L-hardness even for partially ordered commutative automata and unary partially ordered automata, respectively.[10]

Next, let $\mathcal{A} = (Q, \Sigma, \delta)$ be a commutative n-state automaton. By Theorem 7.87, we obtain that all the sets of the form $\delta(Q, u)$ for $u \in \Sigma^*$ can be written as $\delta(Q, a_1^{i_1} \cdots a_k^{i_k})$ for $0 \leq i_1, \ldots, i_k < n$.

Now, the subsets $\delta(Q, a_1^{i_1} \cdots a_k^{i_k})$ for $0 \leq i_1, \ldots, i_k < n$ can be iterated through and tested for equality with a given target set S in deterministic log-space similarly as in the proof of Theorem 8.23 for SetTrans by iterating through the states, the letters, the numbers i_j, and using two memory cells. We also use, additionally, $|S|$ Boolean flags that indicate for each state in S whether it has been hit by some state when applying $a_1^{i_1} \cdots a_k^{i_k}$ to the state set. Then the algorithm returns YES precisely if all these flags are set to true for some $0 \leq i_1, \ldots, i_k < n$. For a fixed alphabet, this gives a deterministic algorithm using only logarithmic space. □

Theorem 8.26 ReachFrom *is* L*-complete for partially ordered commutative automata over a fixed non-empty alphabet, even for an alphabet of size one.*

Proof. By Theorem 8.2, Item 7, we have Reach $\leq_m^{\mathsf{AC}^0}$ ReachFrom and the reduction preserved the automaton type and the alphabet. This yields L-hardness by Theorem 8.25.

Next, the problem on commutative partially ordered automata can be solved with the same algorithm given in the proof of Theorem 8.23 for SetTrans, but where we also have to check for each tuple $(n_1, \ldots, n_k)$ not only that $\delta(q, a_1^{n_1} \cdots a_k^{n_k})$ is in T, but that each state from T appears at least once for some $q \in S$, i.e., the image equals precisely T. □

Theorem 8.27 ReachFrom *is solvable in polynomial time for commutative automata when the alphabet is fixed.*

Proof. In [50, Thm. 1.4], for fixed h and r, commuting $n \times n$ matrices $A_1, \ldots, A_h$, $B_1, \ldots, B_r$, and $n \times l$ matrices A and B, it was proved that it is decidable in polynomial time whether there exist non-negative integers x_i and y_j such that

$$\left(\prod_{i=1}^{h} A_i^{x_i}\right) A = \left(\prod_{j=1}^{r} B_j^{y_j}\right) B. \tag{8.2}$$

Suppose a commutative automaton $\mathcal{A} = (Q, \Sigma, \delta)$ with $\Sigma = \{a_1, \ldots, a_h\}$ and $S, T \subseteq Q$ is given. Let A_i be the transition matrix of a_i, let $r = 1$ and B_1 be the identity matrix, and let A and B be the $n \times 1$ matrices resulting from the characteristic vectors of S and T, respectively. Then Equation (8.2) with these matrices has a solution $x_1, \ldots, x_h \geq 0$ if and only if $\delta(S, a_1^{x_1} \cdots a_h^{x_h}) = T$. Hence, we find that Reach is solvable in polynomial time for fixed h and commutative $\mathcal{A}$. □

[10] For L-hardness the reduction from the proof of Theorem 8.7 can also be used. In this construction, the set $S = \{(1, v_j), \ldots, (n, v_j), v_i, (1, v_i), \ldots, (n, v_i)\}$ is only reachable (by the word a^{n+j} in this case) from the whole state set if $j < i$ since $(n, v_1) \notin S$ and so each word reaching S has to have length at least n. But for $i < j$ this implies $(1, v_j) \notin S$ since the path v_{i+1} to $(1, v_j)$ has length $j - i < n$.

8.3.5 Partially Ordered Automata

Here, supplementing Ryzhikov's [537, Prop. 9 & Thm. 4] NP-completeness results for Intersection and SyncFrom on partially ordered automata, we show the following results for partially ordered input automata in Theorems 8.28 and 8.29: SyncInto, Sync, SyncTrap, and SyncState are in AC^0. Later, in Theorem 8.18, we will also prove that SetTrans is NP-complete for solvable automata, where NP-hardness remains even for partially ordered automata.

Theorem 8.28 SyncInto *is in* AC^0 *for partially ordered input automata.*

Proof. Let $(\mathcal{A}, S)$ be an n-state input instance where $\mathcal{A} = (Q, \Sigma, \delta)$ is partially ordered and $S \subseteq Q$. For an ordering from 1 to n of the states, we assume $\mathcal{A}$ is given by a bit-wise encoding of its transition matrices $[a]_{\mathcal{A}} = (m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$, $a \in \Sigma$, and S by a bit-vector $(s_1, \ldots, s_n)$ encoding the characteristic vector of S. By Theorem 7.54, we have to check whether S contains every trap state of $\mathcal{A}$. The state $i \in \{1, \ldots, n\}$ is a trap state if and only if $\bigwedge_{a\in\Sigma} m_{ii}^{(a)}$ is true. So, an AC^0 circuit implementing the following formula checks that S contains every trap state:

$$\bigwedge_{i=1}^{n} \left[\neg \left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \vee s_i \right] \qquad \left(\text{equivalent to } \bigwedge_{i=1}^{n} \left[\left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \rightarrow s_i \right] \right). \qquad \square$$

Theorem 8.29 Sync, SyncTrap *and* SyncState *are in* AC^0 *for partially ordered input automata.*

Proof. Let $\mathcal{A} = (Q, \Sigma, \delta)$ be an n-state partially ordered input automaton encoded by its set of transition matrices $[a]_{\mathcal{A}} = (m_{ij}^{(a)})_{i,j\in\{1,\ldots,n\}}$ for each letter $a \in \Sigma$. By Corollary 7.58, $\mathcal{A}$ is synchronizing if and only if it has a unique trap state. This condition can be checked by an AC^0 circuit implementing the following formula:[11]

$$\bigvee_{i=1}^{n} \left[\left(\bigwedge_{a\in\Sigma} m_{ii}^{(a)} \right) \wedge \bigwedge_{\substack{j=1 \\ j\neq i}}^{n} \left(\bigvee_{a\in\Sigma} \neg m_{jj}^{(a)} \right) \right].$$

This shows the claim for Sync. As SyncTrap $\leq_m^{\mathsf{AC}^0}$ Sync and SyncState $\leq_m^{\mathsf{AC}^0}$ SyncInto by Theorem 8.2, Item 3 & 6 and these reductions preserve the property of being partially ordered, the claim follows for SyncTrap, and for SyncState by Theorem 8.28. $\square$

[11] Alternatively, the partially ordered automaton $\mathcal{A}$ has a synchronizing word if and only if there exists a trap state t such that $(\mathcal{A}, \{t\})$ is a YES-instance of SyncInto. This can be used to give an $\leq_m^{\mathsf{AC}^0}$-reduction from Sync to SyncInto. However, this reduction has to compute a trap state, and so the proof would not be simpler than the direct proof given here.

8.4 Discussion

Here, we discuss some other complexity results on problems related to synchronization from the literature. In view of the correspondence between automata and transformation semigroups from Section 6.8, we also discuss related problems on transformation semigroups. For notions from (parameterized) complexity theory or automata theory not defined before, we refer to the cited papers or other literature as, for example, [41] for classical computational complexity theory, [224] for parameterized complexity theory, or [490] for automata theory.

In this section, we do not discuss complexity issues related to the constrained synchronization problem introduced in Section 2.1.1 as this is done in Part IV of the present work.

Further Complexity Theoretical Results.

NP-completeness of SyncShort was established for the following classes of input automata in the literature:

1. Binary partially ordered automata with a unique trap state [198, 528];
2. SI-automata over an unbounded alphabet [210, 436];
3. Partially ordered commutative automata [210, 211, 436];
4. Eulerian[12] automata over a binary alphabet [621];
5. Circular automata over a binary alphabet [437];
6. Automata whose automaton multigraph is a two-terminal series-parallel multigraph with loops (TTSPL automata) over an alphabet with at least four letters [105].

Rystsov [528] and Eppstein [198], independently, gave the same reduction to show NP-hardness by constructing a binary partially ordered automaton. Essentially the same reduction was also given by Salomaa [546] to show NP-hardness of the problem whether a specific constant function can be composed out of a given set of functions. The reduction also appears as a special case in a paper by Goralčýk and Koubek [257] where the question of realizing idempotent transformations of a certain rank was investigated. Martyugin [436, Prop. 4.2] noted that the automaton resulting from this reduction has a trap state, is aperiodic, has a $\mathcal{J}$-trivial transition monoid, and belongs to the class of automata preserving a linear order on the states except at a single trap state, thus giving NP-hardness for these classes as well.[13] An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is called *locally confluent* if, for any $q \in Q$ and $a, b \in \Sigma$, there exists $u \in \{a, b\}^*$ such that $\delta(q, au) = \delta(q, bu)$. Clearly, every synchronizing automaton over a binary alphabet is locally confluent. So, by the Rystsov-Eppstein-Salomaa reduction, SyncShort is NP-complete for locally confluent partially ordered automata. The NP-hardness result

[12] An automaton $\mathcal{A} = (Q, \Sigma, \delta)$ is *Eulerian* if $\sum_{x \in \Sigma} |\{p \in Q \mid \delta(p, x) = q\}| = |\Sigma|$ for each $q \in Q$.

[13] He called the latter automata partially monotonic automata, but since these automata have a totally defined transition function we avoid this term here to not cause confusion with partial automata where the transition function can be a partial function introduced later in Part IV.

on TTSPL automata was shown in [105] by adding two additional letters and additional states to obtain a TTSPL automaton from the Rystsov-Eppstein-Salomaa construction.

Eppstein [198] showed that for oriented n-state automata[14] over the alphabet Σ the problem SyncShort is solvable in time $O(|\Sigma|n^2)$. Every monotonic automaton is oriented [23], yielding polynomial time solvability for monotonic automata as well. Martyugin [436] showed that SyncShort is solvable in $O(n^{|\Sigma|}\log(n))$ for commutative automata and in $O(n)$ for binary SI-automata with n states (by checking whether it equals one of the two SI-automata from Theorem 7.31).

Note that, while SyncShort remains NP-complete even for partially ordered commutative input automata (Theorem 8.24), as we have shown in Section 8.3, the complexity of Sync drops from NL-completeness in the general case to AC^0 on partially ordered input automata (Theorem 8.29) and to L-completeness for solvable input automata (Theorem 8.20).

Olschweski and Ummels [472] proved that deciding whether a given automaton has a shortest synchronizing word of length *exactly* k is DP-complete. Furthermore, they proved that computing the length of a shortest synchronizing word, a synchronizing word of length k, and a shortest synchronizing word is $\mathsf{FP}^{\mathsf{NP}[\log]}$-complete, FNP-complete, and $\mathsf{FP}^{\mathsf{NP}[\log]}$-hard and in $\mathsf{FP}^{\mathsf{NP}}$, respectively. With respect to approximability, for n-state automata, Gawrychowski and Straszak [239] proved that for every $\epsilon > 0$ it is NP-hard to approximate the length of a shortest synchronizing word within a factor of $n^{1-\epsilon}$ even for a fixed binary alphabet. This result is optimal, as the Eppstein-Natarajan algorithm [198, 457] mentioned in Section 7.1.6 gives an $(n-1)$-approximation algorithm [243]. More generally, Gerbush and Heeringa [243] gave a $O(k|\Sigma|n^k + n^4/k)$-time $\lceil \frac{n-1}{k-1} \rceil$-approximation algorithm, where $k \geq 2$ denotes the size of the intermediate sets used for compression. Historically, before the optimal result by Gawrychowski and Straszak [239], under the assumption $\mathsf{P} \neq \mathsf{NP}$, Berlinkov firstly proved that there exists no polynomial time $O(1)$-approximation [79, 82] over a fixed binary alphabet. Gerbush and Heeringa [243] then proved that there is no polynomial time $O(\log n)$-approximation over an unbounded alphabet, which was improved by Berlinkov [83] to a fixed binary alphabet.

Fernau and Hoffmann [212] introduced, for a given partial order $\prec$ on Σ^*, the following *extension variant* of the synchronization problem.

ExtDFA-SW-$\prec$ (Fernau and Hoffmann [212])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $u \in \Sigma^*$.
Question: Is there $w \in \mathrm{Syn}(\mathcal{A})$ minimal w.r.t. $\prec$ in $\mathrm{Syn}(\mathcal{A})$ such that $u \prec w$?

For the prefix/suffix/lexicographic orders the problem is in P, for the scattered subsequence order even on commutative automata (and even for TTSPL automata) it is NP-hard. For the parameter $|u|$ and the scattered subsequence order it is W[3]-hard even on commutative automata (and even for TTSPL automata). The complementary problem that switches YES and NO answers for the length-lexicographic order is NP-complete even for TTSPL automata and in $\mathsf{WNL} \cap \mathsf{W}[P] \cap \mathsf{A}[2]$ w.r.t. $|u|$. For more information, we refer to [105, 209, 212].

[14] This is the terminology due to Ananichev and Volkov [23] which we use here as well, Eppstein himself called these automata monotonic; see also the discussion on page 223.

Maslennikova [441] showed that it is PSPACE-complete to decide whether a given regular ideal language is the set of synchronizing words of an automaton with at most four states over a binary alphabet.

Türker and Yenigün [609] introduced and investigated variants of the synchronization problem, the following problems being their main problems.

MinimumSynchronizableSubautomaton (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$ and $k \geq 0$.
Question: Does there exist $\Gamma \subseteq \Sigma$, $|\Gamma| \leq k$, such that $\mathcal{A}_{|\Gamma}$ is synchronizing?

ExclusiveSyncWord (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $S, T \subseteq Q$.
Question: Does there exist $u \in \Sigma^*$ such that $\delta^{-1}(T, u) = S$?

MaxF-ReducingWord (Türker and Yenigün [609])
Input: Automaton $\mathcal{A} = (Q, \Sigma, \delta)$, $F \subseteq Q$, $k \geq 0$.
Question: Does there exist $S \subseteq Q$, $|S| \geq k$, $u \in \Sigma^*$ such that $\delta(S, u) \subseteq F$?

They proved that MinimumSynchronizableSubautomaton is NP-complete[15] (which holds even for TTSPL automata as shown in [105, Cor. 5.9]), MaxF-ReducingWord is NP-hard for monotonic automata and ExclusiveSyncWord is PSPACE-complete. The reduction for MaxF-ReducingWord was challenged by Ryzhikov [537, p. 80] as "they reduce the N-Queens Puzzle [78] to this problem to prove its NP-hardness. However, their poof is unclear, since the input has size $O(\log N)$, and the output size is polynomial in N." Ryzhikov [537, Prop. 3] showed that MaxF-ReducingWord is PSPACE-complete for binary automata. In the context of parameterized complexity, MinimumSynchronizableSubautomaton was shown to be W[2]-hard even for TTSPL automata [105, Cor. 5.7 & 5.9] and in WNL $\cap$ W[P] [209, Thm. 12].

Section 8.3.1 collected known results on the subset mapping problems. Next, we give some further background. SyncInto, SyncFrom, and SetTrans were introduced, and shown to be PSPACE-complete, by Rystsov [529] under the names *(global) inclusion problem for non-initial automata* (*NIA*) [529, Thm. 2.1], *(global) inclusion problem for weakly-initial automata with a single final state* (WIA_1) [529, Cor. 3.3], and *(global) inclusion problem for weakly-initial automata* (*WIA*) [529, Thm. 2.1], respectively. Independently, Luks and McKenzie [425] later introduced SetTrans for permutation groups that asks, for a permutation group G on $[n]$ given by generators and subsets $A, B \subseteq [n]$, whether $g(A) = B$ for some $g \in G$. They proved that the problem is in NC for solvable permutation groups. For groups, the sets A and B must have the same size and $g(A) = B$ is equivalent to $g(A) \subseteq B$. We note that the automata-theoretical analogous problem with equality instead of set inclusion was investigated by Rystsov under the name *reachability problem for weakly-initial automata* and proved to be PSPACE-complete [529, Thm. 3.2], even when the starting set is the whole state set

[15] We omitted a cost function on the letters mentioned in [609], which does not change NP-completeness. Also, in [609] only NP-hardness is stated, but, as noted in [105], containment in NP follows as we can guess Γ and check synchronizability of $\mathcal{A}_{|\Gamma}$ in polynomial time.

and the target has size two.[16] In [251], reachability problems in certain discrete event systems were investigated that translate, in our language, to the problem Intersection for unary permutation automata and with different types of final state sets, giving polynomial-time solvable and NP-complete problems.

Blondin et al. [96] investigated different variants and generalizations of Intersection. They named SetTrans the problem whether there exists, in a transformation monoid given by a list of generators, a transformation mapping a given set into a set included in another given set. SetTrans as defined here is the automata-theoretical version of this problem, i.e., we opted to use the naming suggested by the work of Blondin et al. [96]. For a fixed number $k \geq 1$ of automata, Intersection is NL-complete [321, p. 464].

Berlinkov, Ferens and Szykuła [84] classified and investigated various further subset mapping problems. For $\mathcal{A} = (Q, \Sigma, \delta)$ and $S \subseteq Q$, their problems fall into the following broad main categories: *extending problems*, *totally extending problems*, and *resizing problems* that ask for $w \in \Sigma^*$ such that $|\delta^{-1}(S, w)| > |S|$, $\delta^{-1}(S, w) = Q$, and $|\delta^{-1}(S, w)| \neq |S|$, respectively. They investigated those problems with and without a binary encoded bound on $|w|$ and, for the former two categories, with and without a fixed parameter k such that $|S| \leq k$ or $|S| \geq n - k$, which is used, for example, in runtime bounds of algorithms. A word $w \in \Sigma^*$ *avoids* $S \subseteq Q$ if $\delta(Q, w) \cap S = \emptyset$. As this is equivalent to $\delta^{-1}(Q \setminus S, w) = Q$, totally extending problems can be rephrased as problems that ask for avoiding words of the complement. Furthermore, the authors considered *compression*, i.e., $|\delta(S, w)| < |S|$, which they showed to be polynomial-time solvable, and *shrinking*, i.e., $|\delta^{-1}(S, w)| < |S|$, which is equivalent to extending $Q \setminus S$. As $\delta^{-1}(S, w) = Q$ is equivalent to $\delta(Q, w) \subseteq S$, the problem of total extension without parameters is equivalent to SyncInto, which was investigated earlier by Rystsov [529]. For a full overview of the results we refer to their work [84].

A recent trend in fixed parameter tractability [65] and the field of artificial intelligence [288, 336] asks for *diverse solution sets*, giving a user more freedom of selection of a particular solution to fit his/her needs. Let $\mathcal{A}$ be an automaton. Arrighi et al. [43] extended automata synchronization to the diversity setting by saying a finite set $W \subseteq \mathrm{Syn}(\mathcal{A})$ has *synchronization diversity* k if two distinct words from W have edit distance at least k and each word in W is minimal with respect to the scattered subsequence order in $\mathrm{Syn}(\mathcal{A})$. Then, they proved that deciding whether an automaton admits a sets of synchronization diversity at least k is co-NP-hard and that computing the diversity of the set of words minimal w.r.t. scattered subsequence order in $\mathrm{Syn}(\mathcal{A})$ is hard for the counting complexity class #P.

Parameterized Complexity.

The following results on the parameterized complexity of SyncShort are known (the paramterized complexity class WNL is a lesser known class defined via the number of tape cells used by a nondeterministic Turing machine [269]):

[16] The case that the starting set is the whole state set was also shown to be PSPACE-complete independently by Bondar and Volkov [99, Thm. 3].

1. SyncShort is in FPT w.r.t. $(|\Sigma|, l)$ [211, Thm. 7], as we can try all $|\Sigma|^l$ words of length l, and the number of states n [211, Thm. 10], as we can search in the power automaton with 2^n states and $|\Sigma|2^n$ transitions. Furthermore, unless the polynomial-time hierarchy collapses on the third level, SyncShort admits no polynomial-size kernel when parameterized by $(|\Sigma|, l)$ [105, Cor. 4.6] or by n [623].
2. Bruchertseifer and Fernau [209] introduced the parameterized complexity class W[Sync] as the class of problems fpt-reducible to SyncShort with parameter being the length-bound l. They showed that W[Sync] sits between W[2] and WNL$\cap$ W[P] $\cap$ A[2], see [211, Thm. 3], [105, Thm. 4.2 & Prop. 4.4], and [209, Thm. 3, 4 & 5].

Montoya and Nolasco [453, 454] proved that SyncFrom with a length bound on the solution word is WNL-complete when parameterized by the size of the input set.

Wareham [626] investigated *bounded automata intersection* with a parameter l asking for a solution word of length exactly l accepted by m automata each with at most n states. He obtained containment in FPT for the following parameter combinations:

1. $(|\Sigma|, l)$ by trying all words in Σ^l,
2. (m, n) via the product automaton of size n^m of all input automata,
3. $(n, |\Sigma|)$ by noting that there are at most $n^{|\Sigma|n}$ different automata, so that we can assume $m \leq n^{|\Sigma|n}$, and use the reasoning as for (m, n).

In fact, Wareham stated the above results under the assumption $l \leq n$. Clearly, this restriction is not necessary for 1 to hold. But it is also not necessary for 2, and so 3, since the l-th power of the transition matrix of the automaton graph of the product automaton mentioned in 2 can be computed with $\log(l)$ multiplications by a square-and-multiply algorithm, and this number is polynomial even for a binary encoding of l. For hardness, he obtained the following results: $W[1]$-hard for (m, l), $W[2]$-hard for (n, l), $W[t]$-hard for all $t \geq 1$ for the sole parameter m, and not in XP for the sole parameter $|\Sigma|$ (under the assumption P $\neq$ NP); the construction also implies this for the sole parameter n, as noted in [213]. Further, Bruchertseifer and Fernau [209, Thm. 9], Fernau et al. [213, Cor. 2.2], and Guillemot [269, Prop. 10] proved W[Sync]-completeness w.r.t. l, $W[1]$-completeness w.r.t. (m, k), and WNL-completeness w.r.t. m, respectively. Moreover, Fernau et al. [213, Thm. 5.8] proved $W[1]$-completeness w.r.t. m for automata accepting sparse languages even for unary automata (which accept sparse languages).[17] We remark that the co-$W[2]$-hardness proof in [213, Prop. 6.3] is wrong. For further information on synchronization problems, parameterized complexity, and also runtime lower bounds derived from ETH and SETH we refer to [105, 211].

Later, in **????**, we will show that Intersection and SyncShort are solvable in time $O(m|\Sigma|n^{|\Sigma|+1})$ on m commutative partially ordered input automata with at most n states and in time $O(n^{|\Sigma|+2})$ on commutative n-state automata, respectively.[18] So, both problems are in the parameterized complexity class XP w.r.t. $|\Sigma|$ for the mentioned types of input automata. The XP-result for Intersection on commutative partially ordered automata has been extended in [42] to *totally star-free nondeterministic automata* accepting commutative languages.

[17] See **??** for a definition of sparse languages.

[18] Martyugin [436] obtained an algorithm running in time $O(|\Sigma|n^{|\Sigma|}\log(n))$.

Lastly, testing whether m binary automata accept a common word is equivalent to the simulation of a nondeterministic Turing machine that uses $m \log(n)$ bits of space [363] (see also [234, 356, 406, 635, 636]). In [356, 635], for each fixed m the resulting families of automata intersection problems have been used to obtain conditional separation results between NL and P.

Practical Implementations to Compute Shortest Synchronizing Words.

As SyncShort is NP-complete there exists no polynomial time algorithm computing a shortest synchronizing word under the assumption NP $\neq$ P. However, people have tried to come up with various heuristics to obtain algorithms that run efficiently on most instances in practice. The earliest implementations of exact algorithms using several approaches were given by Rho et al. [510], Pixley et al. [493], and Deshmukh and Hawat [174]. Kisielewicz et al. [342] have given an algorithm that combines multiple techniques and performs well on most instances. This algorithm has been recently improved by Szykuła and Zyzik [594] (see also the related forward and backward scheme by Szykuła and Roman [519]). Algorithms based on machine learning were proposed by Podolak et al. [495, 496], and an evolutionary algorithm by Kowalksi and Roman [386]. Tas et al. [597] studied the performance of some algorithms exploiting GPUs and parallelism on automata with up to a billion states. Furthermore, in [151, 355, 549] the authors optimize existing algorithms for real-life applications, for example in model-based testing.

The Natarajan-Eppstein algorithm [198, 457] mentioned in Section 7.1.6 runs in polynomial time and outputs some synchronizing word that is guaranteed to have at most cubic length. Two other algorithms with these properties were suggested by Trahtman [602][19] and Roman [517]. Kudłacik et al [390] compare different exact and non-exact algorithms.

For more information, we refer to the introductory section of [342] that contains further information and additional references to implementations and algorithms.

Related Problems on Transformation Semigroups.

Goralčík and Koubek [257] investigated the problem TransformationRank (resp. IdempotentRank) which asks, for a set $T \subseteq \mathcal{T}_n$ and $r > 0$, whether $\langle T \rangle_S$ contains a transformation (resp. idempotent transformation) of rank r. For the fixed parameter $r = 1$, both problems reduce to Sync. Further, for each fixed $r > 0$ they proved that TransformationRank is NP-hard and IdempotentRank is solvable in time $O((r+1)n^{(2r+1)}|T|)$. In general, IdempotentRank was proved to be PSPACE-complete. The precise complexity of TransformationRank is an *open problem*. A variant of IdempotentRank with a length-bound for a product of transformations from T was proved to be NP-complete

[19] In other works spelled as Trakhtman.

for each fixed r.[20] In the realm of parameterized complexity theory, Bruchertseifer and Fernau [209, Thm. 8] proved TransformationRank complete for W[Sync].

Kozen [387] introduced Membership that asks, for $T \subseteq \mathcal{T}_n$ and $t \in \mathcal{T}_n$, whether $t \in \langle T \rangle_S$. Now, we obtain various variants of Membership by restricting T to generate monoids from some given class. Similarly, for the problem Intersection by focusing the attention to automata whose transformation monoids come from some common class of monoids, we obtain variants of this problem With respect to these variants, Blondin et al. [96] observed the following relation.

> "[DFA-Intersection] is equivalent to [Membership] when both are intractable, but appears harder than [Membership] when [Membership] is efficiently solvable." [96]

In fact, the complexity gap can be quite large. For example, by results of Arvind and Vijayaraghavan [45] it follows that Membership on abelian permutation groups is in NC^2, whereas Intersection even on unary permutation automata is NP-complete [74, 213, 234, 434, 584] (see also Theorem 8.18). More specifically, we have Membership $\leq_m^{\mathsf{AC}^0}$ Intersection as for $t, t_1, \ldots, t_k \colon [n] \to [n]$ with $\mathcal{A} = ([n], \Sigma, \delta)$ given by $\Sigma = \{t, t_1, \ldots, t_k\}$ and $\delta(i, x) = x(i)$ for $x \in \Sigma$ we have

$$t \in \langle \{t_1, \ldots, t_k\} \rangle_S \Leftrightarrow \bigcap_{i=0}^{n-1} L(\mathcal{A}_{i,\{t(i)\}}) \neq \emptyset.$$

Now, suppose $\mathcal{A}_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$ with $i \in \{1, \ldots, k\}$ form an input to Intersection. We can assume the state sets Q_i are pairwise disjoint. Let $t_a \colon Q_1 \cup \cdots \cup Q_k \to Q_1 \cup \cdots \cup Q_k$ for $a \in \Sigma$ be the transformations with $t_a(q) = \delta_i(q, a)$ for the unique $i \in \{1, \ldots, k\}$ such that $q \in Q_i$. Then

$$\bigcap_{i=1}^{k} L(\mathcal{A}_i) \neq \emptyset \Leftrightarrow \exists t \in \langle \{t_a \mid a \in \Sigma\} \rangle_S \; \forall i \in \{1, \ldots, k\} \colon t(q_i) \in F_i.$$

So, we can solve $\bigcap_{i=0}^{n-1} L(\mathcal{A}_i) \neq \emptyset$ by guessing a transformation t and then checking checking whether $t \in \langle \{t_a \mid a \in \Sigma\} \rangle_S$ and $t(q_i) \in F_i$ for all $i \in \{1, \ldots, k\}$. This reduction up to some "nondeterminism" makes the "equivalent when intractable"-statement more precise.[21] Note that, for classes generated by the t_a such that Membership is solvable in polynomial time, this procedure gives an FPT-algorithm for the parameter $\max\{|F_i| \mid i \in \{1, \ldots, k\}\}$.

Closer inspection of Kozen's proof [387, Thm. 3.2.6] reveals that Membership is PSPACE-complete even for aperiodic transformation monoids as input. For aperiodic transformation monoids, the Membership problem was classified into five classes giving problems that are in AC^0, P- , NP-, or PSPACE-complete, or NP-hard, by Beaudry et al. [77], where NP-hardness holds for aperiodic transformation monoids of threshold at least two. Partial results for general threshold-one monoids were obtained by

[20] Their proof [257, Thm. 5] reduces the problem to the special case $r = 1$ and so, basically, SyncShort. They give a reduction from 3SAT which is a little different than the Rystsov-Eppstein construction [198, 528] but similar to constructions from [210, 211, 212, 436].

[21] The construction is due to Lukas Fleischer (personal communication).

Beaudry [76]. For commutative transformation monoids, Beaudry [73] proved NP-completeness.

For permutation groups, MEMBERSHIP was shown to belong to P by Furst et al. [231] (analyzing an algorithm given by Sims [574]), then to NC^3 for abelian groups by McKenzie and Cook [447] and Mulmuley [455] (improved by Arvind and Vijayaraghavan [45] to containment in $\mathsf{ZPL}^{\mathsf{ModL}}$, which is in NC^2). Luks and McKenzie [483] proved membership in NC for solvable groups, Luks [424] extended this to groups with bounded non-abelian composition factors, and Babai et al. [49] finally established membership in NC for all groups.

Now consider the variant of MEMBERSHIP parameterized by an integer l, where one asks whether a given transformation admits a factorization of length at most l. This problem is PSPACE-complete for l in binary encoding and NP-complete for l in unary encoding, even for permutation groups as shown by Jerrum [343, Thm. 2.2] and Even and Goldreich [204, Thm. 1], respectively. Furthermore, Fernau and Bruchertseifer [209] proved W[Sync]-completeness w.r.t. l and general monoids. When asking for a product of length exactly l, Cai et al. [123, Thm. 6 & 7] proved W[2]-hardness in general and W[1]-hardness for permutation groups w.r.t. l. In case of permutation groups, asking for a product of length exactly l was proved DP-complete for a unary encoding of l and PSPACE-complete for a binary encoding by Lohrey and Rosowski [421, Thm. 4.2 & 4.3].

For further information on INTERSECTION and MEMBERSHIP, the paper by Blondin et al. [96] and the master thesis by Blondin [95] give a good overview. For more algorithmic investigations on INTERSECTION, we refer to further work by Wehar and coauthors [213, 470, 471, 635, 636] and others [42, 211, 214, 356, 406, 626]. Intersection problems on transformation semigroups were investigated by Fleischer and Kufleitner [223] and Fleischer [222], see also the thesis by Fleischer [221]. The membership problem for finite monoids and groups has been generalized to the *rational (subset) membership problem* [353, 420].

A Remark on Local Confluence and Synchronization

Simon [572, 573] introduced piecewise testable languages and characterized them as those languages acceptable by automata with $\mathcal{J}$-trivial transition monoids. Klíma [371, Thm. 1] proved that a language is piecewise testable if and only if it is acceptable by a locally confluent partially ordered automaton. Since we mentioned locally confluent automata above, we point out a connection of synchronizing automata to piecewise testable languages that has not been noted before and is based on the following statement.

Proposition 8.30 *Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be partially ordered and accessible. If $\mathcal{A}$ is locally confluent, then it is synchronizing.*

Proof. We do induction on $|Q|$. Clearly, the claim holds for $1 \leq |Q| \leq 2$. Next, suppose $|Q| \geq 3$.

Since $\mathcal{A}$ is accessible, there either exists $a \in \Sigma$ such that $\delta(q_0, a) \neq q_0$ and $\delta(q_0, a) = \delta(q_0, b)$ for each $b \in \Sigma$ with $\delta(q_0, b) \neq q_0$, or there exist $a, b \in \Sigma$ such that for

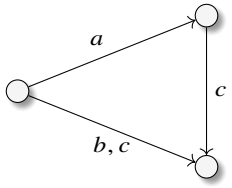


Fig. 8.5: Synchronizing partially ordered automaton that is not locally confluent. Self-loops omitted.

$p = \delta(q_0, a)$ and $q = \delta(q_0, b)$ we have $q_0 \neq p$ and $q_0 \neq q$ and $p \neq q$. In the first case, the set $\uparrow\{\delta(q_0, a)\}$ is a proper upper set of size at least two. In the second case, by assumption, there exists $u \in \{a, b\}^*$ with $\delta(p, u) = \delta(q, u)$. Since $p \neq q$, we have $p \neq \delta(p, u)$ or $q \neq \delta(q, u)$. Thus either $\uparrow\{p\}$ or $\uparrow\{q\}$ has size at least two and neither contains q_0.

Thus, in each case we find $q \in Q$ such that for $S = \uparrow\{q\}$ we have $2 \leq |S| < |Q|$.[22] Then $\mathcal{A}_{|S}$ and $\mathcal{A}/{\sim_S}$ are accessible, partially ordered, and locally confluent. So, inductively, both automata are synchronizing. By Theorem 7.3, Item 4 it follows that $\mathcal{A}$ is synchronizing. □

Corollary 8.31 *An accessible partially ordered automaton over an alphabet of size at most two is synchronizing if and only if it is locally confluent.*

Proof. First, let $\mathcal{A} = (Q, \Sigma, \delta)$ be synchronizing with $|\Sigma| \leq 2$ and synchronizing word $u \in \Sigma^*$. Let $q \in Q$ and $a, b \in \Sigma$. We can assume $a \neq b$, so that $\Sigma = \{a, b\}$. Thus $u \in \{a, b\}^*$ and since u is synchronizing we have $\delta(q, au) = \delta(q, bu)$. Hence, $\mathcal{A}$ is locally confluent.

Since by Theorem 8.30 local confluence implies synchronizability, we obtain the claim. □

Thus, with the result by Klíma [371, Thm. 1] mentioned above, we obtain that a binary language is piecewise testable if and only if its minimal automaton is a synchronizing partially ordered automaton.

Theorem 8.31 does not hold for alphabets with more than two letters, for example, see Figure 8.5.

[22] Alternatively, either there exists $a \in \Sigma$ such that $q = \delta(q_0, a)$ is not a trap state and $q \neq q_0$, in which case $\uparrow\{q\}$ is proper and has size at least two, or $\delta(q_0, a)$ is a trap state for each $a \in \Sigma$. Since for each $a, b \in \Sigma$, $\delta(q_0, au) = \delta(q_0, bu)$ for some $u \in \{a, b\}^*$, in the latter case we must have $|Q| \leq 2$.

Part III
Automata for Sets of Synchronizing Words

"My best tool for efficiency and performance is abstraction."
—Bjarne Stroustrup (1950–present)

Part IV
The Constrained Synchronization Problem

"Du glaubst nicht, wie müde ich bin, wie satt ich die Chemie habe, wie namentlich gewisse Teile mich ordentlich anekeln, mir wenigstens so langweilig sind, dass ich gähnen muss, wenn ich daran denke. Sind wir denn schon so alt, oder wie ist das? Diese nervenschwächende Wirkung muss wirklich der Chemie eigentümlich sein. Ich glaube, die materiellen Influenzen, die Dämpfe, Gerüche und all die Teufelsstinkereien haben großen Anteil daran. Besonders ist es das Praktikum, was einen so herunterbringt."

–Friedrich Wöhler (1800–1882)
Aus einem Brief an Justus von Liebig (22.07.1847)

Index

1-set, 28
G-orbit, 112
L-uniform NC^1 Turing reducible, *see* L-uniform NC^1 oracle reducible
L-uniform NC^1 oracle reducible, 72
L-uniform families of circuits, 67
M-act, 168
R_1-solvable, 220
R_2-solvable, 220
S-congruence, 45
$\leq_T^{\mathsf{NC}^1}$-reduction, 72
$\leq_m^{\log}$-reduction, 72
$\leq_m^{\mathsf{AC}^0}$-reduction, 72
$\leq_m^{\mathsf{NC}^1}$-reduction, 72
$\leq_m^{\mathsf{P}}$-reduction, 72
ρ-automaton, 198
$f(n)$-space bounded, 65
$f(n)$-time bounded, 65
i-fold composition, 38
k-connected graph, 60
k-homogeneous, 112
k-set, 28
k-transitive, 112
m-cycle, *see* cycle
n-tuple, *see* tuple
Sync[L], *see* constrained synchronization problem
k-DAG-ST , 71
k-DAG, 71
3SAT, 71
DAG, 70
Intersection, 226
ReachFrom, 226
Reach, 226
SingleCycle, 70
StronglyConn, 70
SyncFrom, 226
SyncShort, 225
SyncState, 225
SyncWords, 225
Sync, 225
DisjSetTrans, 226
SetTrans, 226
SyncInto, 226
SyncTrap, 225
SAT, 70
Černý automata family, 10
Černý bound, 12
Černý conjecture, 12
IdempotentRank, 254
Membership, 254
OrderBetweenVertices, 70
TransformationRank, 254

abelian group, 43
abstract group, 122
accepted
  language
    by automaton, 159
  word
    by automaton, 159

accepts an input
  Turing machine, 65
accessible
  automaton, 159
  state
    in automaton, 159
acts transitively, 112
acyclic graph, 60
alphabet, 3
alternating group, 112
annihilation law, 41
anti-Leibniz order, 81
antisymmetric relation, 39
Ashby's ghost taming automaton, 7
assignment, 70
associative operation, 40
atom in a graph, 127
automata with output, 169
automaton, *see* deterministic and complete finite semiautomaton
  as language acceptor, 159
automaton graph, 131
automaton with one-point singular idempotents, 186
automaton with simple idempotents, 186
automorphism
  of graphs, 59
  semiautomaton, 137
automorphism group
  of graphs, 59
avoiding word, 252

bijective, 30
binary automaton, 132
binary language, 57
binary relation, 39
block
  in automaton, 132
  in permutation group, 111
Boolean circuit, 66
Boolean matrix, 44
Boolean semiring, 41
Boolean transition matrix, 44
  automaton, 132

canonical map of $\mathcal{A}$ onto MinAut(L), 161
canonical morphism, 149
canonical projection map, 39
cardinality, 3
Cartesian product, 28
center of group, 43
clause, 70
coaccessible state
  in automaton, 159
coarser equivalence relation, 40
codomain
  of function, 3
column index, 44
column of matrix, 44
commutative automaton, 200
commutative language, 57
commutative semigroup, 41
commutativity, 41
complete problem (or language), 72
complex product, *see* product on subsets
complexity class, 71
component projection mapping, 28
component-wise addition, 43
component-wise order, 47
component-wise product, 43
composition
  of total functions, 31
computing a function
  circuit, 66
concatenation product, 3
congruence class, 45
congruent elements, 45
conjugate transformation, 112
conjunction, 27
connected component, 60
connected graph, 60
connectivity, 61
constant function, 30
constrained synchronization problem, 15
constraint language, 15
coprime number, 28
coset, *see* right coset
cycle, 93

cycle contained in transformation
see cycle induced by transformation, 93
cycle in a graph, 60
cycle induced by transformation, 93
cycle notation, 93
cycle point, 38
cyclic automaton, 135
cyclic group, 43
cyclic permutation, *see* cycle
cyclically permuting a set, 93

DAG, *see* directed acyclic graph
deciding a language
circuit, 66
computational mechanism, 69
Turing machine, 65
decision problem, 69
defect
of a function, 36
of a word, 132
degree of permutation group, 111
depth of a circuit, 66
derivative (of a language), *see* language quotient
deterministic and complete finite automaton, *see* deterministic and complete finite semiautomaton
deterministic and complete finite semiautomaton, 4
diagonal action, 122, 123
diagrammatic order, 81
digraph, *see* directed graph
direct product (semigroup), 43
directed acyclic graph, 60
directed graph, 59
disjoint, 28
disjoint cycles, 93
disjoint union, 28
disjunction, 27
distinguishable states, 161
domain
of function, 3
of permutation group, 111
of transformation semigroup, 45
duplicate set
of a function, 30
of a word, 132

edge of a graph, 59
empty function, 30
empty set, 3
empty word, 3
ending vertex, 59
endofunction, 38
endomorphism
semiautomaton, 137
endpoints of edge, 59
equality relation, 40
equivalence, 27
equivalence class, 39
equivalence relation, 39
equivalence relation induced
by partition, 39
by quasiorder, 46
excess
of a subset under a function, 36
excluded set
of a function, 30
of a word, 132
existential quantifier, 27
extending problem, 252
extension of a function to subsets, 30

factor, 57
factor (semi-)automaton, 149
fan-in, 66
fan-out, 66
final state set
of an automaton, 159
final vertex, 59
finer equivalence relation, 40
finite alphabet, *see* alphabet
finite set, 28
finite state set, *see* state set
fixed point (by transformation), *see* fixpoint
fixpoint
of transformation, 38
of transformation semigroup, 45
forest, 60
formal language (over Σ), 3

free monoid (over X^*), 56
full symmetric group, 111
full transformation semigroup, 45
function, 3
function composition, *see* composition
function extended to subsets, 30
function induced by word, 132
functional graph, 62

generated
  subgroup, 43
  submonoid, 43
  subsemigroup, 43
  equivalence relation, 40
  lower set, 46
  upper set, 46
generator, 43
greatest common divisor, 28
greatest element, *see* maximum
group, 41
group of units, 42

hard problem (or language), 72
holonomy decomposition, 170
homing sequence, 169
homogeneous relation, 39

ideal language, 57
idempotent element, 41
idempotent letter/word, 132
idempotent transformation, 102
identity element, 41
identity matrix, 44
identity transformation, 30, 38
image of subset
  of transition function, 4
image of subsets
  of a function, 30
implication, 27
imprimitive permutation group, 112
incoming edge, 59
incomparable elements, 46
indegree of vertex, 59
indistinguishable states, 161
induced partial order, 47
induction proof, 27
infix, *see* factor
initial state, *see* start state
initial vertex, 59
initially connected, *see* accessible automaton
injective, 30
input alphabet
  of an automaton, 4
input instance, 70
integers, 4
intersection, 28
interval, 11
intransitive permutation group, 112
inverse element, 41
inverse function, 30
inverse image of a function, 30
inverse operation, 41
inverse transformation, 91
isomorphic automata, 137
isomorphism
  of monoids, 42
  of semigroups, 41
  order isomorphism, 46
  semiautomaton, 137

kernel relation, 39
kernel type of a function, 36
Kleene plus, 56
Kleene star, 56
Klein four-group, 113
Krasner–Kaloujnine universal embedding theorem, 170
Krohn-Rhodes decomposition theory, 170

language, *see* formal language (over Σ)
language quotient, 57
least common multiple, 28
least element, *see* minimum
left (language) quotient, *see* language quotient
left action, 81
left coset, 42
left ideal language, 57
left regular (Cayley) representation, 122

left-distributivity, 41
Leibniz order, 81
length of word, 3
letter, *see* symbol
linear extension, *see* topological sorting
linear order, *see* total order
literal, 70
lower set
  quasiorder, 46

many-one equivalent
  L-uniform AC^0, 72
  L-uniform NC^1, 72
  log-space, 72
  polynomial-time, 72
many-one reducible
  L-uniform AC^0, 72
  L-uniform NC^1, 72
  log-space, 72
  polynomial-time, 72
map, *see* function
matrix over semiring, 43
matrix product, 44
maximal element, 46
maximal subgroup, 42
maximum, 46
membership problem, 69
minimal automaton, 161
minimal element, 46
minimum, 46
monoid, 41
monoid action, 168
monotone function, *see* morphism
monotonic automaton, 191
morphic image, 139
morphism
  of (semi-)automata, 137
  of groups, 42
  of monoids, 42
  of quasiordered sets, 46
  of semigroups, 41
moved point (by transformation), 38
Myhill-Nerode relation, 161

natural numbers, 4
negation, 27
node of a graph, *see* vertex of a graph
normal subgroup, 43

OI-automaton, *see* automaton with one-point singular idempotents
one-hot unary, 76
one-point singular, 97
oracle equivalent
  L-uniform NC^1, 72
oracle reducible
  L-uniform NC^1, 72
orbit-stabilizer property, 117
orbital graph
  directed, 127
  undirected, 127
order of group, 43
oriented digraph, 60
outdegree of graph, 126
outdegree of vertex, 59
outgoing edge, 59

pair (semi)automaton, 157
pair criterion, 182
Parikh map, 57
Parikh morphism, *see* Parikh map
partial order, 46
partially ordered automaton, 196
partition, 28
path in a graph, 60
permutation, 91
permutation automaton, 190
permutation group, 111
permutation-reset automaton, 190
permutational letter/word, 132
permuted subset, 91, 93
pigeonhole principle, 27
point contained in cycle, 93
point stabilizer, 112
point w.r.t. a transformation, 38
point-symmetric graph, 130
point-wise product, 43
point-wise representation, 78
power (semi-)automaton, 154
power set, 28
prefix, 57

preimage, *see* inverse image
preorder, *see* quasiorder
prime number, 28
prime number theorem, 28
primitive automaton, 132
primitive permutation group, 112
product (semi-)automaton, 154
product in semigroup, 40
product on subsets, 41
projection morphism, 57
proper factor, 57
proper prefix, 57
proper subset, 28
proper suffix, 57

quasiorder, 46
quasipermutation, 91

RAM, *see* random access machine
random access machine, 64
rank
  of a function, 36
  of a word, 132
rational expression, 58
reachability (quasi-)order, *see* reachability order
reachability equivalence, 134
reachability order
  automaton, 134
real numbers, 4
recognized language, *see* accepted language
Rees congruence, 150
reflexive relation, 39
register, 64
register machine, *see* random access machine
regular constraint language, *see* constraint language
regular expression, 58
regular language, 159
regular permutation group, 112
relation, 39
reset automaton, 190
reset complexity, 16
reset length, 10
reset word, *see* synchronizing word
resizing problem, 252
restriction of function to subset, 30
restriction on automaton, 148
right action, 81
right coset, 42
right ideal language, 57
right regular (Cayley) representation, 123
right-distributivity, 41
road coloring theorem, 10
root point, 38
row index, 44
row of matrix, 44

satisfiability problem, *see* SAT
satisfying assignment, 70
saturating a subset, 39
section, 28
self-loop, 131
semiautomaton, *see* deterministic and complete finite semiautomaton
semiconnected graph, *see* unilatterally connected graph
semigroup, 40
semiregular permutation group, 112
semiring, 41
sequential transducer, 169
set builder notation, 3
set difference, 28
set membership, 3
set of duplicate states
  of a function, 30
set of excluded states
  of a function, 30
set of factors, 57
set of fixed points, 38
set of moved points, 38
set of prefixes, 57
set of suffixes, 57
set of synchronizing words, 4
SI-automaton, *see* automaton with simple idempotents
simple automata, 168
simple idempotent, 94
simple idempotent letter/word, 132

simple subgroup, 43
singleton, *see* 1-set
sink component, 135
sink state, *see* trap state
size of a circuit, 66
solvable automaton, 210
source vertex, 59
stability relation, 183
start state
  of an automaton, 159
starting vertex, 59
state complexity, 164
state diagram, 4
state set
  of an automaton, 4
strong connectivity $\kappa(G)$, 126
strongly $\frac{3}{2}$-connected digraph, 60
strongly k-connected graph, 60
strongly connected
  digraph, 60
  in automaton, 134
strongly connected automaton, 135
strongly connected component
  automaton, 134
  in digraph, 60
strongly trap-connected automaton, 136
subautomaton, 148
subgraph, 59
subgroup, 42
submonoid, 42
subsemigroup, 41
subset, 28
suffix, 57
surjective, 30
symbol, 3
symmetric digraph, 60
symmetric group, 111
symmetric relation, 39
synchronization diversity k, 252
synchronizing automaton, 4
synchronizing state, 4
synchronizing word, 4
system of blocks, 111

tail point, 38
tail transformation, 94
terminal vertex, 59
ternary automaton, 132
ternary language, 57
thermometer unary, 76
topological order, *see* topological sorting
topological sorting
  of a DAG, 61
  of a partial order, 46
total function, *see* function
total order, 46
totally extending problem, 252
transformation monoid, 45
  of automaton, 133
transformation of a set, 38
transformation semigroup, 45
  of automaton, 133
transition function
  of an automaton, 4
transition matrix, *see* Boolean transition matrix
transitive on subset, 112
transitive permutation group, 112
transitive relation, 39
transposition, 93
transversal, *see* section
trap state, 131
trivial block, 111
trivial congruences, 45
trivial equivalence relations, 40
trivial group, 42
trivial partition, 28
tuple, 28
Turing machine
  deterministic, 65
  nondeterministic, 65
Turing reducible
  L-uniform NC^1, 72
Turing transducer, 65

unary automaton, 132
unary language, 57
unilaterally connected
  automaton, 134
  digraph, 60

unilaterally connected automaton, 135
union, 28
universal mapping property, 55
universal quantifier, 27
universal relation, 40
upper set
  for quasiorder, 46
  in automaton, 134
  quasiorder, 46

variety of group languages, 222
vertex of a graph, 59
vertex-transitive graph, 126

walk in a graph, 60
weakly k-connected digraph, 60
weakly connected component, 60
well-ordering principle, 27
word, 3

zero matrix, 44